

API docs

...

目錄

1. Create New AI Assistant
2. List Embedding Models
3. List Reranker Models
4. List Large Language Models
5. Get AI Assistant List
6. Get Specific AI Assistant
7. Update AI Assistant
8. Partial Update AI Assistant
9. Delete AI Assistant
10. Duplicate AI Assistant
11. Enable or Disable Evaluation

Create New AI Assistant

POST

/api/v1/chatbots/

Request

Request Parameters

Field	Type	Required	Description
name	string	Yes	The bot's name; in Agent mode it has semantic meaning, in other modes it is only used to distinguish different bots
largeLanguageModel	string (uuid)	Yes	The large language model used by the bot for generating responses (including organization-managed AI Gateway models)
embeddingModel	string (uuid)	No	Embedding model used for vectorizing text, optional field
rerankerModel	string (uuid)	No	Model used for reranking, optional field
instructions	string	No	The bot's role instructions, used to describe the bot's role and behavior
knowledgeBases	array[IdName]	No	List of knowledge bases accessible to the bot
databases	array[IdName]	No	List of SQL databases accessible to the bot (for Text-to-SQL feature)

Field	Type	Required	Description
organization	string (uuid)	No	The organization the bot belongs to; if empty, it is a personal bot
builtInWorkflow	string (uuid)	No	Built-in workflow for predefined processing flows
replyMode	object	No	Reply mode: normal reply or streaming reply normal : Normal ; template : Template ; hybrid : Hybrid ; workflow : Workflow ; agent : Agent;
template	string	No	Template used in template mode and hybrid mode
unanswerableTemplate	string	No	Template used when unable to answer in template mode and hybrid mode
totalWordsCount	integer (int64)	No	Accumulated total word count
outputMode	object	No	Output mode: text, table, or custom format text : Text ; json_schema : JSON Schema;
rawOutputFormat	object	No	JSON schema definition for custom output format
groups	array[IdName]	No	List of groups accessible to the bot
tools	array[ToolSummary]	No	List of tools available to the bot
skills	array[SkillSummary]	No	List of skills associated with the bot
connectors	array[IdName]	No	Connectors associated with this chatbot, gating per-contact OAuth authorization
agentMode	object	No	Agent mode: normal, SQL, or workflow mode normal : Normal ; canvas : Canvas ; team : Team;
numberOfRetrievedChunks	integer	No	Number of retrieved reference chunks, default is 12, minimum is 1
enableEvaluation	boolean	No	
enableInlineCitations	boolean	No	Enable inline citation feature, which inserts [1][2] format citation markers in responses
enableCodeInterpreter	boolean	No	When enabled, provides the Code Interpreter tool to execute code in an isolated container
enableBrowserTool	boolean	No	Enable browser automation tool for web tasks
enableImageSearch	boolean	No	Provide the image_search tool when any attached knowledge base uses a multimodal embedding model. Disable to remove the tool entirely.

Field	Type	Required	Description
enableClaudeStreamRetry	boolean	No	When a Claude streaming call stalls (first-chunk or mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a reset signal so the client discards any partial output...
customMaxLlmOutputTokens	integer	No	Custom maximum LLM output token count, minimum is 512
voiceConfig	object	No	
voiceConfig.enabled	boolean	No	Whether this chatbot can be used as a voice agent. Checked by every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-platform toggle.
voiceConfig.voiceAgentType	string (uuid)	No	Voice agent mode type
voiceConfig.sttProvider	string (uuid)	No	Speech-to-Text service provider
voiceConfig.sttConfig	object	No	Actual configuration parameters for STT (JSON format)
voiceConfig.ttsProvider	string (uuid)	No	Text-to-Speech service provider
voiceConfig.ttsConfig	object	No	Actual configuration parameters for TTS (JSON format)
voiceConfig.realtimeProvider	string (uuid)	No	Realtime end-to-end voice model provider
voiceConfig.realtimeConfig	object	No	Actual configuration parameters for Realtime (JSON format)
voiceConfig.realtimeInstructions	string	No	Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty
voiceConfig.turnHandling	object	No	Voice assistant conversation turn and interruption control settings
voiceConfig.enableVoiceReply	boolean	No	When disabled, the voice agent replies in text only and does not speak
thinkingConfig	object	No	Thinking effort settings (JSON format)
enableToolSearching	boolean	No	Enable dynamic tool search, allowing the agent to dynamically discover and load tools via tool_searching_tool
maxAgentIterations	integer	No	Maximum number of iterations for AgentWorkflow (1-100, default: 20)
agentTimeoutSeconds	integer	No	Timeout in seconds for AgentWorkflow execution (30-1200, default: 600)

Field	Type	Required	Description
agentTimeoutMessage	string	No	Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default.
enableSizeBasedToolResultTruncation	boolean	No	When enabled, tool results exceeding the character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS whitelist approach. Reduces memory bloat from large tool outputs ...
toolResultMaxChars	integer	No	Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True.
enableVectorMemory	boolean	No	When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat hist...
enableCrossConversationSearch	boolean	No	When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation.
crossConversationLookbackDays	integer	No	How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search is True.
useGatewayFallback	boolean	No	When enabled, use the fallback group instead of the single LLM
gatewayFallbackGroup	string (uuid)	No	
enableSearchMessagesTool	boolean	No	Allow the AI to search past messages in the current conversation.
enableGetMessageContextTool	boolean	No	Allow the AI to retrieve messages surrounding a specific message for context.
enableReadAttachmentFullTextTool	boolean	No	Allow the AI to read the full text of plain-text attachments. AGENT mode only.
enableParseAttachmentContentTool	boolean	No	Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only.
enableListAvailableParsersTool	boolean	No	Allow the AI to list available parsers and their capabilities. AGENT mode only.

Field	Type	Required	Description
enableGetDownloadLinkTool	boolean	No	Allow the AI to generate shareable download links for produced content. AGENT mode only.

Request Schema Example

```
{
  "name": string // The bot's name; in Agent mode it has semantic meaning, in other modes it is
  only used to distinguish different bots
  "largeLanguageModel": string (uuid) // The large language model used by the bot for
  generating responses (including organization-managed AI Gateway models)
  "embeddingModel"?: string (uuid) // Embedding model used for vectorizing text, optional field
  (Optional)
  "rerankerModel"?: string (uuid) // Model used for reranking, optional field (Optional)
  "instructions"?: string // The bot's role instructions, used to describe the bot's role and
  behavior (Optional)
  "knowledgeBases"?: [ // List of knowledge bases accessible to the bot (Optional)
    {
      "id": string (uuid)
    }
  ]
  "databases"?: [ // List of SQL databases accessible to the bot (for Text-to-SQL feature)
  (Optional)
    {
      "id": string (uuid)
    }
  ]
  "organization"?: string (uuid) // The organization the bot belongs to; if empty, it is a
  personal bot (Optional)
  "builtInWorkflow"?: string (uuid) // Built-in workflow for predefined processing flows
  (Optional)
  "replyMode"?: // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `workflow` - Workflow
* `agent` - Agent (Optional)
  {
  }
  "template"?: string // Template used in template mode and hybrid mode (Optional)
  "unanswerableTemplate"?: string // Template used when unable to answer in template mode and
  hybrid mode (Optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count (Optional)
  "outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
  {
  }
  "rawOutputFormat"?: object // JSON schema definition for custom output format (Optional)
  "groups"?: [ // List of groups accessible to the bot (Optional)
    {

```

```

        "id": string (uuid)
    }
]
"tools"?: [ // List of tools available to the bot (Optional)
    {
        "id": string (uuid)
    }
]
"skills"?: [ // List of skills associated with the bot (Optional)
    {
        "id": string (uuid)
    }
]
"connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
    {
        "id": string (uuid)
    }
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
    {
    }
    "numberOfRetrievedChunks"?: integer // Number of retrieved reference chunks, default is 12,
minimum is 1 (Optional)
    "enableEvaluation"?: boolean // Optional
    "enableInlineCitations"?: boolean // Enable inline citation feature, which inserts [1][2]
format citation markers in responses (Optional)
    "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to
execute code in an isolated container (Optional)
    "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
    "enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
    "enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
    "customMaxLlmOutputTokens"?: integer // Custom maximum LLM output token count, minimum is 512
(Optional)
    "voiceConfig"?: { // Optional
    {
        "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
        "voiceAgentType"?: string (uuid) // Voice agent mode type (Optional)
        "sttProvider"?: string (uuid) // Speech-to-Text service provider (Optional)
        "sttConfig"?: object // Actual configuration parameters for STT (JSON format) (Optional)
        "ttsProvider"?: string (uuid) // Text-to-Speech service provider (Optional)
        "ttsConfig"?: object // Actual configuration parameters for TTS (JSON format) (Optional)
        "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider (Optional)
        "realtimeConfig"?: object // Actual configuration parameters for Realtime (JSON format)
(Optional)
        "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (Optional)

```

```

    "turnHandling"?: object // Voice assistant conversation turn and interruption control
    settings (Optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
    does not speak (Optional)
  }
}
"thinkingConfig"?: object // Thinking effort settings (JSON format) (Optional)
"enableToolSearching"?: boolean // Enable dynamic tool search, allowing the agent to
dynamically discover and load tools via tool_searching_tool (Optional)
"maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (Optional)
"agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (Optional)
"agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (Optional)
"enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
web search results, etc.). (Optional)
"toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
"enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (Optional)
"enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (Optional)
"crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
90 days. Only effective when enable_cross_conversation_search is True. (Optional)
"useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (Optional)
"gatewayFallbackGroup"?: string (uuid) // Optional
"enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (Optional)
"enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (Optional)
"enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
text attachments. AGENT mode only. (Optional)
"enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (Optional)
"enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (Optional)
"enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (Optional)
}

```

request

Request Example Values

```

{
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",

```

```
"rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
"instructions": "Example String",
"knowledgeBases": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"databases": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"organization": "550e8400-e29b-41d4-a716-446655440000",
"builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
"replyMode": {},
"template": "Example String",
"unanswerableTemplate": "Example String",
"totalWordsCount": 123,
"outputMode": {},
"rawOutputFormat": null,
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"skills": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"connectors": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"agentMode": {},
"numberOfRetrievedChunks": 123,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
  "enabled": true,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
```

```
    "realtimeConfig": null,  
    "realtimeInstructions": "Example String",  
    "turnHandling": null,  
    "enableVoiceReply": true  
  },  
  "thinkingConfig": null,  
  "enableToolSearching": true,  
  "maxAgentIterations": 123,  
  "agentTimeoutSeconds": 123,  
  "agentTimeoutMessage": "Example String",  
  "enableSizeBasedToolResultTruncation": true,  
  "toolResultMaxChars": 123,  
  "enableVectorMemory": true,  
  "enableCrossConversationSearch": true,  
  "crossConversationLookbackDays": 123,  
  "useGatewayFallback": true,  
  "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",  
  "enableSearchMessagesTool": true,  
  "enableGetMessageContextTool": true,  
  "enableReadAttachmentFullTextTool": true,  
  "enableParseAttachmentContentTool": true,  
  "enableListAvailableParsersTool": true,  
  "enableGetDownloadLinkTool": true  
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
}
```

```
"agentMode": {},
"numberOfRetrievedChunks": 123,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
  "enabled": true,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "Example String",
  "turnHandling": null,
  "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example String",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000"
},'
```

Please replace YOUR_API_KEY and verify request data before executing.


```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request (payload)
const data = {
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
```

```

    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 123,
  "enableEvaluation": true,
  "enableInlineCitations": true,
  "enableCodeInterpreter": true,
  "enableBrowserTool": true,
  "enableImageSearch": true,
  "enableClaudeStreamRetry": true,
  "customMaxLlmOutputTokens": 123,
  "voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example String",
    "turnHandling": null,
    "enableVoiceReply": true
  },
  "thinkingConfig": null,
  "enableToolSearching": true,
  "maxAgentIterations": 123,
  "agentTimeoutSeconds": 123,
  "agentTimeoutMessage": "Example String",
  "enableSizeBasedToolResultTruncation": true,
  "toolResultMaxChars": 123,
  "enableVectorMemory": true,
  "enableCrossConversationSearch": true,
  "crossConversationLookbackDays": 123,
  "useGatewayFallback": true,
  "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000"
};

axios.post(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });

```

```

import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request (payload)
data = {
    "name": "Example Name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example String",
    "knowledgeBases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "databases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example String",
    "unanswerableTemplate": "Example String",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": null,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "tools": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "skills": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "connectors": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

```

```

    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 123,
  "enableEvaluation": true,
  "enableInlineCitations": true,
  "enableCodeInterpreter": true,
  "enableBrowserTool": true,
  "enableImageSearch": true,
  "enableClaudeStreamRetry": true,
  "customMaxLlmOutputTokens": 123,
  "voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example String",
    "turnHandling": null,
    "enableVoiceReply": true
  },
  "thinkingConfig": null,
  "enableToolSearching": true,
  "maxAgentIterations": 123,
  "agentTimeoutSeconds": 123,
  "agentTimeoutMessage": "Example String",
  "enableSizeBasedToolResultTruncation": true,
  "toolResultMaxChars": 123,
  "enableVectorMemory": true,
  "enableCrossConversationSearch": true,
  "crossConversationLookbackDays": 123,
  "useGatewayFallback": true,
  "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000"
}

```

```

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)

```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name",
            "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
            "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
            "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
            "instructions": "Example String",
            "knowledgeBases": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "databases": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "organization": "550e8400-e29b-41d4-a716-446655440000",
            "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
            "replyMode": {},
            "template": "Example String",
            "unanswerableTemplate": "Example String",
            "totalWordsCount": 123,
            "outputMode": {},
            "rawOutputFormat": null,
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "tools": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "skills": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "connectors": [

```

```

        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "agentMode": {},
    "numberOfRetrievedChunks": 123,
    "enableEvaluation": true,
    "enableInlineCitations": true,
    "enableCodeInterpreter": true,
    "enableBrowserTool": true,
    "enableImageSearch": true,
    "enableClaudeStreamRetry": true,
    "customMaxLlmOutputTokens": 123,
    "voiceConfig": {
        "enabled": true,
        "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
        "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
        "sttConfig": null,
        "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
        "ttsConfig": null,
        "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
        "realtimeConfig": null,
        "realtimeInstructions": "Example String",
        "turnHandling": null,
        "enableVoiceReply": true
    },
    "thinkingConfig": null,
    "enableToolSearching": true,
    "maxAgentIterations": 123,
    "agentTimeoutSeconds": 123,
    "agentTimeoutMessage": "Example String",
    "enableSizeBasedToolResultTruncation": true,
    "toolResultMaxChars": 123,
    "enableVectorMemory": true,
    "enableCrossConversationSearch": true,
    "crossConversationLookbackDays": 123,
    "useGatewayFallback": true,
    "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000"
    }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "name": string // The bot's name; in Agent mode it has semantic meaning, in other modes it is
only used to distinguish different bots
  "largeLanguageModel": string (uuid) // The large language model used by the bot for
generating responses (including organization-managed AI Gateway models)
  "llm": // The bound model's id and name; read directly from the FK, returned even if the
model is deactivated by the organization
  {
    "id": string (uuid)
    "name": string
  }
  "embeddingModel"?: string (uuid) // Embedding model used for vectorizing text, optional field
(Optional)
  "rerankerModel"?: string (uuid) // Model used for reranking, optional field (Optional)
  "instructions"?: string // The bot's role instructions, used to describe the bot's role and
behavior (Optional)
  "knowledgeBases"?: [ // List of knowledge bases accessible to the bot (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "databases"?: [ // List of SQL databases accessible to the bot (for Text-to-SQL feature)
(Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "updatedAt": string (timestamp)
  "organization"?: string (uuid) // The organization the bot belongs to; if empty, it is a
personal bot (Optional)
  "builtInWorkflow"?: string (uuid) // Built-in workflow for predefined processing flows
(Optional)
  "replyMode"?: // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `workflow` - Workflow
* `agent` - Agent (Optional)
  {
  }
  "template"?: string // Template used in template mode and hybrid mode (Optional)
  "unanswerableTemplate"?: string // Template used when unable to answer in template mode and
hybrid mode (Optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count (Optional)
```

```

"outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
{
}
"rawOutputFormat"?: object // JSON schema definition for custom output format (Optional)
"groups"?: [ // List of groups accessible to the bot (Optional)
{
  "id": string (uuid)
  "name": string
}
]
"tools"?: [ // List of tools available to the bot (Optional)
{
  "id": string (uuid)
  "name": string
  "description": string
  "displayName": string
  "toolType": string
}
]
"skills"?: [ // List of skills associated with the bot (Optional)
{
  "id": string (uuid)
  "name": string
  "description": string
}
]
"connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
{
  "id": string (uuid)
  "name": string
}
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
{
}
"numberOfRetrievedChunks"?: integer // Number of retrieved reference chunks, default is 12,
minimum is 1 (Optional)
"enableEvaluation"?: boolean // Optional
"enableInlineCitations"?: boolean // Enable inline citation feature, which inserts [1][2]
format citation markers in responses (Optional)
"enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to
execute code in an isolated container (Optional)
"enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
"enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
"enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)

```



```
"customMaxLlmOutputTokens"?: integer // Custom maximum LLM output token count, minimum is 512
(Optional)
"voiceConfig"?: { // Optional
{
  "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
  "voiceAgentType"?: string (uuid) // Voice agent mode type (Optional)
  "sttProvider"?: string (uuid) // Speech-to-Text service provider (Optional)
  "sttConfig"?: object // Actual configuration parameters for STT (JSON format) (Optional)
  "ttsProvider"?: string (uuid) // Text-to-Speech service provider (Optional)
  "ttsConfig"?: object // Actual configuration parameters for TTS (JSON format) (Optional)
  "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider (Optional)
  "realtimeConfig"?: object // Actual configuration parameters for Realtime (JSON format)
(Optional)
  "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (Optional)
  "turnHandling"?: object // Voice assistant conversation turn and interruption control
settings (Optional)
  "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (Optional)
}
}
"thinkingConfig"?: object // Thinking effort settings (JSON format) (Optional)
"enableToolSearching"?: boolean // Enable dynamic tool search, allowing the agent to
dynamically discover and load tools via tool_searching_tool (Optional)
"maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (Optional)
"agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (Optional)
"agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (Optional)
"isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
"enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
web search results, etc.). (Optional)
"toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
"enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (Optional)
"enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (Optional)
"crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
90 days. Only effective when enable_cross_conversation_search is True. (Optional)
"useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (Optional)
"gatewayFallbackGroup"?: string (uuid) // Optional
"enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (Optional)
"enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (Optional)
"enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
```

```

text attachments. AGENT mode only. (Optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (Optional)
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (Optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (Optional)
  "isLocked": boolean // Locked instance: config follows the official version; only the
dedicated KB can be modified
  "sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when
instantiated by a rental
  "canUpdate": boolean // Return whether the current user can update this resource.
  "canDelete": boolean // Return whether the current user can delete this resource.
}

```

response

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Response String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "updatedAt": "Response String",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Response String",
  "unanswerableTemplate": "Response String",
  "totalWordsCount": 456,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ]
}

```

```
],
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "description": "Response String",
    "displayName": "Response String",
    "toolType": "Response String"
  }
],
"skills": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "description": "Response String"
  }
],
"connectors": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  }
],
"agentMode": {},
"numberOfRetrievedChunks": 456,
"enableEvaluation": false,
"enableInlineCitations": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
"enableImageSearch": false,
"enableClaudeStreamRetry": false,
"customMaxLlmOutputTokens": 456,
"voiceConfig": {
  "enabled": false,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "Response String",
  "turnHandling": null,
  "enableVoiceReply": false
},
"thinkingConfig": null,
"enableToolSearching": false,
"maxAgentIterations": 456,
"agentTimeoutSeconds": 456,
"agentTimeoutMessage": "Response String",
"isMultimodalLlm": false,
"enableSizeBasedToolResultTruncation": false,
"toolResultMaxChars": 456,
"enableVectorMemory": false,
"enableCrossConversationSearch": false,
"crossConversationLookbackDays": 456,
"useGatewayFallback": false,
```

```
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": false,
"enableGetMessageContextTool": false,
"enableReadAttachmentFullTextTool": false,
"enableParseAttachmentContentTool": false,
"enableListAvailableParsersTool": false,
"enableGetDownloadLinkTool": false,
"isLocked": false,
"sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
"canUpdate": false,
"canDelete": false
}
```

List Embedding Models

GET

</api/v1/embedding-models/>

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/embedding-models/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please replace YOUR_API_KEY and verify request data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/embedding-models/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/embedding-models/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/embedding-models/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```

[
  {
    "id": string (uuid)
    "name": string
    "isDefault"?: boolean // Optional
    "isMultimodal"?: boolean // Whether this embedding model supports multimodal (text + image)
    embeddings (Optional)
    "icon":
    {
      "id": string (uuid)
      "name": string
      "image": string (uri)
    }
  }
]

```

response

Response Example Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "isDefault": false,
    "isMultimodal": false,
    "icon": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "image": "https://example.com/file.jpg"
    }
  }
]
```

List Reranker Models

GET

</api/v1/reranker-models/>

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/reranker-models/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please replace YOUR_API_KEY and verify request data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/reranker-models/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/reranker-models/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/reranker-models/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "id": string (uuid)
    "name": string
    "isDefault?": boolean // Optional
    "icon":
    {
      "id": string (uuid)
      "name": string
      "image": string (uri)
    }
  }
]
```

response

Response Example Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "isDefault": false,
    "icon": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "image": "https://example.com/file.jpg"
    }
  }
]
```

List Large Language Models

GET

</api/v1/large-language-models/>

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/large-language-models/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please replace YOUR_API_KEY and verify request data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/large-language-models/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/large-language-models/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/large-language-models/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```

[
  {
    "id": string (uuid)
    "name": string
    "contextWindow"?: integer // This value is only used for OpenAIIlike or Ollama models; other
models do not use this value (Optional)
    "isDefault"?: boolean // Optional
    "provider": string
    "icon":
    {
      "id": string (uuid)
      "name": string
      "image": string (uri)
    }
    "isMultiModalSupport"?: boolean // Optional
    "isFunctionCallingAvailable"?: boolean // Optional
    "isToolCallingAvailable"?: boolean // Optional
  }
]

```

```

    "isTemplateAvailable": boolean
    "isOutputFormatAvailable": boolean
    "isThinkingAvailable"?: boolean // Optional
    "thinkingConfigSchema": object
    "replyMode": [
      string
    ]
    "outputMode": [
      string
    ]
    "agentMode": [
      string
    ]
    "maxTokens"?: integer // Optional
    "minTokens": integer
    "isCustomModel": boolean // Whether the row is organization-owned (materialized from an AI
Gateway custom model).
  }
]

```

response

Response Example Values

```

[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "contextWindow": 456,
    "isDefault": false,
    "provider": "Response String",
    "icon": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "image": "https://example.com/file.jpg"
    },
    "isMultiModalSupport": false,
    "isFunctionCallingAvailable": false,
    "isToolCallingAvailable": false,
    "isTemplateAvailable": false,
    "isOutputFormatAvailable": false,
    "isThinkingAvailable": false,
    "thinkingConfigSchema": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response Example Name",
      "description": "Response Example Description"
    },
    "replyMode": [
      "Response String"
    ],
    "outputMode": [
      "Response String"
    ],
    "agentMode": [
      "Response String"
    ],
  },
]

```

```
"maxTokens": 456,  
"minTokens": 456,  
"isCustomModel": false  
}  
]
```

Get AI Assistant List

GET

/api/v1/chatbots/

Parameters

Parameter	Required	Type	Description
largeLanguageModel	X	string	
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
pagination	X	string	Whether to paginate (true/false)
query	X	string	
replyMode	X	string	
source	X	string	self_built : self_built; built_in : built_in; all : all;

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

# Please replace YOUR_API_KEY and verify request data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```


Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string // The bot's name; in Agent mode it has semantic meaning, in other modes
it is only used to distinguish different bots
      "groups": [
        {
          "id": string (uuid)
          "name": string
        }
      ]
    }
  ]
  "largeLanguageModel": { // Base serializer for LLM, contains only basic fields
    {
      "id": string (uuid)
      "name": string
    }
  }
  "embeddingModel": {
    {
      "id": string (uuid)
      "name": string
      "isDefault"?: boolean // Optional
      "isMultimodal"?: boolean // Whether this embedding model supports multimodal (text +
image) embeddings (Optional)
      "icon":
        {
          "id": string (uuid)
          "name": string
          "image": string (uri)
        }
    }
  }
  "rerankerModel": {
    {
      "id": string (uuid)
      "name": string
      "isDefault"?: boolean // Optional
      "icon":
        {
          "id": string (uuid)
          "name": string
          "image": string (uri)
        }
    }
  }
}
```

```

    }
    "knowledgeBases": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "tools": [
      {
        "id": string (uuid)
        "name": string
        "description": string
        "displayName": string
        "toolType": string
      }
    ]
    "skills": [
      {
        "id": string (uuid)
        "name": string
        "description": string
      }
    ]
    "agentMode"?: // Agent mode: canvas mode and normal mode; canvas mode uses the canvas to
generate responses, normal mode uses normal mode to generate responses

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
  {
  }
  "replyMode"?: // Reply mode: normal mode, template mode, hybrid mode, workflow mode, or
Agent mode

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `workflow` - Workflow
* `agent` - Agent (Optional)
  {
  }
  "updatedAt": string (timestamp)
  "enableEvaluation"?: boolean // Optional
  "enableInlineCitations"?: boolean // Enable inline citation feature, which inserts [1][2]
format citation markers in responses (Optional)
  "enableToolSearching"?: boolean // Enable dynamic tool search, allowing the agent to
dynamically discover and load tools via tool_searching_tool (Optional)
  "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to
execute code in an isolated container (Optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached
knowledge base uses a multimodal embedding model. Disable to remove the tool entirely.
(Optional)
  "numberOfRetrievedChunks"?: integer // Optional
  "isMultimodalLlm": boolean // Return whether the chatbot's LLM supports multimodal.
  "createdBy":
  {

```

```

    "id": string (uuid)
    "name": string
  }
  "isLocked"?: boolean // Locked instance: config follows the official version; only the
dedicated KB can be modified (Optional)
  "sourceBuildInAgent"?: string (uuid) // Points to the official Build-in Agent blueprint
when instantiated by a rental (Optional)
  "canUpdate": boolean // Return whether the current user can update this resource.
  "canDelete": boolean // Return whether the current user can delete this resource.
}
]
}

```

response

Response Example Values

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "groups": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response String"
        }
      ],
      "largeLanguageModel": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response String"
      },
      "embeddingModel": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response String",
        "isDefault": false,
        "isMultimodal": false,
        "icon": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response String",
          "image": "https://example.com/file.jpg"
        }
      },
      "rerankerModel": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response String",
        "isDefault": false,
        "icon": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response String",
          "image": "https://example.com/file.jpg"
        }
      }
    }
  ],
}

```

```

"knowledgeBases": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  }
],
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "description": "Response String",
    "displayName": "Response String",
    "toolType": "Response String"
  }
],
"skills": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "description": "Response String"
  }
],
"agentMode": {},
"replyMode": {},
"updatedAt": "Response String",
"enableEvaluation": false,
"enableInlineCitations": false,
"enableToolSearching": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
"enableImageSearch": false,
"numberOfRetrievedChunks": 456,
"isMultimodalLlm": false,
"voiceConfig": {
  "enabled": false
},
"createdBy": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String"
},
"isLocked": false,
"sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
"canUpdate": false,
"canDelete": false
}
]
}

```

Get Specific AI Assistant

GET**/api/v1/chatbots/{id}**

Parameters

Parameter	Required	Type	Description
id	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

# Please replace YOUR_API_KEY and verify request data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]));

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```

{
  "id": string (uuid)
  "name": string // The bot's name; in Agent mode it has semantic meaning, in other modes it is
only used to distinguish different bots
  "largeLanguageModel": string (uuid) // The large language model used by the bot for
generating responses (including organization-managed AI Gateway models)
  "llm": // The bound model's id and name; read directly from the FK, returned even if the
model is deactivated by the organization
  {
    "id": string (uuid)
    "name": string
  }
  "embeddingModel"? : string (uuid) // Embedding model used for vectorizing text, optional field
(Optional)
  "rerankerModel"? : string (uuid) // Model used for reranking, optional field (Optional)
  "instructions"? : string // The bot's role instructions, used to describe the bot's role and
behavior (Optional)
}

```

```

"knowledgeBases"?: [ // List of knowledge bases accessible to the bot (Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"databases"?: [ // List of SQL databases accessible to the bot (for Text-to-SQL feature)
(Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"updatedAt": string (timestamp)
"organization"?: string (uuid) // The organization the bot belongs to; if empty, it is a
personal bot (Optional)
"builtInWorkflow"?: string (uuid) // Built-in workflow for predefined processing flows
(Optional)
"replyMode"?: // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `workflow` - Workflow
* `agent` - Agent (Optional)
{
}
"template"?: string // Template used in template mode and hybrid mode (Optional)
"unanswerableTemplate"?: string // Template used when unable to answer in template mode and
hybrid mode (Optional)
"totalWordsCount"?: integer (int64) // Accumulated total word count (Optional)
"outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
{
}
"rawOutputFormat"?: object // JSON schema definition for custom output format (Optional)
"groups"?: [ // List of groups accessible to the bot (Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"tools"?: [ // List of tools available to the bot (Optional)
  {
    "id": string (uuid)
    "name": string
    "description": string
    "displayName": string
    "toolType": string
  }
]
"skills"?: [ // List of skills associated with the bot (Optional)
  {
    "id": string (uuid)
    "name": string
  }
]

```



```

        "description": string
    }
]
"connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
    {
        "id": string (uuid)
        "name": string
    }
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
    {
    }

"numberOfRetrievedChunks"?: integer // Number of retrieved reference chunks, default is 12,
minimum is 1 (Optional)
"enableEvaluation"?: boolean // Optional
"enableInlineCitations"?: boolean // Enable inline citation feature, which inserts [1][2]
format citation markers in responses (Optional)
"enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to
execute code in an isolated container (Optional)
"enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
"enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
"enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
"customMaxLlmOutputTokens"?: integer // Custom maximum LLM output token count, minimum is 512
(Optional)
"voiceConfig"?: { // Optional
    {
        "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
        "voiceAgentType"?: string (uuid) // Voice agent mode type (Optional)
        "sttProvider"?: string (uuid) // Speech-to-Text service provider (Optional)
        "sttConfig"?: object // Actual configuration parameters for STT (JSON format) (Optional)
        "ttsProvider"?: string (uuid) // Text-to-Speech service provider (Optional)
        "ttsConfig"?: object // Actual configuration parameters for TTS (JSON format) (Optional)
        "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider (Optional)
        "realtimeConfig"?: object // Actual configuration parameters for Realtime (JSON format)
(Optional)
        "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (Optional)
        "turnHandling"?: object // Voice assistant conversation turn and interruption control
settings (Optional)
        "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (Optional)
    }
}
"thinkingConfig"?: object // Thinking effort settings (JSON format) (Optional)
"enableToolSearching"?: boolean // Enable dynamic tool search, allowing the agent to
dynamically discover and load tools via tool_searching_tool (Optional)

```

```

"maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (Optional)
"agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (Optional)
"agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (Optional)
"isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
"enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
web search results, etc.). (Optional)
"toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
"enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (Optional)
"enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (Optional)
"crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
90 days. Only effective when enable_cross_conversation_search is True. (Optional)
"useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (Optional)
"gatewayFallbackGroup"?: string (uuid) // Optional
"enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (Optional)
"enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (Optional)
"enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
text attachments. AGENT mode only. (Optional)
"enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (Optional)
"enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (Optional)
"enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (Optional)
"isLocked": boolean // Locked instance: config follows the official version; only the
dedicated KB can be modified
"sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when
instantiated by a rental
"canUpdate": boolean // Return whether the current user can update this resource.
"canDelete": boolean // Return whether the current user can delete this resource.
}

```

response

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",

```

```
    "name": "Response String"
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Response String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "updatedAt": "Response String",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Response String",
  "unanswerableTemplate": "Response String",
  "totalWordsCount": 456,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "description": "Response String",
      "displayName": "Response String",
      "toolType": "Response String"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "description": "Response String"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 456,
  "enableEvaluation": false,
```

```

"enableInlineCitations": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
"enableImageSearch": false,
"enableClaudeStreamRetry": false,
"customMaxLlmOutputTokens": 456,
"voiceConfig": {
  "enabled": false,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "Response String",
  "turnHandling": null,
  "enableVoiceReply": false
},
"thinkingConfig": null,
"enableToolSearching": false,
"maxAgentIterations": 456,
"agentTimeoutSeconds": 456,
"agentTimeoutMessage": "Response String",
"isMultimodalLlm": false,
"enableSizeBasedToolResultTruncation": false,
"toolResultMaxChars": 456,
"enableVectorMemory": false,
"enableCrossConversationSearch": false,
"crossConversationLookbackDays": 456,
"useGatewayFallback": false,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": false,
"enableGetMessageContextTool": false,
"enableReadAttachmentFullTextTool": false,
"enableParseAttachmentContentTool": false,
"enableListAvailableParsersTool": false,
"enableGetDownloadLinkTool": false,
"isLocked": false,
"sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
"canUpdate": false,
"canDelete": false
}

```

Update AI Assistant

PUT

/api/v1/chatbots/{id}

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	

Request

Request Parameters

Field	Type	Required	Description
name	string	Yes	The bot's name; in Agent mode it has semantic meaning, in other modes it is only used to distinguish different bots
largeLanguageModel	string (uuid)	Yes	The large language model used by the bot for generating responses (including organization-managed AI Gateway models)
embeddingModel	string (uuid)	No	Embedding model used for vectorizing text, optional field
rerankerModel	string (uuid)	No	Model used for reranking, optional field
instructions	string	No	The bot's role instructions, used to describe the bot's role and behavior
knowledgeBases	array[IdName]	No	List of knowledge bases accessible to the bot
databases	array[IdName]	No	List of SQL databases accessible to the bot (for Text-to-SQL feature)
organization	string (uuid)	No	The organization the bot belongs to; if empty, it is a personal bot
builtInWorkflow	string (uuid)	No	Built-in workflow for predefined processing flows
replyMode	object	No	Reply mode: normal reply or streaming reply <code>normal</code> : Normal; <code>template</code> : Template; <code>hybrid</code> : Hybrid; <code>workflow</code> : Workflow; <code>agent</code> : Agent;
template	string	No	Template used in template mode and hybrid mode
unanswerableTemplate	string	No	Template used when unable to answer in template mode and hybrid mode
totalWordsCount	integer (int64)	No	Accumulated total word count

Field	Type	Required	Description
outputMode	object	No	Output mode: text, table, or custom format <code>text</code> : Text ; <code>json_schema</code> : JSON Schema;
rawOutputFormat	object	No	JSON schema definition for custom output format
groups	array[IdName]	No	List of groups accessible to the bot
tools	array[ToolSummary]	No	List of tools available to the bot
skills	array[SkillSummary]	No	List of skills associated with the bot
connectors	array[IdName]	No	Connectors associated with this chatbot, gating per-contact OAuth authorization
agentMode	object	No	Agent mode: normal, SQL, or workflow mode <code>normal</code> : Normal ; <code>canvas</code> : Canvas ; <code>team</code> : Team;
numberOfRetrievedChunks	integer	No	Number of retrieved reference chunks, default is 12, minimum is 1
enableEvaluation	boolean	No	
enableInlineCitations	boolean	No	Enable inline citation feature, which inserts [1][2] format citation markers in responses
enableCodeInterpreter	boolean	No	When enabled, provides the Code Interpreter tool to execute code in an isolated container
enableBrowserTool	boolean	No	Enable browser automation tool for web tasks
enableImageSearch	boolean	No	Provide the image_search tool when any attached knowledge base uses a multimodal embedding model. Disable to remove the tool entirely.
enableClaudeStreamRetry	boolean	No	When a Claude streaming call stalls (first-chunk or mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a reset signal so the client discards any partial output...
customMaxLlmOutputTokens	integer	No	Custom maximum LLM output token count, minimum is 512
voiceConfig	object	No	
voiceConfig.enabled	boolean	No	Whether this chatbot can be used as a voice agent. Checked by every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-platform toggle.
voiceConfig.voiceAgentType	string (uuid)	No	Voice agent mode type

Field	Type	Required	Description
voiceConfig.sttProvider	string (uuid)	No	Speech-to-Text service provider
voiceConfig.sttConfig	object	No	Actual configuration parameters for STT (JSON format)
voiceConfig.ttsProvider	string (uuid)	No	Text-to-Speech service provider
voiceConfig.ttsConfig	object	No	Actual configuration parameters for TTS (JSON format)
voiceConfig.realtimeProvider	string (uuid)	No	Realtime end-to-end voice model provider
voiceConfig.realtimeConfig	object	No	Actual configuration parameters for Realtime (JSON format)
voiceConfig.realtimeInstructions	string	No	Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty
voiceConfig.turnHandling	object	No	Voice assistant conversation turn and interruption control settings
voiceConfig.enableVoiceReply	boolean	No	When disabled, the voice agent replies in text only and does not speak
thinkingConfig	object	No	Thinking effort settings (JSON format)
enableToolSearching	boolean	No	Enable dynamic tool search, allowing the agent to dynamically discover and load tools via tool_searching_tool
maxAgentIterations	integer	No	Maximum number of iterations for AgentWorkflow (1-100, default: 20)
agentTimeoutSeconds	integer	No	Timeout in seconds for AgentWorkflow execution (30-1200, default: 600)
agentTimeoutMessage	string	No	Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default.
enableSizeBasedToolResultTruncation	boolean	No	When enabled, tool results exceeding the character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS whitelist approach. Reduces memory bloat from large tool outputs ...
toolResultMaxChars	integer	No	Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True.
enableVectorMemory	boolean	No	When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived

Field	Type	Required	Description
			messages are discarded and only the active chat hist...
enableCrossConversationSearch	boolean	No	When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation.
crossConversationLookbackDays	integer	No	How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search is True.
useGatewayFallback	boolean	No	When enabled, use the fallback group instead of the single LLM
gatewayFallbackGroup	string (uuid)	No	
enableSearchMessagesTool	boolean	No	Allow the AI to search past messages in the current conversation.
enableGetMessageContextTool	boolean	No	Allow the AI to retrieve messages surrounding a specific message for context.
enableReadAttachmentFullTextTool	boolean	No	Allow the AI to read the full text of plain-text attachments. AGENT mode only.
enableParseAttachmentContentTool	boolean	No	Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only.
enableListAvailableParsersTool	boolean	No	Allow the AI to list available parsers and their capabilities. AGENT mode only.
enableGetDownloadLinkTool	boolean	No	Allow the AI to generate shareable download links for produced content. AGENT mode only.

Request Schema Example

```
{
  "name": string // The bot's name; in Agent mode it has semantic meaning, in other modes it is
only used to distinguish different bots
  "largeLanguageModel": string (uuid) // The large language model used by the bot for
generating responses (including organization-managed AI Gateway models)
  "embeddingModel"? : string (uuid) // Embedding model used for vectorizing text, optional field
(Optional)
  "rerankerModel"? : string (uuid) // Model used for reranking, optional field (Optional)
  "instructions"? : string // The bot's role instructions, used to describe the bot's role and
behavior (Optional)
  "knowledgeBases"? : [ // List of knowledge bases accessible to the bot (Optional)
    {
      "id": string (uuid)
```



```

    }
  ]
  "databases"?: [ // List of SQL databases accessible to the bot (for Text-to-SQL feature)
    (Optional)
    {
      "id": string (uuid)
    }
  ]
  "organization"?: string (uuid) // The organization the bot belongs to; if empty, it is a
personal bot (Optional)
  "builtInWorkflow"?: string (uuid) // Built-in workflow for predefined processing flows
(Optional)
  "replyMode"?: // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `workflow` - Workflow
* `agent` - Agent (Optional)
  {
  }
  "template"?: string // Template used in template mode and hybrid mode (Optional)
  "unanswerableTemplate"?: string // Template used when unable to answer in template mode and
hybrid mode (Optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count (Optional)
  "outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
  {
  }
  "rawOutputFormat"?: object // JSON schema definition for custom output format (Optional)
  "groups"?: [ // List of groups accessible to the bot (Optional)
    {
      "id": string (uuid)
    }
  ]
  "tools"?: [ // List of tools available to the bot (Optional)
    {
      "id": string (uuid)
    }
  ]
  "skills"?: [ // List of skills associated with the bot (Optional)
    {
      "id": string (uuid)
    }
  ]
  "connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
    {
      "id": string (uuid)
    }
  ]
  "agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas

```

```

* `team` - Team (Optional)
{
}
  "numberOfRetrievedChunks"?: integer // Number of retrieved reference chunks, default is 12,
  minimum is 1 (Optional)
  "enableEvaluation"?: boolean // Optional
  "enableInlineCitations"?: boolean // Enable inline citation feature, which inserts [1][2]
  format citation markers in responses (Optional)
  "enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to
  execute code in an isolated container (Optional)
  "enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
  "enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
  base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
  "enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
  mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
  reset signal so the client discards any partial output already rendered. Each retry incurs a
  second billing for the same prompt. (Optional)
  "customMaxLlmOutputTokens"?: integer // Custom maximum LLM output token count, minimum is 512
  (Optional)
  "voiceConfig"?: { // Optional
  {
    "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
    conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
    platform toggle. (Optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type (Optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service provider (Optional)
    "sttConfig"?: object // Actual configuration parameters for STT (JSON format) (Optional)
    "ttsProvider"?: string (uuid) // Text-to-Speech service provider (Optional)
    "ttsConfig"?: object // Actual configuration parameters for TTS (JSON format) (Optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider (Optional)
    "realtimeConfig"?: object // Actual configuration parameters for Realtime (JSON format)
    (Optional)
    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
    back to chatbot instructions when empty (Optional)
    "turnHandling"?: object // Voice assistant conversation turn and interruption control
    settings (Optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
    does not speak (Optional)
  }
  }
  "thinkingConfig"?: object // Thinking effort settings (JSON format) (Optional)
  "enableToolSearching"?: boolean // Enable dynamic tool search, allowing the agent to
  dynamically discover and load tools via tool_searching_tool (Optional)
  "maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
  default: 20) (Optional)
  "agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
  default: 600) (Optional)
  "agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
  out (max 500 chars). Leave empty to use the system default. (Optional)
  "enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
  character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
  whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
  web search results, etc.). (Optional)
  "toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
  (500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
  "enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
  processed by fact extraction for semantic retrieval (existing behavior). When disabled,

```

archived messages are discarded and only the active chat history window is used for multi-turn context. (Optional)

"enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation. (Optional)

"crossConversationLookbackDays"?: integer // How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search is True. (Optional)

"useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single LLM (Optional)

"gatewayFallbackGroup"?: string (uuid) // Optional

"enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current conversation. (Optional)

"enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a specific message for context. (Optional)

"enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-text attachments. AGENT mode only. (Optional)

"enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only. (Optional)

"enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and their capabilities. AGENT mode only. (Optional)

"enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links for produced content. AGENT mode only. (Optional)

}

request

Request Example Values

```
{
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

```
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 123,
  "enableEvaluation": true,
  "enableInlineCitations": true,
  "enableCodeInterpreter": true,
  "enableBrowserTool": true,
  "enableImageSearch": true,
  "enableClaudeStreamRetry": true,
  "customMaxLlmOutputTokens": 123,
  "voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example String",
    "turnHandling": null,
    "enableVoiceReply": true
  },
  "thinkingConfig": null,
  "enableToolSearching": true,
  "maxAgentIterations": 123,
  "agentTimeoutSeconds": 123,
  "agentTimeoutMessage": "Example String",
  "enableSizeBasedToolResultTruncation": true,
  "toolResultMaxChars": 123,
  "enableVectorMemory": true,
  "enableCrossConversationSearch": true,
  "crossConversationLookbackDays": 123,
  "useGatewayFallback": true,
  "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
  "enableSearchMessagesTool": true,
  "enableGetMessageContextTool": true,
  "enableReadAttachmentFullTextTool": true,
  "enableParseAttachmentContentTool": true,
  "enableListAvailableParsersTool": true,
```

```
"enableGetDownloadLinkTool": true  
}
```

Code Examples

```
# API call example (Shell)
curl -X PUT "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
}
```

```
"agentMode": {},
"numberOfRetrievedChunks": 123,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
  "enabled": true,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "Example String",
  "turnHandling": null,
  "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example String",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000"
},'
```

Please replace YOUR_API_KEY and verify request data before executing.

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request (payload)
const data = {
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
```



```

    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 123,
  "enableEvaluation": true,
  "enableInlineCitations": true,
  "enableCodeInterpreter": true,
  "enableBrowserTool": true,
  "enableImageSearch": true,
  "enableClaudeStreamRetry": true,
  "customMaxLlmOutputTokens": 123,
  "voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example String",
    "turnHandling": null,
    "enableVoiceReply": true
  },
  "thinkingConfig": null,
  "enableToolSearching": true,
  "maxAgentIterations": 123,
  "agentTimeoutSeconds": 123,
  "agentTimeoutMessage": "Example String",
  "enableSizeBasedToolResultTruncation": true,
  "toolResultMaxChars": 123,
  "enableVectorMemory": true,
  "enableCrossConversationSearch": true,
  "crossConversationLookbackDays": 123,
  "useGatewayFallback": true,
  "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000"
};

axios.put(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });

```

```

import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request (payload)
data = {
    "name": "Example Name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example String",
    "knowledgeBases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "databases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example String",
    "unanswerableTemplate": "Example String",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": null,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "tools": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "skills": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "connectors": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

```

```

    }
],
"agentMode": {},
"numberOfRetrievedChunks": 123,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example String",
    "turnHandling": null,
    "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example String",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000"
}

```

```

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)

```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name",
            "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
            "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
            "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
            "instructions": "Example String",
            "knowledgeBases": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "databases": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "organization": "550e8400-e29b-41d4-a716-446655440000",
            "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
            "replyMode": {},
            "template": "Example String",
            "unanswerableTemplate": "Example String",
            "totalWordsCount": 123,
            "outputMode": {},
            "rawOutputFormat": null,
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "tools": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "skills": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "connectors": [

```

```

        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "agentMode": {},
    "numberOfRetrievedChunks": 123,
    "enableEvaluation": true,
    "enableInlineCitations": true,
    "enableCodeInterpreter": true,
    "enableBrowserTool": true,
    "enableImageSearch": true,
    "enableClaudeStreamRetry": true,
    "customMaxLlmOutputTokens": 123,
    "voiceConfig": {
        "enabled": true,
        "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
        "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
        "sttConfig": null,
        "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
        "ttsConfig": null,
        "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
        "realtimeConfig": null,
        "realtimeInstructions": "Example String",
        "turnHandling": null,
        "enableVoiceReply": true
    },
    "thinkingConfig": null,
    "enableToolSearching": true,
    "maxAgentIterations": 123,
    "agentTimeoutSeconds": 123,
    "agentTimeoutMessage": "Example String",
    "enableSizeBasedToolResultTruncation": true,
    "toolResultMaxChars": 123,
    "enableVectorMemory": true,
    "enableCrossConversationSearch": true,
    "crossConversationLookbackDays": 123,
    "useGatewayFallback": true,
    "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000"
    }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string // The bot's name; in Agent mode it has semantic meaning, in other modes it is
only used to distinguish different bots
  "largeLanguageModel": string (uuid) // The large language model used by the bot for
generating responses (including organization-managed AI Gateway models)
  "llm": // The bound model's id and name; read directly from the FK, returned even if the
model is deactivated by the organization
  {
    "id": string (uuid)
    "name": string
  }
  "embeddingModel?": string (uuid) // Embedding model used for vectorizing text, optional field
(Optional)
  "rerankerModel?": string (uuid) // Model used for reranking, optional field (Optional)
  "instructions?": string // The bot's role instructions, used to describe the bot's role and
behavior (Optional)
  "knowledgeBases?": [ // List of knowledge bases accessible to the bot (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "databases?": [ // List of SQL databases accessible to the bot (for Text-to-SQL feature)
(Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "updatedAt": string (timestamp)
  "organization?": string (uuid) // The organization the bot belongs to; if empty, it is a
personal bot (Optional)
  "builtInWorkflow?": string (uuid) // Built-in workflow for predefined processing flows
(Optional)
  "replyMode?": // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `workflow` - Workflow
* `agent` - Agent (Optional)
  {
  }
  "template?": string // Template used in template mode and hybrid mode (Optional)
  "unanswerableTemplate?": string // Template used when unable to answer in template mode and
hybrid mode (Optional)
  "totalWordsCount?": integer (int64) // Accumulated total word count (Optional)
```

```

"outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
{
}
"rawOutputFormat"?: object // JSON schema definition for custom output format (Optional)
"groups"?: [ // List of groups accessible to the bot (Optional)
{
  "id": string (uuid)
  "name": string
}
]
"tools"?: [ // List of tools available to the bot (Optional)
{
  "id": string (uuid)
  "name": string
  "description": string
  "displayName": string
  "toolType": string
}
]
"skills"?: [ // List of skills associated with the bot (Optional)
{
  "id": string (uuid)
  "name": string
  "description": string
}
]
"connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
{
  "id": string (uuid)
  "name": string
}
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
{
}
"numberOfRetrievedChunks"?: integer // Number of retrieved reference chunks, default is 12,
minimum is 1 (Optional)
"enableEvaluation"?: boolean // Optional
"enableInlineCitations"?: boolean // Enable inline citation feature, which inserts [1][2]
format citation markers in responses (Optional)
"enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to
execute code in an isolated container (Optional)
"enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
"enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
"enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)

```

```
"customMaxLlmOutputTokens"?: integer // Custom maximum LLM output token count, minimum is 512
(Optional)
"voiceConfig"?: { // Optional
{
    "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type (Optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service provider (Optional)
    "sttConfig"?: object // Actual configuration parameters for STT (JSON format) (Optional)
    "ttsProvider"?: string (uuid) // Text-to-Speech service provider (Optional)
    "ttsConfig"?: object // Actual configuration parameters for TTS (JSON format) (Optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider (Optional)
    "realtimeConfig"?: object // Actual configuration parameters for Realtime (JSON format)
(Optional)
    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (Optional)
    "turnHandling"?: object // Voice assistant conversation turn and interruption control
settings (Optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (Optional)
}
}
"thinkingConfig"?: object // Thinking effort settings (JSON format) (Optional)
"enableToolSearching"?: boolean // Enable dynamic tool search, allowing the agent to
dynamically discover and load tools via tool_searching_tool (Optional)
"maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (Optional)
"agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (Optional)
"agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (Optional)
"isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
"enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
web search results, etc.). (Optional)
"toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
"enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (Optional)
"enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (Optional)
"crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
90 days. Only effective when enable_cross_conversation_search is True. (Optional)
"useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (Optional)
"gatewayFallbackGroup"?: string (uuid) // Optional
"enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (Optional)
"enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (Optional)
"enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
```



```

text attachments. AGENT mode only. (Optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (Optional)
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (Optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (Optional)
  "isLocked": boolean // Locked instance: config follows the official version; only the
dedicated KB can be modified
  "sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when
instantiated by a rental
  "canUpdate": boolean // Return whether the current user can update this resource.
  "canDelete": boolean // Return whether the current user can delete this resource.
}

```

response

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Response String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "updatedAt": "Response String",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Response String",
  "unanswerableTemplate": "Response String",
  "totalWordsCount": 456,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ]
}

```

```
],
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "description": "Response String",
    "displayName": "Response String",
    "toolType": "Response String"
  }
],
"skills": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "description": "Response String"
  }
],
"connectors": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  }
],
"agentMode": {},
"numberOfRetrievedChunks": 456,
"enableEvaluation": false,
"enableInlineCitations": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
"enableImageSearch": false,
"enableClaudeStreamRetry": false,
"customMaxLlmOutputTokens": 456,
"voiceConfig": {
  "enabled": false,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "Response String",
  "turnHandling": null,
  "enableVoiceReply": false
},
"thinkingConfig": null,
"enableToolSearching": false,
"maxAgentIterations": 456,
"agentTimeoutSeconds": 456,
"agentTimeoutMessage": "Response String",
"isMultimodalLlm": false,
"enableSizeBasedToolResultTruncation": false,
"toolResultMaxChars": 456,
"enableVectorMemory": false,
"enableCrossConversationSearch": false,
"crossConversationLookbackDays": 456,
"useGatewayFallback": false,
```

```
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": false,
"enableGetMessageContextTool": false,
"enableReadAttachmentFullTextTool": false,
"enableParseAttachmentContentTool": false,
"enableListAvailableParsersTool": false,
"enableGetDownloadLinkTool": false,
"isLocked": false,
"sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
"canUpdate": false,
"canDelete": false
}
```

Partial Update AI Assistant

PATCH

/api/v1/chatbots/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	

Request

Request Parameters

Field	Type	Required	Description
name	string	No	The bot's name; in Agent mode it has semantic meaning, in other modes it is only used to distinguish different bots
largeLanguageModel	string (uuid)	No	The large language model used by the bot for generating responses (including organization-managed AI Gateway models)
embeddingModel	string (uuid)	No	Embedding model used for vectorizing text, optional field
rerankerModel	string (uuid)	No	Model used for reranking, optional field
instructions	string	No	The bot's role instructions, used to describe the bot's role and behavior

Field	Type	Required	Description
knowledgeBases	array[IdName]	No	List of knowledge bases accessible to the bot
databases	array[IdName]	No	List of SQL databases accessible to the bot (for Text-to-SQL feature)
organization	string (uuid)	No	The organization the bot belongs to; if empty, it is a personal bot
builtInWorkflow	string (uuid)	No	Built-in workflow for predefined processing flows
replyMode	object	No	Reply mode: normal reply or streaming reply normal : Normal ; template : Template ; hybrid : Hybrid ; workflow : Workflow ; agent : Agent;
template	string	No	Template used in template mode and hybrid mode
unanswerableTemplate	string	No	Template used when unable to answer in template mode and hybrid mode
totalWordsCount	integer (int64)	No	Accumulated total word count
outputMode	object	No	Output mode: text, table, or custom format text : Text ; json_schema : JSON Schema;
rawOutputFormat	object	No	JSON schema definition for custom output format
groups	array[IdName]	No	List of groups accessible to the bot
tools	array[ToolSummary]	No	List of tools available to the bot
skills	array[SkillSummary]	No	List of skills associated with the bot
connectors	array[IdName]	No	Connectors associated with this chatbot, gating per-contact OAuth authorization
agentMode	object	No	Agent mode: normal, SQL, or workflow mode normal : Normal ; canvas : Canvas ; team : Team;
numberOfRetrievedChunks	integer	No	Number of retrieved reference chunks, default is 12, minimum is 1
enableEvaluation	boolean	No	
enableInlineCitations	boolean	No	Enable inline citation feature, which inserts [1][2] format citation markers in responses
enableCodeInterpreter	boolean	No	When enabled, provides the Code Interpreter tool to execute code in an isolated container
enableBrowserTool	boolean	No	Enable browser automation tool for web tasks

Field	Type	Required	Description
enableImageSearch	boolean	No	Provide the image_search tool when any attached knowledge base uses a multimodal embedding model. Disable to remove the tool entirely.
enableClaudeStreamRetry	boolean	No	When a Claude streaming call stalls (first-chunk or mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a reset signal so the client discards any partial outpu...
customMaxLlmOutputTokens	integer	No	Custom maximum LLM output token count, minimum is 512
voiceConfig	object	No	
voiceConfig.enabled	boolean	No	Whether this chatbot can be used as a voice agent. Checked by every conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-platform toggle.
voiceConfig.voiceAgentType	string (uuid)	No	Voice agent mode type
voiceConfig.sttProvider	string (uuid)	No	Speech-to-Text service provider
voiceConfig.sttConfig	object	No	Actual configuration parameters for STT (JSON format)
voiceConfig.ttsProvider	string (uuid)	No	Text-to-Speech service provider
voiceConfig.ttsConfig	object	No	Actual configuration parameters for TTS (JSON format)
voiceConfig.realtimeProvider	string (uuid)	No	Realtime end-to-end voice model provider
voiceConfig.realtimeConfig	object	No	Actual configuration parameters for Realtime (JSON format)
voiceConfig.realtimeInstructions	string	No	Prompt dedicated to the realtime voice session; falls back to chatbot instructions when empty
voiceConfig.turnHandling	object	No	Voice assistant conversation turn and interruption control settings
voiceConfig.enableVoiceReply	boolean	No	When disabled, the voice agent replies in text only and does not speak
thinkingConfig	object	No	Thinking effort settings (JSON format)
enableToolSearching	boolean	No	Enable dynamic tool search, allowing the agent to dynamically discover and load tools via tool_searching_tool
maxAgentIterations	integer	No	Maximum number of iterations for AgentWorkflow (1-100, default: 20)

Field	Type	Required	Description
agentTimeoutSeconds	integer	No	Timeout in seconds for AgentWorkflow execution (30-1200, default: 600)
agentTimeoutMessage	string	No	Custom user-facing message shown when AgentWorkflow times out (max 500 chars). Leave empty to use the system default.
enableSizeBasedToolResultTruncation	boolean	No	When enabled, tool results exceeding the character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS whitelist approach. Reduces memory bloat from large tool outputs ...
toolResultMaxChars	integer	No	Character threshold for size-based tool result truncation (500-50000). Only effective when enable_size_based_tool_result_truncation is True.
enableVectorMemory	boolean	No	When enabled, archived messages are vectorized and processed by fact extraction for semantic retrieval (existing behavior). When disabled, archived messages are discarded and only the active chat hist...
enableCrossConversationSearch	boolean	No	When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation.
crossConversationLookbackDays	integer	No	How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable_cross_conversation_search is True.
useGatewayFallback	boolean	No	When enabled, use the fallback group instead of the single LLM
gatewayFallbackGroup	string (uuid)	No	
enableSearchMessagesTool	boolean	No	Allow the AI to search past messages in the current conversation.
enableGetMessageContextTool	boolean	No	Allow the AI to retrieve messages surrounding a specific message for context.
enableReadAttachmentFullTextTool	boolean	No	Allow the AI to read the full text of plain-text attachments. AGENT mode only.
enableParseAttachmentContentTool	boolean	No	Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only.

Field	Type	Required	Description
enableListAvailableParsersTool	boolean	No	Allow the AI to list available parsers and their capabilities. AGENT mode only.
enableGetDownloadLinkTool	boolean	No	Allow the AI to generate shareable download links for produced content. AGENT mode only.

Request Schema Example

```
{
  "name"?: string // The bot's name; in Agent mode it has semantic meaning, in other modes it
is only used to distinguish different bots (Optional)
  "largeLanguageModel"?: string (uuid) // The large language model used by the bot for
generating responses (including organization-managed AI Gateway models) (Optional)
  "embeddingModel"?: string (uuid) // Embedding model used for vectorizing text, optional field
(Optional)
  "rerankerModel"?: string (uuid) // Model used for reranking, optional field (Optional)
  "instructions"?: string // The bot's role instructions, used to describe the bot's role and
behavior (Optional)
  "knowledgeBases"?: [ // List of knowledge bases accessible to the bot (Optional)
    {
      "id": string (uuid)
    }
  ]
  "databases"?: [ // List of SQL databases accessible to the bot (for Text-to-SQL feature)
(Optional)
    {
      "id": string (uuid)
    }
  ]
  "organization"?: string (uuid) // The organization the bot belongs to; if empty, it is a
personal bot (Optional)
  "builtInWorkflow"?: string (uuid) // Built-in workflow for predefined processing flows
(Optional)
  "replyMode"?: // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `workflow` - Workflow
* `agent` - Agent (Optional)
  {
  }
  "template"?: string // Template used in template mode and hybrid mode (Optional)
  "unanswerableTemplate"?: string // Template used when unable to answer in template mode and
hybrid mode (Optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count (Optional)
  "outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
  {
  }
  "rawOutputFormat"?: object // JSON schema definition for custom output format (Optional)
```

```

"groups"?: [ // List of groups accessible to the bot (Optional)
  {
    "id": string (uuid)
  }
]
"tools"?: [ // List of tools available to the bot (Optional)
  {
    "id": string (uuid)
  }
]
"skills"?: [ // List of skills associated with the bot (Optional)
  {
    "id": string (uuid)
  }
]
"connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
  {
    "id": string (uuid)
  }
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
{
}
"numberOfRetrievedChunks"?: integer // Number of retrieved reference chunks, default is 12,
minimum is 1 (Optional)
"enableEvaluation"?: boolean // Optional
"enableInlineCitations"?: boolean // Enable inline citation feature, which inserts [1][2]
format citation markers in responses (Optional)
"enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to
execute code in an isolated container (Optional)
"enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
"enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
"enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
"customMaxLlmOutputTokens"?: integer // Custom maximum LLM output token count, minimum is 512
(Optional)
"voiceConfig"?: { // Optional
{
  "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
  "voiceAgentType"?: string (uuid) // Voice agent mode type (Optional)
  "sttProvider"?: string (uuid) // Speech-to-Text service provider (Optional)
  "sttConfig"?: object // Actual configuration parameters for STT (JSON format) (Optional)
  "ttsProvider"?: string (uuid) // Text-to-Speech service provider (Optional)
  "ttsConfig"?: object // Actual configuration parameters for TTS (JSON format) (Optional)
  "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider (Optional)
  "realtimeConfig"?: object // Actual configuration parameters for Realtime (JSON format)
(Optional)
}
}

```



```

    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (Optional)
    "turnHandling"?: object // Voice assistant conversation turn and interruption control
settings (Optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (Optional)
  }
}
"thinkingConfig"?: object // Thinking effort settings (JSON format) (Optional)
"enableToolSearching"?: boolean // Enable dynamic tool search, allowing the agent to
dynamically discover and load tools via tool_searching_tool (Optional)
"maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (Optional)
"agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (Optional)
"agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (Optional)
"enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
web search results, etc.). (Optional)
"toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
"enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (Optional)
"enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (Optional)
"crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
90 days. Only effective when enable_cross_conversation_search is True. (Optional)
"useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (Optional)
"gatewayFallbackGroup"?: string (uuid) // Optional
"enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (Optional)
"enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (Optional)
"enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
text attachments. AGENT mode only. (Optional)
"enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (Optional)
"enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (Optional)
"enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (Optional)
}

```

request

Request Example Values

```

{
  "name": "Example Name",

```

```
"largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
"embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
"rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
"instructions": "Example String",
"knowledgeBases": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"databases": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"organization": "550e8400-e29b-41d4-a716-446655440000",
"builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
"replyMode": {},
"template": "Example String",
"unanswerableTemplate": "Example String",
"totalWordsCount": 123,
"outputMode": {},
"rawOutputFormat": null,
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"skills": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"connectors": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"agentMode": {},
"numberOfRetrievedChunks": 123,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
  "enabled": true,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
```

```
"ttsConfig": null,
"realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
"realtimeConfig": null,
"realtimeInstructions": "Example String",
"turnHandling": null,
"enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example String",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": true,
"enableGetMessageContextTool": true,
"enableReadAttachmentFullTextTool": true,
"enableParseAttachmentContentTool": true,
"enableListAvailableParsersTool": true,
"enableGetDownloadLinkTool": true
}
```

Code Examples

```
# API call example (Shell)
curl -X PATCH "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
}
```

```
"agentMode": {},
"numberOfRetrievedChunks": 123,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
  "enabled": true,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "Example String",
  "turnHandling": null,
  "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example String",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000"
},'
```

Please replace YOUR_API_KEY and verify request data before executing.

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request (payload)
const data = {
  "name": "Example Name",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Example String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Example String",
  "unanswerableTemplate": "Example String",
  "totalWordsCount": 123,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "tools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "skills": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "connectors": [
```

```

    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "agentMode": {},
  "numberOfRetrievedChunks": 123,
  "enableEvaluation": true,
  "enableInlineCitations": true,
  "enableCodeInterpreter": true,
  "enableBrowserTool": true,
  "enableImageSearch": true,
  "enableClaudeStreamRetry": true,
  "customMaxLlmOutputTokens": 123,
  "voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example String",
    "turnHandling": null,
    "enableVoiceReply": true
  },
  "thinkingConfig": null,
  "enableToolSearching": true,
  "maxAgentIterations": 123,
  "agentTimeoutSeconds": 123,
  "agentTimeoutMessage": "Example String",
  "enableSizeBasedToolResultTruncation": true,
  "toolResultMaxChars": 123,
  "enableVectorMemory": true,
  "enableCrossConversationSearch": true,
  "crossConversationLookbackDays": 123,
  "useGatewayFallback": true,
  "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000"
};

```

```

axios.patch(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });

```

```

import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request (payload)
data = {
    "name": "Example Name",
    "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "instructions": "Example String",
    "knowledgeBases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "databases": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
    "replyMode": {},
    "template": "Example String",
    "unanswerableTemplate": "Example String",
    "totalWordsCount": 123,
    "outputMode": {},
    "rawOutputFormat": null,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "tools": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "skills": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "connectors": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

```



```

    }
],
"agentMode": {},
"numberOfRetrievedChunks": 123,
"enableEvaluation": true,
"enableInlineCitations": true,
"enableCodeInterpreter": true,
"enableBrowserTool": true,
"enableImageSearch": true,
"enableClaudeStreamRetry": true,
"customMaxLlmOutputTokens": 123,
"voiceConfig": {
    "enabled": true,
    "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
    "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
    "sttConfig": null,
    "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
    "ttsConfig": null,
    "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
    "realtimeConfig": null,
    "realtimeInstructions": "Example String",
    "turnHandling": null,
    "enableVoiceReply": true
},
"thinkingConfig": null,
"enableToolSearching": true,
"maxAgentIterations": 123,
"agentTimeoutSeconds": 123,
"agentTimeoutMessage": "Example String",
"enableSizeBasedToolResultTruncation": true,
"toolResultMaxChars": 123,
"enableVectorMemory": true,
"enableCrossConversationSearch": true,
"crossConversationLookbackDays": 123,
"useGatewayFallback": true,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000"
}

```

```

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)

```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name",
            "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
            "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
            "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
            "instructions": "Example String",
            "knowledgeBases": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "databases": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "organization": "550e8400-e29b-41d4-a716-446655440000",
            "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
            "replyMode": {},
            "template": "Example String",
            "unanswerableTemplate": "Example String",
            "totalWordsCount": 123,
            "outputMode": {},
            "rawOutputFormat": null,
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "tools": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "skills": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "connectors": [

```

```

        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "agentMode": {},
    "numberOfRetrievedChunks": 123,
    "enableEvaluation": true,
    "enableInlineCitations": true,
    "enableCodeInterpreter": true,
    "enableBrowserTool": true,
    "enableImageSearch": true,
    "enableClaudeStreamRetry": true,
    "customMaxLlmOutputTokens": 123,
    "voiceConfig": {
        "enabled": true,
        "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
        "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
        "sttConfig": null,
        "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
        "ttsConfig": null,
        "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
        "realtimeConfig": null,
        "realtimeInstructions": "Example String",
        "turnHandling": null,
        "enableVoiceReply": true
    },
    "thinkingConfig": null,
    "enableToolSearching": true,
    "maxAgentIterations": 123,
    "agentTimeoutSeconds": 123,
    "agentTimeoutMessage": "Example String",
    "enableSizeBasedToolResultTruncation": true,
    "toolResultMaxChars": 123,
    "enableVectorMemory": true,
    "enableCrossConversationSearch": true,
    "crossConversationLookbackDays": 123,
    "useGatewayFallback": true,
    "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000"
    }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string // The bot's name; in Agent mode it has semantic meaning, in other modes it is
only used to distinguish different bots
  "largeLanguageModel": string (uuid) // The large language model used by the bot for
generating responses (including organization-managed AI Gateway models)
  "llm": // The bound model's id and name; read directly from the FK, returned even if the
model is deactivated by the organization
  {
    "id": string (uuid)
    "name": string
  }
  "embeddingModel"? : string (uuid) // Embedding model used for vectorizing text, optional field
(Optional)
  "rerankerModel"? : string (uuid) // Model used for reranking, optional field (Optional)
  "instructions"? : string // The bot's role instructions, used to describe the bot's role and
behavior (Optional)
  "knowledgeBases"? : [ // List of knowledge bases accessible to the bot (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "databases"? : [ // List of SQL databases accessible to the bot (for Text-to-SQL feature)
(Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "updatedAt": string (timestamp)
  "organization"? : string (uuid) // The organization the bot belongs to; if empty, it is a
personal bot (Optional)
  "builtInWorkflow"? : string (uuid) // Built-in workflow for predefined processing flows
(Optional)
  "replyMode"? : // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `workflow` - Workflow
* `agent` - Agent (Optional)
  {
  }
  "template"? : string // Template used in template mode and hybrid mode (Optional)
  "unanswerableTemplate"? : string // Template used when unable to answer in template mode and
hybrid mode (Optional)
  "totalWordsCount"? : integer (int64) // Accumulated total word count (Optional)
```

```

"outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
{
}
"rawOutputFormat"?: object // JSON schema definition for custom output format (Optional)
"groups"?: [ // List of groups accessible to the bot (Optional)
{
  "id": string (uuid)
  "name": string
}
]
"tools"?: [ // List of tools available to the bot (Optional)
{
  "id": string (uuid)
  "name": string
  "description": string
  "displayName": string
  "toolType": string
}
]
"skills"?: [ // List of skills associated with the bot (Optional)
{
  "id": string (uuid)
  "name": string
  "description": string
}
]
"connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
{
  "id": string (uuid)
  "name": string
}
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
{
}
"numberOfRetrievedChunks"?: integer // Number of retrieved reference chunks, default is 12,
minimum is 1 (Optional)
"enableEvaluation"?: boolean // Optional
"enableInlineCitations"?: boolean // Enable inline citation feature, which inserts [1][2]
format citation markers in responses (Optional)
"enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to
execute code in an isolated container (Optional)
"enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
"enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
"enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)

```

```
"customMaxLlmOutputTokens"?: integer // Custom maximum LLM output token count, minimum is 512
(Optional)
"voiceConfig"?: { // Optional
{
    "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
    "voiceAgentType"?: string (uuid) // Voice agent mode type (Optional)
    "sttProvider"?: string (uuid) // Speech-to-Text service provider (Optional)
    "sttConfig"?: object // Actual configuration parameters for STT (JSON format) (Optional)
    "ttsProvider"?: string (uuid) // Text-to-Speech service provider (Optional)
    "ttsConfig"?: object // Actual configuration parameters for TTS (JSON format) (Optional)
    "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider (Optional)
    "realtimeConfig"?: object // Actual configuration parameters for Realtime (JSON format)
(Optional)
    "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (Optional)
    "turnHandling"?: object // Voice assistant conversation turn and interruption control
settings (Optional)
    "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (Optional)
}
}
"thinkingConfig"?: object // Thinking effort settings (JSON format) (Optional)
"enableToolSearching"?: boolean // Enable dynamic tool search, allowing the agent to
dynamically discover and load tools via tool_searching_tool (Optional)
"maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (Optional)
"agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (Optional)
"agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (Optional)
"isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
"enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
web search results, etc.). (Optional)
"toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
"enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (Optional)
"enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (Optional)
"crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
90 days. Only effective when enable_cross_conversation_search is True. (Optional)
"useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (Optional)
"gatewayFallbackGroup"?: string (uuid) // Optional
"enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (Optional)
"enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (Optional)
"enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
```

```

text attachments. AGENT mode only. (Optional)
  "enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (Optional)
  "enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (Optional)
  "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (Optional)
  "isLocked": boolean // Locked instance: config follows the official version; only the
dedicated KB can be modified
  "sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when
instantiated by a rental
  "canUpdate": boolean // Return whether the current user can update this resource.
  "canDelete": boolean // Return whether the current user can delete this resource.
}

```

response

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  },
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "instructions": "Response String",
  "knowledgeBases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "databases": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ],
  "updatedAt": "Response String",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
  "replyMode": {},
  "template": "Response String",
  "unanswerableTemplate": "Response String",
  "totalWordsCount": 456,
  "outputMode": {},
  "rawOutputFormat": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String"
    }
  ]
}

```

```
],
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "description": "Response String",
    "displayName": "Response String",
    "toolType": "Response String"
  }
],
"skills": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "description": "Response String"
  }
],
"connectors": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  }
],
"agentMode": {},
"numberOfRetrievedChunks": 456,
"enableEvaluation": false,
"enableInlineCitations": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
"enableImageSearch": false,
"enableClaudeStreamRetry": false,
"customMaxLlmOutputTokens": 456,
"voiceConfig": {
  "enabled": false,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "Response String",
  "turnHandling": null,
  "enableVoiceReply": false
},
"thinkingConfig": null,
"enableToolSearching": false,
"maxAgentIterations": 456,
"agentTimeoutSeconds": 456,
"agentTimeoutMessage": "Response String",
"isMultimodalLlm": false,
"enableSizeBasedToolResultTruncation": false,
"toolResultMaxChars": 456,
"enableVectorMemory": false,
"enableCrossConversationSearch": false,
"crossConversationLookbackDays": 456,
"useGatewayFallback": false,
```



```
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": false,
"enableGetMessageContextTool": false,
"enableReadAttachmentFullTextTool": false,
"enableParseAttachmentContentTool": false,
"enableListAvailableParsersTool": false,
"enableGetDownloadLinkTool": false,
"isLocked": false,
"sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
"canUpdate": false,
"canDelete": false
}
```

Delete AI Assistant

DELETE

/api/v1/chatbots/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "

-H "Authorization: Api-Key YOUR_API_KEY"

# Please replace YOUR_API_KEY and verify request data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request (payload)
const data = null;

axios.delete(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Duplicate AI Assistant

POST

</api/v1/chatbots/{id}/duplicate/>

Parameters

Parameter	Required	Type	Description
-----------	----------	------	-------------

id	V	string	
----	---	--------	--

Request

Request Parameters

Field	Type	Required	Description
name	string	Yes	

Request Schema Example

```
{
  "name": string
}
```

request

Request Example Values

```
{
  "name": "Example Name"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name"
}'

# Please replace YOUR_API_KEY and verify request data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request (payload)
const data = {
  "name": "Example Name"
};

axios.post(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request (payload)
data = {
    "name": "Example Name"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 201

Response Schema Example

```

{
  "id": string (uuid)
  "name": string // The bot's name; in Agent mode it has semantic meaning, in other modes it is
only used to distinguish different bots
  "largeLanguageModel": string (uuid) // The large language model used by the bot for
generating responses (including organization-managed AI Gateway models)
  "llm": // The bound model's id and name; read directly from the FK, returned even if the
model is deactivated by the organization
  {
    "id": string (uuid)
    "name": string
  }
  "embeddingModel?": string (uuid) // Embedding model used for vectorizing text, optional field

```

```

(Optional)
  "rerankerModel"?: string (uuid) // Model used for reranking, optional field (Optional)
  "instructions"?: string // The bot's role instructions, used to describe the bot's role and
behavior (Optional)
  "knowledgeBases"?: [ // List of knowledge bases accessible to the bot (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "databases"?: [ // List of SQL databases accessible to the bot (for Text-to-SQL feature)
(Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "updatedAt": string (timestamp)
  "organization"?: string (uuid) // The organization the bot belongs to; if empty, it is a
personal bot (Optional)
  "builtInWorkflow"?: string (uuid) // Built-in workflow for predefined processing flows
(Optional)
  "replyMode"?: // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `workflow` - Workflow
* `agent` - Agent (Optional)
  {
  }
  "template"?: string // Template used in template mode and hybrid mode (Optional)
  "unanswerableTemplate"?: string // Template used when unable to answer in template mode and
hybrid mode (Optional)
  "totalWordsCount"?: integer (int64) // Accumulated total word count (Optional)
  "outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
  {
  }
  "rawOutputFormat"?: object // JSON schema definition for custom output format (Optional)
  "groups"?: [ // List of groups accessible to the bot (Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "tools"?: [ // List of tools available to the bot (Optional)
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
]

```



```

"skills"?: [ // List of skills associated with the bot (Optional)
  {
    "id": string (uuid)
    "name": string
    "description": string
  }
]
"connectors"?: [ // Connectors associated with this chatbot, gating per-contact OAuth
authorization (Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
{
}
"numberOfRetrievedChunks"?: integer // Number of retrieved reference chunks, default is 12,
minimum is 1 (Optional)
"enableEvaluation"?: boolean // Optional
"enableInlineCitations"?: boolean // Enable inline citation feature, which inserts [1][2]
format citation markers in responses (Optional)
"enableCodeInterpreter"?: boolean // When enabled, provides the Code Interpreter tool to
execute code in an isolated container (Optional)
"enableBrowserTool"?: boolean // Enable browser automation tool for web tasks (Optional)
"enableImageSearch"?: boolean // Provide the image_search tool when any attached knowledge
base uses a multimodal embedding model. Disable to remove the tool entirely. (Optional)
"enableClaudeStreamRetry"?: boolean // When a Claude streaming call stalls (first-chunk or
mid-stream), abort and re-issue the call once in non-streaming mode. Mid-stream retries send a
reset signal so the client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
"customMaxLlmOutputTokens"?: integer // Custom maximum LLM output token count, minimum is 512
(Optional)
"voiceConfig"?: { // Optional
{
  "enabled"?: boolean // Whether this chatbot can be used as a voice agent. Checked by every
conversation platform (WebChat, SIP phone, admin/marketplace voice test) instead of a per-
platform toggle. (Optional)
  "voiceAgentType"?: string (uuid) // Voice agent mode type (Optional)
  "sttProvider"?: string (uuid) // Speech-to-Text service provider (Optional)
  "sttConfig"?: object // Actual configuration parameters for STT (JSON format) (Optional)
  "ttsProvider"?: string (uuid) // Text-to-Speech service provider (Optional)
  "ttsConfig"?: object // Actual configuration parameters for TTS (JSON format) (Optional)
  "realtimeProvider"?: string (uuid) // Realtime end-to-end voice model provider (Optional)
  "realtimeConfig"?: object // Actual configuration parameters for Realtime (JSON format)
(Optional)
  "realtimeInstructions"?: string // Prompt dedicated to the realtime voice session; falls
back to chatbot instructions when empty (Optional)
  "turnHandling"?: object // Voice assistant conversation turn and interruption control
settings (Optional)
  "enableVoiceReply"?: boolean // When disabled, the voice agent replies in text only and
does not speak (Optional)
}
}

```

```

}
"thinkingConfig"?: object // Thinking effort settings (JSON format) (Optional)
"enableToolSearching"?: boolean // Enable dynamic tool search, allowing the agent to
dynamically discover and load tools via tool_searching_tool (Optional)
"maxAgentIterations"?: integer // Maximum number of iterations for AgentWorkflow (1-100,
default: 20) (Optional)
"agentTimeoutSeconds"?: integer // Timeout in seconds for AgentWorkflow execution (30-1200,
default: 600) (Optional)
"agentTimeoutMessage"?: string // Custom user-facing message shown when AgentWorkflow times
out (max 500 chars). Leave empty to use the system default. (Optional)
"isMultimodalLlm": boolean // Whether the chatbot LLM supports multimodal (read-only)
"enableSizeBasedToolResultTruncation"?: boolean // When enabled, tool results exceeding the
character threshold are truncated using head+tail strategy instead of the legacy SYSTEM_TOOLS
whitelist approach. Reduces memory bloat from large tool outputs (code interpreter bash stdout,
web search results, etc.). (Optional)
"toolResultMaxChars"?: integer // Character threshold for size-based tool result truncation
(500-50000). Only effective when enable_size_based_tool_result_truncation is True. (Optional)
"enableVectorMemory"?: boolean // When enabled, archived messages are vectorized and
processed by fact extraction for semantic retrieval (existing behavior). When disabled,
archived messages are discarded and only the active chat history window is used for multi-turn
context. (Optional)
"enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools
also reach this contact's other conversations in the same inbox, within the lookback window.
When disabled, they only reach the current conversation. (Optional)
"crossConversationLookbackDays"?: integer // How far back cross-conversation search may
reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to
90 days. Only effective when enable_cross_conversation_search is True. (Optional)
"useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single
LLM (Optional)
"gatewayFallbackGroup"?: string (uuid) // Optional
"enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current
conversation. (Optional)
"enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a
specific message for context. (Optional)
"enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-
text attachments. AGENT mode only. (Optional)
"enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and
other attachments. AGENT mode only. (Optional)
"enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and
their capabilities. AGENT mode only. (Optional)
"enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links
for produced content. AGENT mode only. (Optional)
"isLocked": boolean // Locked instance: config follows the official version; only the
dedicated KB can be modified
"sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when
instantiated by a rental
"canUpdate": boolean // Return whether the current user can update this resource.
"canDelete": boolean // Return whether the current user can delete this resource.
}

```

response

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",

```

```
"name": "Response String",
"largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
"llm": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String"
},
"embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
"rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
"instructions": "Response String",
"knowledgeBases": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  }
],
"databases": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  }
],
"updatedAt": "Response String",
"organization": "550e8400-e29b-41d4-a716-446655440000",
"builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
"replyMode": {},
"template": "Response String",
"unanswerableTemplate": "Response String",
"totalWordsCount": 456,
"outputMode": {},
"rawOutputFormat": null,
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  }
],
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "description": "Response String",
    "displayName": "Response String",
    "toolType": "Response String"
  }
],
"skills": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "description": "Response String"
  }
],
"connectors": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  }
]
```

```

],
"agentMode": {},
"numberOfRetrievedChunks": 456,
"enableEvaluation": false,
"enableInlineCitations": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
"enableImageSearch": false,
"enableClaudeStreamRetry": false,
"customMaxLlmOutputTokens": 456,
"voiceConfig": {
  "enabled": false,
  "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
  "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
  "sttConfig": null,
  "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
  "ttsConfig": null,
  "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
  "realtimeConfig": null,
  "realtimeInstructions": "Response String",
  "turnHandling": null,
  "enableVoiceReply": false
},
"thinkingConfig": null,
"enableToolSearching": false,
"maxAgentIterations": 456,
"agentTimeoutSeconds": 456,
"agentTimeoutMessage": "Response String",
"isMultimodalLlm": false,
"enableSizeBasedToolResultTruncation": false,
"toolResultMaxChars": 456,
"enableVectorMemory": false,
"enableCrossConversationSearch": false,
"crossConversationLookbackDays": 456,
"useGatewayFallback": false,
"gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
"enableSearchMessagesTool": false,
"enableGetMessageContextTool": false,
"enableReadAttachmentFullTextTool": false,
"enableParseAttachmentContentTool": false,
"enableListAvailableParsersTool": false,
"enableGetDownloadLinkTool": false,
"isLocked": false,
"sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
"canUpdate": false,
"canDelete": false
}

```

Enable or Disable Evaluation

PATCH

/api/v1/chatbots/{id}/enable-evaluation/

Parameters

Parameter	Required	Type	Description
id	V	string	

Request

Request Parameters

Field	Type	Required	Description
enableEvaluation	boolean	No	

Request Schema Example

```
{
  "enableEvaluation"?: boolean // Optional
}
```

request

Request Example Values

```
{
  "enableEvaluation": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "enableEvaluation": true
}'

# Please replace YOUR_API_KEY and verify request data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request (payload)
const data = {
  "enableEvaluation": true
};

axios.patch(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request (payload)
data = {
    "enableEvaluation": true
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "enableEvaluation": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "enableEvaluation"?: boolean // Optional
}
```

response

Response Example Values


```
{  
  "enableEvaluation": false  
}
```

API call example (Shell)

...

目錄

- 1. Create Web Chat
- 2. List Web Chat
- 3. Retrieve Specific Web Chat
- 4. Update Web Chat
- 5. Partial Update Web Chat
- 6. Configure Contact Credentials
- 7. Retrieve Voice Assistant Token
- 8. Upload Batch QA File
- 9. Export Batch QA as Excel
- 10. Retrieve LLM Usage Statistics
- 11. Retrieve AI Assistant Statistics

API call example (Shell)

Create Web Chat

POST /api/v1/web-chats/

Request

Request Parameters

Field	Type	Required	Description
avatar	string (uri)	No	
logo	string (uri)	No	
description	string	No	
cover	string (uri)	No	
backUrl	string	No	
isActive	boolean	No	
theme	object	Yes	
theme.primaryColor	string	No	
theme.navbarTextColor	string	No	
theme.conversationBackgroundColor	string	No	
theme.chatbotMessageTextColor	string	No	
theme.userMessageTextColor	string	No	
theme.chatbotMessageBackgroundColor	string	No	

Field	Type	Required	Description
theme.userMessageBackgroundColor	string	No	
theme.chatbotMessageBackgroundShadowEnabled	boolean	No	
theme.userMessageBackgroundShadowEnabled	boolean	No	
theme.chatbotMessageGradientEnabled	boolean	No	
theme.userMessageGradientEnabled	boolean	No	
darkTheme	object	Yes	
darkTheme.primaryColor	string	No	
darkTheme.navbarTextColor	string	No	
darkTheme.conversationBackgroundColor	string	No	
darkTheme.chatbotMessageTextColor	string	No	
darkTheme.userMessageTextColor	string	No	
darkTheme.chatbotMessageBackgroundColor	string	No	
darkTheme.userMessageBackgroundColor	string	No	
darkTheme.chatbotMessageBackgroundShadowEnabled	boolean	No	
darkTheme.userMessageBackgroundShadowEnabled	boolean	No	
darkTheme.chatbotMessageGradientEnabled	boolean	No	
darkTheme.userMessageGradientEnabled	boolean	No	
enableFileUpload	boolean	No	
enableDisplayCitations	boolean	No	
toolDisplayMode	string (enum: full, compact, hidden)	No	<code>full</code> : Full ; <code>compact</code> : Compact ; <code>hidden</code> : Hidden;
enableAskOptionButtons	boolean	No	When enabled, ask_user_question options appear as a row of buttons above the input box (which remains available). When disabled, the current overlay panel is retained. This is a gradual-rollout switch that can be disabled at any time to roll back.
enableDisplayThinking	boolean	No	
enableUserThinkingControl	boolean	No	
enableDisplayQueryMetadata	boolean	No	
enableDisplayHookMarkers	boolean	No	
hookMarkerTextInputModify	string	No	
hookMarkerTextOutputModify	string	No	

Field	Type	Required	Description
hookMarkerTextInputBlock	string	No	
hookMarkerTextOutputBlock	string	No	
enableShareConversation	boolean	No	
enableLocation	boolean	No	
enableShowPoweredBy	boolean	No	
poweredByLogo	string (uri)	No	
enableAnonymous	boolean	No	
enableDisplayPreviousConversation	boolean	No	
enableDisplayConversationHistory	boolean	No	
enableConversationTimer	boolean	No	
conversationTimerDuration	integer	No	Timer timeout duration, range 3-60 minutes
enableAutoNewConversationOnTimeout	boolean	No	
enableSatisfactionSurvey	boolean	No	
satisfactionSurveyDelayMinutes	integer	No	Minutes after the last AI reply before the survey pops up, range 1-60 minutes
satisfactionSurveyPrompt	string	No	Custom survey title text; leave empty to use the frontend multilingual default text
enableThemeModeToggle	boolean	No	
enableDisplayFontSizeSwitch	boolean	No	
defaultThemeMode	object	No	
enableDownloadCitations	boolean	No	
customDomain	object (with 3 properties: id, domain, status)	No	
customDomain.domain	string	Yes	
enableDisplayChatbotAvatar	boolean	No	
enableDisplayChatbotName	boolean	No	
buttonIcon	string (uri)	No	
conversationStartersDisplayCount	integer	No	
preChatFormEnabled	boolean	No	
preChatFormFields	object	No	
preChatFormTitle	object	No	

Field	Type	Required	Description
preChatFormSubtitle	object	No	
loadingDisplayMode	object	No	Waiting-experience style. <code>narrative</code> shows per-stage copy; <code>brand_animation</code> plays the uploaded asset and hides every system narration line. Defaults to <code>narrative</code> , which with no copy configured beh...
loadingStageCopyByLocale	object	No	Locale-keyed per-stage waiting copy, shaped {locale: {stage: text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a partially filled dict is valid a...
loadingLongWaitCopyByLocale	object	No	Locale-keyed line shown once the wait exceeds <code>loading_long_wait_threshold_seconds</code> , shaped {locale: text}. Written in the brand character voice for the animation mode, but applies to both modes. Empt...
loadingLongWaitThresholdSeconds	integer	No	How long a single reply must be pending before the long-wait copy appears.
loadingAnimationUseSharedAsset	boolean	No	True (default) means every stage plays <code>loading_animation_shared_asset_url</code> , so the admin only uploads once. False expands per-stage assets, each falling back to the shared asset when its own slot is ...
loadingAnimationSharedAssetUrl	string (uri)	No	
loadingAnimationAssetUrlsByStage	object	No	Shaped {stage: url} keyed by LoadingStage. Only consulted when <code>loading_animation_use_shared_asset</code> is False; a missing stage falls back to the shared asset.
loadingTypingShowCopy	boolean	No	Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line....
toolCardRunningIconUrl	string (uri)	No	
toolCardCompletedIconUrl	string (uri)	No	

Request Schema Example

```
{
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
}
```

```

"theme": {
{
  "primaryColor"?: string // Optional
  "navbarTextColor"?: string // Optional
  "conversationBackgroundColor"?: string // Optional
  "chatbotMessageTextColor"?: string // Optional
  "userMessageTextColor"?: string // Optional
  "chatbotMessageBackgroundColor"?: string // Optional
  "userMessageBackgroundColor"?: string // Optional
  "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
  "userMessageBackgroundShadowEnabled"?: boolean // Optional
  "chatbotMessageGradientEnabled"?: boolean // Optional
  "userMessageGradientEnabled"?: boolean // Optional
}
}
"darkTheme": {
{
  "primaryColor"?: string // Optional
  "navbarTextColor"?: string // Optional
  "conversationBackgroundColor"?: string // Optional
  "chatbotMessageTextColor"?: string // Optional
  "userMessageTextColor"?: string // Optional
  "chatbotMessageBackgroundColor"?: string // Optional
  "userMessageBackgroundColor"?: string // Optional
  "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
  "userMessageBackgroundShadowEnabled"?: boolean // Optional
  "chatbotMessageGradientEnabled"?: boolean // Optional
  "userMessageGradientEnabled"?: boolean // Optional
}
}
"enableFileUpload"?: boolean // Optional
"enableDisplayCitations"?: boolean // Optional
"toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
"enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
buttons above the input box (which remains available). When disabled, the current overlay panel is
retained. This is a gradual-rollout switch that can be disabled at any time to roll back. (optional)
"enableDisplayThinking"?: boolean // Optional
"enableUserThinkingControl"?: boolean // Optional
"enableDisplayQueryMetadata"?: boolean // Optional
"enableDisplayHookMarkers"?: boolean // Optional
"hookMarkerTextInputModify"?: string // Optional
"hookMarkerTextOutputModify"?: string // Optional
"hookMarkerTextInputBlock"?: string // Optional
"hookMarkerTextOutputBlock"?: string // Optional
"enableShareConversation"?: boolean // Optional
"enableLocation"?: boolean // Optional
"enableShowPoweredBy"?: boolean // Optional
"poweredByLogo"?: string (uri) // Optional
"enableAnonymous"?: boolean // Optional
"enableDisplayPreviousConversation"?: boolean // Optional
"enableDisplayConversationHistory"?: boolean // Optional
"enableConversationTimer"?: boolean // Optional
"conversationTimerDuration"?: integer // Timer timeout duration, range 3-60 minutes (Optional)
"enableAutoNewConversationOnTimeout"?: boolean // Optional
"enableSatisfactionSurvey"?: boolean // Optional
"satisfactionSurveyDelayMinutes"?: integer // Minutes after the last AI reply before the survey
pops up, range 1-60 minutes (Optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text; leave empty to use the frontend

```

```

multilingual default text (Optional)
  "enableThemeModeToggle"?: boolean // Optional
  "enableDisplayFontSizeSwitch"?: boolean // Optional
  "defaultThemeMode"?: // Optional
  {
  }
  "enableDownloadCitations"?: boolean // Optional
  "customDomain"?: // Optional
  {
    "domain": string
  }
  "enableDisplayChatbotAvatar"?: boolean // Optional
  "enableDisplayChatbotName"?: boolean // Optional
  "buttonIcon"?: string (uri) // Optional
  "conversationStartersDisplayCount"?: integer // Optional
  "preChatFormEnabled"?: boolean // Optional
  "preChatFormFields"?: object // Optional
  "preChatFormTitle"?: object // Optional
  "preChatFormSubtitle"?: object // Optional
  "loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
  `brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
  `narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
  {
  }
  "loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
  text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
  so a partially filled dict is valid and never blocks a reply. (Optional)
  "loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
  `loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
  for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
  "loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
  long-wait copy appears. (Optional)
  "loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
  `loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
  each falling back to the shared asset when its own slot is empty. (Optional)
  "loadingAnimationSharedAssetUrl"?: string (uri) // Optional
  "loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
  consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
  shared asset. (Optional)
  "loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
  existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
  only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
  (Optional)
  "toolCardRunningIconUrl"?: string (uri) // Optional
  "toolCardCompletedIconUrl"?: string (uri) // Optional
}

```

Request Example Values

```

{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
}

```



```
"isActive": true,
"theme": {
  "primaryColor": "example string",
  "navbarTextColor": "example string",
  "conversationBackgroundColor": "example string",
  "chatbotMessageTextColor": "example string",
  "userMessageTextColor": "example string",
  "chatbotMessageBackgroundColor": "example string",
  "userMessageBackgroundColor": "example string",
  "chatbotMessageBackgroundShadowEnabled": true,
  "userMessageBackgroundShadowEnabled": true,
  "chatbotMessageGradientEnabled": true,
  "userMessageGradientEnabled": true
},
"darkTheme": {
  "primaryColor": "example string",
  "navbarTextColor": "example string",
  "conversationBackgroundColor": "example string",
  "chatbotMessageTextColor": "example string",
  "userMessageTextColor": "example string",
  "chatbotMessageBackgroundColor": "example string",
  "userMessageBackgroundColor": "example string",
  "chatbotMessageBackgroundShadowEnabled": true,
  "userMessageBackgroundShadowEnabled": true,
  "chatbotMessageGradientEnabled": true,
  "userMessageGradientEnabled": true
},
"enableFileUpload": true,
"enableDisplayCitations": true,
"toolDisplayMode": "full",
"enableAskOptionButtons": true,
"enableDisplayThinking": true,
"enableUserThinkingControl": true,
"enableDisplayQueryMetadata": true,
"enableDisplayHookMarkers": true,
"hookMarkerTextInputModify": "example string",
"hookMarkerTextOutputModify": "example string",
"hookMarkerTextInputBlock": "example string",
"hookMarkerTextOutputBlock": "example string",
"enableShareConversation": true,
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
```

```
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/web-chats/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "example string",
  "hookMarkerTextOutputModify": "example string",
  "hookMarkerTextInputBlock": "example string",
  "hookMarkerTextOutputBlock": "example string",
  "enableShareConversation": true,
  "enableLocation": true,
```

```
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}'
```

Please make sure to replace YOUR_API_KEY and verify the request data before executing.

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "example string",
  "hookMarkerTextOutputModify": "example string",
```

```

    "hookMarkerTextInputBlock": "example string",
    "hookMarkerTextOutputBlock": "example string",
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
      "domain": "example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
  };

```

```

axios.post("https://api.maiagent.ai/api/v1/web-chats/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });

```

```

import requests

url = "https://api.maiagent.ai/api/v1/web-chats/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "example string",
    "isActive": true,
    "theme": {
        "primaryColor": "example string",
        "navbarTextColor": "example string",
        "conversationBackgroundColor": "example string",
        "chatbotMessageTextColor": "example string",
        "userMessageTextColor": "example string",
        "chatbotMessageBackgroundColor": "example string",
        "userMessageBackgroundColor": "example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "darkTheme": {
        "primaryColor": "example string",
        "navbarTextColor": "example string",
        "conversationBackgroundColor": "example string",
        "chatbotMessageTextColor": "example string",
        "userMessageTextColor": "example string",
        "chatbotMessageBackgroundColor": "example string",
        "userMessageBackgroundColor": "example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
    "hookMarkerTextInputModify": "example string",
    "hookMarkerTextOutputModify": "example string",
    "hookMarkerTextInputBlock": "example string",
    "hookMarkerTextOutputBlock": "example string",

```

```

        "enableShareConversation": true,
        "enableLocation": true,
        "enableShowPoweredBy": true,
        "poweredByLogo": "https://example.com/file.jpg",
        "enableAnonymous": true,
        "enableDisplayPreviousConversation": true,
        "enableDisplayConversationHistory": true,
        "enableConversationTimer": true,
        "conversationTimerDuration": 123,
        "enableAutoNewConversationOnTimeout": true,
        "enableSatisfactionSurvey": true,
        "satisfactionSurveyDelayMinutes": 123,
        "satisfactionSurveyPrompt": "example string",
        "enableThemeModeToggle": true,
        "enableDisplayFontSizeSwitch": true,
        "defaultThemeMode": {},
        "enableDownloadCitations": true,
        "customDomain": {
            "domain": "example string"
        },
        "enableDisplayChatbotAvatar": true,
        "enableDisplayChatbotName": true,
        "buttonIcon": "https://example.com/file.jpg",
        "conversationStartersDisplayCount": 123,
        "preChatFormEnabled": true,
        "preChatFormFields": null,
        "preChatFormTitle": null,
        "preChatFormSubtitle": null,
        "loadingDisplayMode": {},
        "loadingStageCopyByLocale": null,
        "loadingLongWaitCopyByLocale": null,
        "loadingLongWaitThresholdSeconds": 123,
        "loadingAnimationUseSharedAsset": true,
        "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
        "loadingAnimationAssetUrlsByStage": null,
        "loadingTypingShowCopy": true,
        "toolCardRunningIconUrl": "https://example.com/file.jpg",
        "toolCardCompletedIconUrl": "https://example.com/file.jpg"
    }
}

```

```

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)

```



```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/web-chats/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "avatar": "https://example.com/file.jpg",
            "logo": "https://example.com/file.jpg",
            "description": "example string",
            "cover": "https://example.com/file.jpg",
            "backUrl": "example string",
            "isActive": true,
            "theme": {
                "primaryColor": "example string",
                "navbarTextColor": "example string",
                "conversationBackgroundColor": "example string",
                "chatbotMessageTextColor": "example string",
                "userMessageTextColor": "example string",
                "chatbotMessageBackgroundColor": "example string",
                "userMessageBackgroundColor": "example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "darkTheme": {
                "primaryColor": "example string",
                "navbarTextColor": "example string",
                "conversationBackgroundColor": "example string",
                "chatbotMessageTextColor": "example string",
                "userMessageTextColor": "example string",
                "chatbotMessageBackgroundColor": "example string",
                "userMessageBackgroundColor": "example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "enableFileUpload": true,
            "enableDisplayCitations": true,
            "toolDisplayMode": "full",
            "enableAskOptionButtons": true,
            "enableDisplayThinking": true,
            "enableUserThinkingControl": true,
            "enableDisplayQueryMetadata": true,
            "enableDisplayHookMarkers": true,
            "hookMarkerTextInputModify": "example string",
            "hookMarkerTextOutputModify": "example string",

```

```

        "hookMarkerTextInputBlock": "example string",
        "hookMarkerTextOutputBlock": "example string",
        "enableShareConversation": true,
        "enableLocation": true,
        "enableShowPoweredBy": true,
        "poweredByLogo": "https://example.com/file.jpg",
        "enableAnonymous": true,
        "enableDisplayPreviousConversation": true,
        "enableDisplayConversationHistory": true,
        "enableConversationTimer": true,
        "conversationTimerDuration": 123,
        "enableAutoNewConversationOnTimeout": true,
        "enableSatisfactionSurvey": true,
        "satisfactionSurveyDelayMinutes": 123,
        "satisfactionSurveyPrompt": "example string",
        "enableThemeModeToggle": true,
        "enableDisplayFontSizeSwitch": true,
        "defaultThemeMode": {},
        "enableDownloadCitations": true,
        "customDomain": {
            "domain": "example string"
        },
        "enableDisplayChatbotAvatar": true,
        "enableDisplayChatbotName": true,
        "buttonIcon": "https://example.com/file.jpg",
        "conversationStartersDisplayCount": 123,
        "preChatFormEnabled": true,
        "preChatFormFields": null,
        "preChatFormTitle": null,
        "preChatFormSubtitle": null,
        "loadingDisplayMode": {},
        "loadingStageCopyByLocale": null,
        "loadingLongWaitCopyByLocale": null,
        "loadingLongWaitThresholdSeconds": 123,
        "loadingAnimationUseSharedAsset": true,
        "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
        "loadingAnimationAssetUrlsByStage": null,
        "loadingTypingShowCopy": true,
        "toolCardRunningIconUrl": "https://example.com/file.jpg",
        "toolCardCompletedIconUrl": "https://example.com/file.jpg"
    }
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "darkTheme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
  * `compact` - Compact
  * `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
  buttons above the input box (which remains available). When disabled, the current overlay panel is
  retained. This is a gradual-rollout switch that can be disabled at any time to roll back. (optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
}
```

```

"hookMarkerTextInputModify"?: string // Optional
"hookMarkerTextOutputModify"?: string // Optional
"hookMarkerTextInputBlock"?: string // Optional
"hookMarkerTextOutputBlock"?: string // Optional
"enableShareConversation"?: boolean // Optional
"enableLocation"?: boolean // Optional
"enableShowPoweredBy"?: boolean // Optional
"poweredByLogo"?: string (uri) // Optional
"enableAnonymous"?: boolean // Optional
"enableDisplayPreviousConversation"?: boolean // Optional
"enableDisplayConversationHistory"?: boolean // Optional
"enableConversationTimer"?: boolean // Optional
"conversationTimerDuration"?: integer // Timer timeout duration, range 3-60 minutes (Optional)
"enableAutoNewConversationOnTimeout"?: boolean // Optional
"enableSatisfactionSurvey"?: boolean // Optional
"satisfactionSurveyDelayMinutes"?: integer // Minutes after the last AI reply before the survey
pops up, range 1-60 minutes (Optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text; leave empty to use the frontend
multilingual default text (Optional)
"accessType":
{
}
"enableThemeModeToggle"?: boolean // Optional
"enableDisplayFontSizeSwitch"?: boolean // Optional
"defaultThemeMode"?: // Optional
{
}
"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "id": string (uuid)
  "domain": string
  "status":
  {
  }
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
"voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
"conversationStartersDisplayCount"?: integer // Optional
"includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered
conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM
context so the agent can resolve short replies like "1" to the matching starter option.
"tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers ``display_name`` (set on MCP tools, where
``name`` is null) and falls back to ``name`` for API tools where
``display_name`` is null. Tools missing both are skipped – SDK
should never show an unnamed tool to a contact.
  object
]
"preChatFormEnabled"?: boolean // Optional
"preChatFormFields"?: object // Optional
"preChatFormTitle"?: object // Optional
"preChatFormSubtitle"?: object // Optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

```

```

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
{
}
"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage: text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a partially filled dict is valid and never blocks a reply. (Optional)
"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds `loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the long-wait copy appears. (Optional)
"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays `loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets, each falling back to the shared asset when its own slot is empty. (Optional)
"loadingAnimationSharedAssetUrl"?: string (uri) // Optional
"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the shared asset. (Optional)
"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected. (Optional)
"toolCardRunningIconUrl"?: string (uri) // Optional
"toolCardCompletedIconUrl"?: string (uri) // Optional
"inbox": object
}

```

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "response string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "response string",
  "isActive": false,
  "enableSpeech": false,
  "displayConversationStarters": [
    "response string"
  ],
  "theme": {
    "primaryColor": "response string",
    "navbarTextColor": "response string",
    "conversationBackgroundColor": "response string",
    "chatbotMessageTextColor": "response string",
    "userMessageTextColor": "response string",
    "chatbotMessageBackgroundColor": "response string",
    "userMessageBackgroundColor": "response string",
    "chatbotMessageBackgroundShadowEnabled": false,
    "userMessageBackgroundShadowEnabled": false,
    "chatbotMessageGradientEnabled": false,
    "userMessageGradientEnabled": false
  },
  "darkTheme": {

```

```
"primaryColor": "response string",
"navbarTextColor": "response string",
"conversationBackgroundColor": "response string",
"chatbotMessageTextColor": "response string",
"userMessageTextColor": "response string",
"chatbotMessageBackgroundColor": "response string",
"userMessageBackgroundColor": "response string",
"chatbotMessageBackgroundShadowEnabled": false,
"userMessageBackgroundShadowEnabled": false,
"chatbotMessageGradientEnabled": false,
"userMessageGradientEnabled": false
},
"enableFileUpload": false,
"enableDisplayCitations": false,
"toolDisplayMode": "full",
"enableAskOptionButtons": false,
"enableDisplayThinking": false,
"enableUserThinkingControl": false,
"enableDisplayQueryMetadata": false,
"enableDisplayHookMarkers": false,
"hookMarkerTextInputModify": "response string",
"hookMarkerTextOutputModify": "response string",
"hookMarkerTextInputBlock": "response string",
"hookMarkerTextOutputBlock": "response string",
"enableShareConversation": false,
"enableLocation": false,
"enableShowPoweredBy": false,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": false,
"enableDisplayPreviousConversation": false,
"enableDisplayConversationHistory": false,
"enableConversationTimer": false,
"conversationTimerDuration": 456,
"enableAutoNewConversationOnTimeout": false,
"enableSatisfactionSurvey": false,
"satisfactionSurveyDelayMinutes": 456,
"satisfactionSurveyPrompt": "response string",
"accessType": {},
"enableThemeModeToggle": false,
"enableDisplayFontSizeSwitch": false,
"defaultThemeMode": {},
"enableDownloadCitations": false,
"customDomain": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "domain": "response string",
  "status": {}
},
"enableDisplayChatbotAvatar": false,
"enableDisplayChatbotName": false,
"buttonIcon": "https://example.com/file.jpg",
"enableCodeInterpreter": false,
"voiceAgentEnabled": false,
"conversationStartersDisplayCount": 456,
"includeGreetingInContext": false,
"tools": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response example name",
    "description": "response example description"
  }
]
```

```
],
"preChatFormEnabled": false,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 456,
"loadingAnimationUseSharedAsset": false,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": false,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg",
"inbox": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response example name",
  "description": "response example description"
}
}
```

List Web Chat

GET

[/api/v1/web-chats/](#)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/web-chats/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/web-chats/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/web-chats/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "id": string (uuid)
    "avatar?": string (uri) // Optional
    "name": string
  }
]
```

Response Example Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "avatar": "https://example.com/file.jpg",
    "name": "response string"
  }
]
```

```
}  
]
```

Retrieve Specific Web Chat

GET

/api/v1/web-chats/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this web chat.

Code Examples

Shell/Bash

```
# API call example (Shell)  
curl -X GET "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"  
  
-H "Authorization: Api-Key YOUR_API_KEY"  
  
# Please make sure to replace YOUR_API_KEY and verify the request data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```

{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
    }
  }
}

```

```

    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
}
"darkTheme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
}
"enableFileUpload"?: boolean // Optional
"enableDisplayCitations"?: boolean // Optional
"toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
  buttons above the input box (which remains available). When disabled, the current overlay panel is
  retained. This is a gradual-rollout switch that can be disabled at any time to roll back. (optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
  "hookMarkerTextInputModify"?: string // Optional
  "hookMarkerTextOutputModify"?: string // Optional
  "hookMarkerTextInputBlock"?: string // Optional
  "hookMarkerTextOutputBlock"?: string // Optional
  "enableShareConversation"?: boolean // Optional
  "enableLocation"?: boolean // Optional
  "enableShowPoweredBy"?: boolean // Optional
  "poweredByLogo"?: string (uri) // Optional
  "enableAnonymous"?: boolean // Optional
  "enableDisplayPreviousConversation"?: boolean // Optional
  "enableDisplayConversationHistory"?: boolean // Optional
  "enableConversationTimer"?: boolean // Optional
  "conversationTimerDuration"?: integer // Timer timeout duration, range 3-60 minutes (Optional)
  "enableAutoNewConversationOnTimeout"?: boolean // Optional
  "enableSatisfactionSurvey"?: boolean // Optional
  "satisfactionSurveyDelayMinutes"?: integer // Minutes after the last AI reply before the survey
  pops up, range 1-60 minutes (Optional)
  "satisfactionSurveyPrompt"?: string // Custom survey title text; leave empty to use the frontend
  multilingual default text (Optional)
  "accessType":
  {
  }
  "enableThemeModeToggle"?: boolean // Optional
  "enableDisplayFontSizeSwitch"?: boolean // Optional

```

```

"defaultThemeMode"?: // Optional
{
}
"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "id": string (uuid)
  "domain": string
  "status":
  {
  }
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
"voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
"conversationStartersDisplayCount"?: integer // Optional
"includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered
conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM
context so the agent can resolve short replies like "1" to the matching starter option.
"tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers ``display_name`` (set on MCP tools, where
``name`` is null) and falls back to ``name`` for API tools where
``display_name`` is null. Tools missing both are skipped – SDK
should never show an unnamed tool to a contact.
  object
]
"preChatFormEnabled"?: boolean // Optional
"preChatFormFields"?: object // Optional
"preChatFormTitle"?: object // Optional
"preChatFormSubtitle"?: object // Optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
{
}
"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
"loadingAnimationSharedAssetUrl"?: string (uri) // Optional
"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)

```

```
"toolCardRunningIconUrl"?: string (uri) // Optional
"toolCardCompletedIconUrl"?: string (uri) // Optional
"inbox": object
}
```

Response Example Values

```
{
  "id": "123e4567-e89b-12d3-a456-426614174000",
  "name": "test doc",
  "avatar": "https://example.com/static/images/default-chatbot-avatar.png",
  "logo": null,
  "description": "",
  "cover": null,
  "backUrl": "",
  "isActive": true,
  "enableSpeech": false,
  "displayGreeting": null,
  "displayConversationStarters": [],
  "theme": {
    "primaryColor": "#3854d8",
    "dialogColor": "#ddeaf6",
    "navbarTextColor": "#ffffff"
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "enableShareConversation": false,
  "enableLocation": false,
  "enableShowPoweredBy": true,
  "poweredByLogo": null,
  "accessType": "web",
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "test doc",
    "signAuth": {
      "id": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
      "signSource": {
        "id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
        "accessType": "keycloak"
      }
    },
    "signParams": {
      "keycloak": {
        "url": "example.com",
        "realm": "example",
        "clientId": "example"
      }
    }
  }
}
```

Update Web Chat

PUT

/api/v1/web-chats/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this web chat.

Request

Request Parameters

Field	Type	Required	Description
avatar	string (uri)	No	
logo	string (uri)	No	
description	string	No	
cover	string (uri)	No	
backUrl	string	No	
isActive	boolean	No	
theme	object	Yes	
theme.primaryColor	string	No	
theme.navbarTextColor	string	No	
theme.conversationBackgroundColor	string	No	
theme.chatbotMessageTextColor	string	No	
theme.userMessageTextColor	string	No	
theme.chatbotMessageBackgroundColor	string	No	
theme.userMessageBackgroundColor	string	No	
theme.chatbotMessageBackgroundShadowEnabled	boolean	No	
theme.userMessageBackgroundShadowEnabled	boolean	No	
theme.chatbotMessageGradientEnabled	boolean	No	
theme.userMessageGradientEnabled	boolean	No	
darkTheme	object	Yes	
darkTheme.primaryColor	string	No	
darkTheme.navbarTextColor	string	No	
darkTheme.conversationBackgroundColor	string	No	

Field	Type	Required	Description
darkTheme.chatbotMessageTextColor	string	No	
darkTheme.userMessageTextColor	string	No	
darkTheme.chatbotMessageBackgroundColor	string	No	
darkTheme.userMessageBackgroundColor	string	No	
darkTheme.chatbotMessageBackgroundShadowEnabled	boolean	No	
darkTheme.userMessageBackgroundShadowEnabled	boolean	No	
darkTheme.chatbotMessageGradientEnabled	boolean	No	
darkTheme.userMessageGradientEnabled	boolean	No	
enableFileUpload	boolean	No	
enableDisplayCitations	boolean	No	
toolDisplayMode	string (enum: full, compact, hidden)	No	full : Full ; compact : Compact ; hidden : Hidden;
enableAskOptionButtons	boolean	No	When enabled, ask_user_question options appear as a row of buttons above the input box (which remains available). When disabled, the current overlay panel is retained. This is a gradual-rollout switch that can be disabled at any time to roll back.
enableDisplayThinking	boolean	No	
enableUserThinkingControl	boolean	No	
enableDisplayQueryMetadata	boolean	No	
enableDisplayHookMarkers	boolean	No	
hookMarkerTextInputModify	string	No	
hookMarkerTextOutputModify	string	No	
hookMarkerTextInputBlock	string	No	
hookMarkerTextOutputBlock	string	No	
enableShareConversation	boolean	No	
enableLocation	boolean	No	
enableShowPoweredBy	boolean	No	
poweredByLogo	string (uri)	No	
enableAnonymous	boolean	No	
enableDisplayPreviousConversation	boolean	No	
enableDisplayConversationHistory	boolean	No	

Field	Type	Required	Description
enableConversationTimer	boolean	No	
conversationTimerDuration	integer	No	Timer timeout duration, range 3-60 minutes
enableAutoNewConversationOnTimeout	boolean	No	
enableSatisfactionSurvey	boolean	No	
satisfactionSurveyDelayMinutes	integer	No	Minutes after the last AI reply before the survey pops up, range 1-60 minutes
satisfactionSurveyPrompt	string	No	Custom survey title text; leave empty to use the frontend multilingual default text
enableThemeModeToggle	boolean	No	
enableDisplayFontSizeSwitch	boolean	No	
defaultThemeMode	object	No	
enableDownloadCitations	boolean	No	
customDomain	object (with 3 properties: id, domain, status)	No	
customDomain.domain	string	Yes	
enableDisplayChatbotAvatar	boolean	No	
enableDisplayChatbotName	boolean	No	
buttonIcon	string (uri)	No	
conversationStartersDisplayCount	integer	No	
preChatFormEnabled	boolean	No	
preChatFormFields	object	No	
preChatFormTitle	object	No	
preChatFormSubtitle	object	No	
loadingDisplayMode	object	No	Waiting-experience style. narrative shows per-stage copy; brand_animation plays the uploaded asset and hides every system narration line. Defaults to narrative , which with no copy configured beh...
loadingStageCopyByLocale	object	No	Locale-keyed per-stage waiting copy, shaped {locale: {stage: text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a partially filled dict is valid a...
loadingLongWaitCopyByLocale	object	No	Locale-keyed line shown once the wait exceeds

Field	Type	Required	Description
			<code>loading_long_wait_threshold_seconds</code> , shaped {locale: text}. Written in the brand character voice for the animation mode, but applies to both modes. Empt...
loadingLongWaitThresholdSeconds	integer	No	How long a single reply must be pending before the long-wait copy appears.
loadingAnimationUseSharedAsset	boolean	No	True (default) means every stage plays <code>loading_animation_shared_asset_url</code> , so the admin only uploads once. False expands per-stage assets, each falling back to the shared asset when its own slot is ...
loadingAnimationSharedAssetUrl	string (uri)	No	
loadingAnimationAssetUrlsByStage	object	No	Shaped {stage: url} keyed by LoadingStage. Only consulted when <code>loading_animation_use_shared_asset</code> is False; a missing stage falls back to the shared asset.
loadingTypingShowCopy	boolean	No	Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line....
toolCardRunningIconUrl	string (uri)	No	
toolCardCompletedIconUrl	string (uri)	No	

Request Schema Example

```
{
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "theme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "darkTheme": {
```

```

{
  "primaryColor"?: string // Optional
  "navbarTextColor"?: string // Optional
  "conversationBackgroundColor"?: string // Optional
  "chatbotMessageTextColor"?: string // Optional
  "userMessageTextColor"?: string // Optional
  "chatbotMessageBackgroundColor"?: string // Optional
  "userMessageBackgroundColor"?: string // Optional
  "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
  "userMessageBackgroundShadowEnabled"?: boolean // Optional
  "chatbotMessageGradientEnabled"?: boolean // Optional
  "userMessageGradientEnabled"?: boolean // Optional
}
}
"enableFileUpload"?: boolean // Optional
"enableDisplayCitations"?: boolean // Optional
"toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
"enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
buttons above the input box (which remains available). When disabled, the current overlay panel is
retained. This is a gradual-rollout switch that can be disabled at any time to roll back. (optional)
"enableDisplayThinking"?: boolean // Optional
"enableUserThinkingControl"?: boolean // Optional
"enableDisplayQueryMetadata"?: boolean // Optional
"enableDisplayHookMarkers"?: boolean // Optional
"hookMarkerTextInputModify"?: string // Optional
"hookMarkerTextOutputModify"?: string // Optional
"hookMarkerTextInputBlock"?: string // Optional
"hookMarkerTextOutputBlock"?: string // Optional
"enableShareConversation"?: boolean // Optional
"enableLocation"?: boolean // Optional
"enableShowPoweredBy"?: boolean // Optional
"poweredByLogo"?: string (uri) // Optional
"enableAnonymous"?: boolean // Optional
"enableDisplayPreviousConversation"?: boolean // Optional
"enableDisplayConversationHistory"?: boolean // Optional
"enableConversationTimer"?: boolean // Optional
"conversationTimerDuration"?: integer // Timer timeout duration, range 3-60 minutes (Optional)
"enableAutoNewConversationOnTimeout"?: boolean // Optional
"enableSatisfactionSurvey"?: boolean // Optional
"satisfactionSurveyDelayMinutes"?: integer // Minutes after the last AI reply before the survey
pops up, range 1-60 minutes (Optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text; leave empty to use the frontend
multilingual default text (Optional)
"enableThemeModeToggle"?: boolean // Optional
"enableDisplayFontSizeSwitch"?: boolean // Optional
"defaultThemeMode"?: // Optional
{
}
"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "domain": string
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"conversationStartersDisplayCount"?: integer // Optional
"preChatFormEnabled"?: boolean // Optional

```

```

"preChatFormFields"?: object // Optional
"preChatFormTitle"?: object // Optional
"preChatFormSubtitle"?: object // Optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
{
}
"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
"loadingAnimationSharedAssetUrl"?: string (uri) // Optional
"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
"toolCardRunningIconUrl"?: string (uri) // Optional
"toolCardCompletedIconUrl"?: string (uri) // Optional
}

```

Request Example Values

```

{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "example string",

```

```
"navbarTextColor": "example string",
"conversationBackgroundColor": "example string",
"chatbotMessageTextColor": "example string",
"userMessageTextColor": "example string",
"chatbotMessageBackgroundColor": "example string",
"userMessageBackgroundColor": "example string",
"chatbotMessageBackgroundShadowEnabled": true,
"userMessageBackgroundShadowEnabled": true,
"chatbotMessageGradientEnabled": true,
"userMessageGradientEnabled": true
},
"enableFileUpload": true,
"enableDisplayCitations": true,
"toolDisplayMode": "full",
"enableAskOptionButtons": true,
"enableDisplayThinking": true,
"enableUserThinkingControl": true,
"enableDisplayQueryMetadata": true,
"enableDisplayHookMarkers": true,
"hookMarkerTextInputModify": "example string",
"hookMarkerTextOutputModify": "example string",
"hookMarkerTextInputBlock": "example string",
"hookMarkerTextOutputBlock": "example string",
"enableShareConversation": true,
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
```

```
"toolCardCompletedIconUrl": "https://example.com/file.jpg"  
}
```

Code Examples

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "example string",
  "hookMarkerTextOutputModify": "example string",
  "hookMarkerTextInputBlock": "example string",
  "hookMarkerTextOutputBlock": "example string",
  "enableShareConversation": true,
  "enableLocation": true,
```



```
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}'
```

Please make sure to replace YOUR_API_KEY and verify the request data before executing.

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "example string",
  "hookMarkerTextOutputModify": "example string",
```

```

    "hookMarkerTextInputBlock": "example string",
    "hookMarkerTextOutputBlock": "example string",
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
      "domain": "example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
  };

```

```

axios.put("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });

```

```

import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "example string",
    "isActive": true,
    "theme": {
        "primaryColor": "example string",
        "navbarTextColor": "example string",
        "conversationBackgroundColor": "example string",
        "chatbotMessageTextColor": "example string",
        "userMessageTextColor": "example string",
        "chatbotMessageBackgroundColor": "example string",
        "userMessageBackgroundColor": "example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "darkTheme": {
        "primaryColor": "example string",
        "navbarTextColor": "example string",
        "conversationBackgroundColor": "example string",
        "chatbotMessageTextColor": "example string",
        "userMessageTextColor": "example string",
        "chatbotMessageBackgroundColor": "example string",
        "userMessageBackgroundColor": "example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
    "hookMarkerTextInputModify": "example string",
    "hookMarkerTextOutputModify": "example string",
    "hookMarkerTextInputBlock": "example string",
    "hookMarkerTextOutputBlock": "example string",

```

```
"enableShareConversation": true,
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
    "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}
```

```
response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "avatar": "https://example.com/file.jpg",
            "logo": "https://example.com/file.jpg",
            "description": "example string",
            "cover": "https://example.com/file.jpg",
            "backUrl": "example string",
            "isActive": true,
            "theme": {
                "primaryColor": "example string",
                "navbarTextColor": "example string",
                "conversationBackgroundColor": "example string",
                "chatbotMessageTextColor": "example string",
                "userMessageTextColor": "example string",
                "chatbotMessageBackgroundColor": "example string",
                "userMessageBackgroundColor": "example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "darkTheme": {
                "primaryColor": "example string",
                "navbarTextColor": "example string",
                "conversationBackgroundColor": "example string",
                "chatbotMessageTextColor": "example string",
                "userMessageTextColor": "example string",
                "chatbotMessageBackgroundColor": "example string",
                "userMessageBackgroundColor": "example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "enableFileUpload": true,
            "enableDisplayCitations": true,
            "toolDisplayMode": "full",
            "enableAskOptionButtons": true,
            "enableDisplayThinking": true,
            "enableUserThinkingControl": true,
            "enableDisplayQueryMetadata": true,
            "enableDisplayHookMarkers": true,
            "hookMarkerTextInputModify": "example string",

```

```

        "hookMarkerTextOutputModify": "example string",
        "hookMarkerTextInputBlock": "example string",
        "hookMarkerTextOutputBlock": "example string",
        "enableShareConversation": true,
        "enableLocation": true,
        "enableShowPoweredBy": true,
        "poweredByLogo": "https://example.com/file.jpg",
        "enableAnonymous": true,
        "enableDisplayPreviousConversation": true,
        "enableDisplayConversationHistory": true,
        "enableConversationTimer": true,
        "conversationTimerDuration": 123,
        "enableAutoNewConversationOnTimeout": true,
        "enableSatisfactionSurvey": true,
        "satisfactionSurveyDelayMinutes": 123,
        "satisfactionSurveyPrompt": "example string",
        "enableThemeModeToggle": true,
        "enableDisplayFontSizeSwitch": true,
        "defaultThemeMode": {},
        "enableDownloadCitations": true,
        "customDomain": {
            "domain": "example string"
        },
        "enableDisplayChatbotAvatar": true,
        "enableDisplayChatbotName": true,
        "buttonIcon": "https://example.com/file.jpg",
        "conversationStartersDisplayCount": 123,
        "preChatFormEnabled": true,
        "preChatFormFields": null,
        "preChatFormTitle": null,
        "preChatFormSubtitle": null,
        "loadingDisplayMode": {},
        "loadingStageCopyByLocale": null,
        "loadingLongWaitCopyByLocale": null,
        "loadingLongWaitThresholdSeconds": 123,
        "loadingAnimationUseSharedAsset": true,
        "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
        "loadingAnimationAssetUrlsByStage": null,
        "loadingTypingShowCopy": true,
        "toolCardRunningIconUrl": "https://example.com/file.jpg",
        "toolCardCompletedIconUrl": "https://example.com/file.jpg"
    }
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "darkTheme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
  * `compact` - Compact
  * `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
  buttons above the input box (which remains available). When disabled, the current overlay panel is
  retained. This is a gradual-rollout switch that can be disabled at any time to roll back. (optional)
```



```

"enableDisplayThinking"?: boolean // Optional
"enableUserThinkingControl"?: boolean // Optional
"enableDisplayQueryMetadata"?: boolean // Optional
"enableDisplayHookMarkers"?: boolean // Optional
"hookMarkerTextInputModify"?: string // Optional
"hookMarkerTextOutputModify"?: string // Optional
"hookMarkerTextInputBlock"?: string // Optional
"hookMarkerTextOutputBlock"?: string // Optional
"enableShareConversation"?: boolean // Optional
"enableLocation"?: boolean // Optional
"enableShowPoweredBy"?: boolean // Optional
"poweredByLogo"?: string (uri) // Optional
"enableAnonymous"?: boolean // Optional
"enableDisplayPreviousConversation"?: boolean // Optional
"enableDisplayConversationHistory"?: boolean // Optional
"enableConversationTimer"?: boolean // Optional
"conversationTimerDuration"?: integer // Timer timeout duration, range 3-60 minutes (Optional)
"enableAutoNewConversationOnTimeout"?: boolean // Optional
"enableSatisfactionSurvey"?: boolean // Optional
"satisfactionSurveyDelayMinutes"?: integer // Minutes after the last AI reply before the survey
pops up, range 1-60 minutes (Optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text; leave empty to use the frontend
multilingual default text (Optional)
"accessType":
{
}
"enableThemeModeToggle"?: boolean // Optional
"enableDisplayFontSizeSwitch"?: boolean // Optional
"defaultThemeMode"?: // Optional
{
}
"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "id": string (uuid)
  "domain": string
  "status":
  {
  }
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
"voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
"conversationStartersDisplayCount"?: integer // Optional
"includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered
conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM
context so the agent can resolve short replies like "1" to the matching starter option.
"tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers ``display_name`` (set on MCP tools, where
``name`` is null) and falls back to ``name`` for API tools where
``display_name`` is null. Tools missing both are skipped – SDK
should never show an unnamed tool to a contact.
  object
]
"preChatFormEnabled"?: boolean // Optional
"preChatFormFields"?: object // Optional
"preChatFormTitle"?: object // Optional

```

```

    "preChatFormSubtitle"?: object // Optional
    "loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
    `brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
    `narrative`, which with no copy configured behaves exactly as before this field existed.

    * `narrative` - Narrative copy
    * `brand_animation` - Brand animation (Optional)
    {
    }

    "loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
    text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
    so a partially filled dict is valid and never blocks a reply. (Optional)

    "loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
    `loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
    for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)

    "loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
    long-wait copy appears. (Optional)

    "loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
    `loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
    each falling back to the shared asset when its own slot is empty. (Optional)

    "loadingAnimationSharedAssetUrl"?: string (uri) // Optional

    "loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
    consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
    shared asset. (Optional)

    "loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
    existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
    only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
    (Optional)

    "toolCardRunningIconUrl"?: string (uri) // Optional
    "toolCardCompletedIconUrl"?: string (uri) // Optional
    "inbox": object
  }

```

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "response string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "response string",
  "isActive": false,
  "enableSpeech": false,
  "displayConversationStarters": [
    "response string"
  ],
  "theme": {
    "primaryColor": "response string",
    "navbarTextColor": "response string",
    "conversationBackgroundColor": "response string",
    "chatbotMessageTextColor": "response string",
    "userMessageTextColor": "response string",
    "chatbotMessageBackgroundColor": "response string",
    "userMessageBackgroundColor": "response string",
    "chatbotMessageBackgroundShadowEnabled": false,
    "userMessageBackgroundShadowEnabled": false,
  }
}

```

```
"chatbotMessageGradientEnabled": false,
"userMessageGradientEnabled": false
},
"darkTheme": {
  "primaryColor": "response string",
  "navbarTextColor": "response string",
  "conversationBackgroundColor": "response string",
  "chatbotMessageTextColor": "response string",
  "userMessageTextColor": "response string",
  "chatbotMessageBackgroundColor": "response string",
  "userMessageBackgroundColor": "response string",
  "chatbotMessageBackgroundShadowEnabled": false,
  "userMessageBackgroundShadowEnabled": false,
  "chatbotMessageGradientEnabled": false,
  "userMessageGradientEnabled": false
},
"enableFileUpload": false,
"enableDisplayCitations": false,
"toolDisplayMode": "full",
"enableAskOptionButtons": false,
"enableDisplayThinking": false,
"enableUserThinkingControl": false,
"enableDisplayQueryMetadata": false,
"enableDisplayHookMarkers": false,
"hookMarkerTextInputModify": "response string",
"hookMarkerTextOutputModify": "response string",
"hookMarkerTextInputBlock": "response string",
"hookMarkerTextOutputBlock": "response string",
"enableShareConversation": false,
"enableLocation": false,
"enableShowPoweredBy": false,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": false,
"enableDisplayPreviousConversation": false,
"enableDisplayConversationHistory": false,
"enableConversationTimer": false,
"conversationTimerDuration": 456,
"enableAutoNewConversationOnTimeout": false,
"enableSatisfactionSurvey": false,
"satisfactionSurveyDelayMinutes": 456,
"satisfactionSurveyPrompt": "response string",
"accessType": {},
"enableThemeModeToggle": false,
"enableDisplayFontSizeSwitch": false,
"defaultThemeMode": {},
"enableDownloadCitations": false,
"customDomain": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "domain": "response string",
  "status": {}
},
"enableDisplayChatbotAvatar": false,
"enableDisplayChatbotName": false,
"buttonIcon": "https://example.com/file.jpg",
"enableCodeInterpreter": false,
"voiceAgentEnabled": false,
"conversationStartersDisplayCount": 456,
"includeGreetingInContext": false,
"tools": [
  {
```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response example name",
    "description": "response example description"
  }
],
"preChatFormEnabled": false,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 456,
"loadingAnimationUseSharedAsset": false,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": false,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg",
"inbox": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response example name",
  "description": "response example description"
}
}
```

Partial Update Web Chat

PATCH `/api/v1/web-chats/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this web chat.

Request

Request Parameters

Field	Type	Required	Description
avatar	string (uri)	No	
logo	string (uri)	No	
description	string	No	

Field	Type	Required	Description
cover	string (uri)	No	
backUrl	string	No	
isActive	boolean	No	
theme	object	No	
theme.primaryColor	string	No	
theme.navbarTextColor	string	No	
theme.conversationBackgroundColor	string	No	
theme.chatbotMessageTextColor	string	No	
theme.userMessageTextColor	string	No	
theme.chatbotMessageBackgroundColor	string	No	
theme.userMessageBackgroundColor	string	No	
theme.chatbotMessageBackgroundShadowEnabled	boolean	No	
theme.userMessageBackgroundShadowEnabled	boolean	No	
theme.chatbotMessageGradientEnabled	boolean	No	
theme.userMessageGradientEnabled	boolean	No	
darkTheme	object	No	
darkTheme.primaryColor	string	No	
darkTheme.navbarTextColor	string	No	
darkTheme.conversationBackgroundColor	string	No	
darkTheme.chatbotMessageTextColor	string	No	
darkTheme.userMessageTextColor	string	No	
darkTheme.chatbotMessageBackgroundColor	string	No	
darkTheme.userMessageBackgroundColor	string	No	
darkTheme.chatbotMessageBackgroundShadowEnabled	boolean	No	
darkTheme.userMessageBackgroundShadowEnabled	boolean	No	
darkTheme.chatbotMessageGradientEnabled	boolean	No	
darkTheme.userMessageGradientEnabled	boolean	No	
enableFileUpload	boolean	No	
enableDisplayCitations	boolean	No	
toolDisplayMode	string (enum: full, compact, hidden)	No	full : Full ; compact : Compact ; hidden : Hidden;

Field	Type	Required	Description
enableAskOptionButtons	boolean	No	When enabled, ask_user_question options appear as a row of buttons above the input box (which remains available). When disabled, the current overlay panel is retained. This is a gradual-rollout switch that can be disabled at any time to roll back.
enableDisplayThinking	boolean	No	
enableUserThinkingControl	boolean	No	
enableDisplayQueryMetadata	boolean	No	
enableDisplayHookMarkers	boolean	No	
hookMarkerTextInputModify	string	No	
hookMarkerTextOutputModify	string	No	
hookMarkerTextInputBlock	string	No	
hookMarkerTextOutputBlock	string	No	
enableShareConversation	boolean	No	
enableLocation	boolean	No	
enableShowPoweredBy	boolean	No	
poweredByLogo	string (uri)	No	
enableAnonymous	boolean	No	
enableDisplayPreviousConversation	boolean	No	
enableDisplayConversationHistory	boolean	No	
enableConversationTimer	boolean	No	
conversationTimerDuration	integer	No	Timer timeout duration, range 3-60 minutes
enableAutoNewConversationOnTimeout	boolean	No	
enableSatisfactionSurvey	boolean	No	
satisfactionSurveyDelayMinutes	integer	No	Minutes after the last AI reply before the survey pops up, range 1-60 minutes
satisfactionSurveyPrompt	string	No	Custom survey title text; leave empty to use the frontend multilingual default text
enableThemeModeToggle	boolean	No	
enableDisplayFontSizeSwitch	boolean	No	
defaultThemeMode	object	No	
enableDownloadCitations	boolean	No	
customDomain	object (with 3 properties:	No	

Field	Type	Required	Description
	id, domain, status)		
customDomain.domain	string	Yes	
enableDisplayChatbotAvatar	boolean	No	
enableDisplayChatbotName	boolean	No	
buttonIcon	string (uri)	No	
conversationStartersDisplayCount	integer	No	
preChatFormEnabled	boolean	No	
preChatFormFields	object	No	
preChatFormTitle	object	No	
preChatFormSubtitle	object	No	
loadingDisplayMode	object	No	Waiting-experience style. <code>narrative</code> shows per-stage copy; <code>brand_animation</code> plays the uploaded asset and hides every system narration line. Defaults to <code>narrative</code> , which with no copy configured beh...
loadingStageCopyByLocale	object	No	Locale-keyed per-stage waiting copy, shaped {locale: {stage: text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a partially filled dict is valid a...
loadingLongWaitCopyByLocale	object	No	Locale-keyed line shown once the wait exceeds <code>loading_long_wait_threshold_seconds</code> , shaped {locale: text}. Written in the brand character voice for the animation mode, but applies to both modes. Empt...
loadingLongWaitThresholdSeconds	integer	No	How long a single reply must be pending before the long-wait copy appears.
loadingAnimationUseSharedAsset	boolean	No	True (default) means every stage plays <code>loading_animation_shared_asset_url</code> , so the admin only uploads once. False expands per-stage assets, each falling back to the shared asset when its own slot is ...
loadingAnimationSharedAssetUrl	string (uri)	No	
loadingAnimationAssetUrlsByStage	object	No	Shaped {stage: url} keyed by LoadingStage. Only consulted when <code>loading_animation_use_shared_asset</code> is False; a missing stage falls back to the shared asset.
loadingTypingShowCopy	boolean	No	Only affects the first "typing" stage. True (default, the pre-existing behavior)

Field	Type	Required	Description
			shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line....
toolCardRunningIconUrl	string (uri)	No	
toolCardCompletedIconUrl	string (uri)	No	

Request Schema Example

```
{
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "theme"?: { // Optional
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "darkTheme"?: { // Optional
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
  * `compact` - Compact
  * `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
  buttons above the input box (which remains available). When disabled, the current overlay panel is
  retained. This is a gradual-rollout switch that can be disabled at any time to roll back. (optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
}
```



```

"enableDisplayQueryMetadata"?: boolean // Optional
"enableDisplayHookMarkers"?: boolean // Optional
"hookMarkerTextInputModify"?: string // Optional
"hookMarkerTextOutputModify"?: string // Optional
"hookMarkerTextInputBlock"?: string // Optional
"hookMarkerTextOutputBlock"?: string // Optional
"enableShareConversation"?: boolean // Optional
"enableLocation"?: boolean // Optional
"enableShowPoweredBy"?: boolean // Optional
"poweredByLogo"?: string (uri) // Optional
"enableAnonymous"?: boolean // Optional
"enableDisplayPreviousConversation"?: boolean // Optional
"enableDisplayConversationHistory"?: boolean // Optional
"enableConversationTimer"?: boolean // Optional
"conversationTimerDuration"?: integer // Timer timeout duration, range 3-60 minutes (Optional)
"enableAutoNewConversationOnTimeout"?: boolean // Optional
"enableSatisfactionSurvey"?: boolean // Optional
"satisfactionSurveyDelayMinutes"?: integer // Minutes after the last AI reply before the survey
pops up, range 1-60 minutes (Optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text; leave empty to use the frontend
multilingual default text (Optional)
"enableThemeModeToggle"?: boolean // Optional
"enableDisplayFontSizeSwitch"?: boolean // Optional
"defaultThemeMode"?: // Optional
{
}
"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "domain": string
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"conversationStartersDisplayCount"?: integer // Optional
"preChatFormEnabled"?: boolean // Optional
"preChatFormFields"?: object // Optional
"preChatFormTitle"?: object // Optional
"preChatFormSubtitle"?: object // Optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
{
}
"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
"loadingAnimationSharedAssetUrl"?: string (uri) // Optional
"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only

```

consulted when `loading\_animation\_use\_shared\_asset` is False; a missing stage falls back to the shared asset. (Optional)

"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected. (Optional)

"toolCardRunningIconUrl"?: string (uri) // Optional
"toolCardCompletedIconUrl"?: string (uri) // Optional
}

Request Example Values

```
{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "example string",
  "hookMarkerTextOutputModify": "example string",
  "hookMarkerTextInputBlock": "example string",
  "hookMarkerTextOutputBlock": "example string",
  "enableShareConversation": true,
```

```
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}
```

Code Examples

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "example string",
  "hookMarkerTextOutputModify": "example string",
  "hookMarkerTextInputBlock": "example string",
  "hookMarkerTextOutputBlock": "example string",
  "enableShareConversation": true,
  "enableLocation": true,
```

```
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}'
```

Please make sure to replace YOUR_API_KEY and verify the request data before executing.

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "example string",
  "hookMarkerTextOutputModify": "example string",
```

```

    "hookMarkerTextInputBlock": "example string",
    "hookMarkerTextOutputBlock": "example string",
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
      "domain": "example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
  };

```

```

axios.patch("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });

```

```

import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "example string",
    "isActive": true,
    "theme": {
        "primaryColor": "example string",
        "navbarTextColor": "example string",
        "conversationBackgroundColor": "example string",
        "chatbotMessageTextColor": "example string",
        "userMessageTextColor": "example string",
        "chatbotMessageBackgroundColor": "example string",
        "userMessageBackgroundColor": "example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "darkTheme": {
        "primaryColor": "example string",
        "navbarTextColor": "example string",
        "conversationBackgroundColor": "example string",
        "chatbotMessageTextColor": "example string",
        "userMessageTextColor": "example string",
        "chatbotMessageBackgroundColor": "example string",
        "userMessageBackgroundColor": "example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
    "hookMarkerTextInputModify": "example string",
    "hookMarkerTextOutputModify": "example string",
    "hookMarkerTextInputBlock": "example string",
    "hookMarkerTextOutputBlock": "example string",

```



```

"enableShareConversation": true,
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
    "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}

```

```

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)

```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "avatar": "https://example.com/file.jpg",
            "logo": "https://example.com/file.jpg",
            "description": "example string",
            "cover": "https://example.com/file.jpg",
            "backUrl": "example string",
            "isActive": true,
            "theme": {
                "primaryColor": "example string",
                "navbarTextColor": "example string",
                "conversationBackgroundColor": "example string",
                "chatbotMessageTextColor": "example string",
                "userMessageTextColor": "example string",
                "chatbotMessageBackgroundColor": "example string",
                "userMessageBackgroundColor": "example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "darkTheme": {
                "primaryColor": "example string",
                "navbarTextColor": "example string",
                "conversationBackgroundColor": "example string",
                "chatbotMessageTextColor": "example string",
                "userMessageTextColor": "example string",
                "chatbotMessageBackgroundColor": "example string",
                "userMessageBackgroundColor": "example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "enableFileUpload": true,
            "enableDisplayCitations": true,
            "toolDisplayMode": "full",
            "enableAskOptionButtons": true,
            "enableDisplayThinking": true,
            "enableUserThinkingControl": true,
            "enableDisplayQueryMetadata": true,
            "enableDisplayHookMarkers": true,
            "hookMarkerTextInputModify": "example string",

```

```

        "hookMarkerTextOutputModify": "example string",
        "hookMarkerTextInputBlock": "example string",
        "hookMarkerTextOutputBlock": "example string",
        "enableShareConversation": true,
        "enableLocation": true,
        "enableShowPoweredBy": true,
        "poweredByLogo": "https://example.com/file.jpg",
        "enableAnonymous": true,
        "enableDisplayPreviousConversation": true,
        "enableDisplayConversationHistory": true,
        "enableConversationTimer": true,
        "conversationTimerDuration": 123,
        "enableAutoNewConversationOnTimeout": true,
        "enableSatisfactionSurvey": true,
        "satisfactionSurveyDelayMinutes": 123,
        "satisfactionSurveyPrompt": "example string",
        "enableThemeModeToggle": true,
        "enableDisplayFontSizeSwitch": true,
        "defaultThemeMode": {},
        "enableDownloadCitations": true,
        "customDomain": {
            "domain": "example string"
        },
        "enableDisplayChatbotAvatar": true,
        "enableDisplayChatbotName": true,
        "buttonIcon": "https://example.com/file.jpg",
        "conversationStartersDisplayCount": 123,
        "preChatFormEnabled": true,
        "preChatFormFields": null,
        "preChatFormTitle": null,
        "preChatFormSubtitle": null,
        "loadingDisplayMode": {},
        "loadingStageCopyByLocale": null,
        "loadingLongWaitCopyByLocale": null,
        "loadingLongWaitThresholdSeconds": 123,
        "loadingAnimationUseSharedAsset": true,
        "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
        "loadingAnimationAssetUrlsByStage": null,
        "loadingTypingShowCopy": true,
        "toolCardRunningIconUrl": "https://example.com/file.jpg",
        "toolCardCompletedIconUrl": "https://example.com/file.jpg"
    }
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "darkTheme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
  * `compact` - Compact
  * `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
  buttons above the input box (which remains available). When disabled, the current overlay panel is
  retained. This is a gradual-rollout switch that can be disabled at any time to roll back. (optional)
```

```

"enableDisplayThinking"?: boolean // Optional
"enableUserThinkingControl"?: boolean // Optional
"enableDisplayQueryMetadata"?: boolean // Optional
"enableDisplayHookMarkers"?: boolean // Optional
"hookMarkerTextInputModify"?: string // Optional
"hookMarkerTextOutputModify"?: string // Optional
"hookMarkerTextInputBlock"?: string // Optional
"hookMarkerTextOutputBlock"?: string // Optional
"enableShareConversation"?: boolean // Optional
"enableLocation"?: boolean // Optional
"enableShowPoweredBy"?: boolean // Optional
"poweredByLogo"?: string (uri) // Optional
"enableAnonymous"?: boolean // Optional
"enableDisplayPreviousConversation"?: boolean // Optional
"enableDisplayConversationHistory"?: boolean // Optional
"enableConversationTimer"?: boolean // Optional
"conversationTimerDuration"?: integer // Timer timeout duration, range 3-60 minutes (Optional)
"enableAutoNewConversationOnTimeout"?: boolean // Optional
"enableSatisfactionSurvey"?: boolean // Optional
"satisfactionSurveyDelayMinutes"?: integer // Minutes after the last AI reply before the survey
pops up, range 1-60 minutes (Optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text; leave empty to use the frontend
multilingual default text (Optional)
"accessType":
{
}
"enableThemeModeToggle"?: boolean // Optional
"enableDisplayFontSizeSwitch"?: boolean // Optional
"defaultThemeMode"?: // Optional
{
}
"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "id": string (uuid)
  "domain": string
  "status":
  {
  }
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
"voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
"conversationStartersDisplayCount"?: integer // Optional
"includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered
conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM
context so the agent can resolve short replies like "1" to the matching starter option.
"tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers ``display_name`` (set on MCP tools, where
``name`` is null) and falls back to ``name`` for API tools where
``display_name`` is null. Tools missing both are skipped – SDK
should never show an unnamed tool to a contact.
  object
]
"preChatFormEnabled"?: boolean // Optional
"preChatFormFields"?: object // Optional
"preChatFormTitle"?: object // Optional

```

```

"preChatFormSubtitle"?: object // Optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
{
}

"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)

"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)

"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)

"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)

"loadingAnimationSharedAssetUrl"?: string (uri) // Optional

"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)

"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)

"toolCardRunningIconUrl"?: string (uri) // Optional
"toolCardCompletedIconUrl"?: string (uri) // Optional
"inbox": object
}

```

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "response string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "response string",
  "isActive": false,
  "enableSpeech": false,
  "displayConversationStarters": [
    "response string"
  ],
  "theme": {
    "primaryColor": "response string",
    "navbarTextColor": "response string",
    "conversationBackgroundColor": "response string",
    "chatbotMessageTextColor": "response string",
    "userMessageTextColor": "response string",
    "chatbotMessageBackgroundColor": "response string",
    "userMessageBackgroundColor": "response string",
    "chatbotMessageBackgroundShadowEnabled": false,
    "userMessageBackgroundShadowEnabled": false,
  }
}

```

```
"chatbotMessageGradientEnabled": false,
"userMessageGradientEnabled": false
},
"darkTheme": {
  "primaryColor": "response string",
  "navbarTextColor": "response string",
  "conversationBackgroundColor": "response string",
  "chatbotMessageTextColor": "response string",
  "userMessageTextColor": "response string",
  "chatbotMessageBackgroundColor": "response string",
  "userMessageBackgroundColor": "response string",
  "chatbotMessageBackgroundShadowEnabled": false,
  "userMessageBackgroundShadowEnabled": false,
  "chatbotMessageGradientEnabled": false,
  "userMessageGradientEnabled": false
},
"enableFileUpload": false,
"enableDisplayCitations": false,
"toolDisplayMode": "full",
"enableAskOptionButtons": false,
"enableDisplayThinking": false,
"enableUserThinkingControl": false,
"enableDisplayQueryMetadata": false,
"enableDisplayHookMarkers": false,
"hookMarkerTextInputModify": "response string",
"hookMarkerTextOutputModify": "response string",
"hookMarkerTextInputBlock": "response string",
"hookMarkerTextOutputBlock": "response string",
"enableShareConversation": false,
"enableLocation": false,
"enableShowPoweredBy": false,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": false,
"enableDisplayPreviousConversation": false,
"enableDisplayConversationHistory": false,
"enableConversationTimer": false,
"conversationTimerDuration": 456,
"enableAutoNewConversationOnTimeout": false,
"enableSatisfactionSurvey": false,
"satisfactionSurveyDelayMinutes": 456,
"satisfactionSurveyPrompt": "response string",
"accessType": {},
"enableThemeModeToggle": false,
"enableDisplayFontSizeSwitch": false,
"defaultThemeMode": {},
"enableDownloadCitations": false,
"customDomain": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "domain": "response string",
  "status": {}
},
"enableDisplayChatbotAvatar": false,
"enableDisplayChatbotName": false,
"buttonIcon": "https://example.com/file.jpg",
"enableCodeInterpreter": false,
"voiceAgentEnabled": false,
"conversationStartersDisplayCount": 456,
"includeGreetingInContext": false,
"tools": [
  {
```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response example name",
    "description": "response example description"
  }
],
"preChatFormEnabled": false,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 456,
"loadingAnimationUseSharedAsset": false,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": false,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg",
"inbox": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response example name",
  "description": "response example description"
}
}
```

Configure Contact Credentials

POST </api/v1/web-chats/{id}/setup-contact-credentials/>

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this web chat.

Request

Request Parameters

Field	Type	Required	Description
<code>sourceId</code>	string	Yes	Unique identifier for the contact
<code>name</code>	string	No	Contact display name
<code>mcpCredentials</code>	object (with 2 properties: <code>toolId</code> , <code>headers</code>)	No	MCP credentials configuration

Field	Type	Required	Description
mcpCredentials.toolId	string (uuid)	Yes	MCP Tool ID
mcpCredentials.headers	object	No	Authentication headers

Request Schema Example

```
{
  "sourceId": string // Unique identifier for the contact
  "name"? : string // Contact display name (Optional)
  "mcpCredentials"? : // MCP credentials configuration (Optional)
  {
    "toolId": string (uuid) // MCP Tool ID
    "headers"? : object // Authentication headers (Optional)
  }
}
```

Request Example Values

```
{
  "sourceId": "example string",
  "name": "example name",
  "mcpCredentials": {
    "toolId": "550e8400-e29b-41d4-a716-446655440000",
    "headers": {}
  }
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/setup-contact-credentials/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "sourceId": "example string",
  "name": "example name",
  "mcpCredentials": {
    "toolId": "550e8400-e29b-41d4-a716-446655440000",
    "headers": {}
  }
}'

# Please make sure to replace YOUR_API_KEY and verify the request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "sourceId": "example string",
  "name": "example name",
  "mcpCredentials": {
    "toolId": "550e8400-e29b-41d4-a716-446655440000",
    "headers": {}
  }
};

axios.post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/setup-contact-credentials/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/setup-  
contact-credentials/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "sourceId": "example string",
    "name": "example name",
    "mcpCredentials": {
        "toolId": "550e8400-e29b-41d4-a716-446655440000",
        "headers": {}
    }
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/setup-contact-credentials/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "sourceId": "example string",
            "name": "example name",
            "mcpCredentials": {
                "toolId": "550e8400-e29b-41d4-a716-446655440000",
                "headers": {}
            }
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "contactId": string (uuid) // The created or existing contact ID
}
```

Response Example Values

```
{
  "contactId": "550e8400-e29b-41d4-a716-446655440000"
```

```
}
```

Status Code: **400** - No response body

Retrieve Voice Assistant Token

POST

/api/v1/web-chats/{id}/voice-agent/token/

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this web chat.

Request

Request Parameters

Field	Type	Required	Description
avatar	string (uri)	No	
logo	string (uri)	No	
description	string	No	
cover	string (uri)	No	
backUrl	string	No	
isActive	boolean	No	
theme	object	Yes	
theme.primaryColor	string	No	
theme.navbarTextColor	string	No	
theme.conversationBackgroundColor	string	No	
theme.chatbotMessageTextColor	string	No	
theme.userMessageTextColor	string	No	
theme.chatbotMessageBackgroundColor	string	No	
theme.userMessageBackgroundColor	string	No	
theme.chatbotMessageBackgroundShadowEnabled	boolean	No	
theme.userMessageBackgroundShadowEnabled	boolean	No	

Field	Type	Required	Description
theme.chatbotMessageGradientEnabled	boolean	No	
theme.userMessageGradientEnabled	boolean	No	
darkTheme	object	Yes	
darkTheme.primaryColor	string	No	
darkTheme.navbarTextColor	string	No	
darkTheme.conversationBackgroundColor	string	No	
darkTheme.chatbotMessageTextColor	string	No	
darkTheme.userMessageTextColor	string	No	
darkTheme.chatbotMessageBackgroundColor	string	No	
darkTheme.userMessageBackgroundColor	string	No	
darkTheme.chatbotMessageBackgroundShadowEnabled	boolean	No	
darkTheme.userMessageBackgroundShadowEnabled	boolean	No	
darkTheme.chatbotMessageGradientEnabled	boolean	No	
darkTheme.userMessageGradientEnabled	boolean	No	
enableFileUpload	boolean	No	
enableDisplayCitations	boolean	No	
toolDisplayMode	string (enum: full, compact, hidden)	No	full : Full ; compact : Compact ; hidden : Hidden;
enableAskOptionButtons	boolean	No	When enabled, ask_user_question options appear as a row of buttons above the input box (which remains available). When disabled, the current overlay panel is retained. This is a gradual-rollout switch that can be disabled at any time to roll back.
enableDisplayThinking	boolean	No	
enableUserThinkingControl	boolean	No	
enableDisplayQueryMetadata	boolean	No	
enableDisplayHookMarkers	boolean	No	
hookMarkerTextInputModify	string	No	
hookMarkerTextOutputModify	string	No	
hookMarkerTextInputBlock	string	No	
hookMarkerTextOutputBlock	string	No	
enableShareConversation	boolean	No	

Field	Type	Required	Description
enableLocation	boolean	No	
enableShowPoweredBy	boolean	No	
poweredByLogo	string (uri)	No	
enableAnonymous	boolean	No	
enableDisplayPreviousConversation	boolean	No	
enableDisplayConversationHistory	boolean	No	
enableConversationTimer	boolean	No	
conversationTimerDuration	integer	No	Timer timeout duration, range 3-60 minutes
enableAutoNewConversationOnTimeout	boolean	No	
enableSatisfactionSurvey	boolean	No	
satisfactionSurveyDelayMinutes	integer	No	Minutes after the last AI reply before the survey pops up, range 1-60 minutes
satisfactionSurveyPrompt	string	No	Custom survey title text; leave empty to use the frontend multilingual default text
enableThemeModeToggle	boolean	No	
enableDisplayFontSizeSwitch	boolean	No	
defaultThemeMode	object	No	
enableDownloadCitations	boolean	No	
customDomain	object (with 3 properties: id, domain, status)	No	
customDomain.domain	string	Yes	
enableDisplayChatbotAvatar	boolean	No	
enableDisplayChatbotName	boolean	No	
buttonIcon	string (uri)	No	
conversationStartersDisplayCount	integer	No	
preChatFormEnabled	boolean	No	
preChatFormFields	object	No	
preChatFormTitle	object	No	
preChatFormSubtitle	object	No	
loadingDisplayMode	object	No	Waiting-experience style. narrative shows per-stage copy; brand_animation plays the uploaded asset and hides every system narration line. Defaults to

Field	Type	Required	Description
			<code>narrative</code> , which with no copy configured beh...
loadingStageCopyByLocale	object	No	Locale-keyed per-stage waiting copy, shaped {locale: {stage: text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy, so a partially filled dict is valid a...
loadingLongWaitCopyByLocale	object	No	Locale-keyed line shown once the wait exceeds <code>loading_long_wait_threshold_seconds</code> , shaped {locale: text}. Written in the brand character voice for the animation mode, but applies to both modes. Empt...
loadingLongWaitThresholdSeconds	integer	No	How long a single reply must be pending before the long-wait copy appears.
loadingAnimationUseSharedAsset	boolean	No	True (default) means every stage plays <code>loading_animation_shared_asset_url</code> , so the admin only uploads once. False expands per-stage assets, each falling back to the shared asset when its own slot is ...
loadingAnimationSharedAssetUrl	string (uri)	No	
loadingAnimationAssetUrlsByStage	object	No	Shaped {stage: url} keyed by LoadingStage. Only consulted when <code>loading_animation_use_shared_asset</code> is False; a missing stage falls back to the shared asset.
loadingTypingShowCopy	boolean	No	Only affects the first "typing" stage. True (default, the pre-existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays only the animation with no text line....
toolCardRunningIconUrl	string (uri)	No	
toolCardCompletedIconUrl	string (uri)	No	

Request Schema Example

```
{
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "theme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
    }
  }
}
```

```

    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
}
"darkTheme": {
  {
    "primaryColor"?: string // Optional
    "navbarTextColor"?: string // Optional
    "conversationBackgroundColor"?: string // Optional
    "chatbotMessageTextColor"?: string // Optional
    "userMessageTextColor"?: string // Optional
    "chatbotMessageBackgroundColor"?: string // Optional
    "userMessageBackgroundColor"?: string // Optional
    "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
    "userMessageBackgroundShadowEnabled"?: boolean // Optional
    "chatbotMessageGradientEnabled"?: boolean // Optional
    "userMessageGradientEnabled"?: boolean // Optional
  }
}
"enableFileUpload"?: boolean // Optional
"enableDisplayCitations"?: boolean // Optional
"toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
* `compact` - Compact
* `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
  buttons above the input box (which remains available). When disabled, the current overlay panel is
  retained. This is a gradual-rollout switch that can be disabled at any time to roll back. (optional)
  "enableDisplayThinking"?: boolean // Optional
  "enableUserThinkingControl"?: boolean // Optional
  "enableDisplayQueryMetadata"?: boolean // Optional
  "enableDisplayHookMarkers"?: boolean // Optional
  "hookMarkerTextInputModify"?: string // Optional
  "hookMarkerTextOutputModify"?: string // Optional
  "hookMarkerTextInputBlock"?: string // Optional
  "hookMarkerTextOutputBlock"?: string // Optional
  "enableShareConversation"?: boolean // Optional
  "enableLocation"?: boolean // Optional
  "enableShowPoweredBy"?: boolean // Optional
  "poweredByLogo"?: string (uri) // Optional
  "enableAnonymous"?: boolean // Optional
  "enableDisplayPreviousConversation"?: boolean // Optional
  "enableDisplayConversationHistory"?: boolean // Optional
  "enableConversationTimer"?: boolean // Optional
  "conversationTimerDuration"?: integer // Timer timeout duration, range 3-60 minutes (Optional)
  "enableAutoNewConversationOnTimeout"?: boolean // Optional
  "enableSatisfactionSurvey"?: boolean // Optional
  "satisfactionSurveyDelayMinutes"?: integer // Minutes after the last AI reply before the survey
  pops up, range 1-60 minutes (Optional)
  "satisfactionSurveyPrompt"?: string // Custom survey title text; leave empty to use the frontend
  multilingual default text (Optional)
  "enableThemeModeToggle"?: boolean // Optional
  "enableDisplayFontSizeSwitch"?: boolean // Optional
  "defaultThemeMode"?: // Optional
  {
  }
}

```

```

"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "domain": string
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"conversationStartersDisplayCount"?: integer // Optional
"preChatFormEnabled"?: boolean // Optional
"preChatFormFields"?: object // Optional
"preChatFormTitle"?: object // Optional
"preChatFormSubtitle"?: object // Optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
{
}
"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)
"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)
"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)
"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)
"loadingAnimationSharedAssetUrl"?: string (uri) // Optional
"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)
"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)
"toolCardRunningIconUrl"?: string (uri) // Optional
"toolCardCompletedIconUrl"?: string (uri) // Optional
}

```

Request Example Values

```

{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",

```

```
"userMessageTextColor": "example string",
"chatbotMessageBackgroundColor": "example string",
"userMessageBackgroundColor": "example string",
"chatbotMessageBackgroundShadowEnabled": true,
"userMessageBackgroundShadowEnabled": true,
"chatbotMessageGradientEnabled": true,
"userMessageGradientEnabled": true
},
"darkTheme": {
  "primaryColor": "example string",
  "navbarTextColor": "example string",
  "conversationBackgroundColor": "example string",
  "chatbotMessageTextColor": "example string",
  "userMessageTextColor": "example string",
  "chatbotMessageBackgroundColor": "example string",
  "userMessageBackgroundColor": "example string",
  "chatbotMessageBackgroundShadowEnabled": true,
  "userMessageBackgroundShadowEnabled": true,
  "chatbotMessageGradientEnabled": true,
  "userMessageGradientEnabled": true
},
"enableFileUpload": true,
"enableDisplayCitations": true,
"toolDisplayMode": "full",
"enableAskOptionButtons": true,
"enableDisplayThinking": true,
"enableUserThinkingControl": true,
"enableDisplayQueryMetadata": true,
"enableDisplayHookMarkers": true,
"hookMarkerTextInputModify": "example string",
"hookMarkerTextOutputModify": "example string",
"hookMarkerTextInputBlock": "example string",
"hookMarkerTextOutputBlock": "example string",
"enableShareConversation": true,
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
```

```
"preChatFormSubtitle": null,  
"loadingDisplayMode": {},  
"loadingStageCopyByLocale": null,  
"loadingLongWaitCopyByLocale": null,  
"loadingLongWaitThresholdSeconds": 123,  
"loadingAnimationUseSharedAsset": true,  
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",  
"loadingAnimationAssetUrlsByStage": null,  
"loadingTypingShowCopy": true,  
"toolCardRunningIconUrl": "https://example.com/file.jpg",  
"toolCardCompletedIconUrl": "https://example.com/file.jpg"  
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/voice-agent/token/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "example string",
  "hookMarkerTextOutputModify": "example string",
  "hookMarkerTextInputBlock": "example string",
  "hookMarkerTextOutputBlock": "example string",
  "enableShareConversation": true,
```

```
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
}'
```

```
# Please make sure to replace YOUR_API_KEY and verify the request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "example string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "example string",
  "isActive": true,
  "theme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "darkTheme": {
    "primaryColor": "example string",
    "navbarTextColor": "example string",
    "conversationBackgroundColor": "example string",
    "chatbotMessageTextColor": "example string",
    "userMessageTextColor": "example string",
    "chatbotMessageBackgroundColor": "example string",
    "userMessageBackgroundColor": "example string",
    "chatbotMessageBackgroundShadowEnabled": true,
    "userMessageBackgroundShadowEnabled": true,
    "chatbotMessageGradientEnabled": true,
    "userMessageGradientEnabled": true
  },
  "enableFileUpload": true,
  "enableDisplayCitations": true,
  "toolDisplayMode": "full",
  "enableAskOptionButtons": true,
  "enableDisplayThinking": true,
  "enableUserThinkingControl": true,
  "enableDisplayQueryMetadata": true,
  "enableDisplayHookMarkers": true,
  "hookMarkerTextInputModify": "example string",
  "hookMarkerTextOutputModify": "example string",
```



```

"hookMarkerTextInputBlock": "example string",
"hookMarkerTextOutputBlock": "example string",
"enableShareConversation": true,
"enableLocation": true,
"enableShowPoweredBy": true,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": true,
"enableDisplayPreviousConversation": true,
"enableDisplayConversationHistory": true,
"enableConversationTimer": true,
"conversationTimerDuration": 123,
"enableAutoNewConversationOnTimeout": true,
"enableSatisfactionSurvey": true,
"satisfactionSurveyDelayMinutes": 123,
"satisfactionSurveyPrompt": "example string",
"enableThemeModeToggle": true,
"enableDisplayFontSizeSwitch": true,
"defaultThemeMode": {},
"enableDownloadCitations": true,
"customDomain": {
  "domain": "example string"
},
"enableDisplayChatbotAvatar": true,
"enableDisplayChatbotName": true,
"buttonIcon": "https://example.com/file.jpg",
"conversationStartersDisplayCount": 123,
"preChatFormEnabled": true,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 123,
"loadingAnimationUseSharedAsset": true,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": true,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg"
};

```

```

axios.post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/voice-agent/token/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });

```

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/voice-agent/token/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "avatar": "https://example.com/file.jpg",
    "logo": "https://example.com/file.jpg",
    "description": "example string",
    "cover": "https://example.com/file.jpg",
    "backUrl": "example string",
    "isActive": true,
    "theme": {
        "primaryColor": "example string",
        "navbarTextColor": "example string",
        "conversationBackgroundColor": "example string",
        "chatbotMessageTextColor": "example string",
        "userMessageTextColor": "example string",
        "chatbotMessageBackgroundColor": "example string",
        "userMessageBackgroundColor": "example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "darkTheme": {
        "primaryColor": "example string",
        "navbarTextColor": "example string",
        "conversationBackgroundColor": "example string",
        "chatbotMessageTextColor": "example string",
        "userMessageTextColor": "example string",
        "chatbotMessageBackgroundColor": "example string",
        "userMessageBackgroundColor": "example string",
        "chatbotMessageBackgroundShadowEnabled": true,
        "userMessageBackgroundShadowEnabled": true,
        "chatbotMessageGradientEnabled": true,
        "userMessageGradientEnabled": true
    },
    "enableFileUpload": true,
    "enableDisplayCitations": true,
    "toolDisplayMode": "full",
    "enableAskOptionButtons": true,
    "enableDisplayThinking": true,
    "enableUserThinkingControl": true,
    "enableDisplayQueryMetadata": true,
    "enableDisplayHookMarkers": true,
    "hookMarkerTextInputModify": "example string",
    "hookMarkerTextOutputModify": "example string",
    "hookMarkerTextInputBlock": "example string",
```

```

    "hookMarkerTextOutputBlock": "example string",
    "enableShareConversation": true,
    "enableLocation": true,
    "enableShowPoweredBy": true,
    "poweredByLogo": "https://example.com/file.jpg",
    "enableAnonymous": true,
    "enableDisplayPreviousConversation": true,
    "enableDisplayConversationHistory": true,
    "enableConversationTimer": true,
    "conversationTimerDuration": 123,
    "enableAutoNewConversationOnTimeout": true,
    "enableSatisfactionSurvey": true,
    "satisfactionSurveyDelayMinutes": 123,
    "satisfactionSurveyPrompt": "example string",
    "enableThemeModeToggle": true,
    "enableDisplayFontSizeSwitch": true,
    "defaultThemeMode": {},
    "enableDownloadCitations": true,
    "customDomain": {
        "domain": "example string"
    },
    "enableDisplayChatbotAvatar": true,
    "enableDisplayChatbotName": true,
    "buttonIcon": "https://example.com/file.jpg",
    "conversationStartersDisplayCount": 123,
    "preChatFormEnabled": true,
    "preChatFormFields": null,
    "preChatFormTitle": null,
    "preChatFormSubtitle": null,
    "loadingDisplayMode": {},
    "loadingStageCopyByLocale": null,
    "loadingLongWaitCopyByLocale": null,
    "loadingLongWaitThresholdSeconds": 123,
    "loadingAnimationUseSharedAsset": true,
    "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
    "loadingAnimationAssetUrlsByStage": null,
    "loadingTypingShowCopy": true,
    "toolCardRunningIconUrl": "https://example.com/file.jpg",
    "toolCardCompletedIconUrl": "https://example.com/file.jpg"
}

```

```

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)

```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/voice-agent/token/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "avatar": "https://example.com/file.jpg",
            "logo": "https://example.com/file.jpg",
            "description": "example string",
            "cover": "https://example.com/file.jpg",
            "backUrl": "example string",
            "isActive": true,
            "theme": {
                "primaryColor": "example string",
                "navbarTextColor": "example string",
                "conversationBackgroundColor": "example string",
                "chatbotMessageTextColor": "example string",
                "userMessageTextColor": "example string",
                "chatbotMessageBackgroundColor": "example string",
                "userMessageBackgroundColor": "example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "darkTheme": {
                "primaryColor": "example string",
                "navbarTextColor": "example string",
                "conversationBackgroundColor": "example string",
                "chatbotMessageTextColor": "example string",
                "userMessageTextColor": "example string",
                "chatbotMessageBackgroundColor": "example string",
                "userMessageBackgroundColor": "example string",
                "chatbotMessageBackgroundShadowEnabled": true,
                "userMessageBackgroundShadowEnabled": true,
                "chatbotMessageGradientEnabled": true,
                "userMessageGradientEnabled": true
            },
            "enableFileUpload": true,
            "enableDisplayCitations": true,
            "toolDisplayMode": "full",
            "enableAskOptionButtons": true,
            "enableDisplayThinking": true,
            "enableUserThinkingControl": true,
            "enableDisplayQueryMetadata": true,
            "enableDisplayHookMarkers": true,
            "hookMarkerTextInputModify": "example string",

```

```

        "hookMarkerTextOutputModify": "example string",
        "hookMarkerTextInputBlock": "example string",
        "hookMarkerTextOutputBlock": "example string",
        "enableShareConversation": true,
        "enableLocation": true,
        "enableShowPoweredBy": true,
        "poweredByLogo": "https://example.com/file.jpg",
        "enableAnonymous": true,
        "enableDisplayPreviousConversation": true,
        "enableDisplayConversationHistory": true,
        "enableConversationTimer": true,
        "conversationTimerDuration": 123,
        "enableAutoNewConversationOnTimeout": true,
        "enableSatisfactionSurvey": true,
        "satisfactionSurveyDelayMinutes": 123,
        "satisfactionSurveyPrompt": "example string",
        "enableThemeModeToggle": true,
        "enableDisplayFontSizeSwitch": true,
        "defaultThemeMode": {},
        "enableDownloadCitations": true,
        "customDomain": {
            "domain": "example string"
        },
        "enableDisplayChatbotAvatar": true,
        "enableDisplayChatbotName": true,
        "buttonIcon": "https://example.com/file.jpg",
        "conversationStartersDisplayCount": 123,
        "preChatFormEnabled": true,
        "preChatFormFields": null,
        "preChatFormTitle": null,
        "preChatFormSubtitle": null,
        "loadingDisplayMode": {},
        "loadingStageCopyByLocale": null,
        "loadingLongWaitCopyByLocale": null,
        "loadingLongWaitThresholdSeconds": 123,
        "loadingAnimationUseSharedAsset": true,
        "loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
        "loadingAnimationAssetUrlsByStage": null,
        "loadingTypingShowCopy": true,
        "toolCardRunningIconUrl": "https://example.com/file.jpg",
        "toolCardCompletedIconUrl": "https://example.com/file.jpg"
    }
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "logo"?: string (uri) // Optional
  "description"?: string // Optional
  "cover"?: string (uri) // Optional
  "backUrl"?: string // Optional
  "isActive"?: boolean // Optional
  "enableSpeech": boolean
  "displayConversationStarters": [
    string
  ]
  "theme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "darkTheme": {
    {
      "primaryColor"?: string // Optional
      "navbarTextColor"?: string // Optional
      "conversationBackgroundColor"?: string // Optional
      "chatbotMessageTextColor"?: string // Optional
      "userMessageTextColor"?: string // Optional
      "chatbotMessageBackgroundColor"?: string // Optional
      "userMessageBackgroundColor"?: string // Optional
      "chatbotMessageBackgroundShadowEnabled"?: boolean // Optional
      "userMessageBackgroundShadowEnabled"?: boolean // Optional
      "chatbotMessageGradientEnabled"?: boolean // Optional
      "userMessageGradientEnabled"?: boolean // Optional
    }
  }
  "enableFileUpload"?: boolean // Optional
  "enableDisplayCitations"?: boolean // Optional
  "toolDisplayMode"?: string (enum: full, compact, hidden) // * `full` - Full
  * `compact` - Compact
  * `hidden` - Hidden (Optional)
  "enableAskOptionButtons"?: boolean // When enabled, ask_user_question options appear as a row of
  buttons above the input box (which remains available). When disabled, the current overlay panel is
  retained. This is a gradual-rollout switch that can be disabled at any time to roll back. (optional)
```

```

"enableDisplayThinking"?: boolean // Optional
"enableUserThinkingControl"?: boolean // Optional
"enableDisplayQueryMetadata"?: boolean // Optional
"enableDisplayHookMarkers"?: boolean // Optional
"hookMarkerTextInputModify"?: string // Optional
"hookMarkerTextOutputModify"?: string // Optional
"hookMarkerTextInputBlock"?: string // Optional
"hookMarkerTextOutputBlock"?: string // Optional
"enableShareConversation"?: boolean // Optional
"enableLocation"?: boolean // Optional
"enableShowPoweredBy"?: boolean // Optional
"poweredByLogo"?: string (uri) // Optional
"enableAnonymous"?: boolean // Optional
"enableDisplayPreviousConversation"?: boolean // Optional
"enableDisplayConversationHistory"?: boolean // Optional
"enableConversationTimer"?: boolean // Optional
"conversationTimerDuration"?: integer // Timer timeout duration, range 3-60 minutes (Optional)
"enableAutoNewConversationOnTimeout"?: boolean // Optional
"enableSatisfactionSurvey"?: boolean // Optional
"satisfactionSurveyDelayMinutes"?: integer // Minutes after the last AI reply before the survey
pops up, range 1-60 minutes (Optional)
"satisfactionSurveyPrompt"?: string // Custom survey title text; leave empty to use the frontend
multilingual default text (Optional)
"accessType":
{
}
"enableThemeModeToggle"?: boolean // Optional
"enableDisplayFontSizeSwitch"?: boolean // Optional
"defaultThemeMode"?: // Optional
{
}
"enableDownloadCitations"?: boolean // Optional
"customDomain"?: // Optional
{
  "id": string (uuid)
  "domain": string
  "status":
  {
  }
}
"enableDisplayChatbotAvatar"?: boolean // Optional
"enableDisplayChatbotName"?: boolean // Optional
"buttonIcon"?: string (uri) // Optional
"enableCodeInterpreter": boolean // Read enable_code_interpreter from the inbox's chatbot.
"voiceAgentEnabled": boolean // Whether the bound chatbot has the voice agent turned on (read-only)
"conversationStartersDisplayCount"?: integer // Optional
"includeGreetingInContext": boolean // When enabled, the conversation greeting and the numbered
conversation starters (in configured order, unshuffled and untruncated) are injected into the LLM
context so the agent can resolve short replies like "1" to the matching starter option.
"tools": [ // Expose chatbot tools to SDK for the tool_mask selector.

Name resolution prefers ``display_name`` (set on MCP tools, where
``name`` is null) and falls back to ``name`` for API tools where
``display_name`` is null. Tools missing both are skipped – SDK
should never show an unnamed tool to a contact.
  object
]
"preChatFormEnabled"?: boolean // Optional
"preChatFormFields"?: object // Optional
"preChatFormTitle"?: object // Optional

```

```

"preChatFormSubtitle"?: object // Optional
"loadingDisplayMode"?: // Waiting-experience style. `narrative` shows per-stage copy;
`brand_animation` plays the uploaded asset and hides every system narration line. Defaults to
`narrative`, which with no copy configured behaves exactly as before this field existed.

* `narrative` - Narrative copy
* `brand_animation` - Brand animation (Optional)
{
}

"loadingStageCopyByLocale"?: object // Locale-keyed per-stage waiting copy, shaped {locale: {stage:
text}} where stage is one of LoadingStage. An absent locale or stage falls back to the built-in copy,
so a partially filled dict is valid and never blocks a reply. (Optional)

"loadingLongWaitCopyByLocale"?: object // Locale-keyed line shown once the wait exceeds
`loading_long_wait_threshold_seconds`, shaped {locale: text}. Written in the brand character voice
for the animation mode, but applies to both modes. Empty means no long-wait line is shown. (Optional)

"loadingLongWaitThresholdSeconds"?: integer // How long a single reply must be pending before the
long-wait copy appears. (Optional)

"loadingAnimationUseSharedAsset"?: boolean // True (default) means every stage plays
`loading_animation_shared_asset_url`, so the admin only uploads once. False expands per-stage assets,
each falling back to the shared asset when its own slot is empty. (Optional)

"loadingAnimationSharedAssetUrl"?: string (uri) // Optional

"loadingAnimationAssetUrlsByStage"?: object // Shaped {stage: url} keyed by LoadingStage. Only
consulted when `loading_animation_use_shared_asset` is False; a missing stage falls back to the
shared asset. (Optional)

"loadingTypingShowCopy"?: boolean // Only affects the first "typing" stage. True (default, the pre-
existing behavior) shows the typing-stage copy next to the pre-first-char animation/dot; False plays
only the animation with no text line. Thinking / analyzing / long-wait stages are unaffected.
(Optional)

"toolCardRunningIconUrl"?: string (uri) // Optional
"toolCardCompletedIconUrl"?: string (uri) // Optional
"inbox": object
}

```

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "avatar": "https://example.com/file.jpg",
  "logo": "https://example.com/file.jpg",
  "description": "response string",
  "cover": "https://example.com/file.jpg",
  "backUrl": "response string",
  "isActive": false,
  "enableSpeech": false,
  "displayConversationStarters": [
    "response string"
  ],
  "theme": {
    "primaryColor": "response string",
    "navbarTextColor": "response string",
    "conversationBackgroundColor": "response string",
    "chatbotMessageTextColor": "response string",
    "userMessageTextColor": "response string",
    "chatbotMessageBackgroundColor": "response string",
    "userMessageBackgroundColor": "response string",
    "chatbotMessageBackgroundShadowEnabled": false,
    "userMessageBackgroundShadowEnabled": false,
  }
}

```



```
"chatbotMessageGradientEnabled": false,
"userMessageGradientEnabled": false
},
"darkTheme": {
  "primaryColor": "response string",
  "navbarTextColor": "response string",
  "conversationBackgroundColor": "response string",
  "chatbotMessageTextColor": "response string",
  "userMessageTextColor": "response string",
  "chatbotMessageBackgroundColor": "response string",
  "userMessageBackgroundColor": "response string",
  "chatbotMessageBackgroundShadowEnabled": false,
  "userMessageBackgroundShadowEnabled": false,
  "chatbotMessageGradientEnabled": false,
  "userMessageGradientEnabled": false
},
"enableFileUpload": false,
"enableDisplayCitations": false,
"toolDisplayMode": "full",
"enableAskOptionButtons": false,
"enableDisplayThinking": false,
"enableUserThinkingControl": false,
"enableDisplayQueryMetadata": false,
"enableDisplayHookMarkers": false,
"hookMarkerTextInputModify": "response string",
"hookMarkerTextOutputModify": "response string",
"hookMarkerTextInputBlock": "response string",
"hookMarkerTextOutputBlock": "response string",
"enableShareConversation": false,
"enableLocation": false,
"enableShowPoweredBy": false,
"poweredByLogo": "https://example.com/file.jpg",
"enableAnonymous": false,
"enableDisplayPreviousConversation": false,
"enableDisplayConversationHistory": false,
"enableConversationTimer": false,
"conversationTimerDuration": 456,
"enableAutoNewConversationOnTimeout": false,
"enableSatisfactionSurvey": false,
"satisfactionSurveyDelayMinutes": 456,
"satisfactionSurveyPrompt": "response string",
"accessType": {},
"enableThemeModeToggle": false,
"enableDisplayFontSizeSwitch": false,
"defaultThemeMode": {},
"enableDownloadCitations": false,
"customDomain": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "domain": "response string",
  "status": {}
},
"enableDisplayChatbotAvatar": false,
"enableDisplayChatbotName": false,
"buttonIcon": "https://example.com/file.jpg",
"enableCodeInterpreter": false,
"voiceAgentEnabled": false,
"conversationStartersDisplayCount": 456,
"includeGreetingInContext": false,
"tools": [
  {
```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response example name",
    "description": "response example description"
  }
],
"preChatFormEnabled": false,
"preChatFormFields": null,
"preChatFormTitle": null,
"preChatFormSubtitle": null,
"loadingDisplayMode": {},
"loadingStageCopyByLocale": null,
"loadingLongWaitCopyByLocale": null,
"loadingLongWaitThresholdSeconds": 456,
"loadingAnimationUseSharedAsset": false,
"loadingAnimationSharedAssetUrl": "https://example.com/file.jpg",
"loadingAnimationAssetUrlsByStage": null,
"loadingTypingShowCopy": false,
"toolCardRunningIconUrl": "https://example.com/file.jpg",
"toolCardCompletedIconUrl": "https://example.com/file.jpg",
"inbox": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response example name",
  "description": "response example description"
}
}
```

Upload Batch QA File

POST

/api/v1/web-chats/{webchatPk}/batch-qas/

Parameters

Parameter	Required	Type	Description
webchatPk	V	string	

Request

Request Parameters

Field	Type	Required	Description
file	string (uri)	Yes	

Request Schema Example

```
{
  "file": string (uri)
}
```

Request Example Values

```
{
  "file": "https://example.com/file.jpg"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "file": "https://example.com/file.jpg"
}'

# Please make sure to replace YOUR_API_KEY and verify the request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "file": "https://example.com/file.jpg"
};

axios.post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "file": "https://example.com/file.jpg"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "file": "https://example.com/file.jpg"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "file": string (uri)
  "createdAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "file": "https://example.com/file.jpg",
```

```
"createdAt": "response string"
}
```

Export Batch QA as Excel

GET

/api/v1/web-chats/{webchatPk}/batch-qas/{id}/export-excel/

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Chatbot batch QA file.
webchatPk	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/550e8400-e29b-41d4-a716-446655440000/export-excel/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/550e8400-e29b-41d4-a716-446655440000/export-excel/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/550e8400-e29b-41d4-a716-446655440000/export-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/web-chats/550e8400-e29b-41d4-a716-446655440000/batch-qas/550e8400-e29b-41d4-a716-446655440000/export-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
200	Excel file download

Retrieve LLM Usage Statistics

GET /api/v1/chatbots/{chatbotPk}/llm-usage-statistics/

Parameters

Parameter	Required	Type	Description
chatbotPk	V	string	A UUID string identifying this Chatbot ID
endDate	X	string	End date, format: YYYY-MM-DD

startDate	V	string	Start date, format: YYYY-MM-DD
timeGranularity	V	string	Time granularity (day/month)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "date": string (date)
  }
]
```

Response Example Values

```
[
  [
    {
      "totalWordsCount": 511,
      "month": "2025-03-01",
      "llmDetails": [
        {
          "chatbotId": "5b646600-f6e0-4f07-aec7-0207a26c0a9b",
          "wordsCount": 511,
          "llmModelId": "d4282719-f50e-4a82-88d0-953e202a3383",
          "llmModelName": "GPT-4o-2024-08-06",
          "llmModelProvider": "openai"
        }
      ]
    }
  ]
]
```

Status Code: 400

Response Schema Example

```
{
  "detail"?: string // Optional
  "valid_types"?: [ // Optional
    string
  ]
}
```

Response Example Values

```
[
  [
    {
      "totalWordsCount": 511,
      "month": "2025-03-01",
      "llmDetails": [
        {
          "chatbotId": "5b646600-f6e0-4f07-aec7-0207a26c0a9b",
          "wordsCount": 511,
          "llmModelId": "d4282719-f50e-4a82-88d0-953e202a3383",
          "llmModelName": "GPT-4o-2024-08-06",

```

```
      "llmModelProvider": "openai"
    }
  ]
}
]
```

Retrieve AI Assistant Statistics

GET

`/api/v1/chatbots/{chatbotPk}/statistics/`

Parameters

Parameter	Required	Type	Description
<code>chatbotPk</code>	V	string	A UUID string identifying this Chatbot ID
<code>datetimeFrom</code>	X	string	Start time
<code>datetimeTo</code>	X	string	End time
<code>timeGranularity</code>	X	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "datetime": integer // Timestamp (milliseconds)
    "messagesCount": integer // Message count
    "conversationsCount": integer // Conversation count
    "likesCount": integer // Like count
    "dislikesCount": integer // Dislike count
  }
]
```

Response Example Values

```
[
  {
    "datetime": 456,
    "messagesCount": 456,
    "conversationsCount": 456,
    "likesCount": 456,
```

```
    "dislikesCount": 456  
  }  
]
```


API call example (Shell)

...

目錄

- 1. Send Message (Streaming)
- 2. Send Message (Create)
- 3. Send an Outgoing Message
- 4. Create a New Conversation
- 5. List Messages
- 6. Retrieve a Specific Message
- 7. List conversations
- 8. Retrieve a Specific Conversation
- 9. Retrieve a Specific Conversation
- 10. Rename Conversation
- 11. Share Conversation
- 12. Quick Share Conversation
- 13. List Shared Conversations
- 14. Get a Specific Shared Conversation
- 15. Update Shared Conversation
- 16. Delete Shared Conversation
- 17. Copy Shared Conversation
- 18. Get My Shared Conversations
- 19. Create Message Feedback
- 20. Update Message Feedback
- 21. Delete Message Feedback
- 22. List Conversation Records
- 23. Retrieve a Specific Conversation Record
- 24. Retrieve Specific Conversation Records
- 25. Create a Conversation Record Export Request
- 26. Get Conversation Record Export Status

API call example (Shell)

Send Message (Streaming)

POST `/api/v1/chatbots/{chatbotId}/completions/`

Parameters

Parameter Name	Required	Type	Description
<code>chatbotId</code>	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
conversation	string (uuid)	No	Unique conversation identifier. If empty, a new conversation is created (optional)
message	object (with 8 properties: content, contentPayload, queryMetadata...)	Yes	Message content to send
message.content	string	Yes	Text content of the message; it may be an empty string when attachments are provided
message.contentPayload	object	No	Additional message content payload in JSON format (optional)
message.queryMetadata	object	No	Query metadata in JSON format (optional)
message.metadata	object	No	Message environment metadata, such as time zone information, in JSON format (optional)
message.attachments	array[AttachmentInput]	No	List of message attachments (optional)
message.toolIds	array[string]	No	List of tool IDs enabled for this message (optional)
message.clientTools	array[any]	No	List of tool definitions provided by the client (such as a Chrome extension) for this message (optional). Each item must include name (str), description (str), and input_schema (dict). The backend does not execute these tools during the LLM call; instead, it returns the clientToolCall content and ends the current turn.
message.sender	string (uuid)	No	Sender Contact ID (optional)
isStreaming	boolean	No	Whether to respond in streaming mode. Defaults to false (optional)
waitForAttachments	boolean	No	Whether to wait for attachment processing to complete before entering the agent workflow. Defaults to true (optional)

Request Schema Example

```
{
  "conversation"?: string (uuid) // Unique conversation identifier. If empty, a new
  conversation is created (optional) (optional)
```

```

"message": // Message content to send
{
  "content": string // Text content of the message; it may be an empty string when
attachments are provided
  "contentPayload"?: object // Additional message content payload in JSON format (optional)
(optional)
  "queryMetadata"?: object // Query metadata in JSON format (optional) (optional)
  "metadata"?: object // Message environment metadata, such as time zone information, in JSON
format (optional) (optional)
  "attachments"?: [ // List of message attachments (optional) (optional)
    {
      "id": string (uuid) // Unique attachment identifier
      "type": // Attachment type. Available values: image, video (under development and not
yet supported), audio, sticker (under development and not yet supported), and other

* `image` - Image
* `video` - Video
* `audio` - Audio
* `sticker` - Sticker
* `other` - Other
      {
      }
      "filename": string // Attachment filename
      "file": string (uri) // URL of the attachment file
    }
  ]
  "toolIds"?: [ // List of tool IDs enabled for this message (optional) (optional)
    string (uuid)
  ]
  "clientTools"?: [ // List of tool definitions provided by the client (such as a Chrome
extension) for this message (optional). Each item must include name (str), description (str),
and input_schema (dict). The backend does not execute these tools during the LLM call; instead,
it returns the clientToolCall content and ends the current turn. (optional)
    object
  ]
  "sender"?: string (uuid) // Sender Contact ID (optional) (optional)
}
  "isStreaming"?: boolean // Whether to respond in streaming mode. Defaults to false (optional)
(optional)
  "waitForAttachments"?: boolean // Whether to wait for attachment processing to complete
before entering the agent workflow. Defaults to true (optional) (optional)
}

```

Request Example Value

```

{
  "message": {
    "content": "Hello, please introduce yourself"
  },
  "is_streaming": true
}

```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "message": {
    "content": "Hello, please introduce yourself"
  },
  "is_streaming": true
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "message": {
    "content": "Hello, please introduce yourself"
  },
  "is_streaming": true
};

axios.post(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "message": {
        "content": "Hello, please introduce yourself"
    },
    "is_streaming": true
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "message": {
                "content": "Hello, please introduce yourself"
            },
            "is_streaming": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response Body

Status Code	Description
200	Conversation response content; an event stream when streaming
400	Invalid request parameters. Possible causes include the Contact ID not belonging to the organization

Send Message (Create)

POST

</api/v1/messages/>

Request Body

Request Parameters

Field	Type	Required	Description
conversation	string (uuid)	Yes	
type	string	No	
content	string	No	
contentPayload	object	No	
attachments	array[AttachmentCreateInput]	No	
canvas	object	No	
canvas.name	string	Yes	
canvas.canvasType	object	Yes	
canvas.title	string	Yes	
canvas.content	string	Yes	
slideTemplate	object (with 3 properties: slug, displayName, thumbnailUrls)	No	
slideTemplate.slug	string	Yes	
slideTemplate.displayName	string	Yes	
slideTemplate.thumbnailUrls	array[string]	No	
queryMetadata	object	No	Query metadata that controls the scope of queryable knowledge; see each endpoint description for details
queryMetadata.knowledgeBases	array[object]	No	List of queryable knowledge bases; an empty array or null means none are queryable (no permission), while omission means unrestricted
queryMetadata.labelRelations	object	No	Label filtering criteria; an empty conditions array means no label filtering (not no permission). This must be provided together with knowledgeBases —when provided

Field	Type	Required	Description
			alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How label criteria are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label criteria; supports nested operator/conditions combinations
metadata	object	No	Message metadata, such as timezone and other environment information
broadcast	boolean	No	
skipCopilotTrigger	boolean	No	
suggestedForId	string (uuid)	No	
skillIds	array[string]	No	Per-message selected skill IDs from the WebChat client (camelCase: skillIds).
toolIds	array[string]	No	Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
templateId	string (uuid)	No	Document/sheet template ID for file-level compilation (camelCase: templateId).

Request Schema Example

```
{
  "conversation": string (uuid)
  "type"?: string // optional
  "content"?: string // optional
  "contentPayload"?: object // optional
  "attachments"?: [ // optional
    {
      "type"?: // optional
      {
      }
      "filename": string
      "file": string (uri)
      "conversation"?: string (uuid) // optional
    }
  ]
}
```

```

    }
  ]
  "canvas"?: { // optional
    {
      "name": string
      "canvasType":
        {
        }
      "title": string
      "content": string
    }
  }
  "slideTemplate"?: // optional
  {
    "slug": string
    "displayName": string
    "thumbnailUrls"?: [ // optional
      string
    ]
  }
  "queryMetadata"?: { // Query metadata that controls the scope of queryable knowledge; see
each endpoint description for details (optional)
    {
      "knowledgeBases"?: [ // List of queryable knowledge bases; an empty array or null means
none are queryable (no permission), while omission means unrestricted (optional)
        {
          "knowledgeBaseId": string (uuid) // Knowledge base ID
          "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; this
field takes precedence when both are provided) (optional)
            string (uuid)
          ]
          "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base and
exclude the listed items; false = select only the listed items (optional)
        }
      ]
      "labelRelations"?: { // Label filtering criteria; an empty conditions array means no label
filtering (not no permission). This must be provided together with knowledgeBases—when provided
alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
(optional)
        {
          "operator": string (enum: AND, OR) // How label criteria are combined
          "conditions"?: [ // List of label criteria; supports nested operator/conditions
combinations (optional)
            object
          ]
        }
      }
    }
  }
}

```

```

}
"metadata"?: object // Message metadata, such as timezone and other environment information
(optional)
"broadcast"?: boolean // optional
"skipCopilotTrigger"?: boolean // optional
"suggestedForId"?: string (uuid) // optional
"skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (optional)
    string (uuid)
]
"toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
(optional)
    string (uuid)
]
"templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (optional)
}

```

Request Example Value

```

{
  "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
  "sender": {
    "id": 2.992461161002561e+38,
    "name": "jessiekuo",
    "avatar": "https://media-dev.maiaagent.ai/static/images/default-user-avatar.png"
  },
  "type": "incoming",
  "content": "This is a test message",
  "contentPayload": {
    "content": "This is a test message",
    "items": [
      {
        "type": "text",
        "text": "This is a test message",
        "startTimestamp": null,
        "citations": []
      }
    ]
  },
  "metadata": null
},
"feedback": "like",
"createdAt": "1748314926000",
"attachments": [],
"citations": [],
"citationNodes": [],
"canvas": {
  "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
  "name": "system-architecture-diagram",
  "canvasType": "markdown",
  "title": "System Architecture Diagram",
  "content": "<!-- MERMAID_PLACEHOLDER_START -->
<div class='mermaid-diagram'>
  <div class='mermaid'>
    \ngraph TD\n      A[User] --> B[Frontend Interface]\n      B --> C[API Layer]\n      C -->

```

```
D[Database]\n
</div>
<p class="mermaid-caption">Mermaid 圖表</p>
</div>
<!-- MERMAID_PLACEHOLDER_END --> ",
  "createdAt": "1748314926000"
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/messages/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
  "sender": {
    "id": 2.992461161002561e+38,
    "name": "jessiekuo",
    "avatar": "https://media-dev.maiagent.ai/static/images/default-user-avatar.png"
  },
  "type": "incoming",
  "content": "This is a test message",
  "contentPayload": {
    "content": "This is a test message",
    "items": [
      {
        "type": "text",
        "text": "This is a test message",
        "startTimestamp": null,
        "citations": []
      }
    ]
  },
  "metadata": null
},
"feedback": "like",
"createdAt": "1748314926000",
"attachments": [],
"citations": [],
"citationNodes": [],
"canvas": {
  "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
  "name": "system-architecture-diagram",
  "canvasType": "markdown",
  "title": "System Architecture Diagram",
  "content": "
```

```
\ngraph TD\n    A[User] --> B[Frontend Interface]\n    B --> C[API Layer]\n    C --> D[Database]\n
```

Mermaid 圖表

```
",
  "createdAt": "1748314926000"
```

```
}  
}'
```

```
# Before running, replace YOUR_API_KEY and verify the request data.
```

```

const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
  "sender": {
    "id": 2.992461161002561e+38,
    "name": "jessiekuo",
    "avatar": "https://media-dev.maiagent.ai/static/images/default-user-avatar.png"
  },
  "type": "incoming",
  "content": "This is a test message",
  "contentPayload": {
    "content": "This is a test message",
    "items": [
      {
        "type": "text",
        "text": "This is a test message",
        "startTimestamp": null,
        "citations": []
      }
    ],
    "metadata": null
  },
  "feedback": "like",
  "createdAt": "1748314926000",
  "attachments": [],
  "citations": [],
  "citationNodes": [],
  "canvas": {
    "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
    "name": "system-architecture-diagram",
    "canvasType": "markdown",
    "title": "System Architecture Diagram",
    "content": "

```

```

\ng
graph TD
  A[User] --> B[Frontend Interface]
  B --> C[API Layer]
  C --> D[Database]

```



```
    ",  
    "createdAt": "1748314926000"  
  }  
};  
  
axios.post("https://api.maiagent.ai/api/v1/messages/", data, config)  
  .then(response => {  
    console.log('Response received successfully:');  
    console.log(response.data);  
  })  
  .catch(error => {  
    console.error('An error occurred while making the request:');  
    console.error(error.response?.data || error.message);  
  });
```

```

import requests

url = "https://api.maiagent.ai/api/v1/messages/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
    "sender": {
        "id": 2.992461161002561e+38,
        "name": "jessiekuo",
        "avatar": "https://media-dev.maiagent.ai/static/images/default-user-avatar.png"
    },
    "type": "incoming",
    "content": "This is a test message",
    "contentPayload": {
        "content": "This is a test message",
        "items": [
            {
                "type": "text",
                "text": "This is a test message",
                "startTimestamp": null,
                "citations": []
            }
        ],
        "metadata": null
    },
    "feedback": "like",
    "createdAt": "1748314926000",
    "attachments": [],
    "citations": [],
    "citationNodes": [],
    "canvas": {
        "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
        "name": "system-architecture-diagram",
        "canvasType": "markdown",
        "title": "System Architecture Diagram",
        "content": "

```

```

\ngraph TD\n    A[User] --> B[Frontend Interface]\n    B --> C[API Layer]\n    C --> D[Database]\n
```

Mermaid 圖表

```
"",  
    "createdAt": "1748314926000"  
  }  
}
```

```
response = requests.post(url, json=data, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("An error occurred while making the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/messages/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
            "sender": {
                "id": 2.992461161002561e+38,
                "name": "jessiekuo",
                "avatar": "https://media-dev.maiagent.ai/static/images/default-user-
avatar.png"
            },
            "type": "incoming",
            "content": "This is a test message",
            "contentPayload": {
                "content": "This is a test message",
                "items": [
                    {
                        "type": "text",
                        "text": "This is a test message",
                        "startTimestamp": null,
                        "citations": []
                    }
                ]
            },
            "metadata": null
        },
        "feedback": "like",
        "createdAt": "1748314926000",
        "attachments": [],
        "citations": [],
        "citationNodes": [],
        "canvas": {
            "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
            "name": "system-architecture-diagram",
            "canvasType": "markdown",
            "title": "System Architecture Diagram",
            "content": "

```

```

\ngraph TD\n    A[User] --> B[Frontend Interface]\n    B --> C[API Layer]\n    C -->
D[Database]\n

```

Mermaid 圖表

```
",
        "createdAt": "1748314926000"
    }
}
]);

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "conversation": string (uuid)
  "sender":
  {
    "id": string (uuid)
    "name": string
    "avatar": string
    "email"?: string (email) // optional
    "phoneNumber"?: string // optional
  }
  "type"?: string // optional
  "content"?: string // optional
  "contentPayload"?: object // optional
  "feedback":
  {
    "id": string (uuid)
    "type":
    {
    }
    "suggestion"?: string // optional
    "updatedAt": string (timestamp)
  }
  "createdAt": string (timestamp)
```

```
"attachments"?: [ // optional
```

```
{
  "id": string (uuid)
  "type"?: // optional
  {
  }
  "filename": string
  "file": string (uri)
  "expiresAt": string // ISO datetime when this attachment will be auto-deleted.
```

For conversation-bound attachments: conversation.last_message_created_at + retention_days.

For attachments without a conversation: created_at + retention_days.

Returns None when cleanup is disabled (effective_from not set).

```
  "conversation"?: string (uuid) // optional
}
]
```

```
"citations": [
```

```
{
  "id": string (uuid)
  "filename": string // Filename
  "file": string (uri) // File to upload
  "fileType": string
  "knowledgeBase"?: // optional
```

```
{
  "id": string (uuid)
  "name": string
}
```

```
"size": integer
"status":
```

```
{
}
```

```
"parser":
```

```
{
  "id": string (uuid)
  "name": string
  "provider":
```

```
{
}
```

```
"isTimestampSttProvider": boolean
```

```
}
```

```
"labels"?: [ // optional
```

```
{
  "id": string (uuid)
  "name": string
}
```

```
]
```

```
"rawUserDefineMetadata"?: object // optional
```

```
"speakerLabels": [
```

```
{
  "id": string (uuid)
  "originalLabel": string // Original speaker label from diarization (e.g., SPEAKER_00)
  "customName"?: string // User-defined speaker name (e.g., John) (optional)
  "displayName": string
}
```

```
]
```

```
"vectorStorageSize": integer // Size of vectors for this file in Elasticsearch (bytes)
```

```
"chunksCount": integer // Number of chunks/nodes generated from this file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"previewUrl": string // For presentations (pptx/ppt), returns the URL of a derived,
previewable PDF file (displayed in the frontend PDF viewer); None otherwise.
```

Generate the URL using the same CustomizedFileFieldSerializer as the `file` field to ensure a consistent format (including each storage provider's presign/host rules). Do not use `get_absolute_url()`, as it produces an incorrect URL format.

```
"createdAt": string (timestamp)
}
]
"citationNodes": [
{
  "chatbotTextNode": {
    {
      "id": string (uuid)
      "charactersCount": integer
      "hitsCount": integer
      "text": string
      "updatedAt": string (timestamp)
      "filename": string
      "chatbotFile": object // Returns complete ChatbotFile information, including the file
URL and type, to support frontend image previews
      "knowledgeBaseFile": object // Same as get_chatbot_file, provided for backward
compatibility
      "pageNumber": integer // Backward-compatible alias of ``page_start``.
      "pageStart": integer
      "pageEnd": integer
      "citationTitle": string // Source title displayed in the inline citation hover card
      "citationDescription": string // Source description displayed in the inline citation
hover card
      "citationQuote": string // Quoted text excerpt displayed in the inline citation hover
card
      "hasImage": boolean // Indicates whether an image is included (checks the file type or
Markdown images in text)
      "imageUrl": string // Extracts the image URL (uses the file URL first; otherwise
extracts it from Markdown)
      "displayText": string // Returns the full text with Markdown image markup removed
      "displayTitle": string // Returns the display title (fallback: citation_title ->
filename)
      "highlightedText": string // Return text with matched terms wrapped in ``<mark>`` tags.
```

Priority:

1. ES/OpenSearch native highlight (set by `retrieve_api` via ``\_es\_highlighted\_text``)
2. Python regex fallback (keyword-based, works for all backends)

```
"labels": [
{
  "id": string (uuid)
  "name": string
}
]
"rawUserDefineMetadata?": object // Users can define their own metadata keys and values
(optional)
```

```
    "metadataEnabledForSearch": object // Map each user-defined metadata key on this node to whether it was sent into the LLM.
```

A key reaches the LLM only when its ``MetadataKey.enabled\_for\_search`` is True *and* the node carries a value for it. Since this maps over ``raw\_user\_define\_metadata`` (the keys the cited-documents block displays, all of which have a value), ``enabled\_for\_search`` alone decides the flag. The source of truth is the node's knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.

```
    "startCharIdx": integer
    "endCharIdx": integer
  }
}
"score"?: number (double) // optional
"displayScore": integer
"citationNumber"?: integer // Citation number matching the [1], [2] markers in the reply content (optional)
"highlightedText"?: string // ES/OpenSearch highlight result with <mark> tags (optional)
}
]
"canvas"?: { // optional
{
  "id": string (uuid)
  "name": string
  "canvasType":
  {
  }
  "title": string
  "content": string
  "createdAt": string (timestamp)
}
}
"slideTemplate"?: // optional
{
  "slug": string
  "displayName": string
  "thumbnailUrls"?: [ // optional
    string
  ]
}
}
"queryMetadata"?: { // Query metadata that controls the searchable knowledge scope; see each endpoint description for details (optional)
{
  "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means none are searchable (no permission), while omission means unrestricted (optional)
  {
    "knowledgeBaseId": string (uuid) // Knowledge base ID
    "chatbotFileIds"?: [ // List of file IDs (excluded or selected exclusively depending on hasUserSelectedAll) (optional)
      string (uuid)
    ]
    "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; this field takes precedence when both are provided) (optional)
      string (uuid)
    ]
    "faqIds"?: [ // List of FAQ IDs (excluded or selected exclusively depending on hasUserSelectedAll) (optional)
```



```

        string (uuid)
    ]
    "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base and
exclude listed items; false = select only listed items (optional)
}
]
"labelRelations"?: { // Label filter criteria; an empty conditions array means no label
filtering (not no permission). Must be provided together with knowledgeBases—when provided
alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
(optional)
{
    "operator": string (enum: AND, OR) // How label conditions are combined
    "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (optional)
        object
    ]
}
}
}
}
"metadata"?: object // Message metadata, such as timezone and other environment information
(optional)
"recordId": string // Returns the ChatbotRecord ID corresponding to the Message

```

Note:

```

    For unsaved Messages (such as virtual Messages during streaming), return None directly
    to avoid triggering a DB query on the OneToOneField reverse relation (N+1 issue).
    "broadcast"?: boolean // optional
    "skipCopilotTrigger"?: boolean // optional
    "activeRevisionNumber": integer
    "revisions": [
        {
            "id": string (uuid)
            "revisionNumber": integer
            "content": string
            "contentPayload": object
            "isActive": boolean
            "createdAt": string (timestamp)
        }
    ]
    "suggestedForId"?: string (uuid) // optional
    "suggestionStatus": string
    "suggestionTriggerSource": string
    "suggestionError": string
    "retryCount": integer // How many times the LLM call was auto-retried during this message
generation. Currently only Bedrock first-chunk stall triggers retry (max 1). Detailed per-
attempt records live in metadata["retry_attempts"].
    "skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (optional)
        string (uuid)
    ]
    "toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
(optional)
        string (uuid)
    ]
    "templateId"?: string (uuid) // Document/sheet template ID for file-level compilation

```

```
(camelCase: templateId). (optional)
  "videoGeneration": object // Return the latest GeneratedVideo state for FE reload recovery;
  ``errorMessage`` is blank to match the socket emit (raw value stays on the DB row).
}
```

Example Response Value

```
{
  "conversation": "c96a0fb9-d106-4b16-8706-3906533bafa2",
  "sender": {
    "id": 2.992461161002561e+38,
    "name": "jessiekuo",
    "avatar": "https://media-dev.maiagent.ai/static/images/default-user-avatar.png"
  },
  "type": "incoming",
  "content": "This is a test message",
  "contentPayload": {
    "content": "This is a test message",
    "items": [
      {
        "type": "text",
        "text": "This is a test message",
        "startTimestamp": null,
        "citations": []
      }
    ],
    "metadata": null
  },
  "feedback": "like",
  "createdAt": "1748314926000",
  "attachments": [],
  "citations": [],
  "citationNodes": [],
  "canvas": {
    "id": "034eecd7-e0f4-46e9-88cb-4fefdb5b7ddd",
    "name": "system-architecture-diagram",
    "canvasType": "markdown",
    "title": "System Architecture Diagram",
    "content": "<!-- MERMAID_PLACEHOLDER_START -->
<div class='mermaid-diagram'>
  <div class='mermaid'>
    \ngraph TD\n  A[User] --> B[Frontend Interface]\n  B --> C[API Layer]\n  C -->
    D[Database]\n
  </div>
  <p class='mermaid-caption'>Mermaid 圖表</p>
</div>
<!-- MERMAID_PLACEHOLDER_END -->",
    "createdAt": "1748314926000"
  }
}
```

Send an Outgoing Message

POST

/api/v1/messages/outgoing/

Request Body

Request Parameters

Field	Type	Required	Description
conversation	string (uuid)	Yes	
type	string	No	
content	string	No	
contentPayload	object	No	
attachments	array[AttachmentCreateInput]	No	
canvas	object	No	
canvas.name	string	Yes	
canvas.canvasType	object	Yes	
canvas.title	string	Yes	
canvas.content	string	Yes	
slideTemplate	object (with 3 properties: slug, displayName, thumbnailUrls)	No	
slideTemplate.slug	string	Yes	
slideTemplate.displayName	string	Yes	
slideTemplate.thumbnailUrls	array[string]	No	
queryMetadata	object	No	Query metadata that controls the searchable knowledge scope; see each endpoint description for details

Field	Type	Required	Description
queryMetadata.knowledgeBases	array[object]	No	List of searchable knowledge bases; an empty array or null means none are searchable (no permission), while omission means unrestricted
queryMetadata.labelRelations	object	No	Label filter criteria; an empty conditions array means no label filtering (not no permission). Must be provided together with knowledgeBases—when provided alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions; supports nested operator/conditions combinations
metadata	object	No	Message metadata, such as timezone and other environment information
broadcast	boolean	No	
skipCopilotTrigger	boolean	No	
suggestedForId	string (uuid)	No	
skillIds	array[string]	No	Per-message selected skill IDs from the WebChat client (camelCase: skillIds).
toolIds	array[string]	No	Per-message selected tool IDs from the WebChat client (camelCase: toolIds).

Field	Type	Required	Description
templateId	string (uuid)	No	Document/sheet template ID for file-level compilation (camelCase: templateId).

Example Request Schema

```
{
  "conversation": string (uuid)
  "type"?: string // optional
  "content"?: string // optional
  "contentPayload"?: object // optional
  "attachments"?: [ // optional
    {
      "type"?: // optional
      {
      }
      "filename": string
      "file": string (uri)
      "conversation"?: string (uuid) // optional
    }
  ]
  "canvas"?: { // optional
    {
      "name": string
      "canvasType":
      {
      }
      "title": string
      "content": string
    }
  }
  "slideTemplate"?: // optional
  {
    "slug": string
    "displayName": string
    "thumbnailUrls"?: [ // optional
      string
    ]
  }
  "queryMetadata"?: { // Query metadata that controls the searchable knowledge scope; see each endpoint description for details (optional)
    {
      "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means none are searchable (no permission), while omission means unrestricted (optional)
        {
          "knowledgeBaseId": string (uuid) // Knowledge base ID
          "chatbotFileIds"?: [ // List of file IDs (excluded or selected exclusively depending on hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; this
```

```

field takes precedence when both are provided) (optional)
    string (uuid)
  ]
  "faqIds"?: [ // List of FAQ IDs (excluded or selected exclusively depending on
hasUserSelectedAll) (optional)
    string (uuid)
  ]
  "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base and
exclude listed items; false = select only listed items (optional)
}
]
"labelRelations"?: { // Label filter criteria; an empty conditions array means no label
filtering (not no permission). Must be provided together with knowledgeBases—when provided
alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
(optional)
  {
    "operator": string (enum: AND, OR) // How label conditions are combined
    "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (optional)
      object
    ]
  }
}
}
}
"metadata"?: object // Message metadata, such as timezone and other environment information
(optional)
"broadcast"?: boolean // optional
"skipCopilotTrigger"?: boolean // optional
"suggestedForId"?: string (uuid) // optional
"skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (optional)
  string (uuid)
]
"toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
(optional)
  string (uuid)
]
"templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (optional)
}

```

Example Request Value

```

{
  "conversation": "550e8400-e29b-41d4-a716-446655440000",
  "type": "example string",
  "content": "Hello! I would like to learn more about the product.",
  "contentPayload": null,
  "attachments": [
    {
      "type": {},
      "filename": "document.pdf",
      "file": "https://example.com/file.jpg",
      "conversation": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}

```

```
}
],
"canvas": {
  "name": "Example Name",
  "canvasType": {},
  "title": "Example Name",
  "content": "Hello! I would like to learn more about the product."
},
"slideTemplate": {
  "slug": "example string",
  "displayName": "example string",
  "thumbnailUrls": [
    "example string"
  ]
},
"queryMetadata": {
  "knowledgeBases": [
    {
      "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
      "chatbotFileIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "knowledgeBaseFileIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "faqIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "hasUserSelectedAll": true
    }
  ],
  "labelRelations": {
    "operator": "AND",
    "conditions": [
      null
    ]
  }
},
"metadata": null,
"broadcast": true,
"skipCopilotTrigger": true,
"suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
"skillIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"toolIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"templateId": "550e8400-e29b-41d4-a716-446655440000"
}
```

Code Examples


```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/messages/outgoing/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "conversation": "550e8400-e29b-41d4-a716-446655440000",
  "type": "example string",
  "content": "Hello! I would like to learn more about the product.",
  "contentPayload": null,
  "attachments": [
    {
      "type": {},
      "filename": "document.pdf",
      "file": "https://example.com/file.jpg",
      "conversation": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "canvas": {
    "name": "Example Name",
    "canvasType": {},
    "title": "Example Name",
    "content": "Hello! I would like to learn more about the product."
  },
  "slideTemplate": {
    "slug": "example string",
    "displayName": "example string",
    "thumbnailUrls": [
      "example string"
    ]
  },
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
        "chatbotFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "knowledgeBaseFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "faqIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "hasUserSelectedAll": true
      }
    ],
    "labelRelations": {
      "operator": "AND",
```

```
        "conditions": [  
            null  
        ]  
    }  
},  
"metadata": null,  
"broadcast": true,  
"skipCopilotTrigger": true,  
"suggestedForId": "550e8400-e29b-41d4-a716-446655440000",  
"skillIds": [  
    "550e8400-e29b-41d4-a716-446655440000"  
],  
"toolIds": [  
    "550e8400-e29b-41d4-a716-446655440000"  
],  
"templateId": "550e8400-e29b-41d4-a716-446655440000"  
}'
```

Before running, replace YOUR_API_KEY and verify the request data.

```

const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "conversation": "550e8400-e29b-41d4-a716-446655440000",
  "type": "example string",
  "content": "Hello! I would like to learn more about the product.",
  "contentPayload": null,
  "attachments": [
    {
      "type": {},
      "filename": "document.pdf",
      "file": "https://example.com/file.jpg",
      "conversation": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "canvas": {
    "name": "Example Name",
    "canvasType": {},
    "title": "Example Name",
    "content": "Hello! I would like to learn more about the product."
  },
  "slideTemplate": {
    "slug": "example string",
    "displayName": "example string",
    "thumbnailUrls": [
      "example string"
    ]
  },
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
        "chatbotFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "knowledgeBaseFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "faqIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "hasUserSelectedAll": true
      }
    ]
  }
};

```

```

    }
  ],
  "labelRelations": {
    "operator": "AND",
    "conditions": [
      null
    ]
  }
},
"metadata": null,
"broadcast": true,
"skipCopilotTrigger": true,
"suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
"skillIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"toolIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"templateId": "550e8400-e29b-41d4-a716-446655440000"
};

axios.post("https://api.maiagent.ai/api/v1/messages/outgoing/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });

```

```

import requests

url = "https://api.maiagent.ai/api/v1/messages/outgoing/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "conversation": "550e8400-e29b-41d4-a716-446655440000",
    "type": "example string",
    "content": "Hello! I would like to learn more about the product.",
    "contentPayload": null,
    "attachments": [
        {
            "type": {},
            "filename": "document.pdf",
            "file": "https://example.com/file.jpg",
            "conversation": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "canvas": {
        "name": "Example Name",
        "canvasType": {},
        "title": "Example Name",
        "content": "Hello! I would like to learn more about the product."
    },
    "slideTemplate": {
        "slug": "example string",
        "displayName": "example string",
        "thumbnailUrls": [
            "example string"
        ]
    },
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                "chatbotFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "knowledgeBaseFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "faqIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "hasUserSelectedAll": true
            }
        ]
    },
    ],

```

```
        "labelRelations": {
            "operator": "AND",
            "conditions": [
                null
            ]
        },
    },
    "metadata": null,
    "broadcast": true,
    "skipCopilotTrigger": true,
    "suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
    "skillIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "toolIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "templateId": "550e8400-e29b-41d4-a716-446655440000"
}
```

```
response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/messages/outgoing/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "conversation": "550e8400-e29b-41d4-a716-446655440000",
            "type": "example string",
            "content": "Hello! I would like to learn more about the product.",
            "contentPayload": null,
            "attachments": [
                {
                    "type": {},
                    "filename": "document.pdf",
                    "file": "https://example.com/file.jpg",
                    "conversation": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "canvas": {
                "name": "Example Name",
                "canvasType": {},
                "title": "Example Name",
                "content": "Hello! I would like to learn more about the product."
            },
            "slideTemplate": {
                "slug": "example string",
                "displayName": "example string",
                "thumbnailUrls": [
                    "example string"
                ]
            },
            "queryMetadata": {
                "knowledgeBases": [
                    {
                        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                        "chatbotFileIds": [
                            "550e8400-e29b-41d4-a716-446655440000"
                        ],
                        "knowledgeBaseFileIds": [
                            "550e8400-e29b-41d4-a716-446655440000"
                        ],
                        "faqIds": [
                            "550e8400-e29b-41d4-a716-446655440000"
                        ],
                        "hasUserSelectedAll": true
                    }
                ]
            }
        }
    ]);
} catch (\Exception $e) {
    // Handle exception
}

```

```

        }
    ],
    "labelRelations": {
        "operator": "AND",
        "conditions": [
            null
        ]
    }
},
"metadata": null,
"broadcast": true,
"skipCopilotTrigger": true,
"suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
"skillIds": [
    "550e8400-e29b-41d4-a716-446655440000"
],
"toolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
],
"templateId": "550e8400-e29b-41d4-a716-446655440000"
}
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response Body

Status Code: 201

Example Response Schema

```

{
  "id": string (uuid)
  "conversation": string (uuid)
  "sender":
  {
    "id": string (uuid)
    "name": string
    "avatar": string
    "email?": string (email) // Optional
    "phoneNumber?": string // Optional
  }
}

```



```

"type"?: string // Optional
"content"?: string // Optional
"contentPayload"?: object // Optional
"feedback":
{
  "id": string (uuid)
  "type":
  {
  }
  "suggestion"?: string // Optional
  "updatedAt": string (timestamp)
}
"createdAt": string (timestamp)
"attachments"?: [ // Optional
{
  "id": string (uuid)
  "type"?: // Optional
  {
  }
  "filename": string
  "file": string (uri)
  "expiresAt": string // ISO datetime when this attachment will be auto-deleted.

```

For conversation-bound attachments: conversation.last_message_created_at + retention_days.

For attachments without a conversation: created_at + retention_days.

Returns None when cleanup is disabled (effective_from not set).

```

"conversation"?: string (uuid) // Optional
}
]
"quotations": [
{
  "id": string (uuid)
  "filename": string // File name
  "file": string (uri) // File to upload
  "fileType": string
  "knowledgeBase"?: // Optional
  {
    "id": string (uuid)
    "name": string
  }
  "size": integer
  "status":
  {
  }
  "parser":
  {
    "id": string (uuid)
    "name": string
    "provider":
    {
    }
    "isTimestampSttProvider": boolean
  }
  "labels"?: [ // Optional
  {
    "id": string (uuid)

```

```

        "name": string
    }
]
"rawUserDefineMetadata"?: object // Optional
"speakerLabels": [
    {
        "id": string (uuid)
        "originalLabel": string // Original speaker label from diarization (e.g., SPEAKER_00)
        "customName"?: string // User-defined speaker name (e.g., John) (Optional)
        "displayName": string
    }
]
"vectorStorageSize": integer // Size of vectors for this file in Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from this file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"previewUrl": string // For presentations (pptx/ppt), returns the URL of a derived PDF
file that can be previewed (displayed in the frontend PDF viewer); returns None for other file
types.

```

Generate the URL with the same CustomizedFileFieldSerializer used by the `file` field to ensure a consistent format (including each storage provider's presign/host rules). Do not use `get_absolute_url()`—it generates an incorrect URL format.

```

        "createdAt": string (timestamp)
    }
]
"citationNodes": [
    {
        "chatbotTextNode": {
            {
                "id": string (uuid)
                "charactersCount": integer
                "hitsCount": integer
                "text": string
                "updatedAt": string (timestamp)
                "filename": string
                "chatbotFile": object // Returns complete ChatbotFile information, including the file
URL and type, to support image previews in the frontend
                "knowledgeBaseFile": object // Same as get_chatbot_file, provided for backward
compatibility
                "pageNumber": integer // Backward-compatible alias of ``page_start``.
                "pageStart": integer
                "pageEnd": integer
                "citationTitle": string // Source title displayed in the inline citation hover card
                "citationDescription": string // Source description displayed in the inline citation
hover card
                "citationQuote": string // Quoted text snippet displayed in the inline citation hover
card
                "hasImage": boolean // Determines whether an image is included (checks the file type or
Markdown images in text)
                "imageUrl": string // Extracts the image URL (uses the file URL first; otherwise
extracts it from Markdown)
                "displayText": string // Returns the complete text with image Markdown removed
                "displayTitle": string // Returns the display title (fallback: citation_title ->

```

```

filename)
    "highlightedText": string // Return text with matched terms wrapped in ``<mark>`` tags.

Priority:
1. ES/OpenSearch native highlight (set by retrieve_api via ``_es_highlighted_text``)
2. Python regex fallback (keyword-based, works for all backends)
    "labels": [
        {
            "id": string (uuid)
            "name": string
        }
    ]
    "rawUserDefineMetadata"?: object // Users can define their own metadata keys and values
(Optional)
    "metadataEnabledForSearch": object // Map each user-defined metadata key on this node
to whether it was sent into the LLM.

A key reaches the LLM only when its ``MetadataKey.enabled_for_search`` is True *and*
the node carries a value for it. Since this maps over ``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the node's
knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.
    "startCharIdx": integer
    "endCharIdx": integer
}
}
"score"?: number (double) // Optional
"displayScore": integer
"citationNumber"?: integer // Citation number matching the [1], [2] markers in the reply
content (Optional)
    "highlightedText"?: string // ES/OpenSearch highlight result with <mark> tags (Optional)
}
]
"canvas"?: { // Optional
{
    "id": string (uuid)
    "name": string
    "canvasType":
    {
    }
    "title": string
    "content": string
    "createdAt": string (timestamp)
}
}
"slideTemplate"?: // Optional
{
    "slug": string
    "displayName": string
    "thumbnailUrls"?: [ // Optional
        string
    ]
}
}
"queryMetadata"?: { // Query metadata that controls the scope of queryable knowledge; see
each endpoint description for details (Optional)
{

```

```

    "knowledgeBases"?: [ // List of queryable knowledge bases; an empty array or null means
none are queryable (no permission), while omitted means unrestricted (Optional)
    {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
            string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; this
field takes precedence when both are provided) (Optional)
            string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
            string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base
except items in the list; false = select only items in the list (Optional)
    }
]
    "labelRelations"?: { // Label filtering criteria; an empty conditions array means no label
filtering (not no permission). Must be provided together with knowledgeBases—when provided
alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
(Optional)
    {
        "operator": string (enum: AND, OR) // How label conditions are combined
        "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (Optional)
            object
        ]
    }
}
    "metadata"?: object // Message metadata, such as timezone and other environment information
(Optional)
    "recordId": string // Returns the ChatbotRecord ID corresponding to the Message

```

Note:

For an unsaved Message (such as a virtual Message during streaming), return None directly to avoid triggering a DB query through the OneToOneField reverse relation (N+1 issue).

```

"broadcast"?: boolean // Optional
"skipCopilotTrigger"?: boolean // Optional
"activeRevisionNumber": integer
"revisions": [
    {
        "id": string (uuid)
        "revisionNumber": integer
        "content": string
        "contentPayload": object
        "isActive": boolean
        "createdAt": string (timestamp)
    }
]
"suggestedForId"?: string (uuid) // Optional
"suggestionStatus": string

```

```

"suggestionTriggerSource": string
"suggestionError": string
"retryCount": integer // How many times the LLM call was auto-retried during this message
generation. Currently only Bedrock first-chunk stall triggers retry (max 1). Detailed per-
attempt records live in metadata["retry_attempts"].
"skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (Optional)
  string (uuid)
]
"toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
(Optional)
  string (uuid)
]
"templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (Optional)
"videoGeneration": object // Return the latest GeneratedVideo state for FE reload recovery;
`errorMessage` is blank to match the socket emit (raw value stays on the DB row).
}

```

Example Response Value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "conversation": "550e8400-e29b-41d4-a716-446655440000",
  "sender": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "avatar": "Response string",
    "email": "response@example.com",
    "phoneNumber": "Response string"
  },
  "type": "Response string",
  "content": "Response string",
  "contentPayload": null,
  "feedback": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": {},
    "suggestion": "Response string",
    "updatedAt": "Response string"
  },
  "createdAt": "Response string",
  "attachments": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "type": {},
      "filename": "Response string",
      "file": "https://example.com/file.jpg",
      "expiresAt": "Response string",
      "conversation": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "citations": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "Response string",

```

```
"file": "https://example.com/file.jpg",
"fileType": "Response string",
"knowledgeBase": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
},
"size": 456,
"status": {},
"parser": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "provider": {},
  "isTimestampSttProvider": false
},
"labels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"rawUserDefineMetadata": null,
"speakerLabels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "originalLabel": "Response string",
    "customName": "Response string",
    "displayName": "Response string"
  }
],
"vectorStorageSize": 456,
"chunksCount": 456,
"waitingTime": 456,
"processingTime": 456,
"processingTimeDetails": null,
"previewUrl": "Response string",
"createdAt": "Response string"
}
],
"citationNodes": [
  {
    "chatbotTextNode": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "charactersCount": 456,
      "hitsCount": 456,
      "text": "Response string",
      "updatedAt": "Response string",
      "filename": "Response string",
      "chatbotFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response example name",
        "description": "Response example description"
      },
      "knowledgeBaseFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response example name",
        "description": "Response example description"
      }
    }
  }
]
```

```

    },
    "pageNumber": 456,
    "pageStart": 456,
    "pageEnd": 456,
    "citationTitle": "Response string",
    "citationDescription": "Response string",
    "citationQuote": "Response string",
    "hasImage": false,
    "imageUrl": "Response string",
    "displayText": "Response string",
    "displayTitle": "Response string",
    "highlightedText": "Response string",
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      }
    ],
    "rawUserDefineMetadata": null,
    "metadataEnabledForSearch": {
      "key1": "value1",
      "key2": "value2",
      "createdAt": "2025-01-01T00:00:00.000Z"
    },
    "startCharIdx": 456,
    "endCharIdx": 456
  },
  "score": 456,
  "displayScore": 456,
  "citationNumber": 456,
  "highlightedText": "Response string"
}
],
"canvas": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "canvasType": {},
  "title": "Response string",
  "content": "Response string",
  "createdAt": "Response string"
},
"slideTemplate": {
  "slug": "Response string",
  "displayName": "Response string",
  "thumbnailUrls": [
    "Response string"
  ]
},
"queryMetadata": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
},
"metadata": null,
"recordId": "Response string",
"broadcast": false,

```

```
"skipCopilotTrigger": false,
"activeRevisionNumber": 456,
"revisions": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "revisionNumber": 456,
    "content": "Response string",
    "contentPayload": null,
    "isActive": false,
    "createdAt": "Response string"
  }
],
"suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
"suggestionStatus": "Response string",
"suggestionTriggerSource": "Response string",
"suggestionError": "Response string",
"retryCount": 456,
"skillIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"toolIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"templateId": "550e8400-e29b-41d4-a716-446655440000",
"videoGeneration": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
}
}
```

Create a New Conversation

POST </api/v1/conversations/>

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/conversations/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{}'
```

Before running, replace YOUR_API_KEY and verify the request data.

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {};

axios.post("https://api.maiagent.ai/api/v1/conversations/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/conversations/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {}
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response

Status Code: 201

Example Response Schema

```

{
  "id": string (uuid)
  "contact":
  {
    "id": string (uuid)
    "name": string
    "sourceId?": string // Optional
  }
  "lastMessageCreatedAt": string (timestamp)
  "autoReplyEnabled": boolean
  "thinkingConfigSchema": object
  "effectiveThinkingConfig": object // Return the effective thinking config after applying the
  priority chain:
  conversation override > chatbot config > LLM metadata.
}

```

This allows the frontend to display the actual thinking config in use, even when the conversation has no explicit override.

Returns None when nothing is configured at any level, so the frontend can distinguish "Default" (None = use model/assistant default) from "Off" ({} = explicitly disabled). Collapsing the unconfigured case to {} would make a fresh conversation display as Off instead of Default.

```
"deepResearchStatus": // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
{
}
"deepResearchOutputFormat": // May have different types
string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas – no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
"callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "sourceId": "Response string"
  },
  "lastMessageCreatedAt": "Response string",
  "autoReplyEnabled": false,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
}
```

```
"callerOrganizationId": "Response string"
}
```

List Messages

GET

/api/v1/messages/

Parameters

Parameter	Required	Type	Description
conversation	V	string	Conversation ID
cursor	X	string	The pagination cursor value.
pageSize	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/messages/?
conversation=example&cursor=example&pageSize=1"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/messages/?
conversation=example&cursor=example&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/?
conversation=example&cursor=example&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/messages/?
conversation=example&cursor=example&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Example Response Schema

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "conversation": string (uuid)
      "sender":
        {
          "id": string (uuid)
          "name": string
          "avatar": string
          "email"?: string (email) // Optional
          "phoneNumber"?: string // Optional
        }
    }
  ]
}

```

```

    }
    "type"?: string // Optional
    "content"?: string // Optional
    "contentPayload"?: object // Optional
    "feedback":
    {
        "id": string (uuid)
        "type":
        {
        }
        "suggestion"?: string // Optional
        "updatedAt": string (timestamp)
    }
    "createdAt": string (timestamp)
    "attachments"?: [ // Optional
    {
        "id": string (uuid)
        "type"?: // Optional
        {
        }
        "filename": string
        "file": string (uri)
        "expiresAt": string // ISO datetime when this attachment will be auto-deleted.
    }

```

For conversation-bound attachments: conversation.last_message_created_at + retention_days.

For attachments without a conversation: created_at + retention_days.

Returns None when cleanup is disabled (effective_from not set).

```

    "conversation"?: string (uuid) // optional
    }
    ]
    "citations": [
    {
        "id": string (uuid)
        "filename": string // File name
        "file": string (uri) // File to upload
        "fileType": string
        "knowledgeBase"?: // optional
        {
            "id": string (uuid)
            "name": string
        }
        "size": integer
        "status":
        {
        }
        "parser":
        {
            "id": string (uuid)
            "name": string
            "provider":
            {
            }
            "isTimestampSttProvider": boolean
        }
        "labels"?: [ // optional
        {

```



```

        "id": string (uuid)
        "name": string
    }
]
"rawUserDefineMetadata"?: object // optional
"speakerLabels": [
    {
        "id": string (uuid)
        "originalLabel": string // Original speaker label from diarization (e.g.,
SPEAKER_00)
        "customName"?: string // User-defined speaker name (e.g., John) (optional)
        "displayName": string
    }
]
"vectorStorageSize": integer // Size of vectors for this file in Elasticsearch
(bytes)
"chunksCount": integer // Number of chunks/nodes generated from this file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"previewUrl": string // For presentations (pptx/ppt), returns the URL of a derived
PDF file that can be previewed (displayed using the PDF viewer on the frontend); otherwise,
returns None.

```

Generate the URL using the same CustomizedFileFieldSerializer as the `file` field to ensure a consistent format (including the presign/host rules for each storage provider). Do not use `get_absolute_url()`—it produces an incorrect URL format.

```

        "createdAt": string (timestamp)
    }
]
"citationNodes": [
    {
        "chatbotTextNode": {
            {
                "id": string (uuid)
                "charactersCount": integer
                "hitsCount": integer
                "text": string
                "updatedAt": string (timestamp)
                "filename": string
                "chatbotFile": object // Returns complete ChatbotFile information, including the
file URL and type, to support frontend image previews
                "knowledgeBaseFile": object // Same as get_chatbot_file, provided for backward
compatibility
                "pageNumber": integer // Backward-compatible alias of ``page_start``.
                "pageStart": integer
                "pageEnd": integer
                "citationTitle": string // Source title displayed in the inline citation hover card
                "citationDescription": string // Source description displayed in the inline
citation hover card
                "citationQuote": string // Quoted text excerpt displayed in the inline citation
hover card
                "hasImage": boolean // Indicates whether an image is included (checks the file type
or Markdown images in text)
                "imageUrl": string // Extracts the image URL (prefers the file URL; otherwise
extracts it from Markdown)

```

```
    "displayText": string // Returns the complete text with Markdown image markup removed
    "displayTitle": string // Returns the display title (fallback: citation_title -> filename)
    "highlightedText": string // Return text with matched terms wrapped in ``<mark>`` tags.
```

Priority:

1. ES/OpenSearch native highlight (set by retrieve_api via ``\_es\_highlighted\_text``)
2. Python regex fallback (keyword-based, works for all backends)

```
    "labels": [
        {
            "id": string (uuid)
            "name": string
        }
    ]
    "rawUserDefineMetadata"?: object // Users can define their own metadata keys and values (optional)
    "metadataEnabledForSearch": object // Map each user-defined metadata key on this node to whether it was sent into the LLM.
```

A key reaches the LLM only when its ``MetadataKey.enabled\_for\_search`` is True *and* the node carries a value for it. Since this maps over ``raw\_user\_define\_metadata`` (the keys the cited-documents block displays, all of which have a value), ``enabled\_for\_search`` alone decides the flag. The source of truth is the node's knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.

```
    "startCharIdx": integer
    "endCharIdx": integer
    }
    }
    "score"?: number (double) // optional
    "displayScore": integer
    "citationNumber"?: integer // Citation number matching the [1], [2] markers in the reply content (optional)
    "highlightedText"?: string // ES/OpenSearch highlight result with <mark> tags (optional)
    }
    ]
    "canvas"?: { // optional
    {
        "id": string (uuid)
        "name": string
        "canvasType":
        {
        }
        "title": string
        "content": string
        "createdAt": string (timestamp)
    }
    }
    "slideTemplate"?: // optional
    {
        "slug": string
        "displayName": string
        "thumbnailUrls"?: [ // optional
            string
```

```

    ]
  }
  "queryMetadata"?: { // Query metadata that controls the scope of queryable knowledge; see
each endpoint description for details (optional)
    {
      "knowledgeBases"?: [ // List of queryable knowledge bases; an empty array or null means
none can be queried (no permission), while omission means unrestricted (optional)
        {
          "knowledgeBaseId": string (uuid) // Knowledge base ID
          "chatbotFileIds"?: [ // List of file IDs (excluded or selected only, depending on
hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds;
when both are provided, this field takes precedence) (optional)
            string (uuid)
          ]
          "faqIds"?: [ // List of FAQ IDs (excluded or selected only, depending on
hasUserSelectedAll) (optional)
            string (uuid)
          ]
          "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base
and exclude listed items; false = select only listed items (optional)
        }
      ]
      "labelRelations"?: { // Label filtering criteria; an empty conditions array means no
label filtering (not no permission). Must be provided together with knowledgeBases—when
provided alone, label filtering is not applied to knowledge base retrieval (equivalent to
unrestricted) (optional)
        {
          "operator": string (enum: AND, OR) // How label conditions are combined
          "conditions"?: [ // List of label conditions supporting nested operator/conditions
combinations (optional)
            object
          ]
        }
      }
    }
  }
  "metadata"?: object // Message metadata, such as timezone and other environment
information (optional)
  "recordId": string // Returns the ChatbotRecord ID corresponding to the Message

```

Note:

For an unsaved Message (such as a virtual Message during streaming), return None directly to avoid triggering a DB query on the OneToOneField reverse relation (the N+1 problem).

```

"broadcast"?: boolean // optional
"skipCopilotTrigger"?: boolean // optional
"activeRevisionNumber": integer
"revisions": [
  {
    "id": string (uuid)
    "revisionNumber": integer
    "content": string
    "contentPayload": object
    "isActive": boolean
  }
]

```

```

        "createdAt": string (timestamp)
    }
]
"suggestedForId"?: string (uuid) // optional
"suggestionStatus": string
"suggestionTriggerSource": string
"suggestionError": string
"retryCount": integer // How many times the LLM call was auto-retried during this message
generation. Currently only Bedrock first-chunk stall triggers retry (max 1). Detailed per-
attempt records live in metadata["retry_attempts"].
"skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:
skillIds). (optional)
    string (uuid)
]
"toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase:
toolIds). (optional)
    string (uuid)
]
"templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (optional)
"videoGeneration": object // Return the latest GeneratedVideo state for FE reload
recovery;
``errorMessage`` is blank to match the socket emit (raw value stays on the DB row).
    }
]
}

```

Example Response Value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "conversation": "550e8400-e29b-41d4-a716-446655440000",
      "sender": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "avatar": "response string",
        "email": "response@example.com",
        "phoneNumber": "response string"
      },
      "type": "response string",
      "content": "response string",
      "contentPayload": null,
      "feedback": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "type": {},
        "suggestion": "response string",
        "updatedAt": "response string"
      },
      "createdAt": "response string",
      "attachments": [
    
```

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": {},
  "filename": "response string",
  "file": "https://example.com/file.jpg",
  "expiresAt": "response string",
  "conversation": "550e8400-e29b-41d4-a716-446655440000"
}
],
"citations": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "filename": "response string",
    "file": "https://example.com/file.jpg",
    "fileType": "response string",
    "knowledgeBase": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    },
    "size": 456,
    "status": {},
    "parser": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "provider": {},
      "isTimestampSttProvider": false
    },
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ],
    "rawUserDefineMetadata": null,
    "speakerLabels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "originalLabel": "response string",
        "customName": "response string",
        "displayName": "response string"
      }
    ],
    "vectorStorageSize": 456,
    "chunksCount": 456,
    "waitingTime": 456,
    "processingTime": 456,
    "processingTimeDetails": null,
    "previewUrl": "response string",
    "createdAt": "response string"
  }
],
"citationNodes": [
  {
    "chatbotTextNode": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "charactersCount": 456,

```

```
"hitsCount": 456,
"text": "response string",
"updatedAt": "response string",
"filename": "response string",
"chatbotFile": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"knowledgeBaseFile": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"pageNumber": 456,
"pageStart": 456,
"pageEnd": 456,
"citationTitle": "response string",
"citationDescription": "response string",
"citationQuote": "response string",
"hasImage": false,
"imageUrl": "response string",
"displayText": "response string",
"displayTitle": "response string",
"highlightedText": "response string",
"labels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"rawUserDefineMetadata": null,
"metadataEnabledForSearch": {
  "key1": "value1",
  "key2": "value2",
  "createdAt": "2025-01-01T00:00:00.000Z"
},
"startCharIdx": 456,
"endCharIdx": 456
},
"score": 456,
"displayScore": 456,
"citationNumber": 456,
"highlightedText": "response string"
}
],
"canvas": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "canvasType": {},
  "title": "response string",
  "content": "response string",
  "createdAt": "response string"
},
"slideTemplate": {
  "slug": "response string",
```

```

    "displayName": "response string",
    "thumbnailUrls": [
      "response string"
    ]
  },
  "queryMetadata": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "metadata": null,
  "recordId": "response string",
  "broadcast": false,
  "skipCopilotTrigger": false,
  "activeRevisionNumber": 456,
  "revisions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "revisionNumber": 456,
      "content": "response string",
      "contentPayload": null,
      "isActive": false,
      "createdAt": "response string"
    }
  ],
  "suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
  "suggestionStatus": "response string",
  "suggestionTriggerSource": "response string",
  "suggestionError": "response string",
  "retryCount": 456,
  "skillIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "toolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "templateId": "550e8400-e29b-41d4-a716-446655440000",
  "videoGeneration": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
}
]
}

```

Retrieve a Specific Message

GET

/api/v1/messages/{id}

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Message.
<code>conversation</code>	V	string	Conversation ID

Code Examples

Shell/Bash

```
# Call the API (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/messages/550e8400-e29b-41d4-a716-446655440000/?conversation=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running the example.
```


JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/messages/550e8400-e29b-41d4-a716-446655440000/?
conversation=example", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/550e8400-e29b-41d4-a716-446655440000/?
conversation=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/messages/550e8400-e29b-41d4-
a716-446655440000/?conversation=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Example Response Schema

```
{
  "id": string (uuid)
  "conversation": string (uuid)
  "sender":
  {
    "id": string (uuid)
    "name": string
    "avatar": string
    "email?": string (email) // optional
    "phoneNumber?": string // optional
  }
  "type?": string // optional
  "content?": string // optional
  "contentPayload?": object // optional
  "feedback":
```

```

{
  "id": string (uuid)
  "type":
  {
  }
  "suggestion"?: string // optional
  "updatedAt": string (timestamp)
}
"createdAt": string (timestamp)
"attachments"?: [ // optional
  {
    "id": string (uuid)
    "type"?: // optional
    {
    }
    "filename": string
    "file": string (uri)
    "expiresAt": string // ISO datetime when this attachment will be auto-deleted.
  }
]

```

For conversation-bound attachments: conversation.last_message_created_at + retention_days.

For attachments without a conversation: created_at + retention_days.

Returns None when cleanup is disabled (effective_from not set).

```

  "conversation"?: string (uuid) // optional
}
]
" Citations": [
  {
    "id": string (uuid)
    "filename": string // File name
    "file": string (uri) // File to upload
    "fileType": string
    "knowledgeBase"?: // optional
    {
      "id": string (uuid)
      "name": string
    }
    "size": integer
    "status":
    {
    }
    "parser":
    {
      "id": string (uuid)
      "name": string
      "provider":
      {
      }
      "isTimestampSttProvider": boolean
    }
    "labels"?: [ // optional
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "rawUserDefineMetadata"?: object // optional
  }
]

```

```

"speakerLabels": [
  {
    "id": string (uuid)
    "originalLabel": string // Original speaker label from diarization (e.g., SPEAKER_00)
    "customName?": string // User-defined speaker name (e.g., John) (optional)
    "displayName": string
  }
]
"vectorStorageSize": integer // Size of vectors for this file in Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from this file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"previewUrl": string // For presentations (pptx/ppt), returns the URL of a derived PDF
file that can be previewed (displayed using the PDF viewer on the frontend); otherwise, returns
None.

```

Generate the URL using the same CustomizedFileFieldSerializer as the `file` field to ensure a consistent format (including the presign/host rules for each storage provider). Do not use `get_absolute_url()`—it produces an incorrect URL format.

```

"createdAt": string (timestamp)
}
]
"citationNodes": [
  {
    "chatbotTextNode": {
      {
        "id": string (uuid)
        "charactersCount": integer
        "hitsCount": integer
        "text": string
        "updatedAt": string (timestamp)
        "filename": string
        "chatbotFile": object // Returns complete ChatbotFile information, including the file
URL and type, to support frontend image previews
        "knowledgeBaseFile": object // Same as get_chatbot_file, provided for backward
compatibility
        "pageNumber": integer // Backward-compatible alias of ``page_start``.
        "pageStart": integer
        "pageEnd": integer
        "citationTitle": string // Source title displayed in the inline citation hover card
        "citationDescription": string // Source description displayed in the inline citation
hover card
        "citationQuote": string // Quoted text excerpt displayed in the inline citation hover
card
        "hasImage": boolean // Indicates whether an image is included (checks the file type or
Markdown images in text)
        "imageUrl": string // Extracts the image URL (prefers the file URL; otherwise extracts
it from Markdown)
        "displayText": string // Returns the complete text with Markdown image markup removed
        "displayTitle": string // Returns the display title (fallback: citation_title ->
filename)
        "highlightedText": string // Return text with matched terms wrapped in ``<mark>`` tags.

```

Priority:

1. ES/OpenSearch native highlight (set by `retrieve_api` via ``\_es\_highlighted\_text``)

2. Python regex fallback (keyword-based, works for all backends)

```
"labels": [
  {
    "id": string (uuid)
    "name": string
  }
]
"rawUserDefineMetadata"?: object // Users can define their own metadata keys and values
(optional)
"metadataEnabledForSearch": object // Map each user-defined metadata key on this node
to whether it was sent into the LLM.
A key reaches the LLM only when its ``MetadataKey.enabled_for_search`` is True *and*
the node carries a value for it. Since this maps over ``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the node's
knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.
"startCharIdx": integer
"endCharIdx": integer
}
}
"score"?: number (double) // optional
"displayScore": integer
"citationNumber"?: integer // Citation number matching the [1], [2] markers in the reply
content (optional)
"highlightedText"?: string // ES/OpenSearch highlight result with <mark> tags (optional)
}
]
"canvas"?: { // optional
{
  "id": string (uuid)
  "name": string
  "canvasType":
  {
  }
  "title": string
  "content": string
  "createdAt": string (timestamp)
}
}
"slideTemplate"?: // optional
{
  "slug": string
  "displayName": string
  "thumbnailUrls"?: [ // optional
    string
  ]
}
}
"queryMetadata"?: { // Query metadata that controls the searchable knowledge scope; see each
endpoint description for details (optional)
{
  "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means
none are searchable (no permission), while omission means unrestricted (optional)
  {
    "knowledgeBaseId": string (uuid) // Knowledge base ID
    "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected according to
hasUserSelectedAll) (optional)
```

```

        string (uuid)
    ]
    "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; when
both are provided, this field takes precedence) (optional)
        string (uuid)
    ]
    "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected according to
hasUserSelectedAll) (optional)
        string (uuid)
    ]
    "hasUserSelectedAll"?: boolean // true=select all content in the knowledge base and
exclude listed items; false = select only listed items (optional)
}
]
"labelRelations"?: { // Label filter criteria; an empty conditions array means no label
filtering (not no permission). Must be provided together with knowledgeBases—when provided
alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
(optional)
    {
        "operator": string (enum: AND, OR) // How label conditions are combined
        "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (optional)
            object
        ]
    }
}
}
}
"metadata"?: object // Message metadata, such as timezone and other environment information
(optional)
"recordId": string // Return the ChatbotRecord ID corresponding to the Message

```

Note:

For an unsaved Message (such as a virtual Message during streaming), return None directly, to avoid triggering a DB query on the OneToOneField reverse relation (the N+1 problem).

```

"broadcast"?: boolean // optional
"skipCopilotTrigger"?: boolean // optional
"activeRevisionNumber": integer
"revisions": [
    {
        "id": string (uuid)
        "revisionNumber": integer
        "content": string
        "contentPayload": object
        "isActive": boolean
        "createdAt": string (timestamp)
    }
]
"suggestedForId"?: string (uuid) // optional
"suggestionStatus": string
"suggestionTriggerSource": string
"suggestionError": string
"retryCount": integer // How many times the LLM call was auto-retried during this message
generation. Currently only Bedrock first-chunk stall triggers retry (max 1). Detailed per-
attempt records live in metadata["retry_attempts"].
"skillIds"?: [ // Per-message selected skill IDs from the WebChat client (camelCase:

```

```

skillIds). (optional)
  string (uuid)
]
"toolIds"?: [ // Per-message selected tool IDs from the WebChat client (camelCase: toolIds).
(optional)
  string (uuid)
]
"templateId"?: string (uuid) // Document/sheet template ID for file-level compilation
(camelCase: templateId). (optional)
"videoGeneration": object // Return the latest GeneratedVideo state for FE reload recovery;
``errorMessage`` is blank to match the socket emit (raw value stays on the DB row).
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "conversation": "550e8400-e29b-41d4-a716-446655440000",
  "sender": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "avatar": "response string",
    "email": "response@example.com",
    "phoneNumber": "response string"
  },
  "type": "response string",
  "content": "response string",
  "contentPayload": null,
  "feedback": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": {},
    "suggestion": "response string",
    "updatedAt": "response string"
  },
  "createdAt": "response string",
  "attachments": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "type": {},
      "filename": "response string",
      "file": "https://example.com/file.jpg",
      "expiresAt": "response string",
      "conversation": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "citations": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "response string",
      "file": "https://example.com/file.jpg",
      "fileType": "response string",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    }
  ],

```

```
"size": 456,
"status": {},
"parser": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "provider": {},
  "isTimestampSttProvider": false
},
"labels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  }
],
"rawUserDefineMetadata": null,
"speakerLabels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "originalLabel": "response string",
    "customName": "response string",
    "displayName": "response string"
  }
],
"vectorStorageSize": 456,
"chunksCount": 456,
"waitingTime": 456,
"processingTime": 456,
"processingTimeDetails": null,
"previewUrl": "response string",
"createdAt": "response string"
}
],
"citationNodes": [
  {
    "chatbotTextNode": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "charactersCount": 456,
      "hitsCount": 456,
      "text": "response string",
      "updatedAt": "response string",
      "filename": "response string",
      "chatbotFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "knowledgeBaseFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
      },
      "pageNumber": 456,
      "pageStart": 456,
      "pageEnd": 456,
      "citationTitle": "response string",
      "citationDescription": "response string",
    }
  }
]
```



```
    "citationQuote": "response string",
    "hasImage": false,
    "imageUrl": "response string",
    "displayText": "response string",
    "displayTitle": "response string",
    "highlightedText": "response string",
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ],
    "rawUserDefineMetadata": null,
    "metadataEnabledForSearch": {
      "key1": "value1",
      "key2": "value2",
      "createdAt": "2025-01-01T00:00:00.000Z"
    },
    "startCharIdx": 456,
    "endCharIdx": 456
  },
  "score": 456,
  "displayScore": 456,
  "citationNumber": 456,
  "highlightedText": "response string"
}
],
"canvas": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "canvasType": {},
  "title": "response string",
  "content": "response string",
  "createdAt": "response string"
},
"slideTemplate": {
  "slug": "response string",
  "displayName": "response string",
  "thumbnailUrls": [
    "response string"
  ]
},
"queryMetadata": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"metadata": null,
"recordId": "response string",
"broadcast": false,
"skipCopilotTrigger": false,
"activeRevisionNumber": 456,
"revisions": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "revisionNumber": 456,
```

```
    "content": "response string",
    "contentPayload": null,
    "isActive": false,
    "createdAt": "response string"
  }
],
"suggestedForId": "550e8400-e29b-41d4-a716-446655440000",
"suggestionStatus": "response string",
"suggestionTriggerSource": "response string",
"suggestionError": "response string",
"retryCount": 456,
"skillIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"toolIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"templateId": "550e8400-e29b-41d4-a716-446655440000",
"videoGeneration": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
}
}
```

List conversations

GET

/api/v1/conversations/

Parameters

Parameter	Required	Type	Description
assignee	X	string	
chatbot	X	string	
contact	X	string	
contactTags	X	array	Multiple values may be separated by commas.
cursor	X	string	The pagination cursor value.
endDate	X	string	
externalConversationId	X	string	

externalSource	X	string	
inbox	X	string	
isPinned	X	string	
mine	X	boolean	
mode	X	string	`chat`: Chat; `cowork`: Cowork;
pageSize	X	integer	Number of results to return per page.
startDate	X	string	
status	X	array	`open`: Open; `resolved`: Resolved; `queued`: Queued;
tags	X	array	Multiple values may be separated by commas.
unassigned	X	boolean	
updatedAt	X	string	

Code examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/conversations/?assignee=550e8400-e29b-41d4-a716-446655440000&chatbot=550e8400-e29b-41d4-a716-446655440000&contact=550e8400-e29b-41d4-a716-446655440000&contactTags=example&cursor=example&endDate=example&externalConversationId=example&externalSource=example&inbox=550e8400-e29b-41d4-a716-446655440000&isPinned=example&mine=true&mode=chat&pageSize=1&startDate=example&status=example&tags=example&unassigned=true&updatedAt=2025-01-01T00:00:00.000Z"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/conversations/?assignee=550e8400-e29b-41d4-a716-446655440000&chatbot=550e8400-e29b-41d4-a716-446655440000&contact=550e8400-e29b-41d4-a716-446655440000&contactTags=example&cursor=example&endDate=example&externalConversationId=example&externalSource=example&inbox=550e8400-e29b-41d4-a716-446655440000&isPinned=example&mine=true&mode=chat&pageSize=1&startDate=example&status=example&tags=example&unassigned=true&updatedAt=2025-01-01T00:00:00.000Z", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/?assignee=550e8400-e29b-41d4-a716-446655440000&chatbot=550e8400-e29b-41d4-a716-446655440000&contact=550e8400-e29b-41d4-a716-446655440000&contactTags=example&cursor=example&endDate=example&externalConversationId=example&externalSource=example&inbox=550e8400-e29b-41d4-a716-446655440000&isPinned=example&mine=true&mode=chat&pageSize=1&startDate=example&status=example&tags=example&unassigned=true&updatedAt=2025-01-01T00:00:00.000Z"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/conversations/?
assignee=550e8400-e29b-41d4-a716-446655440000&chatbot=550e8400-e29b-41d4-a716-
446655440000&contact=550e8400-e29b-41d4-a716-
446655440000&contactTags=example&cursor=example&endDate=example&externalConversationId=ex
ample&externalSource=example&inbox=550e8400-e29b-41d4-a716-
446655440000&isPinned=example&mine=true&mode=chat&pageSize=1&startDate=example&status=exa
mple&tags=example&unassigned=true&updatedAt=2025-01-01T00:00:00.000Z", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>

```

Response

Status code: 200

Example response structure

```

{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "contact": { // Lightweight Contact Serializer for the Conversation List view

```

Includes only the fields required by the List view, excluding inboxes, mcp_credentials, and

api_credentials

to avoid N+1 query issues. email / phone_number / metadata are scalar fields on Contact itself, which the frontend conversation sidebar needs to display, with no N+1 cost.

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string // Stores our own media as a relative storage key, or an external CDN
URL as-is. (optional)
  "email"?: // May have different types (optional)
string (email) // optional
  "phoneNumber"?: string // optional
  "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (optional)
  "sourceId"?: string // optional
  "tags": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
}
}
"inbox": { // Inbox summary embedded in a conversation payload.
```

``unread\_conversations\_count`` stays here because it is part of the documented REST conversation API. It is an org-wide denormalized counter, so WS emit paths must run the payload through :func:`strip\_org\_wide\_unread\_count` before broadcasting it to rooms that include members without ``GroupInbox.can\_view\_conversations``.

```
{
  "id": string (uuid)
  "name": string
  "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
  "unreadConversationsCount"?: integer // optional
  "signAuth"?: // optional
  {
    "id": string (uuid)
    "signSource": string (uuid)
    "signParams": {
      {
        "keycloak":
        {
          "clientId": string
          "url": string
```

```

    "realm": string
  }
  "line":
  {
    "liffId": string
  }
  "ad":
  {
    "saml": string
  }
  "google"?: // optional
  {
    "enabled": boolean
  }
}
}

"sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a
Source ID (verified per the verify mode below) may chat without going through the login flow.
Disabled keeps every existing behavior unchanged. (optional)
"sourceIdVerifyMode"?: // "Require signature verification": the embedding site must
send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID
can act as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (optional)
{
}

"sourceIdSignatureTtlMinutes"?: // How long a signature can be used to obtain the
contact identity. Not a session lifetime: once exchanged, the session follows the existing
WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (optional)
{
}

"keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a
visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API
tools on their behalf, so tools do not require a separate per-tool login. Only for the Keycloak
sign source. Never overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the conversation at
login, never the contact Source ID. (optional)
"keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com"
wildcards) of the tools the visitor Keycloak token may be sent to (optional)
  string
]
}
}

"enableAnalysis"?: boolean // optional
}
}

"chatbot": object // Chatbot (Agent) summary from the conversation's inbox, already
eager-loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon, feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
"title": string

```



```

    "lastMessage"?: { // optional
    {
        "id": string (uuid)
        "type"?: string // optional
        "content"?: string // optional
        "createdAt": string (timestamp)
    }
    }
    "callRecord":
    {
        "id": string (uuid)
        "status"?: string (enum: initiated, ringing, in-progress, completed, failed, busy, no-
answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (optional)
        "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
        "isActive": boolean
        "canHangup": boolean
    }
    "lastMessageCreatedAt": string (timestamp)
    "unreadMessagesCount": integer
    "autoReplyEnabled": boolean
    "isAutoReplyNow": boolean
    "lastReadAt": string (timestamp)
    "createdAt": string (timestamp)
    "isGroupChat": boolean
    "enableGroupMention"?: boolean // optional
    "queryMetadata"?: { // Query metadata that controls the searchable knowledge scope; see
each endpoint description for details (optional)
    {
        "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null
means none are searchable (no permission), while omission means unrestricted (optional)
        {
            "knowledgeBaseId": string (uuid) // Knowledge base ID
            "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected
according to hasUserSelectedAll) (optional)
                string (uuid)
            ]
            "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds;
when both are provided, this field takes precedence) (optional)
                string (uuid)
            ]
            "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected according to
hasUserSelectedAll) (optional)
                string (uuid)
            ]
            "hasUserSelectedAll"?: boolean // true=select all content in the knowledge base
and exclude listed items; false = select only listed items (optional)
        }
    ]
}

```

```

    "labelRelations"?: { // Label filter criteria; an empty conditions array means no label
filtering (not no permission). Must be provided together with knowledgeBases—when provided
alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
(optional)
    {
        "operator": string (enum: AND, OR) // How label conditions are combined
        "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (optional)
            object
        ]
    }
}
}
}
"toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (optional)
"connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(optional)
"thinkingConfig"?: object // None = unset (inherit the chatbot setting), {} = explicitly
off, {...} = custom override (optional)
"thinkingConfigSchema": object
"effectiveThinkingConfig": object // Return the effective thinking config after applying
the priority chain:
conversation override > chatbot config > LLM metadata.

```

This allows the frontend to display the actual thinking config in use, even when the conversation has no explicit override.

Returns None when nothing is configured at any level, so the frontend can distinguish "Default" (None = use model/assistant default) from "Off" ({} = explicitly disabled). Collapsing the unconfigured case to {} would make a fresh conversation display as Off instead of Default.

```

    "llm": string (uuid)
    "deepResearchStatus": // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
    {
    }
    "deepResearchOutputFormat": // May have different types
    string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set
only when the user clicks a format in the pre-research choice card; reset to NULL each time
Deep Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and
falls back to Canvas at render time, but stays distinct from an explicit canvas pick so the
frontend can lock the card only after a real selection. To roll back to fixed-Canvas behavior,
force this to NULL/canvas – no redeploy needed.
* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
    "ipAddress": string
    "ipCountryCode": string
    "ipRecordedAt": string (timestamp)
    "assignee":
    {

```

```

        "id": string (uuid)
        "name": string
        "userId": string (uuid)
    }
    "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
    "tags": [
        {
            "id": string (uuid)
            "name": string
            "memberIds"?: [ // Optional
                string (uuid)
            ]
        }
    ]
    "progressStatus": string
    "firstResponseAt"?: string (timestamp) // Optional
    "queuedAt"?: string (timestamp) // Optional
    "assignedAt"?: string (timestamp) // Optional
    "transferReason"?: string // Optional
    "satisfactionScore": integer // Conversation satisfaction rating on a 5-point scale
(1=very dissatisfied, 5=very satisfied)
    "satisfactionComment": string
    "satisfactionSubmittedAt": string (timestamp)
    "latestAnalysisSummary": string // The conversation's current analysis summary (its
active summary revision).

```

Reads the `latest\_analysis\_summary` annotation added by `setup\_eager\_loading` to avoid an N+1 query. All read paths (list / retrieve / create) eager-load, so the annotation is present; it falls back to '' only for instances built outside `setup\_eager\_loading`.

```

        "clientPlatform": // Client platform that created this conversation (web, desktop)

* `web` - Web
* `desktop` - Desktop
    {
    }
    "workingDirectory": string // Absolute path of the local folder bound to a desktop
conversation. Only set for desktop conversations; null for web conversations.
    "slideProjectId": string
    "matchedField": string // Cached per instance: ``get_matched_message_id`` /
``get_matched_snippet``
both read this, so resolving it once avoids recomputing the keyword and
title-match logic three times per serialized conversation.
    "matchedSnippet": string // Keyword-centred excerpt of the matched message, with
ellipses.

```

Reads the ``matched\_message\_content`` annotation added by ``ConversationViewSet.\_annotate\_keyword\_match``; no extra query.

```

        "matchedMessageId": string
        "pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.
        "folder": string (uuid) // The user-defined sidebar folder this conversation is grouped
into. Null means unclassified (shown under the time groups). Deleting the folder resets this to
null without deleting the conversation.
        "hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to

```

resume.

Lets the frontend know it should proactively re-request the browser panel (emit ``browser:start``) after a page reload, instead of only ever showing the panel when a live ``browser:frame`` push happens to arrive first.

```
"callerOrganizationId": string // Bound caller organization (stamped at conversation start); null when unbound
"handoff": object // Handoff relationship: a continued conversation contains source_conversation_id/source_title; the original handed-off conversation contains continued_to_conversation_id/continued_to_title; null for conversations without a handoff
}
]
}
```

Example Response Value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "contact": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "avatar": "response string",
        "email": "response@example.com",
        "phoneNumber": "response string",
        "metadata": null,
        "sourceId": "response string",
        "tags": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "response string"
          }
        ]
      },
    },
    {
      "inbox": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "channelType": "line",
        "unreadConversationsCount": 456,
        "signAuth": {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "signSource": "550e8400-e29b-41d4-a716-446655440000",
          "signParams": {
            "keycloak": {
              "clientId": "response string",
              "url": "response string",
              "realm": "response string"
            }
          },
          "line": {
            "liffId": "response string"
          }
        }
      }
    }
  ]
}
```

```
    },
    "ad": {
      "saml": "response string"
    },
    "google": {
      "enabled": false
    }
  },
  "sourceIdAccessEnabled": false,
  "sourceIdVerifyMode": {},
  "sourceIdSignatureTtlMinutes": {},
  "keycloakTokenPassthrough": false,
  "keycloakTokenPassthroughAllowedHosts": [
    "response string"
  ]
},
"enableAnalysis": false
},
"chatbot": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"title": "response string",
"lastMessage": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "response string",
  "content": "response string",
  "createdAt": "response string"
},
"callRecord": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "initiated",
  "direction": "inbound",
  "isActive": false,
  "canHangup": false
},
"lastMessageCreatedAt": "response string",
"unreadMessagesCount": 456,
"autoReplyEnabled": false,
"isAutoReplyNow": false,
"lastReadAt": "response string",
"createdAt": "response string",
"isGroupChat": false,
"enableGroupMention": false,
"queryMetadata": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"toolMask": null,
"connectorMask": null,
"thinkingConfig": null,
"thinkingConfigSchema": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
```

```
    "description": "Example response description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "response string",
  "ipCountryCode": "response string",
  "ipRecordedAt": "response string",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "status": "open",
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string",
      "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ]
    }
  ],
  "progressStatus": "response string",
  "firstResponseAt": "response string",
  "queuedAt": "response string",
  "assignedAt": "response string",
  "transferReason": "response string",
  "satisfactionScore": 456,
  "satisfactionComment": "response string",
  "satisfactionSubmittedAt": "response string",
  "latestAnalysisSummary": "response string",
  "clientPlatform": {},
  "workingDirectory": "response string",
  "slideProjectId": "response string",
  "matchedField": "response string",
  "matchedSnippet": "response string",
  "matchedMessageId": "response string",
  "pinnedAt": "response string",
  "folder": "550e8400-e29b-41d4-a716-446655440000",
  "hasActiveBrowserSession": false,
  "callerOrganizationId": "response string",
  "handoff": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  }
}
]
```

Retrieve a Specific Conversation

GET

/api/v1/conversations/{id}

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Conversation.

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```



```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```

{
  "id": string (uuid)
  "contact":
  {
    "id": string (uuid)
    "name": string
    "sourceId?": string // Optional
  }
  "lastMessageCreatedAt": string (timestamp)
  "autoReplyEnabled": boolean
  "thinkingConfigSchema": object
  "effectiveThinkingConfig": object // Return the effective thinking config after applying the
  priority chain:
  conversation override > chatbot config > LLM metadata.
}

```

This allows the frontend to display the actual thinking config in use, even when the conversation has no explicit override.

Returns None when nothing is configured at any level, so the frontend can distinguish "Default" (None = use model/assistant default) from "Off" ({} = explicitly disabled). Collapsing the unconfigured case to {} would make a fresh conversation display as Off instead of Default.

```
"deepResearchStatus": // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
{
}
"deepResearchOutputFormat": // May have different types
string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas – no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
"callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "sourceId": "response string"
  },
  "lastMessageCreatedAt": "response string",
  "autoReplyEnabled": false,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
}
```

```
"callerOrganizationId": "response string"
}
```

Retrieve a Specific Conversation

GET

[/api/v1/conversations/tab-counts/](#)

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/conversations/tab-counts/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/conversations/tab-counts/", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/tab-counts/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/conversations/tab-counts/",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]));

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```

{
  "id": string (uuid)
  "contact": { // Lightweight Contact Serializer for the Conversation List view

Includes only the fields required by the List view, removing inboxes, mcp_credentials, and
api_credentials
to avoid N+1 query issues. email / phone_number / metadata are scalar fields on Contact itself,
which the frontend conversation sidebar needs to display without any N+1 cost.
    {
      "id": string (uuid)
      "name": string
      "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (Optional)
      "email"?: // May have different types (Optional)
      string (email) // Optional
    }
  }
}

```

```

    "phoneNumber"?: string // Optional
    "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
    "sourceId"?: string // Optional
    "tags": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
  }
}
"inbox": { // Inbox summary embedded in a conversation payload.

```

``unread\_conversations\_count`` stays here because it is part of the documented REST conversation API. It is an org-wide denormalized counter, so WS emit paths must run the payload through :func:`strip\_org\_wide\_unread\_count` before broadcasting it to rooms that include members without ``GroupInbox.can\_view\_conversations``.

```

  {
    "id": string (uuid)
    "name": string
    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // Optional
    "signAuth"?: // Optional
    {
      "id": string (uuid)
      "signSource": string (uuid)
      "signParams": {
        {
          "keycloak":
          {
            "clientId": string
            "url": string
            "realm": string
          }
          "line":
          {
            "liffId": string
          }
          "ad":
          {
            "saml": string
          }
        }
      }
    }
  }

```

```

    "google"?: // Optional
    {
        "enabled": boolean
    }
}
}

"sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source
ID (verified per the verify mode below) may chat without going through the login flow. Disabled
keeps every existing behavior unchanged. (Optional)

"sourceIdVerifyMode"?: // "Require signature verification": the embedding site must send
an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act
as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
    {
    }

    "sourceIdSignatureTtlMinutes"?: // How long a signature can be used to obtain the
contact identity. Not a session lifetime: once exchanged, the session follows the existing
WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
    {
    }

    "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor
logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on
their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign
source. Never overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the conversation at
login, never the contact Source ID. (Optional)

    "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (Optional)
        string
    ]
}
"enableAnalysis"?: boolean // Optional
}
}

"chatbot": object // Chatbot (Agent) summary from the conversation's inbox, already eager-
loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon, feature
gate) without comparing against the organization's ``maigpt_inbox`` id.

"title": string
"lastMessage"?: { // Optional
    {
        "id": string (uuid)
        "type"?: string // Optional
        "content"?: string // Optional
        "createdAt": string (timestamp)
    }
}
}
"callRecord":
{

```

```

    "id": string (uuid)
    "status"?: string (enum: initiated, ringing, in-progress, completed, failed, busy, no-
answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (Optional)
    "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
    "isActive": boolean
    "canHangup": boolean
}
"lastMessageCreatedAt": string (timestamp)
"unreadMessagesCount": integer
"autoReplyEnabled": boolean
"isAutoReplyNow": boolean
"lastReadAt": string (timestamp)
"createdAt": string (timestamp)
"isGroupChat": boolean
"enableGroupMention"?: boolean // Optional
"queryMetadata"?: { // Query metadata that controls the searchable knowledge scope; see each
endpoint description for details (Optional)
{
    "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means
none are searchable (no permission), while omission means unrestricted (Optional)
{
    "knowledgeBaseId": string (uuid) // Knowledge base ID
    "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
        string (uuid)
    ]
    "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; this
field takes precedence when both are provided) (Optional)
        string (uuid)
    ]
    "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
        string (uuid)
    ]
    "hasUserSelectedAll"?: boolean // true = select all knowledge base content except the
listed items; false = select only the listed items (Optional)
}
}
    "labelRelations"?: { // Label filter criteria; an empty conditions array means no label
filtering (not no permission). Must be provided together with knowledgeBases—when provided
alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
(Optional)
{
    "operator": string (enum: AND, OR) // How label conditions are combined
    "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (Optional)
        object
    ]
}
}

```



```

    }
  }
}
"toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
"connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
"thinkingConfig"?: object // None = unset (inherit the chatbot setting), {} = explicitly off,
{...} = custom override (Optional)
"thinkingConfigSchema": object
"effectiveThinkingConfig": object // Return the effective thinking config after applying the
priority chain:
conversation override > chatbot config > LLM metadata.

```

This allows the frontend to display the actual thinking config in use, even when the conversation has no explicit override.

Returns None when nothing is configured at any level, so the frontend can distinguish "Default" (None = use model/assistant default) from "Off" ({} = explicitly disabled). Collapsing the unconfigured case to {} would make a fresh conversation display as Off instead of Default.

```

"llm": string (uuid)
"deepResearchStatus": // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
{
}
"deepResearchOutputFormat": // May have different types
string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only
when the user clicks a format in the pre-research choice card; reset to NULL each time Deep
Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls
back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend
can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force
this to NULL/canvas – no redeploy needed.

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
"ipAddress": string
"ipCountryCode": string
"ipRecordedAt": string (timestamp)
"assignee":
{
  "id": string (uuid)
  "name": string
  "userId": string (uuid)
}
"status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (optional)
"tags": [
  {

```

```

    "id": string (uuid)
    "name": string
    "memberIds"?: [ // optional
        string (uuid)
    ]
}
]
"progressStatus": string
"firstResponseAt"?: string (timestamp) // optional
"queuedAt"?: string (timestamp) // optional
"assignedAt"?: string (timestamp) // optional
"transferReason"?: string // optional
"satisfactionScore": integer // Conversation satisfaction rating on a 5-point scale (1=very
dissatisfied, 5=very satisfied)
"satisfactionComment": string
"satisfactionSubmittedAt": string (timestamp)
"latestAnalysisSummary": string // The conversation's current analysis summary (its active
summary revision).

```

Reads the `latest\_analysis\_summary` annotation added by `setup\_eager\_loading` to avoid an N+1 query. All read paths (list / retrieve / create) eager-load, so the annotation is present; it falls back to '' only for instances built outside `setup\_eager\_loading`.

```

"clientPlatform": // Client platform that created this conversation (web, desktop)

```

```

* `web` - Web

```

```

* `desktop` - Desktop

```

```

{
}

```

```

"workingDirectory": string // Absolute path of the local folder bound to a desktop
conversation. Only set for desktop conversations; null for web conversations.

```

```

"slideProjectId": string

```

```

"matchedField": string // Cached per instance: ``get_matched_message_id`` /
``get_matched_snippet``

```

both read this, so resolving it once avoids recomputing the keyword and title-match logic three times per serialized conversation.

```

"matchedSnippet": string // Keyword-centred excerpt of the matched message, with ellipses.

```

Reads the ``matched\_message\_content`` annotation added by ``ConversationViewSet.annotate\_keyword\_match``; no extra query.

```

"matchedMessageId": string

```

```

"pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.

```

```

"folder": string (uuid) // The user-defined sidebar folder this conversation is grouped into.
Null means unclassified (shown under the time groups). Deleting the folder resets this to null
without deleting the conversation.

```

```

"hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to
resume.

```

Lets the frontend know it should proactively re-request the browser panel (emit ``browser:start``) after a page reload, instead of only ever showing the panel when a live ``browser:frame`` push happens to arrive first.

```

"callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound

```

```

"handoff": object // Handoff relationship: a continued conversation contains
source_conversation_id/source_title; the original handed-off conversation contains

```

```
continued_to_conversation_id/continued_to_title; null for conversations without a handoff
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "avatar": "response string",
    "email": "response@example.com",
    "phoneNumber": "response string",
    "metadata": null,
    "sourceId": "response string",
    "tags": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ]
  },
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "channelType": "line",
    "unreadConversationsCount": 456,
    "signAuth": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "response string",
          "url": "response string",
          "realm": "response string"
        },
        "line": {
          "liffId": "response string"
        },
        "ad": {
          "saml": "response string"
        },
        "google": {
          "enabled": false
        }
      },
      "sourceIdAccessEnabled": false,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": false,
      "keycloakTokenPassthroughAllowedHosts": [
        "response string"
      ]
    },
    "enableAnalysis": false
  }
}
```

```
},
"chatbot": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"title": "response string",
"lastMessage": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "response string",
  "content": "response string",
  "createdAt": "response string"
},
"callRecord": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "initiated",
  "direction": "inbound",
  "isActive": false,
  "canHangup": false
},
"lastMessageCreatedAt": "response string",
"unreadMessagesCount": 456,
"autoReplyEnabled": false,
"isAutoReplyNow": false,
"lastReadAt": "response string",
"createdAt": "response string",
"isGroupChat": false,
"enableGroupMention": false,
"queryMetadata": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"toolMask": null,
"connectorMask": null,
"thinkingConfig": null,
"thinkingConfigSchema": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"effectiveThinkingConfig": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"llm": "550e8400-e29b-41d4-a716-446655440000",
"deepResearchStatus": {},
"deepResearchOutputFormat": "canvas",
"ipAddress": "response string",
"ipCountryCode": "response string",
"ipRecordedAt": "response string",
"assignee": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "userId": "550e8400-e29b-41d4-a716-446655440000"
```

```
},
"status": "open",
"tags": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string",
    "memberIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }
],
"progressStatus": "response string",
"firstResponseAt": "response string",
"queuedAt": "response string",
"assignedAt": "response string",
"transferReason": "response string",
"satisfactionScore": 456,
"satisfactionComment": "response string",
"satisfactionSubmittedAt": "response string",
"latestAnalysisSummary": "response string",
"clientPlatform": {},
"workingDirectory": "response string",
"slideProjectId": "response string",
"matchedField": "response string",
"matchedSnippet": "response string",
"matchedMessageId": "response string",
"pinnedAt": "response string",
"folder": "550e8400-e29b-41d4-a716-446655440000",
"hasActiveBrowserSession": false,
"callerOrganizationId": "response string",
"handoff": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
}
}
```

Rename Conversation

PATCH </api/v1/conversations/{id}/rename/>

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Conversation.

Request Body

Request Parameters

Field	Type	Required	Description
contact	object	No	Lightweight Contact Serializer for the Conversation List view. It includes only the fields required by the List view and removes inboxes, mcp_credentials, and api_credentials to avoid N+1 query issues. email / phone_number / metadata are scalar fields on Contact itself that the frontend conversation sidebar needs to display, with no N+1 cost.
contact.name	string	Yes	
contact.avatar	string	No	Stores our own media as a relative storage key, or an external CDN URL as-is.
contact.email	object	No	
contact.phoneNumber	string	No	
contact.metadata	object	No	Custom customer information shown in the customer panel of the conversation page
contact.sourceId	string	No	
inbox	object	No	Inbox summary embedded in a conversation payload. unread_conversations_count stays here because it is part of the documented REST conversation API. It is an org-wide denormalized counter, so WS em...
inbox.name	string	Yes	
inbox.channelType	string (enum: line, telegram, teams, web, messenger, instagram, email, whatsapp, slack, team_plus,	Yes	line : LINE; telegram : Telegram; teams : Teams; web : Web; messenger : Messenger; instagram : Instagram; email : Email; whatsapp : WhatsApp; slack : Slack; team_plus : Team+; `lin...

Field	Type	Required	Description
	line_works, viber)		
inbox.unreadConversationsCount	integer	No	
inbox.signAuth	object (with 8 properties: id, signSource, signParams...)	No	
inbox.signAuth.signSource	string (uuid)	Yes	
inbox.signAuth.signParams	object	Yes	
inbox.signAuth.signParams.keycloak	object (with 3 properties: clientId, url, realm)	Yes	
inbox.signAuth.signParams.line	object (with 1 property: liffId)	Yes	
inbox.signAuth.signParams.ad	object (with 1 property: saml)	Yes	
inbox.signAuth.signParams.google	object (with 1 property: enabled)	No	
inbox.signAuth.sourceIdAccessEnabled	boolean	No	When enabled, embedded visitors presenting a Source ID (verified per the verify mode below) may chat without going through the login flow. Disabled keeps every existing behavior unchanged.
inbox.signAuth.sourceIdVerifyMode	object	No	"Require signature verification": the embedding site must send an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act as that contact; suitable only for testin...
inbox.signAuth.sourceIdSignatureTtlMinutes	object	No	How long a signature can be used to obtain the contact identity. Not a session lifetime: once exchanged, the session follows the existing WebChat mechanism. 1 : 1 ; 5 : 5 ; 15 : 15;

Field	Type	Required	Description
inbox.signAuth.keycloakTokenPassthrough	boolean	No	When enabled, the Keycloak access token a visitor logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on their behalf, so tools do not require a separate per-to...
inbox.signAuth.keycloakTokenPassthroughAllowedHosts	array[string]	No	Hostnames (or "*.example.com" wildcards) of the tools the visitor Keycloak token may be sent to
inbox.enableAnalysis	boolean	No	
lastMessage	object	No	
lastMessage.type	string	No	
lastMessage.content	string	No	
enableGroupMention	boolean	No	
queryMetadata	object	No	Query metadata that controls the searchable knowledge scope; see each endpoint description for details
queryMetadata.knowledgeBases	array[object]	No	List of searchable knowledge bases; an empty array or null means none are searchable (no permission), while omission means unrestricted
queryMetadata.labelRelations	object	No	Label filtering conditions; an empty conditions array means no label filtering (not no permission). This must be provided together with knowledgeBases—if provided alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
queryMetadata.labelRelations.operator	string (enum: AND, OR)	Yes	How label conditions are combined
queryMetadata.labelRelations.conditions	array[any]	No	List of label conditions; supports nested operator/conditions combinations
toolMask	object	No	Enabled/disabled state per tool: {tool_id: bool}

Field	Type	Required	Description
connectorMask	object	No	Enabled/disabled state per connector: {connector_id: bool}
thinkingConfig	object	No	None = unset (inherit the chatbot setting), {} = explicitly off, {...} = custom override
status	string (enum: open, resolved, queued)	No	<code>open</code> : Open ; <code>resolved</code> : Resolved ; <code>queued</code> : Queued;
firstResponseAt	string (timestamp)	No	
queuedAt	string (timestamp)	No	
assignedAt	string (timestamp)	No	
transferReason	string	No	

Request Schema Example

```
{
  "contact"?: { // Lightweight Contact Serializer for the Conversation List view

    Includes only the fields required by the List view and removes inboxes, mcp_credentials, and
    api_credentials
    to avoid N+1 query issues. email / phone_number / metadata are scalar fields on Contact itself
    that the frontend conversation sidebar needs to display, with no N+1 cost. (optional)
    {
      "name": string
      "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
      as-is. (optional)
      "email"?: // May have different types (optional)
      string (email) // optional
      "phoneNumber"?: string // optional
      "metadata"?: object // Custom customer information shown in the customer panel of the
      conversation page (optional)
      "sourceId"?: string // optional
    }
  }
  "inbox"?: { // Inbox summary embedded in a conversation payload.

    ``unread_conversations_count`` stays here because it is part of the
    documented REST conversation API. It is an org-wide denormalized
    counter, so WS emit paths must run the payload through
    :func:`strip_org_wide_unread_count` before broadcasting it to rooms
    that include members without ``GroupInbox.can_view_conversations``. (optional)
    {
      "name": string
```

```

    "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
    whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
    "unreadConversationsCount"?: integer // optional
    "signAuth"?: // optional
    {
        "signSource": string (uuid)
        "signParams": {
            {
                "keycloak":
                {
                    "clientId": string
                    "url": string
                    "realm": string
                }
                "line":
                {
                    "liffId": string
                }
                "ad":
                {
                    "saml": string
                }
                "google"?: // optional
                {
                    "enabled": boolean
                }
            }
        }
        "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source
        ID (verified per the verify mode below) may chat without going through the login flow. Disabled
        keeps every existing behavior unchanged. (optional)
        "sourceIdVerifyMode"?: // "Require signature verification": the embedding site must send
        an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act
        as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (optional)
    {
    }
    "sourceIdSignatureTtlMinutes"?: // How long a signature can be used to obtain the
    contact identity. Not a session lifetime: once exchanged, the session follows the existing
    WebChat mechanism.

* `1` - 1
* `5` - 5

```

```

* `15` - 15 (optional)
{
}
"keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor
logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on
their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign
source. Never overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the conversation at
login, never the contact Source ID. (optional)
"keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (optional)
  string
]
}
"enableAnalysis"?: boolean // optional
}
}
"lastMessage"?: { // optional
{
  "type"?: string // optional
  "content"?: string // optional
}
}
"enableGroupMention"?: boolean // optional
"queryMetadata"?: { // Query metadata that controls the searchable knowledge scope; see each
endpoint description for details (optional)
{
  "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means
none are searchable (no permission), while omission means unrestricted (optional)
  {
    "knowledgeBaseId": string (uuid) // Knowledge base ID
    "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (optional)
      string (uuid)
    ]
    "knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; this
field takes precedence when both are provided) (optional)
      string (uuid)
    ]
    "faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (optional)
      string (uuid)
    ]
    "hasUserSelectedAll"?: boolean // true = select all content in the knowledge base
except the listed items; false = select only the listed items (optional)
  }
}
}
"labelRelations"?: { // Label filtering conditions; an empty conditions array means no
label filtering (not no permission). This must be provided together with knowledgeBases—if
provided alone, label filtering is not applied to knowledge base retrieval (equivalent to
unrestricted) (optional)
{
  "operator": string (enum: AND, OR) // How label conditions are combined
  "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (optional)
    object

```

```

    ]
  }
}
}
"toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (optional)
"connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(optional)
"thinkingConfig"?: object // None = unset (inherit the chatbot setting), {} = explicitly off,
{...} = custom override (optional)
"status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (optional)
"firstResponseAt"?: string (timestamp) // optional
"queuedAt"?: string (timestamp) // optional
"assignedAt"?: string (timestamp) // optional
"transferReason"?: string // optional
}

```

Example Request Value

```

{
  "contact": {
    "name": "Example Name",
    "avatar": "example string",
    "email": null,
    "phoneNumber": "example string",
    "metadata": null,
    "sourceId": "example string"
  },
  "inbox": {
    "name": "Example Name",
    "channelType": "line",
    "unreadConversationsCount": 123,
    "signAuth": {
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "example string",
          "url": "example string",
          "realm": "example string"
        },
        "line": {
          "liffId": "example string"
        },
        "ad": {
          "saml": "example string"
        },
        "google": {
          "enabled": true
        }
      },
      "sourceIdAccessEnabled": true,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {}
    }
  }
}

```

```

    "keycloakTokenPassthrough": true,
    "keycloakTokenPassthroughAllowedHosts": [
      "example string"
    ]
  },
  "enableAnalysis": true
},
"lastMessage": {
  "type": "example string",
  "content": "Hello! I would like to learn about the product."
},
"enableGroupMention": true,
"queryMetadata": {
  "knowledgeBases": [
    {
      "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
      "chatbotFileIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "knowledgeBaseFileIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "faqIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ],
      "hasUserSelectedAll": true
    }
  ],
  "labelRelations": {
    "operator": "AND",
    "conditions": [
      null
    ]
  }
},
"toolMask": null,
"connectorMask": null,
"thinkingConfig": null,
"status": "open",
"firstResponseAt": "example string",
"queuedAt": "example string",
"assignedAt": "example string",
"transferReason": "example string"
}

```

Code Examples

```
# Call API example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/rename/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "contact": {
    "name": "Example Name",
    "avatar": "example string",
    "email": null,
    "phoneNumber": "example string",
    "metadata": null,
    "sourceId": "example string"
  },
  "inbox": {
    "name": "Example Name",
    "channelType": "line",
    "unreadConversationsCount": 123,
    "signAuth": {
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "example string",
          "url": "example string",
          "realm": "example string"
        },
        "line": {
          "liffId": "example string"
        },
        "ad": {
          "saml": "example string"
        },
        "google": {
          "enabled": true
        }
      },
      "sourceIdAccessEnabled": true,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": true,
      "keycloakTokenPassthroughAllowedHosts": [
        "example string"
      ]
    },
    "enableAnalysis": true
  },
  "lastMessage": {
    "type": "example string",
```

```
    "content": "Hello! I would like to learn about the product."
  },
  "enableGroupMention": true,
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
        "chatbotFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "knowledgeBaseFileIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "faqIds": [
          "550e8400-e29b-41d4-a716-446655440000"
        ],
        "hasUserSelectedAll": true
      }
    ],
    "labelRelations": {
      "operator": "AND",
      "conditions": [
        null
      ]
    }
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "status": "open",
  "firstResponseAt": "example string",
  "queuedAt": "example string",
  "assignedAt": "example string",
  "transferReason": "example string"
}'
```

Before running, replace YOUR_API_KEY and verify the request data.

```

const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "contact": {
    "name": "Example name",
    "avatar": "Example string",
    "email": null,
    "phoneNumber": "Example string",
    "metadata": null,
    "sourceId": "Example string"
  },
  "inbox": {
    "name": "Example name",
    "channelType": "line",
    "unreadConversationsCount": 123,
    "signAuth": {
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Example string",
          "url": "Example string",
          "realm": "Example string"
        },
        "line": {
          "liffId": "Example string"
        },
        "ad": {
          "saml": "Example string"
        },
        "google": {
          "enabled": true
        }
      },
      "sourceIdAccessEnabled": true,
      "sourceIdVerifyMode": {},
      "sourceIdSignatureTtlMinutes": {},
      "keycloakTokenPassthrough": true,
      "keycloakTokenPassthroughAllowedHosts": [
        "Example string"
      ]
    },
    "enableAnalysis": true
  },
};

```



```

    "lastMessage": {
      "type": "Example string",
      "content": "Hello! I'd like to learn more about the product."
    },
    "enableGroupMention": true,
    "queryMetadata": {
      "knowledgeBases": [
        {
          "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
          "chatbotFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "knowledgeBaseFileIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "faqIds": [
            "550e8400-e29b-41d4-a716-446655440000"
          ],
          "hasUserSelectedAll": true
        }
      ],
      "labelRelations": {
        "operator": "AND",
        "conditions": [
          null
        ]
      }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
  };

```

```

axios.patch("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/rename/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });

```

```

import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/rename/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "contact": {
        "name": "Example name",
        "avatar": "Example string",
        "email": null,
        "phoneNumber": "Example string",
        "metadata": null,
        "sourceId": "Example string"
    },
    "inbox": {
        "name": "Example name",
        "channelType": "line",
        "unreadConversationsCount": 123,
        "signAuth": {
            "signSource": "550e8400-e29b-41d4-a716-446655440000",
            "signParams": {
                "keycloak": {
                    "clientId": "Example string",
                    "url": "Example string",
                    "realm": "Example string"
                },
                "line": {
                    "liffId": "Example string"
                },
                "ad": {
                    "saml": "Example string"
                },
                "google": {
                    "enabled": true
                }
            },
            "sourceIdAccessEnabled": true,
            "sourceIdVerifyMode": {},
            "sourceIdSignatureTtlMinutes": {},
            "keycloakTokenPassthrough": true,
            "keycloakTokenPassthroughAllowedHosts": [
                "Example string"
            ]
        },
        "enableAnalysis": true
    },
}

```

```

    "lastMessage": {
        "type": "Example string",
        "content": "Hello! I'd like to learn more about the product."
    },
    "enableGroupMention": true,
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                "chatbotFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "knowledgeBaseFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "faqIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "hasUserSelectedAll": true
            }
        ],
        "labelRelations": {
            "operator": "AND",
            "conditions": [
                null
            ]
        }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
}

```

```

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)

```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/conversations/550e8400-
e29b-41d4-a716-446655440000/rename/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "contact": {
                "name": "Example name",
                "avatar": "Example string",
                "email": null,
                "phoneNumber": "Example string",
                "metadata": null,
                "sourceId": "Example string"
            },
            "inbox": {
                "name": "Example name",
                "channelType": "line",
                "unreadConversationsCount": 123,
                "signAuth": {
                    "signSource": "550e8400-e29b-41d4-a716-446655440000",
                    "signParams": {
                        "keycloak": {
                            "clientId": "Example string",
                            "url": "Example string",
                            "realm": "Example string"
                        },
                        "line": {
                            "liffId": "Example string"
                        },
                        "ad": {
                            "saml": "Example string"
                        },
                        "google": {
                            "enabled": true
                        }
                    },
                    "sourceIdAccessEnabled": true,
                    "sourceIdVerifyMode": {},
                    "sourceIdSignatureTtlMinutes": {},
                    "keycloakTokenPassthrough": true,
                    "keycloakTokenPassthroughAllowedHosts": [
                        "Example string"
                    ]
                }
            },
        ]
    ]
}

```

```

        "enableAnalysis": true
    },
    "lastMessage": {
        "type": "Example string",
        "content": "Hello! I'd like to learn more about the product."
    },
    "enableGroupMention": true,
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "550e8400-e29b-41d4-a716-446655440000",
                "chatbotFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "knowledgeBaseFileIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "faqIds": [
                    "550e8400-e29b-41d4-a716-446655440000"
                ],
                "hasUserSelectedAll": true
            }
        ],
        "labelRelations": {
            "operator": "AND",
            "conditions": [
                null
            ]
        }
    },
    "toolMask": null,
    "connectorMask": null,
    "thinkingConfig": null,
    "status": "open",
    "firstResponseAt": "Example string",
    "queuedAt": "Example string",
    "assignedAt": "Example string",
    "transferReason": "Example string"
}
]);

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>

```

Response

Status code: 200

Example response schema

```
{
  "id": string (uuid)
  "contact": { // Lightweight Contact Serializer used for the Conversation List view

Includes only the fields required by the List view and removes inboxes, mcp_credentials, and
api_credentials
to avoid N+1 query issues. email / phone_number / metadata are scalar fields on the Contact
itself,
which the frontend conversation sidebar needs to display, with no N+1 cost.

    {
      "id": string (uuid)
      "name": string
      "avatar"?: string // Stores our own media as a relative storage key, or an external CDN URL
as-is. (Optional)
      "email"?: // May have different types (Optional)
string (email) // Optional
      "phoneNumber"?: string // Optional
      "metadata"?: object // Custom customer information shown in the customer panel of the
conversation page (Optional)
      "sourceId"?: string // Optional
      "tags": [
        {
          "id": string (uuid)
          "name": string
        }
      ]
    }
  }
  "inbox": { // Inbox summary embedded in a conversation payload.

``unread_conversations_count`` stays here because it is part of the
documented REST conversation API. It is an org-wide denormalized
counter, so WS emit paths must run the payload through
:func:`strip_org_wide_unread_count` before broadcasting it to rooms
that include members without ``GroupInbox.can_view_conversations``.

    {
      "id": string (uuid)
      "name": string
      "channelType": string (enum: line, telegram, teams, web, messenger, instagram, email,
whatsapp, slack, team_plus, line_works, viber) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
```

```

* `slack` - Slack
* `team_plus` - Team+
* `line_works` - LINE WORKS
* `viber` - Viber
  "unreadConversationsCount"?: integer // Optional
  "signAuth"?: // Optional
  {
    "id": string (uuid)
    "signSource": string (uuid)
    "signParams": {
      {
        "keycloak":
          {
            "clientId": string
            "url": string
            "realm": string
          }
        "line":
          {
            "liffId": string
          }
        "ad":
          {
            "saml": string
          }
        "google"?: // Optional
          {
            "enabled": boolean
          }
      }
    }
    "sourceIdAccessEnabled"?: boolean // When enabled, embedded visitors presenting a Source
ID (verified per the verify mode below) may chat without going through the login flow. Disabled
keeps every existing behavior unchanged. (Optional)
    "sourceIdVerifyMode"?: // "Require signature verification": the embedding site must send
an HMAC signature computed on its backend. "Source ID only": anyone knowing a Source ID can act
as that contact; suitable only for testing or closed networks.

* `signature` - Require signature verification
* `raw` - Source ID only (Optional)
  {
  }
  "sourceIdSignatureTtlMinutes"?: // How long a signature can be used to obtain the
contact identity. Not a session lifetime: once exchanged, the session follows the existing
WebChat mechanism.

* `1` - 1
* `5` - 5
* `15` - 15 (Optional)
  {
  }
  "keycloakTokenPassthrough"?: boolean // When enabled, the Keycloak access token a visitor
logged in with is forwarded as the Authorization header when the AI calls MCP and API tools on
their behalf, so tools do not require a separate per-tool login. Only for the Keycloak sign
source. Never overrides an Authorization header a tool already resolves from its own
credentials. The visitor identity is the organization member bound to the conversation at

```

```

login, never the contact Source ID. (Optional)
    "keycloakTokenPassthroughAllowedHosts"?: [ // Hostnames (or "*.example.com" wildcards) of
the tools the visitor Keycloak token may be sent to (Optional)
        string
    ]
}
"enableAnalysis"?: boolean // Optional
}
}

"chatbot": object // Chatbot (Agent) summary from the conversation's inbox, already eager-
loaded.

``is_maigpt`` lets the client tell the default MaiGPT agent apart (icon, feature
gate) without comparing against the organization's ``maigpt_inbox`` id.
"title": string
"lastMessage"?: { // Optional
{
    "id": string (uuid)
    "type"?: string // Optional
    "content"?: string // Optional
    "createdAt": string (timestamp)
}
}
"callRecord":
{
    "id": string (uuid)
    "status"?: string (enum: initiated, ringing, in-progress, completed, failed, busy, no-
answer, canceled) // * `initiated` - Initiated
* `ringing` - Ringing
* `in-progress` - In Progress
* `completed` - Completed
* `failed` - Failed
* `busy` - Busy
* `no-answer` - No Answer
* `canceled` - Canceled (Optional)
    "direction": string (enum: inbound, outbound) // * `inbound` - Inbound
* `outbound` - Outbound
    "isActive": boolean
    "canHangup": boolean
}
"lastMessageCreatedAt": string (timestamp)
"unreadMessagesCount": integer
"autoReplyEnabled": boolean
"isAutoReplyNow": boolean
"lastReadAt": string (timestamp)
"createdAt": string (timestamp)
"isGroupChat": boolean
"enableGroupMention"?: boolean // Optional
"queryMetadata"?: { // Query metadata that controls the searchable knowledge scope; see each
endpoint's description for details (Optional)
{
    "knowledgeBases"?: [ // List of searchable knowledge bases; an empty array or null means
none are searchable (no permission), while omission means unrestricted (Optional)
        {
            "knowledgeBaseId": string (uuid) // Knowledge base ID
            "chatbotFileIds"?: [ // List of file IDs (excluded or exclusively selected depending on

```



```

hasUserSelectedAll) (Optional)
    string (uuid)
]
"knowledgeBaseFileIds"?: [ // List of file IDs (the new name for chatbotFileIds; this
field takes precedence when both are provided) (Optional)
    string (uuid)
]
"faqIds"?: [ // List of FAQ IDs (excluded or exclusively selected depending on
hasUserSelectedAll) (Optional)
    string (uuid)
]
"hasUserSelectedAll"?: boolean // true = select all content in the knowledge base
except the listed items; false = select only the listed items (Optional)
}
]
"labelRelations"?: { // Label filtering criteria; an empty conditions array means no label
filtering (not no permission). Must be provided together with knowledgeBases—when provided
alone, label filtering is not applied to knowledge base retrieval (equivalent to unrestricted)
(Optional)
    {
        "operator": string (enum: AND, OR) // How label conditions are combined
        "conditions"?: [ // List of label conditions; supports nested operator/conditions
combinations (Optional)
            object
        ]
    }
}
}
}
"toolMask"?: object // Enabled/disabled state per tool: {tool_id: bool} (Optional)
"connectorMask"?: object // Enabled/disabled state per connector: {connector_id: bool}
(Optional)
"thinkingConfig"?: object // None = unset (inherit the chatbot setting), {} = explicitly off,
{...} = custom override (Optional)
"thinkingConfigSchema": object
"effectiveThinkingConfig": object // Return the effective thinking config after applying the
priority chain:
conversation override > chatbot config > LLM metadata.

```

This allows the frontend to display the actual thinking config in use, even when the conversation has no explicit override.

Returns None when nothing is configured at any level, so the frontend can distinguish "Default" (None = use model/assistant default) from "Off" ({} = explicitly disabled). Collapsing the unconfigured case to {} would make a fresh conversation display as Off instead of Default.

```

"llm": string (uuid)
"deepResearchStatus": // Status of deep research for this conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed
{
}

```

"deepResearchOutputFormat": // May have different types
string (enum: canvas, chat, file) // Chosen delivery form for the finished report. Set only when the user clicks a format in the pre-research choice card; reset to NULL each time Deep Research is (re-)armed. NULL means no explicit choice (or a plan-skipping request) and falls back to Canvas at render time, but stays distinct from an explicit canvas pick so the frontend can lock the card only after a real selection. To roll back to fixed-Canvas behavior, force this to NULL/canvas – no redeploy needed.

```
* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file
"ipAddress": string
"ipCountryCode": string
"ipRecordedAt": string (timestamp)
"assignee":
{
  "id": string (uuid)
  "name": string
  "userId": string (uuid)
}
"status"? : string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
"tags": [
  {
    "id": string (uuid)
    "name": string
    "memberIds"? : [ // Optional
      string (uuid)
    ]
  }
]
"progressStatus": string
"firstResponseAt"? : string (timestamp) // Optional
"queuedAt"? : string (timestamp) // Optional
"assignedAt"? : string (timestamp) // Optional
"transferReason"? : string // Optional
"satisfactionScore": integer // Conversation satisfaction rating on a 5-point scale (1=very dissatisfied, 5=very satisfied)
"satisfactionComment": string
"satisfactionSubmittedAt": string (timestamp)
"latestAnalysisSummary": string // The conversation's current analysis summary (its active summary revision).
```

Reads the `latest\_analysis\_summary` annotation added by `setup\_eager\_loading` to avoid an N+1 query. All read paths (list / retrieve / create) eager-load, so the annotation is present; it falls back to '' only for instances built outside `setup\_eager\_loading`.

"clientPlatform": // Client platform that created this conversation (web, desktop)

```
* `web` - Web
* `desktop` - Desktop
{
}
"workingDirectory": string // Absolute path of the local folder bound to a desktop conversation. Only set for desktop conversations; null for web conversations.
```

```

"slideProjectId": string
"matchedField": string // Cached per instance: ``get_matched_message_id`` /
``get_matched_snippet``
both read this, so resolving it once avoids recomputing the keyword and
title-match logic three times per serialized conversation.
"matchedSnippet": string // Keyword-centred excerpt of the matched message, with ellipses.

Reads the ``matched_message_content`` annotation added by
``ConversationViewSet._annotate_keyword_match``; no extra query.
"matchedMessageId": string
"pinnedAt": string (timestamp) // When the conversation was pinned. Null means unpinned.
"folder": string (uuid) // The user-defined sidebar folder this conversation is grouped into.
Null means unclassified (shown under the time groups). Deleting the folder resets this to null
without deleting the conversation.
"hasActiveBrowserSession": boolean // Whether a browser_use session snapshot exists to
resume.

Lets the frontend know it should proactively re-request the browser
panel (emit ``browser:start``) after a page reload, instead of only
ever showing the panel when a live ``browser:frame`` push happens to
arrive first.
"callerOrganizationId": string // Bound caller organization (stamped at conversation start);
null when unbound
"handoff": object // Handoff relationship: the continuing conversation contains
source_conversation_id/source_title; the original handed-off conversation contains
continued_to_conversation_id/continued_to_title; null for conversations without a handoff
}

```

Example response value

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "avatar": "Response string",
    "email": "response@example.com",
    "phoneNumber": "Response string",
    "metadata": null,
    "sourceId": "Response string",
    "tags": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      }
    ]
  },
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "channelType": "line",
    "unreadConversationsCount": 456,
    "signAuth": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "signSource": "550e8400-e29b-41d4-a716-446655440000",

```

```
"signParams": {
  "keycloak": {
    "clientId": "Response string",
    "url": "Response string",
    "realm": "Response string"
  },
  "line": {
    "liffId": "Response string"
  },
  "ad": {
    "saml": "Response string"
  },
  "google": {
    "enabled": false
  }
},
"sourceIdAccessEnabled": false,
"sourceIdVerifyMode": {},
"sourceIdSignatureTtlMinutes": {},
"keycloakTokenPassthrough": false,
"keycloakTokenPassthroughAllowedHosts": [
  "Response string"
],
"enableAnalysis": false
},
"chatbot": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"title": "Response string",
"lastMessage": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "Response string",
  "content": "Response string",
  "createdAt": "Response string"
},
"callRecord": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "initiated",
  "direction": "inbound",
  "isActive": false,
  "canHangup": false
},
"lastMessageCreatedAt": "Response string",
"unreadMessagesCount": 456,
"autoReplyEnabled": false,
"isAutoReplyNow": false,
"lastReadAt": "Response string",
"createdAt": "Response string",
"isGroupChat": false,
"enableGroupMention": false,
"queryMetadata": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
```

```
    "description": "Example response description"
  },
  "toolMask": null,
  "connectorMask": null,
  "thinkingConfig": null,
  "thinkingConfigSchema": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "effectiveThinkingConfig": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
  },
  "llm": "550e8400-e29b-41d4-a716-446655440000",
  "deepResearchStatus": {},
  "deepResearchOutputFormat": "canvas",
  "ipAddress": "Response string",
  "ipCountryCode": "Response string",
  "ipRecordedAt": "Response string",
  "assignee": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "userId": "550e8400-e29b-41d4-a716-446655440000"
  },
  "status": "open",
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
      ]
    }
  ],
  "progressStatus": "Response string",
  "firstResponseAt": "Response string",
  "queuedAt": "Response string",
  "assignedAt": "Response string",
  "transferReason": "Response string",
  "satisfactionScore": 456,
  "satisfactionComment": "Response string",
  "satisfactionSubmittedAt": "Response string",
  "latestAnalysisSummary": "Response string",
  "clientPlatform": {},
  "workingDirectory": "Response string",
  "slideProjectId": "Response string",
  "matchedField": "Response string",
  "matchedSnippet": "Response string",
  "matchedMessageId": "Response string",
  "pinnedAt": "Response string",
  "folder": "550e8400-e29b-41d4-a716-446655440000",
  "hasActiveBrowserSession": false,
  "callerOrganizationId": "Response string",
  "handoff": {
```

```
"id": "550e8400-e29b-41d4-a716-446655440000",
"name": "Example response name",
"description": "Example response description"
}
}
```

Share Conversation

POST

/api/v1/conversations/{conversationPk}/share/

Parameters

Parameter Name	Required	Type	Description
conversationPk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
shareScope	string (enum: organization, group, member)	Yes	<code>organization</code> : Organization ; <code>group</code> : Group ; <code>member</code> : Member;
permission	object	No	
title	string	No	
description	string	No	
groupIds	array[string]	No	
memberIds	array[string]	No	

Request Schema Example

```
{
  "shareScope": string (enum: organization, group, member) // * `organization` - Organization
  * `group` - Group
  * `member` - Member
  "permission"? : // Optional
  {
  }
```

```
"title"?: string // Optional
"description"?: string // Optional
"groupIds"?: [ // Optional
  string (uuid)
]
"memberIds"?: [ // Optional
  string (uuid)
]
}
```

Request Example Value

```
{
  "shareScope": "organization",
  "permission": {},
  "title": "Example name",
  "description": "Example string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

```
# Call API Example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "shareScope": "organization",
  "permission": {},
  "title": "Example name",
  "description": "Example string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "shareScope": "organization",
  "permission": {},
  "title": "Example name",
  "description": "Example string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "shareScope": "organization",
    "permission": {},
    "title": "Example name",
    "description": "Example string",
    "groupIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "shareScope": "organization",
            "permission": {},
            "title": "Example name",
            "description": "Example string",
            "groupIds": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "memberIds": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response Body

Status Code: 201

Response Schema Example

```

{
  "shareScope": string (enum: organization, group, member) // * `organization` - Organization

```

```
* `group` - Group
* `member` - Member
"permission"?: // Optional
{
}
"title"?: string // Optional
"description"?: string // Optional
"groupIds"?: [ // Optional
  string (uuid)
]
"memberIds"?: [ // Optional
  string (uuid)
]
}
```

Response Example Value

```
{
  "shareScope": "organization",
  "permission": {},
  "title": "Response string",
  "description": "Response string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Quick Share Conversation

POST </api/v1/conversations/{conversationPk}/share/quick/>

Parameters

Parameter Name	Required	Type	Description
<code>conversationPk</code>	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
shareScope	string (enum: organization, group, member)	Yes	<code>organization</code> : Organization ; <code>group</code> : Group ; <code>member</code> : Member;
permission	object	No	
title	string	No	
description	string	No	
groupIds	array[string]	No	
memberIds	array[string]	No	

Request Schema Example

```
{
  "shareScope": string (enum: organization, group, member) // * `organization` - Organization
  * `group` - Group
  * `member` - Member
  "permission"? : // Optional
  {
  }
  "title"? : string // Optional
  "description"? : string // Optional
  "groupIds"? : [ // Optional
    string (uuid)
  ]
  "memberIds"? : [ // Optional
    string (uuid)
  ]
}
```

Request Example Value

```
{
  "shareScope": "organization",
  "permission": {},
  "title": "Example name",
  "description": "Example string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/quick/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "shareScope": "organization",
  "permission": {},
  "title": "Example name",
  "description": "Example string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'

# Replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "shareScope": "organization",
  "permission": {},
  "title": "Example name",
  "description": "Example string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/quick/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/quick/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "shareScope": "organization",
    "permission": {},
    "title": "Example name",
    "description": "Example string",
    "groupIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "memberIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```



```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/conversations/{conversationPk}/share/quick/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "shareScope": "organization",
            "permission": {},
            "title": "Example name",
            "description": "Example string",
            "groupIds": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "memberIds": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response Body

Status Code: 200

Response Schema Example

```

{
  "shareScope": string (enum: organization, group, member) // * `organization` - Organization

```

```
* `group` - Group
* `member` - Member
"permission"?: // Optional
{
}
"title"?: string // Optional
"description"?: string // Optional
"groupIds"?: [ // Optional
  string (uuid)
]
"memberIds"?: [ // Optional
  string (uuid)
]
}
```

Response Example Value

```
{
  "shareScope": "organization",
  "permission": {},
  "title": "Response string",
  "description": "Response string",
  "groupIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "memberIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

List Shared Conversations

GET

</api/v1/shared-conversations/>

Parameters

Parameter Name	Required	Type	Description
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/shared-conversations/?page=1&pageSize=1"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/shared-conversations/?page=1&pageSize=1",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/?page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/shared-conversations/?
page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response Body

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "title": string // Custom title for the shared conversation
      "description": string // Custom description for the shared conversation
      "shareScope":
        {
        }
      "permission":
        {
        }
      "sharedBy":
        {
          "id": string (uuid)
          "name": string
        }
      "messageCount": integer
      "createdAt": string (timestamp)
    }
  ]
}
```

Response Example Value

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "title": "Response string",
      "description": "Response string",
      "shareScope": {},
      "permission": {},
      "sharedBy": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "messageCount": 456,
      "createdAt": "Response string"
    }
  ]
}
```

```
]
}
```

Get a Specific Shared Conversation

GET

`/api/v1/shared-conversations/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Shared Conversation.

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/shared-
conversations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response Body

Status Code: 200

Response Schema Example

```

{
  "id": string (uuid)
  "title": string // Custom title for the shared conversation
  "description": string // Custom description for the shared conversation
  "permission":
  {
  }
  "sharedBy":
  {
    "id": string (uuid)
    "name": string
  }
  "createdAt": string (timestamp)
}

```



```
"conversationSnapshot": object // Frozen conversation snapshot: {title, messages[]}
}
```

Response Example Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "title": "Response string",
  "description": "Response string",
  "permission": {},
  "sharedBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "createdAt": "Response string",
  "conversationSnapshot": null
}
```

Update Shared Conversation

PATCH `/api/v1/shared-conversations/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Shared Conversation.

Request Body

Request Parameters

Field	Type	Required	Description
title	string	No	
description	string	No	
permission	string (enum: readonly, copy)	No	<code>readonly</code> : Read Only ; <code>copy</code> : Copyable;

Request Schema Example

```
{
  "title?": string // Optional
  "description?": string // Optional
  "permission?": string (enum: readonly, copy) // * `readonly` - Read Only
  * `copy` - Copyable (Optional)
}
```

Request Example Value

```
{
  "title": "Example Name",
  "description": "example string",
  "permission": "readonly"
}
```

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "title": "Example Name",
  "description": "example string",
  "permission": "readonly"
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
  "title": "Example Name",
  "description": "example string",
  "permission": "readonly"
};

axios.patch("https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "title": "Example Name",
    "description": "example string",
    "permission": "readonly"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/shared-
conversations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "title": "Example Name",
            "description": "example string",
            "permission": "readonly"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "title"?: string // Optional
  "description"?: string // Optional
  "permission"?: string (enum: readonly, copy) // * `readonly` - Read Only
  * `copy` - Copyable (Optional)
}
```

Response Example Value

```
{
  "title": "response string",
  "description": "response string",
  "permission": "readonly"
}
```

Delete Shared Conversation

DELETE`/api/v1/shared-conversations/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Shared Conversation.

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/shared-
conversations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Copy Shared Conversation

POST

</api/v1/shared-conversations/{id}/fork/>

Parameters

Parameter Name	Required	Type	Description
----------------	----------	------	-------------

id	V	string	A UUID string identifying this Shared Conversation.
----	---	--------	---

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/fork/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{}'
```

Before running, replace YOUR_API_KEY and verify the request data.

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {};

axios.post("https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/fork/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/550e8400-e29b-41d4-a716-446655440000/fork/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/shared-
conversations/550e8400-e29b-41d4-a716-446655440000/fork/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {}
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "title": string // Custom title for the shared conversation
  "description": string // Custom description for the shared conversation
  "permission":
  {
  }
  "sharedBy":
  {
    "id": string (uuid)
    "name": string
  }
  "createdAt": string (timestamp)
```

```
"conversationSnapshot": object // Frozen conversation snapshot: {title, messages[]}
}
```

Response Example Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "title": "response string",
  "description": "response string",
  "permission": {},
  "sharedBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "createdAt": "response string",
  "conversationSnapshot": null
}
```

Get My Shared Conversations

GET

/api/v1/shared-conversations/my-shares/

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/shared-conversations/my-shares/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/shared-conversations/my-shares/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/shared-conversations/my-shares/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/shared-conversations/my-shares/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}

?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "title": string // Custom title for the shared conversation
  "description": string // Custom description for the shared conversation
  "shareScope":
  {
  }
  "permission":
  {
  }
  "sharedBy":
  {
    "id": string (uuid)
    "name": string
  }
}
```

```
"messageCount": integer
"createdAt": string (timestamp)
}
```

Response Example Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "title": "response string",
  "description": "response string",
  "shareScope": {},
  "permission": {},
  "sharedBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "messageCount": 456,
  "createdAt": "response string"
}
```

Create Message Feedback

POST

/api/v1/messages/{messagePk}/feedback/

Parameters

Parameter Name	Required	Type	Description
messagePk	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
type	object	Yes	
suggestion	string	No	

Request Schema Example


```
{
  "type":
  {
  }
  "suggestion"?: string // Optional
}
```

Request Example Value

```
{
  "type": {},
  "suggestion": "example string"
}
```

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "type": {},
  "suggestion": "example string"
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
  "type": {},
  "suggestion": "example string"
};

axios.post("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "type": {},
    "suggestion": "example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/", [
    'headers' => [
        'Authorization' => 'Api-Key YOUR_API_KEY',
        'Content-Type' => 'application/json'
    ],
    'json' => {
        "type": {},
        "suggestion": "example string"
    }
]);

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "type":
  {
  }
  "suggestion"?: string // Optional
  "updatedAt": string (timestamp)
}
```

Response Example Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": {},
  "suggestion": "response string",
  "updatedAt": "response string"
}
```

Status Code: **400** - This message already has feedback. Use the PATCH method to update it.

Update Message Feedback

PUT `/api/v1/messages/{messagePk}/feedback/{id}`

Parameters

Parameter Name	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Message Feedback.
<code>messagePk</code>	V	string	

Request Body

Request Parameters

Field	Type	Required	Description
type	object	Yes	
suggestion	string	No	

Request Schema Example

```
{
  "type":
  {
  }
```

```
"suggestion"?: string // Optional
}
```

Request Example Value

```
{
  "type": {},
  "suggestion": "example string"
}
```

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "type": {},
  "suggestion": "example string"
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request Body (payload)
const data = {
  "type": {},
  "suggestion": "example string"
};

axios.put("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request Body (payload)
data = {
    "type": {},
    "suggestion": "example string"
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```



```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "type": {},
            "suggestion": "example string"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```

{
  "id": string (uuid)
  "type":
  {
  }
  "suggestion?": string // Optional
  "updatedAt": string (timestamp)
}

```

Response Example Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": {},
  "suggestion": "response string",
  "updatedAt": "response string"
}
```

Delete Message Feedback

DELETE

/api/v1/messages/{messagePk}/feedback/{id}

Parameters

Parameter Name	Required	Type	Description
id	V	string	A UUID string identifying this Message Feedback.
messagePk	V	string	

Code Examples

Shell/Bash

```
# Call API Example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request Body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/messages/{messagePk}/feedback/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	Feedback deleted successfully
400	This feedback does not belong to the specified message

List Conversation Records

GET </api/v1/records/>

Parameters

Parameter	Required	Type	Description
callerMember	X	array	Multiple values may be separated by commas.
chatbot	X	string	Filter by a specific AI assistant (UUID)
endDate	X	string	End date (format: YYYY-MM-DD, e.g., 2025-01-31)
endDatetime	X	string	
errorType	X	array	Multiple values may be separated by commas. `system`: System; `llm`: LLM; `embedding`: Embedding; `reranker`: Reranker; `vector_db`: Vector DB; `workflow`: Workflow; `tool`: Tool; `timeout`: Ti...
feedbackType	X	array	Multiple values may be separated by commas. `like`: Like; `dislike`: Dislike;
hasError	X	boolean	
hasFeedback	X	boolean	
keyword	X	string	Keyword search (searches both user messages and chatbot responses)
largeLanguageModel	X	string	Filter by a specific large language model (UUID)
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
source	X	string	Filter record sources, separated by commas. If omitted, both sources are returned. interactive = user conversation; scheduled = scheduled execution.
startDate	X	string	Start date (format: YYYY-MM-DD, e.g., 2025-01-01)
startDatetime	X	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/records/?
callerMember=example&chatbot=example&endDate=example&endDatetime=2025-01-
01T00:00:00.000Z&errorType=example&feedbackType=example&hasError=true&hasFeedback=true&ke
yword=example&largeLanguageModel=example&page=1&pageSize=1&source=interactive&startDate=e
xample&startDatetime=2025-01-01T00:00:00.000Z"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/records/?
callerMember=example&chatbot=example&endDate=example&endDatetime=2025-01-
01T00:00:00.000Z&errorType=example&feedbackType=example&hasError=true&hasFeedback=true&ke
yword=example&largeLanguageModel=example&page=1&pageSize=1&source=interactive&startDate=e
xample&startDatetime=2025-01-01T00:00:00.000Z", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/records/?
callerMember=example&chatbot=example&endDate=example&endDatetime=2025-01-
01T00:00:00.000Z&errorType=example&feedbackType=example&hasError=true&hasFeedback=true&ke
yword=example&largeLanguageModel=example&page=1&pageSize=1&source=interactive&startDate=e
xample&startDatetime=2025-01-01T00:00:00.000Z"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/records/?
callerMember=example&chatbot=example&endDate=example&endDatetime=2025-01-
01T00:00:00.000Z&errorType=example&feedbackType=example&hasError=true&hasFeedback=true&ke
yword=example&largeLanguageModel=example&page=1&pageSize=1&source=interactive&startDate=e
xample&startDatetime=2025-01-01T00:00:00.000Z", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Example Response Schema

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "senderName": string // Returns the name of the user message sender
      "callerMemberId": string
      "callerMemberName": string
      "feedback":
      {

```



```

    "id": string (uuid)
    "type":
    {
    }
    "suggestion"?: string // Optional
    "updatedAt": string (timestamp)
  }
  "inputMessage": string
  "condenseMessage": string // The refined user message after combining the chat history
and context
  "outputMessage": string
  "faithfulnessScore": number (double) // Determines whether the chatbot response matches
the database content or is fabricated
  "displayFaithfulnessScore": integer
  "answerRelevancyScore": number (double) // Determines whether the chatbot response is
relevant to the user question
  "displayAnswerRelevancyScore": integer
  "contextPrecisionScore": number (double) // Determines whether the chatbot response is
relevant to the reference data
  "displayContextPrecisionScore": integer
  "answerCorrectnessScore": number (double) // Determines whether the chatbot response is
correct (requires a reference answer)
  "displayAnswerCorrectnessScore": integer
  "answerSimilarityScore": number (double) // Determines whether the chatbot response is
similar to the reference answer
  "displayAnswerSimilarityScore": integer
  "contextRecallScore": number (double) // Determines whether the chatbot response can
retrieve relevant information from the reference data
  "displayContextRecallScore": integer
  "replyTime": string
  "createdAt": string (timestamp)
  "error": string // Stores error messages generated during execution
  "errorType": // May have different types
    string (enum: system, llm, embedding, reranker, vector_db, workflow, tool, timeout,
client_interrupt, hook_blocked, evaluation, other) // Distinguishes error sources: System, LLM,
Embedding, Reranker, etc.

* `system` - System
* `llm` - LLM
* `embedding` - Embedding
* `reranker` - Reranker
* `vector_db` - Vector DB
* `workflow` - Workflow
* `tool` - Tool
* `timeout` - Timeout
* `client_interrupt` - Client Interrupt
* `hook_blocked` - Hook Blocked
* `evaluation` - Evaluation
* `other` - Other

  "processingTime": object // Returns timing statistics for each processing stage,
including the streaming_metrics waterfall
  "usage": object // Returns usage statistics, including character counts, token counts,
and token details for each LLM call
  "citationNodes": [
    {
      "id": string

```

```
    "text": string
    "chatbotFileId": string
    "fileName": string
    "url": string
  }
]

```

```

    "conversationId": string (uuid)
    "inboxId": string (uuid)
    "botMessageId": string (uuid)
    "userMessageId": string (uuid)
    "creditCosts": [ // Return credit costs aggregated by item_code for credit-mode
organizations.
    object
  ]
  "cacheSavedCredits": number (double) // Net credits saved by prompt caching for this
reply; ``None`` outside credit mode.

Reuses the usage-statistics saving semantics (``compute_cache_saved_credits``)
on the reply's own billing batch, so the per-reply figure and the hourly
aggregate are defined by the same formula. The net value may be negative
(cache-creation premium exceeding read savings); the frontend hides
non-positive values by design.

    "trace": object // Return the round trace timeline for the detail view; ``None`` for
legacy records.
    "llmRawRequest": object // Raw messages array sent to the LLM on the final round,
including system prompt and memory
    "source": string // Origin discriminator for the unified /records/ response.
    "schedule": object // Shape parity with scheduled rows, which link their owning
ChatbotSchedule here.
  }
}
}

```

Example Response Value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "senderName": "Response string",
      "callerMemberId": "Response string",
      "callerMemberName": "Response string",
      "feedback": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "type": {},
        "suggestion": "Response string",
        "updatedAt": "Response string"
      },
      "inputMessage": "Response string",
      "condenseMessage": "Response string",
      "outputMessage": "Response string",
      "faithfulnessScore": 456,
      "displayFaithfulnessScore": 456,
      "answerRelevancyScore": 456,
      "displayAnswerRelevancyScore": 456,
      "contextPrecisionScore": 456,
      "displayContextPrecisionScore": 456,
      "answerCorrectnessScore": 456,
    }
  ]
}

```

```
"displayAnswerCorrectnessScore": 456,
"answerSimilarityScore": 456,
"displayAnswerSimilarityScore": 456,
"contextRecallScore": 456,
"displayContextRecallScore": 456,
"replyTime": "Response string",
"createdAt": "Response string",
"error": "Response string",
"errorType": "system",
"processingTime": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"usage": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example response name",
  "description": "Example response description"
},
"citationNodes": [
  {
    "id": "Response string",
    "text": "Response string",
    "chatbotFileId": "Response string",
    "fileName": "Response string",
    "url": "Response string"
  }
],
"inputHookLogs": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "hook": "550e8400-e29b-41d4-a716-446655440000",
    "hookName": "Response string",
    "hookTypeLabel": "Response string",
    "hookAction": "Response string",
    "conversation": "550e8400-e29b-41d4-a716-446655440000",
    "message": "550e8400-e29b-41d4-a716-446655440000",
    "messageContent": "Response string",
    "chatbotId": "Response string",
    "chatbotName": "Response string",
    "metadata": null,
    "triggerPoint": "Response string",
    "createdAt": "Response string",
    "updatedAt": "Response string"
  }
],
"outputHookLogs": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "hook": "550e8400-e29b-41d4-a716-446655440000",
    "hookName": "Response string",
    "hookTypeLabel": "Response string",
    "hookAction": "Response string",
    "conversation": "550e8400-e29b-41d4-a716-446655440000",
    "message": "550e8400-e29b-41d4-a716-446655440000",
    "messageContent": "Response string",
```

```

        "chatbotId": "Response string",
        "chatbotName": "Response string",
        "metadata": null,
        "triggerPoint": "Response string",
        "createdAt": "Response string",
        "updatedAt": "Response string"
    }
],
"instructionId": "Response string",
"llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
},
"effectiveMaxLlmOutputTokens": 456,
"chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
},
"context": "Response string",
"conversationId": "550e8400-e29b-41d4-a716-446655440000",
"inboxId": "550e8400-e29b-41d4-a716-446655440000",
"botMessageId": "550e8400-e29b-41d4-a716-446655440000",
"userMessageId": "550e8400-e29b-41d4-a716-446655440000",
"creditCosts": [
    {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Example response name",
        "description": "Example response description"
    }
],
"cacheSavedCredits": 456,
"trace": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
},
"llmRawRequest": null,
"source": "Response string",
"schedule": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example response name",
    "description": "Example response description"
}
}
]
}

```

Retrieve a Specific Conversation Record

GET

/api/v1/records/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Chatbot record.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiaagent.ai/api/v1/records/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/records/550e8400-e29b-41d4-a716-446655440000/",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/records/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/records/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Example Response Schema

```

{
  "id": string (uuid)
  "senderName": string // Returns the name of the user message sender
  "callerMemberId": string
  "callerMemberName": string
  "feedback":
  {
    "id": string (uuid)
    "type":
    {
    }
    "suggestion?": string // Optional
    "updatedAt": string (timestamp)
  }
  "inputMessage": string
}

```



```

"condenseMessage": string // The refined user message after combining the chat history and
context
"outputMessage": string
"faithfulnessScore": number (double) // Determines whether the chatbot response matches the
database content or is fabricated
"displayFaithfulnessScore": integer
"answerRelevancyScore": number (double) // Determines whether the chatbot response is
relevant to the user question
"displayAnswerRelevancyScore": integer
"contextPrecisionScore": number (double) // Determines whether the chatbot response is
relevant to the reference data
"displayContextPrecisionScore": integer
"answerCorrectnessScore": number (double) // Determines whether the chatbot response is
correct (requires a reference answer)
"displayAnswerCorrectnessScore": integer
"answerSimilarityScore": number (double) // Determines whether the chatbot response is
similar to the reference answer
"displayAnswerSimilarityScore": integer
"contextRecallScore": number (double) // Determines whether the chatbot response can retrieve
relevant information from the reference data
"displayContextRecallScore": integer
"replyTime": string
"createdAt": string (timestamp)
"error": string // Stores error messages generated during execution
"errorType": // May have different types
string (enum: system, llm, embedding, reranker, vector_db, workflow, tool, timeout,
client_interrupt, hook_blocked, evaluation, other) // Distinguishes error sources: System, LLM,
Embedding, Reranker, etc.

* `system` - System
* `llm` - LLM
* `embedding` - Embedding
* `reranker` - Reranker
* `vector_db` - Vector DB
* `workflow` - Workflow
* `tool` - Tool
* `timeout` - Timeout
* `client_interrupt` - Client Interrupt
* `hook_blocked` - Hook Blocked
* `evaluation` - Evaluation
* `other` - Other

"processingTime": object // Returns timing statistics for each processing stage, including
the streaming_metrics waterfall
"usage": object // Returns usage statistics, including character counts, token counts, and
token details for each LLM call
"citationNodes": [
  {
    "id": string
    "text": string
    "chatbotFileId": string
    "fileName": string
    "url": string
  }
]
"inputHookLogs": [
  {

```

```

    "id": string (uuid)
    "hook": string (uuid)
    "hookName": string
    "hookTypeLabel": string
    "hookAction": string
    "conversation": string (uuid)
    "message": string (uuid)
    "messageContent": string // Return the raw message text.
    "chatbotId": string
    "chatbotName": string
    "metadata": object
    "triggerPoint": string
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"outputHookLogs": [
  {
    "id": string (uuid)
    "hook": string (uuid)
    "hookName": string
    "hookTypeLabel": string
    "hookAction": string
    "conversation": string (uuid)
    "message": string (uuid)
    "messageContent": string // Return the raw message text.
    "chatbotId": string
    "chatbotName": string
    "metadata": object
    "triggerPoint": string
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"instructionId": string
"llm":
{
  "id": string (uuid)
  "name": string
}
"effectiveMaxLlmOutputTokens": integer // The max_tokens setting actually used for this
exchange, which may come from Chatbot.custom_max_llm_output_tokens or LLM.max_tokens
"chatbot":
{
  "id": string (uuid)
  "name": string
}
"context": string
"conversationId": string (uuid)
"inboxId": string (uuid)
"botMessageId": string (uuid)
"userMessageId": string (uuid)
"creditCosts": [ // Return credit costs aggregated by item_code for credit-mode
organizations.
  object
]

```

```
"cacheSavedCredits": number (double) // Net credits saved by prompt caching for this reply;
`None` outside credit mode.
```

Reuses the usage-statistics saving semantics (`compute\_cache\_saved\_credits`) on the reply's own billing batch, so the per-reply figure and the hourly aggregate are defined by the same formula. The net value may be negative (cache-creation premium exceeding read savings); the frontend hides non-positive values by design.

```
"trace": object // Return the round trace timeline for the detail view; `None` for legacy records.
```

```
"llmRawRequest": object // Raw messages array sent to the LLM on the final round, including system prompt and memory
```

```
"source": string // Origin discriminator for the unified /records/ response.
```

```
"schedule": object // Shape parity with scheduled rows, which link their owning ChatbotSchedule here.
```

```
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "senderName": "response string",
  "callerMemberId": "response string",
  "callerMemberName": "response string",
  "feedback": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": {},
    "suggestion": "response string",
    "updatedAt": "response string"
  },
  "inputMessage": "response string",
  "condenseMessage": "response string",
  "outputMessage": "response string",
  "faithfulnessScore": 456,
  "displayFaithfulnessScore": 456,
  "answerRelevancyScore": 456,
  "displayAnswerRelevancyScore": 456,
  "contextPrecisionScore": 456,
  "displayContextPrecisionScore": 456,
  "answerCorrectnessScore": 456,
  "displayAnswerCorrectnessScore": 456,
  "answerSimilarityScore": 456,
  "displayAnswerSimilarityScore": 456,
  "contextRecallScore": 456,
  "displayContextRecallScore": 456,
  "replyTime": "response string",
  "createdAt": "response string",
  "error": "response string",
  "errorType": "system",
  "processingTime": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Response Name",
    "description": "Example response description"
  },
  "usage": {
```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Response Name",
    "description": "Example response description"
  },
  "citationNodes": [
    {
      "id": "response string",
      "text": "response string",
      "chatbotFileId": "response string",
      "fileName": "response string",
      "url": "response string"
    }
  ],
  "inputHookLogs": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "hook": "550e8400-e29b-41d4-a716-446655440000",
      "hookName": "response string",
      "hookTypeLabel": "response string",
      "hookAction": "response string",
      "conversation": "550e8400-e29b-41d4-a716-446655440000",
      "message": "550e8400-e29b-41d4-a716-446655440000",
      "messageContent": "response string",
      "chatbotId": "response string",
      "chatbotName": "response string",
      "metadata": null,
      "triggerPoint": "response string",
      "createdAt": "response string",
      "updatedAt": "response string"
    }
  ],
  "outputHookLogs": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "hook": "550e8400-e29b-41d4-a716-446655440000",
      "hookName": "response string",
      "hookTypeLabel": "response string",
      "hookAction": "response string",
      "conversation": "550e8400-e29b-41d4-a716-446655440000",
      "message": "550e8400-e29b-41d4-a716-446655440000",
      "messageContent": "response string",
      "chatbotId": "response string",
      "chatbotName": "response string",
      "metadata": null,
      "triggerPoint": "response string",
      "createdAt": "response string",
      "updatedAt": "response string"
    }
  ],
  "instructionId": "response string",
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "effectiveMaxLlmOutputTokens": 456,
  "chatbot": {
```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "context": "response string",
  "conversationId": "550e8400-e29b-41d4-a716-446655440000",
  "inboxId": "550e8400-e29b-41d4-a716-446655440000",
  "botMessageId": "550e8400-e29b-41d4-a716-446655440000",
  "userMessageId": "550e8400-e29b-41d4-a716-446655440000",
  "creditCosts": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example Response Name",
      "description": "Example response description"
    }
  ],
  "cacheSavedCredits": 456,
  "trace": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Response Name",
    "description": "Example response description"
  },
  "llmRawRequest": null,
  "source": "response string",
  "schedule": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Response Name",
    "description": "Example response description"
  }
}
```

Status Code: **404** - The specified conversation record was not found

Retrieve Specific Conversation Records

GET

/api/v1/records/export-excel-requests/

Parameters

Parameter Name	Required	Type	Description
endDate	X	string	End date (format: YYYY-MM-DD, e.g., 2025-01-31)
keyword	X	string	Filter conversation records containing specific keywords (searches both user messages and bot replies)
startDate	X	string	Start date (format: YYYY-MM-DD, e.g., 2025-01-01)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/records/export-excel-requests/?
endDate=example&keyword=example&startDate=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/records/export-excel-requests/?
endDate=example&keyword=example&startDate=example", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/records/export-excel-requests/?
endDate=example&keyword=example&startDate=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

PHP

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/records/export-excel-
requests/?endDate=example&keyword=example&startDate=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
200	Excel File Download

Create a Conversation Record Export Request

POST

/api/v1/records/export-excel-requests/

Parameters

Parameter Name	Required	Type	Description
endDate	X	string	End date (format: YYYY-MM-DD, e.g., 2025-01-31)
keyword	X	string	Filter conversation records containing specific keywords (searches both user messages and bot replies)
startDate	X	string	Start date (format: YYYY-MM-DD, e.g., 2025-01-01)

Request Body

Request Parameters

Field	Type	Required	Description
chatbot	string	No	Specify the Chatbot UUIDs to export, separated by commas. Leave blank to export all authorized Chatbots in the organization.
chatbotId	string (uuid)	No	Specify the Chatbot UUID to export (an alias of chatbot; supports only one UUID).
largeLanguageModel	string	No	Specify the large language model UUIDs to export, separated by commas.
startDatetime	string (timestamp)	No	Exact start time (ISO 8601, inclusive), with the same semantics as the list drill-down filter.
endDatetime	string (timestamp)	No	Exact end time (ISO 8601, exclusive), with the same semantics as the list drill-down filter.
hasError	boolean	No	Filter records by whether they contain errors (true includes only records with errors; false includes only records without

Field	Type	Required	Description
			errors).
errorType	string	No	Filter by error type; supports comma-separated multiple selections (OR logic), using the same value range as the conversation records list.
hasFeedback	boolean	No	Filter by whether user feedback exists (true includes only likes/dislikes; false includes only records without feedback).
feedbackType	string	No	Filter by user feedback type (like / dislike); supports comma-separated multiple selections (OR logic).
callerMember	string	No	Filter by caller member UUID; supports comma-separated multiple selections (OR logic).

Request Schema Example

```
{
  "chatbot"?: string // Specify the Chatbot UUIDs to export, separated by commas. Leave blank
to export all authorized Chatbots in the organization. (Optional)
  "chatbotId"?: string (uuid) // Specify the Chatbot UUID to export (an alias of chatbot;
supports only one UUID). (Optional)
  "largeLanguageModel"?: string // Specify the large language model UUIDs to export, separated
by commas. (Optional)
  "startDatetime"?: string (timestamp) // Exact start time (ISO 8601, inclusive), with the same
semantics as the list drill-down filter. (Optional)
  "endDatetime"?: string (timestamp) // Exact end time (ISO 8601, exclusive), with the same
semantics as the list drill-down filter. (Optional)
  "hasError"?: boolean // Filter records by whether they contain errors (true includes only
records with errors; false includes only records without errors). (Optional)
  "errorType"?: string // Filter by error type; supports comma-separated multiple selections
(OR logic), using the same value range as the conversation records list. (Optional)
  "hasFeedback"?: boolean // Filter by whether user feedback exists (true includes only
likes/dislikes; false includes only records without feedback). (Optional)
  "feedbackType"?: string // Filter by user feedback type (like / dislike); supports comma-
separated multiple selections (OR logic). (Optional)
  "callerMember"?: string // Filter by caller member UUID; supports comma-separated multiple
selections (OR logic). (Optional)
}
```

Example Request Value

```
{
  "chatbot": "example string",
  "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
  "largeLanguageModel": "example string",
  "startDatetime": "example string",
  "endDatetime": "example string",
  "hasError": true,
  "errorType": "example string",
  "hasFeedback": true,
  "feedbackType": "example string",
}
```

```
"callerMember": "example string"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/records/export-excel-requests/?
endDate=example&keyword=example&startDate=example"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "chatbot": "example string",
  "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
  "keyword": "example string",
  "largeLanguageModel": "example string",
  "startDate": "example string",
  "endDate": "example string",
  "startDatetime": "example string",
  "endDatetime": "example string",
  "hasError": true,
  "errorType": "example string",
  "hasFeedback": true,
  "feedbackType": "example string",
  "callerMember": "example string"
}'

# Before running, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "chatbot": "example string",
  "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
  "keyword": "example string",
  "largeLanguageModel": "example string",
  "startDate": "example string",
  "endDate": "example string",
  "startDatetime": "example string",
  "endDatetime": "example string",
  "hasError": true,
  "errorType": "example string",
  "hasFeedback": true,
  "feedbackType": "example string",
  "callerMember": "example string"
};

axios.post("https://api.maiagent.ai/api/v1/records/export-excel-requests/?
endDate=example&keyword=example&startDate=example", data, config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/records/export-excel-requests/?\
endDate=example&keyword=example&startDate=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "chatbot": "example string",
    "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
    "keyword": "example string",
    "largeLanguageModel": "example string",
    "startDate": "example string",
    "endDate": "example string",
    "startDatetime": "example string",
    "endDatetime": "example string",
    "hasError": true,
    "errorType": "example string",
    "hasFeedback": true,
    "feedbackType": "example string",
    "callerMember": "example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/records/export-excel-requests/?endDate=example&keyword=example&startDate=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "chatbot": "example string",
            "chatbotId": "550e8400-e29b-41d4-a716-446655440000",
            "keyword": "example string",
            "largeLanguageModel": "example string",
            "startDate": "example string",
            "endDate": "example string",
            "startDatetime": "example string",
            "endDatetime": "example string",
            "hasError": true,
            "errorType": "example string",
            "hasFeedback": true,
            "feedbackType": "example string",
            "callerMember": "example string"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
200	Excel File Download

Get Conversation Record Export Status

GET /api/v1/records/export-excel-requests/{exportId}

Parameters

Parameter Name	Required	Type	Description
exportId	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/records/export-excel-requests/{exportId}"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running, replace YOUR_API_KEY and verify the request data.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/records/export-excel-requests/{exportId}", config)
  .then(response => {
    console.log('Response retrieved successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/records/export-excel-requests/{exportId}"

headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response retrieved successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/records/export-excel-requests/{exportId}
", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response retrieved successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```

{
  "id": string (uuid)
  "senderName": string // Returns the name of the user who sent the message
  "callerMemberId": string
  "callerMemberName": string
  "inputMessage": string
  "outputMessage": string
  "chatbot":
  {
    "id": string (uuid)
    "name": string
  }
  "condenseMessage": string // The refined user message after combining chat history and context
}

```



```

"feedback":
{
  "id": string (uuid)
  "type":
  {
  }
  "suggestion?": string // Optional
  "updatedAt": string (timestamp)
}
"displayFaithfulnessScore": integer
"displayAnswerRelevancyScore": integer
"displayContextPrecisionScore": integer
"displayAnswerCorrectnessScore": integer
"displayAnswerSimilarityScore": integer
"displayContextRecallScore": integer
"replyTime": string
"processingTime": object // Returns processing-time statistics for the list view, using the
same formula as the detail view but excluding the streaming_metrics waterfall
"usage": object // Returns usage statistics, including character count, token count, and
token details for each LLM call
"llm":
{
  "id": string (uuid)
  "name": string
}
"createdAt": string (timestamp)
"conversationId": string (uuid)
"errorType": // May have different types
string (enum: system, llm, embedding, reranker, vector_db, workflow, tool, timeout,
client_interrupt, hook_blocked, evaluation, other) // Distinguishes the error source: system,
LLM, Embedding, Reranker, etc.

* `system` - System
* `llm` - LLM
* `embedding` - Embedding
* `reranker` - Reranker
* `vector_db` - Vector DB
* `workflow` - Workflow
* `tool` - Tool
* `timeout` - Timeout
* `client_interrupt` - Client Interrupt
* `hook_blocked` - Hook Blocked
* `evaluation` - Evaluation
* `other` - Other
"creditCosts": [
  object
]
"source": string // Origin discriminator for the unified /records/ response.
"schedule": object // Shape parity with scheduled rows, which link their owning
ChatbotSchedule here.
}

```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "senderName": "response string",
  "callerMemberId": "response string",
  "callerMemberName": "response string",
  "inputMessage": "response string",
  "outputMessage": "response string",
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "condenseMessage": "response string",
  "feedback": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": {},
    "suggestion": "response string",
    "updatedAt": "response string"
  },
  "displayFaithfulnessScore": 456,
  "displayAnswerRelevancyScore": 456,
  "displayContextPrecisionScore": 456,
  "displayAnswerCorrectnessScore": 456,
  "displayAnswerSimilarityScore": 456,
  "displayContextRecallScore": 456,
  "replyTime": "response string",
  "processingTime": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Response Name",
    "description": "Example response description"
  },
  "usage": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Response Name",
    "description": "Example response description"
  },
  "llm": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response string"
  },
  "createdAt": "response string",
  "conversationId": "550e8400-e29b-41d4-a716-446655440000",
  "errorType": "system",
  "creditCosts": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Example Response Name",
      "description": "Example response description"
    }
  ],
  "source": "response string",
  "schedule": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Response Name",
    "description": "Example response description"
  }
}
```

```
}  
}
```

API call example (Shell)

...

目錄

1. Create Tool
2. List Tools
3. Retrieve Tool
4. Retrieve Tool
5. Retrieve Tool
6. Retrieve Tool
7. Update Tool
8. Update Tool
9. Partially Update Tool
10. Delete Tool
11. Create Connector
12. List Connectors
13. Retrieve Connector
14. Retrieve Connector
15. Update Connector
16. Partially Update Connector
17. Delete Connector
18. List Connector Authorized Members
19. Remove Connector Authorized Member
20. List MCP Tool Registries
21. Retrieve MCP Tool Registry
22. List Tool Execution Records
23. Retrieve Tool Execution Record
24. Retrieve Tool Execution Record
25. Retrieve Tool Execution Record
26. List Execution Records for a Specific Tool
27. Retrieve Specific Tool Execution Record
28. Retrieve Specific Tool Execution Record
29. Retrieve Specific Tool Execution Record

API call example (Shell)

Create Tool

POST

/api/v1/tools/

Request

Request Parameters

Field	Type	Required	Description
name	object	No	Tool name can only contain letters, numbers, underscores (_), and hyphens (-). Not required for MCP type tools. This field is used for API type tools by the LLM.
displayName	string	Yes	A name that can contain any characters, used for user configuration and management.
description	string	No	Tool description displayed to users. Not required for MCP type tools.
prompt	string	No	Tool description/prompt for LLM use. If empty, the system will use the description field. Not required for MCP type tools.
toolType	object	No	
apiUrl	object	No	
httpMethod	object	No	
rawHeaders	object	No	
rawQueryParams	object	No	Query string parameters appended to every request of this API tool. Values support the same {{variable}} template rendering as raw_headers. On key collision with LLM-

Field	Type	Required	Description
			generated arguments, these values ...
rawParametersSchema	object	No	
functionCode	string	No	
mcpUrl	string	No	
mcpRegistry	string (uuid)	No	Registry backing this MCP tool. Null for manual-URL MCP tools.
mcpAllowedTools	object	No	
rawMcpHeader	object	No	Default headers used when a Contact has no corresponding MCP Credential
authSubject	object	No	Shared uses the tool level token plus per contact credentials; contact requires per contact authentication only; caller mints a short-lived token for the conversation-bound member (falls back to per c...
flexTemplateJson	object	No	FlexMessage JSON template. Required for FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble / carousel / box / text / image / button etc.).
flexSchema	object	No	AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be e...

Field	Type	Required	Description
quickReplyConfig	object	No	Authoring config for QUICK_REPLY tools. Structured JSON describing the chip buttons: static list or dynamic (LLM-generated) mode, each field either a fixed value or an LLM prompt.
category	string (enum: knowledge_base, web_search, web_fetch, code_execution, file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other)	No	knowledge_base : Knowledge base retrieval ; web_search : Web search ; web_fetch : Web page fetch ; code_execution : Code execution ; file_processing : File processing ; image_video_generation : ...
displayCopyByLocale	object	No	
outputRenderTemplateByLocale	object	No	
toolDisplayMode	object	No	
groups	array[IdName]	No	Groups this tool is assigned to. Non-owner members must pick at least one group they belong to.

Request Schema Example

```
{
  "name"? : // May have different types (Optional)
    string // Tool name can only contain letters, numbers, underscores (_), and
    hyphens (-). Not required for MCP type tools. This field is used for API type
    tools by the LLM. (Optional)
  "displayName": string // A name that can contain any characters, used for user
  configuration and management.
  "description"? : string // Tool description displayed to users. Not required for
  MCP type tools. (Optional)
  "prompt"? : string // Tool description/prompt for LLM use. If empty, the system
  will use the description field. Not required for MCP type tools. (Optional)
```



```

"toolType"?: // Optional
{
}
"apiUrl"?: // May have different types (Optional)
string (uri) // Optional
"httpMethod"?: // May have different types (Optional)
string (enum: get, post, put, delete, patch) // Optional
"rawHeaders"?: object // Optional
"rawQueryParams"?: object // Query string parameters appended to every request
of this API tool. Values support the same {{variable}} template rendering as
raw_headers. On key collision with LLM-generated arguments, these values win –
use this field for API keys that must travel in the query string. (Optional)
"rawParametersSchema"?: object // Optional
"functionCode"?: string // Optional
"mcpUrl"?: string // Optional
"mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (Optional)
"mcpAllowedTools"?: object // Optional
"rawMcpHeader"?: object // Default headers used when a Contact has no
corresponding MCP Credential (Optional)
"authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
{
}
"flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (Optional)
"flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (Optional)
"quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (Optional)
"category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution

```

```

* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
  "displayCopyByLocale"?: object // Optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // may have different types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
  least one group they belong to. (Optional)
    {
      "id": string (uuid)
    }
  ]
}

```

Request Example

```

{
  "name": null,
  "displayName": "example_string",
  "description": "example_string",
  "prompt": "example_string",
  "toolType": {},
  "apiUrl": null,
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "example_string",
  "mcpUrl": "example_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}

```

```
}  
]  
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/tools/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": null,
  "displayName": "example_string",
  "description": "example_string",
  "prompt": "example_string",
  "toolType": {},
  "apiUrl": null,
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "example_string",
  "mcpUrl": "example_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": null,
  "displayName": "example_string",
  "description": "example_string",
  "prompt": "example_string",
  "toolType": {},
  "apiUrl": null,
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "example_string",
  "mcpUrl": "example_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.post("https://api.maiagent.ai/api/v1/tools/", data, config)
  .then(response => {
```

```
    console.log('Successfully retrieved response:');  
    console.log(response.data);  
  })  
  .catch(error => {  
    console.error('Request error:');  
    console.error(error.response?.data || error.message);  
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": null,
    "displayName": "example_string",
    "description": "example_string",
    "prompt": "example_string",
    "toolType": {},
    "apiUrl": null,
    "httpMethod": null,
    "rawHeaders": null,
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "example_string",
    "mcpUrl": "example_string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
    "category": "knowledge_base",
    "displayCopyByLocale": null,
    "outputRenderTemplateByLocale": null,
    "toolDisplayMode": null,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
```

```
except Exception as e:  
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/tools/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": null,
            "displayName": "example_string",
            "description": "example_string",
            "prompt": "example_string",
            "toolType": {},
            "apiUrl": null,
            "httpMethod": null,
            "rawHeaders": null,
            "rawQueryParams": null,
            "rawParametersSchema": null,
            "functionCode": "example_string",
            "mcpUrl": "example_string",
            "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
            "mcpAllowedTools": null,
            "rawMcpHeader": null,
            "authSubject": {},
            "flexTemplateJson": null,
            "flexSchema": null,
            "quickReplyConfig": null,
            "category": "knowledge_base",
            "displayCopyByLocale": null,
            "outputRenderTemplateByLocale": null,
            "toolDisplayMode": null,
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
```

```

        echo "Successfully retrieved response:\n";
        print_r($data);
    } catch (Exception $e) {
        echo 'Request error: ' . $e->getMessage();
    }
?>

```

Response

Status Code: 201

Response Schema Example

```

{
  "id": string (uuid)
  "name"?: // May have different types (Optional)
    string // Tool name can only contain letters, numbers, underscores (_), and
    hyphens (-). Not required for MCP type tools. This field is used for API type
    tools by the LLM. (Optional)
  "displayName": string // A name that can contain any characters, used for user
  configuration and management.
  "description"?: string // Tool description displayed to users. Not required for
  MCP type tools. (Optional)
  "prompt"?: string // Tool description/prompt for LLM use. If empty, the system
  will use the description field. Not required for MCP type tools. (Optional)
  "toolType"?: // Optional
    {
    }
  "apiUrl"?: // May have different types (Optional)
    string (uri) // Optional
  "httpMethod"?: // May have different types (Optional)
    string (enum: get, post, put, delete, patch) // Optional
  "rawHeaders"?: object // Optional
  "rawHeadersMasked": object
  "rawQueryParams"?: object // Query string parameters appended to every request
  of this API tool. Values support the same {{variable}} template rendering as
  raw_headers. On key collision with LLM-generated arguments, these values win –
  use this field for API keys that must travel in the query string. (Optional)
  "rawQueryParamsMasked": object
  "rawParametersSchema"?: object // Optional
  "functionCode"?: string // Optional
}

```

```

"organization": string (uuid)
"createdBy": string (uuid)
"isGlobal": boolean // If set as global tool, all organizations can use this
tool
"mcpUrl"?: string // Optional
"mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (Optional)
"mcpAllowedTools"?: object // Optional
"rawMcpHeader"?: object // Default headers used when a Contact has no
corresponding MCP Credential (Optional)
"rawMcpHeaderMasked": object
"authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
{
}
"flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (Optional)
"flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (Optional)
"quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (Optional)
"category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
"displayCopyByLocale"?: object // Optional
"outputRenderTemplateByLocale"?: object // optional

```

```

"toolDisplayMode"?: // may have different types (optional)
string (enum: full, compact, hidden) // optional
"groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string",
  "displayName": "response_string",
  "description": "response_string",
  "prompt": "response_string",
  "toolType": {},
  "apiUrl": "https://example.com/file.jpg",
  "httpMethod": "get",
  "rawHeaders": null,
  "rawHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example_response_name",
    "description": "example_response_description"
  },
  "rawQueryParams": null,
  "rawQueryParamsMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example_response_name",
    "description": "example_response_description"
  },
  "rawParametersSchema": null,
  "functionCode": "response_string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "isGlobal": false,
  "mcpUrl": "response_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
}

```

```
"rawMcpHeader": null,
"rawMcpHeaderMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example_response_name",
  "description": "example_response_description"
},
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string"
  }
],
"createdAt": "response_string",
"updatedAt": "response_string",
"canUpdate": false,
"canDelete": false
}
```

List Tools

GET

</api/v1/tools/>

Parameters

Parameter	Required	Type	Description
<code>clientPlatform</code>	X	string	Client platform. When "desktop" is passed, desktop type tools are included; otherwise desktop tools are excluded by default

isGlobal	X	boolean	Set to "true" to retrieve global tools, otherwise retrieve organization tools
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
toolType	X	string	Filter by tool type: api, function, mcp, desktop

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/?
clientPlatform=desktop&isGlobal=true&page=1&pageSize=1&toolType=api"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/?
clientPlatform=desktop&isGlobal=true&page=1&pageSize=1&toolType=api",
config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/?
clientPlatform=desktop&isGlobal=true&page=1&pageSize=1&toolType=api"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/?
clientPlatform=desktop&isGlobal=true&page=1&pageSize=1&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name"?: // May have different types (Optional)
    }
  ]
}
```

```

    string // Tool name can only contain letters, numbers, underscores (_), and
    hyphens (-). Not required for MCP type tools. This field is used for API type
    tools by the LLM. (Optional)
    "displayName": string // A name that can contain any characters, used for
    user configuration and management.
    "description"?: string // Tool description displayed to users. Not required
    for MCP type tools. (Optional)
    "prompt"?: string // Tool description/prompt for LLM use. If empty, the
    system will use the description field. Not required for MCP type tools.
    (Optional)
    "toolType"?: // Optional
    {
    }
    "apiUrl"?: // May have different types (Optional)
    string (uri) // Optional
    "httpMethod"?: // May have different types (Optional)
    string (enum: get, post, put, delete, patch) // Optional
    "rawHeaders"?: object // Optional
    "rawHeadersMasked": object
    "rawQueryParams"?: object // Query string parameters appended to every
    request of this API tool. Values support the same {{variable}} template rendering
    as raw_headers. On key collision with LLM-generated arguments, these values win –
    use this field for API keys that must travel in the query string. (Optional)
    "rawQueryParamsMasked": object
    "rawParametersSchema"?: object // Optional
    "functionCode"?: string // Optional
    "organization": string (uuid)
    "createdBy": string (uuid)
    "isGlobal": boolean // If set as global tool, all organizations can use
    this tool
    "mcpUrl"?: string // Optional
    "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
    manual-URL MCP tools. (Optional)
    "mcpAllowedTools"?: object // Optional
    "rawMcpHeader"?: object // Default headers used when a Contact has no
    corresponding MCP Credential (Optional)
    "rawMcpHeaderMasked": object
    "authSubject"?: // Shared uses the tool level token plus per contact
    credentials; contact requires per contact authentication only; caller mints a
    short-lived token for the conversation-bound member (falls back to per contact
    credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
    {
    }
    "flexTemplateJson"?: object // FlexMessage JSON template. Required for
    FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /

```

```

carousel / box / text / image / button etc.). (Optional)
    "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON
object whose keys are field paths parsed from flex_template_json (e.g. text /
image url / button label / button uri / button message). Each value may be either
a plain string (the AI prompt) or an object carrying additional metadata such as
a field tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to
the template default. (Optional)
    "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (Optional)
    "category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
    "displayCopyByLocale"?: object // Optional
    "outputRenderTemplateByLocale"?: object // optional
    "toolDisplayMode"?: // may have different types (optional)
string (enum: full, compact, hidden) // optional
    "groups"?: [ // Groups this tool is assigned to. Non-owner members must
pick at least one group they belong to. (Optional)
    {
        "id": string (uuid)
        "name": string
    }
]
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
    "canUpdate": boolean // Return whether the current user can update this
resource.
    "canDelete": boolean // Return whether the current user can delete this
resource.
    }
]
}

```

Response Example

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response_string",
      "displayName": "response_string",
      "description": "response_string",
      "prompt": "response_string",
      "toolType": {},
      "apiUrl": "https://example.com/file.jpg",
      "httpMethod": "get",
      "rawHeaders": null,
      "rawHeadersMasked": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "example_response_name",
        "description": "example_response_description"
      },
      "rawQueryParams": null,
      "rawQueryParamsMasked": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "example_response_name",
        "description": "example_response_description"
      },
      "rawParametersSchema": null,
      "functionCode": "response_string",
      "organization": "550e8400-e29b-41d4-a716-446655440000",
      "createdBy": "550e8400-e29b-41d4-a716-446655440000",
      "isGlobal": false,
      "mcpUrl": "response_string",
      "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
      "mcpAllowedTools": null,
      "rawMcpHeader": null,
      "rawMcpHeaderMasked": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "example_response_name",
        "description": "example_response_description"
      },
      "authSubject": {},
      "flexTemplateJson": null,
      "flexSchema": null,
      "quickReplyConfig": null,
      "category": "knowledge_base",
      "displayCopyByLocale": null,
      "outputRenderTemplateByLocale": null,
      "toolDisplayMode": "full",
    }
  ]
}
```

```
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string"
  }
],
"createdAt": "response_string",
"updatedAt": "response_string",
"canUpdate": false,
"canDelete": false
}
]
```

Retrieve Tool

GET

`/api/v1/tools/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this tool.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name"?: // May have different types (Optional)
  string // Tool name can only contain letters, numbers, underscores (_), and
hyphens (-). Not required for MCP type tools. This field is used for API type
tools by the LLM. (Optional)
  "displayName": string // A name that can contain any characters, used for user
configuration and management.
```



```

    "description"?: string // Tool description displayed to users. Not required for
MCP type tools. (Optional)
    "prompt"?: string // Tool description/prompt for LLM use. If empty, the system
will use the description field. Not required for MCP type tools. (Optional)
    "toolType"?: // Optional
    {
    }
    "apiUrl"?: // May have different types (Optional)
    string (uri) // Optional
    "httpMethod"?: // May have different types (Optional)
    string (enum: get, post, put, delete, patch) // Optional
    "rawHeaders"?: object // Optional
    "rawHeadersMasked": object
    "rawQueryParams"?: object // Query string parameters appended to every request
of this API tool. Values support the same {{variable}} template rendering as
raw_headers. On key collision with LLM-generated arguments, these values win –
use this field for API keys that must travel in the query string. (Optional)
    "rawQueryParamsMasked": object
    "rawParametersSchema"?: object // Optional
    "functionCode"?: string // Optional
    "organization": string (uuid)
    "createdBy": string (uuid)
    "isGlobal": boolean // If set as global tool, all organizations can use this
tool
    "mcpUrl"?: string // Optional
    "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (Optional)
    "mcpAllowedTools"?: object // Optional
    "rawMcpHeader"?: object // Default headers used when a Contact has no
corresponding MCP Credential (Optional)
    "rawMcpHeaderMasked": object
    "authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
    {
    }
    "flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (Optional)
    "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the

```

```

template default. (Optional)
  "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
  Structured JSON describing the chip buttons: static list or dynamic (LLM-
  generated) mode, each field either a fixed value or an LLM prompt. (Optional)
  "category"?: string (enum: knowledge_base, web_search, web_fetch,
  code_execution, file_processing, image_video_generation, database_query,
  third_party_integration, browser_operation, agent_collaboration, other) // *
  `knowledge_base` - Knowledge base retrieval
  * `web_search` - Web search
  * `web_fetch` - Web page fetch
  * `code_execution` - Code execution
  * `file_processing` - File processing
  * `image_video_generation` - Image/video generation
  * `database_query` - Database query
  * `third_party_integration` - Third-party integration
  * `browser_operation` - Browser operation
  * `agent_collaboration` - Agent collaboration
  * `other` - Other (catch-all) (Optional)
  "displayCopyByLocale"?: object // Optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // may have different types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
  least one group they belong to. (Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
  resource.
  "canDelete": boolean // Return whether the current user can delete this
  resource.
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string",
  "displayName": "response_string",
  "description": "response_string",
  "prompt": "response_string",
  "toolType": {},
  "apiUrl": "https://example.com/file.jpg",
  "httpMethod": "get",

```

```
"rawHeaders": null,
"rawHeadersMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example_response_name",
  "description": "example_response_description"
},
"rawQueryParams": null,
"rawQueryParamsMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example_response_name",
  "description": "example_response_description"
},
"rawParametersSchema": null,
"functionCode": "response_string",
"organization": "550e8400-e29b-41d4-a716-446655440000",
"createdBy": "550e8400-e29b-41d4-a716-446655440000",
"isGlobal": false,
"mcpUrl": "response_string",
"mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
"mcpAllowedTools": null,
"rawMcpHeader": null,
"rawMcpHeaderMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example_response_name",
  "description": "example_response_description"
},
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string"
  }
],
"createdAt": "response_string",
"updatedAt": "response_string",
"canUpdate": false,
"canDelete": false
}
```

Retrieve Tool

GET

/api/v1/tools/available-tools/

Parameters

Parameter	Required	Type	Description
availableTools	V	array	List of all available MCP tools
mcpAllowedTools	X	array	List of allowed MCP tools
mcpUrl	V	string	MCP server URL (Required)
rawMcpHeader	X		MCP Header (JSON format)
toolId	X	string	Tool ID for per-member OAuth token lookup

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/available-tools/?
availableTools=example&mcpAllowedTools=example&mcpUrl=example&rawMcpHeader=e
xample&toolId=550e8400-e29b-41d4-a716-446655440000"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/available-tools/?
availableTools=example&mcpAllowedTools=example&mcpUrl=example&rawMcpHeader=e
xample&toolId=550e8400-e29b-41d4-a716-446655440000", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/available-tools/?availableTools=example&mcpAllowedTools=example&mcpUrl=example&rawMcpHeader=example&toolId=550e8400-e29b-41d4-a716-446655440000"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/tools/available-tools/?
availableTools=example&mcpAllowedTools=example&mcpUrl=example&rawMcpHeader=e
xample&toolId=550e8400-e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "mcpUrl": string // MCP server URL (Required)
  "rawMcpHeader"?: object // MCP Header (JSON format) (Optional)
  "mcpAllowedTools"?: [ // List of allowed MCP tools (Optional)
    string
  ]
}
```

```
"toolId"?: string (uuid) // Tool ID for per-member OAuth token lookup
(Optional)
"availableTools": [ // List of all available MCP tools
  string
]
}
```

Response Example

```
{
  "mcpUrl": "response_string",
  "rawMcpHeader": null,
  "mcpAllowedTools": [
    "response_string"
  ],
  "toolId": "550e8400-e29b-41d4-a716-446655440000",
  "availableTools": [
    "response_string"
  ]
}
```

Status Code: **400**

Response Schema Example

```
{
  "detail"?: string // Optional
}
```

Response Example

```
{
  "detail": "response_string"
}
```

Retrieve Tool

GET

/api/v1/tools/category-copy-overrides/

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/category-copy-overrides/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/category-copy-overrides/",
config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/category-copy-overrides/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/category-
copy-overrides/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name"?: // May have different types (Optional)
  string // Tool name can only contain letters, numbers, underscores (_), and
hyphens (-). Not required for MCP type tools. This field is used for API type
tools by the LLM. (Optional)
  "displayName": string // A name that can contain any characters, used for user
configuration and management.
```

```

    "description"?: string // Tool description displayed to users. Not required for
MCP type tools. (Optional)
    "prompt"?: string // Tool description/prompt for LLM use. If empty, the system
will use the description field. Not required for MCP type tools. (Optional)
    "toolType"?: // Optional
    {
    }
    "apiUrl"?: // May have different types (Optional)
    string (uri) // Optional
    "httpMethod"?: // May have different types (Optional)
    string (enum: get, post, put, delete, patch) // Optional
    "rawHeaders"?: object // Optional
    "rawHeadersMasked": object
    "rawQueryParams"?: object // Query string parameters appended to every request
of this API tool. Values support the same {{variable}} template rendering as
raw_headers. On key collision with LLM-generated arguments, these values win –
use this field for API keys that must travel in the query string. (Optional)
    "rawQueryParamsMasked": object
    "rawParametersSchema"?: object // Optional
    "functionCode"?: string // Optional
    "organization": string (uuid)
    "createdBy": string (uuid)
    "isGlobal": boolean // If set as global tool, all organizations can use this
tool
    "mcpUrl"?: string // Optional
    "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (Optional)
    "mcpAllowedTools"?: object // Optional
    "rawMcpHeader"?: object // Default headers used when a Contact has no
corresponding MCP Credential (Optional)
    "rawMcpHeaderMasked": object
    "authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
    {
    }
    "flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (Optional)
    "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the

```

```

template default. (Optional)
  "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
  Structured JSON describing the chip buttons: static list or dynamic (LLM-
  generated) mode, each field either a fixed value or an LLM prompt. (Optional)
  "category"?: string (enum: knowledge_base, web_search, web_fetch,
  code_execution, file_processing, image_video_generation, database_query,
  third_party_integration, browser_operation, agent_collaboration, other) // *
  `knowledge_base` - Knowledge base retrieval
  * `web_search` - Web search
  * `web_fetch` - Web page fetch
  * `code_execution` - Code execution
  * `file_processing` - File processing
  * `image_video_generation` - Image/video generation
  * `database_query` - Database query
  * `third_party_integration` - Third-party integration
  * `browser_operation` - Browser operation
  * `agent_collaboration` - Agent collaboration
  * `other` - Other (catch-all) (Optional)
  "displayCopyByLocale"?: object // Optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // may have different types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
  least one group they belong to. (Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
  resource.
  "canDelete": boolean // Return whether the current user can delete this
  resource.
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string",
  "displayName": "response_string",
  "description": "response_string",
  "prompt": "response_string",
  "toolType": {},
  "apiUrl": "https://example.com/file.jpg",
  "httpMethod": "get",

```

```
"rawHeaders": null,
"rawHeadersMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example_response_name",
  "description": "example_response_description"
},
"rawQueryParams": null,
"rawQueryParamsMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example_response_name",
  "description": "example_response_description"
},
"rawParametersSchema": null,
"functionCode": "response_string",
"organization": "550e8400-e29b-41d4-a716-446655440000",
"createdBy": "550e8400-e29b-41d4-a716-446655440000",
"isGlobal": false,
"mcpUrl": "response_string",
"mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
"mcpAllowedTools": null,
"rawMcpHeader": null,
"rawMcpHeaderMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example_response_name",
  "description": "example_response_description"
},
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string"
  }
],
"createdAt": "response_string",
"updatedAt": "response_string",
"canUpdate": false,
"canDelete": false
}
```

Retrieve Tool

GET

/api/v1/tools/category-default-copies/

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/category-default-copies/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/category-default-copies/",
config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/category-default-copies/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/category-
default-copies/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name"?: // May have different types (Optional)
  string // Tool name can only contain letters, numbers, underscores (_), and
hyphens (-). Not required for MCP type tools. This field is used for API type
tools by the LLM. (Optional)
  "displayName": string // A name that can contain any characters, used for user
configuration and management.
```

```

    "description"?: string // Tool description displayed to users. Not required for
MCP type tools. (Optional)
    "prompt"?: string // Tool description/prompt for LLM use. If empty, the system
will use the description field. Not required for MCP type tools. (Optional)
    "toolType"?: // Optional
    {
    }
    "apiUrl"?: // May have different types (Optional)
    string (uri) // Optional
    "httpMethod"?: // May have different types (Optional)
    string (enum: get, post, put, delete, patch) // Optional
    "rawHeaders"?: object // Optional
    "rawHeadersMasked": object
    "rawQueryParams"?: object // Query string parameters appended to every request
of this API tool. Values support the same {{variable}} template rendering as
raw_headers. On key collision with LLM-generated arguments, these values win –
use this field for API keys that must travel in the query string. (Optional)
    "rawQueryParamsMasked": object
    "rawParametersSchema"?: object // Optional
    "functionCode"?: string // Optional
    "organization": string (uuid)
    "createdBy": string (uuid)
    "isGlobal": boolean // If set as global tool, all organizations can use this
tool
    "mcpUrl"?: string // Optional
    "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (Optional)
    "mcpAllowedTools"?: object // Optional
    "rawMcpHeader"?: object // Default headers used when a Contact has no
corresponding MCP Credential (Optional)
    "rawMcpHeaderMasked": object
    "authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
    {
    }
    "flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (Optional)
    "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the

```

```

template default. (Optional)
  "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
  Structured JSON describing the chip buttons: static list or dynamic (LLM-
  generated) mode, each field either a fixed value or an LLM prompt. (Optional)
  "category"?: string (enum: knowledge_base, web_search, web_fetch,
  code_execution, file_processing, image_video_generation, database_query,
  third_party_integration, browser_operation, agent_collaboration, other) // *
  `knowledge_base` - Knowledge base retrieval
  * `web_search` - Web search
  * `web_fetch` - Web page fetch
  * `code_execution` - Code execution
  * `file_processing` - File processing
  * `image_video_generation` - Image/video generation
  * `database_query` - Database query
  * `third_party_integration` - Third-party integration
  * `browser_operation` - Browser operation
  * `agent_collaboration` - Agent collaboration
  * `other` - Other (catch-all) (Optional)
  "displayCopyByLocale"?: object // Optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // may have different types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
  least one group they belong to. (Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
  "canUpdate": boolean // Return whether the current user can update this
  resource.
  "canDelete": boolean // Return whether the current user can delete this
  resource.
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string",
  "displayName": "response_string",
  "description": "response_string",
  "prompt": "response_string",
  "toolType": {},
  "apiUrl": "https://example.com/file.jpg",
  "httpMethod": "get",

```

```
"rawHeaders": null,
"rawHeadersMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example_response_name",
  "description": "example_response_description"
},
"rawQueryParams": null,
"rawQueryParamsMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example_response_name",
  "description": "example_response_description"
},
"rawParametersSchema": null,
"functionCode": "response_string",
"organization": "550e8400-e29b-41d4-a716-446655440000",
"createdBy": "550e8400-e29b-41d4-a716-446655440000",
"isGlobal": false,
"mcpUrl": "response_string",
"mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
"mcpAllowedTools": null,
"rawMcpHeader": null,
"rawMcpHeaderMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example_response_name",
  "description": "example_response_description"
},
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string"
  }
],
"createdAt": "response_string",
"updatedAt": "response_string",
"canUpdate": false,
"canDelete": false
}
```

Update Tool

PUT

/api/v1/tools/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this tool.

Request

Request Parameters

Field	Type	Required	Description
name	object	No	Tool name can only contain letters, numbers, underscores (_), and hyphens (-). Not required for MCP type tools. This field is used for API type tools by the LLM.
displayName	string	Yes	A name that can contain any characters, used for user configuration and management.
description	string	No	Tool description displayed to users. Not required for MCP type tools.
prompt	string	No	Tool description/prompt for LLM use. If empty, the system will use the description field. Not required for MCP type tools.

Field	Type	Required	Description
toolType	object	No	
apiUrl	object	No	
httpMethod	object	No	
rawHeaders	object	No	
rawQueryParams	object	No	Query string parameters appended to every request of this API tool. Values support the same {{variable}} template rendering as raw_headers. On key collision with LLM-generated arguments, these values ...
rawParametersSchema	object	No	
functionCode	string	No	
mcpUrl	string	No	
mcpRegistry	string (uuid)	No	Registry backing this MCP tool. Null for manual-URL MCP tools.
mcpAllowedTools	object	No	
rawMcpHeader	object	No	Default headers used when a Contact has no corresponding MCP Credential
authSubject	object	No	Shared uses the tool level token plus per contact credentials; contact requires per contact authentication only; caller mints a short-lived token for the conversation-bound member (falls back to per c...
flexTemplateJson	object	No	FlexMessage JSON template. Required for FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON

Field	Type	Required	Description
			structure (bubble / carousel / box / text / image / button etc.).
flexSchema	object	No	AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be e...
quickReplyConfig	object	No	Authoring config for QUICK_REPLY tools. Structured JSON describing the chip buttons: static list or dynamic (LLM-generated) mode, each field either a fixed value or an LLM prompt.
category	string (enum: knowledge_base, web_search, web_fetch, code_execution, file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other)	No	knowledge_base : Knowledge base retrieval ; web_search : Web search ; web_fetch : Web page fetch ; code_execution : Code execution ; file_processing : File processing ; image_video_generation : ...
displayCopyByLocale	object	No	
outputRenderTemplateByLocale	object	No	
toolDisplayMode	object	No	
groups	array[IdName]	No	Groups this tool is assigned to. Non-owner members must pick at least one group they belong to.

```

{
  "name"?: // May have different types (Optional)
    string // Tool name can only contain letters, numbers, underscores (_), and
    hyphens (-). Not required for MCP type tools. This field is used for API type
    tools by the LLM. (Optional)
    "displayName": string // A name that can contain any characters, used for user
    configuration and management.
    "description"?: string // Tool description displayed to users. Not required for
    MCP type tools. (Optional)
    "prompt"?: string // Tool description/prompt for LLM use. If empty, the system
    will use the description field. Not required for MCP type tools. (Optional)
    "toolType"?: // Optional
    {
    }
    "apiUrl"?: // May have different types (Optional)
    string (uri) // Optional
    "httpMethod"?: // May have different types (Optional)
    string (enum: get, post, put, delete, patch) // Optional
    "rawHeaders"?: object // Optional
    "rawQueryParams"?: object // Query string parameters appended to every request
    of this API tool. Values support the same {{variable}} template rendering as
    raw_headers. On key collision with LLM-generated arguments, these values win –
    use this field for API keys that must travel in the query string. (Optional)
    "rawParametersSchema"?: object // Optional
    "functionCode"?: string // Optional
    "mcpUrl"?: string // Optional
    "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
    manual-URL MCP tools. (Optional)
    "mcpAllowedTools"?: object // Optional
    "rawMcpHeader"?: object // Default headers used when a Contact has no
    corresponding MCP Credential (Optional)
    "authSubject"?: // Shared uses the tool level token plus per contact
    credentials; contact requires per contact authentication only; caller mints a
    short-lived token for the conversation-bound member (falls back to per contact
    credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
  {
  }
  "flexTemplateJson"?: object // FlexMessage JSON template. Required for
  FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
  carousel / box / text / image / button etc.). (Optional)
  "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
  whose keys are field paths parsed from flex_template_json (e.g. text / image url
  / button label / button uri / button message). Each value may be either a plain
  string (the AI prompt) or an object carrying additional metadata such as a field

```

```

tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (Optional)
  "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (Optional)
  "category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
  "displayCopyByLocale"?: object // Optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // may have different types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (Optional)
    {
      "id": string (uuid)
    }
  ]
}

```

Request Example

```

{
  "name": null,
  "displayName": "example_string",
  "description": "example_string",
  "prompt": "example_string",
  "toolType": {},
  "apiUrl": null,
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "example_string",
  "mcpUrl": "example_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,

```

```
"rawMcpHeader": null,  
"authSubject": {},  
"flexTemplateJson": null,  
"flexSchema": null,  
"quickReplyConfig": null,  
"category": "knowledge_base",  
"displayCopyByLocale": null,  
"outputRenderTemplateByLocale": null,  
"toolDisplayMode": null,  
"groups": [  
  {  
    "id": "550e8400-e29b-41d4-a716-446655440000"  
  }  
]  
}
```

Code Examples

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": null,
  "displayName": "example_string",
  "description": "example_string",
  "prompt": "example_string",
  "toolType": {},
  "apiUrl": null,
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "example_string",
  "mcpUrl": "example_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": null,
  "displayName": "example_string",
  "description": "example_string",
  "prompt": "example_string",
  "toolType": {},
  "apiUrl": null,
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "example_string",
  "mcpUrl": "example_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.put("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/", data, config)
```

```
.then(response => {  
  console.log('Successfully retrieved response:');  
  console.log(response.data);  
})  
.catch(error => {  
  console.error('Request error:');  
  console.error(error.response?.data || error.message);  
});
```



```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": null,
    "displayName": "example_string",
    "description": "example_string",
    "prompt": "example_string",
    "toolType": {},
    "apiUrl": null,
    "httpMethod": null,
    "rawHeaders": null,
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "example_string",
    "mcpUrl": "example_string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
    "category": "knowledge_base",
    "displayCopyByLocale": null,
    "outputRenderTemplateByLocale": null,
    "toolDisplayMode": null,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
```

```
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/tools/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": null,
            "displayName": "example_string",
            "description": "example_string",
            "prompt": "example_string",
            "toolType": {},
            "apiUrl": null,
            "httpMethod": null,
            "rawHeaders": null,
            "rawQueryParams": null,
            "rawParametersSchema": null,
            "functionCode": "example_string",
            "mcpUrl": "example_string",
            "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
            "mcpAllowedTools": null,
            "rawMcpHeader": null,
            "authSubject": {},
            "flexTemplateJson": null,
            "flexSchema": null,
            "quickReplyConfig": null,
            "category": "knowledge_base",
            "displayCopyByLocale": null,
            "outputRenderTemplateByLocale": null,
            "toolDisplayMode": null,
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]);
}
```

```
$data = json_decode($response->getBody(), true);
echo "Successfully retrieved response:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name"?: // May have different types (Optional)
    string // Tool name can only contain letters, numbers, underscores (_), and
    hyphens (-). Not required for MCP type tools. This field is used for API type
    tools by the LLM. (Optional)
  "displayName": string // A name that can contain any characters, used for user
  configuration and management.
  "description"?: string // Tool description displayed to users. Not required for
  MCP type tools. (Optional)
  "prompt"?: string // Tool description/prompt for LLM use. If empty, the system
  will use the description field. Not required for MCP type tools. (Optional)
  "toolType"?: // Optional
    {
    }
  "apiUrl"?: // May have different types (Optional)
    string (uri) // Optional
  "httpMethod"?: // May have different types (Optional)
    string (enum: get, post, put, delete, patch) // Optional
  "rawHeaders"?: object // Optional
  "rawHeadersMasked": object
  "rawQueryParams"?: object // Query string parameters appended to every request
  of this API tool. Values support the same {{variable}} template rendering as
  raw_headers. On key collision with LLM-generated arguments, these values win –
  use this field for API keys that must travel in the query string. (Optional)
  "rawQueryParamsMasked": object
  "rawParametersSchema"?: object // Optional
```

```

"functionCode"?: string // Optional
"organization": string (uuid)
"createdBy": string (uuid)
"isGlobal": boolean // If set as global tool, all organizations can use this
tool
"mcpUrl"?: string // Optional
"mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (Optional)
"mcpAllowedTools"?: object // Optional
"rawMcpHeader"?: object // Default headers used when a Contact has no
corresponding MCP Credential (Optional)
"rawMcpHeaderMasked": object
"authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
{
}
"flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (Optional)
"flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (Optional)
"quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (Optional)
"category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
"displayCopyByLocale"?: object // Optional

```

```

"outputRenderTemplateByLocale"?: object // optional
"toolDisplayMode"?: // may have different types (optional)
string (enum: full, compact, hidden) // optional
"groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string",
  "displayName": "response_string",
  "description": "response_string",
  "prompt": "response_string",
  "toolType": {},
  "apiUrl": "https://example.com/file.jpg",
  "httpMethod": "get",
  "rawHeaders": null,
  "rawHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example_response_name",
    "description": "example_response_description"
  },
  "rawQueryParams": null,
  "rawQueryParamsMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example_response_name",
    "description": "example_response_description"
  },
  "rawParametersSchema": null,
  "functionCode": "response_string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "isGlobal": false,
  "mcpUrl": "response_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",

```

```
"mcpAllowedTools": null,
"rawMcpHeader": null,
"rawMcpHeaderMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example_response_name",
  "description": "example_response_description"
},
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string"
  }
],
"createdAt": "response_string",
"updatedAt": "response_string",
"canUpdate": false,
"canDelete": false
}
```

Update Tool

PUT

</api/v1/tools/category-copy-overrides/>

Request

Request Parameters

Field	Type	Required	Description
name	object	No	Tool name can only contain letters, numbers, underscores (_), and

Field	Type	Required	Description
			hyphens (-). Not required for MCP type tools. This field is used for API type tools by the LLM.
displayName	string	Yes	A name that can contain any characters, used for user configuration and management.
description	string	No	Tool description displayed to users. Not required for MCP type tools.
prompt	string	No	Tool description/prompt for LLM use. If empty, the system will use the description field. Not required for MCP type tools.
toolType	object	No	
apiUrl	object	No	
httpMethod	object	No	
rawHeaders	object	No	
rawQueryParams	object	No	Query string parameters appended to every request of this API tool. Values support the same {{variable}} template rendering as raw_headers. On key collision with LLM-generated arguments, these values ...
rawParametersSchema	object	No	
functionCode	string	No	
mcpUrl	string	No	
mcpRegistry	string (uuid)	No	Registry backing this MCP tool. Null for manual-URL MCP tools.
mcpAllowedTools	object	No	

Field	Type	Required	Description
rawMcpHeader	object	No	Default headers used when a Contact has no corresponding MCP Credential
authSubject	object	No	Shared uses the tool level token plus per contact credentials; contact requires per contact authentication only; caller mints a short-lived token for the conversation-bound member (falls back to per c...
flexTemplateJson	object	No	FlexMessage JSON template. Required for FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble / carousel / box / text / image / button etc.).
flexSchema	object	No	AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be e...
quickReplyConfig	object	No	Authoring config for QUICK_REPLY tools. Structured JSON describing the chip buttons: static list or dynamic (LLM-generated) mode, each field either a fixed value or an LLM prompt.
category	string (enum: knowledge_base, web_search, web_fetch, code_execution,	No	knowledge_base : Knowledge base retrieval ; web_search : Web search ; web_fetch : Web page

Field	Type	Required	Description
	file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other)		fetch ; code_execution : Code execution ; file_processing : File processing ; image_video_generation : ...
displayCopyByLocale	object	No	
outputRenderTemplateByLocale	object	No	
toolDisplayMode	object	No	
groups	array[IdName]	No	Groups this tool is assigned to. Non-owner members must pick at least one group they belong to.

Request Schema Example

```
{
  "name"? : // May have different types (Optional)
    string // Tool name can only contain letters, numbers, underscores (_), and
    hyphens (-). Not required for MCP type tools. This field is used for API type
    tools by the LLM. (Optional)
  "displayName": string // A name that can contain any characters, used for user
  configuration and management.
  "description"? : string // Tool description displayed to users. Not required for
  MCP type tools. (Optional)
  "prompt"? : string // Tool description/prompt for LLM use. If empty, the system
  will use the description field. Not required for MCP type tools. (Optional)
  "toolType"? : // Optional
    {
    }
  "apiUrl"? : // May have different types (Optional)
    string (uri) // Optional
  "httpMethod"? : // May have different types (Optional)
    string (enum: get, post, put, delete, patch) // Optional
  "rawHeaders"? : object // Optional
  "rawQueryParams"? : object // Query string parameters appended to every request
  of this API tool. Values support the same {{variable}} template rendering as
  raw_headers. On key collision with LLM-generated arguments, these values win –
  use this field for API keys that must travel in the query string. (Optional)
  "rawParametersSchema"? : object // Optional
  "functionCode"? : string // Optional
}
```

```

    "mcpUrl"?: string // Optional
    "mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (Optional)
    "mcpAllowedTools"?: object // Optional
    "rawMcpHeader"?: object // Default headers used when a Contact has no
corresponding MCP Credential (Optional)
    "authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
{
}
    "flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (Optional)
    "flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (Optional)
    "quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (Optional)
    "category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
    "displayCopyByLocale"?: object // Optional
    "outputRenderTemplateByLocale"?: object // optional
    "toolDisplayMode"?: // may have different types (optional)
string (enum: full, compact, hidden) // optional
    "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (Optional)
{

```

```
    "id": string (uuid)
  }
]
}
```

Request Example

```
{
  "name": null,
  "displayName": "example_string",
  "description": "example_string",
  "prompt": "example_string",
  "toolType": {},
  "apiUrl": null,
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "example_string",
  "mcpUrl": "example_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

Code Examples

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/tools/category-copy-overrides/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": null,
  "displayName": "example_string",
  "description": "example_string",
  "prompt": "example_string",
  "toolType": {},
  "apiUrl": null,
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "example_string",
  "mcpUrl": "example_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": null,
  "displayName": "example_string",
  "description": "example_string",
  "prompt": "example_string",
  "toolType": {},
  "apiUrl": null,
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "example_string",
  "mcpUrl": "example_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.put("https://api.maiagent.ai/api/v1/tools/category-copy-overrides/",
data, config)
```

```
.then(response => {  
  console.log('Successfully retrieved response:');  
  console.log(response.data);  
})  
.catch(error => {  
  console.error('Request error:');  
  console.error(error.response?.data || error.message);  
});
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/category-copy-overrides/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": null,
    "displayName": "example_string",
    "description": "example_string",
    "prompt": "example_string",
    "toolType": {},
    "apiUrl": null,
    "httpMethod": null,
    "rawHeaders": null,
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "example_string",
    "mcpUrl": "example_string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
    "category": "knowledge_base",
    "displayCopyByLocale": null,
    "outputRenderTemplateByLocale": null,
    "toolDisplayMode": null,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
```



```
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/tools/category-
copy-overrides/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": null,
            "displayName": "example_string",
            "description": "example_string",
            "prompt": "example_string",
            "toolType": {},
            "apiUrl": null,
            "httpMethod": null,
            "rawHeaders": null,
            "rawQueryParams": null,
            "rawParametersSchema": null,
            "functionCode": "example_string",
            "mcpUrl": "example_string",
            "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
            "mcpAllowedTools": null,
            "rawMcpHeader": null,
            "authSubject": {},
            "flexTemplateJson": null,
            "flexSchema": null,
            "quickReplyConfig": null,
            "category": "knowledge_base",
            "displayCopyByLocale": null,
            "outputRenderTemplateByLocale": null,
            "toolDisplayMode": null,
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]);
}
```

```
$data = json_decode($response->getBody(), true);
echo "Successfully retrieved response:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name"?: // May have different types (Optional)
    string // Tool name can only contain letters, numbers, underscores (_), and
    hyphens (-). Not required for MCP type tools. This field is used for API type
    tools by the LLM. (Optional)
  "displayName": string // A name that can contain any characters, used for user
  configuration and management.
  "description"?: string // Tool description displayed to users. Not required for
  MCP type tools. (Optional)
  "prompt"?: string // Tool description/prompt for LLM use. If empty, the system
  will use the description field. Not required for MCP type tools. (Optional)
  "toolType"?: // Optional
    {
    }
  "apiUrl"?: // May have different types (Optional)
    string (uri) // Optional
  "httpMethod"?: // May have different types (Optional)
    string (enum: get, post, put, delete, patch) // Optional
  "rawHeaders"?: object // Optional
  "rawHeadersMasked": object
  "rawQueryParams"?: object // Query string parameters appended to every request
  of this API tool. Values support the same {{variable}} template rendering as
  raw_headers. On key collision with LLM-generated arguments, these values win –
  use this field for API keys that must travel in the query string. (Optional)
  "rawQueryParamsMasked": object
  "rawParametersSchema"?: object // Optional
```

```

"functionCode"?: string // Optional
"organization": string (uuid)
"createdBy": string (uuid)
"isGlobal": boolean // If set as global tool, all organizations can use this
tool
"mcpUrl"?: string // Optional
"mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (Optional)
"mcpAllowedTools"?: object // Optional
"rawMcpHeader"?: object // Default headers used when a Contact has no
corresponding MCP Credential (Optional)
"rawMcpHeaderMasked": object
"authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
{
}
"flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (Optional)
"flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (Optional)
"quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (Optional)
"category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
"displayCopyByLocale"?: object // Optional

```

```

"outputRenderTemplateByLocale"?: object // optional
"toolDisplayMode"?: // may have different types (optional)
string (enum: full, compact, hidden) // optional
"groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string",
  "displayName": "response_string",
  "description": "response_string",
  "prompt": "response_string",
  "toolType": {},
  "apiUrl": "https://example.com/file.jpg",
  "httpMethod": "get",
  "rawHeaders": null,
  "rawHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example_response_name",
    "description": "example_response_description"
  },
  "rawQueryParams": null,
  "rawQueryParamsMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example_response_name",
    "description": "example_response_description"
  },
  "rawParametersSchema": null,
  "functionCode": "response_string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "isGlobal": false,
  "mcpUrl": "response_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",

```

```
"mcpAllowedTools": null,
"rawMcpHeader": null,
"rawMcpHeaderMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example_response_name",
  "description": "example_response_description"
},
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string"
  }
],
"createdAt": "response_string",
"updatedAt": "response_string",
"canUpdate": false,
"canDelete": false
}
```

Partially Update Tool

PATCH

`/api/v1/tools/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this tool.

Request

Request Parameters

Field	Type	Required	Description
name	object	No	Tool name can only contain letters, numbers, underscores (_), and hyphens (-). Not required for MCP type tools. This field is used for API type tools by the LLM.
displayName	string	No	A name that can contain any characters, used for user configuration and management.
description	string	No	Tool description displayed to users. Not required for MCP type tools.
prompt	string	No	Tool description/prompt for LLM use. If empty, the system will use the description field. Not required for MCP type tools.
toolType	object	No	
apiUrl	object	No	
httpMethod	object	No	
rawHeaders	object	No	
rawQueryParams	object	No	Query string parameters appended to every request of this API tool. Values support the same {{variable}} template rendering as raw_headers. On key collision with LLM-generated arguments, these values ...
rawParametersSchema	object	No	

Field	Type	Required	Description
functionCode	string	No	
mcpUrl	string	No	
mcpRegistry	string (uuid)	No	Registry backing this MCP tool. Null for manual-URL MCP tools.
mcpAllowedTools	object	No	
rawMcpHeader	object	No	Default headers used when a Contact has no corresponding MCP Credential
authSubject	object	No	Shared uses the tool level token plus per contact credentials; contact requires per contact authentication only; caller mints a short-lived token for the conversation-bound member (falls back to per c...
flexTemplateJson	object	No	FlexMessage JSON template. Required for FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble / carousel / box / text / image / button etc.).
flexSchema	object	No	AI dynamic field schema for FlexMessage. JSON object whose keys are field paths parsed from flex_template_json (e.g. text / image url / button label / button uri / button message). Each value may be e...
quickReplyConfig	object	No	Authoring config for QUICK_REPLY tools. Structured JSON describing the chip buttons: static list or

Field	Type	Required	Description
			dynamic (LLM-generated) mode, each field either a fixed value or an LLM prompt.
category	string (enum: knowledge_base, web_search, web_fetch, code_execution, file_processing, image_video_generation, database_query, third_party_integration, browser_operation, agent_collaboration, other)	No	knowledge_base : Knowledge base retrieval ; web_search : Web search ; web_fetch : Web page fetch ; code_execution : Code execution ; file_processing : File processing ; image_video_generation : ...
displayCopyByLocale	object	No	
outputRenderTemplateByLocale	object	No	
toolDisplayMode	object	No	
groups	array[IdName]	No	Groups this tool is assigned to. Non-owner members must pick at least one group they belong to.

Request Schema Example

```
{
  "name"? : // May have different types (Optional)
    string // Tool name can only contain letters, numbers, underscores (_), and
    hyphens (-). Not required for MCP type tools. This field is used for API type
    tools by the LLM.
    | null
  "displayName"? : string // A name that can contain any characters, used for user
  configuration and management. (Optional)
  "description"? : string // Tool description displayed to users. Not required for
  MCP type tools. (Optional)
  "prompt"? : string // Tool description/prompt for LLM use. If empty, the system
  will use the description field. Not required for MCP type tools. (Optional)
  "toolType"? : // Optional
    {
    }
  "apiUrl"? : // May have different types (Optional)
  string (uri) // Optional
```

```

"httpMethod"?: // May have different types (Optional)
string (enum: get, post, put, delete, patch) // Optional
"rawHeaders"?: object // Optional
"rawQueryParams"?: object // Query string parameters appended to every request
of this API tool. Values support the same {{variable}} template rendering as
raw_headers. On key collision with LLM-generated arguments, these values win –
use this field for API keys that must travel in the query string. (Optional)
"rawParametersSchema"?: object // Optional
"functionCode"?: string // Optional
"mcpUrl"?: string // Optional
"mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (Optional)
"mcpAllowedTools"?: object // Optional
"rawMcpHeader"?: object // Default headers used when a Contact has no
corresponding MCP Credential (Optional)
"authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
{
}
"flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (Optional)
"flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (Optional)
"quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (Optional)
"category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation

```

```

* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
  "displayCopyByLocale"?: object // Optional
  "outputRenderTemplateByLocale"?: object // optional
  "toolDisplayMode"?: // may have different types (optional)
  string (enum: full, compact, hidden) // optional
  "groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (Optional)
    {
      "id": string (uuid)
    }
  ]
}

```

Request Example

```

{
  "name": null,
  "displayName": "example_string",
  "description": "example_string",
  "prompt": "example_string",
  "toolType": {},
  "apiUrl": null,
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "example_string",
  "mcpUrl": "example_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}

```

```
]
}
```

Code Examples

```
# API call example (Shell)
curl -X PATCH "https://api.maiaagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": null,
  "displayName": "example_string",
  "description": "example_string",
  "prompt": "example_string",
  "toolType": {},
  "apiUrl": null,
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "example_string",
  "mcpUrl": "example_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": null,
  "displayName": "example_string",
  "description": "example_string",
  "prompt": "example_string",
  "toolType": {},
  "apiUrl": null,
  "httpMethod": null,
  "rawHeaders": null,
  "rawQueryParams": null,
  "rawParametersSchema": null,
  "functionCode": "example_string",
  "mcpUrl": "example_string",
  "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
  "mcpAllowedTools": null,
  "rawMcpHeader": null,
  "authSubject": {},
  "flexTemplateJson": null,
  "flexSchema": null,
  "quickReplyConfig": null,
  "category": "knowledge_base",
  "displayCopyByLocale": null,
  "outputRenderTemplateByLocale": null,
  "toolDisplayMode": null,
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.patch("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/", data, config)
```

```
.then(response => {  
  console.log('Successfully retrieved response:');  
  console.log(response.data);  
})  
.catch(error => {  
  console.error('Request error:');  
  console.error(error.response?.data || error.message);  
});
```



```

import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": null,
    "displayName": "example_string",
    "description": "example_string",
    "prompt": "example_string",
    "toolType": {},
    "apiUrl": null,
    "httpMethod": null,
    "rawHeaders": null,
    "rawQueryParams": null,
    "rawParametersSchema": null,
    "functionCode": "example_string",
    "mcpUrl": "example_string",
    "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
    "mcpAllowedTools": null,
    "rawMcpHeader": null,
    "authSubject": {},
    "flexTemplateJson": null,
    "flexSchema": null,
    "quickReplyConfig": null,
    "category": "knowledge_base",
    "displayCopyByLocale": null,
    "outputRenderTemplateByLocale": null,
    "toolDisplayMode": null,
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")

```

```
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": null,
            "displayName": "example_string",
            "description": "example_string",
            "prompt": "example_string",
            "toolType": {},
            "apiUrl": null,
            "httpMethod": null,
            "rawHeaders": null,
            "rawQueryParams": null,
            "rawParametersSchema": null,
            "functionCode": "example_string",
            "mcpUrl": "example_string",
            "mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
            "mcpAllowedTools": null,
            "rawMcpHeader": null,
            "authSubject": {},
            "flexTemplateJson": null,
            "flexSchema": null,
            "quickReplyConfig": null,
            "category": "knowledge_base",
            "displayCopyByLocale": null,
            "outputRenderTemplateByLocale": null,
            "toolDisplayMode": null,
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]);
```

```

        $data = json_decode($response->getBody(), true);
        echo "Successfully retrieved response:\n";
        print_r($data);
    } catch (Exception $e) {
        echo 'Request error: ' . $e->getMessage();
    }
?>

```

Response

Status Code: 200

Response Schema Example

```

{
  "id": string (uuid)
  "name"?: // May have different types (Optional)
    string // Tool name can only contain letters, numbers, underscores (_), and
    hyphens (-). Not required for MCP type tools. This field is used for API type
    tools by the LLM. (Optional)
  "displayName": string // A name that can contain any characters, used for user
  configuration and management.
  "description"?: string // Tool description displayed to users. Not required for
  MCP type tools. (Optional)
  "prompt"?: string // Tool description/prompt for LLM use. If empty, the system
  will use the description field. Not required for MCP type tools. (Optional)
  "toolType"?: // Optional
    {
    }
  "apiUrl"?: // May have different types (Optional)
    string (uri) // Optional
  "httpMethod"?: // May have different types (Optional)
    string (enum: get, post, put, delete, patch) // Optional
  "rawHeaders"?: object // Optional
  "rawHeadersMasked": object
  "rawQueryParams"?: object // Query string parameters appended to every request
  of this API tool. Values support the same {{variable}} template rendering as
  raw_headers. On key collision with LLM-generated arguments, these values win –
  use this field for API keys that must travel in the query string. (Optional)
  "rawQueryParamsMasked": object

```

```

"rawParametersSchema"?: object // Optional
"functionCode"?: string // Optional
"organization": string (uuid)
"createdBy": string (uuid)
"isGlobal": boolean // If set as global tool, all organizations can use this
tool
"mcpUrl"?: string // Optional
"mcpRegistry"?: string (uuid) // Registry backing this MCP tool. Null for
manual-URL MCP tools. (Optional)
"mcpAllowedTools"?: object // Optional
"rawMcpHeader"?: object // Default headers used when a Contact has no
corresponding MCP Credential (Optional)
"rawMcpHeaderMasked": object
"authSubject"?: // Shared uses the tool level token plus per contact
credentials; contact requires per contact authentication only; caller mints a
short-lived token for the conversation-bound member (falls back to per contact
credentials when the conversation has no bound member). MCP tools only.

* `shared` - Shared
* `contact` - Contact
* `caller` - Caller (Optional)
{
}
"flexTemplateJson"?: object // FlexMessage JSON template. Required for
FLEX_MESSAGE tools. Stores the full LINE FlexMessage JSON structure (bubble /
carousel / box / text / image / button etc.). (Optional)
"flexSchema"?: object // AI dynamic field schema for FlexMessage. JSON object
whose keys are field paths parsed from flex_template_json (e.g. text / image url
/ button label / button uri / button message). Each value may be either a plain
string (the AI prompt) or an object carrying additional metadata such as a field
tag, e.g. {"prompt": "...", "tag": "title"}. Empty values fall back to the
template default. (Optional)
"quickReplyConfig"?: object // Authoring config for QUICK_REPLY tools.
Structured JSON describing the chip buttons: static list or dynamic (LLM-
generated) mode, each field either a fixed value or an LLM prompt. (Optional)
"category"?: string (enum: knowledge_base, web_search, web_fetch,
code_execution, file_processing, image_video_generation, database_query,
third_party_integration, browser_operation, agent_collaboration, other) // *
`knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)

```

```

"displayCopyByLocale"?: object // Optional
"outputRenderTemplateByLocale"?: object // optional
"toolDisplayMode"?: // may have different types (optional)
string (enum: full, compact, hidden) // optional
"groups"?: [ // Groups this tool is assigned to. Non-owner members must pick at
least one group they belong to. (Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string",
  "displayName": "response_string",
  "description": "response_string",
  "prompt": "response_string",
  "toolType": {},
  "apiUrl": "https://example.com/file.jpg",
  "httpMethod": "get",
  "rawHeaders": null,
  "rawHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example_response_name",
    "description": "example_response_description"
  },
  "rawQueryParams": null,
  "rawQueryParamsMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example_response_name",
    "description": "example_response_description"
  },
  "rawParametersSchema": null,
  "functionCode": "response_string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "isGlobal": false,
  "mcpUrl": "response_string",
}

```

```
"mcpRegistry": "550e8400-e29b-41d4-a716-446655440000",
"mcpAllowedTools": null,
"rawMcpHeader": null,
"rawMcpHeaderMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "example_response_name",
  "description": "example_response_description"
},
"authSubject": {},
"flexTemplateJson": null,
"flexSchema": null,
"quickReplyConfig": null,
"category": "knowledge_base",
"displayCopyByLocale": null,
"outputRenderTemplateByLocale": null,
"toolDisplayMode": "full",
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string"
  }
],
"createdAt": "response_string",
"updatedAt": "response_string",
"canUpdate": false,
"canDelete": false
}
```

Delete Tool

DELETE

`/api/v1/tools/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this tool.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Create Connector

POST

/api/v1/connectors/

Request

Request Parameters

Field	Type	Required	Description
mcpRegistryId	string (uuid)	Yes	
displayName	string	No	Custom display name for this connector (defaults to MCP Registry name)
description	string	No	Custom description for this connector (defaults to MCP Registry description)
defaultCategory	object	No	Org-level override of the MCP Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category. knowledge_base : Knowledge base retrieval ; web_search : We...
defaultDisplayGroupNameByLocale	object	No	Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"": str}. The headline for the conversatio...
defaultToolIcon	string	No	Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon.
authHeaders	object	No	
allowedContextDataKeys	object	No	Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.}}. An empty list

Field	Type	Required	Description
			disables the feature for this connector: the placeholder is left verbat...
exposedMcpTools	object	No	Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only ...
isEnabled	boolean	No	Whether this connector is enabled for contacts in the organization

Request Schema Example

```
{
  "mcpRegistryId": string (uuid)
  "displayName"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
  "description"?: string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
  "defaultCategory"?: // May have different types (Optional)
    string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

  * `knowledge_base` - Knowledge base retrieval
  * `web_search` - Web search
  * `web_fetch` - Web page fetch
  * `code_execution` - Code execution
  * `file_processing` - File processing
  * `image_video_generation` - Image/video generation
  * `database_query` - Database query
  * `third_party_integration` - Third-party integration
  * `browser_operation` - Browser operation
  * `agent_collaboration` - Agent collaboration
  * `other` - Other (catch-all) (Optional)
  "defaultDisplayNameByLocale"?: object // Locale-keyed server-level default
group headline shown in chat for this connector's dynamic MCP tools (those
without an individual Tool row). Shape: {"<locale>": str}. The headline for the
conversation locale is resolved server-side (see
whizchat.tools.services.tool_category.resolve_localized_text): requested locale -
> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as
displayName. Empty = frontend falls back to its name-based heuristic. An
```

```

individual Tool row overrides this. (optional)
  "defaultToolIcon"?: string // Server-level default icon identifier (a frontend
icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty =
frontend uses its default MCP icon. Streamed as toolIcon. (optional)
  "authHeaders"?: object // Optional
  "allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this
connector may render into auth_headers through {{context_data.<key>}}. An empty
list disables the feature for this connector: the placeholder is left verbatim,
exactly as before the feature existed. Keys are matched literally and are case-
sensitive; at most 20 keys of letters, digits and underscores. (Optional)
  "exposedMcpTools"?: object // Allowlist of MCP tool names this connector
exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the
server tools. Use this to limit a connector to a safe subset (e.g. read-only
tools). (Optional)
  "isEnabled"?: boolean // Whether this connector is enabled for contacts in the
organization (Optional)
}

```

Request Example

```

{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "example_string",
  "description": "example_string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
}

```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/connectors/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "example_string",
  "description": "example_string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
}'

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "example_string",
  "description": "example_string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
};

axios.post("https://api.maiagent.ai/api/v1/connectors/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "example_string",
    "description": "example_string",
    "defaultCategory": null,
    "defaultDisplayGroupNameByLocale": null,
    "defaultToolIcon": "Example string",
    "authHeaders": null,
    "allowedContextDataKeys": null,
    "exposedMcpTools": null,
    "isEnabled": true
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/connectors/",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
            "displayName": "example_string",
            "description": "example_string",
            "defaultCategory": null,
            "defaultDisplayGroupNameByLocale": null,
            "defaultToolIcon": "Example string",
            "authHeaders": null,
            "allowedContextDataKeys": null,
            "exposedMcpTools": null,
            "isEnabled": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
    alphanumeric, underscore and hyphen allowed.
    "displayName?": string // Human-readable display name (defaults to name if
    empty) (Optional)
    "description?": string // Brief description of the MCP service (Optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?:// May have different types (Optional)
    string (uri) // Manual override for the OAuth authorization server URL. Leave
    empty to auto-discover from the MCP server (RFC 9728 protected resource
    metadata). Only fill in when auto-discovery fails or the server advertises
    incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
    api.supabase.com) (Optional)
    "logoUrl"?:// May have different types (Optional)
    string (uri) // CDN URL for the MCP service logo (for frontend rendering)
    (Optional)
    "isActive?": boolean // Whether this MCP service is currently available for
    use (Optional)
    "isGlobal?": boolean // Global registries are maintained by the platform and
    available to all organizations. (Optional)
    "authType?": // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
  "mcpRegistryId": string (uuid)
  "name": string
  "displayName?": string // Custom display name for this connector (defaults to
  MCP Registry name) (Optional)
  "description?": string // Custom description for this connector (defaults to
  MCP Registry description) (Optional)
  "defaultCategory?": // May have different types (Optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
  file_processing, image_video_generation, database_query, third_party_integration,
  browser_operation, agent_collaboration, other) // Org-level override of the MCP
```

Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category.

- * `knowledge\_base` - Knowledge base retrieval
- * `web\_search` - Web search
- * `web\_fetch` - Web page fetch
- * `code\_execution` - Code execution
- * `file\_processing` - File processing
- * `image\_video\_generation` - Image/video generation
- * `database\_query` - Database query
- * `third\_party\_integration` - Third-party integration
- * `browser\_operation` - Browser operation
- * `agent\_collaboration` - Agent collaboration
- * `other` - Other (catch-all) (Optional)

"defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"<locale>": str}. The headline for the conversation locale is resolved server-side (see whizchat.tools.services.tool_category.resolve_localized_text): requested locale -> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as displayGroupName. Empty = frontend falls back to its name-based heuristic. An individual Tool row overrides this. (optional)

"defaultToolIcon"?: string // Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon. (optional)

"effectiveDescription": string

"remoteMcpUrl": string

"oauthServerUrl": string

"logoUrl": string

"authType": string

"authHeaders"?: object // Optional

"authHeadersMasked": object

"allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.<key>}}. An empty list disables the feature for this connector: the placeholder is left verbatim, exactly as before the feature existed. Keys are matched literally and are case-sensitive; at most 20 keys of letters, digits and underscores. (Optional)

"exposedMcpTools"?: object // Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only tools). (Optional)

"isEnabled"?: boolean // Whether this connector is enabled for contacts in the organization (Optional)

"organization": string (uuid)

"createdBy": string (uuid)

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

Response Example

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string",
    "displayName": "response_string",
    "description": "response_string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "response_string",
    "updatedAt": "response_string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string",
  "displayName": "response_string",
  "description": "response_string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "response_string",
  "remoteMcpUrl": "response_string",
  "oauthServerUrl": "response_string",
  "logoUrl": "response_string",
  "authType": "response_string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example_response_name",
    "description": "example_response_description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "response_string",
  "updatedAt": "response_string"
}
```

List Connectors

GET

/api/v1/connectors/

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/connectors/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/connectors/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/connectors/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "id": string (uuid)
    "displayName"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
    "mcpRegistryName": string
    "mcpRegistryId": string
    "mcpRegistryIsGlobal": boolean
    "logoUrl": string
```

```

    "defaultCategory"?: // May have different types (Optional)
    string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)
    "isEnabled"?: boolean // Whether this connector is enabled for contacts in
the organization (Optional)
    "authHeadersMasked": object
    "allowedContextDataKeys": object // Allowlist of webchat contextData keys
this connector may render into auth_headers through {{context_data.<key>}}. An
empty list disables the feature for this connector: the placeholder is left
verbatim, exactly as before the feature existed. Keys are matched literally and
are case-sensitive; at most 20 keys of letters, digits and underscores.
    "authorizedMembersCount": integer
    "createdBy":
    {
        "id": string (uuid)
        "name": string
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
}
]

```

Response Example

```

[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "response_string",
    "mcpRegistryName": "response_string",
    "mcpRegistryId": "response_string",
    "mcpRegistryIsGlobal": false,
    "logoUrl": "response_string",
    "defaultCategory": "knowledge_base",
  }
]

```

```
"isEnabled": false,
"authHeadersMasked": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_example_name",
  "description": "response_example_description"
},
"authorizedMembersCount": 456,
"createdBy": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string"
},
"createdAt": "response_string",
"updatedAt": "response_string"
}
]
```

Retrieve Connector

GET

`/api/v1/connectors/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
    alphanumeric, underscore and hyphen allowed.
```

```

    "displayName?": string // Human-readable display name (defaults to name if
empty) (Optional)
    "description?": string // Brief description of the MCP service (Optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (Optional)
        string (uri) // Manual override for the OAuth authorization server URL. Leave
empty to auto-discover from the MCP server (RFC 9728 protected resource
metadata). Only fill in when auto-discovery fails or the server advertises
incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
api.supabase.com) (Optional)
    "logoUrl"?: // May have different types (Optional)
        string (uri) // CDN URL for the MCP service logo (for frontend rendering)
(Optional)
    "isActive?": boolean // Whether this MCP service is currently available for
use (Optional)
    "isGlobal?": boolean // Global registries are maintained by the platform and
available to all organizations. (Optional)
    "authType"?: // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
    }
    "mcpRegistryId": string (uuid)
    "name": string
    "displayName?": string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
    "description?": string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
    "defaultCategory"?: // May have different types (Optional)
        string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation

```

```

* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)

"defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default
group headline shown in chat for this connector's dynamic MCP tools (those
without an individual Tool row). Shape: {"<locale>": str}. The headline for the
conversation locale is resolved server-side (see
whizchat.tools.services.tool_category.resolve_localized_text): requested locale -
> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as
displayGroupName. Empty = frontend falls back to its name-based heuristic. An
individual Tool row overrides this. (optional)

"defaultToolIcon"?: string // Server-level default icon identifier (a frontend
icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty =
frontend uses its default MCP icon. Streamed as toolIcon. (optional)

"effectiveDescription": string
"remoteMcpUrl": string
"oauthServerUrl": string
"logoUrl": string
"authType": string
"authHeaders"?: object // Optional
"authHeadersMasked": object

"allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this
connector may render into auth_headers through {{context_data.<key>}}. An empty
list disables the feature for this connector: the placeholder is left verbatim,
exactly as before the feature existed. Keys are matched literally and are case-
sensitive; at most 20 keys of letters, digits and underscores. (Optional)

"exposedMcpTools"?: object // Allowlist of MCP tool names this connector
exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the
server tools. Use this to limit a connector to a safe subset (e.g. read-only
tools). (Optional)

"isEnabled"?: boolean // Whether this connector is enabled for contacts in the
organization (Optional)
"organization": string (uuid)
"createdBy": string (uuid)
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string",
    "displayName": "response_string",
    "description": "response_string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
  }
}

```



```

    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "response_string",
    "updatedAt": "response_string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string",
  "displayName": "response_string",
  "description": "response_string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "response_string",
  "remoteMcpUrl": "response_string",
  "oauthServerUrl": "response_string",
  "logoUrl": "response_string",
  "authType": "response_string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example_response_name",
    "description": "example_response_description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "response_string",
  "updatedAt": "response_string"
}

```

Retrieve Connector

GET

</api/v1/connectors/enablement-requests/>

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/connectors/enablement-requests/" \
  -H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/connectors/enablement-requests/",
config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/enablement-requests/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/connectors/enablement-requests/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
    alphanumeric, underscore and hyphen allowed.
    "displayName?": string // Human-readable display name (defaults to name if
```

```

empty) (Optional)
    "description"?: string // Brief description of the MCP service (Optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (Optional)
        string (uri) // Manual override for the OAuth authorization server URL. Leave
        empty to auto-discover from the MCP server (RFC 9728 protected resource
        metadata). Only fill in when auto-discovery fails or the server advertises
        incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
        api.supabase.com) (Optional)
    "logoUrl"?: // May have different types (Optional)
        string (uri) // CDN URL for the MCP service logo (for frontend rendering)
        (Optional)
    "isActive"?: boolean // Whether this MCP service is currently available for
    use (Optional)
    "isGlobal"?: boolean // Global registries are maintained by the platform and
    available to all organizations. (Optional)
    "authType"?: // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
}
"mcpRegistryId": string (uuid)
"name": string
"displayname"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
"description"?: string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
"defaultCategory"?: // May have different types (Optional)
    string (enum: knowledge_base, web_search, web_fetch, code_execution,
    file_processing, image_video_generation, database_query, third_party_integration,
    browser_operation, agent_collaboration, other) // Org-level override of the MCP
    Registry default category for this connector's dynamic tools. Empty = use the MCP
    Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration

```

```

* `other` - Other (catch-all) (Optional)
  "defaultDisplayNameByLocale"?: object // Locale-keyed server-level default
  group headline shown in chat for this connector's dynamic MCP tools (those
  without an individual Tool row). Shape: {"<locale>": str}. The headline for the
  conversation locale is resolved server-side (see
  whizchat.tools.services.tool_category.resolve_localized_text): requested locale -
  > settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as
  displayName. Empty = frontend falls back to its name-based heuristic. An
  individual Tool row overrides this. (optional)
  "defaultToolIcon"?: string // Server-level default icon identifier (a frontend
  icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty =
  frontend uses its default MCP icon. Streamed as toolIcon. (optional)
  "effectiveDescription": string
  "remoteMcpUrl": string
  "oauthServerUrl": string
  "logoUrl": string
  "authType": string
  "authHeaders"?: object // Optional
  "authHeadersMasked": object
  "allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this
  connector may render into auth_headers through {{context_data.<key>}}. An empty
  list disables the feature for this connector: the placeholder is left verbatim,
  exactly as before the feature existed. Keys are matched literally and are case-
  sensitive; at most 20 keys of letters, digits and underscores. (Optional)
  "exposedMcpTools"?: object // Allowlist of MCP tool names this connector
  exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the
  server tools. Use this to limit a connector to a safe subset (e.g. read-only
  tools). (Optional)
  "isEnabled"?: boolean // Whether this connector is enabled for contacts in the
  organization (Optional)
  "organization": string (uuid)
  "createdBy": string (uuid)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string",
    "displayName": "response_string",
    "description": "response_string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
  }
}

```

```

    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "response_string",
    "updatedAt": "response_string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string",
  "displayName": "response_string",
  "description": "response_string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "response_string",
  "remoteMcpUrl": "response_string",
  "oauthServerUrl": "response_string",
  "logoUrl": "response_string",
  "authType": "response_string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example_response_name",
    "description": "example_response_description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "response_string",
  "updatedAt": "response_string"
}

```

Update Connector

PUT

</api/v1/connectors/{id}>

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	

Request

Request Parameters

Field	Type	Required	Description
<code>mcpRegistryId</code>	string (uuid)	Yes	
<code>displayName</code>	string	No	Custom display name for this connector (defaults to MCP Registry name)
<code>description</code>	string	No	Custom description for this connector (defaults to MCP Registry description)
<code>defaultCategory</code>	object	No	Org-level override of the MCP Registry default category for this connector's dynamic tools. Empty = use the MCP Registry <code>default_category</code> . <code>knowledge_base</code> : Knowledge base retrieval ; <code>web_search</code> : We...
<code>defaultDisplayGroupNameByLocale</code>	object	No	Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: <code>{"": str}</code> . The headline for the conversatio...
<code>defaultToolIcon</code>	string	No	Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as <code>toolIcon</code> .
<code>authHeaders</code>	object	No	
<code>allowedContextDataKeys</code>	object	No	Allowlist of webchat <code>contextData</code> keys this connector may render

Field	Type	Required	Description
			into auth_headers through {{context_data.}}. An empty list disables the feature for this connector: the placeholder is left verbat...
exposedMcpTools	object	No	Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only ...
isEnabled	boolean	No	Whether this connector is enabled for contacts in the organization

Request Schema Example

```
{
  "mcpRegistryId": string (uuid)
  "displayName"?: string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
  "description"?: string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
  "defaultCategory"?: // May have different types (Optional)
    string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)

  "defaultDisplayNameByLocale"?: object // Locale-keyed server-level default
group headline shown in chat for this connector's dynamic MCP tools (those
without an individual Tool row). Shape: {"<locale>": str}. The headline for the
conversation locale is resolved server-side (see
whizchat.tools.services.tool_category.resolve_localized_text): requested locale -
```

```

> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as
displayGroupName. Empty = frontend falls back to its name-based heuristic. An
individual Tool row overrides this. (optional)
  "defaultToolIcon"?: string // Server-level default icon identifier (a frontend
icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty =
frontend uses its default MCP icon. Streamed as toolIcon. (optional)
  "authHeaders"?: object // Optional
  "allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this
connector may render into auth_headers through {{context_data.<key>}}. An empty
list disables the feature for this connector: the placeholder is left verbatim,
exactly as before the feature existed. Keys are matched literally and are case-
sensitive; at most 20 keys of letters, digits and underscores. (Optional)
  "exposedMcpTools"?: object // Allowlist of MCP tool names this connector
exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the
server tools. Use this to limit a connector to a safe subset (e.g. read-only
tools). (Optional)
  "isEnabled"?: boolean // Whether this connector is enabled for contacts in the
organization (Optional)
}

```

Request Example

```

{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "example_string",
  "description": "example_string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
}

```

Code Examples

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "example_string",
  "description": "example_string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
}'

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "example_string",
  "description": "example_string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
};

axios.put("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "example_string",
    "description": "example_string",
    "defaultCategory": null,
    "defaultDisplayGroupNameByLocale": null,
    "defaultToolIcon": "Example string",
    "authHeaders": null,
    "allowedContextDataKeys": null,
    "exposedMcpTools": null,
    "isEnabled": true
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
            "displayName": "example_string",
            "description": "example_string",
            "defaultCategory": null,
            "defaultDisplayGroupNameByLocale": null,
            "defaultToolIcon": "Example string",
            "authHeaders": null,
            "allowedContextDataKeys": null,
            "exposedMcpTools": null,
            "isEnabled": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
    alphanumeric, underscore and hyphen allowed.
    "displayName?": string // Human-readable display name (defaults to name if
    empty) (Optional)
    "description?": string // Brief description of the MCP service (Optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (Optional)
    string (uri) // Manual override for the OAuth authorization server URL. Leave
    empty to auto-discover from the MCP server (RFC 9728 protected resource
    metadata). Only fill in when auto-discovery fails or the server advertises
    incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
    api.supabase.com) (Optional)
    "logoUrl"?: // May have different types (Optional)
    string (uri) // CDN URL for the MCP service logo (for frontend rendering)
    (Optional)
    "isActive?": boolean // Whether this MCP service is currently available for
    use (Optional)
    "isGlobal?": boolean // Global registries are maintained by the platform and
    available to all organizations. (Optional)
    "authType"?: // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
  "mcpRegistryId": string (uuid)
  "name": string
  "displayName?": string // Custom display name for this connector (defaults to
  MCP Registry name) (Optional)
  "description?": string // Custom description for this connector (defaults to
  MCP Registry description) (Optional)
  "defaultCategory"?: // May have different types (Optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
  file_processing, image_video_generation, database_query, third_party_integration,
  browser_operation, agent_collaboration, other) // Org-level override of the MCP
```

Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category.

- * `knowledge\_base` - Knowledge base retrieval
- * `web\_search` - Web search
- * `web\_fetch` - Web page fetch
- * `code\_execution` - Code execution
- * `file\_processing` - File processing
- * `image\_video\_generation` - Image/video generation
- * `database\_query` - Database query
- * `third\_party\_integration` - Third-party integration
- * `browser\_operation` - Browser operation
- * `agent\_collaboration` - Agent collaboration
- * `other` - Other (catch-all) (Optional)

"defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"<locale>": str}. The headline for the conversation locale is resolved server-side (see whizchat.tools.services.tool_category.resolve_localized_text): requested locale -> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as displayGroupName. Empty = frontend falls back to its name-based heuristic. An individual Tool row overrides this. (optional)

"defaultToolIcon"?: string // Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon. (optional)

"effectiveDescription": string

"remoteMcpUrl": string

"oauthServerUrl": string

"logoUrl": string

"authType": string

"authHeaders"?: object // Optional

"authHeadersMasked": object

"allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.<key>}}. An empty list disables the feature for this connector: the placeholder is left verbatim, exactly as before the feature existed. Keys are matched literally and are case-sensitive; at most 20 keys of letters, digits and underscores. (Optional)

"exposedMcpTools"?: object // Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only tools). (Optional)

"isEnabled"?: boolean // Whether this connector is enabled for contacts in the organization (Optional)

"organization": string (uuid)

"createdBy": string (uuid)

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

Response Example

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string",
    "displayName": "response_string",
    "description": "response_string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "response_string",
    "updatedAt": "response_string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string",
  "displayName": "response_string",
  "description": "response_string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "response_string",
  "remoteMcpUrl": "response_string",
  "oauthServerUrl": "response_string",
  "logoUrl": "response_string",
  "authType": "response_string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example_response_name",
    "description": "example_response_description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "response_string",
  "updatedAt": "response_string"
}
```

Partially Update Connector

PATCH /api/v1/connectors/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	

Request

Request Parameters

Field	Type	Required	Description
mcpRegistryId	string (uuid)	No	
displayName	string	No	Custom display name for this connector (defaults to MCP Registry name)
description	string	No	Custom description for this connector (defaults to MCP Registry description)
defaultCategory	object	No	Org-level override of the MCP Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category. knowledge_base : Knowledge base retrieval ; web_search : We...
defaultDisplayGroupNameByLocale	object	No	Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual

Field	Type	Required	Description
			Tool row). Shape: {"": str}. The headline for the conversatio...
defaultToolIcon	string	No	Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon.
authHeaders	object	No	
allowedContextDataKeys	object	No	Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.}}. An empty list disables the feature for this connector: the placeholder is left verbat...
exposedMcpTools	object	No	Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only ...
isEnabled	boolean	No	Whether this connector is enabled for contacts in the organization

Request Schema Example

```
{
  "mcpRegistryId?": string (uuid) // Optional
  "displayName?": string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
  "description?": string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
  "defaultCategory"?: // May have different types (Optional)
    string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
```

```

* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation
* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)

"defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default
group headline shown in chat for this connector's dynamic MCP tools (those
without an individual Tool row). Shape: {"<locale>": str}. The headline for the
conversation locale is resolved server-side (see
whizchat.tools.services.tool_category.resolve_localized_text): requested locale -
> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as
displayGroupName. Empty = frontend falls back to its name-based heuristic. An
individual Tool row overrides this. (optional)

"defaultToolIcon"?: string // Server-level default icon identifier (a frontend
icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty =
frontend uses its default MCP icon. Streamed as toolIcon. (optional)

"authHeaders"?: object // Optional

"allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this
connector may render into auth_headers through {{context_data.<key>}}. An empty
list disables the feature for this connector: the placeholder is left verbatim,
exactly as before the feature existed. Keys are matched literally and are case-
sensitive; at most 20 keys of letters, digits and underscores. (Optional)

"exposedMcpTools"?: object // Allowlist of MCP tool names this connector
exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the
server tools. Use this to limit a connector to a safe subset (e.g. read-only
tools). (Optional)

"isEnabled"?: boolean // Whether this connector is enabled for contacts in the
organization (Optional)
}

```

Request Example

```

{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "example_string",
  "description": "example_string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,

```

```
"isEnabled": true
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiaagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "example_string",
  "description": "example_string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
}'

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "displayName": "example_string",
  "description": "example_string",
  "defaultCategory": null,
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Example string",
  "authHeaders": null,
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": true
};

axios.patch("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
    "displayName": "example_string",
    "description": "example_string",
    "defaultCategory": null,
    "defaultDisplayGroupNameByLocale": null,
    "defaultToolIcon": "Example string",
    "authHeaders": null,
    "allowedContextDataKeys": null,
    "exposedMcpTools": null,
    "isEnabled": true
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
            "displayName": "example_string",
            "description": "example_string",
            "defaultCategory": null,
            "defaultDisplayGroupNameByLocale": null,
            "defaultToolIcon": "Example string",
            "authHeaders": null,
            "allowedContextDataKeys": null,
            "exposedMcpTools": null,
            "isEnabled": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
    alphanumeric, underscore and hyphen allowed.
    "displayName?": string // Human-readable display name (defaults to name if
    empty) (Optional)
    "description?": string // Brief description of the MCP service (Optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?:// May have different types (Optional)
    string (uri) // Manual override for the OAuth authorization server URL. Leave
    empty to auto-discover from the MCP server (RFC 9728 protected resource
    metadata). Only fill in when auto-discovery fails or the server advertises
    incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
    api.supabase.com) (Optional)
    "logoUrl"?:// May have different types (Optional)
    string (uri) // CDN URL for the MCP service logo (for frontend rendering)
    (Optional)
    "isActive?": boolean // Whether this MCP service is currently available for
    use (Optional)
    "isGlobal?": boolean // Global registries are maintained by the platform and
    available to all organizations. (Optional)
    "authType?": // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
  "mcpRegistryId": string (uuid)
  "name": string
  "displayName?": string // Custom display name for this connector (defaults to
  MCP Registry name) (Optional)
  "description?": string // Custom description for this connector (defaults to
  MCP Registry description) (Optional)
  "defaultCategory?": // May have different types (Optional)
  string (enum: knowledge_base, web_search, web_fetch, code_execution,
  file_processing, image_video_generation, database_query, third_party_integration,
  browser_operation, agent_collaboration, other) // Org-level override of the MCP
```

Registry default category for this connector's dynamic tools. Empty = use the MCP Registry default_category.

- * `knowledge\_base` - Knowledge base retrieval
- * `web\_search` - Web search
- * `web\_fetch` - Web page fetch
- * `code\_execution` - Code execution
- * `file\_processing` - File processing
- * `image\_video\_generation` - Image/video generation
- * `database\_query` - Database query
- * `third\_party\_integration` - Third-party integration
- * `browser\_operation` - Browser operation
- * `agent\_collaboration` - Agent collaboration
- * `other` - Other (catch-all) (Optional)

"defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default group headline shown in chat for this connector's dynamic MCP tools (those without an individual Tool row). Shape: {"<locale>": str}. The headline for the conversation locale is resolved server-side (see whizchat.tools.services.tool_category.resolve_localized_text): requested locale -> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as displayGroupName. Empty = frontend falls back to its name-based heuristic. An individual Tool row overrides this. (optional)

"defaultToolIcon"?: string // Server-level default icon identifier (a frontend icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty = frontend uses its default MCP icon. Streamed as toolIcon. (optional)

"effectiveDescription": string

"remoteMcpUrl": string

"oauthServerUrl": string

"logoUrl": string

"authType": string

"authHeaders"?: object // Optional

"authHeadersMasked": object

"allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this connector may render into auth_headers through {{context_data.<key>}}. An empty list disables the feature for this connector: the placeholder is left verbatim, exactly as before the feature existed. Keys are matched literally and are case-sensitive; at most 20 keys of letters, digits and underscores. (Optional)

"exposedMcpTools"?: object // Allowlist of MCP tool names this connector exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the server tools. Use this to limit a connector to a safe subset (e.g. read-only tools). (Optional)

"isEnabled"?: boolean // Whether this connector is enabled for contacts in the organization (Optional)

"organization": string (uuid)

"createdBy": string (uuid)

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

Response Example

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string",
    "displayName": "response_string",
    "description": "response_string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "response_string",
    "updatedAt": "response_string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string",
  "displayName": "response_string",
  "description": "response_string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "response_string",
  "remoteMcpUrl": "response_string",
  "oauthServerUrl": "response_string",
  "logoUrl": "response_string",
  "authType": "response_string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example_response_name",
    "description": "example_response_description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "response_string",
  "updatedAt": "response_string"
}
```

Delete Connector

DELETE `/api/v1/connectors/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	Y	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

List Connector Authorized Members

GET**/api/v1/connectors/{id}/authorized-members/**

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/authorized-members/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/authorized-members/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/authorized-members/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/authorized-members/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "mcpRegistry":
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
    alphanumeric, underscore and hyphen allowed.
```

```

    "displayName?": string // Human-readable display name (defaults to name if
empty) (Optional)
    "description?": string // Brief description of the MCP service (Optional)
    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (Optional)
        string (uri) // Manual override for the OAuth authorization server URL. Leave
empty to auto-discover from the MCP server (RFC 9728 protected resource
metadata). Only fill in when auto-discovery fails or the server advertises
incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
api.supabase.com) (Optional)
    "logoUrl"?: // May have different types (Optional)
        string (uri) // CDN URL for the MCP service logo (for frontend rendering)
(Optional)
    "isActive?": boolean // Whether this MCP service is currently available for
use (Optional)
    "isGlobal?": boolean // Global registries are maintained by the platform and
available to all organizations. (Optional)
    "authType"?: // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
    }
    "mcpRegistryId": string (uuid)
    "name": string
    "displayName?": string // Custom display name for this connector (defaults to
MCP Registry name) (Optional)
    "description?": string // Custom description for this connector (defaults to
MCP Registry description) (Optional)
    "defaultCategory"?: // May have different types (Optional)
        string (enum: knowledge_base, web_search, web_fetch, code_execution,
file_processing, image_video_generation, database_query, third_party_integration,
browser_operation, agent_collaboration, other) // Org-level override of the MCP
Registry default category for this connector's dynamic tools. Empty = use the MCP
Registry default_category.

* `knowledge_base` - Knowledge base retrieval
* `web_search` - Web search
* `web_fetch` - Web page fetch
* `code_execution` - Code execution
* `file_processing` - File processing
* `image_video_generation` - Image/video generation
* `database_query` - Database query
* `third_party_integration` - Third-party integration
* `browser_operation` - Browser operation

```

```

* `agent_collaboration` - Agent collaboration
* `other` - Other (catch-all) (Optional)

"defaultDisplayGroupNameByLocale"?: object // Locale-keyed server-level default
group headline shown in chat for this connector's dynamic MCP tools (those
without an individual Tool row). Shape: {"<locale>": str}. The headline for the
conversation locale is resolved server-side (see
whizchat.tools.services.tool_category.resolve_localized_text): requested locale -
> settings.LANGUAGE_CODE locale -> empty, then streamed verbatim as
displayGroupName. Empty = frontend falls back to its name-based heuristic. An
individual Tool row overrides this. (optional)

"defaultToolIcon"?: string // Server-level default icon identifier (a frontend
icon key, not an uploaded image) for this connector's dynamic MCP tools. Empty =
frontend uses its default MCP icon. Streamed as toolIcon. (optional)

"effectiveDescription": string
"remoteMcpUrl": string
"oauthServerUrl": string
"logoUrl": string
"authType": string
"authHeaders"?: object // Optional
"authHeadersMasked": object

"allowedContextDataKeys"?: object // Allowlist of webchat contextData keys this
connector may render into auth_headers through {{context_data.<key>}}. An empty
list disables the feature for this connector: the placeholder is left verbatim,
exactly as before the feature existed. Keys are matched literally and are case-
sensitive; at most 20 keys of letters, digits and underscores. (Optional)

"exposedMcpTools"?: object // Allowlist of MCP tool names this connector
exposes (to the agent and to SEP-1865 UI cards). Empty list = expose all of the
server tools. Use this to limit a connector to a safe subset (e.g. read-only
tools). (Optional)

"isEnabled"?: boolean // Whether this connector is enabled for contacts in the
organization (Optional)
"organization": string (uuid)
"createdBy": string (uuid)
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "mcpRegistry": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string",
    "displayName": "response_string",
    "description": "response_string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
  }
}

```

```

    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "response_string",
    "updatedAt": "response_string"
  },
  "mcpRegistryId": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string",
  "displayName": "response_string",
  "description": "response_string",
  "defaultCategory": "knowledge_base",
  "defaultDisplayGroupNameByLocale": null,
  "defaultToolIcon": "Response string",
  "effectiveDescription": "response_string",
  "remoteMcpUrl": "response_string",
  "oauthServerUrl": "response_string",
  "logoUrl": "response_string",
  "authType": "response_string",
  "authHeaders": null,
  "authHeadersMasked": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example_response_name",
    "description": "example_response_description"
  },
  "allowedContextDataKeys": null,
  "exposedMcpTools": null,
  "isEnabled": false,
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "response_string",
  "updatedAt": "response_string"
}

```

Remove Connector Authorized Member

DELETE

`/api/v1/connectors/{id}/authorized-members/{memberId}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	
<code>memberId</code>	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/authorized-members/{memberId}"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/authorized-members/{memberId}", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-446655440000/authorized-members/{memberId}"

headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/connectors/550e8400-e29b-41d4-a716-
446655440000/authorized-members/{memberId}
", [
    'headers' => [
        'Authorization' => 'Api-Key YOUR_API_KEY'
    ]
]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

List MCP Tool Registries

GET

/api/v1/mcp/registry/

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/mcp/registry/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/mcp/registry/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/mcp/registry/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/mcp/registry/",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "id": string (uuid)
    "name": string // MCP service name (e.g., Supabase, Sentry). Only
    alphanumeric, underscore and hyphen allowed.
    "displayName?": string // Human-readable display name (defaults to name if
    empty) (Optional)
    "description?": string // Brief description of the MCP service (Optional)
```

```

    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (Optional)
        string (uri) // Manual override for the OAuth authorization server URL. Leave
        empty to auto-discover from the MCP server (RFC 9728 protected resource
        metadata). Only fill in when auto-discovery fails or the server advertises
        incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
        api.supabase.com) (Optional)
    "logoUrl"?: // May have different types (Optional)
        string (uri) // CDN URL for the MCP service logo (for frontend rendering)
        (Optional)
    "isActive"?: boolean // Whether this MCP service is currently available for
    use (Optional)
    "isGlobal"?: boolean // Global registries are maintained by the platform and
    available to all organizations. (Optional)
    "authType"?: // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]

```

Response Example

```

[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string",
    "displayName": "response_string",
    "description": "response_string",
    "remoteMcpUrl": "https://example.com/file.jpg",
    "oauthServerUrl": "https://example.com/file.jpg",
    "logoUrl": "https://example.com/file.jpg",
    "isActive": false,
    "isGlobal": false,
    "authType": {},
    "createdAt": "response_string",
    "updatedAt": "response_string"
  }
]

```

Retrieve MCP Tool Registry

GET

/api/v1/mcp/registry/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/mcp/registry/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/mcp/registry/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/mcp/registry/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/mcp/registry/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string // MCP service name (e.g., Supabase, Sentry). Only alphanumeric,
underscore and hyphen allowed.
  "displayName"?: string // Human-readable display name (defaults to name if
empty) (Optional)
  "description"?: string // Brief description of the MCP service (Optional)
```

```

    "remoteMcpUrl": string (uri) // The URL of the Remote MCP service
    "oauthServerUrl"?: // May have different types (Optional)
        string (uri) // Manual override for the OAuth authorization server URL. Leave
        empty to auto-discover from the MCP server (RFC 9728 protected resource
        metadata). Only fill in when auto-discovery fails or the server advertises
        incorrect metadata. (e.g., Supabase MCP is at mcp.supabase.com but OAuth is at
        api.supabase.com) (Optional)
    "logoUrl"?: // May have different types (Optional)
        string (uri) // CDN URL for the MCP service logo (for frontend rendering)
    (Optional)
    "isActive"?: boolean // Whether this MCP service is currently available for use
    (Optional)
    "isGlobal"?: boolean // Global registries are maintained by the platform and
    available to all organizations. (Optional)
    "authType"?: // How to authenticate with this MCP server.

* `oauth` - OAuth 2.0
* `service_account` - Service Account
* `none` - No Authentication (Optional)
    {
    }
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_string",
  "displayName": "response_string",
  "description": "response_string",
  "remoteMcpUrl": "https://example.com/file.jpg",
  "oauthServerUrl": "https://example.com/file.jpg",
  "logoUrl": "https://example.com/file.jpg",
  "isActive": false,
  "isGlobal": false,
  "authType": {},
  "createdAt": "response_string",
  "updatedAt": "response_string"
}

```

List Tool Execution Records

GET

/api/v1/tool-execution-records/

Parameters

Parameter	Required	Type	Description
chatbot	X	string	Filter by AI assistant ID
endDate	X	string	End date (inclusive), format: YYYY-MM-DD
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	Search by AI assistant name, tool name, or message content
startDate	X	string	Start date (inclusive), format: YYYY-MM-DD
status	X	string	Filter by execution status: success, failure, pending, timeout
tool	X	string	Filter by tool ID
toolType	X	string	Filter by tool type: api, function, mcp

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tool-execution-records/?
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=ex
ample&status=failure&tool=example&toolType=api"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tool-execution-records/?
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=ex
ample&status=failure&tool=example&toolType=api", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tool-execution-records/?  
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=ex  
ample&status=failure&tool=example&toolType=api"  
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.get(url, headers=headers)  
try:  
    print("Successfully retrieved response:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tool-execution-records/?
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=example&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"? : string (uri) // Optional
  "previous"? : string (uri) // Optional
  "results": [
    {
```



```

    "id": string (uuid)
    "tool": object // Retrieve tool information
    "chatbot": object // Retrieve AI assistant information
    "messageContent": string // Retrieve message content
    "status"?: // Optional
    {
    }
    "executionTime": integer // Retrieve execution time (milliseconds)
    "errorMessage"?: string // Optional
    "retryCount"?: integer // How many times the platform auto-retried this
    tool call before the final outcome (0 = the first attempt was the final one). API
    tools count tenacity retries inside APIRequestService; MCP tools count transient-
    error retries (read-classified tools only). Function tools are never auto-
    retried. (optional)
    "createdAt": string (timestamp)
  }
]
}

```

Response Example

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "tool": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response_example_name",
        "description": "response_example_description"
      },
      "chatbot": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response_example_name",
        "description": "response_example_description"
      },
      "messageContent": "response_string",
      "status": {},
      "executionTime": 456,
      "retryCount": 456,
      "errorMessage": "response_string",
      "createdAt": "response_string"
    }
  ]
}

```

```
]
}
```

Retrieve Tool Execution Record

GET

`/api/v1/tool-execution-records/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	Y	string	A UUID string identifying this tool execution record.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tool-execution-records/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tool-execution-records/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "tool": object // Retrieve tool information, including operated_tool_name
  "chatbot": object // Retrieve AI assistant information
  "message":
  {
    "id": string (uuid)
    "content": string
  }
}
```

```

}
"inputParameters": object
"outputResult"?: object // Optional
"errorMessage"?: string // Optional
"status"?: // Optional
{
}
"executionTime": integer // Retrieve execution time (milliseconds)
"retryCount"?: integer // How many times the platform auto-retried this tool
call before the final outcome (0 = the first attempt was the final one). API
tools count tenacity retries inside APIRequestService; MCP tools count transient-
error retries (read-classified tools only). Function tools are never auto-
retried. (optional)
"requestUrl"?: // May have different types (Optional)
string (uri) // Final rendered URL used when calling an API tool. Null for non-
API tools. (Optional)
"requestMethod"?: string // HTTP method (GET/POST/...) used when calling an API
tool. Null for non-API tools. (Optional)
"requestHeaders": object // Return rendered request headers with sensitive
values masked.

APICredential keys for the (contact, tool) pair are masked automatically
in addition to ``settings.TOOL_API_SENSITIVE_HEADERS``.
"requestBody"?: object // Final rendered body sent to the external API. Null
for non-API tools or read-only methods. (Optional)
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "tool": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_example_name",
    "description": "response_example_description"
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_example_name",
    "description": "response_example_description"
  },
  "message": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "content": "response_string"
  },
  "inputParameters": null,

```

```
"outputResult": null,
"errorMessage": "response_string",
"status": {},
"executionTime": 456,
"retryCount": 456,
"requestUrl": "https://example.com/file.jpg",
"requestMethod": "response_string",
"requestHeaders": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_example_name",
  "description": "response_example_description"
},
"requestBody": null,
"createdAt": "response_string",
"updatedAt": "response_string"
}
```

Retrieve Tool Execution Record

GET

</api/v1/tool-execution-records/chatbots/>

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tool-execution-records/chatbots/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tool-execution-records/chatbots/",
config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/tool-execution-records/chatbots/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tool-execution-records/chatbots/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "id"?: string (uuid) // Optional
    "name"?: string // Optional
  }
]
```

Response Example

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string"
  }
]
```

Retrieve Tool Execution Record

GET /api/v1/tool-execution-records/tools/

Parameters

Parameter	Required	Type	Description
chatbot	X	string	
endDate	X	string	
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
startDate	X	string	
status	X	string	`success`: Success; `failure`: Failure; `pending`: Pending; `timeout`: Timeout;
tool	X	string	
toolType	X	string	`api`: API; `function`: Function; `mcp`: MCP; `desktop`: Desktop; `flex`: Flex Message; `quick_reply`: Quick Reply;

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tool-execution-records/tools/?
chatbot=550e8400-e29b-41d4-a716-
446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=examp
le&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"? : string (uri) // Optional
  "previous"? : string (uri) // Optional
  "results": [
    {
```

```
    "id": string (uuid)
    "displayName": string
  }
]
```

Response Example

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "displayName": "response_string"
    }
  ]
}
```

List Execution Records for a Specific Tool

GET

/api/v1/tools/{toolPk}/tool-execution-records/

Parameters

Parameter	Required	Type	Description
toolPk	V	string	
chatbot	X	string	Filter by AI assistant ID
endDate	X	string	End date (inclusive), format: YYYY-MM-DD
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.

query	X	string	Search by AI assistant name, tool name, or message content
startDate	X	string	Start date (inclusive), format: YYYY-MM-DD
status	X	string	Filter by execution status: success, failure, pending, timeout
tool	X	string	Filter by tool ID
toolType	X	string	Filter by tool type: api, function, mcp

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/?
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=example&toolType=api"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/?chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=example&toolType=api", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/?chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=example&toolType=api"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-
e29b-41d4-a716-446655440000/tool-execution-records/?
chatbot=example&endDate=example&page=1&pageSize=1&query=example&startDate=ex
ample&status=failure&tool=example&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"? : string (uri) // Optional
  "previous"? : string (uri) // Optional
  "results": [
    {
```

```

    "id": string (uuid)
    "tool": object // Retrieve tool information
    "chatbot": object // Retrieve AI assistant information
    "messageContent": string // Retrieve message content
    "status"?: // Optional
    {
    }
    "executionTime": integer // Retrieve execution time (milliseconds)
    "errorMessage"?: string // Optional
    "retryCount"?: integer // How many times the platform auto-retried this
    tool call before the final outcome (0 = the first attempt was the final one). API
    tools count tenacity retries inside APIRequestService; MCP tools count transient-
    error retries (read-classified tools only). Function tools are never auto-
    retried. (optional)
    "createdAt": string (timestamp)
  }
]
}

```

Response Example

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "tool": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response_example_name",
        "description": "response_example_description"
      },
      "chatbot": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response_example_name",
        "description": "response_example_description"
      },
      "messageContent": "response_string",
      "status": {},
      "executionTime": 456,
      "retryCount": 456,
      "errorMessage": "response_string",
      "createdAt": "response_string"
    }
  ]
}

```

```
]
}
```

Retrieve Specific Tool Execution Record

GET

/api/v1/tools/{toolPk}/tool-execution-records/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this tool execution record.
toolPk	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/",
config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-
e29b-41d4-a716-446655440000/tool-execution-records/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "tool": object // Retrieve tool information, including operated_tool_name
  "chatbot": object // Retrieve AI assistant information
  "message":
  {
    "id": string (uuid)
```

```

    "content": string
  }
  "inputParameters": object
  "outputResult"?: object // Optional
  "errorMessage"?: string // Optional
  "status"?: // Optional
  {
  }
  "executionTime": integer // Retrieve execution time (milliseconds)
  "retryCount"?: integer // How many times the platform auto-retried this tool
  call before the final outcome (0 = the first attempt was the final one). API
  tools count tenacity retries inside APIRequestService; MCP tools count transient-
  error retries (read-classified tools only). Function tools are never auto-
  retried. (optional)
  "requestUrl"?: // May have different types (Optional)
    string (uri) // Final rendered URL used when calling an API tool. Null for non-
  API tools. (Optional)
  "requestMethod"?: string // HTTP method (GET/POST/...) used when calling an API
  tool. Null for non-API tools. (Optional)
  "requestHeaders": object // Return rendered request headers with sensitive
  values masked.

  APICredential keys for the (contact, tool) pair are masked automatically
  in addition to ``settings.TOOL_API_SENSITIVE_HEADERS``.
  "requestBody"?: object // Final rendered body sent to the external API. Null
  for non-API tools or read-only methods. (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "tool": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_example_name",
    "description": "response_example_description"
  },
  "chatbot": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_example_name",
    "description": "response_example_description"
  },
  "message": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "content": "response_string"
  },
}

```

```
"inputParameters": null,
"outputResult": null,
"errorMessage": "response_string",
"status": {},
"executionTime": 456,
"retryCount": 456,
"requestUrl": "https://example.com/file.jpg",
"requestMethod": "response_string",
"requestHeaders": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response_example_name",
  "description": "response_example_description"
},
"requestBody": null,
"createdAt": "response_string",
"updatedAt": "response_string"
}
```

Retrieve Specific Tool Execution Record

GET

`/api/v1/tools/{toolPk}/tool-execution-records/chatbots/`

Parameters

Parameter	Required	Type	Description
<code>toolPk</code>	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/chatbots/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request data
before executing.
```

JavaScript

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/chatbots/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/chatbots/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-
e29b-41d4-a716-446655440000/tool-execution-records/chatbots/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "id"?: string (uuid) // Optional
    "name"?: string // Optional
  }
]
```

Response Example

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "response_string"
  }
]
```

Retrieve Specific Tool Execution Record

GET `/api/v1/tools/{toolPk}/tool-execution-records/tools/`

Parameters

Parameter	Required	Type	Description
<code>toolPk</code>	V	string	
<code>chatbot</code>	X	string	
<code>endDate</code>	X	string	
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.
<code>query</code>	X	string	
<code>startDate</code>	X	string	
<code>status</code>	X	string	`success`: Success; `failure`: Failure; `pending`: Pending; `timeout`: Timeout;
<code>tool</code>	X	string	

toolType

X

string

```
`api`: API; `function`: Function; `mcp`: MCP;  
`desktop`: Desktop; `flex`: Flex Message;  
`quick_reply`: Quick Reply;
```

Code Examples

Shell/Bash

```
# API call example (Shell)  
curl -X GET "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-  
446655440000/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-  
446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=exam  
ple&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api"  
  
-H "Authorization: Api-Key YOUR_API_KEY"  
  
# Please make sure to replace YOUR_API_KEY and verify the request data  
before executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api",
config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/tools/550e8400-e29b-41d4-a716-446655440000/tool-execution-records/tools/?chatbot=550e8400-e29b-41d4-a716-446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=example&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/tools/550e8400-
e29b-41d4-a716-446655440000/tool-execution-records/tools/?chatbot=550e8400-
e29b-41d4-a716-
446655440000&endDate=example&page=1&pageSize=1&query=example&startDate=exam
ple&status=failure&tool=550e8400-e29b-41d4-a716-446655440000&toolType=api", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"? : string (uri) // Optional
  "previous"? : string (uri) // Optional
  "results": [
```

```
{
  "id": string (uuid)
  "displayName": string
}
]
```

Response Example

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "displayName": "response_string"
    }
  ]
}
```

Call API example (Shell)

...

目錄

1. Upload Skill
2. List Skills
3. Get Specific Skill
4. Update Skill
5. Partially Update Skill
6. Delete Skill
7. Export Skill
8. Re-upload Skill
9. List Skill Resources
10. Delete Skill Resource
11. Download Skill Resource

Call API example (Shell)

Upload Skill

POST

`/api/v1/skills/upload/`

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/skills/upload/"

-H "Authorization: Api-Key YOUR_API_KEY"

-F "file=@example_file.pdf"

-F "attached_tool_ids=Example String"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');
const FormData = require('form-data');

// Create FormData object
const formData = new FormData();
// Please replace with actual file
formData.append('file',
* actual file object *
, 'example_file.pdf');
formData.append('attached_tool_ids', 'Example String');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/skills/upload/",
formData, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/skills/upload/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'file': ('example_file.pdf', open('example_file.pdf', 'rb'),
    'application/pdf'),
    'attached_tool_ids': (None, 'Example String')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/skills/upload/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example_file.pdf', 'r'),
                'filename' => 'example_file.pdf'
            ],
            [
                'name' => 'attached_tool_ids',
                'contents' => 'Example String'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "id"?: string (uuid) // Optional
  "name"?: string // Optional
  "description"?: string // Optional
  "attached_tools"?: [ // Tool names automatically bound from the skill
package front matter (optional)
    string
  ]
  "missing_tools"?: [ // Tool names declared by the skill package but not
found in this organization and requiring manual binding (optional)
    string
  ]
  "message"?: string // Optional
}
```

Response Example

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "attached_tools": [
    "Response string"
  ],
  "missing_tools": [
    "Response string"
  ],
  "name": "Response String",
  "description": "Response String",
  "message": "Response String"
}
```

Status Code: 400

Response Schema Example

```
{
  "detail"?: string // Error message (Optional)
}
```

Response Example

```
{
  "detail": "Response String"
}
```

List Skills

GET

/api/v1/skills/

Parameters

Parameter	Required	Type	Description
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
search	X	string	

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/skills/?
page=1&pageSize=1&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/?
page=1&pageSize=1&search=example", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/?
page=1&pageSize=1&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/skills/?
page=1&pageSize=1&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
```



```

"previous"?: string (uri) // Optional
"results": [
  {
    "id": string (uuid)
    "name": string
    "description": string
    "whenToUse": string // Describe when the LLM should trigger this
skill. Max 200 characters recommended.
    "source": // How the skill was created: upload (.skill/.zip) or
manual

* `upload` - Upload
* `manual` - Manual
    {
    }
    "attachedTools": [
      {
        "id": string (uuid)
        "name": string
        "description": string
        "displayName": string
        "toolType": string
      }
    ]
    "groups": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "createdBy": object // Show 'System' as the creator for global
skills (uploaded by platform staff).
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
    "lastUsedAt": string (timestamp)
    "canUpdate": boolean // Return whether the current user can update
this resource.
    "canDelete": boolean // Return whether the current user can delete
this resource.
  }
]
}

```

Response Example

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "description": "Response String",
      "whenToUse": "Response String",
      "source": {},
      "attachedTools": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response String",
          "description": "Response String",
          "displayName": "Response String",
          "toolType": "Response String"
        }
      ],
      "groups": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response String"
        }
      ],
      "createdBy": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response Example Name",
        "description": "Response Example Description"
      },
      "createdAt": "Response String",
      "updatedAt": "Response String",
      "lastUsedAt": "Response String",
      "canUpdate": false,
      "canDelete": false
    }
  ]
}
```

Get Specific Skill

GET

`/api/v1/skills/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Skill.

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
}
```

```

"description": string
"whenToUse": string // Describe when the LLM should trigger this skill.
Max 200 characters recommended.
"instructions": string // Markdown text with instructions for the skill
"compatibility": string // Optional environment requirements from the
Agent Skills Spec frontmatter (max 500 chars).
"metadata": object // Optional arbitrary key-value mapping from the
Agent Skills Spec frontmatter.
"allowedTools": object // Optional pre-approved tool list from the
Agent Skills Spec frontmatter (experimental).
"source": // How the skill was created: upload (.skill/.zip) or manual

* `upload` - Upload
* `manual` - Manual
{
}
"skillPackagePath": string // S3 path for the uploaded .skill/.zip file
"skillPackageName": string // Original filename of the uploaded skill
package
"skillPackageSize": integer // Size of the uploaded skill package in
bytes
"skillPackageUploadedAt": string (timestamp)
"attachedTools": [
{
"id": string (uuid)
"name": string
"description": string
"displayName": string
"toolType": string
}
]
"resources": [
{
"id": string (uuid)
"path": string // Relative path within the skill directory, e.g.
scripts/helper.py
"filePath": string // S3 path:
skills/{organization_id}/{skill_id}/resources/
"fileSize": integer // File size in bytes
"mimeType": string
"createdAt": string (timestamp)
}
]
"groups": [
{

```

```

        "id": string (uuid)
        "name": string
    }
]
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
"lastUsedAt": string (timestamp)
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "description": "Response String",
  "whenToUse": "Response String",
  "instructions": "Response String",
  "compatibility": "Response String",
  "metadata": null,
  "allowedTools": null,
  "source": {},
  "skillPackagePath": "Response String",
  "skillPackageName": "Response String",
  "skillPackageSize": 456,
  "skillPackageUploadedAt": "Response String",
  "attachedTools": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response String",
      "description": "Response String",
      "displayName": "Response String",
      "toolType": "Response String"
    }
  ],
  "resources": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "path": "Response String",
      "filePath": "Response String",
      "fileSize": 456,

```



```
    "mimeType": "Response String",
    "createdAt": "Response String"
  },
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  }
],
"createdAt": "Response String",
"updatedAt": "Response String",
"lastUsedAt": "Response String",
"canUpdate": false,
"canDelete": false
}
```

Update Skill

PUT

/api/v1/skills/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Skill.

Request

Request Parameters

Field	Type	Required	Description
name	string	Yes	
description	string	Yes	
whenToUse	string	No	Describe when the LLM should trigger this skill. Max 200 characters recommended.
instructions	string	No	Markdown text with instructions for the skill
attachedToolIds	array[string]	No	
groups	array[IdName]	No	Groups this skill is assigned to. Non-owner members must pick at least one group they belong to.

Request Schema Example

```
{
  "name": string
  "description": string
  "whenToUse"? : string // Describe when the LLM should trigger this
skill. Max 200 characters recommended. (Optional)
  "instructions"? : string // Markdown text with instructions for the
skill (Optional)
  "attachedToolIds"? : [ // Optional
    string (uuid)
  ]
  "groups"? : [ // Groups this skill is assigned to. Non-owner members
must pick at least one group they belong to. (Optional)
    {
      "id": string (uuid)
    }
  ]
}
```

Request Example

```
{
  "name": "Example Name",
  "description": "Example String",
```

```
"whenToUse": "Example String",
"instructions": "Example String",
"attachedToolIds": [
  "550e8400-e29b-41d4-a716-446655440000"
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
]
}
```

Code Examples

```
# Call API example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name",
  "description": "Example String",
  "whenToUse": "Example String",
  "instructions": "Example String",
  "attachedToolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example Name",
  "description": "Example String",
  "whenToUse": "Example String",
  "instructions": "Example String",
  "attachedToolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.put("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name",
    "description": "Example String",
    "whenToUse": "Example String",
    "instructions": "Example String",
    "attachedToolIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name",
            "description": "Example String",
            "whenToUse": "Example String",
            "instructions": "Example String",
            "attachedToolIds": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "description": string
  "whenToUse": string // Describe when the LLM should trigger this skill.
  Max 200 characters recommended.
  "instructions": string // Markdown text with instructions for the skill
  "compatibility": string // Optional environment requirements from the
  Agent Skills Spec frontmatter (max 500 chars).
  "metadata": object // Optional arbitrary key-value mapping from the
  Agent Skills Spec frontmatter.
  "allowedTools": object // Optional pre-approved tool list from the
  Agent Skills Spec frontmatter (experimental).
  "source": // How the skill was created: upload (.skill/.zip) or manual

  * `upload` - Upload
  * `manual` - Manual
  {
  }
  "skillPackagePath": string // S3 path for the uploaded .skill/.zip file
  "skillPackageName": string // Original filename of the uploaded skill
  package
  "skillPackageSize": integer // Size of the uploaded skill package in
  bytes
  "skillPackageUploadedAt": string (timestamp)
  "attachedTools": [
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
  "resources": [
```



```

{
  "id": string (uuid)
  "path": string // Relative path within the skill directory, e.g.
scripts/helper.py
  "filePath": string // S3 path:
skills/{organization_id}/{skill_id}/resources/
  "fileSize": integer // File size in bytes
  "mimeType": string
  "createdAt": string (timestamp)
}
]
"groups": [
  {
    "id": string (uuid)
    "name": string
  }
]
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
"lastUsedAt": string (timestamp)
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "description": "Response String",
  "whenToUse": "Response String",
  "instructions": "Response String",
  "compatibility": "Response String",
  "metadata": null,
  "allowedTools": null,
  "source": {},
  "skillPackagePath": "Response String",
  "skillPackageName": "Response String",
  "skillPackageSize": 456,
  "skillPackageUploadedAt": "Response String",
  "attachedTools": [
    {

```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "description": "Response String",
    "displayName": "Response String",
    "toolType": "Response String"
  }
],
"resources": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "path": "Response String",
    "filePath": "Response String",
    "fileSize": 456,
    "mimeType": "Response String",
    "createdAt": "Response String"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  }
],
"createdAt": "Response String",
"updatedAt": "Response String",
"lastUsedAt": "Response String",
"canUpdate": false,
"canDelete": false
}
```

Partially Update Skill

PATCH

</api/v1/skills/{id}>

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Skill.

Request

Request Parameters

Field	Type	Required	Description
name	string	No	
description	string	No	
whenToUse	string	No	Describe when the LLM should trigger this skill. Max 200 characters recommended.
instructions	string	No	Markdown text with instructions for the skill
attachedToolIds	array[string]	No	
groups	array[IdName]	No	Groups this skill is assigned to. Non-owner members must pick at least one group they belong to.

Request Schema Example

```
{
  "name"?: string // Optional
  "description"?: string // Optional
  "whenToUse"?: string // Describe when the LLM should trigger this
skill. Max 200 characters recommended. (Optional)
  "instructions"?: string // Markdown text with instructions for the
skill (Optional)
  "attachedToolIds"?: [ // Optional
    string (uuid)
  ]
  "groups"?: [ // Groups this skill is assigned to. Non-owner members
must pick at least one group they belong to. (Optional)
    {
```

```
        "id": string (uuid)
    }
]
}
```

Request Example

```
{
  "name": "Example Name",
  "description": "Example String",
  "whenToUse": "Example String",
  "instructions": "Example String",
  "attachedToolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

Code Examples

```
# Call API example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name",
  "description": "Example String",
  "whenToUse": "Example String",
  "instructions": "Example String",
  "attachedToolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example Name",
  "description": "Example String",
  "whenToUse": "Example String",
  "instructions": "Example String",
  "attachedToolIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
};

axios.patch("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name",
    "description": "Example String",
    "whenToUse": "Example String",
    "instructions": "Example String",
    "attachedToolIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    patch("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-
    a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name",
            "description": "Example String",
            "whenToUse": "Example String",
            "instructions": "Example String",
            "attachedToolIds": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```


Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "description": string
  "whenToUse": string // Describe when the LLM should trigger this skill.
  Max 200 characters recommended.
  "instructions": string // Markdown text with instructions for the skill
  "compatibility": string // Optional environment requirements from the
  Agent Skills Spec frontmatter (max 500 chars).
  "metadata": object // Optional arbitrary key-value mapping from the
  Agent Skills Spec frontmatter.
  "allowedTools": object // Optional pre-approved tool list from the
  Agent Skills Spec frontmatter (experimental).
  "source": // How the skill was created: upload (.skill/.zip) or manual

* `upload` - Upload
* `manual` - Manual
  {
  }
  "skillPackagePath": string // S3 path for the uploaded .skill/.zip file
  "skillPackageName": string // Original filename of the uploaded skill
  package
  "skillPackageSize": integer // Size of the uploaded skill package in
  bytes
  "skillPackageUploadedAt": string (timestamp)
  "attachedTools": [
    {
      "id": string (uuid)
      "name": string
      "description": string
      "displayName": string
      "toolType": string
    }
  ]
  "resources": [
```

```

{
  "id": string (uuid)
  "path": string // Relative path within the skill directory, e.g.
scripts/helper.py
  "filePath": string // S3 path:
skills/{organization_id}/{skill_id}/resources/
  "fileSize": integer // File size in bytes
  "mimeType": string
  "createdAt": string (timestamp)
}
]
"groups": [
  {
    "id": string (uuid)
    "name": string
  }
]
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
"lastUsedAt": string (timestamp)
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
}

```

Response Example

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "description": "Response String",
  "whenToUse": "Response String",
  "instructions": "Response String",
  "compatibility": "Response String",
  "metadata": null,
  "allowedTools": null,
  "source": {},
  "skillPackagePath": "Response String",
  "skillPackageName": "Response String",
  "skillPackageSize": 456,
  "skillPackageUploadedAt": "Response String",
  "attachedTools": [
    {

```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String",
    "description": "Response String",
    "displayName": "Response String",
    "toolType": "Response String"
  }
],
"resources": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "path": "Response String",
    "filePath": "Response String",
    "fileSize": 456,
    "mimeType": "Response String",
    "createdAt": "Response String"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response String"
  }
],
"createdAt": "Response String",
"updatedAt": "Response String",
"lastUsedAt": "Response String",
"canUpdate": false,
"canDelete": false
}
```

Delete Skill

DELETE

/api/v1/skills/{id}

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Skill.

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X DELETE "https://api.maiaagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Export Skill

GET

`/api/v1/skills/{id}/export/`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Skill.

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/export/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/export/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/export/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/export/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
200	.skill file download

Re-upload Skill

POST

`/api/v1/skills/{id}/reupload/`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Skill.

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/reupload/"

-H "Authorization: Api-Key YOUR_API_KEY"

-F "file=@example_file.pdf"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');
const FormData = require('form-data');

// Create FormData object
const formData = new FormData();
// Please replace with actual file
formData.append('file',
* actual file object *
, 'example_file.pdf');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/reupload/", formData, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/reupload/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'file': ('example_file.pdf', open('example_file.pdf', 'rb'),
    'application/pdf')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
post("https://api.maiagent.ai/api/v1/skills/550e8400-e29b-41d4-a716-446655440000/reupload/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example_file.pdf', 'r'),
                'filename' => 'example_file.pdf'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id"?: string (uuid) // Optional
  "name"?: string // Optional
  "description"?: string // Optional
  "message"?: string // Optional
}
```

Response Example

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response String",
  "description": "Response String",
  "message": "Response String"
}
```

Status Code: **400**

Response Schema Example

```
{
  "detail"?: string // Error message (Optional)
}
```

Response Example

```
{
  "detail": "Response String"
}
```

List Skill Resources

GET**/api/v1/skills/{skillPk}/resources/**

Parameters

Parameter	Required	Type	Description
skillPk	V	string	

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "id": string (uuid)
    "path": string // Relative path within the skill directory, e.g.
```

```
scripts/helper.py
    "filePath": string // S3 path:
skills/{organization_id}/{skill_id}/resources/
    "fileSize": integer // File size in bytes
    "mimeType": string
    "createdAt": string (timestamp)
}
]
```

Response Example

```
[
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "path": "Response String",
  "filePath": "Response String",
  "fileSize": 456,
  "mimeType": "Response String",
  "createdAt": "Response String"
}
]
```

Delete Skill Resource

DELETE`/api/v1/skills/{skillPk}/resources/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	
<code>skillPk</code>	V	string	

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url =
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Download Skill Resource

GET

/api/v1/skills/{skillPk}/resources/{id}/download/

Parameters

Parameter	Required	Type	Description
id	V	string	
skillPk	V	string	

Code Examples

Shell/Bash

```
# Call API example (Shell)
curl -X GET
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-
e29b-41d4-a716-446655440000/download/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please confirm you have replaced YOUR_API_KEY and verified the
request data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-e29b-41d4-a716-446655440000/download/", config)
  .then(response => {
    console.log('Successfully retrieved response:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url =
"https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e8400-
e29b-41d4-a716-446655440000/download/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Successfully retrieved response:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/skills/{skillPk}/resources/550e
8400-e29b-41d4-a716-446655440000/download/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Successfully retrieved response:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
200	File download

API call example (Shell)

...

目錄

1. Create Knowledge Base
2. List All Knowledge Bases
3. Get Knowledge Base Details
4. Update Knowledge Base
5. Get Specific Knowledge Base Details
6. Partially Update Knowledge Base
7. Delete Knowledge Base
8. Check RRF Availability
9. Export Labels Excel
10. Import Labels Excel
11. Export Metadata Excel
12. Import Metadata Excel
13. Create New Label
14. List Labels in Knowledge Base
15. Get Label Details
16. Update Label
17. Partially Update Label
18. Delete Label
19. Create Metadata Field
20. List Metadata Fields
21. Get Metadata Field Details
22. Update Metadata Field
23. Upload Files to Knowledge Base
24. Batch Delete Files
25. Batch Re-parse Files
26. Batch Re-parse Files
27. List Files in Knowledge Base
28. Get File Details
29. Get File Move Options
30. Get File Latest Failure Reason
31. Get File Transcription
32. Update File Information
33. Delete File
34. Create File Translation
35. List File Translations

- 36. Get Translation Details
- 37. Delete Translation
- 38. Download Translated File
- 39. List Knowledge Base Documents
- 40. Update Document
- 41. Partially Update Document
- 42. Delete Document
- 43. Create New FAQ
- 44. Batch Delete FAQ
- 45. List All FAQ in Knowledge Base
- 46. Get FAQ Details
- 47. Update FAQ
- 48. Partially Update FAQ
- 49. Delete FAQ

API call example (Shell)

Create Knowledge Base

POST

/api/v1/knowledge-bases/

Request

Request Parameters

Field	Type	Required	Description
embeddingModel	string (uuid)	Yes	
rerankerModel	string (uuid)	Yes	
name	string	Yes	
description	string	No	

Field	Type	Required	Description
numberOfRetrievedChunks	integer	No	Number of retrieved reference chunks, default is 12, minimum is 1
enableHyde	boolean	No	Enable HyDE, default is False
similarityCutoff	number (double)	No	Similarity cutoff, default is 0.0, range is 0.0-1.0
enableRerank	boolean	No	Whether to enable reranking, default is True
enableRrf	boolean	No	Enable Reciprocal Rank Fusion for hybrid search. Disable this if using a custom Elasticsearch without enterprise license. If not provided, uses system default.
vectorDatabaseType	object	No	Type of vector database: elasticsearch or opensearch. elasticsearch : Elasticsearch ; opensearch : OpenSearch ; oracle : Oracle AI Database 26ai ; synxdb : SynxDB;
vectorDatabaseUrl	string	No	Custom vector database endpoint URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g., "https://host:9200"). For Oracle: DSN format (e.g., "host:1521/service_name").
vectorDatabaseApiKey	string	No	API Key for Elasticsearch. Not used for OpenSearch.
opensearchUsername	string	No	Username for OpenSearch Basic Auth.

Field	Type	Required	Description
opensearchPassword	string	No	Password for OpenSearch Basic Auth.
oracleUser	string	No	Oracle database username.
oraclePassword	string	No	Oracle database password.
synxdbUser	string	No	SynxDB database username.
synxdbPassword	string	No	SynxDB database password.
chatbots	array[IdName]	No	
groups	array[IdName]	No	

Request Schema Example

```
{
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description"?: string // Optional
  "numberOfRetrievedChunks"?: integer // Number of retrieved reference
chunks, default is 12, minimum is 1 (Optional)
  "enableHyde"?: boolean // Enable HyDE, default is False (Optional)
  "similarityCutoff"?: number (double) // Similarity cutoff, default is
0.0, range is 0.0-1.0 (Optional)
  "enableRerank"?: boolean // Whether to enable reranking, default is
True (Optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. If not provided, uses system default. (Optional)
  "vectorDatabaseType"?: // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
```

```

Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
  "oracleUser"?: string // Oracle database username. (Optional)
  "oraclePassword"?: string // Oracle database password. (Optional)
  "synxdbUser"?: string // SynxDB database username. (Optional)
  "synxdbPassword"?: string // SynxDB database password. (Optional)
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "groups"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
}

```

Request Example Values

```

{
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example Name",
  "description": "Example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Example string",
  "vectorDatabaseApiKey": "Example string",
  "opensearchUsername": "Example string",
  "opensearchPassword": "Example string",
  "oracleUser": "Example string",
  "oraclePassword": "Example string",
}

```

```
"synxdbUser": "Example string",
"synxdbPassword": "Example string",
"chatbots": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
]
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-bases/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example Name",
  "description": "Example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Example string",
  "vectorDatabaseApiKey": "Example string",
  "opensearchUsername": "Example string",
  "opensearchPassword": "Example string",
  "oracleUser": "Example string",
  "oraclePassword": "Example string",
  "synxdbUser": "Example string",
  "synxdbPassword": "Example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'
```

```
# Make sure to replace YOUR_API_KEY and verify the request data  
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example Name",
  "description": "Example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Example string",
  "vectorDatabaseApiKey": "Example string",
  "opensearchUsername": "Example string",
  "opensearchPassword": "Example string",
  "oracleUser": "Example string",
  "oraclePassword": "Example string",
  "synxdbUser": "Example string",
  "synxdbPassword": "Example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

```
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Name",
    "description": "Example string",
    "numberOfRetrievedChunks": 123,
    "enableHyde": true,
    "similarityCutoff": 123,
    "enableRerank": true,
    "enableRrf": true,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "Example string",
    "vectorDatabaseApiKey": "Example string",
    "opensearchUsername": "Example string",
    "opensearchPassword": "Example string",
    "oracleUser": "Example string",
    "oraclePassword": "Example string",
    "synxdbUser": "Example string",
    "synxdbPassword": "Example string",
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}
```

```
response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "embeddingModel": "550e8400-e29b-41d4-a716-
446655440000",
            "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Example Name",
            "description": "Example string",
            "numberOfRetrievedChunks": 123,
            "enableHyde": true,
            "similarityCutoff": 123,
            "enableRerank": true,
            "enableRrf": true,
            "vectorDatabaseType": {},
            "vectorDatabaseUrl": "Example string",
            "vectorDatabaseApiKey": "Example string",
            "opensearchUsername": "Example string",
            "opensearchPassword": "Example string",
            "oracleUser": "Example string",
            "oraclePassword": "Example string",
            "synxdbUser": "Example string",
            "synxdbPassword": "Example string",
            "chatbots": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "groups": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ]
        }
    ]
}

```

```

        }
    ]
}
]);

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 201

Response Schema Example

```

{
  "id": string (uuid)
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "organization?": string (uuid) // Optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description?": string // Optional
  "numberOfRetrievedChunks?": integer // Number of retrieved reference
chunks, default is 12, minimum is 1 (Optional)
  "enableHyde?": boolean // Enable HyDE, default is False (Optional)
  "similarityCutoff?": number (double) // Similarity cutoff, default is
0.0, range is 0.0-1.0 (Optional)
  "enableRerank?": boolean // Whether to enable reranking, default is

```

```
True (Optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
  "vectorDatabaseType"?: // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
  "oracleUser"?: string // Oracle database username. (Optional)
  "oraclePassword"?: string // Oracle database password. (Optional)
  "synxdbUser"?: string // SynxDB database username. (Optional)
  "synxdbPassword"?: string // SynxDB database password. (Optional)
  "labels": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups"?: [ // List of groups that can access this Knowledge Base
(Optional)
    {
      "id": string (uuid)
      "name": string
    }
  ]
}
```

```
]
  "filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
  "vectorStorageSize": integer
  "chunksCount": integer
  "canUpdate": boolean // Return whether the current user can update this
resource.
  "canDelete": boolean // Return whether the current user can delete this
resource.
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,
  "enableRerank": false,
  "enableRrf": false,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Response string",
  "vectorDatabaseApiKey": "Response string",
  "opensearchUsername": "Response string",
  "opensearchPassword": "Response string",
  "oracleUser": "Response string",
  "oraclePassword": "Response string",
  "synxdbUser": "Response string",
  "synxdbPassword": "Response string",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ]
}
```

```
}
],
"chatbots": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"filesCount": 456,
"vectorStorageSize": 456,
"chunksCount": 456,
"canUpdate": false,
"canDelete": false,
"createdAt": "Response string",
"updatedAt": "Response string"
}
```

List All Knowledge Bases

GET

</api/v1/knowledge-bases/>

Parameters

Parameter	Required	Type	Description
<code>embeddingModel</code>	X	string	
<code>page</code>	X	integer	A page number within the paginated result set.

pageSize	X	integer	Number of results to return per page.
query	X	string	
rerankerModel	X	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-bases/?
embeddingModel=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-
e29b-41d4-a716-446655440000"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/?
embeddingModel=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-
e29b-41d4-a716-446655440000", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/?\nembeddingModel=550e8400-e29b-41d4-a716-\n446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-\ne29b-41d4-a716-446655440000"\nheaders = {\n    "Authorization": "Api-Key YOUR_API_KEY"\n}\n\nresponse = requests.get(url, headers=headers)\ntry:\n    print("Response received successfully:")\n    print(response.json())\nexcept Exception as e:\n    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/?
embeddingModel=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&rerankerModel=550e8400-
e29b-41d4-a716-446655440000", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "createdBy":
        {
          "id": string (uuid)
          "name": string
        }
      "organization"?: string (uuid) // Optional
      "embeddingModel": string (uuid)
      "rerankerModel": string (uuid)
      "name": string
      "description"?: string // Optional
      "numberOfRetrievedChunks"?: integer // Number of retrieved
reference chunks, default is 12, minimum is 1 (Optional)
      "enableHyde"?: boolean // Enable HyDE, default is False (Optional)
      "similarityCutoff"?: number (double) // Similarity cutoff, default
is 0.0, range is 0.0-1.0 (Optional)
      "enableRerank"?: boolean // Whether to enable reranking, default is
True (Optional)
      "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
      "vectorDatabaseType"?: // Type of vector database: elasticsearch
or opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
      {
      }
      "vectorDatabaseUrl"?: string // Custom vector database endpoint URL
or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
      "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not
used for OpenSearch. (Optional)
      "opensearchUsername"?: string // Username for OpenSearch Basic
Auth. (Optional)

```

```

    "opensearchPassword"?: string // Password for OpenSearch Basic
Auth. (Optional)
    "oracleUser"?: string // Oracle database username. (Optional)
    "oraclePassword"?: string // Oracle database password. (Optional)
    "synxdbUser"?: string // SynxDB database username. (Optional)
    "synxdbPassword"?: string // SynxDB database password. (Optional)
    "synxdbUser"?: string // SynxDB database username. (Optional)
    "synxdbPassword"?: string // SynxDB database password. (Optional)
    "labels": [
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "chatbots"?: [ // Optional
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "groups"?: [ // List of groups that can access this Knowledge Base
(Optional)
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "filesCount": integer // Return count of KnowledgeBaseFile
excluding processed files, conversation attachments, and soft-deleted.
    "vectorStorageSize": integer
    "chunksCount": integer
    "canUpdate": boolean // Return whether the current user can update
this resource.
    "canDelete": boolean // Return whether the current user can delete
this resource.
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
}

```

Response Example Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "createdBy": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "organization": "550e8400-e29b-41d4-a716-446655440000",
      "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
      "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "description": "Response string",
      "numberOfRetrievedChunks": 456,
      "enableHyde": false,
      "similarityCutoff": 456,
      "enableRerank": false,
      "enableRrf": false,
      "vectorDatabaseType": {},
      "vectorDatabaseUrl": "Response string",
      "vectorDatabaseApiKey": "Response string",
      "opensearchUsername": "Response string",
      "opensearchPassword": "Response string",
      "oracleUser": "Response string",
      "oraclePassword": "Response string",
      "synxdbUser": "Response string",
      "synxdbPassword": "Response string",
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      ],
      "chatbots": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      ],
      "groups": [
        {
```

```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "filesCount": 456,
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "canUpdate": false,
  "canDelete": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
]
```

Get Knowledge Base Details

GET

`/api/v1/knowledge-bases/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "createdBy":
```

```

{
  "id": string (uuid)
  "name": string
}
"organization"?: string (uuid) // Optional
"embeddingModel": string (uuid)
"rerankerModel": string (uuid)
"name": string
"description"?: string // Optional
"numberOfRetrievedChunks"?: integer // Number of retrieved reference
chunks, default is 12, minimum is 1 (Optional)
"enableHyde"?: boolean // Enable HyDE, default is False (Optional)
"similarityCutoff"?: number (double) // Similarity cutoff, default is
0.0, range is 0.0-1.0 (Optional)
"enableRerank"?: boolean // Whether to enable reranking, default is
True (Optional)
"enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
"vectorDatabaseType"?: // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
{
}
"vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
"vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
"opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
"opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
"oracleUser"?: string // Oracle database username. (Optional)
"oraclePassword"?: string // Oracle database password. (Optional)
"synxdbUser"?: string // SynxDB database username. (Optional)
"synxdbPassword"?: string // SynxDB database password. (Optional)
"labels": [
  {
    "id": string (uuid)

```

```

        "name": string
      }
    ]
    "chatbots"?: [ // Optional
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "groups"?: [ // List of groups that can access this Knowledge Base
(Optional)
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
    "vectorStorageSize": integer
    "chunksCount": integer
    "canUpdate": boolean // Return whether the current user can update this
resource.
    "canDelete": boolean // Return whether the current user can delete this
resource.
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }

```

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,

```

```
"enableRerank": false,
"enableRrf": false,
"vectorDatabaseType": {},
"vectorDatabaseUrl": "Response string",
"vectorDatabaseApiKey": "Response string",
"opensearchUsername": "Response string",
"opensearchPassword": "Response string",
"oracleUser": "Response string",
"oraclePassword": "Response string",
"synxdbUser": "Response string",
"synxdbPassword": "Response string",
"labels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"chatbots": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"filesCount": 456,
"vectorStorageSize": 456,
"chunksCount": 456,
"canUpdate": false,
"canDelete": false,
"createdAt": "Response string",
"updatedAt": "Response string"
}
```

Update Knowledge Base

PUT**/api/v1/knowledge-bases/{id}**

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Knowledge Base.

Request

Request Parameters

Field	Type	Required	Description
organization	string (uuid)	No	
embeddingModel	string (uuid)	Yes	
rerankerModel	string (uuid)	Yes	
name	string	Yes	
description	string	No	
numberOfRetrievedChunks	integer	No	Number of retrieved reference chunks, default is 12, minimum is 1
enableHyde	boolean	No	Enable HyDE, default is False
similarityCutoff	number (double)	No	Similarity cutoff, default is 0.0, range is 0.0-1.0
enableRerank	boolean	No	Whether to enable reranking, default is True
enableRrf	boolean	No	Enable Reciprocal Rank Fusion for hybrid search. Disable this if using a

Field	Type	Required	Description
			custom Elasticsearch without enterprise license.
vectorDatabaseType	object	No	Type of vector database: elasticsearch or opensearch. elasticsearch : Elasticsearch ; opensearch : OpenSearch ; oracle : Oracle AI Database 26ai ; synxdb : SynxDB;
vectorDatabaseUrl	string	No	Custom vector database endpoint URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g., "https://host:9200"). For Oracle: DSN format (e.g., "host:1521/service_name").
vectorDatabaseApiKey	string	No	API Key for Elasticsearch. Not used for OpenSearch.
opensearchUsername	string	No	Username for OpenSearch Basic Auth.
opensearchPassword	string	No	Password for OpenSearch Basic Auth.
oracleUser	string	No	Oracle database username.
oraclePassword	string	No	Oracle database password.
synxdbUser	string	No	SynxDB database username.
synxdbPassword	string	No	SynxDB database password.
chatbots	array[IdName]	No	
groups	array[IdName]	No	List of groups that can access this Knowledge Base

Request Schema Example

```
{
  "organization?": string (uuid) // Optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description?": string // Optional
  "numberOfRetrievedChunks?": integer // Number of retrieved reference
chunks, default is 12, minimum is 1 (Optional)
  "enableHyde?": boolean // Enable HyDE, default is False (Optional)
  "similarityCutoff?": number (double) // Similarity cutoff, default is
0.0, range is 0.0-1.0 (Optional)
  "enableRerank?": boolean // Whether to enable reranking, default is
True (Optional)
  "enableRrf?": boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
  "vectorDatabaseType?": // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
  {
  }
  "vectorDatabaseUrl?": string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
  "vectorDatabaseApiKey?": string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
  "opensearchUsername?": string // Username for OpenSearch Basic Auth.
(Optional)
  "opensearchPassword?": string // Password for OpenSearch Basic Auth.
(Optional)
  "oracleUser?": string // Oracle database username. (Optional)
  "oraclePassword?": string // Oracle database password. (Optional)
  "synxdbUser?": string // SynxDB database username. (Optional)
  "synxdbPassword?": string // SynxDB database password. (Optional)
  "chatbots?": [ // Optional
  {
    "id": string (uuid)
  }
}
```

```

]
"groups"?: [ // List of groups that can access this Knowledge Base
(Optional)
  {
    "id": string (uuid)
  }
]
}

```

Request Example Values

```

{
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example Name",
  "description": "Example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Example string",
  "vectorDatabaseApiKey": "Example string",
  "opensearchUsername": "Example string",
  "opensearchPassword": "Example string",
  "oracleUser": "Example string",
  "oraclePassword": "Example string",
  "synxdbUser": "Example string",
  "synxdbPassword": "Example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}

```

```
]
}
```

Code Examples

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example Name",
  "description": "Example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Example string",
  "vectorDatabaseApiKey": "Example string",
  "opensearchUsername": "Example string",
  "opensearchPassword": "Example string",
  "oracleUser": "Example string",
  "oraclePassword": "Example string",
  "synxdbUser": "Example string",
  "synxdbPassword": "Example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'
```

```
# Make sure to replace YOUR_API_KEY and verify the request data  
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example Name",
  "description": "Example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Example string",
  "vectorDatabaseApiKey": "Example string",
  "opensearchUsername": "Example string",
  "opensearchPassword": "Example string",
  "oracleUser": "Example string",
  "oraclePassword": "Example string",
  "synxdbUser": "Example string",
  "synxdbPassword": "Example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

```
]  
};
```

```
axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-  
e29b-41d4-a716-446655440000/", data, config)  
  .then(response => {  
    console.log('Response received successfully:');  
    console.log(response.data);  
  })  
  .catch(error => {  
    console.error('Request error:');  
    console.error(error.response?.data || error.message);  
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Name",
    "description": "Example string",
    "numberOfRetrievedChunks": 123,
    "enableHyde": true,
    "similarityCutoff": 123,
    "enableRerank": true,
    "enableRrf": true,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "Example string",
    "vectorDatabaseApiKey": "Example string",
    "opensearchUsername": "Example string",
    "opensearchPassword": "Example string",
    "oracleUser": "Example string",
    "oraclePassword": "Example string",
    "synxdbUser": "Example string",
    "synxdbPassword": "Example string",
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}
```



```
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "organization": "550e8400-e29b-41d4-a716-446655440000",
            "embeddingModel": "550e8400-e29b-41d4-a716-
446655440000",
            "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Example Name",
            "description": "Example string",
            "numberOfRetrievedChunks": 123,
            "enableHyde": true,
            "similarityCutoff": 123,
            "enableRerank": true,
            "enableRrf": true,
            "vectorDatabaseType": {},
            "vectorDatabaseUrl": "Example string",
            "vectorDatabaseApiKey": "Example string",
            "opensearchUsername": "Example string",
            "opensearchPassword": "Example string",
            "oracleUser": "Example string",
            "oraclePassword": "Example string",
            "synxdbUser": "Example string",
            "synxdbPassword": "Example string",
            "chatbots": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "groups": [

```

```

        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

l);

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```

{
  "id": string (uuid)
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "organization?": string (uuid) // Optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description?": string // Optional
  "numberOfRetrievedChunks?": integer // Number of retrieved reference
chunks, default is 12, minimum is 1 (Optional)
  "enableHyde?": boolean // Enable HyDE, default is False (Optional)
  "similarityCutoff?": number (double) // Similarity cutoff, default is

```

```
0.0, range is 0.0-1.0 (Optional)
  "enableRerank"?: boolean // Whether to enable reranking, default is
True (Optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
  "vectorDatabaseType"?: // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
  {
  }
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
  "oracleUser"?: string // Oracle database username. (Optional)
  "oraclePassword"?: string // Oracle database password. (Optional)
  "synxdbUser"?: string // SynxDB database username. (Optional)
  "synxdbPassword"?: string // SynxDB database password. (Optional)
  "labels": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups"?: [ // List of groups that can access this Knowledge Base
(Optional)
    {
      "id": string (uuid)
```

```

    "name": string
  }
]
"filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
"vectorStorageSize": integer
"chunksCount": integer
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,
  "enableRerank": false,
  "enableRrf": false,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Response string",
  "vectorDatabaseApiKey": "Response string",
  "opensearchUsername": "Response string",
  "opensearchPassword": "Response string",
  "oracleUser": "Response string",
  "oraclePassword": "Response string",
  "synxdbUser": "Response string",
  "synxdbPassword": "Response string",
  "labels": [
    {

```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"chatbots": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"filesCount": 456,
"vectorStorageSize": 456,
"chunksCount": 456,
"canUpdate": false,
"canDelete": false,
"createdAt": "Response string",
"updatedAt": "Response string"
}
```

Get Specific Knowledge Base Details

GET

</api/v1/knowledge-bases/move-options/>

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-bases/move-
options/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please replace YOUR_API_KEY before executing and verify the
request data.
```

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/move-
options/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/move-options/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/move-options/",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
}
```

```
"embeddingModel": string (uuid)
"vectorDatabaseType"?: // Type of vector database: Elasticsearch or
OpenSearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (optional)
{
}
"canUpdate": boolean // Return whether the current user can update this
resource.
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "vectorDatabaseType": {},
  "canUpdate": false
}
```

Partially Update Knowledge Base

PATCH </api/v1/knowledge-bases/{id}>

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base.

Request

Request Parameters

Field	Type	Required	Description
organization	string (uuid)	No	
embeddingModel	string (uuid)	No	
rerankerModel	string (uuid)	No	
name	string	No	
description	string	No	
numberOfRetrievedChunks	integer	No	Number of retrieved reference chunks, default is 12, minimum is 1
enableHyde	boolean	No	Enable Hyde, default is False
similarityCutoff	number (double)	No	Similarity cutoff, default is 0.0, range is 0.0-1.0
enableRerank	boolean	No	Whether to enable reranking, default is True
enableRrf	boolean	No	Enable Reciprocal Rank Fusion for hybrid search. Disable this if using a custom Elasticsearch without enterprise license.
vectorDatabaseType	object	No	Type of vector database: elasticsearch or opensearch. <code>elasticsearch</code> : Elasticsearch ; <code>opensearch</code> : OpenSearch ; <code>oracle</code> : Oracle AI Database 26ai ; <code>synxdb</code> : SynxDB;

Field	Type	Required	Description
vectorDatabaseUrl	string	No	Custom vector database endpoint URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g., "https://host:9200"). For Oracle: DSN format (e.g., "host:1521/service_name").
vectorDatabaseApiKey	string	No	API Key for Elasticsearch. Not used for OpenSearch.
opensearchUsername	string	No	Username for OpenSearch Basic Auth.
opensearchPassword	string	No	Password for OpenSearch Basic Auth.
oracleUser	string	No	Oracle database username.
oraclePassword	string	No	Oracle database password.
synxdbUser	string	No	SynxDB database username.
synxdbPassword	string	No	SynxDB database password.
chatbots	array[IdName]	No	
groups	array[IdName]	No	List of groups that can access this Knowledge Base

Request Schema Example

```
{
  "organization?": string (uuid) // Optional
  "embeddingModel?": string (uuid) // Optional
  "rerankerModel?": string (uuid) // Optional
  "name?": string // Optional
  "description?": string // Optional
  "numberOfRetrievedChunks?": integer // Number of retrieved reference
chunks, default is 12, minimum is 1 (Optional)
  "enableHyde?": boolean // Enable HyDE, default is False (Optional)
```

```

    "similarityCutoff"?: number (double) // Similarity cutoff, default is
    0.0, range is 0.0-1.0 (Optional)
    "enableRerank"?: boolean // Whether to enable reranking, default is
    True (Optional)
    "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
    search. Disable this if using a custom Elasticsearch without enterprise
    license. (Optional)
    "vectorDatabaseType"?: // Type of vector database: elasticsearch or
    opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
    {
    }
    "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
    Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
    "https://host:9200"). For Oracle: DSN format (e.g.,
    "host:1521/service_name"). (Optional)
    "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
    for OpenSearch. (Optional)
    "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
    (Optional)
    "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
    (Optional)
    "oracleUser"?: string // Oracle database username. (Optional)
    "oraclePassword"?: string // Oracle database password. (Optional)
    "synxdbUser"?: string // SynxDB database username. (Optional)
    "synxdbPassword"?: string // SynxDB database password. (Optional)
    "chatbots"?: [ // Optional
    {
        "id": string (uuid)
    }
    ]
    "groups"?: [ // List of groups that can access this Knowledge Base
    (Optional)
    {
        "id": string (uuid)
    }
    ]
}

```

```

{
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example Name",
  "description": "Example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Example string",
  "vectorDatabaseApiKey": "Example string",
  "opensearchUsername": "Example string",
  "opensearchPassword": "Example string",
  "oracleUser": "Example string",
  "oraclePassword": "Example string",
  "synxdbUser": "Example string",
  "synxdbPassword": "Example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}

```

Code Examples

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example Name",
  "description": "Example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Example string",
  "vectorDatabaseApiKey": "Example string",
  "opensearchUsername": "Example string",
  "opensearchPassword": "Example string",
  "oracleUser": "Example string",
  "oraclePassword": "Example string",
  "synxdbUser": "Example string",
  "synxdbPassword": "Example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}'
```



```
# Make sure to replace YOUR_API_KEY and verify the request data  
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Example Name",
  "description": "Example string",
  "numberOfRetrievedChunks": 123,
  "enableHyde": true,
  "similarityCutoff": 123,
  "enableRerank": true,
  "enableRrf": true,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Example string",
  "vectorDatabaseApiKey": "Example string",
  "opensearchUsername": "Example string",
  "opensearchPassword": "Example string",
  "oracleUser": "Example string",
  "oraclePassword": "Example string",
  "synxdbUser": "Example string",
  "synxdbPassword": "Example string",
  "chatbots": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ]
}
```

```
]  
};
```

```
axios.patch("https://api.maiagent.ai/api/v1/knowledge-  
bases/550e8400-e29b-41d4-a716-446655440000/", data, config)  
  .then(response => {  
    console.log('Response received successfully:');  
    console.log(response.data);  
  })  
  .catch(error => {  
    console.error('Request error:');  
    console.error(error.response?.data || error.message);  
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
    "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Example Name",
    "description": "Example string",
    "numberOfRetrievedChunks": 123,
    "enableHyde": true,
    "similarityCutoff": 123,
    "enableRerank": true,
    "enableRrf": true,
    "vectorDatabaseType": {},
    "vectorDatabaseUrl": "Example string",
    "vectorDatabaseApiKey": "Example string",
    "opensearchUsername": "Example string",
    "opensearchPassword": "Example string",
    "oracleUser": "Example string",
    "oraclePassword": "Example string",
    "synxdbUser": "Example string",
    "synxdbPassword": "Example string",
    "chatbots": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "groups": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}
```

```
}
```

```
response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "organization": "550e8400-e29b-41d4-a716-446655440000",
            "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
            "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Example Name",
            "description": "Example string",
            "numberOfRetrievedChunks": 123,
            "enableHyde": true,
            "similarityCutoff": 123,
            "enableRerank": true,
            "enableRrf": true,
            "vectorDatabaseType": {},
            "vectorDatabaseUrl": "Example string",
            "vectorDatabaseApiKey": "Example string",
            "opensearchUsername": "Example string",
            "opensearchPassword": "Example string",
            "oracleUser": "Example string",
            "oraclePassword": "Example string",
            "synxdbUser": "Example string",
            "synxdbPassword": "Example string",
            "chatbots": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "groups": [

```

```

        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]
}

l);

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```

{
  "id": string (uuid)
  "createdBy":
  {
    "id": string (uuid)
    "name": string
  }
  "organization?": string (uuid) // Optional
  "embeddingModel": string (uuid)
  "rerankerModel": string (uuid)
  "name": string
  "description?": string // Optional
  "numberOfRetrievedChunks?": integer // Number of retrieved reference
chunks, default is 12, minimum is 1 (Optional)
  "enableHyde?": boolean // Enable HyDE, default is False (Optional)
  "similarityCutoff?": number (double) // Similarity cutoff, default is

```

```
0.0, range is 0.0-1.0 (Optional)
  "enableRerank"?: boolean // Whether to enable reranking, default is
True (Optional)
  "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
  "vectorDatabaseType"?: // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
{
}
  "vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
  "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
  "opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
  "opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
  "oracleUser"?: string // Oracle database username. (Optional)
  "oraclePassword"?: string // Oracle database password. (Optional)
  "synxdbUser"?: string // SynxDB database username. (Optional)
  "synxdbPassword"?: string // SynxDB database password. (Optional)
  "labels": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "chatbots"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "groups"?: [ // List of groups that can access this Knowledge Base
(Optional)
    {
      "id": string (uuid)
```



```

    "name": string
  }
]
"filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
"vectorStorageSize": integer
"chunksCount": integer
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,
  "enableRerank": false,
  "enableRrf": false,
  "vectorDatabaseType": {},
  "vectorDatabaseUrl": "Response string",
  "vectorDatabaseApiKey": "Response string",
  "opensearchUsername": "Response string",
  "opensearchPassword": "Response string",
  "oracleUser": "Response string",
  "oraclePassword": "Response string",
  "synxdbUser": "Response string",
  "synxdbPassword": "Response string",
  "labels": [
    {

```

```
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
],
"chatbots": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"filesCount": 456,
"vectorStorageSize": 456,
"chunksCount": 456,
"canUpdate": false,
"canDelete": false,
"createdAt": "Response string",
"updatedAt": "Response string"
}
```

Delete Knowledge Base

DELETE

`/api/v1/knowledge-bases/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Check RRF Availability

GET

`/api/v1/knowledge-bases/{id}/check-rrf-availability/`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/check-rrf-availability/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/check-rrf-availability/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```


Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/check-rrf-availability/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/check-rrf-availability/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "createdBy":
```

```

{
  "id": string (uuid)
  "name": string
}
"organization"?: string (uuid) // Optional
"embeddingModel": string (uuid)
"rerankerModel": string (uuid)
"name": string
"description"?: string // Optional
"numberOfRetrievedChunks"?: integer // Number of retrieved reference
chunks, default is 12, minimum is 1 (Optional)
"enableHyde"?: boolean // Enable HyDE, default is False (Optional)
"similarityCutoff"?: number (double) // Similarity cutoff, default is
0.0, range is 0.0-1.0 (Optional)
"enableRerank"?: boolean // Whether to enable reranking, default is
True (Optional)
"enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
"vectorDatabaseType"?: // Type of vector database: elasticsearch or
opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai
* `synxdb` - SynxDB (Optional)
{
}
"vectorDatabaseUrl"?: string // Custom vector database endpoint URL or
Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
"vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not used
for OpenSearch. (Optional)
"opensearchUsername"?: string // Username for OpenSearch Basic Auth.
(Optional)
"opensearchPassword"?: string // Password for OpenSearch Basic Auth.
(Optional)
"oracleUser"?: string // Oracle database username. (Optional)
"oraclePassword"?: string // Oracle database password. (Optional)
"synxdbUser"?: string // SynxDB database username. (Optional)
"synxdbPassword"?: string // SynxDB database password. (Optional)
"labels": [
  {
    "id": string (uuid)

```

```

    "name": string
  }
]
"chatbots"?: [ // Optional
  {
    "id": string (uuid)
    "name": string
  }
]
"groups"?: [ // List of groups that can access this Knowledge Base
(Optional)
  {
    "id": string (uuid)
    "name": string
  }
]
"filesCount": integer // Return count of KnowledgeBaseFile excluding
processed files, conversation attachments, and soft-deleted.
"vectorStorageSize": integer
"chunksCount": integer
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "createdBy": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
  "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "description": "Response string",
  "numberOfRetrievedChunks": 456,
  "enableHyde": false,
  "similarityCutoff": 456,

```

```
"enableRerank": false,
"enableRrf": false,
"vectorDatabaseType": {},
"vectorDatabaseUrl": "Response string",
"vectorDatabaseApiKey": "Response string",
"opensearchUsername": "Response string",
"opensearchPassword": "Response string",
"oracleUser": "Response string",
"oraclePassword": "Response string",
"synxdbUser": "Response string",
"synxdbPassword": "Response string",
"labels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"chatbots": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"filesCount": 456,
"vectorStorageSize": 456,
"chunksCount": 456,
"canUpdate": false,
"canDelete": false,
"createdAt": "Response string",
"updatedAt": "Response string"
}
```

Export Labels Excel

GET**/api/v1/knowledge-bases/{id}/export-labels-excel/**

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/export-labels-excel/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/export-labels-excel/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/export-labels-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/export-labels-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
200	Excel file download

Import Labels Excel

POST

`/api/v1/knowledge-bases/{id}/import-labels-excel/`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiaagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/import-labels-excel/"

-H "Authorization: Api-Key YOUR_API_KEY"

-F "excelFile=@example_file.txt"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');
const FormData = require('form-data');

// Create FormData object
const formData = new FormData();
// Replace with actual file
formData.append('excelFile',
* actual file object *
, 'example_file.txt');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/import-labels-excel/", formData, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/import-labels-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'excelFile': ('example_file.txt', open('example_file.txt',
    'rb'), 'text/plain')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/import-labels-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'excelFile',
                'contents' => fopen('example_file.txt', 'r'),
                'filename' => 'example_file.txt'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "success"?: boolean // Optional
  "updated_count"?: integer // Optional
  "total_rows"?: integer // Optional
  "errors"?: [ // Optional
    {
      "row"?: integer // Optional
      "file_id"?: string // Optional
      "error"?: string // Optional
    }
  ]
}
```

Response Example Values

```
{
  "success": false,
  "updated_count": 456,
  "total_rows": 456,
  "errors": [
    {
      "row": 456,
      "file_id": "Response string",
      "error": "Response string"
    }
  ]
}
```

Export Metadata Excel

GET

</api/v1/knowledge-bases/{id}/export-metadata-excel/>

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/export-metadata-excel/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/export-metadata-excel/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/export-metadata-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/export-metadata-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
200	Excel file download

Import Metadata Excel

POST

/api/v1/knowledge-bases/{id}/import-metadata-excel/

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Knowledge Base.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/import-metadata-excel/"

-H "Authorization: Api-Key YOUR_API_KEY"

-F "excelFile=@example_file.txt"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');
const FormData = require('form-data');

// Create FormData object
const formData = new FormData();
// Replace with actual file
formData.append('excelFile',
* actual file object *
, 'example_file.txt');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/import-metadata-excel/", formData,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/import-metadata-excel/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'excelFile': ('example_file.txt', open('example_file.txt',
    'rb'), 'text/plain')
}

response = requests.post(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/import-metadata-excel/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'excelFile',
                'contents' => fopen('example_file.txt', 'r'),
                'filename' => 'example_file.txt'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "success"?: boolean // Optional
  "updated_count"?: integer // Optional
  "total_rows"?: integer // Optional
  "errors"?: [ // Optional
    {
      "row"?: integer // Optional
      "file_id"?: string // Optional
      "error"?: string // Optional
    }
  ]
}
```

Response Example Values

```
{
  "success": false,
  "updated_count": 456,
  "total_rows": 456,
  "errors": [
    {
      "row": 456,
      "file_id": "Response string",
      "error": "Response string"
    }
  ]
}
```

Create New Label

POST

/api/v1/knowledge-bases/{knowledgeBasePk}/labels/

Parameters

Parameter	Required	Type	Description
knowledgeBasePk	V	string	

Request

Request Parameters

Field	Type	Required	Description
name	string	Yes	

Request Schema Example

```
{
  "name": string
}
```

Request Example Values

```
{
  "name": "Example Name"
}
```

Code Examples


```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name"
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example Name"
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/labels/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "name"?: string // Optional
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
}
```

List Labels in Knowledge Base

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/labels/`

Parameters

Parameter	Required	Type	Description
<code>knowledgeBasePk</code>	V	string	
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/?
page=1&pageSize=1"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/labels/?page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/?page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/?page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next?": string (uri) // Optional
}
```

```
"previous"?: string (uri) // Optional
"results": [
  {
    "id": string (uuid)
    "name"?: string // Optional
  }
]
}
```

Response Example Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ]
}
```

Get Label Details

GET

</api/v1/knowledge-bases/{knowledgeBasePk}/labels/{id}>

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this knowledge base label.

knowledgeBasePk

V

string

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
"name"?: string // Optional
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
}
```

Update Label

PUT`/api/v1/knowledge-bases/{knowledgeBasePk}/labels/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this knowledge base label.
<code>knowledgeBasePk</code>	V	string	

Request

Request Parameters

Field	Type	Required	Description
name	string	Yes	

Request Schema Example

```
{
  "name": string
}
```

Request Example Values

```
{
  "name": "Example Name"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name"
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example Name"
};

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name"
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name"?: string // Optional
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
}
```

Partially Update Label

PATCH

`/api/v1/knowledge-bases/{knowledgeBasePk}/labels/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this knowledge base label.
<code>knowledgeBasePk</code>	V	string	

Request

Request Parameters

Field	Type	Required	Description
name	string	No	

Request Schema Example

```
{
  "name"?: string // Optional
}
```

Request Example Values

```
{
  "name": "Example Name"
}
```

Code Examples

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name"
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example Name"
};

axios.patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
    e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-
    446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name"?: string // Optional
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
}
```

Delete Label

DELETE`/api/v1/knowledge-bases/{knowledgeBasePk}/labels/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this knowledge base label.
<code>knowledgeBasePk</code>	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/labels/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}

?>
```

Response

Status Code	Description
204	No response body

Create Metadata Field

POST

/api/v1/knowledge-bases/{knowledgeBasePk}/metadata-keys/

Parameters

Parameter	Required	Type	Description
knowledgeBasePk	V	string	

Request

Request Parameters

Field	Type	Required	Description
keyName	string	Yes	Metadata key name
description	string	No	Description of this field's purpose
isRequired	boolean	No	Whether this field is required
defaultValue	string	No	Default value for new files
enabledForSearch	boolean	No	Whether to use this field during retrieval

Request Schema Example

```
{
  "keyName": string // Metadata key name
  "description"?: string // Description of this field's purpose
  (Optional)
  "isRequired"?: boolean // Whether this field is required (Optional)
  "defaultValue"?: string // Default value for new files (Optional)
  "enabledForSearch"?: boolean // Whether to use this field during
```

```
retrieval (Optional)
}
```

Request Example Values

```
{
  "keyName": "Example string",
  "description": "Example string",
  "isRequired": true,
  "defaultValue": "Example string",
  "enabledForSearch": true
}
```

Code Examples


```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "keyName": "Example string",
  "description": "Example string",
  "isRequired": true,
  "defaultValue": "Example string",
  "enabledForSearch": true
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "keyName": "Example string",
  "description": "Example string",
  "isRequired": true,
  "defaultValue": "Example string",
  "enabledForSearch": true
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/metadata-keys/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "keyName": "Example string",
    "description": "Example string",
    "isRequired": True,
    "defaultValue": "Example string",
    "enabledForSearch": True
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "keyName": "Example string",
            "description": "Example string",
            "isRequired": true,
            "defaultValue": "Example string",
            "enabledForSearch": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "knowledgeBase":
  {
    "id": string (uuid)
    "name": string
  }
  "keyName": string // Metadata key name
  "keyType": string // Automatically determined by field name (default
field or custom field)
  "description"?: string // Description of this field's purpose
(Optional)
  "isRequired"?: boolean // Whether this field is required (Optional)
  "defaultValue"?: string // Default value for new files (Optional)
  "enabledForSearch"?: boolean // Whether to use this field during
retrieval (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "keyName": "Response string",
  "keyType": "Response string",
  "description": "Response string",
  "isRequired": false,
  "defaultValue": "Response string",
  "enabledForSearch": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

List Metadata Fields

GET

/api/v1/knowledge-bases/{knowledgeBasePk}/metadata-keys/

Parameters

Parameter	Required	Type	Description
knowledgeBasePk	V	string	
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/?page=1&pageSize=1"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/metadata-keys/?page=1&pageSize=1",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/?page=1&pageSize=1"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/?page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next?": string (uri) // Optional
}
```

```

"previous"?: string (uri) // Optional
"results": [
  {
    "id": string (uuid)
    "knowledgeBase":
    {
      "id": string (uuid)
      "name": string
    }
    "keyName": string // Metadata key name
    "keyType": string // Automatically determined by field name
    (default field or custom field)
    "description"?: string // Description of this field's purpose
    (Optional)
    "isRequired"?: boolean // Whether this field is required (Optional)
    "defaultValue"?: string // Default value for new files (Optional)
    "enabledForSearch"?: boolean // Whether to use this field during
    retrieval (Optional)
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
}

```

Response Example Values

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "keyName": "Response string",
      "keyType": "Response string",
      "description": "Response string",
      "isRequired": false,
      "defaultValue": "Response string",
      "enabledForSearch": false,
      "createdAt": "Response string",

```

```
      "updatedAt": "Response string"
    }
  ]
}
```

Get Metadata Field Details

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/metadata-keys/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Metadata Key.
<code>knowledgeBasePk</code>	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
"knowledgeBase":
{
  "id": string (uuid)
  "name": string
}
"keyName": string // Metadata key name
"keyType": string // Automatically determined by field name (default
field or custom field)
"description"?: string // Description of this field's purpose
(Optional)
"isRequired"?: boolean // Whether this field is required (Optional)
"defaultValue"?: string // Default value for new files (Optional)
"enabledForSearch"?: boolean // Whether to use this field during
retrieval (Optional)
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "keyName": "Response string",
  "keyType": "Response string",
  "description": "Response string",
  "isRequired": false,
  "defaultValue": "Response string",
  "enabledForSearch": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Update Metadata Field

PUT**/api/v1/knowledge-bases/{knowledgeBasePk}/metadata-keys/{id}**

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Metadata Key.
<code>knowledgeBasePk</code>	V	string	

Request

Request Parameters

Field	Type	Required	Description
keyName	string	Yes	Metadata key name
description	string	No	Description of this field's purpose
isRequired	boolean	No	Whether this field is required
defaultValue	string	No	Default value for new files
enabledForSearch	boolean	No	Whether to use this field during retrieval

Request Schema Example

```
{
  "keyName": string // Metadata key name
  "description"? : string // Description of this field's purpose
  (Optional)
  "isRequired"? : boolean // Whether this field is required (Optional)
  "defaultValue"? : string // Default value for new files (Optional)
  "enabledForSearch"? : boolean // Whether to use this field during
```

```
retrieval (Optional)
}
```

Request Example Values

```
{
  "keyName": "Example string",
  "description": "Example string",
  "isRequired": true,
  "defaultValue": "Example string",
  "enabledForSearch": true
}
```

Code Examples

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "keyName": "Example string",
  "description": "Example string",
  "isRequired": true,
  "defaultValue": "Example string",
  "enabledForSearch": true
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "keyName": "Example string",
  "description": "Example string",
  "isRequired": true,
  "defaultValue": "Example string",
  "enabledForSearch": true
};

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "keyName": "Example string",
    "description": "Example string",
    "isRequired": true,
    "defaultValue": "Example string",
    "enabledForSearch": true
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/metadata-keys/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "keyName": "Example string",
            "description": "Example string",
            "isRequired": true,
            "defaultValue": "Example string",
            "enabledForSearch": true
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "knowledgeBase":
  {
    "id": string (uuid)
    "name": string
  }
  "keyName": string // Metadata key name
  "keyType": string // Automatically determined by field name (default
field or custom field)
  "description"?: string // Description of this field's purpose
(Optional)
  "isRequired"?: boolean // Whether this field is required (Optional)
  "defaultValue"?: string // Default value for new files (Optional)
  "enabledForSearch"?: boolean // Whether to use this field during
retrieval (Optional)
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "keyName": "Response string",
  "keyType": "Response string",
  "description": "Response string",
  "isRequired": false,
  "defaultValue": "Response string",
  "enabledForSearch": false,
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Upload Files to Knowledge Base

POST

/api/v1/knowledge-bases/{knowledgeBasePk}/files/

Parameters

Parameter	Required	Type	Description
knowledgeBasePk	V	string	
fileType	X	string	File type filter, e.g.: pdf, docx, txt, xlsx, etc.
knowledgeBase	X	string	Knowledge Base ID
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	Search keywords, supports multi-condition queries separated by spaces. Searchable: file ID, file name, label name, Metadata value
status	X	string	File status filter. Available values: initial, processing, done, deleting, deleted, failed `initial`: Initial ; `processing`: Processing ; `done`: Done ; `deleting`: Deleting ; `deleted`: Deleted ; `faile...

Request

Request Parameters

Field	Type	Required	Description
files	array[KnowledgeBaseFileCreate]	Yes	Supports batch upload of multiple files, up to 50 files per request

Request Schema Example

```
{
  "files": [ // Supports batch upload of multiple files, up to 50 files
per request
    {
      "filename": string // File name
      "file": string (uri) // File to upload
      "parser"?: string (uuid) // Optional
      "labels"?: [ // Optional
        {
          "id": string (uuid)
        }
      ]
      "rawUserDefineMetadata"?: object // Optional
    }
  ]
}
```

Request Example Values

```
{
  "files": [
    {
      "filename": "my-file.pdf",
      "file": "string",
      "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "labels": [
        {
          "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
          "name": "my-label"
        }
      ],
      "rawUserDefineMetadata": {
        "key1": "value1",
        "key2": "value2"
      }
    }
  ]
}
```

```
    },  
    {  
      "filename": "you-can-upload-multiple-files.pdf",  
      "file": "string",  
      "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6"  
    }  
  ]  
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "files": [
    {
      "filename": "my-file.pdf",
      "file": "string",
      "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "labels": [
        {
          "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
          "name": "my-label"
        }
      ],
      "rawUserDefineMetadata": {
        "key1": "value1",
        "key2": "value2"
      }
    },
    {
      "filename": "you-can-upload-multiple-files.pdf",
      "file": "string",
      "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
    }
  ]
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```



```

const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "files": [
    {
      "filename": "my-file.pdf",
      "file": "string",
      "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "labels": [
        {
          "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
          "name": "my-label"
        }
      ],
      "rawUserDefineMetadata": {
        "key1": "value1",
        "key2": "value2"
      }
    },
    {
      "filename": "you-can-upload-multiple-files.pdf",
      "file": "string",
      "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
    }
  ]
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed",

```

```
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```

import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&status=delete_failed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "files": [
        {
            "filename": "my-file.pdf",
            "file": "string",
            "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
            "labels": [
                {
                    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
                    "name": "my-label"
                }
            ],
            "rawUserDefineMetadata": {
                "key1": "value1",
                "key2": "value2"
            }
        },
        {
            "filename": "you-can-upload-multiple-files.pdf",
            "file": "string",
            "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
        }
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")

```

```
print(response.json())  
except Exception as e:  
    print("Request error:", e)
```



```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files?fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&status=delete_failed", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "files": [
                {
                    "filename": "my-file.pdf",
                    "file": "string",
                    "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
                    "labels": [
                        {
                            "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
                            "name": "my-label"
                        }
                    ],
                    "rawUserDefineMetadata": {
                        "key1": "value1",
                        "key2": "value2"
                    }
                },
                {
                    "filename": "you-can-upload-multiple-files.pdf",
                    "file": "string",
                    "parser": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
                }
            ]
        }
    ]);
}

```

```

        ]
    }
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 201

Response Schema Example

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "filename": string // File name
      "file": string (uri) // File to upload
      "fileType": string
      "knowledgeBase"?: // Optional
      {
        "id": string (uuid)
        "name": string
      }
      "size": integer
      "status":
      {
      }
    }
  ]
}

```

```

"parser": {
  {
    "id": string (uuid)
    "name": string
    "provider":
    {
    }
    "order"?: integer // Optional
    "supportsDiarization": boolean
    "isTimestampSttProvider": boolean
  }
}
"labels"?: [ // Optional
  {
    "id": string (uuid)
    "name": string
  }
]
"rawUserDefineMetadata"?: object // Optional
"speakerLabels": [
  {
    "id": string (uuid)
    "originalLabel": string // Original speaker label from
diarization (e.g., SPEAKER_00)
    "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
    "displayName": string
  }
]
"vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from
this file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"createdAt": string (timestamp)
}
]
}

```

Response Example Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "Response string",
      "file": "https://example.com/file.jpg",
      "fileType": "Response string",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "size": 456,
      "status": {},
      "parser": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "provider": {},
        "order": 456,
        "supportsDiarization": false,
        "isTimestampSttProvider": false
      },
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "Response string"
        }
      ],
      "rawUserDefineMetadata": null,
      "speakerLabels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "originalLabel": "Response string",
          "customName": "Response string",
          "displayName": "Response string"
        }
      ],
      "vectorStorageSize": 456,
      "chunksCount": 456,
      "waitingTime": 456,
      "processingTime": 456,
      "processingTimeDetails": null,
    }
  ]
}
```

```
      "createdAt": "Response string"
    }
  ]
}
```

Batch Delete Files

POST

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/batch-delete/`

Parameters

Parameter	Required	Type	Description
<code>knowledgeBasePk</code>	V	string	

Request

Request Parameters

Field	Type	Required	Description
<code>ids</code>	<code>array[string]</code>	Yes	

Request Schema Example

```
{
  "ids": [
    string (uuid)
  ]
}
```

Request Example Values

```
{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/batch-delete/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/batch-delete/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/batch-delete/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "ids": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/batch-delete/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "ids": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}

?>
```

Response

Status Code	Description
-------------	-------------

204

No response body

Batch Re-parse Files

PATCH`/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base File.
<code>knowledgeBasePk</code>	V	string	

Request

Request Parameters

Field	Type	Required	Description
filename	string	No	File name
file	string (uri)	No	File to upload
parser	string (uuid)	No	
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	

Request Schema Example

```
{
  "filename"?: string // File name (Optional)
  "file"?: string (uri) // File to upload (Optional)
  "parser"?: string (uuid) // Optional
  "labels"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
}
```

Request Example Values

```
{
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "parser": "550e8400-e29b-41d4-a716-446655440000",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}
```

Code Examples

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "parser": "550e8400-e29b-41d4-a716-446655440000",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "parser": "550e8400-e29b-41d4-a716-446655440000",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
};

axios.patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg",
    "parser": "550e8400-e29b-41d4-a716-446655440000",
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": null
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
    e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-
    446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "filename": "document.pdf",
            "file": "https://example.com/file.jpg",
            "parser": "550e8400-e29b-41d4-a716-446655440000",
            "labels": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "rawUserDefineMetadata": null
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "filename": string // File name
  "file": string (uri) // File to upload
  "fileType": string
  "knowledgeBase"?: // Optional
  {
    "id": string (uuid)
    "name": string
  }
  "size": integer
  "status":
  {
  }
  "parser": {
  {
    "id": string (uuid)
    "name": string
    "provider":
    {
    }
    "order"?: integer // Optional
    "supportsDiarization": boolean
    "isTimestampSttProvider": boolean
  }
  }
  "labels"?: [ // Optional
  {
    "id": string (uuid)
    "name": string
  }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "speakerLabels": [
  {
    "id": string (uuid)
```



```

    "originalLabel": string // Original speaker label from diarization
    (e.g., SPEAKER_00)
    "customName"?: string // User-defined speaker name (e.g., John)
    (Optional)
    "displayName": string
  }
]
"vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from this
file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"createdAt": string (timestamp)
}

```

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "Response string",
  "file": "https://example.com/file.jpg",
  "fileType": "Response string",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "size": 456,
  "status": {},
  "parser": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "provider": {},
    "order": 456,
    "supportsDiarization": false,
    "isTimestampSttProvider": false
  },
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
}

```

```
"rawUserDefineMetadata": null,
"speakerLabels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "originalLabel": "Response string",
    "customName": "Response string",
    "displayName": "Response string"
  }
],
"vectorStorageSize": 456,
"chunksCount": 456,
"waitingTime": 456,
"processingTime": 456,
"processingTimeDetails": null,
"createdAt": "Response string"
}
```

Batch Re-parse Files

PATCH

</api/v1/knowledge-bases/{knowledgeBasePk}/files/batch-reparse/>

Parameters

Parameter	Required	Type	Description
<code>knowledgeBasePk</code>	V	string	
<code>fileType</code>	X	string	File type filter, e.g.: pdf, docx, txt, xlsx, etc.
<code>knowledgeBase</code>	X	string	Knowledge Base ID
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.

query	X	string	Search keywords, supports multi-condition queries separated by spaces. Searchable: file ID, file name, label name, Metadata value
status	X	string	File status filter. Available values: initial, processing, done, deleting, deleted, failed `initial`: Initial ; `processing`: Processing ; `done`: Done ; `deleting`: Deleting ; `deleted`: Deleted ; `faile...

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/batch-reparse/?fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&status=delete_failed"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '[]'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = [];

axios.patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/batch-reparse/?fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&status=delete_failed", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/batch-reparse/?fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&status=delete_failed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = []

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
    e29b-41d4-a716-446655440000/files/batch-reparse/?
    fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
    446655440000&page=1&pageSize=1&query=example&status=delete_failed",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => []
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "filename": string // File name
      "file": string (uri) // File to upload
      "fileType": string
      "knowledgeBase"?: // Optional
      {
        "id": string (uuid)
        "name": string
      }
      "size": integer
      "status":
      {
      }
      "parser": {
        {
          "id": string (uuid)
          "name": string
          "provider":
          {
          }
          "order"?: integer // Optional
          "supportsDiarization": boolean
          "isTimestampSttProvider": boolean
        }
      }
      "labels"?: [ // Optional
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "rawUserDefineMetadata"?: object // Optional
      "speakerLabels": [
        {
          "id": string (uuid)
          "originalLabel": string // Original speaker label from
          diarization (e.g., SPEAKER_00)
          "customName"?: string // User-defined speaker name (e.g., John)
        }
      ]
    }
  ]
}

```

```

(Optional)
    "displayName": string
  }
]
"vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from
this file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"createdAt": string (timestamp)
}
]
}

```

Response Example Values

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "Response string",
      "file": "https://example.com/file.jpg",
      "fileType": "Response string",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "size": 456,
      "status": {},
      "parser": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "provider": {},
        "order": 456,
        "supportsDiarization": false,
        "isTimestampSttProvider": false
      },
      "labels": [
        {

```



```
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "speakerLabels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "originalLabel": "Response string",
      "customName": "Response string",
      "displayName": "Response string"
    }
  ],
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "waitingTime": 456,
  "processingTime": 456,
  "processingTimeDetails": null,
  "createdAt": "Response string"
}
]
```

List Files in Knowledge Base

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/`

Parameters

Parameter	Required	Type	Description
<code>knowledgeBasePk</code>	V	string	
<code>fileType</code>	X	string	File type filter, e.g.: pdf, docx, txt, xlsx, etc.
<code>knowledgeBase</code>	X	string	Knowledge Base ID

page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	Search keywords, supports multi-condition queries separated by spaces. Searchable: file ID, file name, label name, Metadata value
status	X	string	File status filter. Available values: initial, processing, done, deleting, deleted, failed `initial`: Initial ; `processing`: Processing ; `done`: Done ; `deleting`: Deleting ; `deleted`: Deleted ; `faile...

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/?fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&status=delete_failed"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/?fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-446655440000&page=1&pageSize=1&query=example&status=delete_failed"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/?
fileType=example&knowledgeBase=550e8400-e29b-41d4-a716-
446655440000&page=1&pageSize=1&query=example&status=delete_failed",
[
    'headers' => [
        'Authorization' => 'Api-Key YOUR_API_KEY'
    ]
]);

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "filename": string // File name
      "file": string (uri) // File to upload
      "fileType": string
      "knowledgeBase"?: // Optional
      {
        "id": string (uuid)
        "name": string
      }
      "size": integer
      "status":
      {
      }
      "parser": {
      {
        "id": string (uuid)
        "name": string
        "provider":
        {
        }
        "order"?: integer // Optional
        "supportsDiarization": boolean
        "isTimestampSttProvider": boolean
      }
      }
      "labels"?: [ // Optional
      {
        "id": string (uuid)
        "name": string
      }
      ]
      "rawUserDefineMetadata"?: object // Optional
      "speakerLabels": [
      {
        "id": string (uuid)
        "originalLabel": string // Original speaker label from
diarization (e.g., SPEAKER_00)
        "customName"?: string // User-defined speaker name (e.g., John)

```

```

(Optional)
    "displayName": string
  }
]
"vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from
this file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"createdAt": string (timestamp)
}
]
}

```

Response Example Values

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "Response string",
      "file": "https://example.com/file.jpg",
      "fileType": "Response string",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "size": 456,
      "status": {},
      "parser": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "provider": {},
        "order": 456,
        "supportsDiarization": false,
        "isTimestampSttProvider": false
      },
      "labels": [
        {

```

```
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      }
    ],
    "rawUserDefineMetadata": null,
    "speakerLabels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "originalLabel": "Response string",
        "customName": "Response string",
        "displayName": "Response string"
      }
    ],
    "vectorStorageSize": 456,
    "chunksCount": 456,
    "waitingTime": 456,
    "processingTime": 456,
    "processingTimeDetails": null,
    "createdAt": "Response string"
  }
]
}
```

Get File Details

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base File.
<code>knowledgeBasePk</code>	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/",
[
    'headers' => [
        'Authorization' => 'Api-Key YOUR_API_KEY'
    ]
]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
"filename": string // File name
"file": string (uri) // File to upload
"fileType": string
"knowledgeBase"?: // Optional
{
  "id": string (uuid)
  "name": string
}
"size": integer
"status":
{
}
"parser": {
{
  "id": string (uuid)
  "name": string
  "provider":
  {
  }
  "order"?: integer // Optional
  "supportsDiarization": boolean
  "isTimestampSttProvider": boolean
}
}
"labels"?: [ // Optional
{
  "id": string (uuid)
  "name": string
}
]
"rawUserDefineMetadata"?: object // Optional
"speakerLabels": [
{
  "id": string (uuid)
  "originalLabel": string // Original speaker label from diarization
(e.g., SPEAKER_00)
  "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
  "displayName": string
}
]
"vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from this
file
```

```
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"createdAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "Response string",
  "file": "https://example.com/file.jpg",
  "fileType": "Response string",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "size": 456,
  "status": {},
  "parser": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "provider": {},
    "order": 456,
    "supportsDiarization": false,
    "isTimestampSttProvider": false
  },
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "speakerLabels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "originalLabel": "Response string",
      "customName": "Response string",
      "displayName": "Response string"
    }
  ],
  "vectorStorageSize": 456,
  "chunksCount": 456,
}
```

```
"waitingTime": 456,  
"processingTime": 456,  
"processingTimeDetails": null,  
"createdAt": "Response string"  
}
```

Get File Move Options

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/move-options/`

Parameters

Parameter	Required	Type	Description
<code>knowledgeBasePk</code>	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/move-options/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY before running and verify the
request data.
```



```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/files/move-options/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/move-options/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/move-options/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response Content

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "filename": string
}
```

```
"size": integer
"status"?: string (enum: initial, processing, done, deleting, deleted,
failed, delete_failed) // * `initial` - Initial
* `processing` - Processing
* `done` - Done
* `deleting` - Deleting
* `deleted` - Deleted
* `failed` - Failed
* `delete_failed` - Delete Failed (optional)
"chunksCount"?: integer // Number of chunks/nodes generated from this
file (optional)
"labels": [
  {
    "id": string (uuid)
    "name": string
  }
]
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "response string",
  "size": 456,
  "status": "initial",
  "chunksCount": 456,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  ]
}
```

Get File Latest Failure Reason

GET**/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}/latest-failed-reason/**

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base File.
<code>knowledgeBasePk</code>	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/latest-failed-reason/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/latest-failed-reason/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/latest-failed-reason/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-
446655440000/latest-failed-reason/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```



```
"errorMessage": string
"createdAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "errorMessage": "Response string",
  "createdAt": "Response string"
}
```

Get File Transcription

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}/transcription/`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base File.
<code>knowledgeBasePk</code>	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/transcription/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/transcription/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/transcription/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/transcription/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
"filename": string // File name
"file": string (uri) // File to upload
"fileType": string
"knowledgeBase"?: // Optional
{
  "id": string (uuid)
  "name": string
}
"size": integer
"status":
{
}
"parser": {
{
  "id": string (uuid)
  "name": string
  "provider":
  {
  }
  "order"?: integer // Optional
  "supportsDiarization": boolean
  "isTimestampSttProvider": boolean
}
}
"labels"?: [ // Optional
{
  "id": string (uuid)
  "name": string
}
]
"rawUserDefineMetadata"?: object // Optional
"speakerLabels": [
{
  "id": string (uuid)
  "originalLabel": string // Original speaker label from diarization
(e.g., SPEAKER_00)
  "customName"?: string // User-defined speaker name (e.g., John)
(Optional)
  "displayName": string
}
]
"vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from this
file
```

```
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"createdAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "Response string",
  "file": "https://example.com/file.jpg",
  "fileType": "Response string",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "size": 456,
  "status": {},
  "parser": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "provider": {},
    "order": 456,
    "supportsDiarization": false,
    "isTimestampSttProvider": false
  },
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "speakerLabels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "originalLabel": "Response string",
      "customName": "Response string",
      "displayName": "Response string"
    }
  ],
  "vectorStorageSize": 456,
  "chunksCount": 456,
}
```

```
"waitingTime": 456,  
"processingTime": 456,  
"processingTimeDetails": null,  
"createdAt": "Response string"  
}
```

Update File Information

PUT

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base File.
<code>knowledgeBasePk</code>	V	string	

Request

Request Parameters

Field	Type	Required	Description
filename	string	Yes	File name
file	string (uri)	Yes	File to upload
parser	string (uuid)	No	
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	

Request Schema Example

```
{
  "filename": string // File name
  "file": string (uri) // File to upload
  "parser"?: string (uuid) // Optional
  "labels"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
}
```

Request Example Values

```
{
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "parser": "550e8400-e29b-41d4-a716-446655440000",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}
```

Code Examples

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "parser": "550e8400-e29b-41d4-a716-446655440000",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "filename": "document.pdf",
  "file": "https://example.com/file.jpg",
  "parser": "550e8400-e29b-41d4-a716-446655440000",
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null
};

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "filename": "document.pdf",
    "file": "https://example.com/file.jpg",
    "parser": "550e8400-e29b-41d4-a716-446655440000",
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": null
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "filename": "document.pdf",
            "file": "https://example.com/file.jpg",
            "parser": "550e8400-e29b-41d4-a716-446655440000",
            "labels": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "rawUserDefineMetadata": null
        }
    ]
);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "filename": string // File name
  "file": string (uri) // File to upload
  "fileType": string
  "knowledgeBase"?: // Optional
  {
    "id": string (uuid)
    "name": string
  }
  "size": integer
  "status":
  {
  }
  "parser": {
  {
    "id": string (uuid)
    "name": string
    "provider":
    {
    }
    "order"?: integer // Optional
    "supportsDiarization": boolean
    "isTimestampSttProvider": boolean
  }
  }
  "labels"?: [ // Optional
  {
    "id": string (uuid)
    "name": string
  }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "speakerLabels": [
  {
    "id": string (uuid)
```

```

    "originalLabel": string // Original speaker label from diarization
    (e.g., SPEAKER_00)
    "customName"?: string // User-defined speaker name (e.g., John)
    (Optional)
    "displayName": string
  }
]
"vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
"chunksCount": integer // Number of chunks/nodes generated from this
file
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"createdAt": string (timestamp)
}

```

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "Response string",
  "file": "https://example.com/file.jpg",
  "fileType": "Response string",
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "size": 456,
  "status": {},
  "parser": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "provider": {},
    "order": 456,
    "supportsDiarization": false,
    "isTimestampSttProvider": false
  },
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
}

```

```
"rawUserDefineMetadata": null,
"speakerLabels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "originalLabel": "Response string",
    "customName": "Response string",
    "displayName": "Response string"
  }
],
"vectorStorageSize": 456,
"chunksCount": 456,
"waitingTime": 456,
"processingTime": 456,
"processingTimeDetails": null,
"createdAt": "Response string"
}
```

Delete File

DELETE

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Knowledge Base File.
<code>knowledgeBasePk</code>	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/files/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}

?>
```

Response

Status Code	Description
204	No response body

Create File Translation

POST

/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/

Parameters

Parameter	Required	Type	Description
knowledgeBaseFilePk	V	string	
knowledgeBasePk	V	string	

Request

Request Parameters

Field	Type	Required	Description
sourceLanguage	string	No	
targetLanguage	string	Yes	
customPrompt	string	No	
largeLanguageModelId	string (uuid)	Yes	

Request Schema Example

```
{
  "sourceLanguage"?: string // Optional
  "targetLanguage": string
  "customPrompt"?: string // Optional
  "largeLanguageModelId": string (uuid)
}
```

Request Example Values

```
{
  "sourceLanguage": "Example String",
  "targetLanguage": "Example String",
  "customPrompt": "Example String",
  "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "sourceLanguage": "Example String",
  "targetLanguage": "Example String",
  "customPrompt": "Example String",
  "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "sourceLanguage": "Example String",
  "targetLanguage": "Example String",
  "customPrompt": "Example String",
  "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-  
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "sourceLanguage": "Example String",
    "targetLanguage": "Example String",
    "customPrompt": "Example String",
    "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "sourceLanguage": "Example String",
            "targetLanguage": "Example String",
            "customPrompt": "Example String",
            "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "sourceLanguage?": string // Optional
  "targetLanguage": string
  "customPrompt?": string // Optional
  "largeLanguageModelId": string (uuid)
}
```

Response Example Values

```
{
  "sourceLanguage": "Response string",
  "targetLanguage": "Response string",
  "customPrompt": "Response string",
  "largeLanguageModelId": "550e8400-e29b-41d4-a716-446655440000"
}
```

List File Translations

GET

/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/

Parameters

Parameter	Required	Type	Description
knowledgeBaseFilePk	V	string	
knowledgeBasePk	V	string	

page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/?
page=1&pageSize=1"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/?
page=1&pageSize=1", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-  
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/?  
page=1&pageSize=1"  
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.get(url, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/?
page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
}
```

```
"next"?: string (uri) // Optional
"previous"?: string (uri) // Optional
"results": [
  {
    "id": string (uuid)
    "chatbotFileId": string (uuid)
    "chatbotFileFilename": string
    "organization": string (uuid)
    "largeLanguageModel":
    {
      "id": string (uuid)
      "name": string
    }
    "sourceLanguage": string
    "targetLanguage": string
    "customPrompt": string // User-provided terminology/glossary rules
    "status":
    {
    }
    "errorMessage": string
    "translatedFileUrl": string
    "bilingualFileUrl": string
    "processDuration": string
    "processingDurationSeconds": number (double)
    "pageCount": integer
    "originalWordCount": integer
    "translatedWordCount": integer
    "inputTokensUsed": integer
    "outputTokensUsed": integer
    "totalTokensUsed": integer
    "originalFileSize": integer
    "translatedFileSize": integer
    "bilingualFileSize": integer
    "llmTranslationDuration": string // Total time spent on LLM API
calls
    "llmTranslationDurationSeconds": number (double)
    "llmCallCount": integer // Number of LLM API calls made
    "avgLlmCallDuration": number (double) // Average time per LLM call
in seconds
    "pdfParsingDuration": string // Time spent on PDF parsing and
processing
    "pdfParsingDurationSeconds": number (double)
    "fileSavingDuration": string // Time spent on saving output files
    "fileSavingDurationSeconds": number (double)
    "startedAt": string (timestamp)
```

```
    "completedAt": string (timestamp)
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
}
```

Response Example Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "chatbotFileId": "550e8400-e29b-41d4-a716-446655440000",
      "chatbotFileFilename": "Response string",
      "organization": "550e8400-e29b-41d4-a716-446655440000",
      "largeLanguageModel": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      },
      "sourceLanguage": "Response string",
      "targetLanguage": "Response string",
      "customPrompt": "Response string",
      "status": {},
      "errorMessage": "Response string",
      "translatedFileUrl": "Response string",
      "bilingualFileUrl": "Response string",
      "processDuration": "Response string",
      "processingDurationSeconds": 456,
      "pageCount": 456,
      "originalWordCount": 456,
      "translatedWordCount": 456,
      "inputTokensUsed": 456,
      "outputTokensUsed": 456,
      "totalTokensUsed": 456,
      "originalFileSize": 456,
      "translatedFileSize": 456,
      "bilingualFileSize": 456,
      "llmTranslationDuration": "Response string",
      "llmTranslationDurationSeconds": 456,
      "llmCallCount": 456,
    }
  ]
}
```



```
    "avgLlmCallDuration": 456,
    "pdfParsingDuration": "Response string",
    "pdfParsingDurationSeconds": 456,
    "fileSavingDuration": "Response string",
    "fileSavingDurationSeconds": 456,
    "startedAt": "Response string",
    "completedAt": "Response string",
    "createdAt": "Response string",
    "updatedAt": "Response string"
  }
]
```

Get Translation Details

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this File Translation.
<code>knowledgeBaseFilePk</code>	V	string	
<code>knowledgeBasePk</code>	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-  
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550  
e8400-e29b-41d4-a716-446655440000/"
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.get(url, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```

"chatbotFileId": string (uuid)
"chatbotFileFilename": string
"organization": string (uuid)
"largeLanguageModel":
{
  "id": string (uuid)
  "name": string
}
"sourceLanguage": string
"targetLanguage": string
"customPrompt": string // User-provided terminology/glossary rules
"status":
{
}
"errorMessage": string
"translatedFileUrl": string
"bilingualFileUrl": string
"processDuration": string
"processingDurationSeconds": number (double)
"pageCount": integer
"originalWordCount": integer
"translatedWordCount": integer
"inputTokensUsed": integer
"outputTokensUsed": integer
"totalTokensUsed": integer
"originalFileSize": integer
"translatedFileSize": integer
"bilingualFileSize": integer
"llmTranslationDuration": string // Total time spent on LLM API calls
"llmTranslationDurationSeconds": number (double)
"llmCallCount": integer // Number of LLM API calls made
"avgLlmCallDuration": number (double) // Average time per LLM call in
seconds
"pdfParsingDuration": string // Time spent on PDF parsing and
processing
"pdfParsingDurationSeconds": number (double)
"fileSavingDuration": string // Time spent on saving output files
"fileSavingDurationSeconds": number (double)
"startedAt": string (timestamp)
"completedAt": string (timestamp)
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "chatbotFileId": "550e8400-e29b-41d4-a716-446655440000",
  "chatbotFileFilename": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "largeLanguageModel": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "sourceLanguage": "Response string",
  "targetLanguage": "Response string",
  "customPrompt": "Response string",
  "status": {},
  "errorMessage": "Response string",
  "translatedFileUrl": "Response string",
  "bilingualFileUrl": "Response string",
  "processDuration": "Response string",
  "processingDurationSeconds": 456,
  "pageCount": 456,
  "originalWordCount": 456,
  "translatedWordCount": 456,
  "inputTokensUsed": 456,
  "outputTokensUsed": 456,
  "totalTokensUsed": 456,
  "originalFileSize": 456,
  "translatedFileSize": 456,
  "bilingualFileSize": 456,
  "llmTranslationDuration": "Response string",
  "llmTranslationDurationSeconds": 456,
  "llmCallCount": 456,
  "avgLlmCallDuration": 456,
  "pdfParsingDuration": "Response string",
  "pdfParsingDurationSeconds": 456,
  "fileSavingDuration": "Response string",
  "fileSavingDurationSeconds": 456,
  "startedAt": "Response string",
  "completedAt": "Response string",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

Delete Translation

DELETE

/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this File Translation.
knowledgeBaseFilePk	V	string	
knowledgeBasePk	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-  
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550  
e8400-e29b-41d4-a716-446655440000/"
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.delete(url, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}

?>
```

Response

Status Code	Description
204	No response body

Download Translated File

GET

/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/{id}/download/

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this File Translation.
knowledgeBaseFilePk	V	string	
knowledgeBasePk	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550e8400-e29b-41d4-a716-446655440000/download/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/download/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-  
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550  
e8400-e29b-41d4-a716-446655440000/download/"
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.get(url, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-
bases/{knowledgeBasePk}/files/{knowledgeBaseFilePk}/translations/550
e8400-e29b-41d4-a716-446655440000/download/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```

"chatbotFileId": string (uuid)
"chatbotFileFilename": string
"organization": string (uuid)
"largeLanguageModel":
{
  "id": string (uuid)
  "name": string
}
"sourceLanguage": string
"targetLanguage": string
"customPrompt": string // User-provided terminology/glossary rules
"status":
{
}
"errorMessage": string
"translatedFileUrl": string
"bilingualFileUrl": string
"processDuration": string
"processingDurationSeconds": number (double)
"pageCount": integer
"originalWordCount": integer
"translatedWordCount": integer
"inputTokensUsed": integer
"outputTokensUsed": integer
"totalTokensUsed": integer
"originalFileSize": integer
"translatedFileSize": integer
"bilingualFileSize": integer
"llmTranslationDuration": string // Total time spent on LLM API calls
"llmTranslationDurationSeconds": number (double)
"llmCallCount": integer // Number of LLM API calls made
"avgLlmCallDuration": number (double) // Average time per LLM call in
seconds
"pdfParsingDuration": string // Time spent on PDF parsing and
processing
"pdfParsingDurationSeconds": number (double)
"fileSavingDuration": string // Time spent on saving output files
"fileSavingDurationSeconds": number (double)
"startedAt": string (timestamp)
"completedAt": string (timestamp)
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Response Example Values


```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "chatbotFileId": "550e8400-e29b-41d4-a716-446655440000",
  "chatbotFileFilename": "Response string",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "largeLanguageModel": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "sourceLanguage": "Response string",
  "targetLanguage": "Response string",
  "customPrompt": "Response string",
  "status": {},
  "errorMessage": "Response string",
  "translatedFileUrl": "Response string",
  "bilingualFileUrl": "Response string",
  "processDuration": "Response string",
  "processingDurationSeconds": 456,
  "pageCount": 456,
  "originalWordCount": 456,
  "translatedWordCount": 456,
  "inputTokensUsed": 456,
  "outputTokensUsed": 456,
  "totalTokensUsed": 456,
  "originalFileSize": 456,
  "translatedFileSize": 456,
  "bilingualFileSize": 456,
  "llmTranslationDuration": "Response string",
  "llmTranslationDurationSeconds": 456,
  "llmCallCount": 456,
  "avgLlmCallDuration": 456,
  "pdfParsingDuration": "Response string",
  "pdfParsingDurationSeconds": 456,
  "fileSavingDuration": "Response string",
  "fileSavingDurationSeconds": 456,
  "startedAt": "Response string",
  "completedAt": "Response string",
  "createdAt": "Response string",
  "updatedAt": "Response string"
}
```

List Knowledge Base Documents

GET

/api/v1/knowledge-bases/{knowledgeBasePk}/documents/

Parameters

Parameter	Required	Type	Description
knowledgeBasePk	V	string	
chatbotFileId	X	string	[Deprecated] File ID (please use knowledge_base_file_id)
knowledgeBaseFileId	X	string	Knowledge base file ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/?chatbotFileId=example&knowledgeBaseFileId=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/documents/?
chatbotFileId=example&knowledgeBaseFileId=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/?chatbotFileId=example&knowledgeBaseFileId=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/documents/?
chatbotFileId=example&knowledgeBaseFileId=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
```

```

    "id": string (uuid)
    "fileName": string
    "pageNumber"?: integer // Optional
    "chatbotFile": string (uuid)
    "excludedEmbedMetadataKeys"?: object // List[str]: List of metadata
keys to exclude from embedding (Optional)
    "excludedLlmMetadataKeys"?: object // List[str]: List of metadata
keys to exclude from LLM context (Optional)
    "llamaIndexObjectJson"?: object // Optional
    "metadata"?: object // Optional
    "text": string
    "updatedAt": string (timestamp)
  }
]

```

Response Example Values

```

[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "fileName": "Response string",
    "pageNumber": 456,
    "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
    "excludedEmbedMetadataKeys": null,
    "excludedLlmMetadataKeys": null,
    "llamaIndexObjectJson": null,
    "metadata": null,
    "text": "Response string",
    "updatedAt": "Response string"
  }
]

```

Update Document

PUT

/api/v1/knowledge-bases/{knowledgeBasePk}/documents/{id}

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Chatbot Document.
<code>knowledgeBasePk</code>	V	string	

Request

Request Parameters

Field	Type	Required	Description
fileName	string	Yes	
pageNumber	integer	No	
chatbotFile	string (uuid)	Yes	
excludedEmbedMetadataKeys	object	No	List[str]: List of metadata keys to exclude from embedding
excludedLlmMetadataKeys	object	No	List[str]: List of metadata keys to exclude from LLM context
llamaIndexObjectJson	object	No	
metadata	object	No	
text	string	Yes	

Request Schema Example

```
{
  "fileName": string
  "pageNumber?": integer // Optional
  "chatbotFile": string (uuid)
  "excludedEmbedMetadataKeys?": object // List[str]: List of metadata
```

```
keys to exclude from embedding (Optional)
  "excludedLlmMetadataKeys"?: object // List[str]: List of metadata keys
to exclude from LLM context (Optional)
  "llamaIndexObjectJson"?: object // Optional
  "metadata"?: object // Optional
  "text": string
}
```

Request Example Values

```
{
  "fileName": "Example string",
  "pageNumber": 123,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "This is an example message"
}
```

Code Examples


```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "fileName": "Example string",
  "pageNumber": 123,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "This is an example message"
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "fileName": "Example string",
  "pageNumber": 123,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "This is an example message"
};

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "fileName": "Example string",
    "pageNumber": 123,
    "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
    "excludedEmbedMetadataKeys": null,
    "excludedLlmMetadataKeys": null,
    "llamaIndexObjectJson": null,
    "metadata": null,
    "text": "This is an example message"
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "fileName": "Example string",
            "pageNumber": 123,
            "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
            "excludedEmbedMetadataKeys": null,
            "excludedLlmMetadataKeys": null,
            "llamaIndexObjectJson": null,
            "metadata": null,
            "text": "This is an example message"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "fileName": string
  "pageNumber"?: integer // Optional
  "chatbotFile": string (uuid)
  "excludedEmbedMetadataKeys"?: object // List[str]: List of metadata
keys to exclude from embedding (Optional)
  "excludedLlmMetadataKeys"?: object // List[str]: List of metadata keys
to exclude from LLM context (Optional)
  "llamaIndexObjectJson"?: object // Optional
  "metadata"?: object // Optional
  "text": string
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "fileName": "Response string",
  "pageNumber": 456,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "Response string",
  "updatedAt": "Response string"
}
```

Partially Update Document

PATCH

/api/v1/knowledge-bases/{knowledgeBasePk}/documents/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Chatbot Document.
knowledgeBasePk	V	string	

Request

Request Parameters

Field	Type	Required	Description
fileName	string	No	
pageNumber	integer	No	
chatbotFile	string (uuid)	No	
excludedEmbedMetadataKeys	object	No	List[str]: List of metadata keys to exclude from embedding
excludedLlmMetadataKeys	object	No	List[str]: List of metadata keys to exclude from LLM context
llamaIndexObjectJson	object	No	
metadata	object	No	
text	string	No	

Request Schema Example

```
{
  "fileName?": string // Optional
  "pageNumber?": integer // Optional
  "chatbotFile?": string (uuid) // Optional
  "excludedEmbedMetadataKeys?": object // List[str]: List of metadata
keys to exclude from embedding (Optional)
  "excludedLlmMetadataKeys?": object // List[str]: List of metadata keys
to exclude from LLM context (Optional)
  "llamaIndexObjectJson?": object // Optional
  "metadata?": object // Optional
  "text?": string // Optional
}
```

Request Example Values

```
{
  "fileName": "Example string",
  "pageNumber": 123,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "This is an example message"
}
```

Code Examples

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "fileName": "Example string",
  "pageNumber": 123,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "This is an example message"
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "fileName": "Example string",
  "pageNumber": 123,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "This is an example message"
};

axios.patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "fileName": "Example string",
    "pageNumber": 123,
    "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
    "excludedEmbedMetadataKeys": null,
    "excludedLlmMetadataKeys": null,
    "llamaIndexObjectJson": null,
    "metadata": null,
    "text": "This is an example message"
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "fileName": "Example string",
            "pageNumber": 123,
            "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
            "excludedEmbedMetadataKeys": null,
            "excludedLlmMetadataKeys": null,
            "llamaIndexObjectJson": null,
            "metadata": null,
            "text": "This is an example message"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "fileName": string
  "pageNumber"?: integer // Optional
  "chatbotFile": string (uuid)
  "excludedEmbedMetadataKeys"?: object // List[str]: List of metadata
keys to exclude from embedding (Optional)
  "excludedLlmMetadataKeys"?: object // List[str]: List of metadata keys
to exclude from LLM context (Optional)
  "llamaIndexObjectJson"?: object // Optional
  "metadata"?: object // Optional
  "text": string
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "fileName": "Response string",
  "pageNumber": 456,
  "chatbotFile": "550e8400-e29b-41d4-a716-446655440000",
  "excludedEmbedMetadataKeys": null,
  "excludedLlmMetadataKeys": null,
  "llamaIndexObjectJson": null,
  "metadata": null,
  "text": "Response string",
  "updatedAt": "Response string"
}
```

Delete Document

DELETE

/api/v1/knowledge-bases/{knowledgeBasePk}/documents/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Chatbot Document.
knowledgeBasePk	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiaagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/documents/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}

?>
```

Response

Status Code	Description
204	No response body

Create New FAQ

POST

/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/

Parameters

Parameter	Required	Type	Description
knowledgeBasePk	V	string	

Request

Request Parameters

Field	Type	Required	Description
question	string	Yes	
answer	string	Yes	
answerMediaUrls	object	No	URLs for media files such as images and videos in the answer
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	
knowledgeBase	object (with 2 properties: id, name)	No	
knowledgeBase.id	string (uuid)	Yes	

Request Schema Example

```
{
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs for media files such as images and
  videos in the answer (Optional)
  "labels"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "knowledgeBase"?: // Optional
  {
    "id": string (uuid)
  }
}
```

Request Example Values

```
{
  "question": "Example String",
  "answer": "Example String",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "question": "Example String",
  "answer": "Example String",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "question": "Example String",
  "answer": "Example String",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "question": "Example String",
    "answer": "Example String",
    "answerMediaUrls": null,
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": null,
    "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000"
    }
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "question": "Example String",
            "answer": "Example String",
            "answerMediaUrls": null,
            "labels": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "rawUserDefineMetadata": null,
            "knowledgeBase": {
                "id": "550e8400-e29b-41d4-a716-446655440000"
            }
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 201

Response Schema Example

```
{
  "id": string (uuid)
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs for media files such as images and
  videos in the answer (Optional)
  "hitsCount": integer
  "embeddingTokensCount": integer // Number of embedding tokens used when
  creating this FAQ
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "knowledgeBase"?: // Optional
  {
    "id": string (uuid)
    "name": string
  }
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "question": "Response string",
  "answer": "Response string",
  "answerMediaUrls": null,
  "hitsCount": 456,
  "embeddingTokensCount": 456,
  "labels": [
```

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
},
{
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
}
```

Batch Delete FAQ

POST

</api/v1/knowledge-bases/{knowledgeBasePk}/faqs/batch-delete/>

Parameters

Parameter	Required	Type	Description
<code>knowledgeBasePk</code>	V	string	

Request

Request Parameters

Field	Type	Required	Description
<code>ids</code>	<code>array[string]</code>	No	

Request Schema Example


```
{
  "ids?": [ // Optional
    string (uuid)
  ]
}
```

Request Example Values

```
{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/batch-delete/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "ids": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/batch-delete/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/batch-delete/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "ids": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
post("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/batch-delete/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "ids": [
                "550e8400-e29b-41d4-a716-446655440000"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
-------------	-------------

204

No response body

List All FAQ in Knowledge Base

GET

/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/

Parameters

Parameter	Required	Type	Description
knowledgeBasePk	V	string	
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	Search keywords, supports multi-condition queries separated by spaces. Searchable: FAQ ID, question, answer, label name, Metadata value

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/?
page=1&pageSize=1&query=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/?page=1&pageSize=1&query=example",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/?page=1&pageSize=1&query=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-
41d4-a716-446655440000/faqs/?page=1&pageSize=1&query=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
}
```

```

"previous"?: string (uri) // Optional
"results": [
  {
    "id": string (uuid)
    "question": string
    "answer": string
    "answerMediaUrls"?: object // URLs for media files such as images
and videos in the answer (Optional)
    "hitsCount": integer
    "embeddingTokensCount": integer // Number of embedding tokens used
when creating this FAQ
    "labels"?: [ // Optional
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "rawUserDefineMetadata"?: object // Optional
    "knowledgeBase"?: // Optional
    {
      "id": string (uuid)
      "name": string
    }
  }
]
}

```

Response Example Values

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "question": "Response string",
      "answer": "Response string",
      "answerMediaUrls": null,
      "hitsCount": 456,
      "embeddingTokensCount": 456,
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",

```

```
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
}
]
```

Get FAQ Details

GET

`/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this FAQ.
<code>knowledgeBasePk</code>	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]
);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```



```
"question": string
"answer": string
"answerMediaUrls"?: object // URLs for media files such as images and
videos in the answer (Optional)
"hitsCount": integer
"embeddingTokensCount": integer // Number of embedding tokens used when
creating this FAQ
"labels"?: [ // Optional
  {
    "id": string (uuid)
    "name": string
  }
]
"rawUserDefineMetadata"?: object // Optional
"knowledgeBase"?: // Optional
{
  "id": string (uuid)
  "name": string
}
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "question": "Response string",
  "answer": "Response string",
  "answerMediaUrls": null,
  "hitsCount": 456,
  "embeddingTokensCount": 456,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
}
```

```
}  
}
```

Update FAQ

PUT`/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this FAQ.
<code>knowledgeBasePk</code>	V	string	

Request

Request Parameters

Field	Type	Required	Description
question	string	Yes	
answer	string	Yes	
answerMediaUrls	object	No	URLs for media files such as images and videos in the answer
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	

Field	Type	Required	Description
knowledgeBase	object (with 2 properties: id, name)	No	
knowledgeBase.id	string (uuid)	Yes	

Request Schema Example

```
{
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs for media files such as images and
  videos in the answer (Optional)
  "labels"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "knowledgeBase"?: // Optional
  {
    "id": string (uuid)
  }
}
```

Request Example Values

```
{
  "question": "Example String",
  "answer": "Example String",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
}
```

```
}  
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)  
curl -X PUT "https://api.maiagent.ai/api/v1/knowledge-  
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-  
a716-446655440000/"  
  
-H "Authorization: Api-Key YOUR_API_KEY"  
  
-H "Content-Type: application/json"  
  
-d '{  
  "question": "Example String",  
  "answer": "Example String",  
  "answerMediaUrls": null,  
  "labels": [  
    {  
      "id": "550e8400-e29b-41d4-a716-446655440000"  
    }  
  ],  
  "rawUserDefineMetadata": null,  
  "knowledgeBase": {  
    "id": "550e8400-e29b-41d4-a716-446655440000"  
  }  
}'  
  
# Make sure to replace YOUR_API_KEY and verify the request data  
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "question": "Example String",
  "answer": "Example String",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
};

axios.put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "question": "Example String",
    "answer": "Example String",
    "answerMediaUrls": null,
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": null,
    "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000"
    }
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->put("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/",
    [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "question": "Example String",
            "answer": "Example String",
            "answerMediaUrls": null,
            "labels": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "rawUserDefineMetadata": null,
            "knowledgeBase": {
                "id": "550e8400-e29b-41d4-a716-446655440000"
            }
        }
    ]
);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs for media files such as images and
  videos in the answer (Optional)
  "hitsCount": integer
  "embeddingTokensCount": integer // Number of embedding tokens used when
  creating this FAQ
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "knowledgeBase"?: // Optional
  {
    "id": string (uuid)
    "name": string
  }
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "question": "Response string",
  "answer": "Response string",
  "answerMediaUrls": null,
```



```
"hitsCount": 456,
"embeddingTokensCount": 456,
"labels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"rawUserDefineMetadata": null,
"knowledgeBase": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
}
}
```

Partially Update FAQ

PATCH

/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/{id}

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this FAQ.
<code>knowledgeBasePk</code>	V	string	

Request

Request Parameters

Field	Type	Required	Description
question	string	No	
answer	string	No	
answerMediaUrls	object	No	URLs for media files such as images and videos in the answer
labels	array[IdName]	No	
rawUserDefineMetadata	object	No	
knowledgeBase	object (with 2 properties: id, name)	No	
knowledgeBase.id	string (uuid)	Yes	

Request Schema Example

```
{
  "question"?: string // Optional
  "answer"?: string // Optional
  "answerMediaUrls"?: object // URLs for media files such as images and
  videos in the answer (Optional)
  "labels"?: [ // Optional
    {
      "id": string (uuid)
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "knowledgeBase"?: // Optional
  {
    "id": string (uuid)
  }
}
```

Request Example Values

```
{
  "question": "Example String",
  "answer": "Example String",
```

```
"answerMediaUrls": null,  
"labels": [  
  {  
    "id": "550e8400-e29b-41d4-a716-446655440000"  
  }  
],  
"rawUserDefineMetadata": null,  
"knowledgeBase": {  
  "id": "550e8400-e29b-41d4-a716-446655440000"  
}  
}
```

Code Examples

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "question": "Example String",
  "answer": "Example String",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "question": "Example String",
  "answer": "Example String",
  "answerMediaUrls": null,
  "labels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000"
    }
  ],
  "rawUserDefineMetadata": null,
  "knowledgeBase": {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
};

axios.patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "question": "Example String",
    "answer": "Example String",
    "answerMediaUrls": null,
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000"
        }
    ],
    "rawUserDefineMetadata": null,
    "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000"
    }
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    patch("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
    e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-
    446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "question": "Example String",
            "answer": "Example String",
            "answerMediaUrls": null,
            "labels": [
                {
                    "id": "550e8400-e29b-41d4-a716-446655440000"
                }
            ],
            "rawUserDefineMetadata": null,
            "knowledgeBase": {
                "id": "550e8400-e29b-41d4-a716-446655440000"
            }
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "question": string
  "answer": string
  "answerMediaUrls"?: object // URLs for media files such as images and
  videos in the answer (Optional)
  "hitsCount": integer
  "embeddingTokensCount": integer // Number of embedding tokens used when
  creating this FAQ
  "labels"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "rawUserDefineMetadata"?: object // Optional
  "knowledgeBase"?: // Optional
  {
    "id": string (uuid)
    "name": string
  }
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "question": "Response string",
  "answer": "Response string",
  "answerMediaUrls": null,
```



```
"hitsCount": 456,
"embeddingTokensCount": 456,
"labels": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
],
"rawUserDefineMetadata": null,
"knowledgeBase": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string"
}
}
```

Delete FAQ

DELETE

`/api/v1/knowledge-bases/{knowledgeBasePk}/faqs/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this FAQ.
<code>knowledgeBasePk</code>	V	string	

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/knowledge-
bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>delete("https://api.maiagent.ai/api/v1/knowledge-bases/550e8400-
e29b-41d4-a716-446655440000/faqs/550e8400-e29b-41d4-a716-
446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

API call example (Shell)

...

目錄

- 1. List the AI Assistant’ s Knowledge Base Files
- 2. List the AI Assistant’ s Knowledge Base FAQs
- 3. List Text Nodes for All AI Assistant Files
- 4. Get Text Nodes for a Specific AI Assistant File
- 5. Test Search
- 6. List Parsers Supported by File Type

API call example (Shell)

List the AI Assistant’ s Knowledge Base Files

GET `/api/v1/chatbots/{id}/knowledge-bases-files/`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	
<code>largeLanguageModel</code>	X	string	
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.
<code>query</code>	X	string	
<code>replyMode</code>	X	string	
<code>source</code>	X	string	`self_built`: self_built; `built_in`: built_in; `all`: all;

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```



```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e-
    >getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
```

```
"results": [
  {
    "id": string (uuid)
    "filename": string // File name
    "file": string (uri) // File to upload
    "fileType": string
    "knowledgeBase"?: // optional
    {
      "id": string (uuid)
      "name": string
    }
    "size": integer
    "status":
    {
    }
    "parser":
    {
      "id": string (uuid)
      "name": string
      "provider":
      {
      }
      "isTimestampSttProvider": boolean
    }
    "labels"?: [ // optional
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "rawUserDefineMetadata"?: object // optional
    "speakerLabels": [
      {
        "id": string (uuid)
        "originalLabel": string // Original speaker label from
diarization (e.g., SPEAKER_00)
        "customName"?: string // User-defined speaker name (e.g., John)
(optional)
        "displayName": string
      }
    ]
    "vectorStorageSize": integer // Size of vectors for this file in
Elasticsearch (bytes)
    "chunksCount": integer // Number of chunks/nodes generated from
this file
```

```
"waitingTime": number (double)
"processingTime": number (double)
"processingTimeDetails": object
"previewUrl": string // For presentations (pptx/ppt), returns the
URL of a derived, previewable PDF file (displayed in the frontend PDF
viewer); otherwise, None.
```

Generate the URL with the same CustomizedFileFieldSerializer used by the `file` field to ensure a consistent format (including the storage provider's presign/host rules). Do not use `get_absolute_url()`—it produces an incorrect URL format.

```
"createdAt": string (timestamp)
}
]
}
```

Response Example Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "filename": "response string",
      "file": "https://example.com/file.jpg",
      "fileType": "response string",
      "knowledgeBase": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      },
      "size": 456,
      "status": {},
      "parser": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string",
        "provider": {},
        "isTimestampSttProvider": false
      },
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",
          "name": "response string"
        }
      ]
    }
  ]
}
```

```
    }
  ],
  "rawUserDefineMetadata": null,
  "speakerLabels": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "originalLabel": "response string",
      "customName": "response string",
      "displayName": "response string"
    }
  ],
  "vectorStorageSize": 456,
  "chunksCount": 456,
  "waitingTime": 456,
  "processingTime": 456,
  "processingTimeDetails": null,
  "previewUrl": "response string",
  "createdAt": "response string"
}
]
```

List the AI Assistant's Knowledge Base FAQs

GET

</api/v1/chatbots/{id}/knowledge-bases-faqs/>

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	
<code>largeLanguageModel</code>	X	string	
<code>page</code>	X	integer	A page number within the paginated result set.

pageSize	X	integer	Number of results to return per page.
query	X	string	
replyMode	X	string	
source	X	string	`self_built`:self_built;`built_in`:built_in;`all`:all;

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get(" config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e-
    >getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
```

```

"results": [
  {
    "id": string (uuid)
    "question": string
    "answer": string
    "answerMediaUrls"?: object // URLs of media files in the answer,
such as images and videos (optional)
    "hitsCount": integer
    "embeddingTokensCount": integer // Number of embedding tokens used
to create this FAQ
    "labels"?: [ // optional
      {
        "id": string (uuid)
        "name": string
      }
    ]
    "rawUserDefineMetadata"?: object // optional
    "knowledgeBase": // Always taken from the knowledge base in the
URL; a value in the request body is ignored.
    {
      "id": string (uuid)
      "name": string
    }
  }
]
}

```

Response Example Values

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "question": "response string",
      "answer": "response string",
      "answerMediaUrls": null,
      "hitsCount": 456,
      "embeddingTokensCount": 456,
      "labels": [
        {
          "id": "550e8400-e29b-41d4-a716-446655440000",

```

```
        "name": "response string"
      }
    ],
    "rawUserDefineMetadata": null,
    "knowledgeBase": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "response string"
    }
  }
]
}
```

List Text Nodes for All AI Assistant Files

GET

</api/v1/chatbot-text-nodes/>

Parameters

Parameter	Required	Type	Description
<code>chatbotFile</code>	X	string	[Deprecated] Chatbot file ID (use knowledge_base_file)
<code>knowledgeBaseFile</code>	X	string	Knowledge base file ID (recommended)
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/chatbot-text-nodes/?
chatbotFile=example&knowledgeBaseFile=example&page=1&pageSize=1"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/chatbot-text-nodes/?
chatbotFile=example&knowledgeBaseFile=example&page=1&pageSize=1",
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/chatbot-text-nodes/?\nchatbotFile=example&knowledgeBaseFile=example&page=1&pageSize=1"\nheaders = {\n    "Authorization": "Api-Key YOUR_API_KEY"\n}\n\nresponse = requests.get(url, headers=headers)\ntry:\n    print("Response received successfully:")\n    print(response.json())\nexcept Exception as e:\n    print("An error occurred while making the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/chatbot-text-nodes/?
chatbotFile=example&knowledgeBaseFile=example&page=1&pageSize=1", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e-
>getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
}
```



```

"next"?: string (uri) // optional
"previous"?: string (uri) // optional
"results": [
  {
    "id": string (uuid)
    "charactersCount": integer
    "hitsCount": integer
    "text": string
    "updatedAt": string (timestamp)
    "filename": string
    "chatbotFile": object // Returns complete ChatbotFile information,
including the file URL and type, to support image previews in the
frontend
    "knowledgeBaseFile": object // Same as get_chatbot_file for
backward compatibility
    "pageNumber": integer // Backward-compatible alias of
``page_start``.
    "pageStart": integer
    "pageEnd": integer
    "citationTitle": string // Source title displayed in the inline
citation hover card
    "citationDescription": string // Source description displayed in
the inline citation hover card
    "citationQuote": string // Quoted text displayed in the inline
citation hover card
    "hasImage": boolean // Indicates whether an image is included
(checks the file type or Markdown images in text)
    "imageUrl": string // Extracted image URL (uses the file URL first;
otherwise extracts it from Markdown)
    "displayText": string // Returns the complete text with image
Markdown removed
    "displayTitle": string // Returns the display title (fallback:
citation_title -> filename)
    "highlightedText": string // Return text with matched terms wrapped
in ``<mark>`` tags.

```

Priority:

1. ES/OpenSearch native highlight (set by retrieve_api via ``\_es\_highlighted\_text``)
2. Python regex fallback (keyword-based, works for all backends)

```

"labels": [
  {
    "id": string (uuid)
    "name": string
  }

```

```

    ]
    "rawUserDefineMetadata"?: object // Users can define their own
metadata keys and values (optional)
    "metadataEnabledForSearch": object // Map each user-defined
metadata key on this node to whether it was sent into the LLM.

A key reaches the LLM only when its ``MetadataKey.enabled_for_search`` is
True *and*
the node carries a value for it. Since this maps over
``raw_user_define_metadata``
(the keys the cited-documents block displays, all of which have a value),
``enabled_for_search`` alone decides the flag. The source of truth is the
node's
knowledge base ``MetadataKey`` current setting, matching the AI Search
indicator.

    "startCharIdx": integer
    "endCharIdx": integer
  }
]
}

```

Response Example Values

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "charactersCount": 456,
      "hitsCount": 456,
      "text": "response string",
      "updatedAt": "response string",
      "filename": "response string",
      "chatbotFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "example response name",
        "description": "example response description"
      },
      "knowledgeBaseFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "example response name",
        "description": "example response description"
      }
    }
  ]
}

```

```

    },
    "pageNumber": 456,
    "pageStart": 456,
    "pageEnd": 456,
    "citationTitle": "response string",
    "citationDescription": "response string",
    "citationQuote": "response string",
    "hasImage": false,
    "imageUrl": "response string",
    "displayText": "response string",
    "displayTitle": "response string",
    "highlightedText": "response string",
    "labels": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "response string"
      }
    ],
    "rawUserDefineMetadata": null,
    "metadataEnabledForSearch": {
      "key1": "value1",
      "key2": "value2",
      "createdAt": "2025-01-01T00:00:00.000Z"
    },
    "startCharIdx": 456,
    "endCharIdx": 456
  }
]
}

```

Status Code: 403 - You do not have permission to read the text nodes of this file: knowledge base files require `chatbot_knowledge_base_access` and group authorization for the knowledge base; conversation attachments require `chatbot_chatbot_access` for the assistant; inbox files require group authorization for the inbox

Status Code: 404 - The knowledge base file does not exist or does not belong to the current organization

Get Text Nodes for a Specific AI Assistant File

GET

`/api/v1/chatbot-text-nodes/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this ChatbotTextNode.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/chatbot-text-nodes/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Replace YOUR_API_KEY and verify the request data before running
this example.
```

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/chatbot-text-nodes/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred while making the request:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/chatbot-text-nodes/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred while making the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/chatbot-text-nodes/550e8400-
e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred while making the request: ' . $e-
>getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
"charactersCount": integer
"hitsCount": integer
"text": string
"updatedAt": string (timestamp)
"filename": string
"chatbotFile": object // Returns complete ChatbotFile information,
including the file URL and type, to support image previews in the
frontend
"knowledgeBaseFile": object // Same as get_chatbot_file for backward
compatibility
"pageNumber": integer // Backward-compatible alias of ``page_start``.
"pageStart": integer
"pageEnd": integer
"citationTitle": string // Source title displayed in the inline
citation hover card
"citationDescription": string // Source description displayed in the
inline citation hover card
"citationQuote": string // Quoted text displayed in the inline citation
hover card
"hasImage": boolean // Indicates whether an image is included (checks
the file type or Markdown images in text)
"imageUrl": string // Extracted image URL (uses the file URL first;
otherwise extracts it from Markdown)
"displayText": string // Returns the complete text with image Markdown
removed
"displayTitle": string // Returns the display title (fallback:
citation_title -> filename)
"highlightedText": string // Return text with matched terms wrapped in
``<mark>`` tags.
```

Priority:

1. ES/OpenSearch native highlight (set by retrieve_api via
``\_es\_highlighted\_text``)
2. Python regex fallback (keyword-based, works for all backends)

```
"labels": [
  {
    "id": string (uuid)
    "name": string
  }
]
"rawUserDefineMetadata"?: object // Users can define metadata keys and
values. (optional)
"metadataEnabledForSearch": object // Map each user-defined metadata
key on this node to whether it was sent into the LLM.
```


A key reaches the LLM only when its ``MetadataKey.enabled\_for\_search`` is True *and* the node carries a value for it. Since this maps over ``raw\_user\_define\_metadata`` (the keys the cited-documents block displays, all of which have a value), ``enabled\_for\_search`` alone decides the flag. The source of truth is the node's knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.

```
"startCharIdx": integer
"endCharIdx": integer
}
```

Example Response Value

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "charactersCount": 456,
  "hitsCount": 456,
  "text": "response string",
  "updatedAt": "response string",
  "filename": "response string",
  "chatbotFile": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example response name",
    "description": "example response description"
  },
  "knowledgeBaseFile": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "example response name",
    "description": "example response description"
  },
  "pageNumber": 456,
  "pageStart": 456,
  "pageEnd": 456,
  "citationTitle": "response string",
  "citationDescription": "response string",
  "citationQuote": "response string",
  "hasImage": false,
  "imageUrl": "response string",
  "displayText": "response string",
  "displayTitle": "response string",
  "highlightedText": "response string",
  "labels": [
```

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "response string"
},
{
  "rawUserDefineMetadata": null,
  "metadataEnabledForSearch": {
    "key1": "value1",
    "key2": "value2",
    "createdAt": "2025-01-01T00:00:00.000Z"
  },
  "startCharIdx": 456,
  "endCharIdx": 456
}
```

Test Search

POST

/api/v1/chatbots/{id}/search/

Parameters

Parameter	Required	Type	Description
id	V	string	
largeLanguageModel	X	string	
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
replyMode	X	string	

source

X

string

```
`self_built`: self_built; `built_in`:  
built_in; `all`: all;
```

Request Body

Request Parameters

Field	Type	Required	Description
queryMetadata	object	No	Not supported for chatbot-wide search; providing a value returns 400.

Example Request Schema

```
{  
  "queryMetadata"?: object // Not supported for chatbot-wide search;  
  providing a value returns 400. (optional)  
}
```

Example Request Value

```
{  
  "queryMetadata": null  
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "query": "example string",
    "queryMetadata": null
  }'

# Before running this example, replace YOUR_API_KEY and verify the
request data.
```

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "query": "example string",
  "queryMetadata": null
};

axios.post(" data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "query": "example string",
    "queryMetadata": null
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post(" [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "query": "example string",
            "queryMetadata": null
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
    >getMessage();
}
?>
```

Response

Status Code: 200

Example Response Schema

```

{
  "count": integer
  "next"?: string (uri) // optional
  "previous"?: string (uri) // optional
  "results": [
    {
      "id": string (uuid)
      "charactersCount": integer
      "hitsCount": integer
      "text": string
      "updatedAt": string (timestamp)
      "filename": string
      "chatbotFile": object // Returns complete ChatbotFile information,
        including the file URL and type, to support image previews in the
        frontend.
      "knowledgeBaseFile": object // Same as get_chatbot_file; provided
        for backward compatibility.
      "pageNumber": integer // Backward-compatible alias of
        ``page_start``.
      "pageStart": integer
      "pageEnd": integer
      "citationTitle": string // Source title displayed in the inline
        citation hover card.
      "citationDescription": string // Source description displayed in
        the inline citation hover card.
      "citationQuote": string // Quoted text excerpt displayed in the
        inline citation hover card.
      "hasImage": boolean // Indicates whether an image is present
        (checks the file type or Markdown images in text).
      "imageUrl": string // Extracts the image URL (uses the file URL
        first; otherwise extracts it from Markdown).
      "displayText": string // Returns the complete text with Markdown
        image syntax removed.
      "displayTitle": string // Returns the display title (fallback:
        citation_title -> filename).
      "highlightedText": string // Return text with matched terms wrapped
        in ``<mark>`` tags.
    }
  ]
}

```

Priority:

1. ES/OpenSearch native highlight (set by retrieve_api via ``\_es\_highlighted\_text``)
2. Python regex fallback (keyword-based, works for all backends)

```

"labels": [
  {

```



```

        "id": string (uuid)
        "name": string
    }
]
"rawUserDefineMetadata"?: object // Users can define metadata keys
and values. (optional)
"metadataEnabledForSearch": object // Map each user-defined
metadata key on this node to whether it was sent into the LLM.

```

A key reaches the LLM only when its ``MetadataKey.enabled\_for\_search`` is True *and*

the node carries a value for it. Since this maps over

``raw\_user\_define\_metadata``

(the keys the cited-documents block displays, all of which have a value), ``enabled\_for\_search`` alone decides the flag. The source of truth is the node's

knowledge base ``MetadataKey`` current setting, matching the AI Search indicator.

```

        "startCharIdx": integer
        "endCharIdx": integer
    }
]
}

```

Example Response Value

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "charactersCount": 456,
      "hitsCount": 456,
      "text": "response string",
      "updatedAt": "response string",
      "filename": "response string",
      "chatbotFile": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "example response name",
        "description": "example response description"
      },
      "knowledgeBaseFile": {

```

```

        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "example response name",
        "description": "example response description"
    },
    "pageNumber": 456,
    "pageStart": 456,
    "pageEnd": 456,
    "citationTitle": "response string",
    "citationDescription": "response string",
    "citationQuote": "response string",
    "hasImage": false,
    "imageUrl": "response string",
    "displayText": "response string",
    "displayTitle": "response string",
    "highlightedText": "response string",
    "labels": [
        {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "response string"
        }
    ],
    "rawUserDefineMetadata": null,
    "metadataEnabledForSearch": {
        "key1": "value1",
        "key2": "value2",
        "createdAt": "2025-01-01T00:00:00.000Z"
    },
    "startCharIdx": 456,
    "endCharIdx": 456
}
]
}

```

List Parsers Supported by File Type

GET

</api/v1/parsers/supported-file-types/>

Parameters

Parameter	Required	Type	Description
<code>knowledgeBaseId</code>	X	string	Knowledge base ID. When provided and the knowledge base does not support multimodal embeddings, image file types will be filtered out.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/parsers/supported-file-types/?knowledgeBaseId=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Before running this example, replace YOUR_API_KEY and verify the request data.
```

```
const axios = require('axios');

// Configure request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/parsers/supported-file-types/?knowledgeBaseId=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('An error occurred during the request:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/parsers/supported-file-types/?knowledgeBaseId=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("An error occurred during the request:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/parsers/supported-file-types/?
knowledgeBaseId=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'An error occurred during the request: ' . $e-
>getMessage();
}
?>
```

Response

Status Code: 200

Example Response Schema

```
[
  {
```

```

"fileType": string
"parsers": [
  {
    "id": string (uuid)
    "name": string
    "provider":
    {
    }
    "order"?: integer // optional
    "isTimestampSttProvider": boolean
    "asrProvider": object // The provider spec, built once per
    provider row per response.

```

The spec is platform-level, not per parser, and it is large – a whisper-family row carries a 100-entry language catalog in its ``languages`` spec, as codes plus display names. This serializer is nested per row in the knowledge-base file listing and in message citations, so without memoizing, one response rebuilds the same catalog for every row. ``self.context`` is the root serializer's shared dict, which is also what keeps ``get\_active\_diarization\_rows`` at one query per response.

```

    }
  ]
  "autoDetect": boolean // Whether files of this type get parser auto-
  detection after upload
  (the system auto-detects a parser when the user did not choose one).
}
]

```

Example Response Value

```

[
  [
    {
      "fileType": ".pdf",
      "parsers": [
        {
          "id": "ab83f144-5026-4bce-993f-5c982cc19318",
          "name": "MaiAgent Parser",
          "provider": "maiagent",
          "order": 0,
          "isDefault": true
        },
        {

```

```
        "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
        "name": "MaiAgent Parser (Online)",
        "provider": "llama",
        "order": 2,
        "isDefault": false
    }
]
},
{
    "fileType": ".doc",
    "parsers": [
        {
            "id": "ab83f144-5026-4bce-993f-5c982cc19318",
            "name": "MaiAgent Parser",
            "provider": "maiaagent",
            "order": 0,
            "isDefault": true
        },
        {
            "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
            "name": "MaiAgent Parser (Online)",
            "provider": "llama",
            "order": 2,
            "isDefault": false
        }
    ]
},
{
    "fileType": ".docx",
    "parsers": [
        {
            "id": "ab83f144-5026-4bce-993f-5c982cc19318",
            "name": "MaiAgent Parser",
            "provider": "maiaagent",
            "order": 0,
            "isDefault": true
        },
        {
            "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
            "name": "MaiAgent Parser (Online)",
            "provider": "llama",
            "order": 2,
            "isDefault": false
        }
    ]
}
```



```
},
{
  "fileType": ".ppt",
  "parsers": [
    {
      "id": "ab83f144-5026-4bce-993f-5c982cc19318",
      "name": "MaiAgent Parser",
      "provider": "maiagent",
      "order": 0,
      "isDefault": true
    },
    {
      "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
      "name": "MaiAgent Parser (Online)",
      "provider": "llama",
      "order": 2,
      "isDefault": false
    }
  ]
},
{
  "fileType": ".pptx",
  "parsers": [
    {
      "id": "ab83f144-5026-4bce-993f-5c982cc19318",
      "name": "MaiAgent Parser",
      "provider": "maiagent",
      "order": 0,
      "isDefault": true
    },
    {
      "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
      "name": "MaiAgent Parser (Online)",
      "provider": "llama",
      "order": 2,
      "isDefault": false
    }
  ]
},
{
  "fileType": ".xls",
  "parsers": [
    {
      "id": "ab83f144-5026-4bce-993f-5c982cc19318",
      "name": "MaiAgent Parser",
```

```
        "provider": "maiagent",
        "order": 0,
        "isDefault": true
    }
]
},
{
    "fileType": ".xlsx",
    "parsers": [
        {
            "id": "ab83f144-5026-4bce-993f-5c982cc19318",
            "name": "MaiAgent Parser",
            "provider": "maiagent",
            "order": 0,
            "isDefault": true
        }
    ]
},
{
    "fileType": ".csv",
    "parsers": [
        {
            "id": "ab83f144-5026-4bce-993f-5c982cc19318",
            "name": "MaiAgent Parser",
            "provider": "maiagent",
            "order": 0,
            "isDefault": true
        }
    ]
},
{
    "fileType": ".txt",
    "parsers": [
        {
            "id": "ab83f144-5026-4bce-993f-5c982cc19318",
            "name": "MaiAgent Parser",
            "provider": "maiagent",
            "order": 0,
            "isDefault": true
        },
        {
            "id": "22fdccb5-f075-4aad-bb81-048289ca4b25",
            "name": "MaiAgent Parser (Online)",
            "provider": "llama",
            "order": 2,
```

```
        "isDefault": false
    }
]
},
{
    "fileType": ".md",
    "parsers": [
        {
            "id": "ab83f144-5026-4bce-993f-5c982cc19318",
            "name": "MaiAgent Parser",
            "provider": "maiagent",
            "order": 0,
            "isDefault": true
        }
    ]
},
{
    "fileType": ".json",
    "parsers": [
        {
            "id": "ab83f144-5026-4bce-993f-5c982cc19318",
            "name": "MaiAgent Parser",
            "provider": "maiagent",
            "order": 0,
            "isDefault": true
        }
    ]
},
{
    "fileType": ".jsonl",
    "parsers": [
        {
            "id": "ab83f144-5026-4bce-993f-5c982cc19318",
            "name": "MaiAgent Parser",
            "provider": "maiagent",
            "order": 0,
            "isDefault": true
        }
    ]
},
{
    "fileType": ".html",
    "parsers": [
        {
            "id": "22fdccb5-f075-4aad-bb81-048289ca4b25",
```

```
        "name": "MaiAgent Parser (Online)",
        "provider": "llama",
        "order": 2,
        "isDefault": false
    }
]
},
{
    "fileType": ".mp3",
    "parsers": [
        {
            "id": "4c47e305-2eb7-4438-8d3c-d91eb0b06cc0",
            "name": "Azure Speech",
            "provider": "azure",
            "order": 1,
            "isDefault": true
        },
        {
            "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
            "name": "MaiAgent Parser (Online)",
            "provider": "llama",
            "order": 2,
            "isDefault": false
        }
    ]
},
{
    "fileType": ".wav",
    "parsers": [
        {
            "id": "4c47e305-2eb7-4438-8d3c-d91eb0b06cc0",
            "name": "Azure Speech",
            "provider": "azure",
            "order": 1,
            "isDefault": true
        },
        {
            "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
            "name": "MaiAgent Parser (Online)",
            "provider": "llama",
            "order": 2,
            "isDefault": false
        }
    ]
},
```

```
{
  "fileType": ".mp4",
  "parsers": [
    {
      "id": "22fdcbb5-f075-4aad-bb81-048289ca4b25",
      "name": "MaiAgent Parser (Online)",
      "provider": "llama",
      "order": 2,
      "isDefault": false
    }
  ]
}
```

API call example (Shell)

...

目錄

1. Account Registration
2. Change Password
3. Create New Organization
4. Add Member to Specified Organization
5. Get Organization List
6. Get Specific Organization Information
7. Get Current User Details
8. Get Current User Permissions
9. Get Member List for Specified Organization
10. Get Specific Member Details
11. Get Specific Member Details
12. Get Specific Member Details
13. Get Specific Member Details
14. Get Specific Member Details
15. Update Organization Information
16. Update Current User Details
17. Update Member Role Assignment
18. Delete Organization
19. Delete Specified Member
20. Get Member Usage Statistics
21. Get Member Usage Statistics Summary
22. Get AI Assistant Usage Statistics
23. Get AI Assistant Usage Statistics Summary

API call example (Shell)

Account Registration

POST

`/api/v1/auth/registration/`

Request

Request Parameters

Field	Type	Required	Description
email	string (email)	Yes	
password1	string	Yes	
password2	string	Yes	
name	string	Yes	
company	string	Yes	
referralCode	string	No	
authSource	object (with 2 properties: id, name)	No	
authSource.id	string (uuid)	Yes	

Request Schema Example

```
{
  "email": string (email)
  "password1": string
  "password2": string
  "name": string
  "company": string
  "referralCode"?: string // Optional
  "authSource"?: // Optional
  {
    "id": string (uuid)
  }
}
```

Request Example Values

```
{
  "email": "user@example.com",
  "password1": "Example string",
  "password2": "Example string",
```



```
"name": "Example name",
"company": "Example string",
"referralCode": "Example string",
"authSource": {
  "id": "550e8400-e29b-41d4-a716-446655440000"
}
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/auth/registration/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "email": "user@example.com",
  "password1": "Example string",
  "password2": "Example string",
  "name": "Example name",
  "company": "Example string",
  "referralCode": "Example string",
  "authSource": {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
}'

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "email": "user@example.com",
  "password1": "Example string",
  "password2": "Example string",
  "name": "Example name",
  "company": "Example string",
  "referralCode": "Example string",
  "authSource": {
    "id": "550e8400-e29b-41d4-a716-446655440000"
  }
};

axios.post("https://api.maiagent.ai/api/v1/auth/registration/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/auth/registration/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "email": "user@example.com",
    "password1": "Example string",
    "password2": "Example string",
    "name": "Example name",
    "company": "Example string",
    "referralCode": "Example string",
    "authSource": {
        "id": "550e8400-e29b-41d4-a716-446655440000"
    }
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/auth/registration/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "email": "user@example.com",
            "password1": "Example string",
            "password2": "Example string",
            "name": "Example name",
            "company": "Example string",
            "referralCode": "Example string",
            "authSource": {
                "id": "550e8400-e29b-41d4-a716-446655440000"
            }
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "email": string (email)
  "password1": string
  "password2": string
  "name": string
  "company": string
  "referralCode?": string // Optional
  "authSource?": // Optional
  {
    "id": string (uuid)
    "name": string
  }
}
```

Response Example Values

```
{
  "email": "response@example.com",
  "password1": "Response string",
  "password2": "Response string",
  "name": "Response string",
  "company": "Response string",
  "referralCode": "Response string",
  "authSource": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  }
}
```

Change Password

POST

/api/v1/auth/password/change/

Request

Request Parameters

Field	Type	Required	Description
oldPassword	string	Yes	
newPassword1	string	Yes	
newPassword2	string	Yes	

Request Schema Example

```
{
  "oldPassword": string
  "newPassword1": string
  "newPassword2": string
}
```

Request Example Values

```
{
  "oldPassword": "Example string",
  "newPassword1": "Example string",
  "newPassword2": "Example string"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/auth/password/change/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "oldPassword": "Example string",
  "newPassword1": "Example string",
  "newPassword2": "Example string"
}'

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "oldPassword": "Example string",
  "newPassword1": "Example string",
  "newPassword2": "Example string"
};

axios.post("https://api.maiagent.ai/api/v1/auth/password/change/",
data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/auth/password/change/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "oldPassword": "Example string",
    "newPassword1": "Example string",
    "newPassword2": "Example string"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    post("https://api.maiagent.ai/api/v1/auth/password/change/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "oldPassword": "Example string",
            "newPassword1": "Example string",
            "newPassword2": "Example string"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "detail": string
}
```

Response Example Values

```
{
  "detail": "Response string"
}
```

Create New Organization

POST

[/api/v1/organizations/](#)

Request

Request Parameters

Field	Type	Required	Description
name	string	Yes	Organization name
compact_logo	string (binary)	No	Compact logo (optional, available for enterprise plans only)
full_logo	string (binary)	No	Full logo (optional, available for enterprise plans only)

Request Schema Example

```
{
  "name": string // Organization name
  "compact_logo?": string (binary) // Compact logo (optional, available
```

```
for enterprise plans only) (Optional)
  "full_logo"?: string (binary) // Full logo (optional, available for
enterprise plans only) (Optional)
}
```

Request Example Values

```
{
  "name": "My Organization"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/"

  -H "Authorization: Api-Key YOUR_API_KEY"

  -H "Content-Type: application/json"

  -d '{
    "name": "My Organization"
  }'

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "My Organization"
};

axios.post("https://api.maiagent.ai/api/v1/organizations/", data,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "My Organization"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    post("https://api.maiagent.ai/api/v1/organizations/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "My Organization"
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```

{
  "id": string (uuid)
  "name": string
  "createdAt": string (timestamp)
  "compactLogo"?: string (uri) // Optional
  "fullLogo"?: string (uri) // Optional
  "themePrimaryColor"?: string // UI theme color, used for buttons,
links, and other element colors (HEX format, e.g. #0960bd) (Optional)
  "usageStatistics": object
  "organizationPlan": object
  "creditWallet": object // Return credit wallet with ID only.
  "maigptInboxId": string (uuid)
  "canUseCustomElasticsearch": boolean // Allow this organization to
configure custom Elasticsearch endpoints for knowledge bases.
  "canUseVoiceAgent": boolean // Allow this organization to use voice
agent features.
  "canUseMeetingRecords": boolean // Allow this organization to use
meeting records features.
  "canUseCallCenter": boolean // Allow this organization to use call
center features.
  "canUseHookSystem": boolean // Manual override to grant this
organization the hook system regardless of plan. Enterprise-plan
organizations get access automatically without this flag. Layered under
the global Config.hook_system_enabled switch: it must also be enabled.
  "canUseAiGateway": boolean // Allow this organization to call the
OpenAI-compatible `/api/v1/chat/completions` Gateway. Disabled by
default; must be explicitly enabled by platform admin.
  "canUseAgentTeams": boolean // Read-only projection of the platform-
wide Agent Teams switch (Config.enable_agent_teams). Not a per-
organization flag; member-level access is still governed by RBAC.
  "canUseMaigptWorkspaceMount": boolean // Allow MaiGPT conversations to
mount a file workspace as read-only context. Off by default for gradual
rollout; independent of the other file library P2 switches.
  "gatewayServiceFeePercent": string (decimal) // Markup applied on top
of the token price when billing MaiAgent-managed models called through
the AI Gateway (/v1/chat/completions and /v1/embeddings). Managed gateway
markup only; the organization's own models (BYOK / self-hosted) are
billed via the AI Gateway traffic fee percent instead.
  "gatewayTrafficFeePercent": string (decimal) // Pass-through fee for
calls that run on this organization's own AI Gateway models (BYOK / self-
hosted): charged as the call's total tokens (input, output, and prompt-
cache) x this percent in credits, with no LLM token pricing. Adjustable
per organization; changes apply to new calls only.
  "platformProcessingFeePercent": string (decimal) // Surcharge on every

```


credit consumption of this organization, billed as a separate "Platform Processing Fee" line equal to the consumed credits x this percent. 0 disables the fee. Applies to new consumption only; the percent is shown read-only on the customer pricing page.

"platformProcessingFeeEnabled": boolean // Whether new consumption of this organization accrues the platform processing fee. The percent is exposed read-only as platform_processing_fee_percent.

"isTrial": boolean // Whether the organization is in trial period. Logic: the organization has only one trial plan and it is currently active

"expirationDays": integer // Number of days expired. Positive number = days since expiration, null = not yet expired or permanent plan

"showTrialBanner": boolean // Controls whether the frontend displays the trial/free plan banner

"noActivePlanDays": integer // Consecutive days without an active paid plan. null when cleanup is disabled.

"cleanupRemainingDays": integer // Days remaining before organization data is permanently deleted. null when cleanup is disabled, org has an active paid plan, or no_active_plan_days is 0.

"estimatedCleanupDate": string // ISO date when organization data will be permanently deleted. null when cleanup_remaining_days is null.

"approvalStatus": // Trial-eligibility review status. Only approved organizations can receive a trial plan when organization approval is enabled. Paid plan activation auto-approves the organization.

* `pending` - Pending Review

* `approved` - Approved

* `rejected` - Rejected

```
{
}
```

"companyName?": string // Applicant-reported company name collected at organization creation for approval review. (Optional)

"taxId?": // May have different types (Optional)

string // Applicant-reported business tax ID (8 digits, optional).

Stored as-is; not verified. (Optional)

"contactPhone?": string // Applicant-reported contact phone collected at organization creation for approval review. (Optional)

"applicationMetadata?": object // Extra applicant-reported key-value fields collected at organization creation. Lets the frontend collect additional review fields without a schema migration; the core review fields (company name, tax ID, contact phone) stay as named columns. (Optional)

"invitationCode?": string // Optional

```
}
```

Response Example Values

```
{
  "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
  "name": "My Organization",
  "createdAt": "1748336288000",
  "compactLogo": null,
  "fullLogo": null,
  "usageStatistics": {
    "chatbotsCount": 0,
    "countableChatbotsCount": 0,
    "exemptChatbotsByMode": {},
    "canCreateChatbot": true,
    "createChatbotLimit": 1,
    "currentMonthWordsCountTotal": 1000000,
    "currentMonthUsedWordsCountTotal": 0,
    "currentMonthUsedConversationsCountTotal": 0,
    "availableUploadFileSizeTotal": 104857600,
    "usedUploadFileSizeTotal": 0
  },
  "organizationPlan": {
    "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
    "planName": "Free Plan",
    "planType": "free",
    "expiredAt": null
  },
  "maigptInboxId": null,
  "canUseCustomElasticsearch": false,
  "isTrial": true,
  "expirationDays": null
}
```

Add Member to Specified Organization

POST

/api/v1/organizations/{organizationPk}/members/

Parameters

Parameter	Required	Type	Description
<code>organizationPk</code>	V	string	A UUID string identifying this Organization ID

Request

Request Parameters

Field	Type	Required	Description
email	string (email)	Yes	
organization	string (uuid)	No	

Request Schema Example

```
{
  "email": string (email)
  "organization?": string (uuid) // Optional
}
```

Request Example Values

```
{
  "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
  "is_owner": false
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
  "is_owner": false
}'

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
  "is_owner": false
};

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
    "is_owner": false
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
    post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
            "is_owner": false
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 201

Response Schema Example

```
{
  "organization": string (uuid) // Organization ID
  "is_owner": boolean // Whether the member is the organization owner
}
```

Response Example Values

```
{
  "organization": "613c86f7-a45f-4e29-b255-29caff89de32",
  "is_owner": false
}
```

Get Organization List

GET

/api/v1/organizations/

Parameters

Parameter	Required	Type	Description
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
pagination	X	string	Whether to paginate (true/false)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/?
page=1&pageSize=1&pagination=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/?
page=1&pageSize=1&pagination=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/?
page=1&pageSize=1&pagination=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/?page=1&pageSize=1&pagination=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "count": integer
  "next"?: string (uri) // Optional
}
```

```
"previous"?: string (uri) // Optional
"results": [
  {
    "id": string (uuid)
    "name": string
    "createdAt": string (timestamp)
    "memberCreatedAt": string (timestamp)
    "approvalStatus": string
  }
]
}
```

Response Example Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
      "createdAt": "Response string",
      "memberCreatedAt": "Response string",
      "approvalStatus": "Response string"
    }
  ]
}
```

Get Specific Organization Information

GET

[/api/v1/organizations/{id}](#)

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Organization.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}

?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
}
```

```
"createdAt": string (timestamp)
"compactLogo"?: string (uri) // Optional
"fullLogo"?: string (uri) // Optional
"themePrimaryColor"?: string // UI theme color, used for buttons,
links, and other element colors (HEX format, e.g. #0960bd) (Optional)
"usageStatistics": object
"organizationPlan": object
"creditWallet": object // Return credit wallet with ID only.
"maigptInboxId": string (uuid)
"canUseCustomElasticsearch": boolean // Allow this organization to
configure custom Elasticsearch endpoints for knowledge bases.
"canUseVoiceAgent": boolean // Allow this organization to use voice
agent features.
"canUseMeetingRecords": boolean // Allow this organization to use
meeting records features.
"canUseCallCenter": boolean // Allow this organization to use call
center features.
"canUseHookSystem": boolean // Manual override to grant this
organization the hook system regardless of plan. Enterprise-plan
organizations get access automatically without this flag. Layered under
the global Config.hook_system_enabled switch: it must also be enabled.
"canUseAiGateway": boolean // Allow this organization to call the
OpenAI-compatible `/api/v1/chat/completions` Gateway. Disabled by
default; must be explicitly enabled by platform admin.
"canUseAgentTeams": boolean // Read-only projection of the platform-
wide Agent Teams switch (Config.enable_agent_teams). Not a per-
organization flag; member-level access is still governed by RBAC.
"canUseMaigptWorkspaceMount": boolean // Allow MaiGPT conversations to
mount a file workspace as read-only context. Off by default for gradual
rollout; independent of the other file library P2 switches.
"gatewayServiceFeePercent": string (decimal) // Markup applied on top
of the token price when billing MaiAgent-managed models called through
the AI Gateway (/v1/chat/completions and /v1/embeddings). Managed gateway
markup only; the organization's own models (BYOK / self-hosted) are
billed via the AI Gateway traffic fee percent instead.
"gatewayTrafficFeePercent": string (decimal) // Pass-through fee for
calls that run on this organization's own AI Gateway models (BYOK / self-
hosted): charged as the call's total tokens (input, output, and prompt-
cache) x this percent in credits, with no LLM token pricing. Adjustable
per organization; changes apply to new calls only.
"platformProcessingFeePercent": string (decimal) // Surcharge on every
credit consumption of this organization, billed as a separate "Platform
Processing Fee" line equal to the consumed credits x this percent. 0
disables the fee. Applies to new consumption only; the percent is shown
read-only on the customer pricing page.
```

```

    "platformProcessingFeeEnabled": boolean // Whether new consumption of
    this organization accrues the platform processing fee. The percent is
    exposed read-only as platform_processing_fee_percent.
    "isTrial": boolean // Whether the organization is in trial period.
    Logic: the organization has only one trial plan and it is currently
    active
    "expirationDays": integer // Number of days expired. Positive number =
    days since expiration, null = not yet expired or permanent plan
    "showTrialBanner": boolean // Controls whether the frontend displays
    the trial/free plan banner
    "noActivePlanDays": integer // Consecutive days without an active paid
    plan. null when cleanup is disabled.
    "cleanupRemainingDays": integer // Days remaining before organization
    data is permanently deleted. null when cleanup is disabled, org has an
    active paid plan, or no_active_plan_days is 0.
    "estimatedCleanupDate": string // ISO date when organization data will
    be permanently deleted. null when cleanup_remaining_days is null.
    "approvalStatus": // Trial-eligibility review status. Only approved
    organizations can receive a trial plan when organization approval is
    enabled. Paid plan activation auto-approves the organization.

* `pending` - Pending Review
* `approved` - Approved
* `rejected` - Rejected
{
}
    "companyName?": string // Applicant-reported company name collected at
    organization creation for approval review. (Optional)
    "taxId?": // May have different types (Optional)
    string // Applicant-reported business tax ID (8 digits, optional).
    Stored as-is; not verified. (Optional)
    "contactPhone?": string // Applicant-reported contact phone collected
    at organization creation for approval review. (Optional)
    "applicationMetadata?": object // Extra applicant-reported key-value
    fields collected at organization creation. Lets the frontend collect
    additional review fields without a schema migration; the core review
    fields (company name, tax ID, contact phone) stay as named columns.
    (Optional)
    "invitationCode?": string // Optional
}

```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "createdAt": "Response string",
  "compactLogo": "https://example.com/file.jpg",
  "fullLogo": "https://example.com/file.jpg",
  "themePrimaryColor": "Response string",
  "usageStatistics": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "organizationPlan": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "creditWallet": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "maigptInboxId": "550e8400-e29b-41d4-a716-446655440000",
  "canUseCustomElasticsearch": false,
  "canUseVoiceAgent": false,
  "canUseMeetingRecords": false,
  "canUseCallCenter": false,
  "canUseHookSystem": false,
  "canUseAiGateway": false,
  "canUseAgentTeams": false,
  "canUseMaigptWorkspaceMount": false,
  "gatewayServiceFeePercent": "Response string",
  "gatewayTrafficFeePercent": "Response string",
  "platformProcessingFeePercent": "Response string",
  "platformProcessingFeeEnabled": false,
  "isTrial": false,
  "expirationDays": 456,
  "showTrialBanner": false,
  "noActivePlanDays": 456,
  "cleanupRemainingDays": 456,
  "estimatedCleanupDate": "Response string",
  "approvalStatus": {},
  "companyName": "Response string",
  "taxId": "Response string",
}
```

```
"contactPhone": "Response string",  
"applicationMetadata": null,  
"invitationCode": "Response string"  
}
```

Get Current User Details

GET

</api/v1/users/current/>

Code Examples

Shell/Bash

```
# API call example (Shell)  
curl -X GET "https://api.maiagent.ai/api/v1/users/current/"  
  
-H "Authorization: Api-Key YOUR_API_KEY"  
  
# Please make sure to replace YOUR_API_KEY and verify the request  
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/users/current/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/users/current/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/users/current/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "avatar"?: string (uri) // Optional
  "originalAvatar"?: string (uri) // The original avatar image uploaded
```


by the user (long side capped at 2048px). Every edit re-crops from this image to avoid quality loss from repeatedly cropping already-cropped results. Pre-feature data is empty and will not be back-filled.

(Optional)

"avatarCropBox"?: object // Position and size of the last crop box on the original image (x/y/width/height; optional rotate/scale_x/scale_y). Coordinates are relative to the saved original image. (Optional)

"name": string

"email": string (email)

"authSource":

{

"id": string (uuid)

"name": string

}

"company"?: string // Optional

"invitationCode"?: // May have different types (Optional)

string // Optional

"apiKeys": [// Get the user's API key list in the specified organization.

Flow:

1. Get the current organization instance from the request
2. If no organization instance exists (typically occurs during third-party SSO login), look up the default organization for the user's authentication source (is_auth_source_default_organization=True)

Note:

During third-party SSO login, the request does not include organization_instance

The organization must be located through auth_source and the is_auth_source_default_organization flag

object

]

"permissions": object // Get the user's permissions in the current organization (nested parent-child structure).

"resourcePermissions": object // Get the user's resource creation permissions in the current organization.

"isOwner": boolean // Determine whether the user is an Owner in the current organization.

"maigptCapabilities": object // The member's effective built-in feature capabilities in MaiGPT (resolved from role permissions), used for frontend show/hide.

When there is no member context (not bound to an organization), the resolved result is fully open, consistent with the resolver's "don't lock down on missing data" principle; frontend hiding is a UX layer only – authoritative enforcement is at the gate in the reply path.

```
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "avatar": "https://example.com/file.jpg",
  "originalAvatar": "https://example.com/file.jpg",
  "avatarCropBox": null,
  "name": "Response string",
  "email": "response@example.com",
  "authSource": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "company": "Response string",
  "invitationCode": "Response string",
  "apiKeys": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "permissions": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "resourcePermissions": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "isOwner": false,
  "maigptCapabilities": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  }
}
```

```
}  
}
```

Get Current User Permissions

GET

/api/v1/permissions/

Code Examples

Shell/Bash

```
# API call example (Shell)  
curl -X GET "https://api.maiagent.ai/api/v1/permissions/"  
  
-H "Authorization: Api-Key YOUR_API_KEY"  
  
# Please make sure to replace YOUR_API_KEY and verify the request  
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/permissions/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/permissions/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/permissions/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
[
  {
    "id": string (uuid)
    "name": string
```

```
"value": string
"description": string
"children": [
  object
]
}
```

Response Example Values

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "value": "Response string",
    "description": "Response string",
    "children": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response example name",
        "description": "Response example description"
      }
    ]
  }
]
```

Get Member List for Specified Organization

GET

</api/v1/organizations/{organizationPk}/members/>

Parameters

Parameter	Required	Type	Description
-----------	----------	------	-------------

organizationPk	V	string	A UUID string identifying this Organization ID
group	X	string	
ids	X	string	
inbox	X	string	
isActive	X	boolean	
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
pagination	X	string	Whether to paginate (true/false)
query	X	string	
search	X	string	

Code Examples


```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/?group=550e8400-e29b-41d4-a716-
446655440000&ids=example&inbox=550e8400-e29b-41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&pagination=example&quer
y=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/?group=550e8400-e29b-41d4-a716-
446655440000&ids=example&inbox=550e8400-e29b-41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&pagination=example&quer
y=example&search=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/?group=550e8400-e29b-41d4-a716-446655440000&ids=example&inbox=550e8400-e29b-41d4-a716-446655440000&isActive=true&page=1&pageSize=1&pagination=example&query=example&search=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/?group=550e8400-e29b-41d4-a716-
446655440000&ids=example&inbox=550e8400-e29b-41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&pagination=example&quer
y=example&search=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "name": string
      "email": string
      "isActive": boolean
      "isOwner": boolean // Check whether the member is an organization
owner (via OWNER Group)

```

Uses caching to avoid duplicate queries within the same request

Cache lifetime: valid for the duration of the member instance

Preferentially uses annotation results (if available) to avoid duplicate queries

```

  "permissions": [
    object
  ]
  "groups": [
    object
  ]
  "creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (Optional)
  "activeQuota": object // Return the most recent quota summary
(active first), via prefetch to avoid N+1.
  "metadata": object // Organization-scoped member attributes,
injected into LLM system prompt
  "createdAt": string (timestamp)
  }
]
}

```

Response Example Values

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {

```

```
"id": "550e8400-e29b-41d4-a716-446655440000",
"name": "Response string",
"email": "Response string",
"isActive": false,
"isOwner": false,
"permissions": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  }
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  }
],
"creditQuotaRequired": false,
"activeQuota": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
},
"metadata": null,
"createdAt": "Response string"
}
]
```

Get Specific Member Details

GET

</api/v1/organizations/{organizationPk}/members/{id}>

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Member.
<code>organizationPk</code>	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
"name": string
"email": string
"isActive": boolean
"isOwner": boolean // Check whether the member is an organization owner
(via OWNER Group)
```

Uses caching to avoid duplicate queries within the same request

Cache lifetime: valid for the duration of the member instance

Preferentially uses annotation results (if available) to avoid duplicate queries

```
"permissions": [
  object
]
"groups": [
  object
]
"creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (Optional)
"activeQuota": object // Return the most recent quota summary (active
first), via prefetch to avoid N+1.
"metadata": object // Organization-scoped member attributes, injected
into LLM system prompt
"createdAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "email": "Response string",
  "isActive": false,
  "isOwner": false,
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
    "name": "Response example name",
    "description": "Response example description"
  }
],
"creditQuotaRequired": false,
"activeQuota": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
},
"metadata": null,
"createdAt": "Response string"
}
```

Get Specific Member Details

GET

`/api/v1/organizations/{organizationPk}/members/current/`

Parameters

Parameter	Required	Type	Description
<code>organizationPk</code>	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/current/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/current/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/current/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/current/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
}
```



```
"email": string
"isActive": boolean
"isOwner": boolean // Check whether the member is an organization owner
(via OWNER Group)
```

Uses caching to avoid duplicate queries within the same request

Cache lifetime: valid for the duration of the member instance

Preferentially uses annotation results (if available) to avoid duplicate queries

```
"permissions": [
  object
]
"groups": [
  object
]
"creditQuotaRequired?": boolean // When True, member must have an
active quota to consume credits (Optional)
"activeQuota": object // Return the most recent quota summary (active
first), via prefetch to avoid N+1.
"metadata": object // Organization-scoped member attributes, injected
into LLM system prompt
"createdAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "email": "Response string",
  "isActive": false,
  "isOwner": false,
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
```

```
    "description": "Response example description"
  },
],
"creditQuotaRequired": false,
"activeQuota": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
},
"metadata": null,
"createdAt": "Response string"
}
```

Get Specific Member Details

GET

`/api/v1/organizations/{organizationPk}/members/export/`

Parameters

Parameter	Required	Type	Description
<code>organizationPk</code>	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/export/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/export/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/export/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/export/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
string (binary)
```

Response Example Values

"Response string"

Get Specific Member Details

GET

/api/v1/organizations/{organizationPk}/members/export-template/

Parameters

Parameter	Required	Type	Description
organizationPk	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/export-template/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/export-template/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/export-template/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/export-template/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
string (binary)
```

Response Example Values

"Response string"

Get Specific Member Details

GET

/api/v1/organizations/{organizationPk}/members/metadata-template/

Parameters

Parameter	Required	Type	Description
organizationPk	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/metadata-template/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/metadata-template/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/metadata-template/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/members/metadata-template/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
}
```



```
"email": string
"isActive": boolean
"isOwner": boolean // Check whether the member is an organization owner
(via OWNER Group)
```

Uses caching to avoid duplicate queries within the same request

Cache lifetime: valid for the duration of the member instance

Preferentially uses annotation results (if available) to avoid duplicate queries

```
"permissions": [
  object
]
"groups": [
  object
]
"creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (Optional)
"activeQuota": object // Return the most recent quota summary (active
first), via prefetch to avoid N+1.
"metadata": object // Organization-scoped member attributes, injected
into LLM system prompt
"createdAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "email": "Response string",
  "isActive": false,
  "isOwner": false,
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "groups": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
```

```
    "description": "Response example description"
  }
],
"creditQuotaRequired": false,
"activeQuota": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
},
"metadata": null,
"createdAt": "Response string"
}
```

Update Organization Information

PUT

/api/v1/organizations/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Organization.

Request

Request Parameters

Field	Type	Required	Description
name	string	Yes	
compactLogo	string (uri)	No	

Field	Type	Required	Description
fullLogo	string (uri)	No	
themePrimaryColor	string	No	UI theme color, used for buttons, links, and other element colors (HEX format, e.g. #0960bd)
companyName	string	No	Applicant-reported company name collected at organization creation for approval review.
taxId	object	No	Applicant-reported business tax ID (8 digits, optional). Stored as-is; not verified.
contactPhone	string	No	Applicant-reported contact phone collected at organization creation for approval review.
applicationMetadata	object	No	Extra applicant-reported key-value fields collected at organization creation. Lets the frontend collect additional review fields without a schema migration; the core review fields (company name, tax I...
invitationCode	string	No	

Request Schema Example

```
{
  "name": string
  "compactLogo"?: string (uri) // Optional
  "fullLogo"?: string (uri) // Optional
  "themePrimaryColor"?: string // UI theme color, used for buttons,
links, and other element colors (HEX format, e.g. #0960bd) (Optional)
  "companyName"?: string // Applicant-reported company name collected at
organization creation for approval review. (Optional)
  "taxId"?: // May have different types (Optional)
    string // Applicant-reported business tax ID (8 digits, optional).
Stored as-is; not verified. (Optional)
  "contactPhone"?: string // Applicant-reported contact phone collected
at organization creation for approval review. (Optional)
  "applicationMetadata"?: object // Extra applicant-reported key-value
fields collected at organization creation. Lets the frontend collect
additional review fields without a schema migration; the core review
```

```
fields (company name, tax ID, contact phone) stay as named columns.
(Optional)
  "invitationCode"?: string // Optional
}
```

Request Example Values

```
{
  "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
  "name": "Updated Org Name",
  "created_at": "1748336288000",
  "usage_statistics": {
    "chatbots_count": 0,
    "can_create_chatbot": true,
    "has_create_chatbot_limit": true,
    "current_month_words_count_total": 1000000,
    "current_month_used_words_count_total": 0,
    "current_month_used_conversations_count_total": 0,
    "available_upload_files_size_total": 104857600,
    "used_upload_file_size_total": 0
  },
  "organization_plan": {
    "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
    "plan_name": "Free Plan",
    "expired_at": null
  },
  "is_trial": false,
  "expiration_days": null
}
```

Code Examples

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
  "name": "Updated Org Name",
  "created_at": "1748336288000",
  "usage_statistics": {
    "chatbots_count": 0,
    "can_create_chatbot": true,
    "has_create_chatbot_limit": true,
    "current_month_words_count_total": 1000000,
    "current_month_used_words_count_total": 0,
    "current_month_used_conversations_count_total": 0,
    "available_upload_files_size_total": 104857600,
    "used_upload_file_size_total": 0
  },
  "organization_plan": {
    "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
    "plan_name": "Free Plan",
    "expired_at": null
  },
  "is_trial": false,
  "expiration_days": null
}'

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
  "name": "Updated Org Name",
  "created_at": "1748336288000",
  "usage_statistics": {
    "chatbots_count": 0,
    "can_create_chatbot": true,
    "has_create_chatbot_limit": true,
    "current_month_words_count_total": 1000000,
    "current_month_used_words_count_total": 0,
    "current_month_used_conversations_count_total": 0,
    "available_upload_files_size_total": 104857600,
    "used_upload_file_size_total": 0
  },
  "organization_plan": {
    "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
    "plan_name": "Free Plan",
    "expired_at": null
  },
  "is_trial": false,
  "expiration_days": null
};

axios.put("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
```

```
.catch(error => {  
  console.error('Request error occurred:');  
  console.error(error.response?.data || error.message);  
});
```

```

import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
    "name": "Updated Org Name",
    "created_at": "1748336288000",
    "usage_statistics": {
        "chatbots_count": 0,
        "can_create_chatbot": true,
        "has_create_chatbot_limit": true,
        "current_month_words_count_total": 1000000,
        "current_month_used_words_count_total": 0,
        "current_month_used_conversations_count_total": 0,
        "available_upload_files_size_total": 104857600,
        "used_upload_file_size_total": 0
    },
    "organization_plan": {
        "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
        "plan_name": "Free Plan",
        "expired_at": null
    },
    "is_trial": false,
    "expiration_days": null
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)

```




```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
            "name": "Updated Org Name",
            "created_at": "1748336288000",
            "usage_statistics": {
                "chatbots_count": 0,
                "can_create_chatbot": true,
                "has_create_chatbot_limit": true,
                "current_month_words_count_total": 1000000,
                "current_month_used_words_count_total": 0,
                "current_month_used_conversations_count_total": 0,
                "available_upload_files_size_total": 104857600,
                "used_upload_file_size_total": 0
            },
            "organization_plan": {
                "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
                "plan_name": "Free Plan",
                "expired_at": null
            },
            "is_trial": false,
            "expiration_days": null
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);

```

```
} catch (Exception $e) {  
    echo 'Request error occurred: ' . $e->getMessage();  
}  
?>
```

Response

Status Code: 200

Response Schema Example

```
{  
  "id": string (uuid) // Organization ID  
  "name": string // Organization name  
  "created_at": string (timestamp) // Creation timestamp  
  "usage_statistics": { // Usage statistics  
    {  
      "chatbots_count?": integer // Number of chatbots (Optional)  
      "can_create_chatbot?": boolean // Whether chatbots can be created  
      (Optional)  
      "has_create_chatbot_limit?": boolean // Whether there is a chatbot  
      creation limit (Optional)  
      "current_month_words_count_total?": integer // Total word limit for  
      the current month (Optional)  
      "current_month_used_words_count_total?": integer // Words used in the  
      current month (Optional)  
      "current_month_used_conversations_count_total?": integer //  
      Conversations used in the current month (Optional)  
      "available_upload_files_size_total?": integer // Total available  
      upload file size (bytes) (Optional)  
      "used_upload_file_size_total?": integer // Used upload file size  
      (bytes) (Optional)  
    }  
  }  
  "organization_plan": { // Organization plan information  
    {  
      "id?": string (uuid) // Plan ID (Optional)
```

```
"plan_name"?: string // Plan name (Optional)
"expired_at"?: string (date-time) // Expiration time (Optional)
}
}
"is_trial"?: boolean // Whether the organization is in trial period.
Logic: the organization has only one trial plan and it is currently
active (Optional)
"expiration_days"?: integer // Number of days expired. Positive number
= days since expiration, null = not yet expired or permanent plan
(Optional)
}
```

Response Example Values

```
{
  "id": "5b01e0b9-0b0e-4079-8150-d12015576d2c",
  "name": "Updated Org Name",
  "created_at": "1748336288000",
  "usage_statistics": {
    "chatbots_count": 0,
    "can_create_chatbot": true,
    "has_create_chatbot_limit": true,
    "current_month_words_count_total": 1000000,
    "current_month_used_words_count_total": 0,
    "current_month_used_conversations_count_total": 0,
    "available_upload_files_size_total": 104857600,
    "used_upload_file_size_total": 0
  },
  "organization_plan": {
    "id": "cac825e8-cb9b-4a16-b440-4ff6b2e73911",
    "plan_name": "Free Plan",
    "expired_at": null
  },
  "is_trial": false,
  "expiration_days": null
}
```

Update Current User Details

PUT**/api/v1/users/current/**

Request

Request Parameters

Field	Type	Required	Description
avatar	string (uri)	No	
name	string	Yes	
email	string (email)	Yes	
company	string	No	
invitationCode	string	No	

Request Schema Example

```
{
  "avatar"?: string (uri) // Optional
  "name": string
  "email": string (email)
  "company"?: string // Optional
  "invitationCode"?: string // Optional
}
```

Request Example Values

```
{
  "avatar": "https://example.com/file.jpg",
  "name": "Example name",
  "email": "user@example.com",
  "company": "Example string",
  "invitationCode": "Example string"
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/users/current/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');
const FormData = require('form-data');

// Create FormData object
const formData = new FormData();
// Add form fields

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    ...formData.getHeaders()
  }
};

axios.put("https://api.maiagent.ai/api/v1/users/current/", formData,
config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/users/current/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

# Create file and form data
files = {
    'file': ('example.pdf', open('example.pdf', 'rb'),
    'application/pdf')
}

response = requests.put(url, files=files, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/users/current/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ],
        'multipart' => [
            [
                'name' => 'file',
                'contents' => fopen('example.pdf', 'r'),
                'filename' => 'example.pdf'
            ]
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "avatar"?: string (uri) // Optional
  "originalAvatar"?: string (uri) // The original avatar image uploaded
by the user (long side capped at 2048px). Every edit re-crops from this
image to avoid quality loss from repeatedly cropping already-cropped
results. Pre-feature data is empty and will not be back-filled.
(Optional)
  "avatarCropBox"?: object // Position and size of the last crop box on
the original image (x/y/width/height; optional rotate/scale_x/scale_y).
Coordinates are relative to the saved original image. (Optional)
  "name": string
  "email": string (email)
  "authSource":
  {
    "id": string (uuid)
    "name": string
  }
  "company"?: string // Optional
  "invitationCode"?: // May have different types (Optional)
string // Optional
  "apiKeys": [ // Get the user's API key list in the specified
organization.
```

Flow:

1. Get the current organization instance from the request
2. If no organization instance exists (typically occurs during third-party SSO login), look up the default organization for the user's authentication source (is_auth_source_default_organization=True)

Note:

During third-party SSO login, the request does not include organization_instance

The organization must be located through auth_source and the is_auth_source_default_organization flag


```
    object
  ]
  "permissions": object // Get the user's permissions in the current
organization (nested parent-child structure).
  "resourcePermissions": object // Get the user's resource creation
```

permissions in the current organization.

"isOwner": boolean // Determine whether the user is an Owner in the current organization.

"maigptCapabilities": object // The member's effective built-in feature capabilities in MaiGPT (resolved from role permissions), used for frontend show/hide.

When there is no member context (not bound to an organization), the resolved result is fully open, consistent with the resolver's "don't lock down on missing data" principle; frontend hiding is a UX layer only – authoritative enforcement is at the gate in the reply path.

}

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "avatar": "https://example.com/file.jpg",
  "originalAvatar": "https://example.com/file.jpg",
  "avatarCropBox": null,
  "name": "Response string",
  "email": "response@example.com",
  "authSource": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "company": "Response string",
  "invitationCode": "Response string",
  "apiKeys": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "permissions": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  },
  "resourcePermissions": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  }
}
```

```
},
"isOwner": false,
"maigptCapabilities": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
}
}
```

Update Member Role Assignment

POST

/api/v1/organizations/{organizationPk}/members/{id}/update-groups/

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Member.
organizationPk	V	string	A UUID string identifying this Organization ID

Request

Request Parameters

Field	Type	Required	Description
groups	array[string]	Yes	

Request Schema Example

```
{
  "groups": [
    string (uuid)
  ]
}
```

Request Example Values

```
{
  "groups": [
    "4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61",
    "5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72"
  ]
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/update-groups/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "groups": [
    "4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61",
    "5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72"
  ]
}'

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "groups": [
    "4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61",
    "5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72"
  ]
};

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/update-groups/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/update-groups/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "groups": [
        "4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61",
        "5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72"
    ]
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->
post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/update-groups/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "groups": [
                "4e368ab4-c543-48a2-bbb8-f6f5a0bf6c61",
                "5f479bc5-d654-59b3-ccc9-g7g6b1cg7d72"
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "name": string
  "email": string
  "isActive": boolean
  "isOwner": boolean // Check whether the member is an organization owner
                      (via OWNER Group)
```

Uses caching to avoid duplicate queries within the same request

Cache lifetime: valid for the duration of the member instance

Preferentially uses annotation results (if available) to avoid duplicate queries

```
  "permissions": [
    object
  ]
  "groups": [
    object
  ]
  "creditQuotaRequired?": boolean // When True, member must have an
active quota to consume credits (Optional)
  "activeQuota": object // Return the most recent quota summary (active
first), via prefetch to avoid N+1.
  "metadata": object // Organization-scoped member attributes, injected
into LLM system prompt
  "createdAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "email": "Response string",
  "isActive": false,
  "isOwner": false,
  "permissions": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
```

```
    "name": "Response example name",
    "description": "Response example description"
  },
],
"groups": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response example name",
    "description": "Response example description"
  }
],
"creditQuotaRequired": false,
"activeQuota": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
},
"metadata": null,
"createdAt": "Response string"
}
```

Status Code: **400**

Response Schema Example

object

Delete Organization

DELETE

`/api/v1/organizations/{id}`

Parameters

Parameter	Required	Type	Description
-----------	----------	------	-------------

id	V	string	A UUID string identifying this Organization.
----	---	--------	--

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}

?>
```

Response

Status Code	Description
204	No response body

Delete Specified Member

DELETE`/api/v1/organizations/{organizationPk}/members/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Member.
<code>organizationPk</code>	V	string	A UUID string identifying this Organization ID

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-
446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/members/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
204	No response body

Get Member Usage Statistics

GET

/api/v1/organizations/{organizationPk}/usage-statistics/members/

Parameters

Parameter	Required	Type	Description
organizationPk	V	string	A UUID string identifying this Organization ID
chatbot	X	string	Filter by specific AI Assistant ID
endDate	X	string	End date, format: YYYY-MM-DD
includeTime	X	boolean	Whether to return time-aggregated time series data for trend charts (true/false). When set to true, the response does not include entity information, only time fields and statistical values, and is not paginated.
page	X	integer	Page number
pageSize	X	integer	Results per page
search	X	string	Search by member ID, name, or email
sortBy	X	string	Sort field
sortOrder	X	string	Sort order (asc/desc)
startDate	V	string	Start date, format: YYYY-MM-DD
timeGranularity	X	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/members/?
chatbot=example&endDate=example&includeTime=true&page=1&pageSize=1&s
earch=example&sortBy=conversations_count&sortOrder=asc&startDate=exa
mple&timeGranularity=day"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/members/?
chatbot=example&endDate=example&includeTime=true&page=1&pageSize=1&s
earch=example&sortBy=conversations_count&sortOrder=asc&startDate=exa
mple&timeGranularity=day", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/usage-statistics/members/?chatbot=example&endDate=example&includeTime=true&page=1&pageSize=1&search=example&sortBy=conversations_count&sortOrder=asc&startDate=example&timeGranularity=day"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/members/?
chatbot=example&endDate=example&includeTime=true&page=1&pageSize=1&s
earch=example&sortBy=conversations_count&sortOrder=asc&startDate=exa
mple&timeGranularity=day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example


```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "count"?: integer // Result count (Optional)
      "results"?: [ // Optional
        {
          "member"?: { // Optional
            {
              "id"?: string (uuid) // Member ID (Optional)
              "name"?: string // Member name (Optional)
              "email"?: string // Member email (Optional)
            }
          }
          "datetime"?: string (date-time) // Timestamp (only appears when
include_time=true and time_granularity=hour) (Optional)
          "period"?: string (date) // Date (only appears when
include_time=true and time_granularity=day/month) (Optional)
          "input_characters_count"?: integer // Input character count
(Optional)
          "output_characters_count"?: integer // Output character count
(Optional)
          "input_token_usage"?: integer // Input token count (Optional)
          "output_token_usage"?: integer // Output token count (Optional)
          "messages_count"?: integer // Message count (Optional)
          "conversations_count"?: integer // Conversation count
(Optional)
          "total_characters_count"?: integer // Total character count
(Optional)
          "total_token_usage"?: integer // Total token count (Optional)
        }
      ]
    }
  ]
}

```

Response Example Values

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",

```

```
"previous": "http://api.example.org/accounts/?page=2",
"results": [
  {
    "count": 2,
    "results": [
      {
        "member": {
          "id": "3a245511-d5e0-4e06-9dc6-1106915b9a8a",
          "name": "Wang Xiaoming",
          "email": "wang@example.com"
        },
        "input_characters_count": 10000,
        "output_characters_count": 15000,
        "input_token_usage": 2500,
        "output_token_usage": 3750,
        "messages_count": 100,
        "conversations_count": 50,
        "total_characters_count": 25000,
        "total_token_usage": 6250
      }
    ]
  }
]
```

Get Member Usage Statistics Summary

GET

`/api/v1/organizations/{organizationPk}/usage-statistics/members/summary/`

Parameters

Parameter	Required	Type	Description
<code>organizationPk</code>	V	string	A UUID string identifying this Organization ID

endDate	X	string	End date, format: YYYY-MM-DD
startDate	V	string	Start date, format: YYYY-MM-DD
timeGranularity	X	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/members/summary/?
endDate=example&startDate=example&timeGranularity=day"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/members/summary/?
endDate=example&startDate=example&timeGranularity=day", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/usage-statistics/members/summary/?endDate=example&startDate=example&timeGranularity=day"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/usage-statistics/members/summary/?endDate=example&startDate=example&timeGranularity=day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "total_input_characters": integer // Total input character count
```

```
(Optional)
  "total_output_characters"?: integer // Total output character count
(Optional)
  "total_characters"?: integer // Total character count (Optional)
  "total_input_tokens"?: integer // Total input token count (Optional)
  "total_output_tokens"?: integer // Total output token count (Optional)
  "total_tokens"?: integer // Total token count (Optional)
  "total_messages"?: integer // Total message count (Optional)
  "total_conversations"?: integer // Total conversation count (Optional)
  "total_members"?: integer // Total member count (Optional)
}
```

Response Example Values

```
{
  "total_input_characters": 456,
  "total_output_characters": 456,
  "total_characters": 456,
  "total_input_tokens": 456,
  "total_output_tokens": 456,
  "total_tokens": 456,
  "total_messages": 456,
  "total_conversations": 456,
  "total_members": 456
}
```

Get AI Assistant Usage Statistics

GET

</api/v1/organizations/{organizationPk}/usage-statistics/chatbots/>

Parameters

Parameter	Required	Type	Description
-----------	----------	------	-------------

organizationPk	V	string	A UUID string identifying this Organization ID
endDate	X	string	End date, format: YYYY-MM-DD
includeTime	X	boolean	Whether to return time-aggregated time series data for trend charts (true/false). When set to true, the response does not include entity information, only time fields and statistical values, and is not paginated.
page	X	integer	Page number
pageSize	X	integer	Results per page
search	X	string	Search by AI Assistant ID or name
sortBy	X	string	Sort field
sortOrder	X	string	Sort order (asc/desc)
startDate	V	string	Start date, format: YYYY-MM-DD
timeGranularity	X	string	Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/chatbots/?
endDate=example&includeTime=true&page=1&pageSize=1&search=example&so
rtBy=conversations_count&sortOrder=asc&startDate=example&timeGranula
rity=day"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/chatbots/?
endDate=example&includeTime=true&page=1&pageSize=1&search=example&so
rtBy=conversations_count&sortOrder=asc&startDate=example&timeGranula
rity=day", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/usage-statistics/chatbots/?\nendDate=example&includeTime=true&page=1&pageSize=1&search=example&sortBy=conversations_count&sortOrder=asc&startDate=example&timeGranularity=day"\nheaders = {\n    "Authorization": "Api-Key YOUR_API_KEY"\n}\n\nresponse = requests.get(url, headers=headers)\ntry:\n    print("Response received successfully:")\n    print(response.json())\nexcept Exception as e:\n    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/usage-statistics/chatbots/?
endDate=example&includeTime=true&page=1&pageSize=1&search=example&so
rtBy=conversations_count&sortOrder=asc&startDate=example&timeGranula
rity=day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "count"?: integer // Result count (Optional)
      "results"?: [ // Optional
        {
          "chatbot"?: { // Optional
            {
              "id"?: string (uuid) // AI Assistant ID (Optional)
              "name"?: string // AI Assistant name (Optional)
            }
          }
          "datetime"?: string (date-time) // Timestamp (only appears when
include_time=true and time_granularity=hour) (Optional)
          "period"?: string (date) // Date (only appears when
include_time=true and time_granularity=day/month) (Optional)
          "input_characters_count"?: integer // Input character count
(Optional)
          "output_characters_count"?: integer // Output character count
(Optional)
          "input_token_usage"?: integer // Input token count (Optional)
          "output_token_usage"?: integer // Output token count (Optional)
          "messages_count"?: integer // Message count (Optional)
          "conversations_count"?: integer // Conversation count
(Optional)
          "total_characters_count"?: integer // Total character count
(Optional)
          "total_token_usage"?: integer // Total token count (Optional)
        }
      ]
    }
  ]
}

```

Response Example Values

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",

```

```
"results": [
  {
    "count": 2,
    "results": [
      {
        "chatbot": {
          "id": "5b646600-f6e0-4f07-aec7-0207a26c0a9b",
          "name": "Customer Service Assistant"
        },
        "input_characters_count": 10000,
        "output_characters_count": 15000,
        "input_token_usage": 2500,
        "output_token_usage": 3750,
        "messages_count": 100,
        "conversations_count": 50,
        "total_characters_count": 25000,
        "total_token_usage": 6250
      }
    ]
  }
]
```

Get AI Assistant Usage Statistics Summary

GET

`/api/v1/organizations/{organizationPk}/usage-statistics/chatbots/summary/`

Parameters

Parameter	Required	Type	Description
<code>organizationPk</code>	V	string	A UUID string identifying this Organization ID
<code>endDate</code>	X	string	End date, format: YYYY-MM-DD

startDate

V

string

Start date, format: YYYY-MM-DD

timeGranularity

X

string

Time granularity (hour/day/month)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/chatbots/summary/?
endDate=example&startDate=example&timeGranularity=day"

-H "Authorization: Api-Key YOUR_API_KEY"

# Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/usage-statistics/chatbots/summary/?
endDate=example&startDate=example&timeGranularity=day", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/usage-statistics/chatbots/summary/?endDate=example&startDate=example&timeGranularity=day"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/usage-statistics/chatbots/summary/?endDate=example&startDate=example&timeGranularity=day", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "total_input_characters"?: integer // Total input character count
```

```
(Optional)
  "total_output_characters"?: integer // Total output character count
(Optional)
  "total_characters"?: integer // Total character count (Optional)
  "total_input_tokens"?: integer // Total input token count (Optional)
  "total_output_tokens"?: integer // Total output token count (Optional)
  "total_tokens"?: integer // Total token count (Optional)
  "total_messages"?: integer // Total message count (Optional)
  "total_conversations"?: integer // Total conversation count (Optional)
  "total_chatbots"?: integer // Total AI assistant count (Optional)
}
```

Response Example Values

```
{
  "total_input_characters": 456,
  "total_output_characters": 456,
  "total_characters": 456,
  "total_input_tokens": 456,
  "total_output_tokens": 456,
  "total_tokens": 456,
  "total_messages": 456,
  "total_conversations": 456,
  "total_chatbots": 456
}
```

API call example (Shell)

...

目錄

- 1. List Contacts
- 2. Create Contact
- 3. Get Contact Details
- 4. Get Contact Details
- 5. Get Contact Details
- 6. Update Contact
- 7. Partially Update Contact
- 8. Delete Contact
- 9. Get Contact Conversation List
- 10. Get Contact Latest Conversation
- 11. Broadcast Message

API call example (Shell)

List Contacts

GET

/api/v1/contacts/

Parameters

Parameter	Required	Type	Description
channels	X	array	Multiple values may be separated by commas.
inbox	X	string	Deprecated: Use inboxes instead
inboxes	X	string	Inbox ID (used to filter contacts for a specific inbox)
page	X	integer	A page number within the paginated result set.

pageSize	X	integer	Number of results to return per page.
query	X	string	Search by name or source ID
search	X	string	Full-text search
tags	X	array	Multiple values may be separated by commas.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/?
channels=example&inbox=550e8400-e29b-41d4-a716-
446655440000&inboxes=example&page=1&pageSize=1&query=example&search=
example&tags=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/?channels=example&inbox=550e8400-e29b-41d4-a716-446655440000&inboxes=example&page=1&pageSize=1&query=example&search=example&tags=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/?  
channels=example&inbox=550e8400-e29b-41d4-a716-  
446655440000&inboxes=example&page=1&pageSize=1&query=example&search=  
example&tags=example"  
headers = {  
    "Authorization": "Api-Key YOUR_API_KEY"  
}  
  
response = requests.get(url, headers=headers)  
try:  
    print("Response received successfully:")  
    print(response.json())  
except Exception as e:  
    print("Request error occurred:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/contacts/?
channels=example&inbox=550e8400-e29b-41d4-a716-
446655440000&inboxes=example&page=1&pageSize=1&query=example&search=
example&tags=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "inboxes"?: [ // Optional
        {
          "id": string (uuid)
          "name": string
        }
      ]
      "inbox"?: // Backward-compatible field: single inbox; ignored if
inboxes is also provided (Optional)
      {
        "id": string (uuid)
        "name": string
      }
      "name": string
      "avatar"?: // May have different types (Optional)
string (uri) // Contact avatar URL, supports full URL or relative
path (Optional)
      "phoneNumber"?: string // Optional
      "email"?: // May have different types (Optional)
string (email) // Optional
      "sourceId"?: string // Optional
      "queryMetadata"?: { // Query metadata, controls the scope of
queryable knowledge; see each endpoint for details (Optional)
        {
          "knowledgeBases"?: [ // List of queryable knowledge bases; empty
array or null = none queryable (no access), omitted = no restriction
(Optional)
            {
              "knowledgeBaseId": string (uuid) // Knowledge base ID
              "chatbotFileIds"?: [ // List of file IDs (excluded or
selected based on hasUserSelectedAll) (Optional)
                string (uuid)
              ]
              "knowledgeBaseFileIds"?: [ // List of file IDs (new name for
chatbotFileIds; takes precedence when both are provided) (Optional)
                string (uuid)
              ]
            }
          "faqIds"?: [ // List of FAQ IDs (excluded or selected based

```

```

on hasUserSelectedAll) (Optional)
    string (uuid)
]
"hasUserSelectedAll"?: boolean // true = select all knowledge
base content and exclude items in the list; false = select only items in
the list (Optional)
}
]
"labelRelations"?: { // Label filter conditions; empty conditions
array = no label filtering (not the same as no access). Must be provided
together with knowledgeBases – if provided alone, knowledge base
retrieval will not apply label filtering (equivalent to no restriction)
(Optional)
{
    "operator": string (enum: AND, OR) // How label conditions are
combined
    "conditions"?: [ // List of label conditions, supports nested
operator/conditions combinations (Optional)
        object
    ]
}
}
}
}
"metadata"?: object // Custom customer information, displayed in
the customer info section on the conversation page (Optional)
"mcpCredentials": [ // List of MCP credentials for the contact
    object
]
"apiCredentials": [ // List of API credentials for the contact
    object
]
"contactConnectors": [ // List of Connector OAuth authorizations
for the contact (excluding token body)
    object
]
"tags": [
{
    "id": string (uuid)
    "name": string
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
}
]
"preChatFieldIndex": object // Pre-chat form field configuration

```

from the inboxes this Contact belongs to, mapping {field_id: {label: {locale: text}}}; used by the frontend to resolve metadata UUID keys into display names

```
      "createdAt": string (timestamp)
      "updatedAt": string (timestamp)
    }
  ]
}
```

Response Example Values

```
{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    [
      {
        "id": "user1_uuid",
        "inboxes": [
          {
            "id": "inbox_uuid",
            "name": "inbox_name"
          }
        ],
        "name": "user1",
        "avatar": "",
        "sourceId": null,
        "queryMetadata": null,
        "createdAt": "1751875361000",
        "updatedAt": "1751875361000"
      },
      {
        "id": "user2_uuid",
        "inboxes": [
          {
            "id": "inbox_uuid",
            "name": "inbox_name"
          }
        ],
        "name": "user2",
        "avatar": "avatar_url",
        "sourceId": "self defined source id",
        "queryMetadata": {
```

```
    "knowledgeBases": [  
      {  
        "knowledgeBaseId": "5f8c6a99-5515-43aa-8e83-fdeea327a3a2",  
        "hasUserSelectedAll": true  
      }  
    ],  
    "labelRelations": {  
      "operator": "OR",  
      "conditions": [  
        {  
          "labelId": "957998f9-1824-4872-becd-b7f8a7962896"  
        }  
      ]  
    }  
  },  
  "createdAt": "1751875400000",  
  "updatedAt": "1751875400000"  
}  
]  
}
```

Create Contact

POST

[/api/v1/contacts/](#)

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/contacts/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "inboxes": [
    {
      "id": "inbox_uuid"
    }
  ],
  "name": "user1",
  "avatar": "",
  "phoneNumber": 912345678,
  "email": "user1@example.com",
  "sourceId": "self defined source id",
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "5f8c6a99-5515-43aa-8e83-fdeea327a3a2",
        "hasUserSelectedAll": true
      }
    ],
    "labelRelations": {
      "operator": "OR",
      "conditions": [
        {
          "labelId": "957998f9-1824-4872-becd-b7f8a7962896"
        }
      ]
    }
  },
  "metadata": [
    {
      "key": "Customer Level",
      "value": "VIP"
    },
    {
```

```
        "key": "Preferred Language",  
        "value": "Traditional Chinese"  
    }  
]  
'
```

Make sure to replace YOUR_API_KEY and verify the request data before running.

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "inboxes": [
    {
      "id": "inbox_uuid"
    }
  ],
  "name": "user1",
  "avatar": "",
  "phoneNumber": 912345678,
  "email": "user1@example.com",
  "sourceId": "self defined source id",
  "queryMetadata": {
    "knowledgeBases": [
      {
        "knowledgeBaseId": "5f8c6a99-5515-43aa-8e83-fdeea327a3a2",
        "hasUserSelectedAll": true
      }
    ],
    "labelRelations": {
      "operator": "OR",
      "conditions": [
        {
          "labelId": "957998f9-1824-4872-becd-b7f8a7962896"
        }
      ]
    }
  },
  "metadata": [
    {
```



```
    "key": "Customer Level",
    "value": "VIP"
  },
  {
    "key": "Preferred Language",
    "value": "Traditional Chinese"
  }
]
};
```

```
axios.post("https://api.maiagent.ai/api/v1/contacts/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "inboxes": [
        {
            "id": "inbox_uuid"
        }
    ],
    "name": "user1",
    "avatar": "",
    "phoneNumber": 912345678,
    "email": "user1@example.com",
    "sourceId": "self defined source id",
    "queryMetadata": {
        "knowledgeBases": [
            {
                "knowledgeBaseId": "5f8c6a99-5515-43aa-8e83-fdeea327a3a2",
                "hasUserSelectedAll": true
            }
        ],
        "labelRelations": {
            "operator": "OR",
            "conditions": [
                {
                    "labelId": "957998f9-1824-4872-becd-b7f8a7962896"
                }
            ]
        }
    },
    "metadata": [
        {
            "key": "Customer Level",
```

```
        "value": "VIP"
    },
    {
        "key": "Preferred Language",
        "value": "Traditional Chinese"
    }
]
}
```

```
response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->post("https://api.maiagent.ai/api/v1/contacts/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "inboxes": [
                {
                    "id": "inbox_uuid"
                }
            ],
            "name": "user1",
            "avatar": "",
            "phoneNumber": 912345678,
            "email": "user1@example.com",
            "sourceId": "self defined source id",
            "queryMetadata": {
                "knowledgeBases": [
                    {
                        "knowledgeBaseId": "5f8c6a99-5515-43aa-8e83-fdeea327a3a2",
                        "hasUserSelectedAll": true
                    }
                ],
                "labelRelations": {
                    "operator": "OR",
                    "conditions": [
                        {
                            "labelId": "957998f9-1824-4872-becd-b7f8a7962896"
                        }
                    ]
                }
            }
        }
    ]);
}

```

```

        },
        "metadata": [
            {
                "key": "Customer Level",
                "value": "VIP"
            },
            {
                "key": "Preferred Language",
                "value": "Traditional Chinese"
            }
        ]
    }
});

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>

```

Response

Status Code: 201

Response Schema Example

```

{
  "id": string (uuid)
  "inboxes"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "inbox"?: // Backward-compatible field: single inbox; ignored if

```

```

inboxes is also provided (Optional)
{
  "id": string (uuid)
  "name": string
}
"name": string
"avatar"?: // May have different types (Optional)
string (uri) // Contact avatar URL, supports full URL or relative path
(Optional)
"phoneNumber"?: string // Optional
"email"?: // May have different types (Optional)
string (email) // Optional
"sourceId"?: string // Optional
"queryMetadata"?: { // Query metadata, controls the scope of queryable
knowledge; see each endpoint for details (Optional)
{
  "knowledgeBases"?: [ // List of queryable knowledge bases; empty
array or null = none queryable (no access), omitted = no restriction
(Optional)
{
  "knowledgeBaseId": string (uuid) // Knowledge base ID
  "chatbotFileIds"?: [ // List of file IDs (excluded or selected
based on hasUserSelectedAll) (Optional)
string (uuid)
]
  "knowledgeBaseFileIds"?: [ // List of file IDs (new name for
chatbotFileIds; takes precedence when both are provided) (Optional)
string (uuid)
]
  "faqIds"?: [ // List of FAQ IDs (excluded or selected based on
hasUserSelectedAll) (Optional)
string (uuid)
]
  "hasUserSelectedAll"?: boolean // true = select all knowledge
base content and exclude items in the list; false = select only items in
the list (Optional)
}
]
  "labelRelations"?: { // Label filter conditions; empty conditions
array = no label filtering (not the same as no access). Must be provided
together with knowledgeBases – if provided alone, knowledge base
retrieval will not apply label filtering (equivalent to no restriction)
(Optional)
{
  "operator": string (enum: AND, OR) // How label conditions are

```

```

combined
  "conditions"?: [ // List of label conditions, supports nested
operator/conditions combinations (Optional)
    object
  ]
}
}
}
}
"metadata"?: object // Custom customer information, displayed in the
customer info section on the conversation page (Optional)
"mcpCredentials": [ // List of MCP credentials for the contact
  object
]
"apiCredentials": [ // List of API credentials for the contact
  object
]
"contactConnectors": [ // List of Connector OAuth authorizations for
the contact (excluding token body)
  object
]
"tags": [
  {
    "id": string (uuid)
    "name": string
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"preChatFieldIndex": object // Pre-chat form field configuration from
the inboxes this Contact belongs to, mapping {field_id: {label: {locale:
text}}}; used by the frontend to resolve metadata UUID keys into display
names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}

```

Response Example Values

```

{
  "id": "user1_uuid",
  "inboxes": [
    {
      "id": "inbox_uuid",

```

```
    "name": "inbox_name"
  },
  {
    "name": "user1",
    "avatar": "",
    "phoneNumber": 912345678,
    "email": "user1@example.com",
    "sourceId": null,
    "queryMetadata": null,
    "createdAt": "1751875361000",
    "updatedAt": "1751875361000"
  }
}
```

Get Contact Details

GET

/api/v1/contacts/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Contact.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiaagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "inboxes"?: [ // Optional
```

```

    {
        "id": string (uuid)
        "name": string
    }
]
"inbox"?: // Backward-compatible field: single inbox; ignored if
inboxes is also provided (Optional)
{
    "id": string (uuid)
    "name": string
}
"name": string
"avatar"?: // May have different types (Optional)
string (uri) // Contact avatar URL, supports full URL or relative path
(Optional)
"phoneNumber"?: string // Optional
"email"?: // May have different types (Optional)
string (email) // Optional
"sourceId"?: string // Optional
"queryMetadata"?: { // Query metadata, controls the scope of queryable
knowledge; see each endpoint for details (Optional)
    {
        "knowledgeBases"?: [ // List of queryable knowledge bases; empty
array or null = none queryable (no access), omitted = no restriction
(Optional)
            {
                "knowledgeBaseId": string (uuid) // Knowledge base ID
                "chatbotFileIds"?: [ // List of file IDs (excluded or selected
based on hasUserSelectedAll) (Optional)
                    string (uuid)
                ]
                "knowledgeBaseFileIds"?: [ // List of file IDs (new name for
chatbotFileIds; takes precedence when both are provided) (Optional)
                    string (uuid)
                ]
                "faqIds"?: [ // List of FAQ IDs (excluded or selected based on
hasUserSelectedAll) (Optional)
                    string (uuid)
                ]
                "hasUserSelectedAll"?: boolean // true = select all knowledge
base content and exclude items in the list; false = select only items in
the list (Optional)
            }
        ]
    }
    "labelRelations"?: { // Label filter conditions; empty conditions

```

array = no label filtering (not the same as no access). Must be provided together with knowledgeBases – if provided alone, knowledge base retrieval will not apply label filtering (equivalent to no restriction) (Optional)

```
{
  "operator": string (enum: AND, OR) // How label conditions are
combined
  "conditions"? : [ // List of label conditions, supports nested
operator/conditions combinations (Optional)
    object
  ]
}
}
"metadata"? : object // Custom customer information, displayed in the
customer info section on the conversation page (Optional)
"mcpCredentials": [ // List of MCP credentials for the contact
  object
]
"apiCredentials": [ // List of API credentials for the contact
  object
]
"contactConnectors": [ // List of Connector OAuth authorizations for
the contact (excluding token body)
  object
]
"tags": [
  {
    "id": string (uuid)
    "name": string
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"preChatFieldIndex": object // Pre-chat form field configuration from
the inboxes this Contact belongs to, mapping {field_id: {label: {locale:
text}}}; used by the frontend to resolve metadata UUID keys into display
names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "user1_uuid",
  "inboxes": [
    {
      "id": "inbox_uuid",
      "name": "inbox_name"
    }
  ],
  "name": "user1",
  "avatar": "",
  "phoneNumber": 912345678,
  "email": "user1@example.com",
  "sourceId": null,
  "queryMetadata": null,
  "createdAt": "1751875361000",
  "updatedAt": "1751875361000"
}
```

Get Contact Details

GET

</api/v1/contacts/bulk-import-template/>

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/bulk-import-template/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/bulk-import-template/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/bulk-import-template/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/contacts/bulk-import-template/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "inboxes"?: [ // Optional
```

```

    {
        "id": string (uuid)
        "name": string
    }
]
"inbox"?: // Backward-compatible field: single inbox; ignored if
inboxes is also provided (Optional)
{
    "id": string (uuid)
    "name": string
}
"name": string
"avatar"?: // May have different types (Optional)
string (uri) // Contact avatar URL, supports full URL or relative path
(Optional)
"phoneNumber"?: string // Optional
"email"?: // May have different types (Optional)
string (email) // Optional
"sourceId"?: string // Optional
"queryMetadata"?: { // Query metadata, controls the scope of queryable
knowledge; see each endpoint for details (Optional)
    {
        "knowledgeBases"?: [ // List of queryable knowledge bases; empty
array or null = none queryable (no access), omitted = no restriction
(Optional)
            {
                "knowledgeBaseId": string (uuid) // Knowledge base ID
                "chatbotFileIds"?: [ // List of file IDs (excluded or selected
based on hasUserSelectedAll) (Optional)
                    string (uuid)
                ]
                "knowledgeBaseFileIds"?: [ // List of file IDs (new name for
chatbotFileIds; takes precedence when both are provided) (Optional)
                    string (uuid)
                ]
                "faqIds"?: [ // List of FAQ IDs (excluded or selected based on
hasUserSelectedAll) (Optional)
                    string (uuid)
                ]
                "hasUserSelectedAll"?: boolean // true = select all knowledge
base content and exclude items in the list; false = select only items in
the list (Optional)
            }
        ]
    }
    "labelRelations"?: { // Label filter conditions; empty conditions

```

array = no label filtering (not the same as no access). Must be provided together with knowledgeBases – if provided alone, knowledge base retrieval will not apply label filtering (equivalent to no restriction) (Optional)

```
{
  "operator": string (enum: AND, OR) // How label conditions are
combined
  "conditions"? : [ // List of label conditions, supports nested
operator/conditions combinations (Optional)
    object
  ]
}
}
"metadata"? : object // Custom customer information, displayed in the
customer info section on the conversation page (Optional)
"mcpCredentials": [ // List of MCP credentials for the contact
  object
]
"apiCredentials": [ // List of API credentials for the contact
  object
]
"contactConnectors": [ // List of Connector OAuth authorizations for
the contact (excluding token body)
  object
]
"tags": [
  {
    "id": string (uuid)
    "name": string
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"preChatFieldIndex": object // Pre-chat form field configuration from
the inboxes this Contact belongs to, mapping {field_id: {label: {locale:
text}}}; used by the frontend to resolve metadata UUID keys into display
names
  "createdAt": string (timestamp)
  "updatedAt": string (timestamp)
}
```

Response Example Values

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "inboxes": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string"
    }
  ],
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string"
  },
  "name": "Response string",
  "avatar": "https://example.com/file.jpg",
  "phoneNumber": "Response string",
  "email": "response@example.com",
  "sourceId": "Response string",
  "queryMetadata": null,
  "metadata": null,
  "mcpCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "apiCredentials": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "contactConnectors": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response example name",
      "description": "Response example description"
    }
  ],
  "tags": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Response string",
```

```
    "createdAt": "Response string",
    "updatedAt": "Response string"
  },
],
"preChatFieldIndex": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
},
"createdAt": "Response string",
"updatedAt": "Response string"
}
```

Get Contact Details

GET

[/api/v1/contacts/connector-management/](#)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/connector-
management/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/connector-
management/", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/connector-  
management/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/contacts/connector-management/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code	Description
200	No response body

Update Contact

PUT `/api/v1/contacts/{id}`

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Contact.

Request

Request Parameters

Field	Type	Required	Description
name	string	No	Contact name
inboxes	array[object]	No	Inbox list (new format)
inbox	object	No	Single inbox (legacy format, backward-compatible)
inbox.id	string (uuid)	Yes	Inbox ID
avatar	string	No	Avatar URL
sourceId	string	No	Source ID
queryMetadata	string	No	Query metadata
metadata	array[object]	No	Custom contact attribute list

Request Schema Example

```

{
  "name"?: string // Contact name (Optional)
  "inboxes"?: [ // Inbox list (new format) (Optional)
    {
      "id": string (uuid) // Inbox ID
    }
  ]
  "inbox"?: { // Single inbox (legacy format, backward-compatible)
    (Optional)
    {
      "id": string (uuid) // Inbox ID
    }
  }
  "avatar"?: string // Avatar URL (Optional)
  "sourceId"?: string // Source ID (Optional)
  "queryMetadata"?: string // Query metadata (Optional)
  "metadata"?: [ // Custom contact attribute list (Optional)
    {
      "key": string // Attribute name
      "value": string // Attribute value
      "updatedAt"?: string // Update time (ISO format, optional; auto-
filled if not provided) (Optional)
    }
  ]
}

```

Request Example Values

```

{
  "inboxes": [
    {
      "id": "new_inbox_uuid"
    }
  ],
  "name": "updated_user_name",
  "avatar": "updated_avatar_url",
  "phoneNumber": 987654321,
  "email": "updated@example.com",
  "sourceId": "updated_source_id",
  "queryMetadata": "updated_query_metadata",
  "metadata": [
    {
      "key": "Customer Level",

```

```
      "value": "VVIP"  
    },  
    {  
      "key": "Preferred Language",  
      "value": "Japanese"  
    }  
  ]  
}
```

Code Examples

```
# API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "inboxes": [
    {
      "id": "new_inbox_uuid"
    }
  ],
  "name": "updated_user_name",
  "avatar": "updated_avatar_url",
  "phoneNumber": 987654321,
  "email": "updated@example.com",
  "sourceId": "updated_source_id",
  "queryMetadata": "updated_query_metadata",
  "metadata": [
    {
      "key": "Customer Level",
      "value": "VVIP"
    },
    {
      "key": "Preferred Language",
      "value": "Japanese"
    }
  ]
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "inboxes": [
    {
      "id": "new_inbox_uuid"
    }
  ],
  "name": "updated_user_name",
  "avatar": "updated_avatar_url",
  "phoneNumber": 987654321,
  "email": "updated@example.com",
  "sourceId": "updated_source_id",
  "queryMetadata": "updated_query_metadata",
  "metadata": [
    {
      "key": "Customer Level",
      "value": "VVIP"
    },
    {
      "key": "Preferred Language",
      "value": "Japanese"
    }
  ]
};

axios.put("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  });
```

```
})  
  .catch(error => {  
    console.error('Request error occurred:');  
    console.error(error.response?.data || error.message);  
  });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "inboxes": [
        {
            "id": "new_inbox_uuid"
        }
    ],
    "name": "updated_user_name",
    "avatar": "updated_avatar_url",
    "phoneNumber": 987654321,
    "email": "updated@example.com",
    "sourceId": "updated_source_id",
    "queryMetadata": "updated_query_metadata",
    "metadata": [
        {
            "key": "Customer Level",
            "value": "VVIP"
        },
        {
            "key": "Preferred Language",
            "value": "Japanese"
        }
    ]
}

response = requests.put(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
```

```
except Exception as e:  
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>put("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "inboxes": [
                {
                    "id": "new_inbox_uuid"
                }
            ],
            "name": "updated_user_name",
            "avatar": "updated_avatar_url",
            "phoneNumber": 987654321,
            "email": "updated@example.com",
            "sourceId": "updated_source_id",
            "queryMetadata": "updated_query_metadata",
            "metadata": [
                {
                    "key": "Customer Level",
                    "value": "VVIP"
                },
                {
                    "key": "Preferred Language",
                    "value": "Japanese"
                }
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
```

```
        print_r($data);
    } catch (Exception $e) {
        echo 'Request error occurred: ' . $e->getMessage();
    }
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "inboxes"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "inbox"?: // Backward-compatible field: single inbox; ignored if
inboxes is also provided (Optional)
  {
    "id": string (uuid)
    "name": string
  }
  "name": string
  "avatar"?: // May have different types (Optional)
string (uri) // Contact avatar URL, supports full URL or relative path
(Optional)
  "phoneNumber"?: string // Optional
  "email"?: // May have different types (Optional)
string (email) // Optional
  "sourceId"?: string // Optional
  "queryMetadata"?: { // Query metadata, controls the scope of queryable
knowledge; see each endpoint for details (Optional)
  {
```

```
"knowledgeBases"?: [ // List of queryable knowledge bases; empty
array or null = none queryable (no access), omitted = no restriction
(Optional)
  {
    "knowledgeBaseId": string (uuid) // Knowledge base ID
    "chatbotFileIds"?: [ // List of file IDs (excluded or selected
based on hasUserSelectedAll) (Optional)
      string (uuid)
    ]
    "knowledgeBaseFileIds"?: [ // List of file IDs (new name for
chatbotFileIds; takes precedence when both are provided) (Optional)
      string (uuid)
    ]
    "faqIds"?: [ // List of FAQ IDs (excluded or selected based on
hasUserSelectedAll) (Optional)
      string (uuid)
    ]
    "hasUserSelectedAll"?: boolean // true = select all knowledge
base content and exclude items in the list; false = select only items in
the list (Optional)
  }
]
"labelRelations"?: { // Label filter conditions; empty conditions
array = no label filtering (not the same as no access). Must be provided
together with knowledgeBases – if provided alone, knowledge base
retrieval will not apply label filtering (equivalent to no restriction)
(Optional)
  {
    "operator": string (enum: AND, OR) // How label conditions are
combined
    "conditions"?: [ // List of label conditions, supports nested
operator/conditions combinations (Optional)
      object
    ]
  }
}
"metadata"?: object // Custom customer information, displayed in the
customer info section on the conversation page (Optional)
"mcpCredentials": [ // List of MCP credentials for the contact
  object
]
"apiCredentials": [ // List of API credentials for the contact
  object
]
```

```

]
"contactConnectors": [ // List of Connector OAuth authorizations for
the contact (excluding token body)
  object
]
"tags": [
  {
    "id": string (uuid)
    "name": string
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"preChatFieldIndex": object // Pre-chat form field configuration from
the inboxes this Contact belongs to, mapping {field_id: {label: {locale:
text}}}; used by the frontend to resolve metadata UUID keys into display
names
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Response Example Values

```

{
  "id": "user1_uuid",
  "inboxes": [
    {
      "id": "inbox_uuid",
      "name": "inbox_name"
    }
  ],
  "name": "user1",
  "avatar": "",
  "phoneNumber": 912345678,
  "email": "user1@example.com",
  "sourceId": null,
  "queryMetadata": null,
  "createdAt": "1751875361000",
  "updatedAt": "1751875361000"
}

```

Partially Update Contact

PATCH

/api/v1/contacts/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Contact.

Request

Request Parameters

Field	Type	Required	Description
name	string	No	Contact name
inboxes	array[object]	No	Inbox list (new format)
inbox	object	No	Single inbox (legacy format, backward-compatible)
inbox.id	string (uuid)	Yes	Inbox ID
avatar	string	No	Avatar URL
sourceId	string	No	Source ID
queryMetadata	string	No	Query metadata
metadata	array[object]	No	Custom contact attribute list

Request Schema Example

```

{
  "name"?: string // Contact name (Optional)
  "inboxes"?: [ // Inbox list (new format) (Optional)
    {
      "id": string (uuid) // Inbox ID
    }
  ]
  "inbox"?: { // Single inbox (legacy format, backward-compatible)
    (Optional)
    {
      "id": string (uuid) // Inbox ID
    }
  }
  "avatar"?: string // Avatar URL (Optional)
  "sourceId"?: string // Source ID (Optional)
  "queryMetadata"?: string // Query metadata (Optional)
  "metadata"?: [ // Custom contact attribute list (Optional)
    {
      "key": string // Attribute name
      "value": string // Attribute value
      "updatedAt"?: string // Update time (ISO format, optional; auto-
filled if not provided) (Optional)
    }
  ]
}

```

Request Example Values

```

{
  "inboxes": [
    {
      "id": "new_inbox_uuid"
    }
  ],
  "name": "updated_user_name",
  "avatar": "updated_avatar_url",
  "phoneNumber": 987654321,
  "email": "updated@example.com",
  "sourceId": "updated_source_id",
  "queryMetadata": "updated_query_metadata",
  "metadata": [
    {
      "key": "Customer Level",

```



```
    "value": "VVIP"  
  },  
  {  
    "key": "Preferred Language",  
    "value": "Japanese"  
  }  
]  
}
```

Code Examples

```
# API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/contacts/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "inboxes": [
    {
      "id": "new_inbox_uuid"
    }
  ],
  "name": "updated_user_name",
  "avatar": "updated_avatar_url",
  "phoneNumber": 987654321,
  "email": "updated@example.com",
  "sourceId": "updated_source_id",
  "queryMetadata": "updated_query_metadata",
  "metadata": [
    {
      "key": "Customer Level",
      "value": "VVIP"
    },
    {
      "key": "Preferred Language",
      "value": "Japanese"
    }
  ]
}'

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "inboxes": [
    {
      "id": "new_inbox_uuid"
    }
  ],
  "name": "updated_user_name",
  "avatar": "updated_avatar_url",
  "phoneNumber": 987654321,
  "email": "updated@example.com",
  "sourceId": "updated_source_id",
  "queryMetadata": "updated_query_metadata",
  "metadata": [
    {
      "key": "Customer Level",
      "value": "VVIP"
    },
    {
      "key": "Preferred Language",
      "value": "Japanese"
    }
  ]
};

axios.patch("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  });
```

```
})  
.catch(error => {  
  console.error('Request error occurred:');  
  console.error(error.response?.data || error.message);  
});
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "inboxes": [
        {
            "id": "new_inbox_uuid"
        }
    ],
    "name": "updated_user_name",
    "avatar": "updated_avatar_url",
    "phoneNumber": 987654321,
    "email": "updated@example.com",
    "sourceId": "updated_source_id",
    "queryMetadata": "updated_query_metadata",
    "metadata": [
        {
            "key": "Customer Level",
            "value": "VVIP"
        },
        {
            "key": "Preferred Language",
            "value": "Japanese"
        }
    ]
}

response = requests.patch(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
```

```
except Exception as e:  
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>patch("https://api.maiaagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "inboxes": [
                {
                    "id": "new_inbox_uuid"
                }
            ],
            "name": "updated_user_name",
            "avatar": "updated_avatar_url",
            "phoneNumber": 987654321,
            "email": "updated@example.com",
            "sourceId": "updated_source_id",
            "queryMetadata": "updated_query_metadata",
            "metadata": [
                {
                    "key": "Customer Level",
                    "value": "VVIP"
                },
                {
                    "key": "Preferred Language",
                    "value": "Japanese"
                }
            ]
        }
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
```

```
        print_r($data);
    } catch (Exception $e) {
        echo 'Request error occurred: ' . $e->getMessage();
    }
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
  "inboxes"?: [ // Optional
    {
      "id": string (uuid)
      "name": string
    }
  ]
  "inbox"?: // Backward-compatible field: single inbox; ignored if
inboxes is also provided (Optional)
  {
    "id": string (uuid)
    "name": string
  }
  "name": string
  "avatar"?: // May have different types (Optional)
string (uri) // Contact avatar URL, supports full URL or relative path
(Optional)
  "phoneNumber"?: string // Optional
  "email"?: // May have different types (Optional)
string (email) // Optional
  "sourceId"?: string // Optional
  "queryMetadata"?: { // Query metadata, controls the scope of queryable
knowledge; see each endpoint for details (Optional)
    {
```



```

    "knowledgeBases"?: [ // List of queryable knowledge bases; empty
array or null = none queryable (no access), omitted = no restriction
(Optional)
    {
        "knowledgeBaseId": string (uuid) // Knowledge base ID
        "chatbotFileIds"?: [ // List of file IDs (excluded or selected
based on hasUserSelectedAll) (Optional)
            string (uuid)
        ]
        "knowledgeBaseFileIds"?: [ // List of file IDs (new name for
chatbotFileIds; takes precedence when both are provided) (Optional)
            string (uuid)
        ]
        "faqIds"?: [ // List of FAQ IDs (excluded or selected based on
hasUserSelectedAll) (Optional)
            string (uuid)
        ]
        "hasUserSelectedAll"?: boolean // true = select all knowledge
base content and exclude items in the list; false = select only items in
the list (Optional)
    }
]
    "labelRelations"?: { // Label filter conditions; empty conditions
array = no label filtering (not the same as no access). Must be provided
together with knowledgeBases – if provided alone, knowledge base
retrieval will not apply label filtering (equivalent to no restriction)
(Optional)
    {
        "operator": string (enum: AND, OR) // How label conditions are
combined
        "conditions"?: [ // List of label conditions, supports nested
operator/conditions combinations (Optional)
            object
        ]
    }
}
    "metadata"?: object // Custom customer information, displayed in the
customer info section on the conversation page (Optional)
    "mcpCredentials": [ // List of MCP credentials for the contact
        object
    ]
    "apiCredentials": [ // List of API credentials for the contact
        object
    ]

```

```

]
"contactConnectors": [ // List of Connector OAuth authorizations for
the contact (excluding token body)
  object
]
"tags": [
  {
    "id": string (uuid)
    "name": string
    "createdAt": string (timestamp)
    "updatedAt": string (timestamp)
  }
]
"preChatFieldIndex": object // Pre-chat form field configuration from
the inboxes this Contact belongs to, mapping {field_id: {label: {locale:
text}}}; used by the frontend to resolve metadata UUID keys into display
names
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Response Example Values

```

{
  "id": "user1_uuid",
  "inboxes": [
    {
      "id": "inbox_uuid",
      "name": "inbox_name"
    }
  ],
  "name": "user1",
  "avatar": "",
  "phoneNumber": 912345678,
  "email": "user1@example.com",
  "sourceId": null,
  "queryMetadata": null,
  "createdAt": "1751875361000",
  "updatedAt": "1751875361000"
}

```

Delete Contact

DELETE

/api/v1/contacts/{id}

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Contact.

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/contacts/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->delete("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}

?>
```

Response

Status Code	Description
204	No response body
400	The contact still has associated conversations and cannot be deleted

Get Contact Conversation List

GET

/api/v1/contacts/{id}/conversations/

Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Contact.
channels	X	array	Multiple values may be separated by commas.
inbox	X	string	Filter by a single inbox ID (UUID format)
inboxes	X	string	Filter by multiple inbox IDs (comma-separated, e.g.: uuid1,uuid2 or use multiple inboxes parameters)
keyword	X	string	Search message content within conversations (case-insensitive)
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
tags	X	array	Multiple values may be separated by commas.

Code Examples

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/conversations/?
channels=example&inbox=example&inboxes=example&keyword=example&page=
1&pageSize=1&query=example&tags=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/conversations?channels=example&inbox=example&inboxes=example&keyword=example&page=1&pageSize=1&query=example&tags=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/conversations/?channels=example&inbox=example&inboxes=example&keyword=example&page=1&pageSize=1&query=example&tags=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-
a716-446655440000/conversations/?
channels=example&inbox=example&inboxes=example&keyword=example&page=
1&pageSize=1&query=example&tags=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```

{
  "count": integer
  "next"?: string (uri) // Optional
  "previous"?: string (uri) // Optional
  "results": [
    {
      "id": string (uuid)
      "contact": { // Lightweight Contact Serializer for Conversation
List view

```

Only includes fields necessary for the List view, removing inboxes, mcp_credentials, and api_credentials to avoid N+1 query issues. email / phone_number / metadata are scalar fields on the Contact itself; they need to be displayed in the frontend conversation sidebar with no N+1 cost.

```

    {
      "id": string (uuid)
      "name": string
      "avatar"?: string (uri) // Optional
      "email"?: string (email) // Optional
      "phoneNumber"?: string // Optional
      "metadata"?: object // Custom customer information, displayed in
the customer info section on the conversation page (Optional)
      "sourceId"?: string // Optional
      "tags": [
        {
          "id": string (uuid)
          "name": string
        }
      ]
    }
  ]
}
}
"inbox": {
  {
    "id": string (uuid)
    "name": string
    "channelType": string (enum: line, telegram, teams, web,
messenger, instagram, email, whatsapp, slack) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram

```

```
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack

  "unreadConversationsCount"?: integer // Optional
  "signAuth"?: // Optional
  {
    "id": string (uuid)
    "signSource": string (uuid)
    "signParams": {
      {
        "keycloak":
          {
            "clientId": string
            "url": string
            "realm": string
          }
        "line":
          {
            "liffId": string
          }
        "ad":
          {
            "saml": string
          }
      }
    }
  }
  "enableAnalysis"?: boolean // Optional
}
}
"title": string
"lastMessage"?: { // Optional
{
  "id": string (uuid)
  "type"?: string // Optional
  "content"?: string // Optional
  "createdAt": string (timestamp)
}
}
"lastMessageCreatedAt": string (timestamp)
"unreadMessagesCount": integer
"autoReplyEnabled": boolean
"isAutoReplyNow": boolean
"lastReadAt": string (timestamp)
"createdAt": string (timestamp)
```

```

    "isGroupChat": boolean
    "enableGroupMention"?: boolean // Optional
    "queryMetadata"?: object // Optional
    "toolMask"?: object // Records the enabled/disabled status of each
tool {tool_id: bool} (Optional)
    "connectorMask"?: object // Records the enabled/disabled status of
each connector {connector_id: bool} (Optional)
    "thinkingConfig"?: object // None=not set (uses chatbot config),
{}=explicitly disabled, {...}=custom override (Optional)
    "thinkingConfigSchema": object
    "effectiveThinkingConfig": object // Return the effective thinking
config after applying the priority chain:
conversation override > chatbot config > LLM metadata.

```

This allows the frontend to display the actual thinking config in use, even when the conversation has no explicit override.

Returns None when nothing is configured at any level, so the frontend can distinguish "Default" (None = use model/assistant default) from "Off" ({} = explicitly disabled). Collapsing the unconfigured case to {} would make a fresh conversation display as Off instead of Default.

```

    "llm": string (uuid)
    "deepResearchStatus": // Status of deep research for this
conversation

* `not_used` - Not Used
* `started` - Started
* `running` - Running
* `completed` - Completed
* `failed` - Failed

{
}

    "deepResearchOutputFormat": // may have different types
    string (enum: canvas, chat, file) // Chosen delivery form for the
finished report. Set only when the user clicks a format in the pre-
research choice card; reset to NULL each time Deep Research is
(re-)armed. NULL means no explicit choice (or a plan-skipping request)
and falls back to Canvas at render time, but stays distinct from an
explicit canvas pick so the frontend can lock the card only after a real
selection. To roll back to fixed-Canvas behavior, force this to
NULL/canvas – no redeploy needed.

```

```

* `canvas` - Canvas
* `chat` - Chat reply
* `file` - Downloadable file

```

```

    "ipAddress": string
    "ipCountryCode": string
    "ipRecordedAt": string (timestamp)
    "assignee":
    {
        "id": string (uuid)
        "name": string
        "userId": string (uuid)
    }
    "status"?: string (enum: open, resolved, queued) // * `open` - Open
* `resolved` - Resolved
* `queued` - Queued (Optional)
    "tags": [
        {
            "id": string (uuid)
            "name": string
            "memberIds"?: [ // Optional
                string (uuid)
            ]
        }
    ]
    "progressStatus": string
    "firstResponseAt"?: string (timestamp) // Optional
    "queuedAt"?: string (timestamp) // Optional
    "assignedAt"?: string (timestamp) // Optional
    "transferReason"?: string // Optional
    "satisfactionScore": integer // Conversation satisfaction rating on
a 5-point scale (1=very dissatisfied, 5=very satisfied)
    "satisfactionComment": string
    "satisfactionSubmittedAt": string (timestamp)
    "latestAnalysisSummary": string // The conversation's current
analysis summary (its active summary revision).

```

Reads the `latest\_analysis\_summary` annotation added by
`setup\_eager\_loading`
to avoid an N+1 query. All read paths (list / retrieve / create) eager-
load,
so the annotation is present; it falls back to '' only for instances
built
outside `setup\_eager\_loading`.

```

    "clientPlatform": // Client platform that created this
conversation (web, desktop)

```

* `web` - Web

* `desktop` - Desktop

```

    {
    }

    "workingDirectory": string // Absolute path of the local folder
    bound to a desktop conversation. Only set for desktop conversations; null
    for web conversations.

    "slideProjectId": string

    "matchedField": string // Cached per instance:
    ``get_matched_message_id`` / ``get_matched_snippet``
    both read this, so resolving it once avoids recomputing the keyword and
    title-match logic three times per serialized conversation.

    "matchedSnippet": string // Keyword-centred excerpt of the matched
    message, with ellipses.

    Reads the ``matched_message_content`` annotation added by
    ``ConversationViewSet._annotate_keyword_match``; no extra query.

    "matchedMessageId": string

    "pinnedAt": string (timestamp) // When the conversation was pinned.
    Null means unpinned.
    }
  ]
}

```

Response Example Values

```

{
  "count": 123,
  "next": "http://api.example.org/accounts/?page=4",
  "previous": "http://api.example.org/accounts/?page=2",
  "results": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "contact": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string",
        "avatar": "https://example.com/file.jpg",
        "email": "response@example.com",
        "phoneNumber": "Response string",
        "metadata": null,
        "sourceId": "Response string",
        "tags": [
          {
            "id": "550e8400-e29b-41d4-a716-446655440000",
            "name": "Response string"
          }
        ]
      }
    }
  ]
}

```



```
]
},
"inbox": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "channelType": "line",
  "unreadConversationsCount": 456,
  "signAuth": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "signSource": "550e8400-e29b-41d4-a716-446655440000",
    "signParams": {
      "keycloak": {
        "clientId": "Response string",
        "url": "Response string",
        "realm": "Response string"
      },
      "line": {
        "liffId": "Response string"
      },
      "ad": {
        "saml": "Response string"
      }
    }
  },
  "enableAnalysis": false
},
"title": "Response string",
"lastMessage": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "Response string",
  "content": "Response string",
  "createdAt": "Response string"
},
"lastMessageCreatedAt": "Response string",
"unreadMessagesCount": 456,
"autoReplyEnabled": false,
"isAutoReplyNow": false,
"lastReadAt": "Response string",
"createdAt": "Response string",
"isGroupChat": false,
"enableGroupMention": false,
"queryMetadata": null,
"toolMask": null,
"connectorMask": null,
"thinkingConfig": null,
```

```
"thinkingConfigSchema": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
},
"effectiveThinkingConfig": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
},
"llm": "550e8400-e29b-41d4-a716-446655440000",
"deepResearchStatus": {},
"deepResearchOutputFormat": "canvas",
"ipAddress": "Response string",
"ipCountryCode": "Response string",
"ipRecordedAt": "Response string",
"assignee": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "userId": "550e8400-e29b-41d4-a716-446655440000"
},
"status": "open",
"tags": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "memberIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }
],
"progressStatus": "Response string",
"firstResponseAt": "Response string",
"queuedAt": "Response string",
"assignedAt": "Response string",
"transferReason": "Response string",
"satisfactionScore": 456,
"satisfactionComment": "Response string",
"satisfactionSubmittedAt": "Response string",
"latestAnalysisSummary": "Response string",
"clientPlatform": {},
"workingDirectory": "Response string",
"slideProjectId": "Response string",
"matchedField": "Response string",
"matchedSnippet": "Response string",
```

```
    "matchedMessageId": "Response string",
    "pinnedAt": "Response string"
  }
]
```

Get Contact Latest Conversation

GET

</api/v1/contacts/{id}/conversations/latest/>

Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Contact.
<code>inbox</code>	X	string	Filter by a single inbox ID (UUID format)
<code>inboxes</code>	X	string	Filter by multiple inbox IDs (comma-separated, e.g.: uuid1,uuid2 or use multiple inboxes parameters)
<code>keyword</code>	X	string	Search message content within conversations (case-insensitive)

Code Examples

Shell/Bash

```
# API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/conversations/latest/?
inbox=example&inboxes=example&keyword=example"

-H "Authorization: Api-Key YOUR_API_KEY"

# Make sure to replace YOUR_API_KEY and verify the request data
before running.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY'
  }
};

axios.get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/conversations/latest/?inbox=example&inboxes=example&keyword=example", config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/conversations/latest/?inbox=example&inboxes=example&keyword=example"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client->get("https://api.maiagent.ai/api/v1/contacts/550e8400-e29b-41d4-a716-446655440000/conversations/latest/?inbox=example&inboxes=example&keyword=example", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY'
        ]
    ]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "id": string (uuid)
```

```
"contact": { // Lightweight Contact Serializer for Conversation List
view
```

Only includes fields necessary for the List view, removing inboxes, mcp_credentials, and api_credentials to avoid N+1 query issues. email / phone_number / metadata are scalar fields on the Contact itself; they need to be displayed in the frontend conversation sidebar with no N+1 cost.

```
{
  "id": string (uuid)
  "name": string
  "avatar"?: string (uri) // Optional
  "email"?: string (email) // Optional
  "phoneNumber"?: string // Optional
  "metadata"?: object // Custom customer information, displayed in the
customer info section on the conversation page (Optional)
  "sourceId"?: string // Optional
  "tags": [
    {
      "id": string (uuid)
      "name": string
    }
  ]
}
}
"inbox": {
{
  "id": string (uuid)
  "name": string
  "channelType": string (enum: line, telegram, teams, web, messenger,
instagram, email, whatsapp, slack) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
  "unreadConversationsCount"?: integer // Optional
  "signAuth"?: // Optional
  {
    "id": string (uuid)
    "signSource": string (uuid)
```



```

    "signParams": {
      {
        "keycloak":
          {
            "clientId": string
            "url": string
            "realm": string
          }
        "line":
          {
            "liffId": string
          }
        "ad":
          {
            "saml": string
          }
      }
    }
    "enableAnalysis"?: boolean // Optional
  }
}
"title": string
"lastMessage"?: { // Optional
  {
    "id": string (uuid)
    "type"?: string // Optional
    "content"?: string // Optional
    "createdAt": string (timestamp)
  }
}
"lastMessageCreatedAt": string (timestamp)
"unreadMessagesCount": integer
"autoReplyEnabled": boolean
"isAutoReplyNow": boolean
"lastReadAt": string (timestamp)
"createdAt": string (timestamp)
"isGroupChat": boolean
"enableGroupMention"?: boolean // Optional
"queryMetadata"?: object // Optional
"toolMask"?: object // Records the enabled/disabled status of each tool
{tool_id: bool} (Optional)
"connectorMask"?: object // Records the enabled/disabled status of each
connector {connector_id: bool} (Optional)
"thinkingConfig"?: object // None=not set (uses chatbot config),

```

```
{}=explicitly disabled, {...}=custom override (Optional)
  "thinkingConfigSchema": object
  "effectiveThinkingConfig": object // Return the effective thinking
config after applying the priority chain:
conversation override > chatbot config > LLM metadata.
```

This allows the frontend to display the actual thinking config in use, even when the conversation has no explicit override.

Returns None when nothing is configured at any level, so the frontend can distinguish "Default" (None = use model/assistant default) from "Off" ({} = explicitly disabled). Collapsing the unconfigured case to {} would make a fresh conversation display as Off instead of Default.

```
  "llm": string (uuid)
  "deepResearchStatus": // Status of deep research for this conversation
```

```
* `not_used` - Not Used
```

```
* `started` - Started
```

```
* `running` - Running
```

```
* `completed` - Completed
```

```
* `failed` - Failed
```

```
{
}
```

```
  "deepResearchOutputFormat": // may have different types
  string (enum: canvas, chat, file) // Chosen delivery form for the
finished report. Set only when the user clicks a format in the pre-
research choice card; reset to NULL each time Deep Research is
(re-)armed. NULL means no explicit choice (or a plan-skipping request)
and falls back to Canvas at render time, but stays distinct from an
explicit canvas pick so the frontend can lock the card only after a real
selection. To roll back to fixed-Canvas behavior, force this to
NULL/canvas – no redeploy needed.
```

```
* `canvas` - Canvas
```

```
* `chat` - Chat reply
```

```
* `file` - Downloadable file
```

```
  "ipAddress": string
  "ipCountryCode": string
  "ipRecordedAt": string (timestamp)
  "assignee":
  {
    "id": string (uuid)
    "name": string
    "userId": string (uuid)
  }
```

```

    "status"?: string (enum: open, resolved, queued) // * `open` - Open
    * `resolved` - Resolved
    * `queued` - Queued (Optional)
    "tags": [
        {
            "id": string (uuid)
            "name": string
            "memberIds"?: [ // Optional
                string (uuid)
            ]
        }
    ]
    "progressStatus": string
    "firstResponseAt"?: string (timestamp) // Optional
    "queuedAt"?: string (timestamp) // Optional
    "assignedAt"?: string (timestamp) // Optional
    "transferReason"?: string // Optional
    "satisfactionScore": integer // Conversation satisfaction rating on a
5-point scale (1=very dissatisfied, 5=very satisfied)
    "satisfactionComment": string
    "satisfactionSubmittedAt": string (timestamp)
    "latestAnalysisSummary": string // The conversation's current analysis
summary (its active summary revision).

```

Reads the `latest\_analysis\_summary` annotation added by
`setup\_eager\_loading`
to avoid an N+1 query. All read paths (list / retrieve / create) eager-
load,
so the annotation is present; it falls back to `''` only for instances
built
outside `setup\_eager\_loading`.

```

    "clientPlatform": // Client platform that created this conversation
(web, desktop)

    * `web` - Web
    * `desktop` - Desktop
    {
    }
    "workingDirectory": string // Absolute path of the local folder bound
to a desktop conversation. Only set for desktop conversations; null for
web conversations.
    "slideProjectId": string
    "matchedField": string // Cached per instance:
`get_matched_message_id` / `get_matched_snippet`
both read this, so resolving it once avoids recomputing the keyword and

```

```

title-match logic three times per serialized conversation.
    "matchedSnippet": string // Keyword-centred excerpt of the matched
message, with ellipses.

Reads the ``matched_message_content`` annotation added by
``ConversationViewSet._annotate_keyword_match``; no extra query.
    "matchedMessageId": string
    "pinnedAt": string (timestamp) // When the conversation was pinned.
Null means unpinned.
}

```

Response Example Values

```

{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "contact": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "avatar": "Response string",
    "email": "response@example.com",
    "phoneNumber": "Response string",
    "metadata": null,
    "sourceId": "Response string",
    "tags": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "Response string"
      }
    ]
  },
  "inbox": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "channelType": "line",
    "unreadConversationsCount": 456,
    "signAuth": {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "signSource": "550e8400-e29b-41d4-a716-446655440000",
      "signParams": {
        "keycloak": {
          "clientId": "Response string",
          "url": "Response string",
          "realm": "Response string"
        }
      }
    },

```

```
    "line": {
      "liffId": "Response string"
    },
    "ad": {
      "saml": "Response string"
    }
  },
  "enableAnalysis": false
},
"title": "Response string",
"lastMessage": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "Response string",
  "content": "Response string",
  "createdAt": "Response string"
},
"lastMessageCreatedAt": "Response string",
"unreadMessagesCount": 456,
"autoReplyEnabled": false,
"isAutoReplyNow": false,
"lastReadAt": "Response string",
"createdAt": "Response string",
"isGroupChat": false,
"enableGroupMention": false,
"queryMetadata": null,
"toolMask": null,
"connectorMask": null,
"thinkingConfig": null,
"thinkingConfigSchema": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
},
"effectiveThinkingConfig": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response example name",
  "description": "Response example description"
},
"llm": "550e8400-e29b-41d4-a716-446655440000",
"deepResearchStatus": {},
"deepResearchOutputFormat": "canvas",
"ipAddress": "Response string",
"ipCountryCode": "Response string",
"ipRecordedAt": "Response string",
```

```
"assignee": {
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Response string",
  "userId": "550e8400-e29b-41d4-a716-446655440000"
},
"status": "open",
"tags": [
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Response string",
    "memberIds": [
      "550e8400-e29b-41d4-a716-446655440000"
    ]
  }
],
"progressStatus": "Response string",
"firstResponseAt": "Response string",
"queuedAt": "Response string",
"assignedAt": "Response string",
"transferReason": "Response string",
"satisfactionScore": 456,
"satisfactionComment": "Response string",
"satisfactionSubmittedAt": "Response string",
"latestAnalysisSummary": "Response string",
"clientPlatform": {},
"workingDirectory": "Response string",
"slideProjectId": "Response string",
"matchedField": "Response string",
"matchedSnippet": "Response string",
"matchedMessageId": "Response string",
"pinnedAt": "Response string"
}
```

Status Code: **404** - No response body

Broadcast Message

POST

</api/v1/contacts/broadcast-message/>

Request

Request Parameters

Field	Type	Required	Description
contactIds	array[string]	No	[Direct selection mode] List of contact IDs to send the message to
inboxId	string (uuid)	No	[Exclusion/send-all mode] Inbox ID; will send to all contacts under this inbox
excludeContactIds	array[string]	No	[Exclusion mode] List of contact IDs to exclude; must be used together with inbox_id
message	string	Yes	Message content to send

Request Schema Example

```
{
  "contactIds"?: [ // [Direct selection mode] List of contact IDs to send
the message to (Optional)
    string (uuid)
  ]
  "inboxId"?: string (uuid) // [Exclusion/send-all mode] Inbox ID; will
send to all contacts under this inbox (Optional)
  "excludeContactIds"?: [ // [Exclusion mode] List of contact IDs to
exclude; must be used together with inbox_id (Optional)
    string (uuid)
  ]
  "message": string // Message content to send
}
```

Request Example Values

```
{
  "contactIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "inboxId": "550e8400-e29b-41d4-a716-446655440000",
```

```
"excludeContactIds": [  
  "550e8400-e29b-41d4-a716-446655440000"  
],  
"message": "This is an example message"  
}
```

Code Examples

Shell/Bash

```
# API call example (Shell)  
curl -X POST "https://api.maiagent.ai/api/v1/contacts/broadcast-  
message/"  
  
-H "Authorization: Api-Key YOUR_API_KEY"  
  
-H "Content-Type: application/json"  
  
-d '{  
  "contactIds": [  
    "550e8400-e29b-41d4-a716-446655440000"  
  ],  
  "inboxId": "550e8400-e29b-41d4-a716-446655440000",  
  "excludeContactIds": [  
    "550e8400-e29b-41d4-a716-446655440000"  
  ],  
  "message": "This is an example message"  
}'  
  
# Make sure to replace YOUR_API_KEY and verify the request data  
before running.
```



```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "contactIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "inboxId": "550e8400-e29b-41d4-a716-446655440000",
  "excludeContactIds": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "message": "This is an example message"
};

axios.post("https://api.maiagent.ai/api/v1/contacts/broadcast-
message/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error occurred:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/contacts/broadcast-message/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "contactIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "inboxId": "550e8400-e29b-41d4-a716-446655440000",
    "excludeContactIds": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "message": "This is an example message"
}

response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/contacts/broadcast-message/",
[
    'headers' => [
        'Authorization' => 'Api-Key YOUR_API_KEY',
        'Content-Type' => 'application/json'
    ],
    'json' => {
        "contactIds": [
            "550e8400-e29b-41d4-a716-446655440000"
        ],
        "inboxId": "550e8400-e29b-41d4-a716-446655440000",
        "excludeContactIds": [
            "550e8400-e29b-41d4-a716-446655440000"
        ],
        "message": "This is an example message"
    }
]);

    $data = json_decode($response->getBody(), true);
    echo "Response received successfully:\n";
    print_r($data);
} catch (Exception $e) {
    echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

Response

Status Code: 200

Response Schema Example

```
{
  "results"?: [ // Optional
    {
      "contact_id"?: string (uuid) // Optional
      "conversation_id"?: string (uuid) // Optional
      "message_id"?: string (uuid) // Optional
      "status"?: string // Optional
    }
  ]
  "errors"?: [ // Optional
    {
      "contact_id"?: string (uuid) // Optional
      "error"?: string // Optional
    }
  ]
  "total_contacts"?: integer // Optional
  "success_count"?: integer // Optional
  "error_count"?: integer // Optional
}
```

Response Example Values

```
{
  "results": [
    {
      "contact_id": "550e8400-e29b-41d4-a716-446655440000",
      "conversation_id": "550e8400-e29b-41d4-a716-446655440000",
      "message_id": "550e8400-e29b-41d4-a716-446655440000",
      "status": "success"
    }
  ],
  "errors": [
    {
      "contact_id": "550e8400-e29b-41d4-a716-446655440000",
      "error": "Response string"
    }
  ]
}
```

```
    }  
  ],  
  "total_contacts": 456,  
  "success_count": 456,  
  "error_count": 456  
}
```

API call example (Shell)

...

目錄

1. Create Role
2. Bulk Add Role Members
3. Bulk Assign AI Assistants to Role
4. Bulk Assign Inboxes to Role
5. Bulk Assign Knowledge Bases to Role
6. Bulk Assign Databases to Role
7. List Permissions
8. List Roles
9. Get Role Details
10. Get Role Details
11. Get Role Details
12. Get Role Details
13. List Role Members
14. View Specific Role Member
15. List Role's AI Assistants
16. List Role's Inboxes
17. List Role's Knowledge Bases
18. List Role's Databases
19. Update Role Permissions
20. Partially Update Role Permissions
21. Delete Role
22. Remove Role Member
23. Remove AI Assistant from Role
24. Remove Inbox from Role
25. Remove Knowledge Base from Role
26. Remove Database from Role

API call example (Shell)

Create Role

POST**/api/v1/organizations/{organizationPk}/groups/**

Parameters

Parameter	Required	Type	Description
<code>organizationPk</code>	V	string	A UUID string identifying this Organization ID

Request

Request Parameters

Field	Type	Required	Description
name	string	Yes	
organization	string (uuid)	No	
permissions	array[string]	Yes	
canCreateChatbot	boolean	No	
canCreateKnowledgeBase	boolean	No	
canCreateInbox	boolean	No	
canCreateDatabase	boolean	No	
canCreateTool	boolean	No	
canCreateAgentUiTool	boolean	No	
canCreateSkill	boolean	No	
maigptCanCodeInterpreter	boolean	No	Whether members of this role may use the code interpreter built-in tool in MaiGPT.
maigptCanRag	boolean	No	Whether members of this role may use the knowledge base

Field	Type	Required	Description
			RAG built-in tool in MaiGPT.
maigptCanDeepResearch	boolean	No	Whether members of this role may use Deep Research in MaiGPT. Deep Research reaches the public internet through its own built-in search, so a role that revokes the official web-search tools needs this...
maigptCanBrowserTool	boolean	No	Whether members of this role may use the browser built-in tool in MaiGPT.
allowedLlms	array[string]	No	
allowedTools	array[string]	No	
allowedConnectors	array[string]	No	
allowedSkills	array[string]	No	

Request Schema Example

```
{
  "name": string
  "organization?": string (uuid) // Optional
  "permissions": [
    string (uuid)
  ]
  "canCreateChatbot?": boolean // Optional
  "canCreateKnowledgeBase?": boolean // Optional
  "canCreateInbox?": boolean // Optional
  "canCreateDatabase?": boolean // Optional
  "canCreateTool?": boolean // Optional
  "canCreateAgentUiTool?": boolean // Optional
  "canCreateSkill?": boolean // Optional
  "maigptCanCodeInterpreter?": boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (Optional)
  "maigptCanRag?": boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (Optional)
  "maigptCanDeepResearch?": boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
```

through its own built-in search, so a role that revokes the official web-search tools needs this switch too. (optional)

"maigptCanBrowserTool"?: boolean // Whether members of this role may use the browser built-in tool in MaiGPT. (optional)

```
"allowedLlms"?: [ // Optional
  string (uuid)
]
"allowedTools"?: [ // Optional
  string (uuid)
]
"allowedConnectors"?: [ // Optional
  string (uuid)
]
"allowedSkills"?: [ // Optional
  string (uuid)
]
}
```

Request Example Values

```
{
  "name": "Example Name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [
```

```
    "550e8400-e29b-41d4-a716-446655440000"  
  ],  
  "allowedSkills": [  
    "550e8400-e29b-41d4-a716-446655440000"  
  ]  
}
```

Code Examples

```
# API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
  "name": "Example Name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedSkills": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}'
```

```
# Please replace YOUR_API_KEY and verify the request data before  
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
  headers: {
    'Authorization': 'Api-Key YOUR_API_KEY',
    'Content-Type': 'application/json'
  }
};

// Request body (payload)
const data = {
  "name": "Example Name",
  "organization": "550e8400-e29b-41d4-a716-446655440000",
  "permissions": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "canCreateChatbot": true,
  "canCreateKnowledgeBase": true,
  "canCreateInbox": true,
  "canCreateDatabase": true,
  "canCreateTool": true,
  "canCreateAgentUiTool": true,
  "canCreateSkill": true,
  "maigptCanCodeInterpreter": true,
  "maigptCanRag": true,
  "maigptCanDeepResearch": true,
  "maigptCanBrowserTool": true,
  "allowedLlms": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedTools": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedConnectors": [
    "550e8400-e29b-41d4-a716-446655440000"
  ],
  "allowedSkills": [
    "550e8400-e29b-41d4-a716-446655440000"
  ]
}
```

```
};

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/", data, config)
  .then(response => {
    console.log('Response received successfully:');
    console.log(response.data);
  })
  .catch(error => {
    console.error('Request error:');
    console.error(error.response?.data || error.message);
  });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/"
headers = {
    "Authorization": "Api-Key YOUR_API_KEY",
    "Content-Type": "application/json"
}

# Request body (payload)
data = {
    "name": "Example Name",
    "organization": "550e8400-e29b-41d4-a716-446655440000",
    "permissions": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "canCreateChatbot": true,
    "canCreateKnowledgeBase": true,
    "canCreateInbox": true,
    "canCreateDatabase": true,
    "canCreateTool": true,
    "canCreateAgentUiTool": true,
    "canCreateSkill": true,
    "maigptCanCodeInterpreter": true,
    "maigptCanRag": true,
    "maigptCanDeepResearch": true,
    "maigptCanBrowserTool": true,
    "allowedLlms": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedTools": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedConnectors": [
        "550e8400-e29b-41d4-a716-446655440000"
    ],
    "allowedSkills": [
        "550e8400-e29b-41d4-a716-446655440000"
    ]
}
```



```
response = requests.post(url, json=data, headers=headers)
try:
    print("Response received successfully:")
    print(response.json())
except Exception as e:
    print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
    $response = $client-
>post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/", [
        'headers' => [
            'Authorization' => 'Api-Key YOUR_API_KEY',
            'Content-Type' => 'application/json'
        ],
        'json' => {
            "name": "Example Name",
            "organization": "550e8400-e29b-41d4-a716-446655440000",
            "permissions": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "canCreateChatbot": true,
            "canCreateKnowledgeBase": true,
            "canCreateInbox": true,
            "canCreateDatabase": true,
            "canCreateTool": true,
            "canCreateAgentUiTool": true,
            "canCreateSkill": true,
            "maigptCanCodeInterpreter": true,
            "maigptCanRag": true,
            "maigptCanDeepResearch": true,
            "maigptCanBrowserTool": true,
            "allowedLlms": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "allowedTools": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "allowedConnectors": [
                "550e8400-e29b-41d4-a716-446655440000"
            ],
            "allowedSkills": [
```

```

        "550e8400-e29b-41d4-a716-446655440000"
    ]
}
]);

$data = json_decode($response->getBody(), true);
echo "Response received successfully:\n";
print_r($data);
} catch (Exception $e) {
    echo 'Request error: ' . $e->getMessage();
}
?>

```

Response

Status Code: 201

Response Schema Example

```

{
  "id": string (uuid)
  "name": string
  "type":
  {
  }
  "permissions": [ // Returns the group permissions list

```

Processing flow:

1. Split group permissions into parent and child permissions
 2. Group child permissions by parent permission ID
 3. If child permissions are missing their parent, automatically load the missing parent permissions from the database
 4. Serialize parent permissions and corresponding child permissions, sorted by order
- ```

 object
]
 "membersPreview": [

```

```

 string
]
"membersCount": integer // Returns the remaining member count (total
minus 3 displayed), used for frontend +N badge display

Different from resource count:

 members_count: returns total - 3 (for +N badge), None means total
 <= 3
 chatbots_count/knowledge_bases_count/inboxes_count: returns total
 count

"chatbotsCount": integer
"chatbotsPreview": [
 string
]
"knowledgeBasesCount": integer
"knowledgeBasesPreview": [
 string
]
"inboxesCount": integer
"inboxesPreview": [
 string
]
"databasesCount": integer
"databasesPreview": [
 string
]
"canCreateChatbot"?: boolean // Optional
"canCreateKnowledgeBase"?: boolean // Optional
"canCreateInbox"?: boolean // Optional
"canCreateDatabase"?: boolean // Optional
"canCreateTool"?: boolean // Optional
"canCreateAgentUiTool"?: boolean // Optional
"canCreateSkill"?: boolean // Optional
"maigptCanCodeInterpreter"?: boolean // Whether members of this role
may use the code interpreter built-in tool in MaiGPT. (Optional)
"maigptCanRag"?: boolean // Whether members of this role may use the
knowledge base RAG built-in tool in MaiGPT. (Optional)
"maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (optional)
"maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)

```

```
"createdAt": string (timestamp)
}
```

## Response Example Values

```
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "type": {},
 "permissions": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response Example Name",
 "description": "Response Example Description"
 }
],
 "membersPreview": [
 "response string"
],
 "membersCount": 456,
 "chatbotsCount": 456,
 "chatbotsPreview": [
 "response string"
],
 "knowledgeBasesCount": 456,
 "knowledgeBasesPreview": [
 "response string"
],
 "inboxesCount": 456,
 "inboxesPreview": [
 "response string"
],
 "databasesCount": 456,
 "databasesPreview": [
 "response string"
],
 "canCreateChatbot": false,
 "canCreateKnowledgeBase": false,
 "canCreateInbox": false,
 "canCreateDatabase": false,
 "canCreateTool": false,
 "canCreateAgentUiTool": false,
 "canCreateSkill": false,
 "maigptCanCodeInterpreter": false,
}
```

```
"maigptCanRag": false,
"maigptCanDeepResearch": false,
"maigptCanBrowserTool": false,
"createdAt": "response string"
}
```

## Bulk Add Role Members

POST

</api/v1/organizations/{organizationPk}/groups/{groupPk}/group-members/bulk-create/>

### Parameters

| Parameter      | Required | Type   | Description                                    |
|----------------|----------|--------|------------------------------------------------|
| groupPk        | V        | string |                                                |
| organizationPk | V        | string | A UUID string identifying this Organization ID |

### Request

Request Parameters

| Field   | Type          | Required | Description |
|---------|---------------|----------|-------------|
| members | array[string] | Yes      |             |

Request Schema Example

```
{
 "members": [
 string (uuid)
]
}
```

```
]
}
```

### Request Example Values

```
{
 "members": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}
```

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/bulk-create/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "members": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "members": [
 "550e8400-e29b-41d4-a716-446655440000"
]
};

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-members/bulk-create/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-members/bulk-create/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "members": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->
post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-members/bulk-create/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "members": [
 "550e8400-e29b-41d4-a716-446655440000"
]
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

**Status Code: 201**

## Response Schema Example

```
[
 {
 "id"?: string (uuid) // Role member ID (Optional)
 "member"?: { // Optional
 {
 "id"?: string (uuid) // Member ID (Optional)
 "name"?: string // Member name (Optional)
 "email"?: string (email) // Member email (Optional)
 "organization"?: { // Optional
 {
 "id"?: string (uuid) // Organization ID (Optional)
 "name"?: string // Organization name (Optional)
 "createdAt"?: string // Organization creation timestamp
 (Optional)
 "usageStatistics"?: { // Optional
 {
 "chatbotsCount"?: integer // Chatbot count (Optional)
 "countableChatbotsCount"?: integer // Number of chatbots
 counted toward the creation quota (chatbots whose mode is not on the plan
 allowlist are excluded) (Optional)
 "exemptChatbotsByMode"?: object // Number of chatbots not
 counted toward the quota, grouped by the mode that exempts them
 (Optional)
 "canCreateChatbot"?: boolean // Whether chatbots can be created
 (Optional)
 "createChatbotLimit"?: integer // Maximum number of chatbots
 that can be created (-1 means unlimited, 0 means creation is disabled)
 (Optional)
 "currentMonthWordsCountTotal"?: integer // Current month total
 word limit (Optional)
 "currentMonthUsedWordsCountTotal"?: integer // Current month
 used word count (Optional)
 "currentMonthUsedConversationsCountTotal"?: integer // Current
 month used conversation count (Optional)
 "availableUploadFilesSizeTotal"?: integer // Total available
 upload file size (Optional)
 "usedUploadFileSizeTotal"?: integer // Used upload file size
 (Optional)
 }
 }
 "organizationPlan"?: { // Optional
 {
 "id"?: string (uuid) // Plan ID (Optional)
```

```

 "planName"?: string // Plan name (Optional)
 "expiredAt"?: string // Plan expiration time (Optional)
 }
}
}
}
"isOwner"?: boolean // Whether the member is the organization owner
(Optional)
"permissions"?: { // Optional
{
 "hasOrganizationAccessPermission"?: boolean // Whether the member
has organization access permission (Optional)
 "hasChatAccessPermission"?: boolean // Whether the member has
chat access permission (Optional)
 "hasConversationAccessPermission"?: boolean // Whether the member
has conversation access permission (Optional)
 "hasChatbotAccessPermission"?: boolean // Whether the member has
chatbot access permission (Optional)
}
}
 "createdAt"?: string // Member creation timestamp (Optional)
}
}
 "createdAt"?: string // Role member creation timestamp (Optional)
}
}
]

```

## Response Example Values

```

[
{
 "id": "c1e54f04-01fc-45e9-b40c-9d8b37bd0ac0",
 "member": {
 "id": "af4e472d-3f9b-446b-bab8-05de4112d086",
 "name": "h",
 "email": "h@gmail.com",
 "organization": {
 "id": "670e3d30-9d24-47b4-b2cc-6bf224b2f3bb",
 "name": "Updated Org Name",
 "createdAt": "1745288290000",
 "usageStatistics": {
 "chatbotsCount": 1,
 "countableChatbotsCount": 1,
 "exemptChatbotsByMode": {},
 }
 }
 }
}
]

```

```

 "canCreateChatbot": false,
 "createChatbotLimit": 1,
 "currentMonthWordsCountTotal": 1000000,
 "currentMonthUsedWordsCountTotal": 0,
 "currentMonthUsedConversationsCountTotal": 0,
 "availableUploadFileSizeTotal": 104857600,
 "usedUploadFileSizeTotal": 0
 },
 "organizationPlan": {
 "id": "16136181-ae0d-4b3f-aaeb-9a6ed251459f",
 "planName": "Free Plan",
 "expiredAt": null
 }
},
"isOwner": false,
"permissions": {
 "hasMaigptAccessPermission": false,
 "hasChatbotAccessPermission": false,
 "hasAgentopsAccessPermission": false,
 "hasConversationAccessPermission": false,
 "hasChatAccessPermission": false,
 "hasDeveloperAccessPermission": false,
 "hasOrganizationAccessPermission": true
},
"createdAt": "1748335331000"
},
"createdAt": "1748341252000"
}
]

```

## Bulk Assign AI Assistants to Role

POST

</api/v1/organizations/{organizationPk}/groups/{groupPk}/group-chatbots/bulk-create/>

### Parameters

| Parameter      | Required | Type    | Description                                    |
|----------------|----------|---------|------------------------------------------------|
| groupPk        | V        | string  |                                                |
| organizationPk | V        | string  | A UUID string identifying this Organization ID |
| page           | X        | integer | A page number within the paginated result set. |
| pageSize       | X        | integer | Number of results to return per page.          |
| query          | X        | string  |                                                |
| search         | X        | string  |                                                |

## Request

### Request Parameters

| Field    | Type          | Required | Description |
|----------|---------------|----------|-------------|
| chatbots | array[string] | Yes      |             |

### Request Schema Example

```
{
 "chatbots": [
 string (uuid)
]
}
```

### Request Example Values

```
{
 "chatbots": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}
```

```
]
}
```

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/bulk-create/?
page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "chatbots": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "chatbots": [
 "550e8400-e29b-41d4-a716-446655440000"
]
};

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-chatbots/bulk-create/?page=1&pageSize=1&query=example&search=example", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-chatbots/bulk-create/?page=1&pageSize=1&query=example&search=example"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "chatbots": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-chatbots/bulk-create/?page=1&pageSize=1&query=example&search=example", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "chatbots": [
 "550e8400-e29b-41d4-a716-446655440000"
]
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

## Status Code: 201

### Response Schema Example

```
{
 "count": integer
 "next"?: string (uri) // Optional
 "previous"?: string (uri) // Optional
 "results": [
 {
 "id": string (uuid)
 "group": string (uuid)
 "chatbot":
 {
 "id": string (uuid)
 "name": string // Name of the bot; in Agent mode it has semantic
 meaning, in other modes it is used to distinguish different bots
 "largeLanguageModel": string (uuid) // Large language model used
 by the bot for generating responses (including AI Gateway models built by
 the organization)
 "llm": // ID and name of the bound model, read directly from the
 FK; returned even if the model has been disabled by the organization
 {
 "id": string (uuid)
 "name": string
 }
 "embeddingModel"?: string (uuid) // Embedding model for
 vectorizing text, optional (Optional)
 "rerankerModel"?: string (uuid) // Reranker model, optional
 (Optional)
 "instructions"?: string // Bot role instructions, used to
 describe the bot role and behavior (Optional)
 "knowledgeBases"?: [// List of knowledge bases accessible to the
 bot (Optional)
 {
 "id": string (uuid)
 "name": string
 }
]
 "databases"?: [// List of SQL databases accessible to the bot
 (for Text-to-SQL feature) (Optional)
 {
 "id": string (uuid)
 "name": string
 }
]
 }
 }
]
}
```

```

]
 "updatedAt": string (timestamp)
 "organization"?: string (uuid) // Organization the bot belongs
to; if empty, it is a personal bot (Optional)
 "builtInWorkflow"?: string (uuid) // Built-in workflow for
predefined processing flows (Optional)
 "replyMode"?: // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `workflow` - Workflow
* `agent` - Agent (Optional)
 {
 }
 "template"?: string // Template used in template mode and hybrid
mode (Optional)
 "unanswerableTemplate"?: string // Template used in template mode
and hybrid mode when unable to answer (Optional)
 "totalWordsCount"?: integer (int64) // Cumulative total word
count used (Optional)
 "outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
 {
 }
 "rawOutputFormat"?: object // JSON schema definition for custom
output format (Optional)
 "groups"?: [// List of groups accessible to the bot (Optional)
 {
 "id": string (uuid)
 "name": string
 }
]
 "tools"?: [// List of tools available to the bot (Optional)
 {
 "id": string (uuid)
 "name": string
 "description": string
 "displayName": string
 "toolType": string
 }
]
 "skills"?: [// List of skills associated with the bot (Optional)

```

```

 {
 "id": string (uuid)
 "name": string
 "description": string
 }
]
 "connectors"?: [// Connectors associated with this chatbot,
gating per-contact OAuth authorization (Optional)
 {
 "id": string (uuid)
 "name": string
 }
]
 "agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
 {
 }
 "numberOfRetrievedChunks"?: integer // Number of retrieved
reference chunks, default is 12, minimum is 1 (Optional)
 "enableEvaluation"?: boolean // Optional
 "enableInlineCitations"?: boolean // Enable inline citation
feature, inserting [1][2] format citation markers in responses (Optional)
 "enableCodeInterpreter"?: boolean // When enabled, provides Code
Interpreter tool that can execute code in an isolated container
(Optional)
 "enableBrowserTool"?: boolean // Enable browser automation tool
for web tasks (Optional)
 "enableImageSearch"?: boolean // Provide the image_search tool
when any attached knowledge base uses a multimodal embedding model.
Disable to remove the tool entirely. (Optional)
 "enableClaudeStreamRetry"?: boolean // When a Claude streaming
call stalls (first-chunk or mid-stream), abort and re-issue the call once
in non-streaming mode. Mid-stream retries send a reset signal so the
client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
 "customMaxLlmOutputTokens"?: integer // Custom maximum LLM output
token count, minimum is 512 (Optional)
 "voiceConfig"?: { // Optional
 {
 "enabled"?: boolean // Whether this chatbot can be used as a
voice agent. Checked by every conversation platform (WebChat, SIP phone,
admin/marketplace voice test) instead of a per-platform toggle.

```

```
(Optional)
 "voiceAgentType"?: string (uuid) // Voice agent mode type
(Optional)
 "sttProvider"?: string (uuid) // Speech-to-Text service
provider (Optional)
 "sttConfig"?: object // Actual STT configuration parameters
(JSON format) (Optional)
 "ttsProvider"?: string (uuid) // Text-to-Speech service
provider (Optional)
 "ttsConfig"?: object // Actual TTS configuration parameters
(JSON format) (Optional)
 "realtimeProvider"?: string (uuid) // Realtime end-to-end voice
model provider (Optional)
 "realtimeConfig"?: object // Actual Realtime configuration
parameters (JSON format) (Optional)
 "realtimeInstructions"?: string // Prompt dedicated to the
realtime voice session; falls back to chatbot instructions when empty
(Optional)
 "turnHandling"?: object // Voice assistant conversation turn
and interruption control settings (Optional)
 "enableVoiceReply"?: boolean // When disabled, the voice agent
replies in text only and does not speak (Optional)
 }
 }
 "thinkingConfig"?: object // Thinking effort settings (JSON
format) (Optional)
 "enableToolSearching"?: boolean // Enable dynamic tool searching,
allowing agents to dynamically discover and load tools via
tool_searching_tool (Optional)
 "maxAgentIterations"?: integer // Maximum number of iterations
for AgentWorkflow (1-100, default: 20) (Optional)
 "agentTimeoutSeconds"?: integer // Timeout in seconds for
AgentWorkflow execution (30-1200, default: 600) (Optional)
 "agentTimeoutMessage"?: string // Custom user-facing message
shown when AgentWorkflow times out (max 500 chars). Leave empty to use
the system default. (Optional)
 "isMultimodalLlm": boolean // Whether the chatbot LLM supports
multimodal (read-only)
 "enableSizeBasedToolResultTruncation"?: boolean // When enabled,
tool results exceeding the character threshold are truncated using
head+tail strategy instead of the legacy SYSTEM_TOOLS whitelist approach.
Reduces memory bloat from large tool outputs (code interpreter bash
stdout, web search results, etc.). (Optional)
 "toolResultMaxChars"?: integer // Character threshold for size-
based tool result truncation (500-50000). Only effective when
```

```
enable_size_based_tool_result_truncation is True. (Optional)
 "enableVectorMemory"?: boolean // When enabled, archived messages
are vectorized and processed by fact extraction for semantic retrieval
(existing behavior). When disabled, archived messages are discarded and
only the active chat history window is used for multi-turn context.
(Optional)
 "enableCrossConversationSearch"?: boolean // When enabled, the
conversation history tools also reach this contact's other conversations
in the same inbox, within the lookback window. When disabled, they only
reach the current conversation. (Optional)
 "crossConversationLookbackDays"?: integer // How far back cross-
conversation search may reach, measured from each conversation's last
message (1-365 days). Leave empty to fall back to 90 days. Only effective
when enable_cross_conversation_search is True. (Optional)
 "useGatewayFallback"?: boolean // When enabled, use the fallback
group instead of the single LLM (Optional)
 "gatewayFallbackGroup"?: string (uuid) // Optional
 "enableSearchMessagesTool"?: boolean // Allow the AI to search
past messages in the current conversation. (Optional)
 "enableGetMessageContextTool"?: boolean // Allow the AI to
retrieve messages surrounding a specific message for context. (Optional)
 "enableReadAttachmentFullTextTool"?: boolean // Allow the AI to
read the full text of plain-text attachments. AGENT mode only. (Optional)
 "enableParseAttachmentContentTool"?: boolean // Allow the AI to
parse PDF, Office, audio, and other attachments. AGENT mode only.
(Optional)
 "enableListAvailableParsersTool"?: boolean // Allow the AI to
list available parsers and their capabilities. AGENT mode only.
(Optional)
 "enableGetDownloadLinkTool"?: boolean // Allow the AI to generate
shareable download links for produced content. AGENT mode only.
(Optional)
 "isLocked": boolean // Locked instance: config follows the
official version; only the dedicated KB can be modified
 "sourceBuildInAgent": string (uuid) // Points to the official
Build-in Agent blueprint when instantiated by a rental
 "canUpdate": boolean // Return whether the current user can
update this resource.
 "canDelete": boolean // Return whether the current user can
delete this resource.
}
 "canRead"?: boolean // Optional
 "canUpdate"?: boolean // Optional
 "canDelete"?: boolean // Optional
 "createdAt": string (timestamp)
```

```
}
]
}
```

## Response Example Values

```
{
 "count": 123,
 "next": "http://api.example.org/accounts/?page=4",
 "previous": "http://api.example.org/accounts/?page=2",
 "results": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "group": "550e8400-e29b-41d4-a716-446655440000",
 "chatbot": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
 "llm": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 },
 "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
 "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
 "instructions": "response string",
 "knowledgeBases": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
 "databases": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
 "updatedAt": "response string",
 "organization": "550e8400-e29b-41d4-a716-446655440000",
 "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
 "replyMode": {},
 "template": "response string",
 "unanswerableTemplate": "response string",
 "totalWordsCount": 456,
 },
],
 },
 "updated_at": "response string",
 "organization": "550e8400-e29b-41d4-a716-446655440000",
 "built_in_workflow": "550e8400-e29b-41d4-a716-446655440000",
 "reply_mode": {},
 "template": "response string",
 "unanswerable_template": "response string",
 "total_words_count": 456,
}
```



```
"outputMode": {},
"rawOutputFormat": null,
"groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
"tools": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "description": "response string",
 "displayName": "response string",
 "toolType": "response string"
 }
],
"skills": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "description": "response string"
 }
],
"connectors": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
"agentMode": {},
"numberOfRetrievedChunks": 456,
"enableEvaluation": false,
"enableInlineCitations": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
"enableImageSearch": false,
"enableClaudeStreamRetry": false,
"customMaxLlmOutputTokens": 456,
"voiceConfig": {
 "enabled": false,
 "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
 "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
 "sttConfig": null,
 "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
```

```

 "ttsConfig": null,
 "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
 "realtimeConfig": null,
 "realtimeInstructions": "response string",
 "turnHandling": null,
 "enableVoiceReply": false
 },
 "thinkingConfig": null,
 "enableToolSearching": false,
 "maxAgentIterations": 456,
 "agentTimeoutSeconds": 456,
 "agentTimeoutMessage": "response string",
 "isMultimodalLlm": false,
 "enableSizeBasedToolResultTruncation": false,
 "toolResultMaxChars": 456,
 "enableVectorMemory": false,
 "enableCrossConversationSearch": false,
 "crossConversationLookbackDays": 456,
 "useGatewayFallback": false,
 "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
 "enableSearchMessagesTool": false,
 "enableGetMessageContextTool": false,
 "enableReadAttachmentFullTextTool": false,
 "enableParseAttachmentContentTool": false,
 "enableListAvailableParsersTool": false,
 "enableGetDownloadLinkTool": false,
 "isLocked": false,
 "sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
 "canUpdate": false,
 "canDelete": false
},
"canRead": false,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string"
}
]
}

```

## Bulk Assign Inboxes to Role

---

POST

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-inboxes/bulk-create/

## Parameters

| Parameter      | Required | Type   | Description                                    |
|----------------|----------|--------|------------------------------------------------|
| groupPk        | V        | string |                                                |
| organizationPk | V        | string | A UUID string identifying this Organization ID |

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/bulk-create/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

// Request body (payload)
const data = null;

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/bulk-create/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-inboxes/bulk-create/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.post(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-inboxes/bulk-create/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

| Status Code | Description      |
|-------------|------------------|
| 200         | No response body |

# Bulk Assign Knowledge Bases to Role

POST

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-knowledge-bases/bulk-create/

## Parameters

| Parameter      | Required | Type    | Description                                    |
|----------------|----------|---------|------------------------------------------------|
| groupPk        | V        | string  |                                                |
| organizationPk | V        | string  | A UUID string identifying this Organization ID |
| page           | X        | integer | A page number within the paginated result set. |
| pageSize       | X        | integer | Number of results to return per page.          |
| query          | X        | string  |                                                |
| search         | X        | string  |                                                |

## Request

Request Parameters

| Field          | Type          | Required | Description |
|----------------|---------------|----------|-------------|
| knowledgeBases | array[string] | Yes      |             |

Request Schema Example

```
{
 "knowledgeBases": [
```

```
 string (uuid)
]
}
```

#### Request Example Values

```
{
 "knowledgeBases": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}
```

## Code Examples



```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/bulk-create/?
page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "knowledgeBases": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "knowledgeBases": [
 "550e8400-e29b-41d4-a716-446655440000"
]
};

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-knowledge-bases/bulk-create/?page=1&pageSize=1&query=example&search=example", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-knowledge-bases/bulk-create/?page=1&pageSize=1&query=example&search=example"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "knowledgeBases": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-knowledge-bases/bulk-create/?page=1&pageSize=1&query=example&search=example", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "knowledgeBases": [
 "550e8400-e29b-41d4-a716-446655440000"
]
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

## Status Code: 201

### Response Schema Example

```
{
 "count": integer
 "next"?: string (uri) // Optional
 "previous"?: string (uri) // Optional
 "results": [
 {
 "id": string (uuid)
 "group": string (uuid)
 "knowledgeBase":
 {
 "id": string (uuid)
 "createdBy":
 {
 "id": string (uuid)
 "name": string
 }
 "organization"?: string (uuid) // Optional
 "embeddingModel": string (uuid)
 "rerankerModel": string (uuid)
 "name": string
 "description"?: string // Optional
 "numberOfRetrievedChunks"?: integer // Number of retrieved
reference chunks, default is 12, minimum is 1 (Optional)
 "enableHyde"?: boolean // Enable HyDE, default is False
(Optional)
 "similarityCutoff"?: number (double) // Similarity cutoff,
default is 0.0, range is 0.0-1.0 (Optional)
 "enableRerank"?: boolean // Whether to enable reranking, default
is True (Optional)
 "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
 "vectorDatabaseType"?: // Type of vector database: elasticsearch
or opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai (Optional)
 {
 }
 "vectorDatabaseUrl"?: string // Custom vector database endpoint
 }
]
}
```

```
URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)
 "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not
used for OpenSearch. (Optional)
 "opensearchUsername"?: string // Username for OpenSearch Basic
Auth. (Optional)
 "opensearchPassword"?: string // Password for OpenSearch Basic
Auth. (Optional)
 "oracleUser"?: string // Oracle database username. (Optional)
 "oraclePassword"?: string // Oracle database password. (Optional)
 "labels": [
 {
 "id": string (uuid)
 "name": string
 }
]
 "chatbots"?: [// Optional
 {
 "id": string (uuid)
 "name": string
 }
]
 "groups"?: [// List of groups with access to the knowledge base
(Optional)
 {
 "id": string (uuid)
 "name": string
 }
]
 "filesCount": integer // Return count of KnowledgeBaseFile
excluding processed files, conversation attachments, and soft-deleted.
 "vectorStorageSize": integer
 "chunksCount": integer
 "canUpdate": boolean // Return whether the current user can
update this resource.
 "canDelete": boolean // Return whether the current user can
delete this resource.
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
}
"canRead"?: boolean // Optional
"canUpdate"?: boolean // Optional
"canDelete"?: boolean // Optional
"createdAt": string (timestamp)
```

```
}
]
}
```

## Response Example Values

```
{
 "count": 123,
 "next": "http://api.example.org/accounts/?page=4",
 "previous": "http://api.example.org/accounts/?page=2",
 "results": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "group": "550e8400-e29b-41d4-a716-446655440000",
 "knowledgeBase": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "createdBy": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 },
 },
 "organization": "550e8400-e29b-41d4-a716-446655440000",
 "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
 "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "description": "response string",
 "numberOfRetrievedChunks": 456,
 "enableHyde": false,
 "similarityCutoff": 456,
 "enableRerank": false,
 "enableRrf": false,
 "vectorDatabaseType": {},
 "vectorDatabaseUrl": "response string",
 "vectorDatabaseApiKey": "response string",
 "opensearchUsername": "response string",
 "opensearchPassword": "response string",
 "oracleUser": "response string",
 "oraclePassword": "response string",
 "labels": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 },
],
 "chatbots": [

```

```
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
 "filesCount": 456,
 "vectorStorageSize": 456,
 "chunksCount": 456,
 "canUpdate": false,
 "canDelete": false,
 "createdAt": "response string",
 "updatedAt": "response string"
},
"canRead": false,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string"
}
]
```

## Bulk Assign Databases to Role

POST

</api/v1/organizations/{organizationPk}/groups/{groupPk}/group-databases/bulk-create/>

### Parameters

| Parameter | Required | Type | Description |
|-----------|----------|------|-------------|
|-----------|----------|------|-------------|



|                |   |         |                                                |
|----------------|---|---------|------------------------------------------------|
| groupPk        | V | string  |                                                |
| organizationPk | V | string  | A UUID string identifying this Organization ID |
| page           | X | integer | A page number within the paginated result set. |
| pageSize       | X | integer | Number of results to return per page.          |
| query          | X | string  |                                                |
| search         | X | string  |                                                |

# Request

## Request Parameters

| Field     | Type          | Required | Description |
|-----------|---------------|----------|-------------|
| databases | array[string] | Yes      |             |

## Request Schema Example

```
{
 "databases": [
 string (uuid)
]
}
```

## Request Example Values

```
{
 "databases": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}
```

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/bulk-create/?
page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "databases": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "databases": [
 "550e8400-e29b-41d4-a716-446655440000"
]
};

axios.post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/bulk-create/?page=1&pageSize=1&query=example&search=example", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/bulk-create/?page=1&pageSize=1&query=example&search=example"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "databases": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->post("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/bulk-create/?page=1&pageSize=1&query=example&search=example", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "databases": [
 "550e8400-e29b-41d4-a716-446655440000"
]
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

## Status Code: 201

### Response Schema Example

```
{
 "count": integer
 "next"?: string (uri) // Optional
 "previous"?: string (uri) // Optional
 "results": [
 {
 "id": string (uuid)
 "group": string (uuid)
 "database":
 {
 "id": string (uuid)
 "name": string // Display name for this database connection
 "description": string // Description of the database and its
purpose
 "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL,
Oracle, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL

 {
 }
 "databaseUrl": string // Connection string for external
databases. Not required for MaiAgent type as it uses the system default.
 "includeTables": object // Optional list of tables to include.
Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files.
 "isActive": boolean // When disabled, this database will not be
used in Text-to-SQL queries
 "status":
 {
 }
 "deletedAt": string (timestamp)
 "tables": [
 {
 "id": string (uuid)
 "tableName": string // Name of the table in the database
 "description"?: string // Description of the table and its
data for better Text-to-SQL understanding (Optional)
```

```
 "status":
 {
 }
 "errorMessage": string // Error details if table creation or
deletion failed
 "sourceFile":
 {
 "id": string (uuid)
 "name": string
 "isFromKnowledgeBase": boolean // Determine if the source
file originated from a knowledge base upload.
```

Returns True if the file was uploaded via knowledge base,  
False if uploaded directly to the database.

```
 }
 "columns": [
 {
 "id": string (uuid)
 "columnName": string
 "columnType?": string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (Optional)
 "description?": string // Description of the column for
better Text-to-SQL understanding (Optional)
 "isPrimaryKey?": boolean // Optional
 "isNullable?": boolean // Optional
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
 }
]
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
}
]
"chatbots": [
 {
 "id": string (uuid)
 "name": string
 }
]
"groups": [
 {
 "id": string (uuid)
 "name": string
 }
]
]
```

```

 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
 }
 "canRead"?: boolean // Optional
 "canUpdate"?: boolean // Optional
 "canDelete"?: boolean // Optional
 "createdAt": string (timestamp)
}
]
}

```

## Response Example Values

```

{
 "count": 123,
 "next": "http://api.example.org/accounts/?page=4",
 "previous": "http://api.example.org/accounts/?page=2",
 "results": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "group": "550e8400-e29b-41d4-a716-446655440000",
 "database": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "description": "response string",
 "databaseType": {},
 "databaseUrl": "response string",
 "includeTables": null,
 "isActive": false,
 "status": {},
 "deletedAt": "response string",
 "tables": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "tableName": "response string",
 "description": "response string",
 "status": {},
 "errorMessage": "response string",
 "sourceFile": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "isFromKnowledgeBase": false
 },
 "columns": [

```



```

 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "response string",
 "columnType": "response string",
 "description": "response string",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "response string",
 "updatedAt": "response string"
 }
],
 "createdAt": "response string",
 "updatedAt": "response string"
}
],
"chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
"groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
"createdAt": "response string",
"updatedAt": "response string"
},
"canRead": false,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string"
}
]
}

```

## List Permissions

---

GET

/api/v1/permissions/

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/permissions/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/permissions/", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/permissions/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>get("https://api.maiagent.ai/api/v1/permissions/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

#### Response Schema Example

```
[
 {
 "id": string (uuid)
 "name": string
```

```
"value": string
"description": string
"children": [
 object
]
}
```

#### Response Example Values

```
[
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "value": "response string",
 "description": "response string",
 "children": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response Example Name",
 "description": "Response Example Description"
 }
]
 }
]
```

## List Roles

GET

</api/v1/organizations/{organizationPk}/groups/>

### Parameters

| Parameter | Required | Type | Description |
|-----------|----------|------|-------------|
|-----------|----------|------|-------------|

|                |   |         |                                                |
|----------------|---|---------|------------------------------------------------|
| organizationPk | V | string  | A UUID string identifying this Organization ID |
| page           | X | integer | A page number within the paginated result set. |
| pageSize       | X | integer | Number of results to return per page.          |
| query          | X | string  |                                                |
| search         | X | string  |                                                |

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/?
page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/?
page=1&pageSize=1&query=example&search=example", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/?page=1&pageSize=1&query=example&search=example"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/?
page=1&pageSize=1&query=example&search=example", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

Response Schema Example

```
{
 "count": integer
}
```

```

"next"?: string (uri) // Optional
"previous"?: string (uri) // Optional
"results": [
 {
 "id": string (uuid)
 "name": string
 "type":
 {
 }
 "permissions": [// Returns the group permissions list

```

Processing flow:

1. Split group permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If child permissions are missing their parent, automatically load the missing parent permissions from the database
4. Serialize parent permissions and corresponding child permissions, sorted by order

```

 object
]
 "membersPreview": [
 string
]
 "membersCount": integer // Returns the remaining member count
(total minus 3 displayed), used for frontend +N badge display

```

Different from resource count:

```


 members_count: returns total - 3 (for +N badge), None means total
<= 3
 chatbots_count/knowledge_bases_count/inboxes_count: returns total
count


```

```

 "chatbotsCount": integer
 "chatbotsPreview": [
 string
]
 "knowledgeBasesCount": integer
 "knowledgeBasesPreview": [
 string
]
 "inboxesCount": integer
 "inboxesPreview": [
 string
]

```

```

 "databasesCount": integer
 "databasesPreview": [
 string
]
 "canCreateChatbot"?: boolean // Optional
 "canCreateKnowledgeBase"?: boolean // Optional
 "canCreateInbox"?: boolean // Optional
 "canCreateDatabase"?: boolean // Optional
 "canCreateTool"?: boolean // Optional
 "canCreateAgentUiTool"?: boolean // Optional
 "canCreateSkill"?: boolean // Optional
 "maigptCanDeepResearch"?: boolean // Whether members of this role
may use Deep Research in MaiGPT. Deep Research reaches the public
internet through its own built-in search, so a role that revokes the
official web-search tools needs this switch too. (optional)
 "maigptCanBrowserTool"?: boolean // Whether members of this role
may use the browser built-in tool in MaiGPT. (optional)
 "createdAt": string (timestamp)
}
]
}

```

## Response Example Values

```

{
 "count": 123,
 "next": "http://api.example.org/accounts/?page=4",
 "previous": "http://api.example.org/accounts/?page=2",
 "results": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "type": {},
 "permissions": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response Example Name",
 "description": "Response Example Description"
 }
],
 "membersPreview": [
 "response string"
],
 "membersCount": 456,

```

```
"chatbotsCount": 456,
"chatbotsPreview": [
 "response string"
],
"knowledgeBasesCount": 456,
"knowledgeBasesPreview": [
 "response string"
],
"inboxesCount": 456,
"inboxesPreview": [
 "response string"
],
"databasesCount": 456,
"databasesPreview": [
 "response string"
],
"canCreateChatbot": false,
"canCreateKnowledgeBase": false,
"canCreateInbox": false,
"canCreateDatabase": false,
"canCreateTool": false,
"canCreateAgentUiTool": false,
"canCreateSkill": false,
"maigptCanDeepResearch": false,
"maigptCanBrowserTool": false,
"createdAt": "response string"
}
]
}
```

## Get Role Details

GET

</api/v1/organizations/{organizationPk}/groups/{id}>

## Parameters

| Parameter                   | Required | Type   | Description                                    |
|-----------------------------|----------|--------|------------------------------------------------|
| <code>id</code>             | V        | string | A UUID string identifying this Group.          |
| <code>organizationPk</code> | V        | string | A UUID string identifying this Organization ID |

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

## Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

#### Response Schema Example

```
{
 "id": string (uuid)
```

```
"name": string
"type":
{
}
"permissions": [// Returns the group permissions list
```

Processing flow:

1. Split group permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If child permissions are missing their parent, automatically load the missing parent permissions from the database
4. Serialize parent permissions and corresponding child permissions, sorted by order

```
 object
]
 "membersPreview": [
 string
]
 "membersCount": integer // Returns the remaining member count (total
minus 3 displayed), used for frontend +N badge display
```

Different from resource count:

```

 members_count: returns total - 3 (for +N badge), None means total
<= 3
 chatbots_count/knowledge_bases_count/inboxes_count: returns total
count

```

```
"chatbotsCount": integer
"chatbotsPreview": [
 string
]
"knowledgeBasesCount": integer
"knowledgeBasesPreview": [
 string
]
"inboxesCount": integer
"inboxesPreview": [
 string
]
"databasesCount": integer
"databasesPreview": [
 string
]
"canCreateChatbot"?: boolean // Optional
```

```

"canCreateKnowledgeBase"?: boolean // Optional
"canCreateInbox"?: boolean // Optional
"canCreateDatabase"?: boolean // Optional
"canCreateTool"?: boolean // Optional
"canCreateAgentUiTool"?: boolean // Optional
"canCreateSkill"?: boolean // Optional
"maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (optional)
"maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)
"createdAt": string (timestamp)
}

```

## Response Example Values

```

{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "type": {},
 "permissions": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response Example Name",
 "description": "Response Example Description"
 }
],
 "membersPreview": [
 "response string"
],
 "membersCount": 456,
 "chatbotsCount": 456,
 "chatbotsPreview": [
 "response string"
],
 "knowledgeBasesCount": 456,
 "knowledgeBasesPreview": [
 "response string"
],
 "inboxesCount": 456,
 "inboxesPreview": [
 "response string"
],
}

```

```
"databasesCount": 456,
"databasesPreview": [
 "response string"
],
"canCreateChatbot": false,
"canCreateKnowledgeBase": false,
"canCreateInbox": false,
"canCreateDatabase": false,
"canCreateTool": false,
"canCreateAgentUiTool": false,
"canCreateSkill": false,
"maigptCanDeepResearch": false,
"maigptCanBrowserTool": false,
"createdAt": "response string"
}
```

## Get Role Details

GET

</api/v1/organizations/{organizationPk}/groups/export/>

### Parameters

Parameter	Required	Type	Description
<a href="#">organizationPk</a>	V	string	A UUID string identifying this Organization ID

### Code Examples

## Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/export/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/export/", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/export/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/export/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

**Status Code: 200**

Response Schema Example

```
string (binary)
```



Response Example Values

```
"response string"
```

# Get Role Details

GET

/api/v1/organizations/{organizationPk}/groups/export-template/

## Parameters

Parameter	Required	Type	Description
organizationPk	V	string	A UUID string identifying this Organization ID

## Code Examples

## Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/export-template/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/export-template/", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/export-template/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/export-template/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

**Status Code: 200**

Response Schema Example

```
string (binary)
```

Response Example Values

```
"response string"
```

# Get Role Details

GET

/api/v1/organizations/{organizationPk}/groups/lite/

## Parameters

Parameter	Required	Type	Description
organizationPk	V	string	A UUID string identifying this Organization ID

## Code Examples

## Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/lite/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/lite/", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/lite/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/lite/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

#### Response Schema Example

```
{
 "id": string (uuid)
 "name": string
}
```

```
"type":
{
}
"permissions": [// Returns the group permissions list
```

Processing flow:

1. Split group permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If child permissions are missing their parent, automatically load the missing parent permissions from the database
4. Serialize parent permissions and corresponding child permissions, sorted by order

```
 object
]
"membersPreview": [
 string
]
"membersCount": integer // Returns the remaining member count (total
minus 3 displayed), used for frontend +N badge display
```

Different from resource count:

```

 members_count: returns total - 3 (for +N badge), None means total
<= 3
 chatbots_count/knowledge_bases_count/inboxes_count: returns total
count

```

```
"chatbotsCount": integer
"chatbotsPreview": [
 string
]
"knowledgeBasesCount": integer
"knowledgeBasesPreview": [
 string
]
"inboxesCount": integer
"inboxesPreview": [
 string
]
"databasesCount": integer
"databasesPreview": [
 string
]
"canCreateChatbot"?: boolean // Optional
"canCreateKnowledgeBase"?: boolean // Optional
```

```

"canCreateInbox"?: boolean // Optional
"canCreateDatabase"?: boolean // Optional
"canCreateTool"?: boolean // Optional
"canCreateAgentUiTool"?: boolean // Optional
"canCreateSkill"?: boolean // Optional
"maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (optional)
"maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)
"createdAt": string (timestamp)
}

```

## Response Example Values

```

{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "type": {},
 "permissions": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response Example Name",
 "description": "Response Example Description"
 }
],
 "membersPreview": [
 "response string"
],
 "membersCount": 456,
 "chatbotsCount": 456,
 "chatbotsPreview": [
 "response string"
],
 "knowledgeBasesCount": 456,
 "knowledgeBasesPreview": [
 "response string"
],
 "inboxesCount": 456,
 "inboxesPreview": [
 "response string"
],
 "databasesCount": 456,

```

```
"databasesPreview": [
 "response string"
],
"canCreateChatbot": false,
"canCreateKnowledgeBase": false,
"canCreateInbox": false,
"canCreateDatabase": false,
"canCreateTool": false,
"canCreateAgentUiTool": false,
"canCreateSkill": false,
"maigptCanDeepResearch": false,
"maigptCanBrowserTool": false,
"createdAt": "response string"
}
```

## List Role Members

GET

</api/v1/organizations/{organizationPk}/groups/{groupPk}/group-members/>

### Parameters

Parameter	Required	Type	Description
groupPk	V	string	
organizationPk	V	string	A UUID string identifying this Organization ID
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	

search

X

string

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/?
page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/?
page=1&pageSize=1&query=example&search=example", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-members/?page=1&pageSize=1&query=example&search=example"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/?
page=1&pageSize=1&query=example&search=example", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

Response Schema Example

```

{
 "count": integer
 "next"?: string (uri) // Optional
 "previous"?: string (uri) // Optional
 "results": [
 {
 "id": string (uuid)
 "member": {
 {
 "id": string (uuid)
 "name": string
 "email": string
 "isActive": boolean
 "isOwner": boolean // Check if the member is the organization
owner (via OWNER Group)

```

Uses cache to avoid duplicate queries within the same request  
 Cache lifecycle: valid for the duration of the member instance

Prioritizes annotation results (if available) to avoid duplicate queries

```

 "permissions": [
 object
]
 "groups": [
 object
]
 "creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (Optional)
 "activeQuota": object // Return the most recent quota summary
(active first), via prefetch to avoid N+1.
 "metadata": object // Organization-scoped member attributes,
injected into LLM system prompt
 "createdAt": string (timestamp)
 }
 }
 "createdAt": string (timestamp)
}
]
}

```

Response Example Values

```
{
 "count": 123,
 "next": "http://api.example.org/accounts/?page=4",
 "previous": "http://api.example.org/accounts/?page=2",
 "results": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "member": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "email": "response string",
 "isActive": false,
 "isOwner": false,
 "permissions": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response Example Name",
 "description": "Response Example Description"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response Example Name",
 "description": "Response Example Description"
 }
],
 "creditQuotaRequired": false,
 "activeQuota": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response Example Name",
 "description": "Response Example Description"
 },
 "metadata": null,
 "createdAt": "response string"
 },
 "createdAt": "response string"
 }
]
}
```

## View Specific Role Member

GET

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-members/{id}

### Parameters

Parameter	Required	Type	Description
groupPk	V	string	
id	V	string	A UUID string identifying this Group Member.
organizationPk	V	string	A UUID string identifying this Organization ID

### Code Examples

## Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/",
config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

#### Response Schema Example

```
{
 "id": string (uuid)
```



```

"member": {
 {
 "id": string (uuid)
 "name": string
 "email": string
 "isActive": boolean
 "isOwner": boolean // Check if the member is the organization owner
(via OWNER Group)

```

Uses cache to avoid duplicate queries within the same request  
 Cache lifecycle: valid for the duration of the member instance

Prioritizes annotation results (if available) to avoid duplicate queries

```

"permissions": [
 object
]
"groups": [
 object
]
"creditQuotaRequired"?: boolean // When True, member must have an
active quota to consume credits (Optional)
"activeQuota": object // Return the most recent quota summary (active
first), via prefetch to avoid N+1.
"metadata": object // Organization-scoped member attributes, injected
into LLM system prompt
"createdAt": string (timestamp)
}
}
"createdAt": string (timestamp)
}

```

## Response Example Values

```

{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "member": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "email": "response string",
 "isActive": false,
 "isOwner": false,
 "permissions": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",

```

```
 "name": "Response Example Name",
 "description": "Response Example Description"
 },
],
"groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response Example Name",
 "description": "Response Example Description"
 }
],
"creditQuotaRequired": false,
"activeQuota": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response Example Name",
 "description": "Response Example Description"
},
"metadata": null,
"createdAt": "response string"
},
"createdAt": "response string"
}
```

## List Role's AI Assistants

GET

</api/v1/organizations/{organizationPk}/groups/{groupPk}/group-chatbots/>

### Parameters

Parameter	Required	Type	Description
<code>groupPk</code>	V	string	
<code>organizationPk</code>	V	string	A UUID string identifying this Organization ID

page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
search	X	string	

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/?
page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/?
page=1&pageSize=1&query=example&search=example", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-chatbots/?page=1&pageSize=1&query=example&search=example"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/?
page=1&pageSize=1&query=example&search=example", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

Response Schema Example

```

{
 "count": integer
 "next"?: string (uri) // Optional
 "previous"?: string (uri) // Optional
 "results": [
 {
 "id": string (uuid)
 "group": string (uuid)
 "chatbot":
 {
 "id": string (uuid)
 "name": string // Name of the bot; in Agent mode it has semantic
meaning, in other modes it is used to distinguish different bots
 "largeLanguageModel": string (uuid) // Large language model used
by the bot for generating responses (including AI Gateway models built by
the organization)
 "llm": // ID and name of the bound model, read directly from the
FK; returned even if the model has been disabled by the organization
 {
 "id": string (uuid)
 "name": string
 }
 "embeddingModel"?: string (uuid) // Embedding model for
vectorizing text, optional (Optional)
 "rerankerModel"?: string (uuid) // Reranker model, optional
(Optional)
 "instructions"?: string // Bot role instructions, used to
describe the bot role and behavior (Optional)
 "knowledgeBases"?: [// List of knowledge bases accessible to the
bot (Optional)
 {
 "id": string (uuid)
 "name": string
 }
]
 "databases"?: [// List of SQL databases accessible to the bot
(for Text-to-SQL feature) (Optional)
 {
 "id": string (uuid)
 "name": string
 }
]
 "updatedAt": string (timestamp)
 "organization"?: string (uuid) // Organization the bot belongs

```

```
to; if empty, it is a personal bot (Optional)
 "builtInWorkflow"?: string (uuid) // Built-in workflow for
predefined processing flows (Optional)
 "replyMode"?: // Reply mode: normal reply or streaming reply

* `normal` - Normal
* `template` - Template
* `hybrid` - Hybrid
* `workflow` - Workflow
* `agent` - Agent (Optional)
 {
 }
 "template"?: string // Template used in template mode and hybrid
mode (Optional)
 "unanswerableTemplate"?: string // Template used in template mode
and hybrid mode when unable to answer (Optional)
 "totalWordsCount"?: integer (int64) // Cumulative total word
count used (Optional)
 "outputMode"?: // Output mode: text, table, or custom format

* `text` - Text
* `json_schema` - JSON Schema (Optional)
 {
 }
 "rawOutputFormat"?: object // JSON schema definition for custom
output format (Optional)
 "groups"?: [// List of groups accessible to the bot (Optional)
 {
 "id": string (uuid)
 "name": string
 }
]
 "tools"?: [// List of tools available to the bot (Optional)
 {
 "id": string (uuid)
 "name": string
 "description": string
 "displayName": string
 "toolType": string
 }
]
 "skills"?: [// List of skills associated with the bot (Optional)
 {
 "id": string (uuid)
 "name": string
```



```

 "description": string
 }
]
"connectors"?: [// Connectors associated with this chatbot,
gating per-contact OAuth authorization (Optional)
 {
 "id": string (uuid)
 "name": string
 }
]
"agentMode"?: // Agent mode: normal, SQL, or workflow mode

* `normal` - Normal
* `canvas` - Canvas
* `team` - Team (Optional)
 {
 }
 "numberOfRetrievedChunks"?: integer // Number of retrieved
reference chunks, default is 12, minimum is 1 (Optional)
 "enableEvaluation"?: boolean // Optional
 "enableInlineCitations"?: boolean // Enable inline citation
feature, inserting [1][2] format citation markers in responses (Optional)
 "enableCodeInterpreter"?: boolean // When enabled, provides Code
Interpreter tool that can execute code in an isolated container
(Optional)
 "enableBrowserTool"?: boolean // Enable browser automation tool
for web tasks (Optional)
 "enableImageSearch"?: boolean // Provide the image_search tool
when any attached knowledge base uses a multimodal embedding model.
Disable to remove the tool entirely. (Optional)
 "enableClaudeStreamRetry"?: boolean // When a Claude streaming
call stalls (first-chunk or mid-stream), abort and re-issue the call once
in non-streaming mode. Mid-stream retries send a reset signal so the
client discards any partial output already rendered. Each retry incurs a
second billing for the same prompt. (Optional)
 "customMaxLlmOutputTokens"?: integer // Custom maximum LLM output
token count, minimum is 512 (Optional)
 "voiceConfig"?: { // Optional
 {
 "enabled"?: boolean // Whether this chatbot can be used as a
voice agent. Checked by every conversation platform (WebChat, SIP phone,
admin/marketplace voice test) instead of a per-platform toggle.
(Optional)
 "voiceAgentType"?: string (uuid) // Voice agent mode type
(Optional)

```

```
 "sttProvider"?: string (uuid) // Speech-to-Text service
 provider (Optional)
 "sttConfig"?: object // Actual STT configuration parameters
 (JSON format) (Optional)
 "ttsProvider"?: string (uuid) // Text-to-Speech service
 provider (Optional)
 "ttsConfig"?: object // Actual TTS configuration parameters
 (JSON format) (Optional)
 "realtimeProvider"?: string (uuid) // Realtime end-to-end voice
 model provider (Optional)
 "realtimeConfig"?: object // Actual Realtime configuration
 parameters (JSON format) (Optional)
 "realtimeInstructions"?: string // Prompt dedicated to the
 realtime voice session; falls back to chatbot instructions when empty
 (Optional)
 "turnHandling"?: object // Voice assistant conversation turn
 and interruption control settings (Optional)
 "enableVoiceReply"?: boolean // When disabled, the voice agent
 replies in text only and does not speak (Optional)
 }
}
"thinkingConfig"?: object // Thinking effort settings (JSON
format) (Optional)
"enableToolSearching"?: boolean // Enable dynamic tool searching,
allowing agents to dynamically discover and load tools via
tool_searching_tool (Optional)
"maxAgentIterations"?: integer // Maximum number of iterations
for AgentWorkflow (1-100, default: 20) (Optional)
"agentTimeoutSeconds"?: integer // Timeout in seconds for
AgentWorkflow execution (30-1200, default: 600) (Optional)
"agentTimeoutMessage"?: string // Custom user-facing message
shown when AgentWorkflow times out (max 500 chars). Leave empty to use
the system default. (Optional)
"isMultimodalLlm": boolean // Whether the chatbot LLM supports
multimodal (read-only)
"enableSizeBasedToolResultTruncation"?: boolean // When enabled,
tool results exceeding the character threshold are truncated using
head+tail strategy instead of the legacy SYSTEM_TOOLS whitelist approach.
Reduces memory bloat from large tool outputs (code interpreter bash
stdout, web search results, etc.). (Optional)
"toolResultMaxChars"?: integer // Character threshold for size-
based tool result truncation (500-50000). Only effective when
enable_size_based_tool_result_truncation is True. (Optional)
"enableVectorMemory"?: boolean // When enabled, archived messages
are vectorized and processed by fact extraction for semantic retrieval
```

(existing behavior). When disabled, archived messages are discarded and only the active chat history window is used for multi-turn context.

(Optional)

"enableCrossConversationSearch"?: boolean // When enabled, the conversation history tools also reach this contact's other conversations in the same inbox, within the lookback window. When disabled, they only reach the current conversation. (Optional)

"crossConversationLookbackDays"?: integer // How far back cross-conversation search may reach, measured from each conversation's last message (1-365 days). Leave empty to fall back to 90 days. Only effective when enable\_cross\_conversation\_search is True. (Optional)

"useGatewayFallback"?: boolean // When enabled, use the fallback group instead of the single LLM (Optional)

"gatewayFallbackGroup"?: string (uuid) // Optional

"enableSearchMessagesTool"?: boolean // Allow the AI to search past messages in the current conversation. (Optional)

"enableGetMessageContextTool"?: boolean // Allow the AI to retrieve messages surrounding a specific message for context. (Optional)

"enableReadAttachmentFullTextTool"?: boolean // Allow the AI to read the full text of plain-text attachments. AGENT mode only. (Optional)

"enableParseAttachmentContentTool"?: boolean // Allow the AI to parse PDF, Office, audio, and other attachments. AGENT mode only. (Optional)

"enableListAvailableParsersTool"?: boolean // Allow the AI to list available parsers and their capabilities. AGENT mode only. (Optional)

"enableGetDownloadLinkTool"?: boolean // Allow the AI to generate shareable download links for produced content. AGENT mode only. (Optional)

"isLocked": boolean // Locked instance: config follows the official version; only the dedicated KB can be modified

"sourceBuildInAgent": string (uuid) // Points to the official Build-in Agent blueprint when instantiated by a rental

"canUpdate": boolean // Return whether the current user can update this resource.

"canDelete": boolean // Return whether the current user can delete this resource.

}

"canRead"?: boolean // Optional

"canUpdate"?: boolean // Optional

"canDelete"?: boolean // Optional

"createdAt": string (timestamp)

}

```
]
}
```

## Response Example Values

```
{
 "count": 123,
 "next": "http://api.example.org/accounts/?page=4",
 "previous": "http://api.example.org/accounts/?page=2",
 "results": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "group": "550e8400-e29b-41d4-a716-446655440000",
 "chatbot": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "largeLanguageModel": "550e8400-e29b-41d4-a716-446655440000",
 "llm": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 },
 "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
 "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
 "instructions": "response string",
 "knowledgeBases": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
 "databases": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
 "updatedAt": "response string",
 "organization": "550e8400-e29b-41d4-a716-446655440000",
 "builtInWorkflow": "550e8400-e29b-41d4-a716-446655440000",
 "replyMode": {},
 "template": "response string",
 "unanswerableTemplate": "response string",
 "totalWordsCount": 456,
 "outputMode": {},
 }
]
 }
}
```

```
"rawOutputFormat": null,
"groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
"tools": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "description": "response string",
 "displayName": "response string",
 "toolType": "response string"
 }
],
"skills": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "description": "response string"
 }
],
"connectors": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
"agentMode": {},
"numberOfRetrievedChunks": 456,
"enableEvaluation": false,
"enableInlineCitations": false,
"enableCodeInterpreter": false,
"enableBrowserTool": false,
"enableImageSearch": false,
"enableClaudeStreamRetry": false,
"customMaxLlmOutputTokens": 456,
"voiceConfig": {
 "enabled": false,
 "voiceAgentType": "550e8400-e29b-41d4-a716-446655440000",
 "sttProvider": "550e8400-e29b-41d4-a716-446655440000",
 "sttConfig": null,
 "ttsProvider": "550e8400-e29b-41d4-a716-446655440000",
 "ttsConfig": null,
```

```

 "realtimeProvider": "550e8400-e29b-41d4-a716-446655440000",
 "realtimeConfig": null,
 "realtimeInstructions": "response string",
 "turnHandling": null,
 "enableVoiceReply": false
 },
 "thinkingConfig": null,
 "enableToolSearching": false,
 "maxAgentIterations": 456,
 "agentTimeoutSeconds": 456,
 "agentTimeoutMessage": "response string",
 "isMultimodalLlm": false,
 "enableSizeBasedToolResultTruncation": false,
 "toolResultMaxChars": 456,
 "enableVectorMemory": false,
 "enableCrossConversationSearch": false,
 "crossConversationLookbackDays": 456,
 "useGatewayFallback": false,
 "gatewayFallbackGroup": "550e8400-e29b-41d4-a716-446655440000",
 "enableSearchMessagesTool": false,
 "enableGetMessageContextTool": false,
 "enableReadAttachmentFullTextTool": false,
 "enableParseAttachmentContentTool": false,
 "enableListAvailableParsersTool": false,
 "enableGetDownloadLinkTool": false,
 "isLocked": false,
 "sourceBuildInAgent": "550e8400-e29b-41d4-a716-446655440000",
 "canUpdate": false,
 "canDelete": false
},
"canRead": false,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string"
}
]
}

```

## List Role's Inboxes

---

GET

`/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-inboxes/`

## Parameters

Parameter	Required	Type	Description
<code>groupPk</code>	V	string	
<code>organizationPk</code>	V	string	A UUID string identifying this Organization ID
<code>channelType</code>	X	string	<code>`line`</code> : LINE ; <code>`telegram`</code> : Telegram ; <code>`teams`</code> : Teams ; <code>`web`</code> : Web ; <code>`messenger`</code> : Messenger ; <code>`instagram`</code> : Instagram ; <code>`email`</code> : Email ; <code>`whatsapp`</code> : WhatsApp ; <code>`slack`</code> : Slack;
<code>chatbot</code>	X	string	
<code>isActive</code>	X	boolean	
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.
<code>query</code>	X	string	
<code>search</code>	X	string	

## Code Examples

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/?channelType=email&chatbot=550e8400-e29b-
41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&query=example&search=ex
ample"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/?channelType=email&chatbot=550e8400-e29b-
41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&query=example&search=ex
ample", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-inboxes/?channelType=email&chatbot=550e8400-e29b-41d4-a716-446655440000&isActive=true&page=1&pageSize=1&query=example&search=example"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-inboxes/?channelType=email&chatbot=550e8400-e29b-
41d4-a716-
446655440000&isActive=true&page=1&pageSize=1&query=example&search=ex
ample", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

Response Schema Example

```

{
 "count": integer
 "next"?: string (uri) // Optional
 "previous"?: string (uri) // Optional
 "results": [
 {
 "id": string (uuid)
 "group": string (uuid)
 "inbox":
 {
 "id": string (uuid)
 "channelType": string (enum: line, telegram, teams, web,
messenger, instagram, email, whatsapp, slack) // * `line` - LINE
* `telegram` - Telegram
* `teams` - Teams
* `web` - Web
* `messenger` - Messenger
* `instagram` - Instagram
* `email` - Email
* `whatsapp` - WhatsApp
* `slack` - Slack
 "name": string
 "unreadConversationsCount": integer
 "accessType": string
 }
 "canRead"?: boolean // Optional
 "canUpdate"?: boolean // Optional
 "canDelete"?: boolean // Optional
 "createdAt": string (timestamp)
 }
]
}

```

## Response Example Values

```

{
 "count": 123,
 "next": "http://api.example.org/accounts/?page=4",
 "previous": "http://api.example.org/accounts/?page=2",
 "results": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "group": "550e8400-e29b-41d4-a716-446655440000",

```

```
"inbox": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "channelType": "line",
 "name": "response string",
 "unreadConversationsCount": 456,
 "accessType": "response string"
},
"canRead": false,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string"
}
]
```

## List Role's Knowledge Bases

GET

</api/v1/organizations/{organizationPk}/groups/{groupPk}/group-knowledge-bases/>

### Parameters

Parameter	Required	Type	Description
groupPk	V	string	
organizationPk	V	string	A UUID string identifying this Organization ID
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	

search

X

string

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/?
page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/?
page=1&pageSize=1&query=example&search=example", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-knowledge-bases/?page=1&pageSize=1&query=example&search=example"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/?
page=1&pageSize=1&query=example&search=example", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

Response Schema Example

```

{
 "count": integer
 "next"?: string (uri) // Optional
 "previous"?: string (uri) // Optional
 "results": [
 {
 "id": string (uuid)
 "group": string (uuid)
 "knowledgeBase":
 {
 "id": string (uuid)
 "createdBy":
 {
 "id": string (uuid)
 "name": string
 }
 "organization"?: string (uuid) // Optional
 "embeddingModel": string (uuid)
 "rerankerModel": string (uuid)
 "name": string
 "description"?: string // Optional
 "numberOfRetrievedChunks"?: integer // Number of retrieved
reference chunks, default is 12, minimum is 1 (Optional)
 "enableHyde"?: boolean // Enable HyDE, default is False
(Optional)
 "similarityCutoff"?: number (double) // Similarity cutoff,
default is 0.0, range is 0.0-1.0 (Optional)
 "enableRerank"?: boolean // Whether to enable reranking, default
is True (Optional)
 "enableRrf"?: boolean // Enable Reciprocal Rank Fusion for hybrid
search. Disable this if using a custom Elasticsearch without enterprise
license. (Optional)
 "vectorDatabaseType"?: // Type of vector database: elasticsearch
or opensearch.

* `elasticsearch` - Elasticsearch
* `opensearch` - OpenSearch
* `oracle` - Oracle AI Database 26ai (Optional)
 {
 }
 "vectorDatabaseUrl"?: string // Custom vector database endpoint
URL or Oracle DSN. For Elasticsearch/OpenSearch: URL format (e.g.,
"https://host:9200"). For Oracle: DSN format (e.g.,
"host:1521/service_name"). (Optional)

```

```
 "vectorDatabaseApiKey"?: string // API Key for Elasticsearch. Not
used for OpenSearch. (Optional)
 "opensearchUsername"?: string // Username for OpenSearch Basic
Auth. (Optional)
 "opensearchPassword"?: string // Password for OpenSearch Basic
Auth. (Optional)
 "oracleUser"?: string // Oracle database username. (Optional)
 "oraclePassword"?: string // Oracle database password. (Optional)
 "labels": [
 {
 "id": string (uuid)
 "name": string
 }
]
 "chatbots"?: [// Optional
 {
 "id": string (uuid)
 "name": string
 }
]
 "groups"?: [// List of groups with access to the knowledge base
(Optional)
 {
 "id": string (uuid)
 "name": string
 }
]
 "filesCount": integer // Return count of KnowledgeBaseFile
excluding processed files, conversation attachments, and soft-deleted.
 "vectorStorageSize": integer
 "chunksCount": integer
 "canUpdate": boolean // Return whether the current user can
update this resource.
 "canDelete": boolean // Return whether the current user can
delete this resource.
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
 }
 "canRead"?: boolean // Optional
 "canUpdate"?: boolean // Optional
 "canDelete"?: boolean // Optional
 "createdAt": string (timestamp)
}
```

```
]
}
```

## Response Example Values

```
{
 "count": 123,
 "next": "http://api.example.org/accounts/?page=4",
 "previous": "http://api.example.org/accounts/?page=2",
 "results": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "group": "550e8400-e29b-41d4-a716-446655440000",
 "knowledgeBase": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "createdBy": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 },
 "organization": "550e8400-e29b-41d4-a716-446655440000",
 "embeddingModel": "550e8400-e29b-41d4-a716-446655440000",
 "rerankerModel": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "description": "response string",
 "numberOfRetrievedChunks": 456,
 "enableHyde": false,
 "similarityCutoff": 456,
 "enableRerank": false,
 "enableRrf": false,
 "vectorDatabaseType": {},
 "vectorDatabaseUrl": "response string",
 "vectorDatabaseApiKey": "response string",
 "opensearchUsername": "response string",
 "opensearchPassword": "response string",
 "oracleUser": "response string",
 "oraclePassword": "response string",
 "labels": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
 "chatbots": [
 {
```

```
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
 "filesCount": 456,
 "vectorStorageSize": 456,
 "chunksCount": 456,
 "canUpdate": false,
 "canDelete": false,
 "createdAt": "response string",
 "updatedAt": "response string"
},
"canRead": false,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string"
}
]
```

## List Role's Databases

GET

</api/v1/organizations/{organizationPk}/groups/{groupPk}/group-databases/>

### Parameters

Parameter	Required	Type	Description
<code>groupPk</code>	V	string	

organizationPk	V	string	A UUID string identifying this Organization ID
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	
search	X	string	

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/?
page=1&pageSize=1&query=example&search=example"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/?
page=1&pageSize=1&query=example&search=example", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/?page=1&pageSize=1&query=example&search=example"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>get("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-databases/?
page=1&pageSize=1&query=example&search=example", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

Response Schema Example

```

{
 "count": integer
 "next"?: string (uri) // Optional
 "previous"?: string (uri) // Optional
 "results": [
 {
 "id": string (uuid)
 "group": string (uuid)
 "database":
 {
 "id": string (uuid)
 "name": string // Display name for this database connection
 "description": string // Description of the database and its
purpose
 "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL,
Oracle, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
 {
 }
 "databaseUrl": string // Connection string for external
databases. Not required for MaiAgent type as it uses the system default.
 "includeTables": object // Optional list of tables to include.
Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files.
 "isActive": boolean // When disabled, this database will not be
used in Text-to-SQL queries
 "status":
 {
 }
 "deletedAt": string (timestamp)
 "tables": [
 {
 "id": string (uuid)
 "tableName": string // Name of the table in the database
 "description"?: string // Description of the table and its
data for better Text-to-SQL understanding (Optional)
 "status":
 {
 }
 }
]
 }
 }
]
 }
}

```

```
 "errorMessage": string // Error details if table creation or
deletion failed
 "sourceFile":
 {
 "id": string (uuid)
 "name": string
 "isFromKnowledgeBase": boolean // Determine if the source
file originated from a knowledge base upload.
```

Returns True if the file was uploaded via knowledge base,  
False if uploaded directly to the database.

```
 }
 "columns": [
 {
 "id": string (uuid)
 "columnName": string
 "columnType"?: string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (Optional)
 "description"?: string // Description of the column for
better Text-to-SQL understanding (Optional)
 "isPrimaryKey"?: boolean // Optional
 "isNullable"?: boolean // Optional
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
 }
]
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
}
]
"chatbots": [
{
 "id": string (uuid)
 "name": string
}
]
"groups": [
{
 "id": string (uuid)
 "name": string
}
]
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}
```

```
 "canRead"?: boolean // Optional
 "canUpdate"?: boolean // Optional
 "canDelete"?: boolean // Optional
 "createdAt": string (timestamp)
 }
]
}
```

## Response Example Values

```
{
 "count": 123,
 "next": "http://api.example.org/accounts/?page=4",
 "previous": "http://api.example.org/accounts/?page=2",
 "results": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "group": "550e8400-e29b-41d4-a716-446655440000",
 "database": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "description": "response string",
 "databaseType": {},
 "databaseUrl": "response string",
 "includeTables": null,
 "isActive": false,
 "status": {},
 "deletedAt": "response string",
 "tables": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "tableName": "response string",
 "description": "response string",
 "status": {},
 "errorMessage": "response string",
 "sourceFile": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "isFromKnowledgeBase": false
 },
 "columns": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "response string",
```

```

 "columnType": "response string",
 "description": "response string",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "response string",
 "updatedAt": "response string"
 }
],
 "createdAt": "response string",
 "updatedAt": "response string"
 }
],
"chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
"groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string"
 }
],
"createdAt": "response string",
"updatedAt": "response string"
},
"canRead": false,
"canUpdate": false,
"canDelete": false,
"createdAt": "response string"
}
]
}

```

## Update Role Permissions

PUT

/api/v1/organizations/{organizationPk}/groups/{id}

# Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Group.
organizationPk	V	string	A UUID string identifying this Organization ID

# Request

Request Parameters

Field	Type	Required	Description
name	string	Yes	
organization	string (uuid)	No	
permissions	array[string]	Yes	
canCreateChatbot	boolean	No	
canCreateKnowledgeBase	boolean	No	
canCreateInbox	boolean	No	
canCreateDatabase	boolean	No	
canCreateTool	boolean	No	
canCreateAgentUiTool	boolean	No	
canCreateSkill	boolean	No	

Field	Type	Required	Description
maigptCanDeepResearch	boolean	No	Whether members of this role may use Deep Research in MaiGPT. Deep Research reaches the public internet through its own built-in search, so a role that revokes the official web-search tools needs this...
maigptCanBrowserTool	boolean	No	Whether members of this role may use the browser built-in tool in MaiGPT.

#### Request Schema Example

```
{
 "name": string
 "organization?": string (uuid) // Optional
 "permissions": [
 string (uuid)
]
 "canCreateChatbot?": boolean // Optional
 "canCreateKnowledgeBase?": boolean // Optional
 "canCreateInbox?": boolean // Optional
 "canCreateDatabase?": boolean // Optional
 "canCreateTool?": boolean // Optional
 "canCreateAgentUiTool?": boolean // Optional
 "canCreateSkill?": boolean // Optional
 "maigptCanDeepResearch?": boolean // Whether members of this role may
 use Deep Research in MaiGPT. Deep Research reaches the public internet
 through its own built-in search, so a role that revokes the official web-
 search tools needs this switch too. (optional)
 "maigptCanBrowserTool?": boolean // Whether members of this role may
 use the browser built-in tool in MaiGPT. (optional)
}
```

#### Request Example Values

```
{
 "name": "Example Name",
 "organization": "550e8400-e29b-41d4-a716-446655440000",
```

```
"permissions": [
 "550e8400-e29b-41d4-a716-446655440000"
],
"canCreateChatbot": true,
"canCreateKnowledgeBase": true,
"canCreateInbox": true,
"canCreateDatabase": true,
"canCreateTool": true,
"canCreateAgentUiTool": true,
"maigptCanDeepResearch": true,
"maigptCanBrowserTool": true,
"canCreateSkill": true
}
```

## Code Examples



```
API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "name": "Example Name",
 "organization": "550e8400-e29b-41d4-a716-446655440000",
 "permissions": [
 "550e8400-e29b-41d4-a716-446655440000"
],
 "canCreateChatbot": true,
 "canCreateKnowledgeBase": true,
 "canCreateInbox": true,
 "canCreateDatabase": true,
 "canCreateTool": true,
 "canCreateAgentUiTool": true,
 "maigptCanDeepResearch": true,
 "maigptCanBrowserTool": true,
 "canCreateSkill": true
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "name": "Example Name",
 "organization": "550e8400-e29b-41d4-a716-446655440000",
 "permissions": [
 "550e8400-e29b-41d4-a716-446655440000"
],
 "canCreateChatbot": true,
 "canCreateKnowledgeBase": true,
 "canCreateInbox": true,
 "canCreateDatabase": true,
 "canCreateTool": true,
 "canCreateAgentUiTool": true,
 "maigptCanDeepResearch": true,
 "maigptCanBrowserTool": true,
 "canCreateSkill": true
};

axios.put("https://api.maiaagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

## Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "name": "Example Name",
 "organization": "550e8400-e29b-41d4-a716-446655440000",
 "permissions": [
 "550e8400-e29b-41d4-a716-446655440000"
],
 "canCreateChatbot": true,
 "canCreateKnowledgeBase": true,
 "canCreateInbox": true,
 "canCreateDatabase": true,
 "canCreateTool": true,
 "canCreateAgentUiTool": true,
 "maigptCanDeepResearch": true,
 "maigptCanBrowserTool": true,
 "canCreateSkill": true
}

response = requests.put(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>put("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-
41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "name": "Example Name",
 "organization": "550e8400-e29b-41d4-a716-446655440000",
 "permissions": [
 "550e8400-e29b-41d4-a716-446655440000"
],
 "canCreateChatbot": true,
 "canCreateKnowledgeBase": true,
 "canCreateInbox": true,
 "canCreateDatabase": true,
 "canCreateTool": true,
 "canCreateAgentUiTool": true,
 "maigptCanDeepResearch": true,
 "maigptCanBrowserTool": true,
 "canCreateSkill": true
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>

```

## Response

### Status Code: 200

#### Response Schema Example

```
{
 "id": string (uuid)
 "name": string
 "type":
 {
 }
 "permissions": [// Returns the group permissions list
```

Processing flow:

1. Split group permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If child permissions are missing their parent, automatically load the missing parent permissions from the database
4. Serialize parent permissions and corresponding child permissions, sorted by order

```
 object
]
 "membersPreview": [
 string
]
 "membersCount": integer // Returns the remaining member count (total
minus 3 displayed), used for frontend +N badge display
```

Different from resource count:

```

 members_count: returns total - 3 (for +N badge), None means total
<= 3
 chatbots_count/knowledge_bases_count/inboxes_count: returns total
count

 "chatbotsCount": integer
```

```

"chatbotsPreview": [
 string
]
"knowledgeBasesCount": integer
"knowledgeBasesPreview": [
 string
]
"inboxesCount": integer
"inboxesPreview": [
 string
]
"databasesCount": integer
"databasesPreview": [
 string
]
"canCreateChatbot"?: boolean // Optional
"canCreateKnowledgeBase"?: boolean // Optional
"canCreateInbox"?: boolean // Optional
"canCreateDatabase"?: boolean // Optional
"canCreateTool"?: boolean // Optional
"canCreateAgentUiTool"?: boolean // Optional
"canCreateSkill"?: boolean // Optional
"maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (optional)
"maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)
"createdAt": string (timestamp)
}

```

## Response Example Values

```

{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "type": {},
 "permissions": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response Example Name",
 "description": "Response Example Description"
 }
],

```

```
"membersPreview": [
 "response string"
],
"membersCount": 456,
"chatbotsCount": 456,
"chatbotsPreview": [
 "response string"
],
"knowledgeBasesCount": 456,
"knowledgeBasesPreview": [
 "response string"
],
"inboxesCount": 456,
"inboxesPreview": [
 "response string"
],
"databasesCount": 456,
"databasesPreview": [
 "response string"
],
"canCreateChatbot": false,
"canCreateKnowledgeBase": false,
"canCreateInbox": false,
"canCreateDatabase": false,
"canCreateTool": false,
"canCreateAgentUiTool": false,
"canCreateSkill": false,
"maigptCanDeepResearch": false,
"maigptCanBrowserTool": false,
"createdAt": "response string"
}
```

## Partially Update Role Permissions

PATCH

`/api/v1/organizations/{organizationPk}/groups/{id}`

# Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Group.
organizationPk	V	string	A UUID string identifying this Organization ID

# Request

Request Parameters

Field	Type	Required	Description
name	string	No	
organization	string (uuid)	No	
permissions	array[string]	No	
canCreateChatbot	boolean	No	
canCreateKnowledgeBase	boolean	No	
canCreateInbox	boolean	No	
canCreateDatabase	boolean	No	
canCreateTool	boolean	No	
canCreateAgentUiTool	boolean	No	
canCreateSkill	boolean	No	



Field	Type	Required	Description
maigptCanDeepResearch	boolean	No	Whether members of this role may use Deep Research in MaiGPT. Deep Research reaches the public internet through its own built-in search, so a role that revokes the official web-search tools needs this...
maigptCanBrowserTool	boolean	No	Whether members of this role may use the browser built-in tool in MaiGPT.

#### Request Schema Example

```
{
 "name"?: string // Optional
 "organization"?: string (uuid) // Optional
 "permissions"?: [// Optional
 string (uuid)
]
 "canCreateChatbot"?: boolean // Optional
 "canCreateKnowledgeBase"?: boolean // Optional
 "canCreateInbox"?: boolean // Optional
 "canCreateDatabase"?: boolean // Optional
 "canCreateTool"?: boolean // Optional
 "canCreateAgentUiTool"?: boolean // Optional
 "canCreateSkill"?: boolean // Optional
 "maigptCanDeepResearch"?: boolean // Whether members of this role may
 use Deep Research in MaiGPT. Deep Research reaches the public internet
 through its own built-in search, so a role that revokes the official web-
 search tools needs this switch too. (optional)
 "maigptCanBrowserTool"?: boolean // Whether members of this role may
 use the browser built-in tool in MaiGPT. (optional)
}
```

#### Request Example Values

```
{
 "name": "Example Name",
 "organization": "550e8400-e29b-41d4-a716-446655440000",
```

```
"permissions": [
 "550e8400-e29b-41d4-a716-446655440000"
],
"canCreateChatbot": true,
"canCreateKnowledgeBase": true,
"canCreateInbox": true,
"canCreateDatabase": true,
"canCreateTool": true,
"canCreateAgentUiTool": true,
"maigptCanDeepResearch": true,
"maigptCanBrowserTool": true,
"canCreateSkill": true
}
```

## Code Examples

```
API call example (Shell)
curl -X PATCH
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "name": "Example Name",
 "organization": "550e8400-e29b-41d4-a716-446655440000",
 "permissions": [
 "550e8400-e29b-41d4-a716-446655440000"
],
 "canCreateChatbot": true,
 "canCreateKnowledgeBase": true,
 "canCreateInbox": true,
 "canCreateDatabase": true,
 "canCreateTool": true,
 "canCreateAgentUiTool": true,
 "maigptCanDeepResearch": true,
 "maigptCanBrowserTool": true,
 "canCreateSkill": true
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "name": "Example Name",
 "organization": "550e8400-e29b-41d4-a716-446655440000",
 "permissions": [
 "550e8400-e29b-41d4-a716-446655440000"
],
 "canCreateChatbot": true,
 "canCreateKnowledgeBase": true,
 "canCreateInbox": true,
 "canCreateDatabase": true,
 "canCreateTool": true,
 "canCreateAgentUiTool": true,
 "maigptCanDeepResearch": true,
 "maigptCanBrowserTool": true,
 "canCreateSkill": true
};

axios.patch("https://api.maiaagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

## Python

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "name": "Example Name",
 "organization": "550e8400-e29b-41d4-a716-446655440000",
 "permissions": [
 "550e8400-e29b-41d4-a716-446655440000"
],
 "canCreateChatbot": true,
 "canCreateKnowledgeBase": true,
 "canCreateInbox": true,
 "canCreateDatabase": true,
 "canCreateTool": true,
 "canCreateAgentUiTool": true,
 "maigptCanDeepResearch": true,
 "maigptCanBrowserTool": true,
 "canCreateSkill": true
}

response = requests.patch(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->
 patch("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "name": "Example Name",
 "organization": "550e8400-e29b-41d4-a716-446655440000",
 "permissions": [
 "550e8400-e29b-41d4-a716-446655440000"
],
 "canCreateChatbot": true,
 "canCreateKnowledgeBase": true,
 "canCreateInbox": true,
 "canCreateDatabase": true,
 "canCreateTool": true,
 "canCreateAgentUiTool": true,
 "maigptCanDeepResearch": true,
 "maigptCanBrowserTool": true,
 "canCreateSkill": true
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>

```

## Response

### Status Code: 200

#### Response Schema Example

```
{
 "id": string (uuid)
 "name": string
 "type":
 {
 }
 "permissions": [// Returns the group permissions list
```

Processing flow:

1. Split group permissions into parent and child permissions
2. Group child permissions by parent permission ID
3. If child permissions are missing their parent, automatically load the missing parent permissions from the database
4. Serialize parent permissions and corresponding child permissions, sorted by order

```
 object
]
 "membersPreview": [
 string
]
 "membersCount": integer // Returns the remaining member count (total
minus 3 displayed), used for frontend +N badge display
```

Different from resource count:

```

 members_count: returns total - 3 (for +N badge), None means total
<= 3
 chatbots_count/knowledge_bases_count/inboxes_count: returns total
count

 "chatbotsCount": integer
```

```

"chatbotsPreview": [
 string
]
"knowledgeBasesCount": integer
"knowledgeBasesPreview": [
 string
]
"inboxesCount": integer
"inboxesPreview": [
 string
]
"databasesCount": integer
"databasesPreview": [
 string
]
"canCreateChatbot"?: boolean // Optional
"canCreateKnowledgeBase"?: boolean // Optional
"canCreateInbox"?: boolean // Optional
"canCreateDatabase"?: boolean // Optional
"canCreateTool"?: boolean // Optional
"canCreateAgentUiTool"?: boolean // Optional
"canCreateSkill"?: boolean // Optional
"maigptCanDeepResearch"?: boolean // Whether members of this role may
use Deep Research in MaiGPT. Deep Research reaches the public internet
through its own built-in search, so a role that revokes the official web-
search tools needs this switch too. (optional)
"maigptCanBrowserTool"?: boolean // Whether members of this role may
use the browser built-in tool in MaiGPT. (optional)
"createdAt": string (timestamp)
}

```

## Response Example Values

```

{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "response string",
 "type": {},
 "permissions": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response Example Name",
 "description": "Response Example Description"
 }
],

```



```
"membersPreview": [
 "response string"
],
"membersCount": 456,
"chatbotsCount": 456,
"chatbotsPreview": [
 "response string"
],
"knowledgeBasesCount": 456,
"knowledgeBasesPreview": [
 "response string"
],
"inboxesCount": 456,
"inboxesPreview": [
 "response string"
],
"databasesCount": 456,
"databasesPreview": [
 "response string"
],
"canCreateChatbot": false,
"canCreateKnowledgeBase": false,
"canCreateInbox": false,
"canCreateDatabase": false,
"canCreateTool": false,
"canCreateAgentUiTool": false,
"canCreateSkill": false,
"maigptCanDeepResearch": false,
"maigptCanBrowserTool": false,
"createdAt": "response string"
}
```

## Delete Role

**DELETE**

`/api/v1/organizations/{organizationPk}/groups/{id}`

## Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Group.
<code>organizationPk</code>	V	string	A UUID string identifying this Organization ID

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}

?>
```

## Response

Status Code	Description
204	No response body

# Remove Role Member

DELETE

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-members/{id}

## Parameters

Parameter	Required	Type	Description
groupPk	V	string	
id	V	string	A UUID string identifying this Group Member.
organizationPk	V	string	A UUID string identifying this Organization ID

## Code Examples

```
API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
members/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/",
data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-members/550e8400-e29b-41d4-a716-446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}

?>
```

## Response

Status Code	Description
204	No response body

# Remove AI Assistant from Role

DELETE

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-chatbots/{id}

## Parameters

Parameter	Required	Type	Description
groupPk	V	string	
id	V	string	A UUID string identifying this Group Chatbot.
organizationPk	V	string	A UUID string identifying this Organization ID

## Code Examples

```
API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
chatbots/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-chatbots/550e8400-e29b-41d4-a716-446655440000/",
data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-chatbots/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-chatbots/550e8400-e29b-41d4-a716-446655440000/",
 [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

Status Code	Description
204	No response body

# Remove Inbox from Role

DELETE

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-inboxes/{id}

## Parameters

Parameter	Required	Type	Description
groupPk	V	string	
id	V	string	A UUID string identifying this Group Inbox.
organizationPk	V	string	A UUID string identifying this Organization ID

## Code Examples



```
API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
inboxes/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-inboxes/550e8400-e29b-41d4-a716-446655440000/",
data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-inboxes/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-inboxes/550e8400-e29b-41d4-a716-446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}

?>
```

## Response

Status Code	Description
204	No response body

# Remove Knowledge Base from Role

DELETE

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-knowledge-bases/{id}

## Parameters

Parameter	Required	Type	Description
groupPk	V	string	
id	V	string	A UUID string identifying this Group KnowledgeBase.
organizationPk	V	string	A UUID string identifying this Organization ID

## Code Examples

```
API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-
e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-
446655440000/group-knowledge-bases/550e8400-e29b-41d4-a716-
446655440000/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-knowledge-bases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-knowledge-bases/550e8400-e29b-41d4-a716-446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

Status Code	Description
204	No response body

# Remove Database from Role

DELETE

/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-databases/{id}

## Parameters

Parameter	Required	Type	Description
groupPk	V	string	
id	V	string	A UUID string identifying this Group Database.
organizationPk	V	string	A UUID string identifying this Organization ID

## Code Examples

```
API call example (Shell)
curl -X DELETE
"https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-
a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-
databases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/550e8400-e29b-41d4-a716-446655440000/",
data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->delete("https://api.maiagent.ai/api/v1/organizations/550e8400-e29b-41d4-a716-446655440000/groups/550e8400-e29b-41d4-a716-446655440000/group-databases/550e8400-e29b-41d4-a716-446655440000/",
 [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

Status Code	Description
204	No response body



# API call example (Shell)

...



# 目錄

---

1. Create SQL Database Connection
2. List SQL Databases
3. Get Specific SQL Database
4. Update SQL Database
5. Partially Update SQL Database
6. Delete SQL Database
7. Test Database Connection
8. Upload Files to Database
9. Batch Delete SQL Databases
10. List Tables
11. Create Table
12. Get Specific Table
13. Update Table
14. Partially Update Table
15. Delete Table
16. Batch Delete Tables
17. List Columns
18. Get Specific Column
19. Update Column
20. Partially Update Column
21. Test Database Connection (General)
22. List Database Connection String Examples
23. Get Specific Connection String Example
24. List AI Assistant Database Configurations
25. Add AI Assistant Database Configuration
26. Get Specific Database Configuration
27. Update Database Configuration
28. Partially Update Database Configuration
29. Delete Database Configuration

## API call example (Shell)

---

# Create SQL Database Connection

POST

/api/v1/sql-databases/

## Request

Request Parameters

Field	Type	Required	Description
name	string	Yes	Display name for this database connection
description	string	No	Description of the database and its purpose
databaseType	object	Yes	Type of database: PostgreSQL, MySQL, MSSQL, Oracle, SynxDB, or MaiAgent <code>postgresql</code> : PostgreSQL ; <code>mysql</code> : MySQL ; <code>maiagent</code> : MaiAgent ; <code>oracle</code> : Oracle ; <code>mssql</code> : MSSQL ; <code>synxdb</code> : SynxDB;
databaseUrl	string	No	Connection string for external databases. Not required for MaiAgent type as it uses the system default.
includeTables	object	No	Optional list of tables to include. Format: <code>{"tables": ["table1", "table2"]}</code> . For MaiAgent type, this is auto-populated from uploaded files.
isActive	boolean	No	When disabled, this database will not be used in Text-to-SQL queries
chatbots	array[IdName]	No	
groups	array[IdName]	No	

Request Schema Example

```

{
 "name": string // Display name for this database connection
 "description"?: string // Description of the database and its purpose
(optional)
 "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
 {
 }
 "databaseUrl"?: string // Connection string for external databases. Not
required for MaiAgent type as it uses the system default. (optional)
 "includeTables"?: object // Optional list of tables to include. Format:
{"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
populated from uploaded files. (optional)
 "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (optional)
 "chatbots"?: [// optional
 {
 "id": string (uuid)
 }
]
 "groups"?: [// optional
 {
 "id": string (uuid)
 }
]
}

```

#### Request Example Values

```

{
 "name": "Example Name",
 "description": "Example String",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,

```

```
"chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
"groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
}
```

## Code Examples

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-databases/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "name": "Example Name",
 "description": "Example String",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "name": "Example Name",
 "description": "Example String",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
};

axios.post("https://api.maiagent.ai/api/v1/sql-databases/", data,
config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
```

```
console.error(error.response?.data || error.message);
});
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "name": "Example Name",
 "description": "Example String",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->post("https://api.maiagent.ai/api/v1/sql-
databases/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "name": "Example Name",
 "description": "Example String",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 201

#### Response Schema Example

```
{
 "id": string (uuid)
 "name": string // Display name for this database connection
 "description"?: string // Description of the database and its purpose
 (optional)
 "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
 SynxDB, or MaiAgent

 * `postgresql` - PostgreSQL
 * `mysql` - MySQL
 * `maiagent` - MaiAgent
 * `oracle` - Oracle
 * `mssql` - MSSQL
 * `synxdb` - SynxDB
 {
 }
 "databaseUrlMasked": string
 "includeTables"?: object // Optional list of tables to include. Format:
 {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
 populated from uploaded files. (optional)
 "isActive"?: boolean // When disabled, this database will not be used
 in Text-to-SQL queries (optional)
 "status":
 {
 }
 "deletedAt": string (timestamp)
 "tables": [
 {
 "id": string (uuid)
 "tableName": string // Name of the table in the database
 "sheetName": string // Name of the source worksheet/sheet this
```

table was created from. Empty for tables created before this field was added.

"description"?: string // Description of the table and its data for better Text-to-SQL understanding (optional)

"status":

{

}

"errorMessage": string // Error details if table creation or deletion failed

"sourceFile":

{

"id": string (uuid)

"name": string

"isFromKnowledgeBase": boolean // Determine if the source file originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,

False if uploaded directly to the database.

}

"columns": [

{

"id": string (uuid)

"columnName": string

"columnType"?: string // Data type of the column (e.g., VARCHAR, INTEGER, TIMESTAMP) (optional)

"description"?: string // Description of the column for better Text-to-SQL understanding (optional)

"isPrimaryKey"?: boolean // optional

"isNullable"?: boolean // optional

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

]

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

]

"chatbots": [

{

"id": string (uuid)

"name": string

}

]

"groups": [

{

```

 "id": string (uuid)
 "name": string
 }
]
"createdBy":
{
 "id": string (uuid)
 "name": string
}
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

## Response Example Values

```

{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "description": "Response String",
 "databaseType": {},
 "databaseUrlMasked": "Response String",
 "includeTables": null,
 "isActive": false,
 "status": {},
 "deletedAt": "Response String",
 "tables": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "tableName": "Response String",
 "sheetName": "Response String",
 "description": "Response String",
 "status": {},
 "errorMessage": "Response String",
 "sourceFile": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "isFromKnowledgeBase": false
 },
 "columns": [
 {

```

```

 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "Response String",
 "columnType": "Response String",
 "description": "Response String",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
 }
],
 "createdAt": "Response String",
 "updatedAt": "Response String"
}
],
"chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
 }
],
"groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
 }
],
"createdBy": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
},
"canUpdate": false,
"canDelete": false,
"createdAt": "Response String",
"updatedAt": "Response String"
}

```

## List SQL Databases

---

GET

/api/v1/sql-databases/

## Parameters

Parameter	Required	Type	Description
databaseType	X	string	Type of database: PostgreSQL, MySQL, MSSQL, Oracle, SynxDB, or MaiAgent `postgresql`: PostgreSQL; `mysql`: MySQL; `maiagent`: MaiAgent; `oracle`: Oracle; `mssql`: MSSQL; `synxdb`: SynxDB;
isActive	X	boolean	
isExternal	X	boolean	
page	X	integer	A page number within the paginated result set.
pageSize	X	integer	Number of results to return per page.
query	X	string	

## Code Examples

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-databases/?
databaseType=maiagent&isActive=true&isExternal=true&page=1&pageSize=
1&query=example"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/sql-databases/?
databaseType=maiagent&isActive=true&isExternal=true&page=1&pageSize=
1&query=example", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/?\ndatabaseType=maiagent&isActive=true&isExternal=true&page=1&pageSize=\n1&query=example"\nheaders = {\n "Authorization": "Api-Key YOUR_API_KEY"\n}\n\nresponse = requests.get(url, headers=headers)\ntry:\n print("Response received successfully:")\n print(response.json())\nexcept Exception as e:\n print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->get("https://api.maiagent.ai/api/v1/sql-
databases/?
databaseType=maiagent&isActive=true&isExternal=true&page=1&pageSize=
1&query=example", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

Response Schema Example

```
{
 "count": integer
}
```

```

"next"?: string (uri) // optional
"previous"?: string (uri) // optional
"results": [
 {
 "id": string (uuid)
 "name": string // Display name for this database connection
 "description"?: string // Description of the database and its
purpose (optional)
 "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL,
Oracle, SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
 {
 }
 "databaseUrlMasked": string
 "includeTables"?: object // Optional list of tables to include.
Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files. (optional)
 "isActive"?: boolean // When disabled, this database will not be
used in Text-to-SQL queries (optional)
 "status":
 {
 }
 "deletedAt": string (timestamp)
 "tables": [
 {
 "id": string (uuid)
 "tableName": string // Name of the table in the database
 }
]
 "tablesCount": integer
 "chatbots": [
 {
 "id": string (uuid)
 "name": string
 }
]
 "groups": [
 {
 "id": string (uuid)

```

```

 "name": string
 }
]
 "createdBy":
 {
 "id": string (uuid)
 "name": string
 }
 "canUpdate": boolean // Return whether the current user can update
this resource.
 "canDelete": boolean // Return whether the current user can delete
this resource.
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
 }
]
}

```

## Response Example Values

```

{
 "count": 123,
 "next": "http://api.example.org/accounts/?page=4",
 "previous": "http://api.example.org/accounts/?page=2",
 "results": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "description": "Response String",
 "databaseType": {},
 "databaseUrlMasked": "Response String",
 "includeTables": null,
 "isActive": false,
 "status": {},
 "deletedAt": "Response String",
 "tables": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "tableName": "Response String"
 }
],
 "tablesCount": 456,
 "chatbots": [
 {

```

```
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
 }
],
 "createdBy": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
 },
 "canUpdate": false,
 "canDelete": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
}
]
```

## Get Specific SQL Database

GET

`/api/v1/sql-databases/{id}`

### Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this SQL Database.

### Code Examples

## Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->get("https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

#### Response Schema Example

```
{
 "id": string (uuid)
 "name": string // Display name for this database connection
 "description?": string // Description of the database and its purpose
```

```

(optional)
 "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
SynxDB, or MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
 {
 }
 "databaseUrlMasked": string
 "includeTables"?: object // Optional list of tables to include. Format:
{"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
populated from uploaded files. (optional)
 "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (optional)
 "status":
 {
 }
 "deletedAt": string (timestamp)
 "tables": [
 {
 "id": string (uuid)
 "tableName": string // Name of the table in the database
 "sheetName": string // Name of the source worksheet/sheet this
table was created from. Empty for tables created before this field was
added.
 "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
 "status":
 {
 }
 "errorMessage": string // Error details if table creation or
deletion failed
 "sourceFile":
 {
 "id": string (uuid)
 "name": string
 "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

```

Returns True if the file was uploaded via knowledge base,  
False if uploaded directly to the database.

```

 }
 "columns": [
 {
 "id": string (uuid)
 "columnName": string
 "columnType"?: string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (optional)
 "description"?: string // Description of the column for better
Text-to-SQL understanding (optional)
 "isPrimaryKey"?: boolean // optional
 "isNullable"?: boolean // optional
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
 }
]
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
 }
]
"chatbots": [
 {
 "id": string (uuid)
 "name": string
 }
]
"groups": [
 {
 "id": string (uuid)
 "name": string
 }
]
"createdBy":
{
 "id": string (uuid)
 "name": string
}
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

Response Example Values

```
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "description": "Response String",
 "databaseType": {},
 "databaseUrlMasked": "Response String",
 "includeTables": null,
 "isActive": false,
 "status": {},
 "deletedAt": "Response String",
 "tables": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "tableName": "Response String",
 "sheetName": "Response String",
 "description": "Response String",
 "status": {},
 "errorMessage": "Response String",
 "sourceFile": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "isFromKnowledgeBase": false
 },
 "columns": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "Response String",
 "columnType": "Response String",
 "description": "Response String",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
 }
],
 "createdAt": "Response String",
 "updatedAt": "Response String"
 }
],
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
 }
]
}
```

```
],
"groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
 }
],
"createdBy": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
},
"canUpdate": false,
"canDelete": false,
"createdAt": "Response String",
"updatedAt": "Response String"
}
```

## Update SQL Database

PUT

`/api/v1/sql-databases/{id}`

### Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this SQL Database.

### Request

Request Parameters

Field	Type	Required	Description
name	string	Yes	Display name for this database connection
description	string	No	Description of the database and its purpose
databaseUrl	string	No	Connection string for external databases. Not required for MaiAgent type as it uses the system default.
includeTables	object	No	Optional list of tables to include. Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-populated from uploaded files.
isActive	boolean	No	When disabled, this database will not be used in Text-to-SQL queries
chatbots	array[IdName]	No	
groups	array[IdName]	No	

#### Request Schema Example

```
{
 "name": string // Display name for this database connection
 "description"?: string // Description of the database and its purpose
 (optional)
 "databaseUrl"?: string // Connection string for external databases. Not
 required for MaiAgent type as it uses the system default. (optional)
 "includeTables"?: object // Optional list of tables to include. Format:
 {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
 populated from uploaded files. (optional)
 "isActive"?: boolean // When disabled, this database will not be used
 in Text-to-SQL queries (optional)
 "chatbots"?: [// optional
 {
 "id": string (uuid)
 }
]
 "groups"?: [// optional
 {
 "id": string (uuid)
 }
]
}
```

```
]
}
```

#### Request Example Values

```
{
 "name": "Example Name",
 "description": "Example String",
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
}
```

## Code Examples

```
API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "name": "Example Name",
 "description": "Example String",
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "name": "Example Name",
 "description": "Example String",
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
};

axios.put("https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "name": "Example Name",
 "description": "Example String",
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
}

response = requests.put(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->put("https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "name": "Example Name",
 "description": "Example String",
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

#### Response Schema Example

```
{
 "id": string (uuid)
 "name": string // Display name for this database connection
 "description"?: string // Description of the database and its purpose
 (optional)
 "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
 SynxDB, or MaiAgent

 * `postgresql` - PostgreSQL
 * `mysql` - MySQL
 * `maiagent` - MaiAgent
 * `oracle` - Oracle
 * `mssql` - MSSQL
 * `synxdb` - SynxDB
 {
 }
 "databaseUrlMasked": string
 "includeTables"?: object // Optional list of tables to include. Format:
 {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
 populated from uploaded files. (optional)
 "isActive"?: boolean // When disabled, this database will not be used
 in Text-to-SQL queries (optional)
 "status":
 {
 }
 "deletedAt": string (timestamp)
 "tables": [
 {
 "id": string (uuid)
 "tableName": string // Name of the table in the database
 "sheetName": string // Name of the source worksheet/sheet this
```

table was created from. Empty for tables created before this field was added.

"description"?: string // Description of the table and its data for better Text-to-SQL understanding (optional)

"status":

{

}

"errorMessage": string // Error details if table creation or deletion failed

"sourceFile":

{

"id": string (uuid)

"name": string

"isFromKnowledgeBase": boolean // Determine if the source file originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,

False if uploaded directly to the database.

}

"columns": [

{

"id": string (uuid)

"columnName": string

"columnType"?: string // Data type of the column (e.g., VARCHAR, INTEGER, TIMESTAMP) (optional)

"description"?: string // Description of the column for better Text-to-SQL understanding (optional)

"isPrimaryKey"?: boolean // optional

"isNullable"?: boolean // optional

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

]

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

]

"chatbots": [

{

"id": string (uuid)

"name": string

}

]

"groups": [

{

```

 "id": string (uuid)
 "name": string
 }
]
"createdBy":
{
 "id": string (uuid)
 "name": string
}
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

## Response Example Values

```

{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "description": "Response String",
 "databaseType": {},
 "databaseUrlMasked": "Response String",
 "includeTables": null,
 "isActive": false,
 "status": {},
 "deletedAt": "Response String",
 "tables": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "tableName": "Response String",
 "sheetName": "Response String",
 "description": "Response String",
 "status": {},
 "errorMessage": "Response String",
 "sourceFile": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "isFromKnowledgeBase": false
 },
 "columns": [
 {

```

```

 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "Response String",
 "columnType": "Response String",
 "description": "Response String",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
 }
],
 "createdAt": "Response String",
 "updatedAt": "Response String"
}
],
"chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
 }
],
"groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
 }
],
"createdBy": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
},
"canUpdate": false,
"canDelete": false,
"createdAt": "Response String",
"updatedAt": "Response String"
}

```

## Partially Update SQL Database

---



**PATCH****/api/v1/sql-databases/{id}**

## Parameters

Parameter	Required	Type	Description
<b>id</b>	V	string	A UUID string identifying this SQL Database.

## Request

Request Parameters

Field	Type	Required	Description
name	string	No	Display name for this database connection
description	string	No	Description of the database and its purpose
databaseUrl	string	No	Connection string for external databases. Not required for MaiAgent type as it uses the system default.
includeTables	object	No	Optional list of tables to include. Format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-populated from uploaded files.
isActive	boolean	No	When disabled, this database will not be used in Text-to-SQL queries
chatbots	array[IdName]	No	
groups	array[IdName]	No	

Request Schema Example

```

{
 "name"?: string // Display name for this database connection (optional)
 "description"?: string // Description of the database and its purpose
(optional)
 "databaseUrl"?: string // Connection string for external databases. Not
required for MaiAgent type as it uses the system default. (optional)
 "includeTables"?: object // Optional list of tables to include. Format:
{"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
populated from uploaded files. (optional)
 "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (optional)
 "chatbots"?: [// optional
 {
 "id": string (uuid)
 }
]
 "groups"?: [// optional
 {
 "id": string (uuid)
 }
]
}

```

#### Request Example Values

```

{
 "name": "Example Name",
 "description": "Example String",
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
}

```

```
]
}
```

## Code Examples

```
API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "name": "Example Name",
 "description": "Example String",
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "name": "Example Name",
 "description": "Example String",
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
};

axios.patch("https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "name": "Example Name",
 "description": "Example String",
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
}

response = requests.patch(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```

<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->patch("https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "name": "Example Name",
 "description": "Example String",
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true,
 "chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
],
 "groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000"
 }
]
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>

```



## Response

### Status Code: 200

#### Response Schema Example

```
{
 "id": string (uuid)
 "name": string // Display name for this database connection
 "description?": string // Description of the database and its purpose
 (optional)
 "databaseType": // Type of database: PostgreSQL, MySQL, MSSQL, Oracle,
 SynxDB, or MaiAgent

 * `postgresql` - PostgreSQL
 * `mysql` - MySQL
 * `maiagent` - MaiAgent
 * `oracle` - Oracle
 * `mssql` - MSSQL
 * `synxdb` - SynxDB
 {
 }
 "databaseUrlMasked": string
 "includeTables?": object // Optional list of tables to include. Format:
 {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-
 populated from uploaded files. (optional)
 "isActive?": boolean // When disabled, this database will not be used
 in Text-to-SQL queries (optional)
 "status":
 {
 }
 "deletedAt": string (timestamp)
 "tables": [
 {
 "id": string (uuid)
 "tableName": string // Name of the table in the database
 "sheetName": string // Name of the source worksheet/sheet this
```

table was created from. Empty for tables created before this field was added.

"description"?: string // Description of the table and its data for better Text-to-SQL understanding (optional)

"status":

{

}

"errorMessage": string // Error details if table creation or deletion failed

"sourceFile":

{

"id": string (uuid)

"name": string

"isFromKnowledgeBase": boolean // Determine if the source file originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,

False if uploaded directly to the database.

}

"columns": [

{

"id": string (uuid)

"columnName": string

"columnType"?: string // Data type of the column (e.g., VARCHAR, INTEGER, TIMESTAMP) (optional)

"description"?: string // Description of the column for better Text-to-SQL understanding (optional)

"isPrimaryKey"?: boolean // optional

"isNullable"?: boolean // optional

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

]

"createdAt": string (timestamp)

"updatedAt": string (timestamp)

}

]

"chatbots": [

{

"id": string (uuid)

"name": string

}

]

"groups": [

{

```

 "id": string (uuid)
 "name": string
 }
]
"createdBy":
{
 "id": string (uuid)
 "name": string
}
"canUpdate": boolean // Return whether the current user can update this
resource.
"canDelete": boolean // Return whether the current user can delete this
resource.
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

## Response Example Values

```

{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "description": "Response String",
 "databaseType": {},
 "databaseUrlMasked": "Response String",
 "includeTables": null,
 "isActive": false,
 "status": {},
 "deletedAt": "Response String",
 "tables": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "tableName": "Response String",
 "sheetName": "Response String",
 "description": "Response String",
 "status": {},
 "errorMessage": "Response String",
 "sourceFile": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "isFromKnowledgeBase": false
 },
 "columns": [
 {

```

```

 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "Response String",
 "columnType": "Response String",
 "description": "Response String",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
 }
],
 "createdAt": "Response String",
 "updatedAt": "Response String"
}
],
"chatbots": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
 }
],
"groups": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
 }
],
"createdBy": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String"
},
"canUpdate": false,
"canDelete": false,
"createdAt": "Response String",
"updatedAt": "Response String"
}

```

## Delete SQL Database

---

**DELETE****/api/v1/sql-databases/{id}**

## Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this SQL Database.

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->delete("https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

Status Code	Description
204	No response body

## Test Database Connection

---



**POST****/api/v1/sql-databases/{id}/test-connection/**

## Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this SQL Database.

## Request

Request Parameters

Field	Type	Required	Description
databaseType	string (enum: postgresql, mysql, maiagent, oracle, mssql, synxdb)	Yes	<code>postgresql</code> : PostgreSQL ; <code>mysql</code> : MySQL ; <code>maiagent</code> : MaiAgent ; <code>oracle</code> : Oracle ; <code>mssql</code> : MSSQL;
databaseUrl	string	Yes	

Request Schema Example

```
{
 "databaseType": string (enum: postgresql, mysql, maiagent, oracle,
mssql, synxdb) // * `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
 "databaseUrl": string
}
```

Request Example Values

```
{
 "databaseType": "postgresql",
 "databaseUrl": "Example String"
}
```

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/test-connection/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "databaseType": "postgresql",
 "databaseUrl": "Example String"
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "databaseType": "postgresql",
 "databaseUrl": "Example String"
};

axios.post("https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/test-connection/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/test-connection/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "databaseType": "postgresql",
 "databaseUrl": "Example String"
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->post("https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/test-connection/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "databaseType": "postgresql",
 "databaseUrl": "Example String"
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

**Status Code: 200**

Response Schema Example

```
{
 "success"?: boolean // optional
 "message"?: string // optional
}
```

#### Response Example Values

```
{
 "success": false,
 "message": "Response String"
}
```

## Upload Files to Database

POST

/api/v1/sql-databases/{id}/upload-files/

### Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this SQL Database.

### Request

#### Request Parameters

Field	Type	Required	Description
files	array[object]	Yes	

## Request Schema Example

```
{
 "files": [
 {
 "file": string // File path from presigned URL upload (e.g.,
media/chatbotfiles/xxx/file.csv)
 "filename": string // Original filename
 "size": integer // File size in bytes
 }
]
}
```

## Request Example Values

```
{
 "files": [
 {
 "file": "media/chatbotfiles/xxx/data.csv",
 "filename": "data.csv",
 "size": 1024
 },
 {
 "file": "media/chatbotfiles/xxx/report.xlsx",
 "filename": "report.xlsx",
 "size": 2048
 }
]
}
```

## Code Examples

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/upload-files/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "files": [
 {
 "file": "media/chatbotfiles/xxx/data.csv",
 "filename": "data.csv",
 "size": 1024
 },
 {
 "file": "media/chatbotfiles/xxx/report.xlsx",
 "filename": "report.xlsx",
 "size": 2048
 }
]
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "files": [
 {
 "file": "media/chatbotfiles/xxx/data.csv",
 "filename": "data.csv",
 "size": 1024
 },
 {
 "file": "media/chatbotfiles/xxx/report.xlsx",
 "filename": "report.xlsx",
 "size": 2048
 }
]
};

axios.post("https://api.maiagent.ai/api/v1/sql-databases/550e8400-
e29b-41d4-a716-446655440000/upload-files/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/550e8400-e29b-41d4-a716-446655440000/upload-files/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "files": [
 {
 "file": "media/chatbotfiles/xxx/data.csv",
 "filename": "data.csv",
 "size": 1024
 },
 {
 "file": "media/chatbotfiles/xxx/report.xlsx",
 "filename": "report.xlsx",
 "size": 2048
 }
]
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->post("https://api.maiagent.ai/api/v1/sql-
databases/550e8400-e29b-41d4-a716-446655440000/upload-files/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "files": [
 {
 "file": "media/chatbotfiles/xxx/data.csv",
 "filename": "data.csv",
 "size": 1024
 },
 {
 "file": "media/chatbotfiles/xxx/report.xlsx",
 "filename": "report.xlsx",
 "size": 2048
 }
]
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 201

#### Response Schema Example

```
{
 "message"?: string // optional
 "filesCount"?: integer // optional
 "fileIds"?: [// optional
 string (uuid)
]
}
```

#### Response Example Values

```
{
 "message": "Response String",
 "filesCount": 456,
 "fileIds": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}
```

## Batch Delete SQL Databases

POST

</api/v1/sql-databases/batch-delete/>

## Request

Request Parameters

Field	Type	Required	Description
ids	array[string]	Yes	

Request Schema Example

```
{
 "ids": [
 string (uuid)
]
}
```

Request Example Values

```
{
 "ids": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}
```

Code Examples

## Shell/Bash

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-databases/batch-
delete/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "ids": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "ids": [
 "550e8400-e29b-41d4-a716-446655440000"
]
};

axios.post("https://api.maiagent.ai/api/v1/sql-databases/batch-delete/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/batch-delete/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "ids": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->post("https://api.maiagent.ai/api/v1/sql-
databases/batch-delete/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "ids": [
 "550e8400-e29b-41d4-a716-446655440000"
]
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

Status Code	Description
-------------	-------------

204

No response body

## List Tables

GET

`/api/v1/sql-databases/{sqlDatabasePk}/tables/`

### Parameters

Parameter	Required	Type	Description
<code>sqlDatabasePk</code>	V	string	
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.
<code>query</code>	X	string	
<code>status</code>	X	string	`pending`: Pending; `processing`: Processing; `success`: Success; `failed`: Failed; `dropping`: Dropping; `drop_failed`: Drop Failed;

### Code Examples

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/?
page=1&pageSize=1&query=example&status=drop_failed"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/?
page=1&pageSize=1&query=example&status=drop_failed", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/?
page=1&pageSize=1&query=example&status=drop_failed"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/?
page=1&pageSize=1&query=example&status=drop_failed", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

#### Response Schema Example

```
{
 "count": integer
 "next"?: string (uri) // optional
}
```

```

"previous"?: string (uri) // optional
"results": [
 {
 "id": string (uuid)
 "tableName": string // Name of the table in the database
 "sheetName": string // Name of the source worksheet/sheet this
table was created from. Empty for tables created before this field was
added.
 "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
 "status":
 {
 }
 "errorMessage": string // Error details if table creation or
deletion failed
 "sourceFile":
 {
 "id": string (uuid)
 "name": string
 "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
 }
 "columns": [
 {
 "id": string (uuid)
 "columnName": string
 "columnType"?: string // Data type of the column (e.g.,
VARCHAR, INTEGER, TIMESTAMP) (optional)
 "description"?: string // Description of the column for better
Text-to-SQL understanding (optional)
 "isPrimaryKey"?: boolean // optional
 "isNullable"?: boolean // optional
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
 }
]
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
 }
]
}

```

## Response Example Values

```
{
 "count": 123,
 "next": "http://api.example.org/accounts/?page=4",
 "previous": "http://api.example.org/accounts/?page=2",
 "results": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "tableName": "Response String",
 "sheetName": "Response String",
 "description": "Response String",
 "status": {},
 "errorMessage": "Response String",
 "sourceFile": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "isFromKnowledgeBase": false
 },
 "columns": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "Response String",
 "columnType": "Response String",
 "description": "Response String",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
 }
],
 "createdAt": "Response String",
 "updatedAt": "Response String"
 }
]
}
```

## Create Table

---



**POST****/api/v1/sql-databases/{sqlDatabasePk}/tables/**

## Parameters

Parameter	Required	Type	Description
sqlDatabasePk	V	string	

## Request

Request Parameters

Field	Type	Required	Description
tableName	string	Yes	Name of the table in the database
description	string	No	Description of the table and its data for better Text-to-SQL understanding

Request Schema Example

```
{
 "tableName": string // Name of the table in the database
 "sheetName": string // Name of the source worksheet/sheet this table
was created from. Empty for tables created before this field was added.
 "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
}
```

Request Example Values

```
{
 "tableName": "Example String",
```

```
"description": "Example String"
}
```

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "tableName": "Example String",
 "description": "Example String"
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "tableName": "Example String",
 "description": "Example String"
};

axios.post("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "tableName": "Example String",
 "description": "Example String"
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->post("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "tableName": "Example String",
 "description": "Example String"
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 201

Response Schema Example

```

{
 "id": string (uuid)
 "tableName": string // Name of the table in the database
 "sheetName": string // Name of the source worksheet/sheet this table
was created from. Empty for tables created before this field was added.
 "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
 "status":
 {
 }
 "errorMessage": string // Error details if table creation or deletion
failed
 "sourceFile":
 {
 "id": string (uuid)
 "name": string
 "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
 }
 "columns": [
 {
 "id": string (uuid)
 "columnName": string
 "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
 "description"?: string // Description of the column for better
Text-to-SQL understanding (optional)
 "isPrimaryKey"?: boolean // optional
 "isNullable"?: boolean // optional
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
 }
]
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
}

```

Response Example Values

```
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "tableName": "Response String",
 "sheetName": "Response String",
 "description": "Response String",
 "status": {},
 "errorMessage": "Response String",
 "sourceFile": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "isFromKnowledgeBase": false
 },
 "columns": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "Response String",
 "columnType": "Response String",
 "description": "Response String",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
 }
],
 "createdAt": "Response String",
 "updatedAt": "Response String"
}
```

## Get Specific Table

GET

</api/v1/sql-databases/{sqlDatabasePk}/tables/{id}>

## Parameters

Parameter	Required	Type	Description
-----------	----------	------	-------------

id	V	string	A UUID string identifying this Database Table.
sqlDatabasePk	V	string	

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

#### Response Schema Example

```
{
 "id": string (uuid)
 "tableName": string // Name of the table in the database
```

```

"sheetName": string // Name of the source worksheet/sheet this table
was created from. Empty for tables created before this field was added.
"description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
"status":
{
}
"errorMessage": string // Error details if table creation or deletion
failed
"sourceFile":
{
 "id": string (uuid)
 "name": string
 "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
}
"columns": [
 {
 "id": string (uuid)
 "columnName": string
 "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
 "description"?: string // Description of the column for better
Text-to-SQL understanding (optional)
 "isPrimaryKey"?: boolean // optional
 "isNullable"?: boolean // optional
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
 }
]
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

#### Response Example Values

```

{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "tableName": "Response String",
 "sheetName": "Response String",
 "description": "Response String",

```

```
"status": {},
"errorMessage": "Response String",
"sourceFile": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "isFromKnowledgeBase": false
},
"columns": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "Response String",
 "columnType": "Response String",
 "description": "Response String",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
 }
],
"createdAt": "Response String",
"updatedAt": "Response String"
}
```

## Update Table

PUT

/api/v1/sql-databases/{sqlDatabasePk}/tables/{id}

### Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Database Table.
sqlDatabasePk	V	string	

# Request

## Request Parameters

Field	Type	Required	Description
tableName	string	Yes	Name of the table in the database
description	string	No	Description of the table and its data for better Text-to-SQL understanding

## Request Schema Example

```
{
 "tableName": string // Name of the table in the database
 "sheetName": string // Name of the source worksheet/sheet this table
was created from. Empty for tables created before this field was added.
 "description?": string // Description of the table and its data for
better Text-to-SQL understanding (optional)
}
```

## Request Example Values

```
{
 "tableName": "Example String",
 "description": "Example String"
}
```

# Code Examples

```
API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "tableName": "Example String",
 "description": "Example String"
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "tableName": "Example String",
 "description": "Example String"
};

axios.put("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-446655440000/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "tableName": "Example String",
 "description": "Example String"
}

response = requests.put(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->put("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "tableName": "Example String",
 "description": "Example String"
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

**Status Code: 200**

Response Schema Example

```

{
 "id": string (uuid)
 "tableName": string // Name of the table in the database
 "sheetName": string // Name of the source worksheet/sheet this table
was created from. Empty for tables created before this field was added.
 "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
 "status":
 {
 }
 "errorMessage": string // Error details if table creation or deletion
failed
 "sourceFile":
 {
 "id": string (uuid)
 "name": string
 "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
 }
 "columns": [
 {
 "id": string (uuid)
 "columnName": string
 "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
 "description"?: string // Description of the column for better
Text-to-SQL understanding (optional)
 "isPrimaryKey"?: boolean // optional
 "isNullable"?: boolean // optional
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
 }
]
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
}

```

Response Example Values

```
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "tableName": "Response String",
 "sheetName": "Response String",
 "description": "Response String",
 "status": {},
 "errorMessage": "Response String",
 "sourceFile": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "isFromKnowledgeBase": false
 },
 "columns": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "Response String",
 "columnType": "Response String",
 "description": "Response String",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
 }
],
 "createdAt": "Response String",
 "updatedAt": "Response String"
}
```

## Partially Update Table

PATCH

</api/v1/sql-databases/{sqlDatabasePk}/tables/{id}>

### Parameters

Parameter	Required	Type	Description
-----------	----------	------	-------------

id	V	string	A UUID string identifying this Database Table.
sqlDatabasePk	V	string	

## Request

### Request Parameters

Field	Type	Required	Description
tableName	string	No	Name of the table in the database
description	string	No	Description of the table and its data for better Text-to-SQL understanding

### Request Schema Example

```
{
 "tableName?": string // Name of the table in the database (optional)
 "description?": string // Description of the table and its data for
better Text-to-SQL understanding (optional)
}
```

### Request Example Values

```
{
 "tableName": "Example String",
 "description": "Example String"
}
```

## Code Examples

```
API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "tableName": "Example String",
 "description": "Example String"
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "tableName": "Example String",
 "description": "Example String"
};

axios.patch("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "tableName": "Example String",
 "description": "Example String"
}

response = requests.patch(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->patch("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "tableName": "Example String",
 "description": "Example String"
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

**Status Code: 200**

Response Schema Example

```

{
 "id": string (uuid)
 "tableName": string // Name of the table in the database
 "sheetName": string // Name of the source worksheet/sheet this table
was created from. Empty for tables created before this field was added.
 "description"?: string // Description of the table and its data for
better Text-to-SQL understanding (optional)
 "status":
 {
 }
 "errorMessage": string // Error details if table creation or deletion
failed
 "sourceFile":
 {
 "id": string (uuid)
 "name": string
 "isFromKnowledgeBase": boolean // Determine if the source file
originated from a knowledge base upload.

Returns True if the file was uploaded via knowledge base,
False if uploaded directly to the database.
 }
 "columns": [
 {
 "id": string (uuid)
 "columnName": string
 "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
 "description"?: string // Description of the column for better
Text-to-SQL understanding (optional)
 "isPrimaryKey"?: boolean // optional
 "isNullable"?: boolean // optional
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
 }
]
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
}

```

Response Example Values

```
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "tableName": "Response String",
 "sheetName": "Response String",
 "description": "Response String",
 "status": {},
 "errorMessage": "Response String",
 "sourceFile": {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "isFromKnowledgeBase": false
 },
 "columns": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "Response String",
 "columnType": "Response String",
 "description": "Response String",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
 }
],
 "createdAt": "Response String",
 "updatedAt": "Response String"
}
```

## Delete Table

**DELETE**

**/api/v1/sql-databases/{sqlDatabasePk}/tables/{id}**

## Parameters

Parameter	Required	Type	Description
-----------	----------	------	-------------

id	V	string	A UUID string identifying this Database Table.
sqlDatabasePk	V	string	

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X DELETE "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

// Request body (payload)
const data = null;

axios.delete("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->delete("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/550e8400-e29b-41d4-a716-
446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

Status Code	Description
204	No response body

# Batch Delete Tables

POST

/api/v1/sql-databases/{sqlDatabasePk}/tables/batch-delete/

## Parameters

Parameter	Required	Type	Description
sqlDatabasePk	V	string	

## Request

Request Parameters

Field	Type	Required	Description
ids	array[string]	Yes	

Request Schema Example

```
{
 "ids": [
 string (uuid)
]
}
```

Request Example Values

```
{
 "ids": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}
```



```
]
}
```

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/batch-delete/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "ids": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "ids": [
 "550e8400-e29b-41d4-a716-446655440000"
]
};

axios.post("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/batch-delete/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/batch-delete/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "ids": [
 "550e8400-e29b-41d4-a716-446655440000"
]
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->post("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/batch-delete/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "ids": [
 "550e8400-e29b-41d4-a716-446655440000"
]
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

Status Code	Description
-------------	-------------

204

No response body

## List Columns

GET

/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/

### Parameters

Parameter	Required	Type	Description
sqlDatabasePk	V	string	
tablePk	V	string	

### Code Examples

#### Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

#### Response Schema Example

```
[
 {
 "id": string (uuid)
 "columnName": string
```



```
"columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
"description"?: string // Description of the column for better Text-
to-SQL understanding (optional)
"isPrimaryKey"?: boolean // optional
"isNullable"?: boolean // optional
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}
]
```

#### Response Example Values

```
[
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "Response String",
 "columnType": "Response String",
 "description": "Response String",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
 }
]
```

## Get Specific Column

GET

</api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/{id}>

### Parameters

Parameter	Required	Type	Description
-----------	----------	------	-------------

id	V	string	A UUID string identifying this Database Table Column.
sqlDatabasePk	V	string	
tablePk	V	string	

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->get("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

#### Response Schema Example

```
{
 "id": string (uuid)
 "columnName": string
```

```
"columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
"description"?: string // Description of the column for better Text-to-
SQL understanding (optional)
"isPrimaryKey"?: boolean // optional
"isNullable"?: boolean // optional
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}
```

#### Response Example Values

```
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "Response String",
 "columnType": "Response String",
 "description": "Response String",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
}
```

## Update Column

PUT

</api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/{id}>

### Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Database Table Column.

sqlDatabasePk V string

tablePk V string

## Request

### Request Parameters

Field	Type	Required	Description
columnName	string	Yes	
columnType	string	No	Data type of the column (e.g., VARCHAR, INTEGER, TIMESTAMP)
description	string	No	Description of the column for better Text-to-SQL understanding
isPrimaryKey	boolean	No	
isNullable	boolean	No	

### Request Schema Example

```
{
 "columnName": string
 "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
 "description"?: string // Description of the column for better Text-to-
SQL understanding (optional)
 "isPrimaryKey"?: boolean // optional
 "isNullable"?: boolean // optional
}
```

### Request Example Values

```
{
 "columnName": "Example String",
 "columnType": "Example String",
 "description": "Example String",
```

```
"isPrimaryKey": true,
"isNullable": true
}
```

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X PUT "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "columnName": "Example String",
 "columnType": "Example String",
 "description": "Example String",
 "isPrimaryKey": true,
 "isNullable": true
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "columnName": "Example String",
 "columnType": "Example String",
 "description": "Example String",
 "isPrimaryKey": true,
 "isNullable": true
};

axios.put("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-41d4-a716-446655440000/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "columnName": "Example String",
 "columnType": "Example String",
 "description": "Example String",
 "isPrimaryKey": true,
 "isNullable": true
}

response = requests.put(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->put("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "columnName": "Example String",
 "columnType": "Example String",
 "description": "Example String",
 "isPrimaryKey": true,
 "isNullable": true
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

## Status Code: 200

### Response Schema Example

```
{
 "id": string (uuid)
 "columnName": string
 "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
 "description"?: string // Description of the column for better Text-to-
SQL understanding (optional)
 "isPrimaryKey"?: boolean // optional
 "isNullable"?: boolean // optional
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
}
```

### Response Example Values

```
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "Response String",
 "columnType": "Response String",
 "description": "Response String",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
}
```

## Partially Update Column

PATCH

</api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/{id}>

# Parameters

Parameter	Required	Type	Description
id	V	string	A UUID string identifying this Database Table Column.
sqlDatabasePk	V	string	
tablePk	V	string	

# Request

Request Parameters

Field	Type	Required	Description
columnName	string	No	
columnType	string	No	Data type of the column (e.g., VARCHAR, INTEGER, TIMESTAMP)
description	string	No	Description of the column for better Text-to-SQL understanding
isPrimaryKey	boolean	No	
isNullable	boolean	No	

Request Schema Example

```
{
 "columnName"?: string // optional
 "columnType"?: string // Data type of the column (e.g., VARCHAR,
 INTEGER, TIMESTAMP) (optional)
 "description"?: string // Description of the column for better Text-to-
 SQL understanding (optional)
 "isPrimaryKey"?: boolean // optional
```

```
"isNullable"?: boolean // optional
}
```

#### Request Example Values

```
{
 "columnName": "Example String",
 "columnType": "Example String",
 "description": "Example String",
 "isPrimaryKey": true,
 "isNullable": true
}
```

## Code Examples

```
API call example (Shell)
curl -X PATCH "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "columnName": "Example String",
 "columnType": "Example String",
 "description": "Example String",
 "isPrimaryKey": true,
 "isNullable": true
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "columnName": "Example String",
 "columnType": "Example String",
 "description": "Example String",
 "isPrimaryKey": true,
 "isNullable": true
};

axios.patch("https://api.maiagent.ai/api/v1/sql-databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-41d4-a716-446655440000/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```



```
import requests

url = "https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "columnName": "Example String",
 "columnType": "Example String",
 "description": "Example String",
 "isPrimaryKey": true,
 "isNullable": true
}

response = requests.patch(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->patch("https://api.maiagent.ai/api/v1/sql-
databases/{sqlDatabasePk}/tables/{tablePk}/columns/550e8400-e29b-
41d4-a716-446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "columnName": "Example String",
 "columnType": "Example String",
 "description": "Example String",
 "isPrimaryKey": true,
 "isNullable": true
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

## Status Code: 200

### Response Schema Example

```
{
 "id": string (uuid)
 "columnName": string
 "columnType"?: string // Data type of the column (e.g., VARCHAR,
INTEGER, TIMESTAMP) (optional)
 "description"?: string // Description of the column for better Text-to-
SQL understanding (optional)
 "isPrimaryKey"?: boolean // optional
 "isNullable"?: boolean // optional
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
}
```

### Response Example Values

```
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "columnName": "Response String",
 "columnType": "Response String",
 "description": "Response String",
 "isPrimaryKey": false,
 "isNullable": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
}
```

## Test Database Connection (General)

POST

</api/v1/database-connection/test/>

## Request

### Request Parameters

Field	Type	Required	Description
databaseType	object	Yes	Database type (excluding MaiAgent internal type) <code>postgresql</code> : PostgreSQL ; <code>mysql</code> : MySQL ; <code>oracle</code> : Oracle ; <code>mssql</code> : MSSQL;
databaseUrl	string	Yes	Database connection string

### Request Schema Example

```
{
 "databaseType": // Database type (excluding MaiAgent internal type)

 * `postgresql` - PostgreSQL
 * `mysql` - MySQL
 * `oracle` - Oracle
 * `mssql` - MSSQL
 * `synxdb` - SynxDB
 {
 }
 "databaseUrl": string // Database connection string
}
```

### Request Example Values

```
{
 "database_type": "postgresql",
 "database_url": "postgresql://user:password@localhost:5432/dbname"
}
```

## Code Examples

## Shell/Bash

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/database-connection/test/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "database_type": "postgresql",
 "database_url":
"postgresql://user:password@localhost:5432/dbname"
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "database_type": "postgresql",
 "database_url":
"postgresql://user:password@localhost:5432/dbname"
};

axios.post("https://api.maiagent.ai/api/v1/database-connection/test/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/database-connection/test/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "database_type": "postgresql",
 "database_url":
"postgresql://user:password@localhost:5432/dbname"
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>post("https://api.maiagent.ai/api/v1/database-connection/test/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "database_type": "postgresql",
 "database_url":
"postgresql://user:password@localhost:5432/dbname"
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

**Status Code: 200**

Response Schema Example



```
{
 "status": // Connection status

 * `success` - success
 * `failed` - failed
 {
 }
 "responseTime"?: number (double) // Response time (seconds) (optional)
 "message"?: string // Success message (optional)
 "errorMessage"?: string // Error message (optional)
}
```

#### Response Example Values

```
{
 "status": "success",
 "response_time": 0.123,
 "message": "Successfully connected to PostgreSQL database"
}
```

#### Status Code: 400

#### Response Schema Example

```
{
 "detail"?: string // optional
}
```

#### Response Example Values

```
{
 "status": "success",
 "response_time": 0.123,
 "message": "Successfully connected to PostgreSQL database"
}
```

## List Database Connection String Examples

GET

[/api/v1/database-examples/](#)

### Code Examples

#### Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiaagent.ai/api/v1/database-examples/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/database-examples/",
config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/database-examples/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>get("https://api.maiagent.ai/api/v1/database-examples/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

#### Response Schema Example

```
[
 {
 "databaseType"?: // Database type options
```

```
* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL (optional)
 {
 }
 "databaseTypeExample": string // e.g.:
postgresql://db_user:db_password@db_ip:db_port/db_name
}
```

#### Response Example Values

```
[
 {
 "databaseType": {},
 "databaseTypeExample": "Response String"
 }
]
```

#### Status Code: 401

#### Response Schema Example

```
{
 "detail"? : string // optional
}
```

#### Response Example Values

```
{
 "detail": "Response String"
}
```

## Get Specific Connection String Example

GET

`/api/v1/database-examples/{id}`

### Parameters

Parameter	Required	Type	Description
<code>id</code>	V	string	A UUID string identifying this Database Connection Example.

### Code Examples

#### Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/database-examples/550e8400-e29b-41d4-a716-446655440000/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/database-examples/550e8400-e29b-41d4-a716-446655440000/", config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```



## Python

```
import requests

url = "https://api.maiagent.ai/api/v1/database-examples/550e8400-
e29b-41d4-a716-446655440000/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->get("https://api.maiagent.ai/api/v1/database-examples/550e8400-e29b-41d4-a716-446655440000/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

**Status Code: 200**

Response Schema Example

```
{
 "databaseType?": // Database type options
```

```
* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL (optional)
{
}
"databaseTypeExample": string // e.g.:
postgresql://db_user:db_password@db_ip:db_port/db_name
}
```

#### Response Example Values

```
{
 "databaseType": {},
 "databaseTypeExample": "Response String"
}
```

## List AI Assistant Database Configurations

GET

</api/v1/chatbots/{chatbotPk}/databases/>

### Parameters

Parameter	Required	Type	Description
<code>chatbotPk</code>	V	string	A UUID string identifying this Chatbot ID
<code>page</code>	X	integer	A page number within the paginated result set.
<code>pageSize</code>	X	integer	Number of results to return per page.

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get(" config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->get(" [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

Response Schema Example

```
{
 "count": integer
 "next"?: string (uri) // optional
 "previous"?: string (uri) // optional
 "results": [
```

```

{
 "id": string (uuid)
 "name": string // Display name, e.g.: Main Business DB, Analytics
DB
 "databaseType": // Database type options: MySQL, PostgreSQL,
MSSQL, Oracle, MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
 {
 }
 "databaseUrl": string // Database connection string
 "databaseUrlMasked": string
 "includeTables?": object // Specify the list of tables to query,
format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files and does not require manual
configuration (optional)
 "isActive?": boolean // When disabled, this database will not be
used in Text-to-SQL queries (optional)
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
 }
]
}

```

## Response Example Values

```

{
 "count": 123,
 "next": "http://api.example.org/accounts/?page=4",
 "previous": "http://api.example.org/accounts/?page=2",
 "results": [
 {
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "databaseType": {},
 "databaseUrlMasked": "Response String",
 "includeTables": null,
 "isActive": false,
 "createdAt": "Response String",

```



```
 "updatedAt": "Response String"
 }
]
}
```

## Add AI Assistant Database Configuration

POST

/api/v1/chatbots/{chatbotPk}/databases/

### Parameters

Parameter	Required	Type	Description
chatbotPk	V	string	A UUID string identifying this Chatbot ID

### Request

Request Parameters

Field	Type	Required	Description
name	string	Yes	Display name, e.g.: Main Business DB, Analytics DB
databaseType	object	Yes	Database type options: MySQL, PostgreSQL, MSSQL, Oracle, MaiAgent <b>postgresql</b> : PostgreSQL ; <b>mysql</b> : MySQL ; <b>maiagent</b> : MaiAgent ; <b>oracle</b> : Oracle ; <b>mssql</b> : MSSQL;
databaseUrl	string	Yes	Database connection string
includeTables	object	No	Specify the list of tables to query, format: {"tables": ["table1", "table2"]}. For MaiAgent

Field	Type	Required	Description
			type, this is auto-populated from uploaded files and does not require manual configuration
isActive	boolean	No	When disabled, this database will not be used in Text-to-SQL queries

#### Request Schema Example

```
{
 "name": string // Display name, e.g.: Main Business DB, Analytics DB
 "databaseType": // Database type options: MySQL, PostgreSQL, MSSQL,
 Oracle, MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
 {
 }
 "databaseUrl": string // Database connection string
 "databaseUrlMasked": string
 "includeTables"?: object // Specify the list of tables to query,
format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files and does not require manual
configuration (optional)
 "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (optional)
}
```

#### Request Example Values

```
{
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
```

```
"isActive": true
}
```

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X POST "

 -H "Authorization: Api-Key YOUR_API_KEY"

 -H "Content-Type: application/json"

 -d '{
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true
 }'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true
};

axios.post(" data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->post(" [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

**Status Code: 201**

## Response Schema Example

```
{
 "id": string (uuid)
 "name": string // Display name, e.g.: Main Business DB, Analytics DB
 "databaseType": // Database type options: MySQL, PostgreSQL, MSSQL,
Oracle, MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
 {
 }
 "databaseUrl": string // Database connection string
 "databaseUrlMasked": string
 "includeTables"?: object // Specify the list of tables to query,
format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files and does not require manual
configuration (optional)
 "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (optional)
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
}
```

## Response Example Values

```
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "databaseType": {},
 "databaseUrlMasked": "Response String",
 "includeTables": null,
 "isActive": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
}
```

## Get Specific Database Configuration

GET

/api/v1/chatbots/{chatbotPk}/databases/{id}

### Parameters

Parameter	Required	Type	Description
chatbotPk	V	string	A UUID string identifying this Chatbot ID
id	V	string	A UUID string identifying this Chatbot Database Configuration.

### Code Examples

#### Shell/Bash

```
API call example (Shell)
curl -X GET "

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```



```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get(" config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->get(" [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

Response Schema Example

```
{
 "id": string (uuid)
 "name": string // Display name, e.g.: Main Business DB, Analytics DB
 "databaseType": // Database type options: MySQL, PostgreSQL, MSSQL,
 Oracle, MaiAgent
```

```

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
{
}
"databaseUrl": string // Database connection string
"databaseUrlMasked": string
"includeTables"?: object // Specify the list of tables to query,
format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files and does not require manual
configuration (optional)
"isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (optional)
"createdAt": string (timestamp)
"updatedAt": string (timestamp)
}

```

#### Response Example Values

```

{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "databaseType": {},
 "databaseUrlMasked": "Response String",
 "includeTables": null,
 "isActive": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
}

```

## Update Database Configuration

PUT

/api/v1/chatbots/{chatbotPk}/databases/{id}

## Parameters

Parameter	Required	Type	Description
<code>chatbotPk</code>	V	string	A UUID string identifying this Chatbot ID
<code>id</code>	V	string	A UUID string identifying this Chatbot Database Configuration.

## Request

### Request Parameters

Field	Type	Required	Description
name	string	Yes	Display name, e.g.: Main Business DB, Analytics DB
databaseType	object	Yes	Database type options: MySQL, PostgreSQL, MSSQL, Oracle, MaiAgent <code>postgresql</code> : PostgreSQL ; <code>mysql</code> : MySQL ; <code>maiagent</code> : MaiAgent ; <code>oracle</code> : Oracle ; <code>mssql</code> : MSSQL;
databaseUrl	string	Yes	Database connection string
includeTables	object	No	Specify the list of tables to query, format: <code>{"tables": ["table1", "table2"]}</code> . For MaiAgent type, this is auto-populated from uploaded files and does not require manual configuration
isActive	boolean	No	When disabled, this database will not be used in Text-to-SQL queries

### Request Schema Example

```
{
 "name": string // Display name, e.g.: Main Business DB, Analytics DB
 "databaseType": // Database type options: MySQL, PostgreSQL, MSSQL,
 Oracle, MaiAgent
```

```
* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
{
}
"databaseUrl": string // Database connection string
"databaseUrlMasked": string
"includeTables"?: object // Specify the list of tables to query,
format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files and does not require manual
configuration (optional)
"isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (optional)
}
```

#### Request Example Values

```
{
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true
}
```

## Code Examples

```
API call example (Shell)
curl -X PUT "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true
};

axios.put(" data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```



```
import requests

url = "
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true
}

response = requests.put(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->put(" [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

**Status Code: 200**

## Response Schema Example

```
{
 "id": string (uuid)
 "name": string // Display name, e.g.: Main Business DB, Analytics DB
 "databaseType": // Database type options: MySQL, PostgreSQL, MSSQL,
 Oracle, MaiAgent

 * `postgresql` - PostgreSQL
 * `mysql` - MySQL
 * `maiagent` - MaiAgent
 * `oracle` - Oracle
 * `mssql` - MSSQL
 * `synxdb` - SynxDB
 {
 }
 "databaseUrl": string // Database connection string
 "databaseUrlMasked": string
 "includeTables"?: object // Specify the list of tables to query,
 format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
 auto-populated from uploaded files and does not require manual
 configuration (optional)
 "isActive"?: boolean // When disabled, this database will not be used
 in Text-to-SQL queries (optional)
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
}
```

## Response Example Values

```
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "databaseType": {},
 "databaseUrlMasked": "Response String",
 "includeTables": null,
 "isActive": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
}
```

# Partially Update Database Configuration

PATCH

/api/v1/chatbots/{chatbotPk}/databases/{id}

## Parameters

Parameter	Required	Type	Description
chatbotPk	V	string	A UUID string identifying this Chatbot ID
id	V	string	A UUID string identifying this Chatbot Database Configuration.

## Request

Request Parameters

Field	Type	Required	Description
name	string	No	Display name, e.g.: Main Business DB, Analytics DB
databaseType	object	No	Database type options: MySQL, PostgreSQL, MSSQL, Oracle, MaiAgent <b>postgresql</b> : PostgreSQL ; <b>mysql</b> : MySQL ; <b>maiaagent</b> : MaiAgent ; <b>oracle</b> : Oracle ; <b>mssql</b> : MSSQL;
databaseUrl	string	No	Database connection string
includeTables	object	No	Specify the list of tables to query, format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is auto-populated from uploaded files and does not require manual configuration
isActive	boolean	No	When disabled, this database will not be used in Text-to-SQL queries

## Request Schema Example

```
{
 "name"?: string // Display name, e.g.: Main Business DB, Analytics DB
(optional)
 "databaseType"?: // Database type options: MySQL, PostgreSQL, MSSQL,
Oracle, MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL (optional)
 {
 }
 "databaseUrl"?: string // Database connection string (optional)
 "includeTables"?: object // Specify the list of tables to query,
format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files and does not require manual
configuration (optional)
 "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (optional)
}
```

## Request Example Values

```
{
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true
}
```

## Code Examples

```
API call example (Shell)
curl -X PATCH "

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true
}'

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true
};

axios.patch(" data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true
}

response = requests.patch(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->patch(" [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "name": "Example Name",
 "databaseType": {},
 "databaseUrl": "Example String",
 "includeTables": null,
 "isActive": true
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}

?>
```

## Response

**Status Code: 200**

## Response Schema Example

```
{
 "id": string (uuid)
 "name": string // Display name, e.g.: Main Business DB, Analytics DB
 "databaseType": // Database type options: MySQL, PostgreSQL, MSSQL,
Oracle, MaiAgent

* `postgresql` - PostgreSQL
* `mysql` - MySQL
* `maiagent` - MaiAgent
* `oracle` - Oracle
* `mssql` - MSSQL
* `synxdb` - SynxDB
 {
 }
 "databaseUrl": string // Database connection string
 "databaseUrlMasked": string
 "includeTables"?: object // Specify the list of tables to query,
format: {"tables": ["table1", "table2"]}. For MaiAgent type, this is
auto-populated from uploaded files and does not require manual
configuration (optional)
 "isActive"?: boolean // When disabled, this database will not be used
in Text-to-SQL queries (optional)
 "createdAt": string (timestamp)
 "updatedAt": string (timestamp)
}
```

## Response Example Values

```
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "name": "Response String",
 "databaseType": {},
 "databaseUrlMasked": "Response String",
 "includeTables": null,
 "isActive": false,
 "createdAt": "Response String",
 "updatedAt": "Response String"
}
```

## Delete Database Configuration

**DELETE**`/api/v1/chatbots/{chatbotPk}/databases/{id}`

### Parameters

Parameter	Required	Type	Description
<code>chatbotPk</code>	V	string	A UUID string identifying this Chatbot ID
<code>id</code>	V	string	A UUID string identifying this Chatbot Database Configuration.

### Code Examples

#### Shell/Bash

```
API call example (Shell)
curl -X DELETE "

-H "Authorization: Api-Key YOUR_API_KEY"

Please replace YOUR_API_KEY and verify the request data before
executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

// Request body (payload)
const data = null;

axios.delete(" data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.delete(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->delete(" [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error: ' . $e->getMessage();
}
?>
```

## Response

Status Code	Description
204	No response body

# API call example (Shell)

...

# 目錄

---

1. Create New Attachment (Integration)
2. Get Presigned URL for File Upload
3. Presigned Upload Attachment
4. Create Conversation Attachment
5. Get Allowed File Specifications

## API call example (Shell)

---

### Create New Attachment (Integration)

---

POST

/api/v1/attachments-upload/

### Code Examples

#### Shell/Bash

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/attachments-upload/"

-H "Authorization: Api-Key YOUR_API_KEY"

-F "file=@example_file.pdf"

Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```



```
const axios = require('axios');
const FormData = require('form-data');

// Create FormData object
const formData = new FormData();
// Please replace with actual file
formData.append('file',
* actual file object *
, 'example_file.pdf');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 ...formData.getHeaders()
 }
};

axios.post("https://api.maiagent.ai/api/v1/attachments-upload/",
formData, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error occurred:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/attachments-upload/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

Create file and form data
files = {
 'file': ('example_file.pdf', open('example_file.pdf', 'rb'),
 'application/pdf')
}

response = requests.post(url, files=files, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>post("https://api.maiagent.ai/api/v1/attachments-upload/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
],
 'multipart' => [
 [
 'name' => 'file',
 'contents' => fopen('example_file.pdf', 'r'),
 'filename' => 'example_file.pdf'
]
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

## Response

Status Code	Description
-------------	-------------

200

No response body

## Get Presigned URL for File Upload

POST

/api/v1/upload-presigned-url/

### Parameters

Parameter Name	Required	Type	Description
fieldName	X	string	Field name
filename	X	string	File name
modelName	X	string	Model name

### Request

Request Parameters

Field	Type	Required	Description
fileSize	integer	Yes	

Request Structure Example

```
{
 "fileSize": integer
}
```

Request Example Values

```
{
 "fileSize": 123
}
```

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/upload-presigned-url/?
fieldName=example&filename=example&modelName=example"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "modelName": "example string",
 "fieldName": "example string",
 "filename": "document.pdf",
 "fileSize": 123
}'

Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "modelName": "example string",
 "fieldName": "example string",
 "filename": "document.pdf",
 "fileSize": 123
};

axios.post("https://api.maiagent.ai/api/v1/upload-presigned-url/?
fieldName=example&filename=example&modelName=example", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error occurred:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/upload-presigned-url/?
fieldName=example&filename=example&modelName=example"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "modelName": "example string",
 "fieldName": "example string",
 "filename": "document.pdf",
 "fileSize": 123
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->
post("https://api.maiagent.ai/api/v1/upload-presigned-url/?
fieldName=example&filename=example&modelName=example", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "modelName": "example string",
 "fieldName": "example string",
 "filename": "document.pdf",
 "fileSize": 123
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

## Response

**Status Code: 200**



### Response Structure Example

```
{
 "url": string(uri) // S3 bucket URL
 "fields": object // Form fields for POST request
}
```

### Response Example Values

```
{
 "url": "https://s3.example.com/bucket-name",
 "fields": {
 "key": "media/chat/attachment/filename.pdf",
 "x-amz-algorithm": "AWS4-HMAC-SHA256",
 "x-amz-credential":
 "AKIAXXXXXXXXXXXXXXXX/YYMMDD/region/s3/aws4_request",
 "x-amz-date": "YYMMDDTHMMSSZ",
 "policy": "base64-encoded-policy-document",
 "x-amz-signature": "generated-signature-hash"
 }
}
```

## Presigned Upload Attachment

**POST**

**/api/v1/attachments/**

### Request

#### Request Parameters

Field	Type	Required	Description
type	object	No	

Field	Type	Required	Description
filename	string	Yes	
file	string (uri)	Yes	

Request Structure Example

```
{
 "type"?: // Optional
 {
 }
 "filename": string
 "file": string (uri)
}
```

Request Example Values

```
{
 "type": {},
 "filename": "document.pdf",
 "file": "https://example.com/file.jpg"
}
```

Code Examples

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/attachments/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "type": {},
 "filename": "document.pdf",
 "file": "https://example.com/file.jpg"
}'

Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "type": {},
 "filename": "document.pdf",
 "file": "https://example.com/file.jpg"
};

axios.post("https://api.maiagent.ai/api/v1/attachments/", data,
config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error occurred:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/attachments/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "type": {},
 "filename": "document.pdf",
 "file": "https://example.com/file.jpg"
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->
 post("https://api.maiagent.ai/api/v1/attachments/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "type": {},
 "filename": "document.pdf",
 "file": "https://example.com/file.jpg"
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 201

Response Structure Example

```
{
 "id": string (uuid)
 "type"?: // Optional
 {
 }
 "filename": string
 "file": string (uri)
 "expiresAt": string // ISO datetime when this attachment will be auto-deleted.

 For conversation-bound attachments: conversation.last_message_created_at
 + retention_days.
 For attachments without a conversation: created_at + retention_days.
 Returns None when cleanup is disabled (effective_from not set).
 "downloadStatus": string
}
```

#### Response Example Values

```
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "type": {},
 "filename": "response string",
 "file": "https://example.com/file.jpg",
 "expiresAt": "response string",
 "downloadStatus": "response string"
}
```

## Create Conversation Attachment

**POST**

**/api/v1/conversations/{conversationPk}/attachments/**

### Parameters

Parameter Name	Required	Type	Description
conversationPk	V	string	

# Request

## Request Parameters

Field	Type	Required	Description
type	object	No	
filename	string	Yes	
file	string (uri)	Yes	
conversation	string (uuid)	No	

## Request Structure Example

```
{
 "type"?: // Optional
 {
 }
 "filename": string
 "file": string (uri)
 "conversation"?: string (uuid) // Optional
}
```

## Request Example Values

```
{
 "type": {},
 "filename": "document.pdf",
 "file": "https://example.com/file.jpg",
```



```
"conversation": "550e8400-e29b-41d4-a716-446655440000"
}
```

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X POST "https://api.maiagent.ai/api/v1/conversations/550e8400-
e29b-41d4-a716-446655440000/attachments/"

-H "Authorization: Api-Key YOUR_API_KEY"

-H "Content-Type: application/json"

-d '{
 "type": {},
 "filename": "document.pdf",
 "file": "https://example.com/file.jpg",
 "conversation": "550e8400-e29b-41d4-a716-446655440000"
}'

Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY',
 'Content-Type': 'application/json'
 }
};

// Request body (payload)
const data = {
 "type": {},
 "filename": "document.pdf",
 "file": "https://example.com/file.jpg",
 "conversation": "550e8400-e29b-41d4-a716-446655440000"
};

axios.post("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/attachments/", data, config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error occurred:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/attachments/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY",
 "Content-Type": "application/json"
}

Request body (payload)
data = {
 "type": {},
 "filename": "document.pdf",
 "file": "https://example.com/file.jpg",
 "conversation": "550e8400-e29b-41d4-a716-446655440000"
}

response = requests.post(url, json=data, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error occurred:", e)
```

```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client->
post("https://api.maiagent.ai/api/v1/conversations/550e8400-e29b-41d4-a716-446655440000/attachments/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY',
 'Content-Type' => 'application/json'
],
 'json' => {
 "type": {},
 "filename": "document.pdf",
 "file": "https://example.com/file.jpg",
 "conversation": "550e8400-e29b-41d4-a716-446655440000"
 }
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

## Response

**Status Code: 201**

## Response Structure Example

```
{
 "id": string (uuid)
 "type"?: // Optional
 {
 }
 "filename": string
 "file": string (uri)
 "expiresAt": string // ISO datetime when this attachment will be auto-
deleted.

For conversation-bound attachments: conversation.last_message_created_at
+ retention_days.
For attachments without a conversation: created_at + retention_days.
Returns None when cleanup is disabled (effective_from not set).
 "conversation"?: string (uuid) // Optional
}
```

## Response Example Values

```
{
 "id": "550e8400-e29b-41d4-a716-446655440000",
 "type": {},
 "filename": "response string",
 "file": "https://example.com/file.jpg",
 "expiresAt": "response string",
 "conversation": "550e8400-e29b-41d4-a716-446655440000"
}
```

## Get Allowed File Specifications

GET

</api/v1/allowed-file-specs/>

## Code Examples

### Shell/Bash

```
API call example (Shell)
curl -X GET "https://api.maiagent.ai/api/v1/allowed-file-specs/"

-H "Authorization: Api-Key YOUR_API_KEY"

Please make sure to replace YOUR_API_KEY and verify the request
data before executing.
```

```
const axios = require('axios');

// Set request headers
const config = {
 headers: {
 'Authorization': 'Api-Key YOUR_API_KEY'
 }
};

axios.get("https://api.maiagent.ai/api/v1/allowed-file-specs/",
config)
 .then(response => {
 console.log('Response received successfully:');
 console.log(response.data);
 })
 .catch(error => {
 console.error('Request error occurred:');
 console.error(error.response?.data || error.message);
 });
```

```
import requests

url = "https://api.maiagent.ai/api/v1/allowed-file-specs/"
headers = {
 "Authorization": "Api-Key YOUR_API_KEY"
}

response = requests.get(url, headers=headers)
try:
 print("Response received successfully:")
 print(response.json())
except Exception as e:
 print("Request error occurred:", e)
```



```
<?php
require 'vendor/autoload.php';

$client = new GuzzleHttp\Client();

try {
 $response = $client-
>get("https://api.maiagent.ai/api/v1/allowed-file-specs/", [
 'headers' => [
 'Authorization' => 'Api-Key YOUR_API_KEY'
]
]);

 $data = json_decode($response->getBody(), true);
 echo "Response received successfully:\n";
 print_r($data);
} catch (Exception $e) {
 echo 'Request error occurred: ' . $e->getMessage();
}
?>
```

## Response

### Status Code: 200

Response Structure Example

```
{
 "webChat": object
 "organization": object
 "attachment": object
}
```

```
"chatbotFile": object
"knowledgeBaseFile": object
"batchQa": object
"meetingRecord": object
}
```

## Response Example Values

```
{
 "web_chat": {
 "logo": [
 {
 "content_type": {
 ".jpg": "image/jpeg",
 ".jpeg": "image/jpeg",
 ".png": "image/png",
 ".gif": "image/gif"
 },
 "max_size": 5242880
 }
],
 "avatar": [
 {
 "content_type": {
 ".jpg": "image/jpeg",
 ".jpeg": "image/jpeg",
 ".png": "image/png",
 ".gif": "image/gif"
 },
 "max_size": 2097152
 }
],
 "cover": [
 {
 "content_type": {
 ".jpg": "image/jpeg",
 ".jpeg": "image/jpeg",
 ".png": "image/png",
 ".gif": "image/gif"
 },
 "max_size": 10485760
 }
],
 "powered-by-logo": [
```

```
{
 "content_type": {
 ".jpg": "image/jpeg",
 ".jpeg": "image/jpeg",
 ".png": "image/png",
 ".gif": "image/gif"
 },
 "max_size": 5242880
}
],
},
"organization": {
 "compact_logo": [
 {
 "content_type": {
 ".jpg": "image/jpeg",
 ".jpeg": "image/jpeg",
 ".png": "image/png",
 ".gif": "image/gif"
 },
 "max_size": 2097152
 }
],
 "full_logo": [
 {
 "content_type": {
 ".jpg": "image/jpeg",
 ".jpeg": "image/jpeg",
 ".png": "image/png",
 ".gif": "image/gif"
 },
 "max_size": 2097152
 }
]
},
"attachment": {
 "file": [
 {
 "content_type": {
 ".pdf": "application/pdf",
 ".doc": "application/msword",
 ".docx": "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
 ".txt": "text/plain"
 },

```

```
 "max_size": 20971520
 },
 {
 "content_type": {
 ".jpg": "image/jpeg",
 ".jpeg": "image/jpeg",
 ".png": "image/png",
 ".gif": "image/gif"
 },
 "max_size": 5242880
 }
]
},
"chatbot_file": {
 "file": [
 {
 "content_type": {
 ".pdf": "application/pdf",
 ".doc": "application/msword",
 ".docx": "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
 ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
 ".ppt": "application/vnd.ms-powerpoint",
 ".pptx": "application/vnd.openxmlformats-officedocument.presentationml.presentation",
 ".txt": "text/plain"
 },
 "max_size": 104857600
 }
]
},
"batch_qa": {
 "file": [
 {
 "content_type": null,
 "max_size": 104857600
 }
]
},
"faq": {
 "answer": [
 {
 "content_type": {
 ".jpg": "image/jpeg",
```

```
 ".jpeg": "image/jpeg",
 ".png": "image/png",
 ".gif": "image/gif",
 ".webp": "image/webp",
 ".tiff": "image/tiff",
 ".mp4": "video/mp4"
 },
 "max_size": 104857600
}
]
}
}
```